

collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Collector
4 # =====
5 # Description:
6 #   Движок сбора системных метрик, аппаратной структуры и показателей
7 #   производительности.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.collector import SystemCollector
11 #
12 #       service = SystemCollector()
13 #
14 # File: collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 18:33:00
20 # =====
21
22 from __future__ import annotations
23 """Движок сбора системных метрик, аппаратной структуры и показателей
24   производительности."""
25
26 import asyncio
27 import ctypes
28 import getpass
29 import locale
30 import os
```

```
30 import platform
31 import socket
32 import time
33 from datetime import datetime, timezone
34 from typing import Any, Dict, List, Optional
35 try:
36     import psutil
37     PSUTIL_AVAILABLE = True
38 except ImportError:
39     psutil = None
40     PSUTIL_AVAILABLE = False
41 from logger import logger
42 from .models import (
43     AnomalyItem,
44     BatteryMetrics,
45     CloudStorageInfo,
46     CpuMetrics,
47     DiskIoMetrics,
48     DiskPartitionMetrics,
49     DriverInfo,
50     GpuMetrics,
51     HardwareArchiveEntry,
52     HardwareAuditReport,
53     HardwareChangeItem,
54     HardwareDeviceAudit,
55     HardwareNode,
56     HardwareSensor,
57     MemoryMetrics,
58     MonitorInfo,
59     NetworkInterfaceMetrics,
60     NetworkPortMetrics,
61     OfficeSuiteInfo,
62     PhysicalDiskHealth,
63     ProcessLifecycleEvent,
64     ProcessMetrics,
65     ProcessNetworkActivity,
66     ProcessProvenanceInfo,
67     ProcessProvenanceReport,
```

```

68     RamStickInfo,
69     StartupArchiveEntry,
70     StartupChangeItem,
71     SystemCoreMetrics,
72     SystemHardwareQuick,
73     SystemHealthAlerts,
74     SystemSnapshot,
75     WindowsUpdateInfo,
76 )
77 from .sensors import get_hardware_sensors
78 from apps.windows.telemetry_research.hardware_auditor import HardwareAuditor
79 from apps.windows.telemetry_research.hardware_history_manager import HardwareHistoryManager
80 from apps.windows.telemetry_research.startup_history_manager import StartupHistoryManager
81 from .process_token_collector import ProcessTokenCollector
82 from .sqlite import TelemetryStorage
83
84 class SystemCollector:
85     """Telemetry collector for system load, hardware devices,
86     and processes."""
87
88     def __init__(
89         self,
90         auditor: Optional[HardwareAuditor] = None,
91         history_manager: Optional[HardwareHistoryManager] = None,
92         startup_history_manager: Optional[StartupHistoryManager] = None,
93         storage: Optional[TelemetryStorage] = None,
94     ) -> None:
95         """Initialize telemetry collector with timing, I/O baseline, hardware auditor, startup history, and storage."""
96         self._last_disk_io = psutil.disk_io_counters() if PSUTIL_AVAILABLE else None
97         self._last_net_io = psutil.net_io_counters(pernic=True) if PSUTIL_AVAILABLE else None

```

```

97         self._last_proc_net_io: Dict[int, Tuple[float, float,
float]] = {}
98         self._last_time = time.time()
99         self._cpu_model_cached: Optional[str] = None
100        self._identity_cached: Optional[Dict[str, Any]] = None
101        self._monitors_cached: Optional[List[MonitorInfo]] = N
one
102        self._updates_cached: Optional[WindowsUpdateInfo] = No
ne
103        self._office_cached: Optional[OfficeSuiteInfo] = None
104        self._ram_sticks_cached: Optional[List[RamStickInfo]]
= None
105        self._ram_sticks_cache_time: float = 0.0
106        self._physical_disks_cached: Optional[List[PhysicalDis
kHealth]] = None
107        self._physical_disks_cache_time: float = 0.0
108        self.storage = storage or TelemetryStorage.get_instanc
e()
109        self.auditor = auditor or HardwareAuditor(storage=self
.storage)
110        self.history_manager = history_manager or HardwareHist
oryManager()
111        self.startup_history_manager = startup_history_manager
or StartupHistoryManager()
112
113        def get_system_identity(self) -> Dict[str, Any]:
114            """Collect host identity, current user, system languag
e, and locale parameters.
115
116            Returns:
117                Dict[str, Any]: Detailed system identity and local
ization dictionary.
118            """
119            if self._identity_cached:
120                return self._identity_cached
121            hostname = socket.gethostname()
122            domain = os.environ.get('USERDOMAIN', '')
123            username_raw = os.environ.get('USERNAME') or getpass.g

```

```

etuser()
124     full_username = f'{domain}\\{username_raw}' if domain
and domain != hostname else username_raw
125     user_locale = 'ru-RU'
126     system_locale = 'ru-RU'
127     codepage = 'UTF-8'
128     input_languages: List[str] = []
129     if os.name == 'nt':
130         try:
131             buf = ctypes.create_unicode_buffer(100)
132             if ctypes.windll.kernel32.GetUserDefaultLocale
Name(buf, 100):
133                 user_locale = buf.value or 'ru-RU'
134             if ctypes.windll.kernel32.GetSystemDefaultLoca
leName(buf, 100):
135                 system_locale = buf.value or 'ru-RU'
136             acp = ctypes.windll.kernel32.GetACP()
137             oemcp = ctypes.windll.kernel32.GetOEMCP()
138             codepage = f'ACP: {acp} | OEM: {oemcp} (UTF-8)
',
139             count = ctypes.windll.user32.GetKeyboardLayout
List(0, None)
140             if count > 0:
141                 hkls = (ctypes.c_void_p * count)()
142                 ctypes.windll.user32.GetKeyboardLayoutList
(count, hkls)
143                 lang_map = {1049: 'Русский (RU)', 1033: 'E
nglish (US)', 1037: 'İngilizce (IL)', 1058: 'Українська (UA)', 103
1: 'Deutsch (DE)', 1036: 'Français (FR)'}
144                 for h in hkls:
145                     lang_id = (h.value if hasattr(h, 'valu
e') and h.value else int(h)) & 65535
146                     lang_name = lang_map.get(lang_id, f'La
yout 0x{lang_id:04X}')
147                     if lang_name not in input_languages:
148                         input_languages.append(lang_name)
149             except Exception as ex:
150                 logger.debug(f'Failed to query native Windows

```

```

    locale: {ex}')
151         if not input_languages:
152             input_languages = ['Русский (RU)', 'English (US)']
153         try:
154             tz_offset = datetime.now().astimezone().strftime('%z')
155             tz_name = datetime.now().astimezone().tzname() or
time.tzname[0]
156             timezone_str = f'{tz_name} (UTC{tz_offset[:3]}:{tz
_offset[3:]})'
157         except Exception:
158             timezone_str = 'UTC+03:00'
159     def _format_lang_display(locale_tag: str) -> str:
160         if not locale_tag:
161             return ''
162         tag_clean = locale_tag.strip()
163         tag_lower = tag_clean.lower()
164         if 'ru' in tag_lower:
165             return f'Русский (Россия) [{tag_clean}]'
166         elif 'he' in tag_lower:
167             return f'עברית (ישראל) [{tag_clean}]'
168         elif 'en' in tag_lower:
169             return f'English (United States) [{tag_clean}]'
170         elif 'uk' in tag_lower:
171             return f'Українська (Україна) [{tag_clean}]'
172         elif 'de' in tag_lower:
173             return f'Deutsch (Deutschland) [{tag_clean}]'
174         elif 'fr' in tag_lower:
175             return f'Français (France) [{tag_clean}]'
176         elif 'es' in tag_lower:
177             return f'Español (España) [{tag_clean}]'
178         elif 'zh' in tag_lower:
179             return f'中文 (China) [{tag_clean}]'
180         elif 'ja' in tag_lower:
181             return f'日本語 (Japan) [{tag_clean}]'
182         return tag_clean
183

```

```

184 sys_lang_display = _format_lang_display(user_locale)
185 os_build = platform.version() or '10.0.26200'
186 os_install_date = ''
187 os_install_language = ''
188 if os.name == 'nt':
189     try:
190         import winreg
191         # Попытка получить оригинальный язык установки
из реестра (InstallLanguage)
192         for access in (winreg.KEY_READ | winreg.KEY_WO
W64_64KEY, winreg.KEY_READ):
193             try:
194                 with winreg.OpenKey(winreg.HKEY_LOCAL_
MACHINE, r'SYSTEM\CurrentControlSet\Control\Nls\Language', 0,
access) as key:
195                     val, _ = winreg.QueryValueEx(key,
'InstallLanguage')
196                     if val:
197                         lcid = int(str(val), 16)
198                         buf_lcid = ctypes.create_unico
de_buffer(100)
199                         if ctypes.windll.kernel32.LCID
ToLocaleName(lcid, buf_lcid, 100, 0):
200                             tag = buf_lcid.value
201                             if tag:
202                                 os_install_language =
_format_lang_display(tag)
203                             break
204             except Exception:
205                 pass
206         if not os_install_language:
207             sys_ui_lang = ctypes.windll.kernel32.GetSy
stemDefaultUILanguage()
208             buf_lcid = ctypes.create_unicode_buffer(10
0)
209             if ctypes.windll.kernel32.LCIDToLocaleName
(sys_ui_lang, buf_lcid, 100, 0):
210                 tag = buf_lcid.value

```

```

211         if tag:
212             os_install_language = _format_lang
                _display(tag)
213         except Exception as ex:
214             logger.debug(f'Failed to query Windows Install
                Language: {ex}')
215
216         try:
217             import winreg
218             for access in (winreg.KEY_READ | winreg.KEY_WO
                W64_64KEY, winreg.KEY_READ):
219                 try:
220                     with winreg.OpenKey(winreg.HKEY_LOCAL_
                MACHINE, r'SOFTWARE\Microsoft\Windows NT\CurrentVersion', 0, a
                ccess) as key:
221                         try:
222                             val, _ = winreg.QueryValueEx(k
                ey, 'InstallDate')
223                             if val and int(val) > 0:
224                                 os_install_date = datetime
                .fromtimestamp(int(val)).strftime('%d.%m.%Y %H:%M')
225                             break
226                         except Exception:
227                             pass
228                     try:
229                         val, _ = winreg.QueryValueEx(k
                ey, 'InstallTime')
230                         if val and int(val) > 0:
231                             epoch_sec = (int(val) / 10
                000000.0) - 11644473600.0
232                             if epoch_sec > 0:
233                                 os_install_date = date
                time.fromtimestamp(epoch_sec).strftime('%d.%m.%Y %H:%M')
234                             break
235                         except Exception:
236                             pass
237             except Exception:
238                 continue

```



```

239         except Exception as ex:
240             logger.debug(f'Failed to query Windows Install
Date from registry: {ex}')
241             workgroup = 'WORKGROUP'
242             if os.name == 'nt':
243                 try:
244                     from ctypes import wintypes
245                     netapi32 = ctypes.windll.netapi32
246                     lp_buf = wintypes.LPWSTR()
247                     join_stat = wintypes.DWORD()
248                     if netapi32.NetGetJoinInformation(None, ctypes
.byref(lp_buf), ctypes.byref(join_stat)) == 0:
249                         if lp_buf.value:
250                             workgroup = lp_buf.value
251                             netapi32.NetApiBufferFree(lp_buf)
252             except Exception as ex:
253                 logger.debug(f'Failed to query Windows Workgro
up: {ex}')
254             self._identity_cached = {
255                 'hostname': hostname,
256                 'workgroup': workgroup,
257                 'username': full_username,
258                 'os_build': os_build,
259                 'os_install_date': os_install_date,
260                 'system_language': sys_lang_display,
261                 'os_install_language': os_install_language,
262                 'user_locale': user_locale,
263                 'system_locale': system_locale,
264                 'timezone': timezone_str,
265                 'codepage': codepage,
266                 'input_languages': input_languages
267             }
268             return self._identity_cached
269
270     async def _resolve_cpu_model(self) -> str:
271         """Resolve CPU brand/model name from registry, platfor
m or WMI.
272

```

```

273 |     Returns:
274 |         str: Resolved CPU model name.
275 |         """
276 |     if self._cpu_model_cached:
277 |         return self._cpu_model_cached
278 |     model = ''
279 |     if os.name == 'nt':
280 |         try:
281 |             import winreg
282 |             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
283 |                                 r'HARDWARE\DESCRIPTION\System\CentralProcessor\0') as key:
284 |                 val, _ = winreg.QueryValueEx(key, 'Process
orNameString')
285 |                 if val and val.strip():
286 |                     model = val.strip()
287 |         except Exception:
288 |             pass
289 |     if not model:
290 |         model = platform.processor() or ''
291 |     if os.name == 'nt' and (not model or 'Intel64' in mode
l or 'AMD64' in model):
292 |         try:
293 |             from src.utils.com_worker import com_worker
294 |
295 |             def _get_cpu():
296 |                 import pythoncom
297 |                 pythoncom.CoInitialize()
298 |                 import wmi
299 |                 w = wmi.WMI()
300 |                 cpus = w.Win32_Processor()
301 |                 return str(cpus[0].Name).strip() if cpus e
lse None
302 |             res = await com_worker.run(_get_cpu)
303 |             if res:
304 |                 model = res
305 |         except Exception:
306 |             pass
307 |     if not model:

```

```

307         model = f'{platform.machine()} {os.cpu_count() or
1}-Core Processor'
308         self._cpu_model_cached = model
309         return model
310
311     async def get_cpu_metrics(self) -> CpuMetrics:
312         """Collect real-time CPU utilization and architecture
details.
313
314         Returns:
315             CpuMetrics: Current CPU performance metrics.
316         """
317         logical_cores = os.cpu_count() or 1
318         physical_cores = max(1, logical_cores // 2) if logical
_cores > 1 else 1
319         if PSUTIL_AVAILABLE:
320             per_core = psutil.cpu_percent(interval=None, percp
u=True)
321             total = round(sum(per_core) / max(len(per_core), 1
), 1) if per_core else 0.0
322             physical = psutil.cpu_count(logical=False) or phys
ical_cores
323             logical = psutil.cpu_count(logical=True) or logica
l_cores
324             freq_mhz = 0.0
325             try:
326                 freq = psutil.cpu_freq()
327                 if freq:
328                     freq_mhz = round(freq.current, 1)
329             except Exception:
330                 pass
331             return CpuMetrics(model=await self._resolve_cpu_mo
del(), architecture=platform.machine(), physical_cores=physica
l, logical_cores=logical, total_percent=total, per_core_perce
nt=per_core, frequency_mhz=freq_mhz)
332         return CpuMetrics(model=await self._resolve_cpu_model(
), architecture=platform.machine(), physical_cores=physical_co
res, logical_cores=logical_cores, total_percent=10.0, per_core

```

```

    _percent=[10.0] * logical_cores, frequency_mhz=2400.0)
333
334     def get_memory_metrics(self) -> MemoryMetrics:
335         """Collect RAM and swap memory consumption.
336
337         Returns:
338             MemoryMetrics: Current memory usage statistics.
339         """
340         if PSUTIL_AVAILABLE:
341             vm = psutil.virtual_memory()
342             sm = psutil.swap_memory()
343             return MemoryMetrics(total_gb=round(vm.total / 102
4 ** 3, 2), available_gb=round(vm.available / 1024 ** 3, 2), u
sed_gb=round((vm.total - vm.available) / 1024 ** 3, 2), percen
t=round(vm.percent, 1), swap_total_gb=round(sm.total / 1024 **
3, 2), swap_used_gb=round(sm.used / 1024 ** 3, 2), swap_perce
nt=round(sm.percent, 1))
344         total_ram_gb = 16.0
345         used_ram_gb = 8.0
346         if os.name == 'nt':
347             try:
348
349                 class MEMORYSTATUSEX(ctypes.Structure):
350                     _fields_ = [('dwLength', ctypes.c_ulong),
('dwMemoryLoad', ctypes.c_ulong), ('ullTotalPhys', ctypes.c_ul
onglong), ('ullAvailPhys', ctypes.c_ulonglong), ('ullTotalPage
File', ctypes.c_ulonglong), ('ullAvailPageFile', ctypes.c_ulon
glong), ('ullTotalVirtual', ctypes.c_ulonglong), ('ullAvailVir
tual', ctypes.c_ulonglong), ('sullAvailExtendedVirtual', ctype
s.c_ulonglong)]
351                 stat = MEMORYSTATUSEX()
352                 stat.dwLength = ctypes.sizeof(MEMORYSTATUSEX)
353                 if ctypes.windll.kernel32.GlobalMemoryStatusEx
(ctypes.byref(stat)):
354                     total_ram_gb = round(stat.ullTotalPhys / 1
024 ** 3, 2)
355                     avail_ram_gb = round(stat.ullAvailPhys / 1
024 ** 3, 2)

```

```

356         used_ram_gb = round(total_ram_gb - avail_r
am_gb, 2)
357         percent = round(stat.dwMemoryLoad, 1)
358         return MemoryMetrics(total_gb=total_ram_gb
, available_gb=avail_ram_gb, used_gb=used_ram_gb, percent=perc
ent, swap_total_gb=round(stat.ullTotalPageFile / 1024 ** 3, 2)
, swap_used_gb=round((stat.ullTotalPageFile - stat.ullAvailPag
eFile) / 1024 ** 3, 2), swap_percent=round((1.0 - stat.ullAvai
lPageFile / max(stat.ullTotalPageFile, 1)) * 100, 1))
359     except Exception:
360         pass
361     return MemoryMetrics(total_gb=total_ram_gb, available_
gb=total_ram_gb - used_ram_gb, used_gb=used_ram_gb, percent=50
.0, swap_total_gb=16.0, swap_used_gb=2.0, swap_percent=12.5)
362
363     def get_gpu_metrics(self) -> List[GpuMetrics]:
364         """Collect GPU accelerator load, VRAM, and capabilitie
s.
365
366         Returns:
367             List[GpuMetrics]: Detected GPU devices with teleme
try.
368
369         """
370         gpus: List[GpuMetrics] = []
371         try:
372             from apps.windows.modules.hardware.gpu_prober impo
rt GpuProber
373             prober = GpuProber()
374             gpu_list = prober.probe_all()
375             for g in gpu_list:
376                 gpus.append(GpuMetrics(
377                     name=g.name,
378                     memory_total_gb=round((g.memory_total_mb or
0.0) / 1024.0, 2),
379                     memory_used_gb=round((g.memory_used_mb or
0.0) / 1024.0, 2),
380                     load_percent=g.utilization_gpu_pct,
381                     temperature_celsius=g.temperature_gpu_c,

```

```

381         has_cuda=g.vendor.lower() == 'nvidia',
382         has_directml=True,
383         engines=g.engines,
384     ))
385     except Exception as ex:
386         logger.debug(f'Ошибка сбора метрик GPU через GpuPr
ober: {ex}')
387     if not gpus:
388         gpus.append(GpuMetrics(name='Integrated Display Co
ntroller', memory_total_gb=0.0, has_cuda=False, has_directml=F
alse))
389     return gpus
390
391     def get_disk_metrics(self) -> tuple[List[DiskPartitionMetr
ics], DiskIoMetrics]:
392         """Collect disk partitions capacity and read/write rat
es.
393
394         Returns:
395             tuple[List[DiskPartitionMetrics], DiskIoMetrics]:
Partitions and I/O rates.
396         """
397         partitions: List[DiskPartitionMetrics] = []
398         io_metrics = DiskIoMetrics()
399         if PSUTIL_AVAILABLE:
400             for part in psutil.disk_partitions(all=False):
401                 try:
402                     usage = psutil.disk_usage(part.mountpoint)
403                     vol_name = ''
404                     is_virtual = False
405                     drive_type = 'Fixed'
406                     if os.name == 'nt':
407                         try:
408                             import ctypes
409                             vol_buf = ctypes.create_unicode_bu
ffer(1024)
410                             fs_buf = ctypes.create_unicode_bu
ffer(1024)

```

```

411         mount_root = part.mountpoint if part.mountpoint.endswith('\\') else f'{part.mountpoint}\\'
412         if ctypes.windll.kernel32.GetVolumeInformationW(mount_root, vol_buf, 1024, None, None, None, fs_buf, 1024):
413             vol_name = vol_buf.value
414             dos_buf = ctypes.create_unicode_buffer(1024)
415             clean_dev = part.device.rstrip('\\')
416             ctypes.windll.kernel32.QueryDosDeviceW(clean_dev, dos_buf, 1024)
417             dos_dev = dos_buf.value
418             if 'google' in vol_name.lower() or '@' in vol_name or 'onedrive' in vol_name.lower() or ('dropbox' in vol_name.lower()) or ('volume{' in dos_dev.lower() and 'harddiskvolume' not in dos_dev.lower()):
419                 is_virtual = True
420                 drive_type = 'Google Drive' if 'google' in vol_name.lower() or '@' in vol_name else 'Cloud / Virtual'
421             except Exception:
422                 pass
423             partitions.append(DiskPartitionMetrics(device=part.device, mountpoint=part.mountpoint, fstype=part.fstype, total_gb=round(usage.total / 1024 ** 3, 2), used_gb=round(usage.used / 1024 ** 3, 2), free_gb=round(usage.free / 1024 ** 3, 2), percent=round(usage.percent, 1), volume_name=vol_name or None, is_virtual=is_virtual, drive_type=drive_type))
424             except (PermissionError, OSError):
425                 continue
426             now = time.time()
427             elapsed = max(now - self._last_time, 0.1)
428             current_io = psutil.disk_io_counters()
429             if current_io and self._last_disk_io:
430                 io_metrics.read_bytes_per_sec = max(0.0, (current_io.read_bytes - self._last_disk_io.read_bytes) / elapsed)
431                 io_metrics.write_bytes_per_sec = max(0.0, (cur

```

```

    rent_io.write_bytes - self._last_disk_io.write_bytes) / elapsed
    d)
432         io_metrics.read_count_per_sec = max(0.0, (current_io.read_count - self._last_disk_io.read_count) / elapsed)
433         io_metrics.write_count_per_sec = max(0.0, (current_io.write_count - self._last_disk_io.write_count) / elapsed)
434         self._last_disk_io = current_io
435     else:
436         drive = 'C:\\' if os.name == 'nt' else '/'
437         try:
438             import shutil
439             total, used, free = shutil.disk_usage(drive)
440             partitions.append(DiskPartitionMetrics(device='C:' if os.name == 'nt' else '/', mountpoint=drive, fstype='NTFS' if os.name == 'nt' else 'ext4', total_gb=round(total / 1024 ** 3, 2), used_gb=round(used / 1024 ** 3, 2), free_gb=round(free / 1024 ** 3, 2), percent=round(used / max(total, 1) * 100, 1)))
441         except Exception:
442             pass
443     return (partitions, io_metrics)
444
445     def get_network_metrics(self) -> List[NetworkInterfaceMetrics]:
446         """Collect network interface statuses and throughput rates.
447
448         Returns:
449             List[NetworkInterfaceMetrics]: Network adapters metrics.
450         """
451         interfaces: List[NetworkInterfaceMetrics] = []
452         adapter_stats_map = {}
453         try:
454             from apps.windows.network.network_usage import WindowsNetworkUsageCollector
455             usage_collector = WindowsNetworkUsageCollector()

```



```

456         stats_list = usage_collector.get_adapter_statistics()
457         adapter_stats_map = {s.name.lower(): s for s in stats_list}
458     except Exception as ex:
459         logger.debug(f'Failed to fetch adapter stats in SystemCollector: {ex}')
460
461     if PSUTIL_AVAILABLE:
462         addrs = psutil.net_if_addrs()
463         stats = psutil.net_if_stats()
464         current_net_io = psutil.net_io_counters(pernic=True)
465
466         now = time.time()
467         elapsed = max(now - self._last_time, 0.1)
468         for name, addr_list in addrs.items():
469             stat = stats.get(name)
470             is_up = stat.isup if stat else True
471             speed = stat.speed if stat else 0
472             ips = [a.address for a in addr_list if a.family == socket.AF_INET]
473
474             sent_rate = 0.0
475             recv_rate = 0.0
476             if current_net_io and name in current_net_io and self._last_net_io and (name in self._last_net_io):
477                 cur = current_net_io[name]
478                 prev = self._last_net_io[name]
479                 sent_rate = max(0.0, (cur.bytes_sent - prev.bytes_sent) / elapsed)
480                 recv_rate = max(0.0, (cur.bytes_recv - prev.bytes_recv) / elapsed)
481                 st_info = adapter_stats_map.get(name.lower())
482                 interfaces.append(NetworkInterfaceMetrics(name=name, is_up=is_up, speed_mbps=speed, bytes_sent_per_sec=sent_rate, bytes_recv_per_sec=recv_rate, ip_addresses=ips, statistics=st_info))
483
484         self._last_net_io = current_net_io
485     else:

```

```

483         try:
484             host_ip = socket.gethostbyname(socket.gethostn
ame())
485         except Exception:
486             host_ip = '127.0.0.1'
487         interfaces.append(NetworkInterfaceMetrics(name='Pr
imary Network Adapter', is_up=True, speed_mbps=1000, bytes_sen
t_per_sec=0.0, bytes_recv_per_sec=0.0, ip_addresses=[host_ip])
)
488         return interfaces
489
490     def get_network_usage_report(self, period_minutes: int = 1
440):
491         """Получить глубокий отчет сетевого потребления за пер
иод для AI диагностики.
492
493         Args:
494             period_minutes: Интервал отчета в минутах.
495
496         Returns:
497             NetworkUsagePeriodReport: Полная сводка сетевой ст
атистики за период.
498         """
499         from apps.windows.network.network_usage import Windows
NetworkUsageCollector
500         collector = WindowsNetworkUsageCollector()
501         return collector.get_traffic_period_summary(period_min
utes=period_minutes)
502
503     def get_disk_usage_report(self, period_minutes: int = 1440
, force_refresh_smart: bool = False):
504         """Получить глубокий отчет использования дискового I/O
и SMART за период для AI диагностики.
505
506         Args:
507             period_minutes: Интервал отчета в минутах (по умол
чанию 1440 = 24 часа).
508             force_refresh_smart: Принудительно опросить физиче

```

ские диски.

509

510 Returns:

511 DiskUsagePeriodReport: Полная сводка дискового I/O
и SMART-показателей за период.

512 """

513 from apps.windows.telemetry.storage_usage import Windo
wsStorageUsageCollector

514 collector = WindowsStorageUsageCollector()

515 return collector.get_disk_usage_period_report(period_m
inutes=period_minutes, force_refresh_smart=force_refresh_smart
)

516

517

518

519 @staticmethod

520 def _classify_launch_reason(

521 parent_name: Optional[str],

522 process_name: Optional[str] = None,

523 session_id: Optional[int] = None,

524 cmdline: Optional[str] = None,

525) -> str:

526 """Классификация причины/контекста запуска процесса по
его родителю и окружению.

527

528 Args:

529 parent_name: Имя родительского процесса.

530 process_name: Имя текущего процесса.

531 session_id: Номер сессии Windows (0 = системные сл
ужбы, 1+ = интерактивная сессия).

532 cmdline: Командная строка процесса (опционально).

533

534 Returns:

535 str: Категория запуска ('service', 'scheduled_task
, 'shell_interactive', 'terminal_cli', 'ide_developer', 'syst
em_boot', 'spawned_by_parent', 'unknown').

536 """

537 p_name = (parent_name or '').lower()

```

538         if p_name in ('services.exe',) or (p_name in ('svchost
| .exe',) and session_id == 0):
539             return 'service'
540         if p_name in ('taskeng.exe', 'taskhostw.exe', 'taskhos
| t.exe') or (cmdline and '-task' in cmdline.lower()):
541             return 'scheduled_task'
542         if p_name in ('explorer.exe',):
543             return 'shell_interactive'
544         if p_name in ('cmd.exe', 'powershell.exe', 'pwsh.exe',
| 'wt.exe', 'windowsterminal.exe', 'bash.exe', 'zsh.exe', 'conh
| ost.exe'):
545             return 'terminal_cli'
546         if p_name in ('code.exe', 'devenv.exe', 'pycharm64.exe
| ', 'idea64.exe', 'rider64.exe', 'cursor.exe', 'windsurf.exe'):
547             return 'ide_developer'
548         if p_name in ('winlogon.exe', 'userinit.exe', 'smss.ex
| e', 'csrss.exe', 'wininit.exe', 'lsass.exe'):
549             return 'system_boot'
550         if p_name:
551             clean_parent = p_name.replace('.exe', '')
552             return f'spawned_by_{clean_parent}'
553         return 'unknown'
554
555     def get_top_processes(self, limit: int = 25, sort_by: str
| = 'cpu') -> List[ProcessMetrics]:
556         """Retrieve active processes sorted by resource consum
| ption, with token and provenance info.
557
558         Args:
559             limit: Maximum processes to return (0 for all).
560             sort_by: Attribute to sort by ('cpu', 'memory'/'ra
| m', 'handles'/'descriptors').
561
562         Returns:
563             List[ProcessMetrics]: Ranked process metrics, incl
| uding provenance, integrity_level and elevation.
564         """
565         # Collect token information for all processes
566         token_collector = ProcessTokenCollector()

```

```

566 |         token_info_map = {info.pid: info for info in token_col
|lector.collect()}
567 |
568 |         procs: List[ProcessMetrics] = []
569 |         if PSUTIL_AVAILABLE:
570 |             handle_attr = 'num_handles' if os.name == 'nt' els
|e 'num_fds'
571 |             proc_dict: Dict[int, Dict[str, Any]] = {}
572 |             for p in psutil.process_iter(attrs=['pid', 'ppid',
|'name', 'status', 'cpu_percent', 'memory_info', 'memory_perce
|nt', 'num_threads', 'username', 'create_time', 'exe', 'cmdline
|', handle_attr]):
573 |                 try:
574 |                     proc_dict[p.info['pid']] = p.info
575 |                 except (psutil.NoSuchProcess, psutil.AccessDen
|ied):
576 |                     continue
577 |
578 |             for pid, info in proc_dict.items():
579 |                 try:
580 |                     mem_info = info.get('memory_info')
581 |                     rss_mb = round(mem_info.rss / (1024 * 1024
|), 1) if mem_info else 0.0
582 |                     cpu_p = round(info.get('cpu_percent') or 0
|.0, 1)
583 |                     num_h = info.get(handle_attr) or 0
584 |                     token = token_info_map.get(pid)
585 |                     integrity = token.integrity_level if token
|
|                     else None
586 |                     elevation = token.elevation if token else
|None
587 |                     sid = token.sid if token else None
588 |                     session_id = token.session_id if token els
|e None
589 |
590 |                     ppid = info.get('ppid')
591 |                     parent_info = proc_dict.get(ppid) if ppid
|
|                     else None

```

```

592         parent_name = parent_info.get('name') if p
    parent_info else None
593
594         # Построение цепочки предков
595         ancestors = [info.get('name') or str(pid)]
596         curr_pid = ppid
597         visited = {pid}
598         while curr_pid and curr_pid in proc_dict a
    nd curr_pid not in visited:
599             visited.add(curr_pid)
600             curr_p = proc_dict[curr_pid]
601             ancestors.append(curr_p.get('name') or
    f'PID:{curr_pid}')
602             curr_pid = curr_p.get('ppid')
603         ancestors.reverse()
604         ancestor_chain = ' → '.join(ancestors)
605
606         create_time_val = info.get('create_time')
    or 0.0
607         creation_time_str = datetime.fromtimestamp
    (create_time_val, tz=timezone.utc).isoformat() if create_time_
    val > 0 else None
608         process_guid = f'proc_{pid}_{int(create_ti
    me_val)}'
609
610         raw_cmdline = info.get('cmdline')
611         cmdline_str = ' '.join(raw_cmdline) if isi
    nstance(raw_cmdline, list) else (str(raw_cmdline) if raw_cmdli
    ne else None)
612         launch_reason = self._classify_launch_reas
    on(parent_name, info.get('name'), session_id, cmdline_str)
613
614         procs.append(ProcessMetrics(
615             pid=pid,
616             name=info.get('name') or 'unknown',
617             status=info.get('status') or 'running'
    ,
618             cpu_percent=cpu_p,

```

```

619 |             memory_mb=rss_mb,
620 |             memory_percent=round(info.get('memory_
percent') or 0.0, 1),
621 |             num_threads=info.get('num_threads') or
1,
622 |             num_handles=num_h,
623 |             username=info.get('username'),
624 |             integrity_level=integrity,
625 |             elevation=elevation,
626 |             ppid=ppid,
627 |             parent_name=parent_name,
628 |             executable_path=info.get('exe'),
629 |             sid=sid,
630 |             session_id=session_id,
631 |             creation_time=creation_time_str,
632 |             process_guid=process_guid,
633 |             ancestor_chain=ancestor_chain,
634 |             launch_reason=launch_reason,
635 |             ))
636 |         except (psutil.NoSuchProcess, psutil.AccessDen
ied):
637 |             continue
638 |     else:
639 |         # Fallback for non-psutil environments
640 |         now_iso = datetime.now(timezone.utc).isoformat()
641 |         p = ProcessMetrics(
642 |             pid=os.getpid(),
643 |             name='python.exe',
644 |             status='running',
645 |             cpu_percent=1.5,
646 |             memory_mb=120.0,
647 |             memory_percent=1.0,
648 |             num_threads=8,
649 |             num_handles=42,
650 |             username=os.getenv('USERNAME', 'SYSTEM'),
651 |             ppid=os.getppid() if hasattr(os, 'getppid') el
se None,
652 |             parent_name='cmd.exe',

```

```

653         executable_path=os.path.abspath(__file__),
654         sid='S-1-5-21-mock-sid',
655         session_id=1,
656         creation_time=now_iso,
657         process_guid=f'proc_{os.getpid()}_{int(time.time())}',
658         ancestor_chain='cmd.exe → python.exe',
659         launch_reason='terminal_cli',
660     )
661     token = token_info_map.get(p.pid)
662     if token:
663         p.integrity_level = token.integrity_level
664         p.elevation = token.elevation
665         p.sid = token.sid
666         p.session_id = token.session_id
667     procs.append(p)
668
669     # Sorting according to requested key
670     sort_key = (sort_by or 'cpu').lower()
671     if sort_key in ('handles', 'descriptors'):
672         procs.sort(key=lambda x: x.num_handles, reverse=True)
673     elif sort_key in ('memory', 'ram'):
674         procs.sort(key=lambda x: x.memory_mb, reverse=True)
675     else:
676         procs.sort(key=lambda x: x.cpu_percent, reverse=True)
677
678     if limit is None or limit <= 0:
679         return procs
680     return procs[:limit]
681
682     def get_battery_metrics(self) -> BatteryMetrics:
683         """Collect laptop or UPS battery charge level and AC power status.
684
685         Returns:

```



```

686         BatteryMetrics: Current battery and power status.
687         """
688         if PSUTIL_AVAILABLE and hasattr(psutil, 'sensors_battery'):
689             try:
690                 bat = psutil.sensors_battery()
691                 if bat is not None:
692                     return BatteryMetrics(has_battery=True, percent=round(bat.percent, 1), power_plugged=bat.power_plugged, secs_left=bat.secsleft if bat.secsleft > 0 else None, power_profile='AC / Battery')
693             except Exception as ex:
694                 logger.debug(f'Battery probe failed: {ex}')
695             return BatteryMetrics(has_battery=False, percent=None, power_plugged=True, secs_left=None, power_profile='AC Mains / Desktop')
696
697     def get_physical_disks_health(self, force: bool = False) -
698     > List[PhysicalDiskHealth]:
699         """Собрать данные о физических дисках, SMART, типах носителей и здоровье.
700
701         В сценариях API (tc.ps1) данные читаются строго из SQLite базы данных telemetry.db
702         без вызова тяжелого опроса WMI/CIM.
703         Прямой опрос оборудования через WindowsStorageSensor выполняется только при force=True (в демоне tlm.ps1).
704
705         Args:
706             force: Принудительное обновление данных накопителя через WindowsStorageSensor (только для tlm.ps1).
707
708         Returns:
709             List[PhysicalDiskHealth]: Список обнаруженных физических накопителей.
710         """
711         now = time.time()
712         if not force and self._physical_disks_cached is not No

```



```

me_write_bytes"),
733         lifetime_read_tb=d.get("lifetime_r
ead_tb"),
734         lifetime_write_tb=d.get("lifetime_
write_tb"),
735         power_on_hours=d.get("power_on_hou
rs"),
736         wear_percentage=d.get("wear_perce
ntage"),
737     ))
738     if disks:
739         self._physical_disks_cached = disks
740         self._physical_disks_cache_time = now
741         return disks
742     except Exception as ex:
743         logger.debug(f"Не удалось прочитать physical_d
isks из БД: {ex}")
744
745     # 2. Прямой опрос через WindowsStorageSensor при force
=True или отсутствии дисков в БД
746     disks: List[PhysicalDiskHealth] = []
747     if os.name == 'nt':
748         try:
749             from apps.windows.modules.storage_manager.core
.windows_storage_sensor import WindowsStorageSensor
750             sensor = WindowsStorageSensor(ttl_sec=43200.0)
751             phys_disks = sensor.get_physical_disks(force_r
efresh=force, sync=True)
752             for d in phys_disks:
753                 disks.append(PhysicalDiskHealth(
754                     device_id=d.device_id,
755                     model=d.model or d.friendly_name or 'P
hysical Drive',
756                     serial_number=d.serial_number if d.ser
ial_number and d.serial_number != 'N/A' else None,
757                     media_type=d.media_type or 'SSD',
758                     size_gb=round(d.size_gb, 1) if d.size_
gb else 0.0,

```

```

759         health_status=d.health_status or 'Healthy',
760         operational_status=d.operational_status or 'OK',
761         temperature_celsius=d.temperature_c,
762         interface_type=d.bus_type or 'NVMe',
763         lifetime_read_bytes=d.lifetime_read_bytes,
764         lifetime_write_bytes=d.lifetime_write_bytes,
765         lifetime_read_tb=d.lifetime_read_tb,
766         lifetime_write_tb=d.lifetime_write_tb,
767         power_on_hours=d.power_on_hours,
768         wear_percentage=d.wear_percentage,
769     ))
770     except Exception as ex:
771         logger.debug(f'Ошибка сбора физических дисков
772         через WindowsStorageSensor: {ex}')
773
774     if not disks:
775         disks.append(PhysicalDiskHealth(
776             device_id='Disk 0',
777             model='System Drive (NVMe/SSD)',
778             serial_number=None,
779             media_type='SSD',
780             size_gb=512.0,
781             health_status='Healthy',
782             operational_status='OK',
783             interface_type='NVMe',
784         ))
785     self._physical_disks_cached = disks
786     self._physical_disks_cache_time = now
787     return disks
788
789     async def get_ram_sticks(self) -> List[RamStickInfo]:
790         """Collect physical memory stick (SPD) details via WMI

```

```

791         Returns:
792             List[RamStickInfo]: Installed physical RAM modules
793         """
794         now = time.time()
795         if self._ram_sticks_cached is not None and now - self.
            _ram_sticks_cache_time < 60.0:
796             return self._ram_sticks_cached
797         sticks: List[RamStickInfo] = []
798         if os.name == 'nt':
799             from src.utils.com_worker import com_worker
800
801         async def _probe():
802
803             def _do_probe():
804                 try:
805                     import wmi
806                 except ImportError:
807                     return []
808                 _sticks: List[RamStickInfo] = []
809                 try:
810                     w = wmi.WMI()
811                     mems = w.Win32_PhysicalMemory()
812                     type_map = {20: 'DDR', 21: 'DDR2', 24:
                        'DDR3', 26: 'DDR4', 34: 'DDR5'}
813                     for m in mems:
814                         cap_gb = round(int(m.Capacity or 0
                        ) / 1024 ** 3, 1)
815                         speed = int(m.Speed or m.Configure
                        dClockSpeed or 3200)
816                         m_type = type_map.get(m.SMBIOSMemo
                        ryType, 'DDR4/DDR5')
817                         _sticks.append(RamStickInfo(bank_l
                        abel=str(m.BankLabel or m.DeviceLocator or 'DIMM').strip(), ca
                        pacity_gb=cap_gb, speed_mhz=speed, manufacturer=str(m.Manufact
                        urer or 'Generic').strip(), part_number=str(m.PartNumber or ''
                        ).strip(), memory_type=m_type))
818                 except Exception as err:

```

```

819 |             logger.debug(f'WMI Win32_PhysicalMemor
      | y query failed: {err}')
820 |             return _sticks
821 |             return await com_worker.run(_do_probe)
822 |         try:
823 |             sticks = await _probe()
824 |         except Exception as ex:
825 |             logger.debug(f'Failed to query physical memory
      | sticks: {ex}')
826 |             self._ram_sticks_cached = sticks
827 |             self._ram_sticks_cache_time = now
828 |             return sticks
829 |
830 |     def get_listening_ports(self, limit: int=15) -> List[Netwo
      | rkPortMetrics]:
831 |         """Collect active listening TCP/UDP sockets with proce
      | ss attribution.
832 |
833 |         Args:
834 |             limit: Maximum listening ports to return.
835 |
836 |         Returns:
837 |             List[NetworkPortMetrics]: Active listening ports.
838 |         """
839 |         ports: List[NetworkPortMetrics] = []
840 |         if PSUTIL_AVAILABLE:
841 |             try:
842 |                 conns = psutil.net_connections(kind='inet')
843 |                 seen = set()
844 |                 for c in conns:
845 |                     if c.status == 'LISTEN' or c.type == socke
      | t.SOCK_DGRAM:
846 |                         p_num = c.laddr.port
847 |                         if p_num in seen:
848 |                             continue
849 |                         seen.add(p_num)
850 |                         p_name = None
851 |                         if c.pid:

```

```

852 |             try:
853 |                 p_name = psutil.Process(c.pid)
854 |                 .name()
855 |             except Exception:
856 |                 pass
857 |             ports.append(NetworkPortMetrics(port=p
858 | _num, protocol='TCP' if c.type == socket.SOCK_STREAM else 'UDP
859 | ', address=c.laddr.ip or '0.0.0.0', pid=c.pid, process_name=p_
860 | name))
861 |             if len(ports) >= limit:
862 |                 break
863 |             except Exception as ex:
864 |                 logger.debug(f'Failed to query listening ports
865 | : {ex}')
```

```

866 |     return ports
867 |
868 |     def get_process_network_activity(self, limit: int=50, only
869 | _internet: bool=False) -> List[ProcessNetworkActivity]:
870 |         """Collect active network connections and traffic attr
871 | ibution for processes.
872 |
873 |         Args:
874 |             limit: Maximum connections to return.
875 |             only_internet: Whether to restrict only to externa
876 | l internet addresses.
877 |
878 |         Returns:
879 |             List[ProcessNetworkActivity]: Active network conne
880 | ctions with process and traffic description.
881 |         """
882 |         activities: List[ProcessNetworkActivity] = []
883 |         if not PSUTIL_AVAILABLE:
884 |             return activities
885 |         try:
886 |             raw_conns = psutil.net_connections(kind='inet')
887 |         except Exception as ex:
888 |             logger.debug(f'Failed to query network connections
889 | : {ex}')
```

```

880         return activities
881     proc_cache: Dict[int, Dict[str, Any]] = {}
882     now_ts = time.time()
883
884     def _get_proc_info(pid: Optional[int]) -> Dict[str, Any]:
885         if not pid or pid == 0:
886             return {'name': 'System Idle', 'user': 'SYSTEM',
887                     'read_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'delta_write_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0.0}
888         if pid == 4:
889             return {'name': 'System', 'user': 'NT AUTHORITY\\SYSTEM', 'read_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'delta_write_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0.0}
890         if pid in proc_cache:
891             return proc_cache[pid]
892         try:
893             p = psutil.Process(pid)
894             pname = p.name()
895             puser = ''
896             try:
897                 puser = p.username()
898             except Exception:
899                 puser = ''
900             read_kb = 0.0
901             write_kb = 0.0
902             delta_read_kb = 0.0
903             delta_write_kb = 0.0
904             read_speed_kbs = 0.0
905             write_speed_kbs = 0.0
906             try:
907                 io = p.io_counters()
908                 read_kb = round(io.read_bytes / 1024, 1)
909                 write_kb = round(io.write_bytes / 1024, 1)
910                 if hasattr(self, '_last_proc_net_io') and pid in self._last_proc_net_io:

```



```

910         prev_t, prev_r, prev_w = self._last_pr
    | oc_net_io[pid]
911         dt = max(now_ts - prev_t, 0.1)
912         if io.read_bytes >= prev_r:
913             delta_read_kb = round((io.read_byt
    | es - prev_r) / 1024, 1)
914             read_speed_kbs = round(delta_read_
    | kb / dt, 1)
915             if io.write_bytes >= prev_w:
916                 delta_write_kb = round((io.write_b
    | ytes - prev_w) / 1024, 1)
917                 write_speed_kbs = round(delta_writ
    | e_kb / dt, 1)
918             if hasattr(self, '_last_proc_net_io'):
919                 self._last_proc_net_io[pid] = (now_ts,
    | io.read_bytes, io.write_bytes)
920             except Exception:
921                 pass
922             info = {'name': pname, 'user': puser, 'read_kb
    | ': read_kb, 'write_kb': write_kb, 'delta_read_kb': delta_read_
    | kb, 'delta_write_kb': delta_write_kb, 'read_speed_kbs': read_s
    | peed_kbs, 'write_speed_kbs': write_speed_kbs}
923             proc_cache[pid] = info
924             return info
925         except Exception:
926             info = {'name': f'PID {pid}', 'user': '', 'rea
    | d_kb': 0.0, 'write_kb': 0.0, 'delta_read_kb': 0.0, 'delta_writ
    | e_kb': 0.0, 'read_speed_kbs': 0.0, 'write_speed_kbs': 0.0}
927             proc_cache[pid] = info
928             return info
929
930     def _is_ext_internet(ip: str) -> bool:
931         if not ip:
932             return False
933         if ip in ('127.0.0.1', '::1', '0.0.0.0', ':::') or
    | ip.startswith('127.') or ip.startswith('fe80:'):
934             return False
935         return True

```

```

936         for c in raw_conns:
937             laddr_str = f'{c.laddr.ip}:{c.laddr.port}' if c.la
laddr else '-'
938             raddr_str = f'{c.raddr.ip}:{c.raddr.port}' if c.ra
raddr else '-'
939             rip = c.raddr.ip if c.raddr else ''
940             rport = c.raddr.port if c.raddr else 0
941             lport = c.laddr.port if c.laddr else 0
942             is_ext = _is_ext_internet(rip)
943             if only_internet and (not is_ext):
944                 continue
945             proto = 'TCP' if c.type == socket.SOCK_STREAM else
'UDP'
946             status = c.status or ('LISTEN' if not c.raddr else
'ESTABLISHED')
947             p_info = _get_proc_info(c.pid)
948             p_name = p_info['name']
949             p_user = p_info['user']
950             service_type, sent_desc, recv_desc = self._classif
y_traffic(p_name, rport or lport, proto, status, is_ext)
951             activities.append(ProcessNetworkActivity(pid=c.pid
or 0, name=p_name, user=p_user, local_address=laddr_str, remo
te_address=raddr_str, remote_host=None, protocol=proto, status
=status, is_internet=is_ext, service_type=service_type, sent_k
b=p_info.get('write_kb', 0.0), recv_kb=p_info.get('read_kb', 0
.0), delta_sent_kb=p_info.get('delta_write_kb', 0.0), delta_re
cv_kb=p_info.get('delta_read_kb', 0.0), sent_rate_kbs=p_info.g
et('write_speed_kbs', 0.0), recv_rate_kbs=p_info.get('read_spe
ed_kbs', 0.0), sent_summary=sent_desc, recv_summary=recv_desc)
)
952             if len(activities) >= limit:
953                 break
954             return activities
955
956     @staticmethod
957     def _classify_traffic(proc_name: str, port: int, proto: st
r, status: str, is_internet: bool) -> tuple[str, str, str]:
958         """Classify network traffic service type and generate

```

```
human-readable summaries."""
959     p_lower = proc_name.lower()
960     if status == 'LISTEN' or not is_internet:
961         if 'python' in p_lower or 'uvicorn' in p_lower or
'node' in p_lower:
962             return ('Локальный Web API / Сервер', 'HTTP от
веты локальным клиентам / WebSockets', 'Входящие HTTP запросы
от фронтенда / браузера')
963             return ('Локальная служба / Socket', 'Служебные от
веты локальной подсистемы', 'Ожидание входящих подключений (Li
sten)')
964         if port in (443, 8443):
965             if any((b in p_lower for b in ('edge', 'chrome', '
firefox', 'brave', 'opera', 'browser'))):
966                 return ('HTTPS / Веб-навигация', 'Исходящие HT
TPS запросы, URL заголовки, формы, cookies', 'HTML страницы, с
крипты, видео, изображения, медиа')
967             if any((m in p_lower for m in ('telegram', 'discor
d', 'slack', 'whatsapp', 'skype'))):
968                 return ('Мессенджер / Зашифрованный чат', 'Тек
стовые сообщения, медиа-файлы, голосовые пакеты, пинги', 'Вход
ящие сообщения, аудио/видеопоток, обновления чатов')
969             if any((s in p_lower for s in ('onedrive', 'google
drivefs', 'dropbox', 'cloud'))):
970                 return ('Облачная синхронизация / Хранилище',
'Выгрузка локальных файлов, метаданные изменений', 'Синхрониза
ция файлов из облака, снимки папок')
971             if any((d in p_lower for d in ('code', 'devenv', '
language_server', 'python', 'pycharm', 'git'))):
972                 return ('Разработка / LLM API & Git', 'AI пром
пты к LLM моделям, API вызовы, git push, пакеты', 'Сгенерирова
нный AI код, ответы API, пакеты библиотек')
973             if any((w in p_lower for w in ('svchost', 'system'
, 'msmpeng', 'searchindexer', 'spoolsv'))):
974                 return ('Системные службы Windows / Облако MS'
, 'Телеметрия Defender, проверка сертификатов, запросы ОС', 'С
игнатуры безопасности, метаданные обновлений Windows')
975                 return ('HTTPS / Зашифрованный веб-трафик', 'Зашиф
```

```

    рованные HTTPS/TLS запросы и полезная нагрузка', 'Ответы удале
ного веб-сервера, зашифрованные данные')
976         if port in (80, 8080):
977             return ('HTTP / Открытый веб-трафик', 'HTTP GET/PO
ST запросы, клиентские заголовки', 'HTML веб-страницы, незашиф
рованный контент, ответы')
978         if port == 53:
979             return ('DNS / Разрешение имен', 'Запросы на получ
ение IP-адресов доменов (DNS Queries)', 'DNS-ответы с IP-адрес
ами серверов (DNS Answers)')
980         if port in (5222, 5223, 5228):
981             return ('Push-нотификации / XMPP', 'Heartbeat пинг
и, исходящие push-запросы', 'Входящие фоновые push-уведомления
и события')
982         if port == 22:
983             return ('SSH / Защищенный терминал', 'Команды конс
оли, зашифрованные SSH сессии', 'Вывод удаленного терминала, ф
айловые потоки SFTP')
984         if port == 7680:
985             return ('WUDD / P2P обновления Windows', 'P2P разд
ача фрагментов обновлений ОС', 'P2P получение компонентов обно
влений ОС')
986         if port in (465, 587):
987             return ('SMTP / Почтовая отправка', 'Исходящие эле
ктронные письма (MIME/SMTP)', 'Квитанции доставки, подтвержден
ия почтового сервера')
988         if port in (993, 995):
989             return ('IMAP/POP3 / Почтовый прием', 'Команды про
верки новых писем и папок', 'Входящие электронные письма, заго
ловки, вложения')
990         return (f'{proto}/{port}', f'Исходящие пакеты данных (
{proto}) на порт {port}', f'Входящие пакеты и ответы сервера с
порта {port}')
991
992     def get_health_alerts(self) -> SystemHealthAlerts:
993         """Check system reliability indicators and pending reb
oot status.
994

```

```

995         Returns:
996             SystemHealthAlerts: Consolidated system health alert
          indicators.
997         """
998         reboot_pending = False
999         if os.name == 'nt':
1000             try:
1001                 import winreg
1002                 reboot_keys = ['SOFTWARE\\Microsoft\\Windows\\
          CurrentVersion\\WindowsUpdate\\Auto Update\\RebootRequired', '
          SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Component Based
          Servicing\\RebootPending']
1003                 for k in reboot_keys:
1004                     try:
1005                         with winreg.OpenKey(winreg.HKEY_LOCAL_
          MACHINE, k):
1006                             reboot_pending = True
1007                             break
1008                     except OSError:
1009                         pass
1010                 except Exception:
1011                     pass
1012                 alert_msg = 'Требуется перезагрузка для обновлений Win
          dows' if reboot_pending else 'Система работает стабильно'
1013                 return SystemHealthAlerts(reboot_pending=reboot_pendin
          g, critical_events_count=0, latest_alert=alert_msg)
1014
1015     def get_monitors(self) -> List[MonitorInfo]:
1016         """Collect connected display monitors, resolutions and
          refresh rates.
1017
1018         Returns:
1019             List[MonitorInfo]: Detected active display monitor
          s.
1020         """
1021         if self._monitors_cached is not None:
1022             return self._monitors_cached
1023         monitors: List[MonitorInfo] = []

```

```

1024         if os.name == 'nt':
1025             try:
1026                 import ctypes.wintypes
1027                 user32 = ctypes.windll.user32
1028
1029                 class DISPLAY_DEVICEW(ctypes.Structure):
1030                     _fields_ = [('cb', ctypes.wintypes.DWORD),
1031                                ('DeviceName', ctypes.wintypes.WCHAR * 32), ('DeviceString',
1032                                ctypes.wintypes.WCHAR * 128), ('StateFlags', ctypes.wintypes.D
1033                                WORD), ('DeviceID', ctypes.wintypes.WCHAR * 128), ('DeviceKey'
1034                                , ctypes.wintypes.WCHAR * 128)]
1035
1036                 adapter_map: Dict[str, Dict[str, str]] = {}
1037                 for i in range(16):
1038                     disp = DISPLAY_DEVICEW()
1039                     disp.cb = ctypes.sizeof(DISPLAY_DEVICEW)
1040                     if not user32.EnumDisplayDevicesW(None, i,
1041                     ctypes.byref(disp), 0):
1042                         break
1043                     if disp.StateFlags & 1:
1044                         mon = DISPLAY_DEVICEW()
1045                         mon.cb = ctypes.sizeof(DISPLAY_DEVICEW
1046                         )
1047                         mon_name = ''
1048                         if user32.EnumDisplayDevicesW(disp.Dev
1049                         iceName, 0, ctypes.byref(mon), 0):
1050                             mon_name = mon.DeviceString
1051                             adapter_map[disp.DeviceName] = {'adapt
1052                             er': disp.DeviceString, 'mon_name': mon_name or 'Display Monit
1053                             or'}
1054
1055                 class MONITORINFOEXW(ctypes.Structure):
1056                     _fields_ = [('cbSize', ctypes.wintypes.DWO
1057                     RD), ('rcMonitor', ctypes.wintypes.RECT), ('rcWork', ctypes.wi
1058                     ntypes.RECT), ('dwFlags', ctypes.wintypes.DWORD), ('szDevice',
1059                     ctypes.wintypes.WCHAR * 32)]
1060
1061                 class DEVMODEW(ctypes.Structure):
1062                     _fields_ = [('dmDeviceName', ctypes.wintyp

```

```

es.WCHAR * 32), ('dmSpecVersion', ctypes.wintypes.WORD), ('dmD
riverVersion', ctypes.wintypes.WORD), ('dmSize', ctypes.wintyp
es.WORD), ('dmDriverExtra', ctypes.wintypes.WORD), ('dmFields'
, ctypes.wintypes.DWORD), ('dmOrientation', ctypes.c_short), (
'dmPaperSize', ctypes.c_short), ('dmPaperLength', ctypes.c_sho
rt), ('dmPaperWidth', ctypes.c_short), ('dmScale', ctypes.c_sh
ort), ('dmCopies', ctypes.c_short), ('dmDefaultSource', ctypes
.c_short), ('dmPrintQuality', ctypes.c_short), ('dmColor', cty
pes.c_short), ('dmDuplex', ctypes.c_short), ('dmYResolution',
ctypes.c_short), ('dmTTOption', ctypes.c_short), ('dmCollate',
ctypes.c_short), ('dmFormName', ctypes.wintypes.WCHAR * 32),
('dmLogPixels', ctypes.wintypes.WORD), ('dmBitsPerPel', ctypes
.wintypes.DWORD), ('dmPelsWidth', ctypes.wintypes.DWORD), ('dm
PelsHeight', ctypes.wintypes.DWORD), ('dmDisplayFlags', ctypes
.wintypes.DWORD), ('dmDisplayFrequency', ctypes.wintypes.DWORD
)]

```

1050

```

1051         def _enum_cb(h_mon: Any, hdc: Any, lprc: Any,
dw_data: Any) -> bool:

```

```

1052             info = MONITORINFOEXW()

```

```

1053             info.cbSize = ctypes.sizeof(MONITORINFOEXW
)

```

```

1054             user32.GetMonitorInfoW(h_mon, ctypes.byref
(info))

```

```

1055             dev = info.szDevice

```

```

1056             w = info.rcMonitor.right - info.rcMonitor.
left

```

```

1057             h = info.rcMonitor.bottom - info.rcMonitor
.top

```

```

1058             is_prim = bool(info.dwFlags & 1)

```

```

1059             freq = 60

```

```

1060             bits = 32

```

```

1061             dm = DEVMODEW()

```

```

1062             dm.dmSize = ctypes.sizeof(DEVMODEW)

```

```

1063             if user32.EnumDisplaySettingsW(dev, -1, ct
ypes.byref(dm)):

```

```

1064                 if dm.dmDisplayFrequency:

```

```

1065                     freq = dm.dmDisplayFrequency

```

```

1066         if dm.dmBitsPerPel:
1067             bits = dm.dmBitsPerPel
1068         if dm.dmPelsWidth and dm.dmPelsHeight:
1069             w, h = (dm.dmPelsWidth, dm.dmPelsH
eight)
1070         ad_info = adapter_map.get(dev, {})
1071         monitors.append(MonitorInfo(device=dev, na
me=ad_info.get('mon_name', 'Display Monitor'), adapter=ad_info
.get('adapter', ''), width=w, height=h, frequency_hz=freq, bit
s_per_pixel=bits, is_primary=is_prim))
1072         return True
1073         cb_type = ctypes.WINFUNCTYPE(ctypes.c_bool, ct
ypes.wintypes.HMONITOR, ctypes.wintypes.HDC, ctypes.POINTER(ct
ypes.wintypes.RECT), ctypes.wintypes.LPARAM)
1074         user32.EnumDisplayMonitors(None, None, cb_type
(_enum_cb), 0)
1075         except Exception as ex:
1076             logger.debug(f'Failed to query connected monit
ors: {ex}')
1077         if not monitors:
1078             monitors.append(MonitorInfo(device='\\\\.\\DISPLAY
1', name='Primary Monitor', adapter='Display Adapter', width=1
920, height=1080, frequency_hz=60, bits_per_pixel=32, is_prima
ry=True))
1079         self._monitors_cached = monitors
1080         return monitors
1081
1082     def get_updates_info(self) -> WindowsUpdateInfo:
1083         """Collect Windows Update status and recent installed
hotfixes.
1084
1085         Returns:
1086             WindowsUpdateInfo: Summary of updates status and i
nstalled KBs.
1087         """
1088         if self._updates_cached is not None:
1089             return self._updates_cached
1090         hotfixes: List[str] = []

```



```

1091         if os.name == 'nt':
1092             try:
1093                 import winreg
1094                 with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
1095                                     'SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Component Base
1096                                     d Servicing\\Packages') as key:
1097                     count = winreg.QueryInfoKey(key)[0]
1098                     for i in range(count):
1099                         try:
1100                             pkg_name = winreg.EnumKey(key, i)
1101                             if 'Package_for_KB' in pkg_name or
1102                             'Package_for_RollupFix' in pkg_name:
1103                                 parts = pkg_name.split('~')
1104                                 kb = parts[0].replace('Package
1105                                 _for_', '')
1106                                 if kb not in hotfixes:
1107                                     hotfixes.append(kb)
1108                                 except OSError:
1109                                     pass
1110                             except Exception as ex:
1111                                 logger.debug(f'Fast hotfix registry scan excep
1112                                 tion: {ex}')
1113             if not hotfixes:
1114                 hotfixes = ['KB5126052', 'KB5054156', 'KB5071430',
1115                             'KB5129195', 'KB5124007']
1116             info = WindowsUpdateInfo(status='Up to date (Актуально
1117             )', installed_kb_count=len(hotfixes), recent_hotfixes=hotfixes
1118             [:10], latest_installed_on='Недавние исправления установлены')
1119             self._updates_cached = info
1120             return info
1121
1122     def get_ms_office_info(self) -> OfficeSuiteInfo:
1123         """Сбор информации об установленном пакете Microsoft O
1124         ffice / 365.
1125
1126         Returns:
1127             OfficeSuiteInfo: Сведения о версии и статусе пакет
1128             a Office.

```

```

1119 | """
1120 | if self._office_cached is not None:
1121 |     return self._office_cached
1122 | installed = False
1123 | product_name = None
1124 | version = None
1125 | publisher = None
1126 | if os.name == 'nt':
1127 |     try:
1128 |         import winreg
1129 |         ctr_keys = [(winreg.HKEY_LOCAL_MACHINE, 'SOFTWARE\\Microsoft\\Office\\ClickToRun\\Configuration'), (winreg.HKEY_LOCAL_MACHINE, 'SOFTWARE\\WOW6432Node\\Microsoft\\Office\\ClickToRun\\Configuration')]
1130 |         for root, subkey in ctr_keys:
1131 |             try:
1132 |                 with winreg.OpenKey(root, subkey) as k:
1133 |                     for val_name in ('ProductVersion', 'VersionToReport', 'ClientVersionToReport'):
1134 |                         try:
1135 |                             v, _ = winreg.QueryValueEx(k, val_name)
1136 |                             if v:
1137 |                                 version = str(v)
1138 |                                 break
1139 |                         except OSError:
1140 |                             pass
1141 |                     try:
1142 |                         prod_ids, _ = winreg.QueryValueEx(k, 'ProductReleaseIds')
1143 |                         if prod_ids:
1144 |                             product_name = str(prod_ids).replace('Volume', '').replace('Retail', '').strip()
1145 |                         except OSError:
1146 |                             pass
1147 |                     if version:
1148 |                         installed = True

```

```

1149         publisher = 'Microsoft Corpora
tion'
1150         break
1151     except OSError:
1152         pass
1153     if not installed:
1154         for root in (winreg.HKEY_LOCAL_MACHINE, wi
nreg.HKEY_CURRENT_USER):
1155             for sub in ('SOFTWARE\\Microsoft\\Wind
ows\\CurrentVersion\\Uninstall', 'SOFTWARE\\WOW6432Node\\Micro
soft\\Windows\\CurrentVersion\\Uninstall'):
1156                 try:
1157                     with winreg.OpenKey(root, sub)
as key:
1158                         count = winreg.QueryInfoKe
y(key)[0]
1159                         for i in range(count):
1160                             try:
1161                                 sk_name = winreg.E
numKey(key, i)
1162                                 with winreg.OpenKe
y(key, sk_name) as sk:
1163                                     dn, _ = winreg
.QueryValueEx(sk, 'DisplayName')
1164                                     dn_str = str(d
n)
1165                                     if 'microsoft
office' in dn_str.lower() or 'microsoft 365' in dn_str.lower()
or 'office 16' in dn_str.lower() or ('office 15' in dn_str.lo
wer()):
1166                                         try:
1167                                             dv, _
= winreg.QueryValueEx(sk, 'DisplayVersion')
1168                                             versio
n = str(dv)
1169                                         except OSE
rror:
1170                                             pass

```

```

1171 |                                     try:
1172 |                                     pub, _
1173 |                                     = winreg.QueryValueEx(sk, 'Publisher')
1174 |                                     publis
1175 |                                     her = str(pub)
1176 |                                     except OSError
1177 |                                     rror:
1178 |                                     publis
1179 |                                     her = 'Microsoft Corporation'
1180 |                                     product_na
1181 |                                     me = dn_str
1182 |                                     installed
1183 |                                     = True
1184 |                                     break
1185 |                                     except OSError:
1186 |                                     pass
1187 |                                     if installed:
1188 |                                     break
1189 |                                     except OSError:
1190 |                                     pass
1191 |                                     if installed:
1192 |                                     break
1193 |                                     except Exception as ex:
1194 |                                     logger.debug(f'Исключение при поиске MS Office
1195 |                                     : {ex}')
1196 |                                     if installed:
1197 |                                     ver_suffix = f' (v{version})' if version else ''
1198 |                                     summary = f"{product_name or 'Microsoft Office'}{v
1199 |                                     er_suffix}"
1200 |                                     else:
1201 |                                     summary = 'Не установлен'
1202 |                                     res = OfficeSuiteInfo(installed=installed, product_nam
1203 |                                     e=product_name, version=version, publisher=publisher, status=s
1204 |                                     ummary)
1205 |                                     self._office_cached = res
1206 |                                     return res
1207 |
1208 |     def get_onedrive_info(self) -> CloudStorageInfo:

```

```

1199 |         """Сбор информации о папке/диске OneDrive и состоянии
      | синхронизации.
1200 |
1201 |         Returns:
1202 |             CloudStorageInfo: Сведения о локальном пути, емкос
      | ти диска и статусе синхронизации OneDrive.
1203 |         """
1204 |         import shutil
1205 |         od_paths: List[str] = []
1206 |         for env_var in ('OneDrive', 'OneDriveConsumer', 'OneDr
      | iveCommercial'):
1207 |             val = os.environ.get(env_var)
1208 |             if val and os.path.exists(val) and (val not in od_
      | paths):
1209 |                 od_paths.append(val)
1210 |         if not od_paths:
1211 |             home = os.path.expanduser('~')
1212 |             candidate = os.path.join(home, 'OneDrive')
1213 |             if os.path.exists(candidate):
1214 |                 od_paths.append(candidate)
1215 |         if os.name == 'nt':
1216 |             try:
1217 |                 import winreg
1218 |                 for subkey in ('Software\\Microsoft\\OneDrive\\
      | \\Accounts\\Personal', 'Software\\Microsoft\\OneDrive\\Accounts
      | \\Business1', 'Software\\Microsoft\\OneDrive'):
1219 |                     try:
1220 |                         with winreg.OpenKey(winreg.HKEY_CURREN
      | T_USER, subkey) as k:
1221 |                             val, _ = winreg.QueryValueEx(k, 'U
      | serFolder')
1222 |                             if val and os.path.exists(val) and
      | (val not in od_paths):
1223 |                                 od_paths.append(val)
1224 |                     except OSError:
1225 |                         pass
1226 |             except Exception:
1227 |                 pass

```

```

1228
1229         if not od_paths:
1230             return CloudStorageInfo(installed=False, name='One
Drive', path=None, status='Не настроено')
1231
1232         target_path = od_paths[0]
1233
1234         # Проверка активности процесса синхронизации OneDrive
1235         is_running = False
1236         try:
1237             for p in psutil.process_iter(['name']):
1238                 try:
1239                     pname = (p.info.get('name') or '').lower()
1240                     if pname in ('onedrive.exe', 'onedrive.sta
ndalone.exe', 'filecoauth.exe'):
1241                         is_running = True
1242                         break
1243             except (psutil.NoSuchProcess, psutil.AccessDen
ied):
1244                 continue
1245         except Exception:
1246             pass
1247
1248         try:
1249             usage = shutil.disk_usage(target_path)
1250             free_gb = round(usage.free / (1024 ** 3), 1)
1251             total_gb = round(usage.total / (1024 ** 3), 1)
1252             used_gb = round(usage.used / (1024 ** 3), 1)
1253             pct = round(usage.used / max(usage.total, 1) * 100
.0, 1)
1254
1255             if is_running:
1256                 summary = f'{free_gb} GB своб. (Синхронизирова
н)'
1257             else:
1258                 summary = f'Локальный диск ({free_gb} GB своб.
, не синхронизирован)'
1259

```

```

1260         return CloudStorageInfo(
1261             installed=True,
1262             name='OneDrive',
1263             path=target_path,
1264             total_gb=total_gb,
1265             used_gb=used_gb,
1266             free_gb=free_gb,
1267             percent_used=pct,
1268             status=summary,
1269         )
1270     except Exception as ex:
1271         logger.debug(f'Ошибка проверки диска OneDrive: {ex}
1272         })
1273         fallback_status = f'Локальный диск ({target_path},
1274         не синхронизирован)' if not is_running else f'Активен ({target_path})'
1275         return CloudStorageInfo(installed=True, name='OneDrive', path=target_path, status=fallback_status)
1276
1277     async def get_core_metrics(self) -> SystemCoreMetrics:
1278         """Сверхбыстрый сбор базовой телеметрии без вызова тяжелых внешних утилит (< 0.2с).
1279
1280         Returns:
1281             SystemCoreMetrics: Мгновенные показатели загрузки процессора, памяти, дисков и видеокарты.
1282         """
1283         now = time.time()
1284         uptime = round(now - (psutil.boot_time() if PSUTIL_AVAILABLE else now - 3600), 1)
1285         _, disk_io = self.get_disk_metrics()
1286         mems = self.get_memory_metrics()
1287         gpus = self.get_gpu_metrics()
1288         batt = self.get_battery_metrics()
1289         cpu_metrics = await self.get_cpu_metrics()
1290         self._last_time = now
1291         return SystemCoreMetrics(cpu=cpu_metrics, memory=mems, gpus=gpus, disk_io=disk_io, battery=batt, uptime_seconds=uptime)

```

```

me)
1290
1291     async def get_hardware_quick(self) -> SystemHardwareQuick:
1292         """Сбор сводки физических компонентов (диски, слоты па
        мяти, порты, алерты).
1293
1294         Returns:
1295             SystemHardwareQuick: Состояние накопителей, планок
        RAM и сетевых сокетов.
1296         """
1297         ram_sticks = await self.get_ram_sticks()
1298         phys_disks = self.get_physical_disks_health()
1299         ports = self.get_listening_ports()
1300         alerts = self.get_health_alerts()
1301         return SystemHardwareQuick(ram_sticks=ram_sticks, phys
        ical_disks=phys_disks, listening_ports=ports, alerts=alerts)
1302
1303     async def get_snapshot(self, process_limit: int=20) -> Sys
        temSnapshot:
1304         """Capture full point-in-time system telemetry snapsho
        t without blocking event loop.
1305
1306         Args:
1307             process_limit: Number of top active processes to i
        nclude.
1308
1309         Returns:
1310             SystemSnapshot: Consolidated system and hardware s
        napshot.
1311         """
1312
1313         def _collect_sync_telemetry() -> Dict[str, Any]:
1314             now = time.time()
1315             uptime = round(now - (psutil.boot_time() if PSUTIL
        _AVAILABLE else now - 3600), 1)
1316             partitions, disk_io = self.get_disk_metrics()
1317             net_metrics = self.get_network_metrics()
1318             sensors: List[HardwareSensor] = []

```



```

1319         if self.storage:
1320             try:
1321                 db_sensors = self.storage.get_latest_senso
rs()
1322                 for item in db_sensors:
1323                     sensors.append(
1324                         HardwareSensor(
1325                             sensor_id=str(item.get('sensor
_id') or item.get('name') or 'unknown'),
1326                             name=str(item.get('name') or i
tem.get('sensor_id') or 'sensor'),
1327                             sensor_type=str(item.get('sens
or_type') or 'temperature'),
1328                             value=float(item.get('value')
or 0.0),
1329                             unit=str(item.get('unit') or '
°C'),
1330                             )
1331                     )
1332             except Exception:
1333                 sensors = []
1334         top_procs = self.get_top_processes(limit=process_l
imit)
1335         ident = self.get_system_identity()
1336         mems = self.get_memory_metrics()
1337         gpu_list = self.get_gpu_metrics()
1338         mon_list = self.get_monitors()
1339         upd_info = self.get_updates_info()
1340         off_info = self.get_ms_office_info()
1341         od_info = self.get_onedrive_info()
1342         phys_disks = self.get_physical_disks_health()
1343         ports = self.get_listening_ports()
1344         net_activity = self.get_process_network_activity(l
imit=40)
1345         batt = self.get_battery_metrics()
1346         alerts = self.get_health_alerts()
1347         self._last_time = now
1348         return {'now': now, 'uptime': uptime, 'partitions'

```

```

: partitions, 'disk_io': disk_io, 'net_metrics': net_metrics,
'sensors': sensors, 'top_procs': top_procs, 'ident': ident, 'm
emory': mems, 'gpus': gpu_list, 'monitors': mon_list, 'updates
': upd_info, 'office': off_info, 'onedrive': od_info, 'physica
l_disks': phys_disks, 'listening_ports': ports, 'net_activity'
: net_activity, 'battery': batt, 'alerts': alerts}
1349         cpu_task = asyncio.create_task(self.get_cpu_metrics())
1350         ram_task = asyncio.create_task(self.get_ram_sticks())
1351         sync_task = asyncio.create_task(asyncio.to_thread(_col
lect_sync_telemetry))
1352         cpu_metrics, ram_sticks, sync_data = await asyncio.gat
her(cpu_task, ram_task, sync_task)
1353         ident = sync_data['ident']
1354         # Читаем последний зафиксированный аудит оборудования
из базы данных (без живого опроса PnP/реестра/WMI)
1355         hardware_audit_dict: Dict[str, Any] = {}
1356         if self.storage:
1357             try:
1358                 latest_audit = self.storage.get_latest_hardwar
e_audit() or self.storage.get_latest_extended_audit()
1359                 if latest_audit:
1360                     hardware_audit_dict = latest_audit
1361             except Exception:
1362                 pass
1363         return SystemSnapshot(
1364             hostname=ident.get('hostname') or socket.gethostna
me(),
1365             workgroup=ident.get('workgroup') or 'WORKGROUP',
1366             username=ident.get('username') or '',
1367             os_name=f'{platform.system()} {platform.release()}'
',
1368             os_build=ident.get('os_build') or platform.version
(),
1369             system_language=ident.get('system_language') or ''
',
1370             os_install_language=ident.get('os_install_language
') or '',
1371             user_locale=ident.get('user_locale') or '',

```

```

1372         system_locale=ident.get('system_locale') or '',
1373         timezone=ident.get('timezone') or '',
1374         codepage=ident.get('codepage') or '',
1375         input_languages=ident.get('input_languages') or []
1376     ,
1377     os_install_date=ident.get('os_install_date') or ''
1378     ,
1379     uptime_seconds=sync_data['uptime'],
1380     cpu=cpu_metrics,
1381     memory=sync_data['memory'],
1382     ram_sticks=ram_sticks,
1383     gpus=sync_data['gpus'],
1384     monitors=sync_data['monitors'],
1385     updates=sync_data['updates'],
1386     office=sync_data['office'],
1387     onedrive=sync_data['onedrive'],
1388     disks=sync_data['partitions'],
1389     physical_disks=sync_data['physical_disks'],
1390     disk_io=sync_data['disk_io'],
1391     network=sync_data['net_metrics'],
1392     listening_ports=sync_data['listening_ports'],
1393     network_activity=sync_data['net_activity'],
1394     battery=sync_data['battery'],
1395     alerts=sync_data['alerts'],
1396     sensors=sync_data['sensors'],
1397     top_processes=sync_data['top_procs'],
1398     hardware_audit=hardware_audit_dict
1399 )
1400
1401 def get_hardware_sensors(self) -> List[HardwareSensor]:
1402     """Collect real-time hardware sensors readings.
1403
1404     Returns:
1405         List[HardwareSensor]: Hardware sensors and tempera
1406     tures.
1407
1408     """
1409     return get_hardware_sensors()
1410

```

```

1407     async def get_hardware_tree_async(self, force: bool = False) -> List[HardwareNode]:
1408         """Generate AIDA64-like hierarchical component specification tree (Async).
1409
1410         Состояние оборудования сканируется 1 раз при запуске процесса или по отдельному запросу (force=True).
1411
1412         Args:
1413             force: Принудительный повторный опрос оборудования
1414             .
1415         Returns:
1416             List[HardwareNode]: Hardware devices grouped by category.
1417         """
1418         if not force and getattr(self, '_hardware_tree_cached', None) is not None:
1419             return self._hardware_tree_cached
1420         if os.name == 'nt':
1421             try:
1422                 import pythoncom
1423                 pythoncom.CoInitialize()
1424             except Exception as e:
1425                 logger.debug(f'[HardwareTree] CoInitialize warning: {e}')
1426         nodes: List[HardwareNode] = []
1427         ident = self.get_system_identity()
1428         system_props = {
1429             'Компьютер (Host)': ident.get('hostname', ''),
1430             'Пользователь': ident.get('username', ''),
1431             'Версия Windows': platform.version(),
1432             'Номер сборки': ident.get('os_build', ''),
1433             'Дата установки ОС': ident.get('os_install_date', ''),
1434             'Язык системы': ident.get('system_language', ''),
1435             'Язык установки (Оригинал)': ident.get('os_install_language', ''),

```

```

1436         'Локали': f"User: {ident.get('user_locale', '')} |
        Sys: {ident.get('system_locale', '')}",
1437         'Часовой пояс': ident.get('timezone', ''),
1438         'Кодировки': ident.get('codepage', ''),
1439         'Языки ввода': ', '.join(ident.get('input_language
        s', []))),
1440         'Архитектура': platform.machine(),
1441         'Среда Python': platform.python_version()
1442     }
1443     nodes.append(HardwareNode(category='System', name=f"{i
        dent.get('hostname')} ({platform.system()} {platform.release()
        })", properties=system_props))
1444     cpu = await self.get_cpu_metrics()
1445     cpu_props: Dict[str, Any] = {'Модель процессора': cpu.
        model, 'Физические ядра': cpu.physical_cores, 'Логические пото
        ки': cpu.logical_cores, 'Базовая частота': f'{cpu.frequency_mh
        z} MHz', 'Архитектура': cpu.architecture}
1446     if os.name == 'nt':
1447         try:
1448             import win32com.client
1449             wmi_obj = win32com.client.GetObject('winmgmts:
        ')
1450             for p in wmi_obj.InstancesOf('Win32_Processor'
        ):
1451                 if getattr(p, 'SocketDesignation', None):
1452                     cpu_props['Разъем (Socket)'] = p.Socke
        tDesignation
1453                 if getattr(p, 'L2CacheSize', None):
1454                     cpu_props['Кэш L2'] = f'{round(p.L2Cac
        heSize / 1024, 1)} MB ({p.L2CacheSize} KB)'
1455                 if getattr(p, 'L3CacheSize', None):
1456                     cpu_props['Кэш L3'] = f'{round(p.L3Cac
        heSize / 1024, 1)} MB ({p.L3CacheSize} KB)'
1457                 if getattr(p, 'MaxClockSpeed', None):
1458                     cpu_props['Макс. частота'] = f'{p.MaxC
        lockSpeed} MHz'
1459                 if getattr(p, 'Manufacturer', None):
1460                     cpu_props['Производитель'] = p.Manufac

```

```

turer
1461         break
1462     except Exception as e:
1463         logger.debug(f'[HardwareTree] WMI Win32_Process
sor warning: {e}')
1464         nodes.append(HardwareNode(category='Processor (CPU)',
name=cpu.model, properties=cpu_props))
1465         if os.name == 'nt':
1466             mb_props: Dict[str, Any] = {}
1467             mb_name = ''
1468             mb_mfg = ''
1469             try:
1470                 import winreg
1471                 with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
'HARDWARE\\DESCRIPTION\\System\\BIOS') as key:
1472                     mb_mfg, _ = winreg.QueryValueEx(key, 'Base
BoardManufacturer')
1473                     mb_name, _ = winreg.QueryValueEx(key, 'Bas
eBoardProduct')
1474                     bios_ver, _ = winreg.QueryValueEx(key, 'BI
OSVersion')
1475                     if mb_mfg:
1476                         mb_props['Производитель платы'] = mb_m
fg
1477                     if mb_name:
1478                         mb_props['Модель платы'] = mb_name
1479                     if bios_ver:
1480                         mb_props['Версия BIOS'] = bios_ver
1481             except Exception:
1482                 pass
1483             try:
1484                 import win32com.client
1485                 wmi_obj = win32com.client.GetObject('winmgmts:
')
1486                 for b in wmi_obj.InstancesOf('Win32_BaseBoard'
):
1487                     if getattr(b, 'Manufacturer', None):
1488                         mb_props['Производитель платы'] = b.Ma

```

```

    nufacturer
1489         if getattr(b, 'Product', None):
1490             mb_props['Модель платы'] = b.Product
1491         if getattr(b, 'SerialNumber', None):
1492             mb_props['Серийный номер платы'] = (b.
SerialNumber or '').strip()
1493         if getattr(b, 'Version', None):
1494             mb_props['Ревизия / Версия'] = b.Versi
on
1495             mb_name = mb_props.get('Модель платы', mb_
name)
1496             mb_mfg = mb_props.get('Производитель платы
', mb_mfg)
1497             break
1498         for bios in wmi_obj.InstancesOf('Win32_BIOS'):
1499             if getattr(bios, 'Manufacturer', None):
1500                 mb_props['Производитель BIOS'] = bios.
Manufacturer
1501             if getattr(bios, 'SMBIOSBIOSVersion', None
):
1502                 mb_props['Версия SMBIOS BIOS'] = bios.
SMBIOSBIOSVersion
1503             if getattr(bios, 'ReleaseDate', None):
1504                 rel_date = str(bios.ReleaseDate)[:8]
1505                 if len(rel_date) == 8:
1506                     mb_props['Дата выпуска BIOS'] = f'
{rel_date[6:8]}.{rel_date[4:6]}.{rel_date[0:4]}'
1507                 else:
1508                     mb_props['Дата выпуска BIOS'] = st
r(bios.ReleaseDate)
1509             break
1510         except Exception as e:
1511             logger.debug(f'[HardwareTree] WMI BaseBoard/BI
OS warning: {e}')
1512             if mb_props:
1513                 nodes.append(HardwareNode(category='Motherboar
d', name=f'{mb_mfg} {mb_name}'.strip() or 'Motherboard', prope
rties=mb_props))

```

```

1514         mem = self.get_memory_metrics()
1515         mem_overview_props: Dict[str, Any] = {'Общий объем RAM': f'{mem.total_gb} GB', 'Доступная память': f'{mem.available_gb} GB', 'Файл подкачки (Swap)': f'{mem.swap_total_gb} GB'}
1516         ram_modules_found = 0
1517         if os.name == 'nt':
1518             try:
1519                 import win32com.client
1520                 wmi_obj = win32com.client.GetObject('winmgmts:')
1521                 total_slots = 0
1522                 max_capacity_str = ''
1523                 for arr in wmi_obj.InstancesOf('Win32_PhysicalMemoryArray'):
1524                     if getattr(arr, 'MemoryDevices', None):
1525                         total_slots = int(arr.MemoryDevices)
1526                         mem_overview_props['Слотов памяти на плате'] = total_slots
1527                     if getattr(arr, 'MaxCapacity', None):
1528                         max_gb = round(int(arr.MaxCapacity) / 1024 ** 2, 1)
1529                         max_capacity_str = f'{max_gb} GB'
1530                         mem_overview_props['Макс. поддерживаемый объем'] = max_capacity_str
1531                     break
1532                 type_map = {20: 'DDR', 21: 'DDR2', 22: 'DDR2 FB-DIMM', 24: 'DDR3', 26: 'DDR4', 27: 'LPDDR', 28: 'LPDDR2', 29: 'LPDDR3', 30: 'LPDDR4', 34: 'DDR5', 35: 'LPDDR5'}
1533                 form_map = {8: 'DIMM (Desktop)', 12: 'SODIMM (Laptop)', 9: 'TSOP', 10: 'PGA', 11: 'RIMM'}
1534
1535                 raw_modules = list(wmi_obj.InstancesOf('Win32_PhysicalMemory'))
1536                 ram_modules_found = len(raw_modules)
1537                 module_speeds = [int(getattr(m, 'Speed', 0) or 0) for m in raw_modules if getattr(m, 'Speed', 0)]
1538                 module_clocks = [int(getattr(m, 'ConfiguredClockSpeed', 0) or getattr(m, 'Speed', 0) or 0) for m in raw_modules if getattr(m, 'ConfiguredClockSpeed', 0) or getattr(m, 'Speed', 0) or 0)]

```



```

les if (getattr(m, 'ConfiguredClockSpeed', 0) or getattr(m, 'S
peed', 0))]
1539         max_module_speed = max(module_speeds) if modul
e_speeds else 0
1540         min_module_speed = min(module_speeds) if modul
e_speeds else 0
1541
1542         for idx, m in enumerate(raw_modules):
1543             cap_bytes = int(getattr(m, 'Capacity', 0)
or 0)
1544             cap_gb = round(cap_bytes / 1024 ** 3, 1)
1545             smbios_type = int(getattr(m, 'SMBIOSMemory
Type', 0) or 0)
1546             mem_type = type_map.get(smbios_type, f'DDR
(Тип {smbios_type})' if smbios_type else 'DDR')
1547             form_code = int(getattr(m, 'FormFactor', 0
) or 0)
1548             form_name = form_map.get(form_code, 'DIMM'
)
1549             mfg = (getattr(m, 'Manufacturer', '') or '
').strip()
1550             if mfg == '04CB000080CE':
1551                 mfg_display = 'ADATA / Micron (04CB)'
1552             elif mfg:
1553                 mfg_display = mfg
1554             else:
1555                 mfg_display = 'Не определен'
1556             sn = (getattr(m, 'SerialNumber', '') or ''
).strip() or 'N/A'
1557             pn = (getattr(m, 'PartNumber', '') or '').
strip() or 'N/A'
1558             speed = getattr(m, 'Speed', 0) or 0
1559             clock = getattr(m, 'ConfiguredClockSpeed',
0) or speed
1560             locator = (getattr(m, 'DeviceLocator', '')
or f'DIMM {idx + 1}').strip()
1561             bank = (getattr(m, 'BankLabel', '') or '')
.strip()

```

```

1562
1563             # Определение статуса согласования частот
            и выявление узких мест
1564             if speed and clock and clock < speed:
1565                 compat_status = f'␣ Потенциал не раскр
            ыт: планка рассчитана на {speed} MHz, но контроллер/BIOS ограничил до {clock} MHz (потеря {speed - clock} MHz)'
1566                 freq_warning = f' (␣ занижена на {speed - clock} MHz)'
1567                 eff_percent = f'{round((clock / speed)
            * 100, 1)}% от номинала планки'
1568             elif max_module_speed and speed and speed
            < max_module_speed:
1569                 compat_status = f'␣ Узкое место: планка
            а {speed} MHz медленнее других модулей ({max_module_speed} MHz
            ), снижает общую частоту'
1570                 freq_warning = ''
1571                 eff_percent = '100% (номинал)'
1572             else:
1573                 compat_status = f'␣ Оптимально: планка
            работает на паспортной частоте ({clock} MHz)'
1574                 freq_warning = ''
1575                 eff_percent = '100% (номинал)'
1576
1577             module_props: Dict[str, Any] = {
1578                 'Слот / Разъем (Locator)': locator,
1579                 'Объем планки': f'{cap_gb} GB ({cap_bytes:,} байт)',
1580                 'Номинальная частота': f'{speed} MHz'
            if speed else 'N/A',
1581                 'Текущая рабочая частота': f'{clock} MHz{freq_warning}' if clock else 'N/A',
1582                 'Эффективность планки': eff_percent,
1583                 'Состояние согласования частоты': compat_status,
1584                 'Тип памяти': f'{mem_type} (SMBIOS {smbios_type})',
1585                 'Форм-фактор': form_name,

```

```

1586 |             'Производитель': mfg_display,
1587 |             'Серийный номер (S/N)': sn,
1588 |             'Парт-номер (P/N)': pn,
1589 |         }
1590 |         if bank:
1591 |             module_props['Банк памяти (Bank)'] = b
1592 |         nodes.append(HardwareNode(category='Memory
Module (RAM)', name=f'Планка {idx + 1} ({locator}): {cap_gb}
GB {mem_type}-{speed or clock}', properties=module_props))
1593 |     except Exception as e:
1594 |         logger.debug(f'[HardwareTree] WMI PhysicalMemo
ry query warning: {e}')
1595 |     if ram_modules_found > 0:
1596 |         mem_overview_props['Установлено модулей'] = f'{ram
_modules_found} шт.'
1597 |         if total_slots > 0:
1598 |             free_slots = max(0, total_slots - ram_modules_
found)
1599 |             mem_overview_props['Конфигурация слотов'] = f'
Занято {ram_modules_found} из {total_slots} (свободно: {free_s
lots})'
1600 |             if module_clocks and module_speeds:
1601 |                 min_c = min(module_clocks)
1602 |                 max_s = max(module_speeds)
1603 |                 if min_c < max_s:
1604 |                     mem_overview_props['Статус частоты памяти'
] = f'⚠ Частота шины занижена до {min_c} MHz (потенциал планок
: {max_s} MHz, проверьте XMP/BIOS)'
1605 |                 else:
1606 |                     mem_overview_props['Статус частоты памяти'
] = f'⚠ Оптимальный ({min_c} MHz, 100% номинала)'
1607 |             nodes.append(HardwareNode(category='System Memory', na
me=f'{mem.total_gb} GB Physical RAM', properties=mem_overview_
props))
1608 |     wmi_video_map: Dict[str, Dict[str, Any]] = {}
1609 |     if os.name == 'nt':
1610 |         try:

```

```

1611         import win32com.client
1612         wmi_obj = win32com.client.GetObject('winmgmts:
1613         ')
1614         for v in wmi_obj.InstancesOf('Win32_VideoContr
1615         oller'):
1616             v_name = (getattr(v, 'Name', '') or '').st
1617             rip()
1618             v_pnp = (getattr(v, 'PNPDeviceID', '') or
1619             '').strip()
1620             v_mfg = (getattr(v, 'AdapterCompatibility'
1621             , '') or '').strip()
1622             v_drv_ver = (getattr(v, 'DriverVersion', '
1623             ') or '').strip()
1624             v_drv_date_raw = getattr(v, 'DriverDate',
1625             None)
1626             v_drv_date = ''
1627             if v_drv_date_raw:
1628                 ds = str(v_drv_date_raw)[:8]
1629                 if len(ds) == 8 and ds.isdigit():
1630                     v_drv_date = f'{ds[6:8]}.{ds[4:6]}
1631                     .{ds[0:4]}'
1632             v_proc = (getattr(v, 'VideoProcessor', '')
1633             or '').strip()
1634             v_mode = (getattr(v, 'VideoModeDescription
1635             ', '') or '').strip()
1636             v_refresh = getattr(v, 'CurrentRefreshRate
1637             ', None)
1638             v_status = (getattr(v, 'Status', '') or '0
1639             K').strip()
1640             info_dict = {'name': v_name, 'mfg': v_mfg,
1641             'drv_ver': v_drv_ver, 'drv_date': v_drv_date, 'proc': v_proc,
1642             'mode': v_mode, 'refresh': v_refresh, 'status': v_status, 'pn
1643             p': v_pnp}
1644             if v_name:
1645                 wmi_video_map[v_name.lower()] = info_d
1646             ict
1647         except Exception as e:
1648             logger.debug(f'[HardwareTree] WMI VideoControl

```

```

ler query warning: {e}')
1633         gpus = self.get_gpu_metrics()
1634         for idx, gpu in enumerate(gpus):
1635             v_extra = wmi_video_map.get(gpu.name.lower(), {})
1636             gpu_mfg = v_extra.get('mfg') or ('NVIDIA' if 'gefo
rce' in gpu.name.lower() or 'nvidia' in gpu.name.lower() else
'Intel' if 'intel' in gpu.name.lower() else 'AMD' if 'radeon'
in gpu.name.lower() else 'Unknown')
1637             drv_ver = v_extra.get('drv_ver')
1638             drv_date = v_extra.get('drv_date')
1639             vproc = v_extra.get('proc')
1640             vmode = v_extra.get('mode')
1641             gpu_props: Dict[str, Any] = {'Индекс устройства':
idx, 'Модель видеокарты': gpu.name, 'Производитель (Вендор)':
gpu_mfg, 'Объем видеопамяти (VRAM)': f'{gpu.memory_total_gb} G
B'}
1642             if vproc:
1643                 gpu_props['Видеопроцессор'] = vproc
1644             if drv_ver:
1645                 gpu_props['Версия драйвера'] = drv_ver
1646             if drv_date:
1647                 gpu_props['Дата драйвера'] = drv_date
1648             if vmode:
1649                 gpu_props['Текущий видеорежим'] = vmode
1650             elif v_extra.get('refresh'):
1651                 gpu_props['Частота обновления'] = f"{v_extra['
refresh']}] Hz"
1652             gpu_props['Поддержка CUDA'] = 'Да' if gpu.has_cuda
else 'Нет'
1653             gpu_props['Поддержка DirectML'] = 'Да' if gpu.has_
directml else 'Нет'
1654             gpu_props['Статус устройства'] = v_extra.get('stat
us', 'OK (Активно)')
1655             if v_extra.get('pnp'):
1656                 gpu_props['Идентификатор PnP'] = v_extra['pnp'
]
1657             nodes.append(HardwareNode(category='Display Adapte
r', name=gpu.name, properties=gpu_props))

```



```

1679         elif 'toshiba' in model_l or model_l.start
|swith('hdwd') or model_l.startswith('dt01'):
1680             disk_mfg = 'Toshiba'
1681             elif 'wdc' in model_l or model_l.startswit
|h('wd') or 'western digital' in model_l:
1682                 disk_mfg = 'Western Digital'
1683             elif 'kingston' in model_l or model_l.star
|tswith('sa400') or model_l.startswith('skc'):
1684                 disk_mfg = 'Kingston'
1685             elif 'seagate' in model_l or model_l.start
|swith('st'):
1686                 disk_mfg = 'Seagate'
1687             elif 'sandisk' in model_l:
1688                 disk_mfg = 'SanDisk'
1689             elif 'kioxia' in model_l:
1690                 disk_mfg = 'Kioxia'
1691             elif 'intel' in model_l:
1692                 disk_mfg = 'Intel'
1693             elif 'micron' in model_l:
1694                 disk_mfg = 'Micron'
1695             elif 'sk hynix' in model_l or 'hynix' in m
|odel_l:
1696                 disk_mfg = 'SK Hynix'
1697             else:
1698                 disk_mfg = 'Универсальный накопитель'
1699                 disk_props: Dict[str, Any] = {'Модель нако
|пителя': model, 'Производитель (Бренд)': disk_mfg, 'Интерфейс
|подключения': iface, 'Тип носителя': media, 'Емкость': f'{size
|_gb} GB ({size_bytes:,} байт)', 'Серийный номер (S/N)': sn}
1700                 if firmware:
1701                     disk_props['Версия прошивки (Firmware)
|'] = firmware
1702                 disk_props['Число разделов'] = parts
1703                 disk_props['Статус S.M.A.R.T.'] = status
1704                 nodes.append(HardwareNode(category='Physic
|al Disk', name=f'{model} ({size_gb} GB)', properties=disk_prop
|s))
1705             except Exception as e:

```



```

1728 |             atype = (getattr(a, 'AdapterType', '') or
1729 |             '').strip()
1730 |             astatus_num = getattr(a, 'NetConnectionSta
1731 | tus', None)
1732 |             aphys = getattr(a, 'PhysicalAdapter', Fals
1733 | e)
1734 |             item = {'net_id': net_id, 'name': aname, '
1735 | desc': adesc, 'pnp': apnp, 'mfg': amfg, 'mac': amac, 'speed':
1736 | aspeed, 'ad_type': atype, 'status_num': astatus_num, 'is_phys'
1737 | : aphys}
1738 |             if net_id:
1739 |                 wmi_net_adapters[net_id.lower()] = ite
1740 | m
1741 |             if aname:
1742 |                 wmi_net_adapters[aname.lower()] = item
1743 |             if adesc:
1744 |                 wmi_net_adapters[adesc.lower()] = item
1745 | except Exception as e:
1746 |     logger.debug(f'[HardwareTree] WMI NetworkAdapt
1747 | er query warning: {e}')
1748 |     if PSUTIL_AVAILABLE:
1749 |         ps_addrs = psutil.net_if_addrs()
1750 |         ps_stats = psutil.net_if_stats()
1751 |         for if_name, addr_list in ps_addrs.items():
1752 |             stat = ps_stats.get(if_name)
1753 |             w_info = wmi_net_adapters.get(if_name.lower(),
1754 |             {})
1755 |             cfg = wmi_net_configs.get(w_info.get('desc', '
1756 | ').lower()) or wmi_net_configs.get(if_name.lower())
1757 |             mac_addr = w_info.get('mac')
1758 |             if not mac_addr:
1759 |                 for a in addr_list:
1760 |                     fam_name = getattr(a.family, 'name', '
1761 | ')
1762 |                     if fam_name == 'AF_LINK' or a.family ==
1763 | = -1 or a.family == getattr(psutil, 'AF_LINK', -1):
1764 |                         if a.address and len(a.address) in
1765 | (12, 17):

```

```

1753 |             mac_addr = a.address.replace('
| -', ':').upper()
1754 |             break
1755 |             ipv4_list = [a.address for a in addr_list if a
| .family == socket.AF_INET]
1756 |             ipv6_list = [a.address.split('%')[0] for a in
| addr_list if hasattr(socket, 'AF_INET6') and a.family == socke
| t.AF_INET6]
1757 |             gateways = list(getattr(cfg, 'DefaultIPGateway
| ', []) or []) if cfg else []
1758 |             dhcp_server = getattr(cfg, 'DHCPServer', None)
| if cfg else None
1759 |             dhcp_enabled = getattr(cfg, 'DHCPEnabled', Non
| e) if cfg else None
1760 |             dns_servers = list(getattr(cfg, 'DNSServerSear
| chOrder', []) or []) if cfg else []
1761 |             is_up = stat.isup if stat else True
1762 |             status_num = w_info.get('status_num')
1763 |             if status_num == 2 or (status_num is None and
| is_up):
1764 |                 status_str = 'Активно (Up)'
1765 |             elif status_num == 7:
1766 |                 status_str = 'Кабель не подключен (Disconn
| ected)'
1767 |             elif status_num == 0:
1768 |                 status_str = 'Отключено (Disabled)'
1769 |             elif not is_up:
1770 |                 status_str = 'Не активно (Down)'
1771 |             else:
1772 |                 status_str = 'Активно (Up)'
1773 |             speed_mbps = 0
1774 |             if stat and stat.speed and (stat.speed > 0) an
| d (stat.speed < 1000000):
1775 |                 speed_mbps = stat.speed
1776 |             elif w_info.get('speed'):
1777 |                 try:
1778 |                     raw_spd = int(w_info['speed'])
1779 |                     if raw_spd < 1000000000000:

```

```

1780         speed_mbps = round(raw_spd / 10000
1781         00)
1782     except Exception:
1783         pass
1784     if speed_mbps > 0:
1785         if speed_mbps >= 1000:
1786             speed_str = f'{speed_mbps} Mbps ({round
1787             d(speed_mbps / 1000, 1)} Gbps)'
1788         else:
1789             speed_str = f'{speed_mbps} Mbps'
1790     else:
1791         speed_str = 'Автоопределение (0 Mbps)' if
1792         not is_up else '1000 Mbps (АВТО)'
1793     model_name = w_info.get('name') or w_info.get(
1794     'desc') or if_name
1795     mfg_val = w_info.get('mfg')
1796     if not mfg_val:
1797         if 'intel' in if_name.lower() or 'intel' i
1798         n model_name.lower():
1799             mfg_val = 'Intel Corporation'
1800         elif 'realtek' in if_name.lower() or 'real
1801         tek' in model_name.lower():
1802             mfg_val = 'Realtek'
1803         elif 'vethernet' in if_name.lower() or 'hy
1804         per-v' in model_name.lower() or 'microsoft' in model_name.lowe
1805         r() or ('local area connection*' in if_name.lower()):
1806             mfg_val = 'Microsoft Corporation'
1807         else:
1808             mfg_val = 'Универсальный сетевой контр
1809             оллер'
1810     name_l = if_name.lower()
1811     desc_l = (w_info.get('desc') or '').lower()
1812     if 'local area connection*' in name_l:
1813         adapter_type = 'Виртуальный адаптер Wi-Fi
1814         Direct'
1815     if model_name == if_name:
1816         model_name = f'Microsoft Wi-Fi Direct
1817         Virtual Adapter ({if_name})'

```

```

1807         elif 'wi-fi' in name_l or 'wireless' in desc_l
|         or '802.11' in desc_l:
1808             adapter_type = 'Беспроводной адаптер (Wi-F
| i 802.11)'
1809             elif 'bluetooth' in name_l or 'bluetooth' in d
| esc_l:
1810                 adapter_type = 'Bluetooth PAN адаптер'
1811             elif 'vethernet' in name_l or 'virtual' in des
| c_l or 'hyper-v' in desc_l:
1812                 adapter_type = 'Виртуальный сетевой адапте
| р (Hyper-V / WSL)'
1813             elif 'loopback' in name_l:
1814                 adapter_type = 'Программный интерфейс Loop
| back'
1815             elif 'tap' in name_l or 'tun' in name_l or 'vp
| n' in name_l:
1816                 adapter_type = 'VPN / Туннельный адаптер'
1817             else:
1818                 adapter_type = 'Сетевая карта Ethernet (LA
| N)'
1819             net_props: Dict[str, Any] = {'Имя подключения'
| : if_name, 'Модель адаптера': model_name, 'Производитель': mfg
| _val, 'Тип адаптера': adapter_type, 'Физический (MAC) адрес':
| mac_addr or 'N/A', 'Состояние соединения': status_str, 'Скорос
| ть канала': speed_str}
1820             if stat and getattr(stat, 'duplex', None) is n
| ot None:
1821                 d_val = getattr(stat.duplex, 'value', stat
| .duplex)
1822                 if d_val == 2:
1823                     net_props['Режим дуплекса'] = 'Full Du
| plex (Полный)'
1824                 elif d_val == 1:
1825                     net_props['Режим дуплекса'] = 'Half Du
| plex (Полудуплекс)'
1826                 if stat and getattr(stat, 'mtu', None):
1827                     net_props['Размер MTU'] = f'{stat.mtu} бай
| т'

```

```

1828         net_props['IP-адреса (IPv4)'] = ', '.join(ipv4
|_list) or 'N/A'
1829         if ipv6_list:
1830             net_props['IP-адреса (IPv6)'] = ', '.join(
|ipv6_list[:3])
1831         if gateways:
1832             net_props['Основной шлюз'] = ', '.join(gat
|eways)
1833         if dhcp_server or dhcp_enabled is not None:
1834             net_props['DHCP сервер'] = dhcp_server or
|('Включен (DHCP)' if dhcp_enabled else 'Статический IP / Отклю
|чен')
1835         if dns_servers:
1836             net_props['DNS серверы'] = ', '.join(dns_s
|ervers)
1837         if w_info.get('pnp'):
1838             net_props['Идентификатор PnP'] = w_info['p
|np']
1839         display_title = f'{if_name}: {model_name}' if
|model_name != if_name else if_name
1840         nodes.append(HardwareNode(category='Network Ad
|apter', name=display_title, properties=net_props))
1841         if os.name == 'nt':
1842             try:
1843                 import win32com.client
1844                 wmi_obj = win32com.client.GetObject('winmgmts:
|')
1845                 for s in wmi_obj.InstancesOf('Win32_SoundDevic
|e'):
1846                     snd_name = (getattr(s, 'Caption', '') or g
|etattr(s, 'Name', '') or '').strip()
1847                     snd_mfg = (getattr(s, 'Manufacturer', '')
|or '').strip() or 'Аудиоустройство'
1848                     snd_status = (getattr(s, 'Status', '') or
|'OK').strip()
1849                     snd_pnp = (getattr(s, 'PNPDeviceID', '') o
|r '').strip()
1850                     if snd_name:

```

```
1851 |             nodes.append(HardwareNode(category='Audio Device', name=snd_name, properties={'Название устройства':  
| snd_name, 'Производитель': snd_mfg, 'Состояние': f'{snd_status} (Работает)', 'Идентификатор PnP': snd_pnp or 'N/A'}))  
1852 |         except Exception as e:  
1853 |             logger.debug(f'[HardwareTree] WMI SoundDevice  
| query warning: {e}')  
1854 |         if os.name == 'nt':  
1855 |             try:  
1856 |                 import win32com.client  
1857 |                 wmi_obj = win32com.client.GetObject('winmgmts:  
| ')  
1858 |                 for u in wmi_obj.InstancesOf('Win32_USBController'):  
1859 |                     usb_name = (getattr(u, 'Caption', '') or getattr(u, 'Name', '') or '').strip()  
1860 |                     usb_mfg = (getattr(u, 'Manufacturer', '') or '').strip() or 'USB Контроллер'  
1861 |                     usb_status = (getattr(u, 'Status', '') or 'OK').strip()  
1862 |                     usb_pnp = (getattr(u, 'PNPDeviceID', '') or '').strip()  
1863 |                     if usb_name:  
1864 |                         nodes.append(HardwareNode(category='USB Controller', name=usb_name, properties={'Название контроллера':  
| usb_name, 'Производитель': usb_mfg, 'Состояние': f'{usb_status} (Работает)', 'Идентификатор PnP': usb_pnp or 'N/A'}))  
1865 |                     except Exception as e:  
1866 |                         logger.debug(f'[HardwareTree] WMI USBController query warning: {e}')  
1867 |                     upd = self.get_updates_info()  
1868 |                     nodes.append(HardwareNode(category='Windows Updates', name=f'{upd.status} ({upd.installed_kb_count} KBs)', properties={'Статус обновлений': upd.status, 'Число установленных исправлений': upd.installed_kb_count, 'Последние KB пакеты': ', '.join(upd.recent_hotfixes) or 'N/A', 'Дата последней установки': upd.latest_installed_on or 'OK'}))  
1869 |                 self._hardware_tree_cached = nodes
```

```

1870         return nodes
1871
1872     def get_hardware_tree(self, force: bool = False) -> List[HardwareNode]:
1873         """Generate AIDA64-like hierarchical component specification tree.
1874
1875         Args:
1876             force: Принудительный повторный опрос оборудования
1877             .
1878         Returns:
1879             List[HardwareNode]: Hardware devices grouped by category.
1880         """
1881         import asyncio
1882         return asyncio.run(self.get_hardware_tree_async(force=force))
1883
1884     def get_hardware_audit(self) -> HardwareAuditReport:
1885         """Collect deep hardware devices audit with drivers, install dates, and attached sensors.
1886
1887         Returns:
1888             HardwareAuditReport: Complete hardware audit report.
1889         """
1890         return self.auditor.audit_hardware()
1891
1892     def archive_hardware_state(self, auto_diff: bool=True) -> HardwareArchiveEntry:
1893         """Capture current hardware audit and save it to historical archive storage.
1894
1895         Args:
1896             auto_diff: Automatically compute diff with previous archive.
1897

```

```

1898         Returns:
1899             HardwareArchiveEntry: Persisted archive entry with
1900             detected changes.
1901             """
1902             report = self.get_hardware_audit()
1903             return self.history_manager.archive_report(report, auto_diff=auto_diff)
1904
1905     def get_hardware_history(self, limit: int=50) -> List[Dict[str, Any]]:
1906         """Retrieve historical hardware archive snapshots meta data.
1907
1908         Args:
1909             limit: Maximum count of archive entries.
1910
1911         Returns:
1912             List[Dict[str, Any]]: Chronological list of archive entries.
1913             """
1914             return self.history_manager.get_history(limit=limit)
1915
1916     def get_hardware_changes(self, limit: int=100) -> List[HardwareChangeItem]:
1917         """Retrieve historical timeline of all hardware configuration changes.
1918
1919         Args:
1920             limit: Maximum count of changes to return.
1921
1922         Returns:
1923             List[HardwareChangeItem]: Historical changes timeline.
1924             """
1925             return self.history_manager.get_change_timeline(limit=limit)
1926
1927     def get_startup_audit(self) -> Any:

```



```

1927 |         """Collect deep Windows startup and autoruns audit rep
      | ort.
1928 |
1929 |         Returns:
1930 |             AuditReport: Complete startup audit report with ri
      | sk ratings.
1931 |         """
1932 |         from apps.windows.modules.startup.core.auditor import
      | StartupAuditor
1933 |         return StartupAuditor().run_audit()
1934 |
1935 |     def archive_startup_state(self, auto_diff: bool = True) ->
      | StartupArchiveEntry:
1936 |         """Capture current Windows startup state, compute chan
      | ges diff, and save snapshot to storage.
1937 |
1938 |         Args:
1939 |             auto_diff: Automatically compute diff with previou
      | s archive snapshot.
1940 |
1941 |         Returns:
1942 |             StartupArchiveEntry: Persisted archive entry with
      | detected changes.
1943 |         """
1944 |         report = self.get_startup_audit()
1945 |         entry = self.startup_history_manager.archive_report(re
      | port, auto_diff=auto_diff)
1946 |         if self.storage:
1947 |             try:
1948 |                 self.storage.save_startup_archive(entry)
1949 |             except Exception as ex:
1950 |                 logger.debug(f'Не удалось сохранить снимок авт
      | озапуска в БД: {ex}')
1951 |         return entry
1952 |
1953 |     def get_startup_history(self, limit: int = 50) -> List[Dic
      | t[str, Any]]:
1954 |         """Retrieve historical startup archive snapshots metad

```

```

    ata.
1955
1956     Args:
1957         limit: Maximum count of archive entries.
1958
1959     Returns:
1960         List[Dict[str, Any]]: Chronological list of startup
1961         p archive entries.
1962         """
1963     if self.storage:
1964         try:
1965             db_history = self.storage.get_startup_archives
1966             (limit=limit)
1967             if db_history:
1968                 return db_history
1969             except Exception:
1970                 pass
1971     return self.startup_history_manager.get_history(limit=
1972     limit)
1973
1974     def get_startup_changes(self, limit: int = 100) -> List[Di
1975     ct[str, Any]]:
1976         """Retrieve historical timeline of all startup configura
1977         tion changes.
1978
1979     Args:
1980         limit: Maximum count of changes to return.
1981
1982     Returns:
1983         List[Dict[str, Any]]: Historical changes timeline.
1984         """
1985     if self.storage:
1986         try:
1987             db_changes = self.storage.get_startup_changes(
1988             limit=limit)
1989             if db_changes:
1990                 return db_changes
1991             except Exception:

```

```

1986         pass
1987         return [c.model_dump(mode='json') for c in self.startu
p_history_manager.get_change_timeline(limit=limit)]
1988
1989     def get_latest_startup_audit(self) -> Optional[Dict[str, A
ny]]:
1990         """Retrieve latest recorded startup audit snapshot fro
m telemetry storage.
1991
1992         Returns:
1993             Optional[Dict[str, Any]]: Startup audit dict or No
ne.
1994         """
1995         if self.storage:
1996             try:
1997                 latest = self.storage.get_latest_startup_archi
ve()
1998                 if latest:
1999                     return latest
2000             except Exception:
2001                 pass
2002             latest_archive = self.startup_history_manager.get_late
st_archive()
2003             return latest_archive.model_dump(mode='json') if lates
t_archive else None
2004
2005     def save_snapshot_to_db(self, snapshot: Optional[SystemSna
pshot]=None, top_n: int=20) -> int:
2006         """Collect and save system telemetry snapshot directly
into SQLite database.
2007
2008         Args:
2009             snapshot: Optional existing SystemSnapshot (if Non
e, collected synchronously).
2010             top_n: Top processes count.
2011
2012         Returns:
2013             int: Created snapshot ID in database.

```

```

2014         """
2015         snap = snapshot
2016         if snap is None:
2017             import asyncio
2018             try:
2019                 loop = asyncio.get_event_loop()
2020             except RuntimeError:
2021                 loop = None
2022             if loop and loop.is_running():
2023                 snap = asyncio.run_coroutine_threadsafe(self.g
2024 et_snapshot(process_limit=top_n), loop).result(timeout=5.0)
2025             else:
2026                 snap = asyncio.run(self.get_snapshot(process_l
2027 imit=top_n))
2028         return self.storage.save_snapshot(snap, top_n=top_n)
2029
2030     def get_snapshots(self, limit: int=60, since_epoch: Option
2031 al[float]=None) -> List[Dict[str, Any]]:
2032         """Retrieve persisted system snapshots from SQLite dat
2033 abase.
2034
2035         Args:
2036             limit: Maximum number of snapshots.
2037             since_epoch: Optional starting timestamp.
2038
2039         Returns:
2040             List[Dict[str, Any]]: List of system snapshots fro
2041 m database.
2042         """
2043         return self.storage.get_snapshots(limit=limit, since_e
2044 poch=since_epoch)
2045
2046     def get_latest_cpu_from_db(self) -> Optional[Dict[str, Any
2047 ]]:
2048         """Извлекает последние метрики процессора из базы данн
2049 ых SQLite.
2050
2051         Returns:

```

```

2044         Optional[Dict[str, Any]]: Метрики процессора из SQ
    Lite.
2045         """
2046         return self.storage.get_latest_cpu()
2047
2048     def get_cpu_history_from_db(self, limit: int = 60, since_e
    poch: Optional[float] = None) -> List[Dict[str, Any]]:
2049         """Извлекает историю нагрузки процессора из базы данны
    x SQLite.
2050
2051         Args:
2052             limit: Количество записей.
2053             since_epoch: Начальное время UNIX epoch.
2054
2055         Returns:
2056             List[Dict[str, Any]]: Исторические срезы нагрузки
    CPU.
2057         """
2058         return self.storage.get_cpu_history(limit=limit, since
    _epoch=since_epoch)
2059
2060     def get_process_history(self, name: Optional[str]=None, pi
    d: Optional[int]=None, limit: int=100) -> List[Dict[str, Any]]
    :
2061         """Retrieve historical process metrics from SQLite dat
    abase.
2062
2063         Args:
2064             name: Substring of process name.
2065             pid: Process identifier.
2066             limit: Record limit.
2067
2068         Returns:
2069             List[Dict[str, Any]]: History of process metrics.
2070         """
2071         return self.storage.get_process_history(name=name, pid
    =pid, limit=limit)
2072

```

```

2073     def get_reboot_report(self, limit: int = 20, hours: int =
2074 | 720) -> Any:
2075 |         """Сбор сводного отчета анализа перезагрузок и выключе
2076 | ний ОС.
2077 |
2078 |         Args:
2079 |             limit: Максимальное количество сессий.
2080 |             hours: Глубина выборки журналов в часах.
2081 |
2082 |         Returns:
2083 |             RebootAnalysisReport: Сводный аналитический отчет.
2084 |         """
2085 |         from apps.windows.telemetry_research.reboot_analyzer i
2086 | mport WindowsRebootAnalyzer
2087 |         analyzer = WindowsRebootAnalyzer(storage=self.storage)
2088 |         return analyzer.collect_reboot_history(limit=limit, ho
2089 | urs=hours)
2090 |
2091 |     def get_latest_reboot(self) -> Optional[Any]:
2092 |         """Получение данных о последней перезагрузке системы.
2093 |
2094 |         Returns:
2095 |             Optional[RebootSession]: Информация о последнем пе
2096 | резавпуске.
2097 |         """
2098 |         from apps.windows.telemetry_research.reboot_analyzer i
2099 | mport WindowsRebootAnalyzer
2100 |         analyzer = WindowsRebootAnalyzer(storage=self.storage)
2101 |         return analyzer.analyze_current_boot()
2102 |
2103 |     def get_extended_system_audit(self) -> Any:
2104 |         """Сбор расширенного аудита безопасности (Defender), а
2105 | втозагрузки (Startup), VSS и пользователей.
2106 |
2107 |         Returns:
2108 |             ExtendedSystemAuditReport: Сводный отчет расширенн
2109 | ого аудита.
2110 |         """

```

```

2103         from apps.windows.telemetry_research.deep_diagnostics
import DeepDiagnosticsEngine
2104         engine = DeepDiagnosticsEngine()
2105         report = engine.collect_extended_system_audit()
2106         try:
2107             self.storage.save_extended_audit(report)
2108         except Exception as ex:
2109             logger.debug(f'Не удалось сохранить расширенный ау
дит в БД: {ex}')
2110         return report
2111
2112     def get_latest_extended_audit(self) -> Optional[Dict[str,
Any]]:
2113         """Получение последнего сохраненного среза расширенног
о системного аудита.
2114
2115         Returns:
2116             Optional[Dict[str, Any]]: Словарь аудита или None.
2117         """
2118         return self.storage.get_latest_extended_audit()
2119
2120     def get_process_provenance_report(
2121         self,
2122         pid: Optional[int] = None,
2123         name_filter: Optional[str] = None,
2124         user_filter: Optional[str] = None,
2125         limit: int = 100,
2126     ) -> ProcessProvenanceReport:
2127         """Сформировать сводный отчет о происхождении и цепочк
ах запуска активных процессов.
2128
2129         Args:
2130             pid: Опциональный фильтр по PID.
2131             name_filter: Опциональный фильтр по имени процесса
.
2132             user_filter: Опциональный фильтр по пользователю/S
ID.
2133             limit: Максимальное количество процессов в отчете.

```

```

2134 |
2135 |         Returns:
2136 |             ProcessProvenanceReport: Полный отчет с детальным
анализом происхождения процессов.
2137 |         """
2138 |         all_procs = self.get_top_processes(limit=0, sort_by='c
pu')
2139 |         filtered_items: List[ProcessProvenanceInfo] = []
2140 |
2141 |         for p in all_procs:
2142 |             if pid is not None and p.pid != pid:
2143 |                 continue
2144 |             if name_filter and name_filter.lower() not in (p.n
ame or '').lower():
2145 |                 continue
2146 |             if user_filter and user_filter.lower() not in (p.u
sername or '').lower() and user_filter.lower() not in (p.sid o
r '').lower():
2147 |                 continue
2148 |
2149 |             chain_list = [node.strip() for node in (p.ancestor
_chain or "").split(" → ") if node.strip()]
2150 |             user_display = p.username or "Unknown"
2151 |             sid_display = p.sid or "N/A"
2152 |             session_display = f"Session {p.session_id}" if p.s
ession_id is not None else "Session N/A"
2153 |             who_runs = f"{user_display} ({sid_display}, {sessi
on_display})"
2154 |             who_created = f"{p.parent_name or 'Unknown'} (PPID
: {p.ppid})"
2155 |             why_created = p.launch_reason or "unknown"
2156 |
2157 |             item = ProcessProvenanceInfo(
2158 |                 pid=p.pid,
2159 |                 ppid=p.ppid,
2160 |                 name=p.name,
2161 |                 executable_path=p.executable_path,
2162 |                 command_line=p.cmdline,

```



```

2163         user=p.username,
2164         sid=p.sid,
2165         session_id=p.session_id,
2166         integrity_level=p.integrity_level,
2167         elevation=bool(p.elevation) if p.elevation is
not None else None,
2168         creation_time=p.creation_time,
2169         process_guid=p.process_guid or f"proc_{p.pid}"
,
2170         parent_name=p.parent_name,
2171         ancestor_chain=chain_list,
2172         ancestor_chain_str=p.ancestor_chain or "",
2173         who_runs_it=who_runs,
2174         who_created_it=who_created,
2175         why_was_it_created=why_created,
2176     )
2177     filtered_items.append(item)
2178     if limit and len(filtered_items) >= limit:
2179         break
2180
2181     now_iso = datetime.now(timezone.utc).isoformat()
2182     recent_evs: List[ProcessLifecycleEvent] = []
2183     try:
2184         for ev in self.storage.get_process_provenance_history(limit=20):
2185             if isinstance(ev, dict):
2186                 recent_evs.append(ProcessLifecycleEvent(
2187                     event_id=str(ev.get("event_id") or ""),
2188                     timestamp=str(ev.get("timestamp") or now_iso),
2189                     event_type=str(ev.get("event_type") or "ProcessCreated"),
2190                     pid=int(ev.get("pid") or 0),
2191                     ppid=int(ev.get("parent_pid")) if ev.get("parent_pid") is not None else None,
2192                     process_guid=str(ev.get("process_guid") or ""),

```

```

2193         parent_guid=ev.get("parent_guid"),
2194         name=str(ev.get("name") or "unknown"),
2195         executable_path=ev.get("executable_path"),
2196         command_line=ev.get("command_line"),
2197         user=ev.get("user"),
2198         sid=ev.get("sid"),
2199         session_id=int(ev.get("session_id")) if ev
2200         .get("session_id") is not None else None,
2201         integrity_level=ev.get("integrity_level"),
2202         elevation=bool(ev.get("elevation")) if
2203         ev.get("elevation") is not None else None,
2204         parent_name=ev.get("parent_name"),
2205         ancestor_chain=ev.get("ancestor_chain"),
2206         launch_reason=ev.get("launch_reason"),
2207         source=str(ev.get("source") or "system"),
2208         raw_json=str(ev.get("details_json")) if
2209         ev.get("details_json") else None,
2210     ))
2211     except Exception as ex:
2212         logger.debug(f"Не удалось извлечь недавние события
2213         provenance для отчета: {ex}")
2214
2215     summary = f"Всего активных процессов: {len(all_procs)}
2216     . Проанализировано происхождение для {len(filtered_items)} про
2217     цессов."
2218
2219     return ProcessProvenanceReport(
2220         timestamp=now_iso,
2221         total_processes=len(all_procs),
2222         active_provenance=filtered_items,
2223         recent_events=recent_evs,
2224         summary=summary,
2225     )
2226
2227 def get_process_provenance_history(

```

```

2221         self,
2222         guid: Optional[str] = None,
2223         pid: Optional[int] = None,
2224         name: Optional[str] = None,
2225         user: Optional[str] = None,
2226         event_type: Optional[str] = None,
2227         limit: int = 100,
2228     ) -> List[Dict[str, Any]]:
2229         """Получить исторические события жизненного цикла и пр
оисхождения процессов из БД.
2230
2231         Args:
2232             guid: Фильтр по уникальному идентификатору процесс
а.
2233             pid: Фильтр по PID.
2234             name: Фильтр по имени процесса.
2235             user: Фильтр по имени пользователя или SID.
2236             event_type: Фильтр по типу события ('ProcessCreate
d', 'ProcessTerminated').
2237             limit: Максимальное количество записей.
2238
2239         Returns:
2240             List[Dict[str, Any]]: Список исторических событий
происхождения процессов.
2241         """
2242         return self.storage.get_process_provenance_history(
2243             guid=guid,
2244             pid=pid,
2245             name=name,
2246             user=user,
2247             event_type=event_type,
2248             limit=limit,
2249         )
2250
2251     def get_process_lineage(self, guid_or_pid: str) -> List[Di
ct[str, Any]]:
2252         """Построить полную восходящую цепочку предков процесс
а (дерево запуска).

```

```
2253 |
2254 |         Args:
2255 |             guid_or_pid: Идентификатор process_guid или числов
    ой PID.
2256 |
2257 |         Returns:
2258 |             List[Dict[str, Any]]: Упорядоченный список предков
    от корневого процесса к целевому.
2259 |             """
2260 |         return self.storage.get_process_lineage(guid_or_pid=gu
    id_or_pid)
2261 |
2262 |
2263 |
```

compactor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Compact
  or
4 # =====
5 # Description:
6 #   Модуль логарифмического и многоуровневого сжатия (rollup)
  временных рядов.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.compactor import TelemetryCo
  mpactor, compute_percentile
11 #
12 # File: compactor.py
13 # Project: ai-breadboard
14 # Package: apps.windows.telemetry
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-03 23:25:00
18 # =====
19
20 """Модуль `TelemetryCompactor` и вспомогательная функция `comp
  ute_percentile`.
21
22 Перенесены из `telemetry_research` без обертки.
23 """
24
25 from __future__ import annotations
26
27 import math
28 from datetime import datetime
29 from typing import Any, Dict, List, Optional, Sequence, Tuple
```

```

30
31 from apps.windows.telemetry.models import ProcessMetricRollup,
    SystemMetricRollup
32
33
34 def compute_percentile(values: Sequence[float], p: float = 95.
    0) -> float:
35     """Вычисляет перцентиль выборки методом линейной интерполя
    ции.
36
37     Args:
38         values: Последовательность числовых значений.
39         p: Желаемый перцентиль (0..100), по умолчанию 95.0.
40
41     Returns:
42         float: Значение перцентиля (0.0 при пустой выборке).
43     """
44     if not values:
45         return 0.0
46     sorted_vals = sorted(values)
47     n = len(sorted_vals)
48     if n == 1:
49         return float(sorted_vals[0])
50     p_clamped = max(0.0, min(100.0, p))
51     rank = (p_clamped / 100.0) * (n - 1)
52     lower = int(math.floor(rank))
53     upper = int(math.ceil(rank))
54     weight = rank - lower
55     return float(sorted_vals[lower] + weight * (sorted_vals[up
    per] - sorted_vals[lower]))
56
57
58 class TelemetryCompactor:
59     """Компаратор и агрегатор временных рядов телеметрии."""
60
61     @staticmethod
62     def get_tier_for_age(age_seconds: float) -> str:
63         """Определяет гранулярность (tier) в зависимости от во

```

зраста данных.

```
64 |
65 |     Args:
66 |         age_seconds: Возраст данных в секундах.
67 |
68 |     Returns:
69 |         str: Уровень агрегации ('1s', '2s', '5s', '10s', '
70 | 30s', '1m', '5m', '15m', '1h', '6h').
71 |         """
72 |     if age_seconds <= 60.0:
73 |         return "1s"
74 |     elif age_seconds <= 300.0:
75 |         return "2s"
76 |     elif age_seconds <= 900.0:
77 |         return "5s"
78 |     elif age_seconds <= 1800.0:
79 |         return "10s"
80 |     elif age_seconds <= 3600.0:
81 |         return "30s"
82 |     elif age_seconds <= 10800.0:
83 |         return "1m"
84 |     elif age_seconds <= 43200.0:
85 |         return "5m"
86 |     elif age_seconds <= 86400.0:
87 |         return "15m"
88 |     elif age_seconds <= 604800.0:
89 |         return "1h"
90 |     else:
91 |         return "6h"
92 |
93 | @staticmethod
94 | def compact_system_metrics(
95 |     samples: List[Dict[str, Any]],
96 |     period_start: str,
97 |     period_end: str,
98 |     tier: str = "1m",
99 | ) -> SystemMetricRollup:
    """Агрегирует список сырых замеров хоста в один System
```

```

MetricRollup.
100
101     Args:
102         samples: Список словарей замеров (SystemSnapshot и
ли сжатых метрик).
103         period_start: ISO timestamp начала периода.
104         period_end: ISO timestamp окончания периода.
105         tier: Уровень гранулярности бакета.
106
107     Returns:
108         SystemMetricRollup: Сжатый бакет с расчётом avg, m
in, max, p95 и total.
109     """
110     count = len(samples)
111     if count == 0:
112         return SystemMetricRollup(
113             period_start=period_start,
114             period_end=period_end,
115             duration_seconds=0.0,
116             sample_count=0,
117             tier=tier,
118         )
119     # Длительность периода
120     duration_sec = 60.0
121     try:
122         t0 = datetime.fromisoformat(period_start)
123         t1 = datetime.fromisoformat(period_end)
124         duration_sec = max(1.0, (t1 - t0).total_seconds())
125     except Exception:
126         duration_sec = float(count)
127     cpu_vals: List[float] = []
128     ram_used_vals: List[float] = []
129     ram_pct_vals: List[float] = []
130     disk_read_rates: List[float] = []
131     disk_write_rates: List[float] = []
132     net_rx_rates: List[float] = []
133     net_tx_rates: List[float] = []
134     for s in samples:

```



```
135 |         cpu_val = float(s.get("cpu_total_percent") or s.ge
| t("cpu_percent") or 0.0)
136 |         cpu_vals.append(cpu_val)
137 |         ram_used = float(s.get("memory_used_gb") or 0.0)
138 |         ram_pct = float(s.get("memory_percent") or 0.0)
139 |         ram_used_vals.append(ram_used)
140 |         ram_pct_vals.append(ram_pct)
141 |         disk_r = float(s.get("disk_read_bytes_sec") or 0.0
| ) / (1024.0 * 1024.0)
142 |         disk_w = float(s.get("disk_write_bytes_sec") or 0.
| 0) / (1024.0 * 1024.0)
143 |         disk_read_rates.append(disk_r)
144 |         disk_write_rates.append(disk_w)
145 |         net_rx = float(s.get("network_recv_bytes_sec") or
| 0.0) / (1024.0 * 1024.0)
146 |         net_tx = float(s.get("network_sent_bytes_sec") or
| 0.0) / (1024.0 * 1024.0)
147 |         net_rx_rates.append(net_rx)
148 |         net_tx_rates.append(net_tx)
149 |         sample_dt = duration_sec / max(1, count)
150 |         cpu_avg = sum(cpu_vals) / count
151 |         cpu_min = min(cpu_vals)
152 |         cpu_max = max(cpu_vals)
153 |         cpu_p95 = compute_percentile(cpu_vals, 95.0)
154 |         ram_avg = sum(ram_used_vals) / count
155 |         ram_min = min(ram_used_vals)
156 |         ram_max = max(ram_used_vals)
157 |         ram_pct_avg = sum(ram_pct_vals) / count
158 |         ram_pct_max = max(ram_pct_vals)
159 |         disk_r_avg = sum(disk_read_rates) / count
160 |         disk_r_max = max(disk_read_rates)
161 |         disk_r_total = sum(disk_read_rates) * sample_dt
162 |         disk_w_avg = sum(disk_write_rates) / count
163 |         disk_w_max = max(disk_write_rates)
164 |         disk_w_total = sum(disk_write_rates) * sample_dt
165 |         net_rx_avg = sum(net_rx_rates) / count
166 |         net_rx_max = max(net_rx_rates)
167 |         net_rx_total = sum(net_rx_rates) * sample_dt
```

```

168 |         net_tx_avg = sum(net_tx_rates) / count
169 |         net_tx_max = max(net_tx_rates)
170 |         net_tx_total = sum(net_tx_rates) * sample_dt
171 |         return SystemMetricRollup(
172 |             period_start=period_start,
173 |             period_end=period_end,
174 |             duration_seconds=round(duration_sec, 2),
175 |             sample_count=count,
176 |             tier=tier,
177 |             cpu_avg=round(cpu_avg, 2),
178 |             cpu_min=round(cpu_min, 2),
179 |             cpu_max=round(cpu_max, 2),
180 |             cpu_p95=round(cpu_p95, 2),
181 |             ram_avg_gb=round(ram_avg, 3),
182 |             ram_min_gb=round(ram_min, 3),
183 |             ram_max_gb=round(ram_max, 3),
184 |             ram_percent_avg=round(ram_pct_avg, 2),
185 |             ram_percent_max=round(ram_pct_max, 2),
186 |             disk_read_avg_mbs=round(disk_r_avg, 3),
187 |             disk_read_max_mbs=round(disk_r_max, 3),
188 |             disk_read_total_mb=round(disk_r_total, 3),
189 |             disk_write_avg_mbs=round(disk_w_avg, 3),
190 |             disk_write_max_mbs=round(disk_w_max, 3),
191 |             disk_write_total_mb=round(disk_w_total, 3),
192 |             network_rx_avg_mbs=round(net_rx_avg, 3),
193 |             network_rx_max_mbs=round(net_rx_max, 3),
194 |             network_rx_total_mb=round(net_rx_total, 3),
195 |             network_tx_avg_mbs=round(net_tx_avg, 3),
196 |             network_tx_max_mbs=round(net_tx_max, 3),
197 |             network_tx_total_mb=round(net_tx_total, 3),
198 |         )
199 |
200 |     @staticmethod
201 |     def compact_process_metrics(
202 |         process_samples: List[Dict[str, Any]],
203 |         period_start: str,
204 |         period_end: str,
205 |     ) -> List[ProcessMetricRollup]:

```

```

206 |         """Группирует и агрегирует замеры процессов по имени/P
    | ID.
207 |
208 |     Args:
209 |         process_samples: Список замеров процессов.
210 |         period_start: Начало периода.
211 |         period_end: Конец периода.
212 |
213 |     Returns:
214 |         List[ProcessMetricRollup]: Агрегированные показате
    | ли каждого процесса.
215 |         """
216 |         duration_sec = 60.0
217 |         try:
218 |             t0 = datetime.fromisoformat(period_start)
219 |             t1 = datetime.fromisoformat(period_end)
220 |             duration_sec = max(1.0, (t1 - t0).total_seconds())
221 |         except Exception:
222 |             duration_sec = 60.0
223 |         # Группировка по (name, pid)
224 |         grouped: Dict[Tuple[str, Optional[int]], List[Dict[str
    | , Any]]] = {}
225 |         for p in process_samples:
226 |             key = (str(p.get("name", "unknown")), p.get("pid")
    | )
227 |             grouped.setdefault(key, []).append(p)
228 |         results: List[ProcessMetricRollup] = []
229 |         for (name, pid), p_list in grouped.items():
230 |             count = len(p_list)
231 |             cpu_vals = [float(item.get("cpu_percent") or 0.0)
    | for item in p_list]
232 |             ram_vals = [float(item.get("memory_mb") or 0.0) fo
    | r item in p_list]
233 |             read_bytes_sec = [float(item.get("read_bytes_sec")
    | or 0.0) for item in p_list]
234 |             write_bytes_sec = [float(item.get("write_bytes_sec
    | ") or 0.0) for item in p_list]
235 |             rx_bytes_sec = [float(item.get("rx_bytes_sec") or

```

```

0.0) for item in p_list]
236         tx_bytes_sec = [float(item.get("tx_bytes_sec") or
0.0) for item in p_list]
237         sample_dt = duration_sec / max(1, count)
238         path = p_list[0].get("path") or p_list[0].get("exe
")
239         cpu_avg = sum(cpu_vals) / count
240         cpu_max = max(cpu_vals)
241         cpu_p95 = compute_percentile(cpu_vals, 95.0)
242         ram_avg = sum(ram_vals) / count
243         ram_max = max(ram_vals)
244         disk_r_tot = (sum(read_bytes_sec) * sample_dt) / (
1024.0 * 1024.0)
245         disk_w_tot = (sum(write_bytes_sec) * sample_dt) /
(1024.0 * 1024.0)
246         net_rx_tot = (sum(rx_bytes_sec) * sample_dt) / (10
24.0 * 1024.0)
247         net_tx_tot = (sum(tx_bytes_sec) * sample_dt) / (10
24.0 * 1024.0)
248         results.append(
249             ProcessMetricRollup(
250                 name=name,
251                 pid=pid,
252                 period_start=period_start,
253                 period_end=period_end,
254                 duration_seconds=round(duration_sec, 2),
255                 sample_count=count,
256                 path=path,
257                 cpu_avg=round(cpu_avg, 2),
258                 cpu_max=round(cpu_max, 2),
259                 cpu_p95=round(cpu_p95, 2),
260                 ram_avg_mb=round(ram_avg, 2),
261                 ram_max_mb=round(ram_max, 2),
262                 disk_read_total_mb=round(disk_r_tot, 3),
263                 disk_write_total_mb=round(disk_w_tot, 3),
264                 network_rx_total_mb=round(net_rx_tot, 3),
265                 network_tx_total_mb=round(net_tx_tot, 3),
266                 )

```

```
267 |         )
268 |     return results
269 |
```

ctl.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - CLI Con
  troller
4 # =====
5 # Description:
6 #   Консольный контроллер AITelemetryCtl для мониторинга, упра
  вления службой
7 #   Windows Service, проверки базы данных и выполнения диагнос
  тики.
8 #
9 # Usage Examples:
10 #   CLI:
11 #       AITelemetryCtl.exe status
12 #       AITelemetryCtl.exe start
13 #       AITelemetryCtl.exe stop
14 #       AITelemetryCtl.exe restart
15 #       AITelemetryCtl.exe diagnose
16 #       AITelemetryCtl.exe database
17 #
18 # File: ctl.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-06 07:58:00
24 # =====
25
26 from __future__ import annotations
27 """Консольная утилита управления и диагностики службы AITeleme
  try."""
28
29 import argparse
```

```
30 import json
31 import os
32 import subprocess
33 import sys
34 from pathlib import Path
35 from typing import Any, Dict, Optional
36
37 _current_dir = Path(__file__).resolve().parent
38 _project_root = str(_current_dir.parents[2]) if len(_current_dir.parents) >= 3 else str(_current_dir)
39 if _project_root not in sys.path:
40     sys.path.insert(0, _project_root)
41
42
43 def get_service_status_scm(service_name: str = 'AITelemetry')
44     -> Dict[str, Any]:
45     """Запрашивает состояние службы через Windows SCM (sc.exe)
46     .
47
48     Args:
49         service_name: Имя службы в диспетчере Windows.
50
51     Returns:
52         Dict[str, Any]: Словарь со статусом и деталями службы.
53     """
54     try:
55         res = subprocess.run(
56             ['sc.exe', 'query', service_name],
57             capture_output=True,
58             text=True,
59             timeout=5
60         )
61         output = res.stdout
62         is_installed = res.returncode == 0
63         state = 'UNKNOWN'
64         if 'RUNNING' in output:
65             state = 'RUNNING'
66         elif 'STOPPED' in output:
```

```

65         state = 'STOPPED'
66     elif 'START_PENDING' in output:
67         state = 'START_PENDING'
68     elif 'STOP_PENDING' in output:
69         state = 'STOP_PENDING'
70     elif not is_installed:
71         state = 'NOT_INSTALLED'
72
73     return {
74         'installed': is_installed,
75         'state': state,
76         'raw_output': output.strip()
77     }
78     except Exception as ex:
79         return {'installed': False, 'state': 'ERROR', 'error':
80             str(ex)}
81
82 def cmd_status() -> int:
83     """Выводит подробный статус службы и базы данных."""
84     print("=" * 60)
85     print("          AITelemetry - Статус службы и мониторинга")
86     print("=" * 60)
87
88     scm_status = get_service_status_scm('AITelemetry')
89     state = scm_status.get('state', 'UNKNOWN')
90     state_icon = "■ " if state == 'RUNNING' else ("■ " if stat
91 e == 'STOPPED' else "■ ")
92     print(f"Служба Windows (SCM):    {state_icon} {state}")
93     print(f"Установлена в системе: {'Да' if scm_status.get('in
94 stalled') else 'Нет'}")
95
96     try:
97         from apps.windows.telemetry.telemetry_config import Te
98 lemetryConfigManager
99         cfg_mgr = TelemetryConfigManager()
100         cfg_path = cfg_mgr.config_path
101         print(f"Файл конфигурации:    {cfg_path} (существует:

```



```

    {cfg_path.exists()})")
99         print(f"Режим работы:           {cfg_mgr.get_mode()}")
100         print(f"Интервал замеров:       {cfg_mgr.get_interval_s
seconds()} сек")
101     except Exception as ex:
102         print(f"Конфигурация:           Ошибка чтения ({ex})")
103
104     try:
105         from apps.windows.telemetry.sqlite import TelemetrySto
rage
106         storage = TelemetryStorage.get_instance(read_only=True
)
107         db_path = storage.db_path
108         print(f"База данных SQLite:     {db_path} (размер: {db_
path.stat().st_size / 1024 / 1024:.2f} МБ)")
109         stats = storage.get_storage_stats()
110         print(f"Всего снимков (polls): {stats.get('snapshots_c
ount', 0)}")
111         print(f"Метрик процессов:       {stats.get('process_met
rics_count', 0)}")
112         print(f"Событий (events):       {stats.get('events_coun
t', 0)}")
113     except Exception as ex:
114         print(f"База данных:           Недоступна ({ex})")
115
116     print("=" * 60)
117     return 0
118
119
120 def cmd_control(action: str) -> int:
121     """Управляет службой через sc.exe / net.exe.
122
123     Args:
124         action: Действие ('start', 'stop', 'restart').
125
126     Returns:
127         int: Код возврата команды.
128     """

```

```
129 |     if action == 'start':
130 |         print("Запуск службы AITelemetry...")
131 |         res = subprocess.run(['net.exe', 'start', 'AITelemetry
|'])
132 |         return res.returncode
133 |     elif action == 'stop':
134 |         print("Остановка службы AITelemetry...")
135 |         res = subprocess.run(['net.exe', 'stop', 'AITelemetry'
|'])
136 |         return res.returncode
137 |     elif action == 'restart':
138 |         print("Перезапуск службы AITelemetry...")
139 |         subprocess.run(['net.exe', 'stop', 'AITelemetry'])
140 |         res = subprocess.run(['net.exe', 'start', 'AITelemetry
|'])
141 |         return res.returncode
142 |     return 1
143 |
144 |
145 | def cmd_database() -> int:
146 |     """Выводит детальную статистику таблиц SQLite."""
147 |     try:
148 |         from apps.windows.telemetry.sqlite import TelemetrySto
|rage
149 |         storage = TelemetryStorage.get_instance(read_only=True
|)
150 |         stats = storage.get_storage_stats()
151 |         print(json.dumps(stats, indent=2, ensure_ascii=False))
152 |         return 0
153 |     except Exception as ex:
154 |         print(f"Ошибка чтения базы данных: {ex}")
155 |         return 1
156 |
157 |
158 | def cmd_diagnose() -> int:
159 |     """Выполняет экспресс-диагностику системных метрик."""
160 |     print("Выполнение глубокой диагностики системы...")
161 |     try:
```

```

162         from apps.windows.telemetry.collector import SystemCol
lector
163         collector = SystemCollector()
164         snapshot = collector.collect_snapshot(top_processes_li
mit=10)
165         print(f"CPU: {snapshot.cpu.usage_percent:.1f}%")
166         print(f"Память: {snapshot.memory.percent_used:.1f}% (С
вободно: {snapshot.memory.available_mb / 1024:.2f} ГБ)")
167         print(f"Количество процессов: {len(snapshot.processes)
}")
168         print("Диагностика успешно выполнена.")
169         return 0
170     except Exception as ex:
171         print(f"Ошибка при диагностике: {ex}")
172         return 1
173
174
175 def main() -> int:
176     """Точка входа утилиты AITelemetryCtl."""
177     parser = argparse.ArgumentParser(description='AITelemetry
CLI Controller')
178     subparsers = parser.add_subparsers(dest='command', help='К
оманды управления')
179
180     subparsers.add_parser('status', help='Показать статус служ
бы и БД')
181     subparsers.add_parser('start', help='Запустить службу')
182     subparsers.add_parser('stop', help='Остановить службу')
183     subparsers.add_parser('restart', help='Перезапустить служб
у')
184     subparsers.add_parser('database', help='Показать статистик
у базы данных')
185     subparsers.add_parser('diagnose', help='Выполнить экспресс
-диагностику')
186
187     args = parser.parse_args()
188
189     if not args.command or args.command == 'status':

```

```
190 |         return cmd_status()
191 |     elif args.command in ('start', 'stop', 'restart'):
192 |         return cmd_control(args.command)
193 |     elif args.command == 'database':
194 |         return cmd_database()
195 |     elif args.command == 'diagnose':
196 |         return cmd_diagnose()
197 |     return 0
198 |
199 |
200 | if __name__ == '__main__':
201 |     sys.exit(main())
202 |
```

deep_diagnostics.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Deep Di
  agnostics
4 # =====
5 # Description:
6 #   Модуль глубокой аппаратной и системной диагностики, поведе
  нческой форензики и анализа утечек.
7 #
8 # Usage Examples:
9 #   Python API:
10 #
11 #   from apps.windows.telemetry.deep_diagnostics import DeepDi
  agnosticsEngine
12 #
13 #   service = DeepDiagnosticsEngine()
14 #
15 # File: deep_diagnostics.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 06:47:00
21 # =====
22
23 """Модуль глубокой аппаратной и системной диагностики, поведе
  нческой форензики и анализа утечек.
24 """
25
26 from __future__ import annotations
27
28 import codecs
29 import ctypes
```

```
30 import ctypes.wintypes as wintypes
31 import os
32 import re
33 import struct
34 import subprocess
35 import sys
36 import time
37 from datetime import datetime, timezone, timedelta
38 from pathlib import Path
39 from typing import Any, Dict, List, Optional, Tuple
40
41 try:
42     import psutil
43     PSUTIL_AVAILABLE = True
44 except ImportError:
45     psutil = None
46     PSUTIL_AVAILABLE = False
47 try:
48     import winreg
49     WINREG_AVAILABLE = True
50 except ImportError:
51     winreg = None
52     WINREG_AVAILABLE = False
53 try:
54     from logger import logger
55 except ImportError:
56     from logger import logger
57
58 from apps.windows.telemetry.models import (
59     DefenderTelemetrySummary,
60     ExtendedSystemAuditReport,
61     ForensicsActivityReport,
62     KernelThrottlingReport,
63     PeripheralsNetworkReport,
64     ProcessLeakDiagnosticsReport,
65     ProcessLeakItem,
66     StartupTelemetrySummary,
67     StorageBatteryWearReport,
```

```

68     UserAccountsTelemetrySummary,
69     VssTelemetrySummary,
70 )
71
72 GR_GDIOBJECTS = 0
73 GR_USEROBJECTS = 1
74 PROCESS_QUERY_INFORMATION = 1024
75 PROCESS_VM_READ = 16
76
77 class LASTINPUTINFO(ctypes.Structure):
78     """Структура Win32 для GetLastInputInfo."""
79     _fields_ = [('cbSize', wintypes.UINT), ('dwTime', wintypes
80         .DWORD)]
81
82 class PROCESS_MEMORY_COUNTERS_EX(ctypes.Structure):
83     """Структура PSAPI для GetProcessMemoryInfo."""
84     _fields_ = [
85         ('cb', wintypes.DWORD),
86         ('PageFaultCount', wintypes.DWORD),
87         ('PeakWorkingSetSize', ctypes.c_size_t),
88         ('WorkingSetSize', ctypes.c_size_t),
89         ('QuotaPeakPagedPoolUsage', ctypes.c_size_t),
90         ('QuotaPagedPoolUsage', ctypes.c_size_t),
91         ('QuotaPeakNonPagedPoolUsage', ctypes.c_size_t),
92         ('QuotaNonPagedPoolUsage', ctypes.c_size_t),
93         ('PagefileUsage', ctypes.c_size_t),
94         ('PeakPagefileUsage', ctypes.c_size_t),
95         ('PrivateUsage', ctypes.c_size_t),
96     ]
97
98 class DeepDiagnosticsEngine:
99     """Движок глубокой системной диагностики и форензики."""
100
101     def __init__(self) -> None:
102         """Инициализация Win32 API интерфейсов."""
103         self._user32 = getattr(ctypes.windll, 'user32', None)
104         if os.name == 'nt' else None
105         self._kernel32 = getattr(ctypes.windll, 'kernel32', No

```

```

ne) if os.name == 'nt' else None
104         self._psapi = getattr(ctypes.windll, 'psapi', None) if
        os.name == 'nt' else None
105
106     def collect_process_leaks(self, limit: int = 100) -> Proce
ssLeakDiagnosticsReport:
107         """Собрать метрики скрытых утечек процессов (Handles,
        GDI, USER, Page Faults).
108
109         Args:
110             limit: Максимальное количество процессов в общем с
        писке.
111
112         Returns:
113             ProcessLeakDiagnosticsReport: Отчет с ранжирование
        м подозрительных процессов.
114         """
115         if not PSUTIL_AVAILABLE:
116             return ProcessLeakDiagnosticsReport()
117         items: List[ProcessLeakItem] = []
118         suspicious_count = 0
119         for proc in psutil.process_iter(['pid', 'name', 'statu
s', 'num_threads', 'cpu_percent', 'memory_info']):
120             try:
121                 info = proc.info
122                 pid = info.get('pid')
123                 if not pid or pid <= 4:
124                     continue
125                 name = info.get('name') or 'unknown'
126                 status = info.get('status') or 'running'
127                 threads_count = info.get('num_threads') or 1
128                 cpu_percent = info.get('cpu_percent') or 0.0
129                 mem_info = info.get('memory_info')
130                 memory_mb = round(mem_info.rss / (1024 * 1024)
        , 2) if mem_info else 0.0
131                 handles_count = 0
132                 gdi_objects = 0
133                 user_objects = 0

```



```

134         page_faults = 0
135         peak_ws_mb = memory_mb
136         if self._kernel32 and self._user32:
137             h_process = self._kernel32.OpenProcess(PRO
CESS_QUERY_INFORMATION | PROCESS_VM_READ, False, pid)
138             if h_process:
139                 try:
140                     dw_handles = wintypes.DWORD()
141                     if self._kernel32.GetProcessHandle
Count(h_process, ctypes.byref(dw_handles)):
142                         handles_count = int(dw_handles
.value)
143                         gdi_objects = int(self._user32.Get
GuiResources(h_process, GR_GDIOBJECTS))
144                         user_objects = int(self._user32.Ge
tGuiResources(h_process, GR_USEROBJECTS))
145                         if self._psapi:
146                             counters = PROCESS_MEMORY_COUN
TERS_EX()
147                             counters.cb = ctypes.sizeof(PR
OCESS_MEMORY_COUNTERS_EX)
148                             if self._psapi.GetProcessMemor
yInfo(h_process, ctypes.byref(counters), counters.cb):
149                                 page_faults = int(counters
.PageFaultCount)
150                                 peak_ws_mb = round(counter
s.PeakWorkingSetSize / (1024 * 1024), 2)
151                             finally:
152                                 self._kernel32.CloseHandle(h_proce
ss)
153                 if handles_count == 0:
154                     try:
155                         handles_count = proc.num_handles() if
hasattr(proc, 'num_handles') else 0
156                     except Exception:
157                         handles_count = 0
158                 risk_reasons: List[str] = []
159                 risk_score = 'normal'

```

```
160         if handles_count >= 4000:
161             risk_score = 'critical'
162             risk_reasons.append(f'Критическое число де
скрипторов: {handles_count}')
163         elif handles_count >= 1500:
164             if risk_score != 'critical':
165                 risk_score = 'warning'
166             risk_reasons.append(f'Высокое число дескри
пторов: {handles_count}')
167         if gdi_objects >= 2000:
168             risk_score = 'critical'
169             risk_reasons.append(f'Угроза исчерпания GD
I пула: {gdi_objects} (лимит 10 000)')
170         elif gdi_objects >= 800:
171             if risk_score != 'critical':
172                 risk_score = 'warning'
173             risk_reasons.append(f'Повышенное использо
вание GDI: {gdi_objects}')
174         if user_objects >= 500:
175             if risk_score != 'critical':
176                 risk_score = 'warning'
177             risk_reasons.append(f'Много USER объектов:
{user_objects}')
178         if page_faults >= 2000000:
179             risk_reasons.append(f'Высокая нагрузка на
подкачку ({page_faults:,} Hard Faults)')
180         if memory_mb >= 2048:
181             risk_reasons.append(f'Высокое потребление
памяти: {memory_mb:.1f} MB')
182         if risk_score != 'normal':
183             suspicious_count += 1
184         item = ProcessLeakItem(
185             pid=pid,
186             name=name,
187             status=status,
188             handles_count=handles_count,
189             gdi_objects=gdi_objects,
190             user_objects=user_objects,
```

```

191         page_faults_total=page_faults,
192         peak_working_set_mb=peak_ws_mb,
193         memory_mb=memory_mb,
194         threads_count=threads_count,
195         cpu_percent=cpu_percent,
196         leak_risk_score=risk_score,
197         leak_risk_reasons=risk_reasons,
198     )
199     items.append(item)
200     except (psutil.NoSuchProcess, psutil.AccessDenied,
201            psutil.ZombieProcess):
202         continue
203     except Exception as ex:
204         logger.debug(f'Ошибка сбора дескрипторов проце
205            cca: {ex}')
206     top_handles = sorted(items, key=lambda x: x.handles_co
207        unt, reverse=True)[:10]
208     top_gdi = sorted(items, key=lambda x: x.gdi_objects, r
209        everse=True)[:10]
210     top_page_faults = sorted(items, key=lambda x: x.page_f
211        aults_total, reverse=True)[:10]
212     risk_priority = {'critical': 0, 'warning': 1, 'normal'
213        : 2}
214     sorted_all = sorted(items, key=lambda x: (risk_priorit
215        y.get(x.leak_risk_score, 3), -x.handles_count))[:limit]
216     return ProcessLeakDiagnosticsReport(
217         timestamp=datetime.now(timezone.utc).isoformat(),
218         total_processes=len(items),
219         suspicious_count=suspicious_count,
220         top_handle_hogs=top_handles,
221         top_gdi_hogs=top_gdi,
222         top_page_fault_hogs=top_page_faults,
223         all_processes=sorted_all,
224     )
225
226     def collect_forensics_activity(self) -> ForensicsActivityR
227        eport:
228         """Собрать данные активности пользователя, активного о

```

```

    кна, доступа к сенсорам и UserAssist."""
221         fg_window: Dict[str, Any] = {'title': 'Рабочий стол /
Системный интерфейс', 'process_name': 'explorer.exe', 'pid': 0
    }
222         idle_seconds = 0.0
223         if self._user32:
224             try:
225                 hwnd = self._user32.GetForegroundWindow()
226                 if hwnd:
227                     length = self._user32.GetWindowTextLengthW
(hwnd)
228                     buff = ctypes.create_unicode_buffer(length
+ 1)
229                     self._user32.GetWindowTextW(hwnd, buff, le
ngth + 1)
230                     title = buff.value.strip() or 'Без заголов
ка'
231                     lp_pid = wintypes.DWORD()
232                     self._user32.GetWindowThreadProcessId(hwnd
, ctypes.byref(lp_pid))
233                     pid = int(lp_pid.value)
234                     proc_name = 'unknown'
235                     if PSUTIL_AVAILABLE and pid > 0:
236                         try:
237                             proc_name = psutil.Process(pid).na
me()
238                         except Exception:
239                             proc_name = f'PID {pid}'
240                     fg_window = {'title': title, 'process_name
': proc_name, 'pid': pid}
241             except Exception as ex:
242                 logger.debug(f'Ошибка получения ForegroundWind
ow: {ex}')
243         if self._user32 and self._kernel32:
244             try:
245                 lii = LASTINPUTINFO()
246                 lii.cbSize = ctypes.sizeof(LASTINPUTINFO)
247                 if self._user32.GetLastInputInfo(ctypes.byref(

```

```

l ii)):
248         tick_count = self._kernel32.GetTickCount()
249         idle_ms = max(0, tick_count - l ii.dwTime)
250         idle_seconds = round(idle_ms / 1000.0, 1)
251     except Exception as ex:
252         logger.debug(f'Ошибка расчета UserIdleTime: {e
x}')
253     cam_apps, mic_apps = self._get_camera_mic_access()
254     userassist_apps = self._get_userassist_apps(limit=15)
255     return ForensicsActivityReport(
256         timestamp=datetime.now(timezone.utc).isoformat(),
257         foreground_window=fg_window,
258         user_idle_seconds=idle_seconds,
259         camera_active_apps=cam_apps,
260         microphone_active_apps=mic_apps,
261         userassist_top_apps=userassist_apps,
262     )
263
264     def _get_camera_mic_access(self) -> Tuple[List[Dict[str, A
ny]], List[Dict[str, Any]]:
265         """Определить приложения, использующие камеру и микроф
он через реестр Windows."""
266         cam_list: List[Dict[str, Any]] = []
267         mic_list: List[Dict[str, Any]] = []
268         if not WINREG_AVAILABLE:
269             return (cam_list, mic_list)
270         capabilities = [('webcam', cam_list), ('microphone', m
ic_list)]
271         base_path = r'Software\Microsoft\Windows\CurrentVersio
n\CapabilityAccessManager\ConsentStore'
272         for cap_name, target_list in capabilities:
273             try:
274                 cap_key_path = f'{base_path}\\{cap_name}'
275                 with winreg.OpenKey(winreg.HKEY_CURRENT_USER,
cap_key_path, 0, winreg.KEY_READ) as key:
276                     num_subkeys, _, _ = winreg.QueryInfoKey(ke
y)
277                     for i in range(num_subkeys):

```

```

278         try:
279             sub_name = winreg.EnumKey(key, i)
280             if sub_name == 'NonPackaged':
281                 non_pkg_path = f'{cap_key_path
|}\\NonPackaged'
282                 with winreg.OpenKey(winreg.HKEY_CURRENT_USER, non_pkg_path, 0, winreg.KEY_READ) as np_key:
283                     np_count, _, _ = winreg.QueryInfoKey(np_key)
284                     for j in range(np_count):
285                         try:
286                             app_sub = winreg.EnumKey(np_key, j)
287                             with winreg.OpenKey(np_key, app_sub, 0, winreg.KEY_READ) as app_key:
288                                 stop_val, _ = winreg.QueryValueEx(app_key, 'LastUsedTimeStop')
289                                 start_val, _ = winreg.QueryValueEx(app_key, 'LastUsedTimeStart')
290                                 is_active = stop_val == 0 and start_val > 0
291                                 clean_app_name = app_sub.replace('#', '\\').split('\\')[-1]
292                                 target_list.append({
293                                     'app_name': clean_app_name,
294                                     'full_path': app_sub.replace('#', '\\'),
295                                     'is_active': is_active,
296                                     'type': 'Win32 Executable',
297                                 })
298             except Exception:
299                 continue
300         else:
301             with winreg.OpenKey(key, sub_n

```

```

    ame, 0, winreg.KEY_READ) as uwp_key:
302         try:
303             stop_val, _ = winreg.Q
QueryValueEx(uwp_key, 'LastUsedTimeStop')
304             start_val, _ = winreg.
QueryValueEx(uwp_key, 'LastUsedTimeStart')
305             is_active = stop_val =
= 0 and start_val > 0
306             target_list.append({
307                 'app_name': sub_na
me,
308                 'full_path': sub_n
ame,
309                 'is_active_now': i
s_active,
310                 'type': 'UWP / Win
dows App',
311             })
312         except Exception:
313             continue
314         except Exception:
315             continue
316     except Exception as ex:
317         logger.debug(f'Ошибка проверки доступа к {cap_
name}: {ex}')
318     return (cam_list, mic_list)
319
320     def _get_userassist_apps(self, limit: int = 15) -> List[Di
ct[str, Any]]:
321         """Извлечь историю запусков и фокусное время из ключей
UserAssist реестра (ROT13)."""
322         apps: List[Dict[str, Any]] = []
323         if not WINREG_AVAILABLE:
324             return apps
325         ua_path = r'Software\Microsoft\Windows\CurrentVersion\
Explorer\UserAssist'
326         try:
327             with winreg.OpenKey(winreg.HKEY_CURRENT_USER, ua_p

```



```

me,
352         'run_count': run_c
ount,
353         'focus_seconds': f
ocus_seconds,
354         'focus_formatted':
f'{focus_seconds // 60} мин {focus_seconds % 60} с',
355         })
356     except Exception:
357         continue
358     except Exception:
359         continue
360     except Exception as ex:
361         logger.debug(f'Ошибка UserAssist: {ex}')
362     apps.sort(key=lambda x: (x['run_count'], x['focus_seco
nds']), reverse=True)
363     return apps[:limit]
364
365     def collect_kernel_throttling(self) -> KernelThrottlingRep
ort:
366         """Собрать данные по прерываниям DPC/ISR, троттлингу,
Uptime и сбоям."""
367         dpc_pct = 0.0
368         interrupt_pct = 0.0
369         uptime_sec = 0.0
370         uptime_str = '0 дн 0 ч'
371         thermal_throttling = False
372         power_throttling = False
373         bsod_list: List[Dict[str, Any]] = []
374         pcie_info: Dict[str, Any] = {'gpu_name': 'PCIe Device'
, 'current_link_speed': 'Gen 3/4', 'current_link_width': 'x16'
, 'status': 'Optimal'}
375         if PSUTIL_AVAILABLE:
376             try:
377                 boot_time = psutil.boot_time()
378                 uptime_sec = max(0.0, time.time() - boot_time)
379                 days = int(uptime_sec // 86400)
380                 hours = int(uptime_sec % 86400 // 3600)

```

```

381         mins = int(uptime_sec % 3600 // 60)
382         uptime_str = f'{days} дн {hours} ч {mins} мин'
383         cpu_times = psutil.cpu_times_percent(interval=
0.1)
384         dpc_pct = round(getattr(cpu_times, 'dpc', 0.0)
    or 0.0, 2)
385         interrupt_pct = round(getattr(cpu_times, 'inte
rrupt', 0.0) or 0.0, 2)
386         except Exception as ex:
387             logger.debug(f'Ошибка сбора cpu_times: {ex}')
388         dpc_status = 'optimal'
389         if dpc_pct > 5.0 or interrupt_pct > 3.0:
390             dpc_status = 'severe'
391         elif dpc_pct > 1.5 or interrupt_pct > 1.0:
392             dpc_status = 'elevated'
393         minidump_dir = Path(os.environ.get('SystemRoot', 'C:\\
Windows')) / 'Minidump'
394         if minidump_dir.exists() and minidump_dir.is_dir():
395             try:
396                 dumps = sorted(minidump_dir.glob('*.dmp'), key
=lambda p: p.stat().st_mtime, reverse=True)[:5]
397                 for d in dumps:
398                     mtime = datetime.fromtimestamp(d.stat().st
_mtime, tz=timezone.utc).isoformat()
399                     bsod_list.append({
400                         'file_name': d.name,
401                         'timestamp': mtime,
402                         'size_kb': round(d.stat().st_size / 10
24, 1),
403                         'type': 'Crash Dump (BugCheck)',
404                     })
405             except Exception as ex:
406                 logger.debug(f'Ошибка проверки Minidump: {ex}'
    )
407         try:
408             from apps.windows.telemetry.sensors import get_har
dware_sensors
409             sensors = get_hardware_sensors()

```

```

410         for s in sensors:
411             name_l = s.name.lower()
412             if 'prochot' in name_l or 'thermal throttling'
in name_l:
413                 if s.value > 0:
414                     thermal_throttling = True
415                 if 'power limit' in name_l or 'pl1' in name_l
or 'pl2' in name_l:
416                     if s.value > 0:
417                         power_throttling = True
418         except Exception:
419             pass
420         return KernelThrottlingReport(
421             timestamp=datetime.now(timezone.utc).isoformat(),
422             dpc_latency_pct=dpc_pct,
423             interrupt_latency_pct=interrupt_pct,
424             dpc_status=dpc_status,
425             thermal_throttling_detected=thermal_throttling,
426             power_limit_throttling_detected=power_throttling,
427             system_uptime_seconds=uptime_sec,
428             uptime_formatted=uptime_str,
429             last_bsod_crashes=bsod_list,
430             gpu_pcie_link=pcie_info,
431         )
432
433     def collect_storage_battery_wear(self) -> StorageBatteryWe
arReport:
434         """Собрать данные износа SSD/NVMe и батареи питания."""
435
436         disks: List[Dict[str, Any]] = []
437         battery_data: Dict[str, Any] = {
438             'has_battery': False,
439             'design_capacity_mwh': 0.0,
440             'full_charge_capacity_mwh': 0.0,
441             'wear_level_pct': 0.0,
442             'cycle_count': 0,
443             'charge_rate_mw': 0.0,
444             'is_charging': False,

```

```

444         'percent': 0.0,
445         'power_source': 'AC Mains (Стационарное питание)',
446     }
447     try:
448         from apps.windows.modules.storage_manager.core.win
dows_storage_sensor import WindowsStorageSensor
449         sensor = WindowsStorageSensor()
450         snapshot = sensor.collect_snapshot(sync=True)
451         raw_partitions = snapshot.get('sources', {}).get('
partitions', [])
452         raw_logical = snapshot.get('sources', {}).get('log
ical_disks', [])
453
454         # Извлекаем сведения о логических дисках (буква том
а -> ФС и свободное место)
455         logical_info: Dict[str, Dict[str, Any]] = {}
456         for log_d in raw_logical:
457             if isinstance(log_d, dict):
458                 ld_id = str(log_d.get('DeviceID') or '').u
pper().rstrip('\\')
459                 if ld_id:
460                     logical_info[ld_id] = {
461                         'fs': log_d.get('FileSystem') or '
NTFS',
462                         'free_bytes': int(log_d.get('FreeS
pace') or 0),
463                     }
464
465         part_map: Dict[str, List[str]] = {}
466         part_free_map: Dict[str, float] = {}
467         for p in raw_partitions:
468             if isinstance(p, dict):
469                 dn = p.get('DiskNumber')
470                 dl = p.get('DriveLetter')
471                 if dn is not None and dl:
472                     dn_str = str(dn)
473                     letter = f'{dl}.'.upper()
474                     fs_name = logical_info.get(letter, {})

```

```

| .get('fs', 'NTFS')
475 |             part_map.setdefault(dn_str, []).append
| (f'{letter} [{fs_name}]')
476 |             free_b = logical_info.get(letter, {}).
| get('free_bytes', 0)
477 |             if free_b > 0:
478 |                 part_free_map[dn_str] = part_free_
| map.get(dn_str, 0.0) + (free_b / 1024 ** 3)
479 |
480 |             io_counters = psutil.disk_io_counters(perdisk=True
| ) if PSUTIL_AVAILABLE and hasattr(psutil, 'disk_io_counters')
| else {}
481 |             now_dt = datetime.now(timezone.utc)
482 |             physical_disks = sensor.get_physical_disks(sync=Tr
| ue)
483 |             if physical_disks:
484 |                 for d in physical_disks:
485 |                     dev_num_str = ''.join(filter(str.isdigit,
| d.device_id))
486 |                     mounted = part_map.get(dev_num_str, [])
487 |                     has_free = dev_num_str in part_free_map
488 |                     free_total = part_free_map.get(dev_num_str
| , 0.0)
489 |                     if not has_free and PSUTIL_AVAILABLE and m
| ounted:
490 |                         for part_label in mounted:
491 |                             m_letter = part_label.split()[0] i
| f part_label else ''
492 |                             if m_letter and ':' in m_letter:
493 |                                 try:
494 |                                     u = psutil.disk_usage(f'{m
| _letter}\\')
495 |                                     free_total += (u.free / 10
| 24 ** 3)
496 |                                     has_free = True
497 |                                 except Exception:
498 |                                     pass
499 |             io_key = f'PhysicalDrive{dev_num_str}'

```

```

500         io = io_counters.get(io_key)
501         bytes_read = io.read_bytes if io else (d.l
ifetime_read_bytes or 0)
502         bytes_written = io.write_bytes if io else
(d.lifetime_write_bytes or 0)
503         read_count = io.read_count if io else (d.r
ead_errors_total or 0)
504         write_count = io.write_count if io else (d
.write_errors_total or 0)
505         poh = d.power_on_hours
506         first_on_str = (now_dt - timedelta(hours=i
nt(poh))).strftime('%Y-%m-%d') if poh and poh > 0 else None
507         health_pct = 100.0
508         if d.wear_percentage is not None and d.wear_percentage > 0:
509             health_pct = max(0.0, min(100.0, 100.0
- float(d.wear_percentage)))
510             elif d.health_status and d.health_status.l
ower() in ('warning', 'caution'):
511                 health_pct = 70.0
512             elif d.health_status and d.health_status.l
ower() in ('unhealthy', 'critical', 'failed'):
513                 health_pct = 20.0
514                 status_str = 'Healthy (SMART OK)'
515                 if d.health_status and d.health_status not
in ('Healthy', 'OK', '0', 'PASSED'):
516                     status_str = d.health_status
517                 disks.append({
518                     'device_id': f'Disk {dev_num_str}' if
dev_num_str else d.device_id,
519                     'name': d.model or d.friendly_name or
'Физический накопитель',
520                     'model': d.model or d.friendly_name,
521                     'serial_number': d.serial_number if d.
serial_number and d.serial_number != 'N/A' else None,
522                     'bus_type': d.bus_type or 'Unknown',
523                     'media_type': d.media_type or 'SSD',
524                     'partitions': ', '.join(mounted) if mo

```



```

    ** 3, 1),
556         'health_pct': 100.0,
557         'wear_level_pct': 0.0,
558         'temperature_c': None,
559         'power_on_hours': None,
560         'status': 'Healthy (SMART OK)',
561     })
562     except Exception:
563         continue
564     except Exception as ex:
565         logger.debug(f'Ошибка сбора дисков (psutil fal
lback): {ex}')
566     if PSUTIL_AVAILABLE and hasattr(psutil, 'sensors_batte
ry'):
567         try:
568             bat = psutil.sensors_battery()
569             if bat is not None:
570                 battery_data['has_battery'] = True
571                 battery_data['percent'] = round(bat.percen
t, 1)
572                 battery_data['is_charging'] = bat.power_pl
ugged
573                 battery_data['power_source'] = 'AC Adapter
' if bat.power_plugged else 'Battery Discharge'
574                 battery_data['secsleft'] = bat.secsleft if
bat.secsleft != psutil.POWER_TIME_UNLIMITED else -1
575             except Exception as ex:
576                 logger.debug(f'Ошибка опроса батареи: {ex}')
577             if battery_data['has_battery']:
578                 try:
579                     ps_cmd = 'Get-CimInstance -Namespace root/wmi
-ClassName BatteryStaticData -ErrorAction SilentlyContinue | S
elect-Object DesignedCapacity, FullChargeCapacity | ConvertTo-
Json'
580                     res = subprocess.run(f'powershell.exe -NoProfi
le -NonInteractive -Command "{ps_cmd}"', capture_output=True,
text=True, timeout=5, shell=True)
581                     if res.returncode == 0 and res.stdout.strip():

```



```

582         import json
583         b_json = json.loads(res.stdout.strip())
584         if isinstance(b_json, dict):
585             des = float(b_json.get('DesignedCapacity') or 0.0)
586             full = float(b_json.get('FullChargeCapacity') or 0.0)
587             if des > 0 and full > 0:
588                 battery_data['design_capacity_mwh']
589                 ] = des
590                 battery_data['full_charge_capacity_mwh'] = full
591                 wear = max(0.0, min(100.0, (1.0 -
592                 full / des) * 100.0))
593                 battery_data['wear_level_pct'] = round(wear, 1)
594             except Exception:
595                 pass
596             return StorageBatteryWearReport(timestamp=datetime.now(
597             (timezone.utc).isoformat(), disks_wear=disks, battery_wear=battery_data)
598
599     def collect_peripherals_network(self) -> PeripheralsNetworkReport:
600         """Собрать данные по USB устройствам, беспроводной сети Wi-Fi и аудиоустройствам."""
601         usb_devices: List[Dict[str, Any]] = []
602         wifi_data: Dict[str, Any] = {
603             'is_connected': False,
604             'ssid': 'Не подключено',
605             'bssid': '--',
606             'signal_pct': 0,
607             'rssi_dbm': -100,
608             'channel': 0,
609             'radio_type': '--',
610             'auth_cipher': '--',
611         }
612         audio_endpoints: List[Dict[str, Any]] = []

```

```

610         try:
611             from apps.windows.telemetry.win32_ffi.setupapi import SetupAPI
612             pnp = SetupAPI()
613             devs = pnp.get_all_devices()
614             for d in devs:
615                 hw_id = d.hardware_id or ''
616                 if 'USB' in hw_id.upper() or 'USB' in (d.device_class or '').upper():
617                     vid_match = re.search('VID_([0-9A-Fa-f]{4})', hw_id)
618                     pid_match = re.search('PID_([0-9A-Fa-f]{4})', hw_id)
619                     vid = vid_match.group(1) if vid_match else
620                         '----'
621                     pid = pid_match.group(1) if pid_match else
622                         '----'
623                     usb_devices.append({
624                         'name': d.friendly_name or 'USB Устройство',
625                         'device_id': d.device_instance_id,
626                         'vendor_id': vid,
627                         'product_id': pid,
628                         'device_class': d.device_class,
629                         'has_problem': d.has_problem,
630                         'status': 'Error' if d.has_problem else 'Connected & Active',
631                     })
632             except Exception as ex:
633                 logger.debug(f'Ошибка сбора USB через SetupAPI: {ex}')
634             try:
635                 res = subprocess.run('netsh wlan show interfaces',
636                                     capture_output=True, text=True, timeout=4, shell=True)
637                 if res.returncode == 0 and res.stdout:
638                     out = res.stdout
639                     state_match = re.search(r'State\s*:\s*connected|Состояние\s*:\s*подключено', out, re.IGNORECASE)

```

```

637         if state_match:
638             wifi_data['is_connected'] = True
639             ssid_m = re.search(r'SSID\s*:\s*(.+)', out
        )
640             bssid_m = re.search(r'BSSID\s*:\s*(.+)', out)
641             sig_m = re.search(r'Signal|Сигнал\s*:\s*(\
        d+)\%', out)
642             chan_m = re.search(r'Channel|Канал\s*:\s*(
        \d+)\%', out)
643             radio_m = re.search(r'Radio type|Тип радио
        модуля\s*:\s*(.+)', out)
644             auth_m = re.search(r'Authentication|Провер
        ка подлинности\s*:\s*(.+)', out)
645             if ssid_m:
646                 wifi_data['ssid'] = ssid_m.group(1).st
        rip()
647             if bssid_m:
648                 wifi_data['bssid'] = bssid_m.group(1).
        strip()
649             if sig_m:
650                 sig_pct = int(sig_m.group(1))
651                 wifi_data['signal_pct'] = sig_pct
652                 wifi_data['rssi_dbm'] = int(sig_pct /
        2.0 - 100)
653             if chan_m:
654                 wifi_data['channel'] = int(chan_m.grou
        p(1))
655             if radio_m:
656                 wifi_data['radio_type'] = radio_m.grou
        p(1).strip()
657             if auth_m:
658                 wifi_data['auth_cipher'] = auth_m.grou
        p(1).strip()
659         except Exception as ex:
660             logger.debug(f'Ошибка Wi-Fi netsh: {ex}')
661         try:
662             audio_endpoints.append({'name': 'Основное устройст

```

```

        во воспроизведения (Default Playback)', 'type': 'Speakers / He
adphones', 'is_default': True, 'status': 'Active'})
663         audio_endpoints.append({'name': 'Основное устройст
во записи (Default Recording)', 'type': 'Microphone', 'is_defa
ult': True, 'status': 'Active'})
664     except Exception:
665         pass
666     return PeripheralsNetworkReport(timestamp=datetime.now
(timezone.utc).isoformat(), usb_devices=usb_devices, wifi_tele
metry=wifi_data, audio_endpoints=audio_endpoints)
667
668     def collect_defender_telemetry(self) -> DefenderTelemetryS
ummary:
669         """Собрать сводку телеметрии Windows Defender (CFA, AS
R, исключения, угрозы)."""
670         summary = DefenderTelemetrySummary()
671         try:
672             from apps.windows.defender.core.cfa_manager import
ControlledFolderAccessManager
673             cfa_mgr = ControlledFolderAccessManager()
674             cfa_info = cfa_mgr.get_cfa_status()
675             summary.cfa_enabled = bool(cfa_info.enabled)
676             summary.cfa_mode = str(cfa_info.mode.value if hasa
ttr(cfa_info.mode, 'value') else cfa_info.mode)
677             summary.cfa_protected_folders_count = len(cfa_info
.protected_folders)
678         except Exception as ex:
679             logger.debug(f'Ошибка сбора статуса CFA Defender:
{ex}')
680         try:
681             from apps.windows.defender.core.asr_manager import
AttackSurfaceReductionManager
682             asr_mgr = AttackSurfaceReductionManager()
683             rules = asr_mgr.get_all_rules_status()
684             summary.asr_rules_count = len(rules)
685             summary.asr_enabled_count = len([r for r in rules
if getattr(r, 'state', None) and str(r.state).lower() in ('ena
bled', 'block', 'audit', 'warn')])

```

```

686         except Exception as ex:
687             logger.debug(f'Ошибка сбора правил ASR Defender: {
ex}')
688         try:
689             from apps.windows.defender.core.exclusions_auditor
import DefenderExclusionsAuditor
690             excl_auditor = DefenderExclusionsAuditor()
691             excl = excl_auditor.audit_all_exclusions()
692             paths = getattr(excl, 'paths', []) or []
693             processes = getattr(excl, 'processes', []) or []
694             summary.path_exclusions = [str(p) for p in paths]
695             summary.process_exclusions = [str(p) for p in proc
esses]
696             summary.exclusions_count = len(summary.path_exclus
ions) + len(summary.process_exclusions)
697         except Exception as ex:
698             logger.debug(f'Ошибка сбора исключений Defender: {
ex}')
699         try:
700             from apps.windows.defender.core.threat_manager imp
ort ThreatManager
701             threat_mgr = ThreatManager()
702             active_threats = threat_mgr.get_active_threats()
703             threat_history = threat_mgr.get_threat_history(lim
it=50)
704             summary.active_threats_count = len(active_threats)
705             summary.threat_history_count = len(threat_history)
706         except Exception as ex:
707             logger.debug(f'Ошибка сбора угроз Defender: {ex}')
708         return summary
709
710     def collect_startup_telemetry(self) -> StartupTelemetrySum
mary:
711         """Собрать срез точек автозапуска и персистентности пр
ограмм."""
712         summary = StartupTelemetrySummary()
713         try:
714             from apps.windows.startup.core.scanner import Star

```

```

tupScanner
715         scanner = StartupScanner()
716         entries = scanner.scan_all()
717         summary.total_entries = len(entries)
718         serialized_entries: List[Dict[str, Any]] = []
719         for e in entries:
720             loc_type = str(getattr(e, 'location_type', ''))
721         )
722             if 'REGISTRY' in loc_type:
723                 summary.registry_run_count += 1
724             elif 'STARTUP_FOLDER' in loc_type:
725                 summary.startup_folders_count += 1
726             elif 'SCHEDULED_TASK' in loc_type:
727                 summary.scheduled_tasks_count += 1
728             elif 'SERVICE' in loc_type:
729                 summary.startup_services_count += 1
730             serialized_entries.append({
731                 'name': getattr(e, 'name', 'Unknown'),
732                 'command': getattr(e, 'command_line', ''),
733                 'location_type': loc_type,
734                 'is_enabled': getattr(e, 'is_enabled', True),
735                 'publisher': getattr(e, 'publisher', ''),
736             })
737             summary.entries = serialized_entries
738         except Exception as ex:
739             logger.debug(f'Ошибка сбора автозагрузки: {ex}')
740         return summary
741
742     def collect_vss_telemetry(self) -> VssTelemetrySummary:
743         """Собрать состояние теневых копий томов VSS."""
744         summary = VssTelemetrySummary()
745         try:
746             from apps.windows.modules.backup_manager.core.vss_
747             manager import VssManager
748             vss = VssManager()
749             snapshots = vss.list_snapshots()
750             summary.total_snapshots_count = len(snapshots)

```

```

749         vols: List[str] = []
750         serialized_snaps: List[Dict[str, Any]] = []
751         for s in snapshots:
752             vol = getattr(s, 'original_volume', '')
753             if vol and vol not in vols:
754                 vols.append(vol)
755                 serialized_snaps.append({
756                     'snapshot_id': getattr(s, 'snapshot_id', '
757                     '),
758                     'original_volume': vol,
759                     'creation_time': getattr(s, 'creation_time
760                     ', None),
761                     'shadow_volume_path': getattr(s, 'shadow_v
762                     olume_path', None),
763                     })
764                 summary.protected_volumes = vols
765                 summary.snapshots = serialized_snaps
766                 if snapshots and hasattr(snapshots[-1], 'creation_
767                 time'):
768                     summary.latest_snapshot_time = snapshots[-1].c
769                     reation_time
770                 except Exception as ex:
771                     logger.debug(f'Ошибка сбора теневых копий VSS: {ex
772                     }')
773                 return summary
774
775     def collect_users_telemetry(self) -> UserAccountsTelemetry
776     Summary:
777         """Собрать аудит локальных пользователей и привилегий.
778         """
779         summary = UserAccountsTelemetrySummary()
780         try:
781             from apps.windows.sysadmin.src.user_collector impo
782             rt WindowsUserCollector
783             collector = WindowsUserCollector()
784             users = collector.get_all_users()
785             summary.total_users_count = len(users)
786             admins: List[str] = []

```

```

778         logged_in: List[str] = []
779         locked_count = 0
780         serialized_users: List[Dict[str, Any]] = []
781         for u in users:
782             uname = getattr(u, 'name', '')
783             is_admin = getattr(u, 'is_admin', False)
784             is_logged = getattr(u, 'is_logged_in', False)
785             is_locked = getattr(u, 'lockout_status', False
) or not getattr(u, 'enabled', True)
786             if is_admin:
787                 admins.append(uname)
788             if is_logged:
789                 logged_in.append(uname)
790             if is_locked:
791                 locked_count += 1
792             serialized_users.append({
793                 'name': uname,
794                 'full_name': getattr(u, 'full_name', ''),
795                 'is_admin': is_admin,
796                 'is_logged_in': is_logged,
797                 'enabled': getattr(u, 'enabled', True),
798                 'account_type': getattr(u, 'account_type',
'Local'),
799                 'last_logon': getattr(u, 'last_logon', Non
e),
800             })
801         summary.admin_users_count = len(admins)
802         summary.admin_usernames = admins
803         summary.logged_in_users = logged_in
804         summary.locked_users_count = locked_count
805         summary.users = serialized_users
806     except Exception as ex:
807         logger.debug(f'Ошибка сбора локальных пользователе
й: {ex}')
808     return summary
809
810     def collect_extended_system_audit(self) -> ExtendedSystemA
uditReport:

```



```
811 |         """Собрать комплексный расширенный аудит безопасности,  
      | автозагрузки, VSS и пользователей."""  
812 |         import socket  
813 |         return ExtendedSystemAuditReport(  
814 |             timestamp=datetime.now(timezone.utc).isoformat(),  
815 |             hostname=socket.gethostname(),  
816 |             defender=self.collect_defender_telemetry(),  
817 |             startup=self.collect_startup_telemetry(),  
818 |             vss=self.collect_vss_telemetry(),  
819 |             users=self.collect_users_telemetry(),  
820 |         )  
821 |  
822 |
```

device_flapping_sensor.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Device
  Flapping Sensor
4 # =====
5 # Description:
6 #   Сенсор непрерывного мониторинга периферии, обнаружения дре
  безга и PnP-телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.device_flapping_sensor impor
  t DeviceTransitionEvent
11 #
12 #       service = DeviceTransitionEvent()
13 #
14 # File: device_flapping_sensor.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Сенсор непрерывного мониторинга периферии, обнаружения дреб
  езга и PnP-телеметрии."""
24
25 import os
26 import threading
27 import time
28 from dataclasses import asdict, dataclass, field
29 from datetime import datetime, timezone
```

```

30 from typing import Any, Callable, Dict, List, Optional, Set
31 from logger import logger
32 from .win32_ffl.setupapi import PnPDeviceInfo, SetupAPI
33 from .sqlite import TelemetryStorage
34
35 @dataclass
36 class DeviceTransitionEvent:
37     """Модель события изменения состояния устройства в телемет
    | рии."""
38     timestamp: str
39     event_type: str
40     device_instance_id: str
41     friendly_name: str
42     device_class: str
43     category: str
44     has_problem: bool
45     problem_code: int
46     status_code: int
47     manufacturer: str
48     flapping_count_in_window: int = 0
49     uptime_seconds: float = 0.0
50
51     def to_dict(self) -> Dict[str, Any]:
52         """Преобразование события в словарь для сериализации."
    | ""
53         return asdict(self)
54
55 class DeviceFlappingSensor:
56     """Сенсор непрерывного мониторинга и отлова дребезга обору
    | дования."""
57     CATEGORY_MAPPING: Dict[str, str] = {'HIDClass': 'HID & Пер
    | иферия', 'Mouse': 'Мышь / Указатель', 'Keyboard': 'Клавиатура'
    | , 'DiskDrive': 'Дисконкей / Накопитель', 'Volume': 'Том накопи
    | теля', 'USB': 'USB Устройство / Контроллер', 'USBDevice': 'USB
    | Периферия', 'Monitor': 'HDMI / DP / Монитор', 'Display': 'Вид
    | еоадаптер / Дисплей', 'Printer': 'Принтер', 'PrintQueue': 'Оче
    | редь печати', 'Bluetooth': 'Bluetooth Адаптер / Устройство', '
    | Digitizer': 'Wacom / Графический планшет', 'Media': 'Аудио / В

```

```
идео / Микрофон', 'Camera': 'Веб-камера', 'Net': 'Сетевой адаптер']
```

58

```
59     def __init__(self, poll_interval_sec: float=1.0, flapping_
window_sec: float=60.0, flapping_threshold: int=3, storage: Op
tional[TelemetryStorage]=None, on_event_callback: Optional[Cal
lable[[DeviceTransitionEvent], None]]=None, json_logger: Any=None) -> None:
```

```
60         """Инициализирует сенсор телеметрии устройств.
```

61

```
62         Args:
```

```
63             poll_interval_sec: Интервал между опросами оборудо
вания в секундах.
```

```
64             flapping_window_sec: Временное окно для фиксации д
ребезга (сек).
```

```
65             flapping_threshold: Количество отключений за окно
для вызова тревоги FLAPPING_ALERT.
```

```
66             storage: Хранилище SQLite (по умолчанию использует
ся синглтон).
```

```
67             on_event_callback: Опциональный callback-обработчи
к для передачи событий в UI/RAG.
```

```
68             json_logger: Опциональный логгер JSONL для обратно
й совместимости и тестирования.
```

```
69         """
```

```
70         self.poll_interval = max(0.1, poll_interval_sec)
```

```
71         self.flapping_window = flapping_window_sec
```

```
72         self.flapping_threshold = flapping_threshold
```

```
73         self.on_event_callback = on_event_callback
```

```
74         self.json_logger = json_logger
```

```
75         self.storage = storage or TelemetryStorage.get_instanc
```

```
        e()
```

```
76         self._setupapi = SetupAPI() if os.name == 'nt' else No
ne
```

```
77         self._is_running = False
```

```
78         self._thread: Optional[threading.Thread] = None
```

```
79         self._lock = threading.Lock()
```

```
80         self._active_devices: Dict[str, PnPDeviceInfo] = {}
```

```
81         self._connected_since: Dict[str, float] = {}
```

```

82         self._disconnect_history: Dict[str, List[float]] = {}
83
84     def _resolve_category(self, device_class: str, friendly_name: str, instance_id: str) -> str:
85         """Определяет человекочитаемую категорию устройства.
86
87         Args:
88             device_class: PnP-класс из реестра SetupAPI.
89             friendly_name: Имя устройства.
90             instance_id: Идентификатор инстанса (VID/PID).
91
92         Returns:
93             str: Название категории.
94         """
95         combined = f'{device_class} {friendly_name} {instance_id}'.lower()
96         if 'wacom' in combined or 'digitizer' in combined or 'pen' in combined:
97             return 'Wacom / Графический планшет'
98         if 'hid' in combined and 'keyboard' in combined:
99             return 'Клавиатура'
100        if 'hid' in combined and 'mouse' in combined:
101            return 'Мышь / Указатель'
102        if 'bth' in combined or 'bluetooth' in combined:
103            return 'Bluetooth'
104        if 'monitor' in combined or 'display' in combined:
105            return 'HDMI / DP / Монитор'
106        if 'disk' in combined or 'usbstor' in combined:
107            return 'Дисконкей / Накопитель'
108        if 'print' in combined:
109            return 'Принтер'
110        return self.CATEGORY_MAPPING.get(device_class, device_class or 'Прочее оборудование')
111
112    def _get_current_devices_snapshot(self) -> Dict[str, PnPDeviceInfo]:
113        """Получает текущий снимок активных PnP-устройств.
114

```

```

115         Returns:
116             Dict[str, PnPDeviceInfo]: Словарь активных устройств по их Device Instance ID.
117             """
118         if self._setupapi is None:
119             return {}
120         try:
121             devices = self._setupapi.get_all_devices(only_present=True)
122             return {dev.device_instance_id: dev for dev in devices if dev.device_instance_id}
123         except Exception as e:
124             logger.error(f'Ошибка опроса SetupAPI в сенсоре устройств: {e}')
125             return {}
126
127     def _clean_flapping_history(self, now: float) -> None:
128         """Очищает устаревшие временные метки за пределами окна дребезга."""
129         cutoff = now - self.flapping_window
130         for dev_id in list(self._disconnect_history.keys()):
131             self._disconnect_history[dev_id] = [ts for ts in self._disconnect_history[dev_id] if ts >= cutoff]
132             if not self._disconnect_history[dev_id]:
133                 del self._disconnect_history[dev_id]
134
135     def _emit_event(self, event: DeviceTransitionEvent) -> None:
136         """Логирует событие телеметрии и оповещает внешние подписчики.
137
138         Args:
139             event: Экземпляр события изменения состояния устройства.
140             """
141         try:
142             self.storage.save_device_event(event.to_dict())
143         except Exception as ex:

```

```

144         logger.error(f'Ошибка сохранения device_event в БД
: {ex}')
145     if self.json_logger:
146         try:
147             if hasattr(self.json_logger, 'log_event'):
148                 self.json_logger.log_event(event.to_dict()
)
149             elif hasattr(self.json_logger, 'log'):
150                 self.json_logger.log(event.to_dict())
151         except Exception as ex:
152             logger.error(f'Ошибка сохранения device_event
в json_logger: {ex}')
153         log_msg = f"[{event.event_type}] [{event.category}] '{
event.friendly_name}' (ID: {event.device_instance_id}) | Ошибк
а: Code {event.problem_code} | Аптайм: {event.uptime_seconds:.
1f}с | Дребезгов: {event.flapping_count_in_window}"
154         if event.event_type in ('DISCONNECTED', 'FLAPPING_ALER
T') or event.has_problem:
155             logger.warning(log_msg)
156         else:
157             logger.info(log_msg)
158         if self.on_event_callback:
159             try:
160                 self.on_event_callback(event)
161             except Exception as ex:
162                 logger.error(f'Ошибка в on_event_callback: {ex
}')
163
164     def poll_once(self) -> List[DeviceTransitionEvent]:
165         """Выполняет один цикл опроса и детекции разницы состо
яний.
166
167         Returns:
168             List[DeviceTransitionEvent]: Список зафиксированны
х событий за текущий такт.
169         """
170         now = time.time()
171         iso_time = datetime.now(timezone.utc).isoformat()

```

```

172         events: List[DeviceTransitionEvent] = []
173         current_snapshot = self._get_current_devices_snapshot(
174     )
175         with self._lock:
176             self._clean_flapping_history(now)
177             if not self._active_devices and current_snapshot:
178                 self._active_devices = current_snapshot
179                 for dev_id in current_snapshot:
180                     self._connected_since[dev_id] = now
181                 logger.info(f'Сенсор инициализирован: обнаруже
но {len(current_snapshot)} активных устройств')
182                 return events
183                 current_ids = set(current_snapshot.keys())
184                 previous_ids = set(self._active_devices.keys())
185                 disconnected_ids = previous_ids - current_ids
186                 for dev_id in disconnected_ids:
187                     prev_dev = self._active_devices.pop(dev_id)
188                     connect_time = self._connected_since.pop(dev_i
d, now)
189                     uptime = max(0.0, now - connect_time)
190                     if dev_id not in self._disconnect_history:
191                         self._disconnect_history[dev_id] = []
192                         self._disconnect_history[dev_id].append(now)
193                         flapping_count = len(self._disconnect_history[
dev_id])
194                         category = self._resolve_category(prev_dev.dev
ice_class, prev_dev.friendly_name, dev_id)
195                         evt = DeviceTransitionEvent(timestamp=iso_time
, event_type='DISCONNECTED', device_instance_id=dev_id, friend
ly_name=prev_dev.friendly_name or prev_dev.device_class or dev
_id, device_class=prev_dev.device_class, category=category, ha
s_problem=prev_dev.has_problem, problem_code=prev_dev.problem_
code, status_code=prev_dev.status_code, manufacturer=prev_dev.
manufacturer, flapping_count_in_window=flapping_count, uptime_
seconds=uptime)
196                         events.append(evt)
197                         self._emit_event(evt)
198                         if flapping_count >= self.flapping_threshold:

```



```

198         flap_evt = DeviceTransitionEvent(timestamp
=iso_time, event_type='FLAPPING_ALERT', device_instance_id=dev
_id, friendly_name=prev_dev.friendly_name, device_class=prev_d
ev.device_class, category=category, has_problem=True, problem_
code=prev_dev.problem_code, status_code=prev_dev.status_code,
manufacturer=prev_dev.manufacturer, flapping_count_in_window=f
lapping_count, uptime_seconds=uptime)
199         events.append(flap_evt)
200         self._emit_event(flap_evt)
201     connected_ids = current_ids - previous_ids
202     for dev_id in connected_ids:
203         new_dev = current_snapshot[dev_id]
204         self._active_devices[dev_id] = new_dev
205         self._connected_since[dev_id] = now
206         flapping_count = len(self._disconnect_history.
get(dev_id, []))
207         category = self._resolve_category(new_dev.devi
ce_class, new_dev.friendly_name, dev_id)
208         evt = DeviceTransitionEvent(timestamp=iso_time
, event_type='CONNECTED', device_instance_id=dev_id, friendly_
name=new_dev.friendly_name or new_dev.device_class or dev_id,
device_class=new_dev.device_class, category=category, has_prob
lem=new_dev.has_problem, problem_code=new_dev.problem_code, st
atus_code=new_dev.status_code, manufacturer=new_dev.manufactur
er, flapping_count_in_window=flapping_count, uptime_seconds=0.
0)
209         events.append(evt)
210         self._emit_event(evt)
211     common_ids = current_ids & previous_ids
212     for dev_id in common_ids:
213         old_dev = self._active_devices[dev_id]
214         new_dev = current_snapshot[dev_id]
215         if old_dev.has_problem != new_dev.has_problem
or old_dev.problem_code != new_dev.problem_code:
216             self._active_devices[dev_id] = new_dev
217             connect_time = self._connected_since.get(d
ev_id, now)
218             category = self._resolve_category(new_dev.

```

```

device_class, new_dev.friendly_name, dev_id)
219         evt = DeviceTransitionEvent(timestamp=iso_
time, event_type='ERROR_STATE_CHANGED', device_instance_id=dev
_id, friendly_name=new_dev.friendly_name, device_class=new_dev
.device_class, category=category, has_problem=new_dev.has_prob
lem, problem_code=new_dev.problem_code, status_code=new_dev.st
atus_code, manufacturer=new_dev.manufacturer, uptime_seconds=m
ax(0.0, now - connect_time))
220         events.append(evt)
221         self._emit_event(evt)
222     return events
223
224     def _loop(self) -> None:
225         """Фоновый цикл непрерывного опроса."""
226         logger.info(f'Запущен фоновый поток сенсора устройств
(интервал: {self.poll_interval}с, окно дребезга: {self.flappin
g_window}с, порог: {self.flapping_threshold})')
227         while self._is_running:
228             try:
229                 self.poll_once()
230             except Exception as e:
231                 logger.error(f'Необработанная ошибка в цикле о
проса сенсора: {e}')
232                 time.sleep(self.poll_interval)
233
234     def start(self) -> None:
235         """Запускает непрерывный мониторинг в фоновом потоке."
"""
236         with self._lock:
237             if self._is_running:
238                 logger.warning('Сенсор устройств уже запущен')
239                 return
240             self._is_running = True
241             self._thread = threading.Thread(target=self._loop,
name='DeviceFlappingSensorThread', daemon=True)
242             self._thread.start()
243
244     def stop(self) -> None:

```

```
245 |         """Останавливает непрерывный мониторинг."""
246 |     with self._lock:
247 |         if not self._is_running:
248 |             return
249 |         self._is_running = False
250 |     if self._thread and self._thread.is_alive():
251 |         self._thread.join(timeout=3.0)
252 |         logger.info('Сенсор устройств успешно остановлен')
253 |
254 |     def get_status(self) -> Dict[str, Any]:
255 |         """Возвращает текущую статистику сенсора.
256 |
257 |         Returns:
258 |             Dict[str, Any]: Данные о количестве активных устройс
259 |             """
260 |         with self._lock:
261 |             return {'is_running': self._is_running, 'poll_inte
262 |             rval_sec': self.poll_interval, 'active_devices_count': len(sel
263 |             f._active_devices), 'flapping_devices_tracked': len(self._disc
264 |             onnect_history), 'devices_with_problems': sum((1 for d in self
265 |             ._active_devices.values() if d.has_problem))}
```

file_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - File Collector
4 # =====
5 # Description:
6 #   Сбор событий файловой системы через DirectoryWatcher.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.file_collector import FileCollector
11 #
12 #       service = FileCollector()
13 #
14 # File: file_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-03 23:06:40
20 # =====
21
22 from __future__ import annotations
23 """Сбор событий файловой системы через DirectoryWatcher."""
24
25 import threading
26 from datetime import datetime, timezone
27 from pathlib import Path
28 from typing import Any, Dict, List, Optional
29 try:
30     from logger import logger
31 except ImportError:
```

```

32     from logger import logger
33
34
35 def _get_directory_watcher_classes():
36     """Ленивый импорт DirectoryWatcher для избежания циклическ
их зависимостей."""
37     try:
38         from apps.windows.modules.sysadmin.src.directory_watch
er import DirectoryWatcher, LiveFileEvent
39         return DirectoryWatcher, LiveFileEvent
40     except ImportError:
41         try:
42             from apps.windows.sysadmin.src.directory_watcher i
mport DirectoryWatcher, LiveFileEvent
43             return DirectoryWatcher, LiveFileEvent
44         except ImportError as e:
45             logger.warning(f'Не удалось импортировать Director
yWatcher: {e}')
46             return None, None
47
48
49 class FileCollector:
50     """Коллектор событий файловой системы."""
51
52     def __init__(self, watch_dirs: Optional[List[str]]=None, m
ax_history: int=200) -> None:
53         """Инициализирует коллектор файловых событий.
54
55         Args:
56             watch_dirs: Список директорий для мониторинга.
57             max_history: Размер истории событий.
58         """
59         self.watch_dirs = watch_dirs or [str(Path.home() / 'Do
cuments')]
60         self.max_history = max_history
61         self._watchers: Dict[str, Any] = {}
62         self._lock = threading.Lock()
63         self._init_watchers()

```

```

64 |
65 |     def _init_watchers(self) -> None:
66 |         """Инициализирует DirectoryWatcher для каждой директор
ии."""
67 |         DirectoryWatcher, _ = _get_directory_watcher_classes()
68 |         if DirectoryWatcher is None:
69 |             logger.warning('DirectoryWatcher недоступен')
70 |             return
71 |         for watch_dir in self.watch_dirs:
72 |             try:
73 |                 watcher = DirectoryWatcher(watch_dirs=[watch_d
ir], max_history=self.max_history)
74 |                 if watcher.start():
75 |                     self._watchers[watch_dir] = watcher
76 |                     logger.info(f'DirectoryWatcher запущен для
: {watch_dir}')
77 |             except Exception as e:
78 |                 logger.error(f'Ошибка инициализации DirectoryW
atcher для {watch_dir}: {e}')
79 |
80 |     def _convert_event(self, event: LiveFileEvent) -> Dict[str
, Any]:
81 |         """Преобразует LiveFileEvent в словарь для JSON.
82 |
83 |         Args:
84 |             event: Событие от DirectoryWatcher.
85 |
86 |         Returns:
87 |             Dict[str, Any]: Событие в формате словаря.
88 |         """
89 |         return {'timestamp': event.timestamp, 'action': event.
action, 'path': event.path, 'is_deletion': event.is_deletion,
'details': event.details}
90 |
91 |     def get_recent_events(self, limit: int=100) -> List[Dict[s
tr, Any]]:
92 |         """Получает недавние файловые события.
93 |

```

```

94 |         Args:
95 |             limit: Максимальное количество событий.
96 |
97 |         Returns:
98 |             List[Dict[str, Any]]: Список файловых событий.
99 |         """
100 |         with self._lock:
101 |             events: List[Dict[str, Any]] = []
102 |             for watch_dir, watcher in self._watchers.items():
103 |                 try:
104 |                     raw_events = watcher.get_recent_events(lim
it=limit)
105 |                     for event in raw_events:
106 |                         converted = self._convert_event(event)
107 |                         converted['watch_dir'] = watch_dir
108 |                         events.append(converted)
109 |                 except Exception as e:
110 |                     logger.error(f'Ошибка получения событий из
{watch_dir}: {e}')
111 |             events.sort(key=lambda x: x.get('timestamp', ''),
reverse=True)
112 |             return events[:limit]
113 |
114 |     def get_events_by_action(self, action: str, limit: int=50)
-> List[Dict[str, Any]]:
115 |         """Получает события определенного типа.
116 |
117 |         Args:
118 |             action: Тип действия (Created, Deleted, Modified).
119 |             limit: Максимальное количество событий.
120 |
121 |         Returns:
122 |             List[Dict[str, Any]]: Список событий.
123 |         """
124 |         all_events = self.get_recent_events(limit=limit * 2)
125 |         return [e for e in all_events if e.get('action') == ac
tion][:limit]
126 |

```

```

127 |     def get_events_by_type(self, event_type: str, limit: int=5
    | 0) -> List[Dict[str, Any]]:
128 |         """Получает события определенного типа (Created/Delete
    | d).
129 |
130 |         Args:
131 |             event_type: Тип события (Created или Deleted).
132 |             limit: Максимальное количество событий.
133 |
134 |         Returns:
135 |             List[Dict[str, Any]]: Список событий.
136 |         """
137 |         if event_type.lower() == 'created':
138 |             return self.get_events_by_action('Created', limit)
139 |         elif event_type.lower() == 'deleted':
140 |             return self.get_events_by_action('Deleted', limit)
141 |         return []
142 |
143 |     def stop(self) -> None:
144 |         """Останавливает все DirectoryWatcher."""
145 |         with self._lock:
146 |             for watcher in self._watchers.values():
147 |                 watcher.stop()
148 |             self._watchers.clear()
149 |             logger.info('FileCollector остановлен')
150 |
151 |     def restart(self) -> bool:
152 |         """Перезапускает DirectoryWatcher.
153 |
154 |         Returns:
155 |             bool: True если перезапуск успешен.
156 |         """
157 |         self.stop()
158 |         self._init_watchers()
159 |         return True
160 |
161 |     def add_watch_dir(self, watch_dir: str) -> bool:
162 |         """Добавляет директорию для мониторинга.
```



```

163
164     Args:
165         watch_dir: Путь к директории.
166
167     Returns:
168         bool: True если директория успешно добавлена.
169     """
170     DirectoryWatcher, _ = _get_directory_watcher_classes()
171     if DirectoryWatcher is None:
172         logger.warning('DirectoryWatcher недоступен')
173         return False
174     if watch_dir in self._watchers:
175         logger.warning(f'Директория уже отслеживается: {wa
176 tch_dir}')
177         return False
178     try:
179         watcher = DirectoryWatcher(watch_dirs=[watch_dir],
180 max_history=self.max_history)
181         if watcher.start():
182             self._watchers[watch_dir] = watcher
183             logger.info(f'Добавлена директория для мониторинга: {watch_dir}')
184             return True
185         except Exception as e:
186             logger.error(f'Ошибка добавления директории {watch
187 _dir}: {e}')
188             return False
189
190     def remove_watch_dir(self, watch_dir: str) -> bool:
191         """Удаляет директорию из мониторинга.
192
193     Args:
194         watch_dir: Путь к директории.
195
196     Returns:
197         bool: True если директория успешно удалена.
198     """
199     with self._lock:

```

```
197 |         if watch_dir in self._watchers:
198 |             watcher = self._watchers.pop(watch_dir)
199 |             watcher.stop()
200 |             logger.info(f'Удалена директория из мониторинг
a: {watch_dir}')
201 |             return True
202 |         else:
203 |             logger.warning(f'Директория не найдена в монит
оринге: {watch_dir}')
204 |             return False
```

grouped_telemetry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Grouped
  Telemetry Wrapper
4 # =====
5 # Description:
6 #   Обёртка, позволяющая импортировать группы телеметрии из мо
  дуля
7 #   `apps.windows.telemetry_research.grouped_telemetry` через
  путь
8 #   `apps.windows.telemetry.grouped_telemetry`. Это необходимо
  для совместимости
9 #   с существующими тестами, которые ожидают наличие данного м
  одуля в пакете
10 #   `apps.windows.telemetry`.
11 #
12 #   Внутри экспортируются все публичные типы, объявленные в ор
  игинальном модуле.
13 #   Переэкспорт делается без изменения логики - классы и функц
  ии остаются теми
14 #   же, что и в исследовательском подпакете.
15 # =====
16 # Updated: 2026-10-03 23:59:45
17 # =====
18
19 """Обёртка для экспорта групп телеметрии.
20
21 Этот модуль переэкспортирует ключевые классы из
22 `apps.windows.telemetry_research.grouped_telemetry`, обеспечив
  ая обратную
23 совместимость с кодом, который импортирует их напрямую из
24 `apps.windows.telemetry.grouped_telemetry`.
```

```
25 | """
26 |
27 | # Перезэкспорт из исследовательского подпакета
28 | from apps.windows.telemetry_research.grouped_telemetry import
29 | (
30 |     TelemetryGroupInfo,
31 |     GroupedTelemetryBuilder,
32 |     GroupDiagnoseRequest,
33 |     GroupDiagnosticResult,
34 |     SynthesisRequest,
35 |     SynthesisDiagnosticResult,
36 | )
37 | __all__ = [
38 |     "TelemetryGroupInfo",
39 |     "GroupedTelemetryBuilder",
40 |     "GroupDiagnoseRequest",
41 |     "GroupDiagnosticResult",
42 |     "SynthesisRequest",
43 |     "SynthesisDiagnosticResult",
44 | ]
45 |
```

incident_detector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Incident Detector
4 # =====
5 # Description:
6 #   Модуль непрерывного анализа аномалий и автоматического выявления сбоев системы.
7 #
8 # Usage Examples:
9 #   Python API:
10 #   from apps.windows.telemetry.incident_detector import IncidentDetector
11 #
12 # File: incident_detector.py
13 # Project: ai-breadboard
14 # Package: apps.windows.telemetry
15 # Author: hypo69
16 # Copyright: © 2026 hypo69
17 # Updated: 2026-10-03 23:15:00
18 # =====
19
20 """Модуль `IncidentDetector`.
21
22 Содержит полную реализацию детектора аномалий, перенесённую из
23 `telemetry_research`.
24
25 """
26
27 from __future__ import annotations
28
29 import time
30 import uuid
31 from datetime import datetime, timezone
```

```
30 from typing import Any, Dict, List, Optional
31
32 # logger import handling
33 try:
34     from logger import logger
35 except ImportError:
36     from logger import logger
37
38 from apps.windows.telemetry.models import TelemetryIncident
39 from apps.windows.telemetry.ring_buffer import TelemetryRingBu
40 ffer
41
42 class IncidentDetector:
43     """Детектор аномалий в потоках метрик и событий с фиксац
44 ие инцидентов."""
45
46     def __init__(
47         self,
48         cpu_threshold: float = 90.0,
49         disk_write_burst_mb: float = 100.0,
50         net_burst_mb: float = 50.0,
51         ring_buffer: Optional[TelemetryRingBuffer] = None,
52     ) -> None:
53         """Инициализирует детектор пороговыми значениями и ссы
54 лкой на кольцевой буфер.
55
56         Args:
57             cpu_threshold: Порог срабатывания по CPU % (по умо
58 лчанию 90.0).
59             disk_write_burst_mb: Порог дисковой записи в МБ/с
60 (по умолчанию 100.0).
61             net_burst_mb: Порог сетевого трафика в МБ/с (по ум
62 олчанию 50.0).
63             ring_buffer: Ссылка на экземпляр кольцевого буфера
64 памяти.
65         """
66         self.cpu_threshold = cpu_threshold
```

```

61         self.disk_write_burst_mb = disk_write_burst_mb
62         self.net_burst_mb = net_burst_mb
63         self.ring_buffer = ring_buffer or TelemetryRingBuffer(
64         )
65         self._last_incident_epoch: float = 0.0
66         self._cooldown_seconds: float = 60.0 # Не спамить оди
наковыми инцидентами чаще раза в минуту
67
68     def check_anomaly(
69         self,
70         system_snapshot: Dict[str, Any],
71         processes: Optional[List[Dict[str, Any]]] = None,
72         recent_events: Optional[List[Dict[str, Any]]] = None,
73     ) -> Optional[TelemetryIncident]:
74         """Анализирует срез метрик и возвращает TelemetryIncident при обнаружении аномалии.
75
76         Args:
77             system_snapshot: Текущий снимок системы (SystemSnapshot dict).
78             processes: Список процессов текущего среза.
79             recent_events: Последние дискретные события (W64, ETW, Defender).
80
81         Returns:
82             Optional[TelemetryIncident]: Объект инцидента или None при штатной работе.
83
84         """
85         now = time.time()
86         cpu_pct = float(system_snapshot.get("cpu_total_percent") or system_snapshot.get("cpu_percent") or 0.0)
87         disk_w_mbs = float(system_snapshot.get("disk_write_bytes_sec") or 0.0) / (1024.0 * 1024.0)
88         net_tx_mbs = float(system_snapshot.get("network_sent_bytes_sec") or 0.0) / (1024.0 * 1024.0)
89         net_rx_mbs = float(system_snapshot.get("network_recv_bytes_sec") or 0.0) / (1024.0 * 1024.0)

```

```
89         trigger_type: Optional[str] = None
90         severity = "warning"
91         title = ""
92         description = ""
93         suspects: List[Dict[str, Any]] = []
94
95         # 1. Проверка CPU Spike
96         if cpu_pct >= self.cpu_threshold:
97             trigger_type = "cpu_spike"
98             severity = "critical" if cpu_pct >= 95.0 else "war
99 ning"
100             title = f"Критическая нагрузка на процессор: {cpu_
101 pct:.1f}%"
102             description = f"Общая загрузка CPU превысила порог
103 {self.cpu_threshold}%, достигнув {cpu_pct:.1f}%."
104
105         # 2. Проверка Disk Write Burst
106         elif disk_w_mbs >= self.disk_write_burst_mb:
107             trigger_type = "disk_burst"
108             severity = "warning"
109             title = f"Аномальная скорость записи на диск: {dis
110 k_w_mbs:.1f} МБ/с"
111             description = f"Скорость дисковой записи превысила
112 порог {self.disk_write_burst_mb} МБ/с."
113
114         # 3. Проверка Network Burst
115         elif (net_tx_mbs + net_rx_mbs) >= self.net_burst_mb:
116             trigger_type = "network_burst"
117             severity = "warning"
118             title = f"Всплеск сетевой активности: {(net_tx_mbs
119 + net_rx_mbs):.1f} МБ/с"
120             description = f"Суммарный сетевой трафик достиг {(
121 net_tx_mbs + net_rx_mbs):.1f} МБ/с (TX: {net_tx_mbs:.1f}, RX:
122 {net_rx_mbs:.1f})."
123
124         # 4. Проверка подозрительных процессов (AppData, Temp,
125 Public)
126         if processes:
```



```

118         for p in processes:
119             exe_path = str(p.get("path") or p.get("exe") or "").lower()
120             p_cpu = float(p.get("cpu_percent") or 0.0)
121             is_suspicious_location = any(
122                 loc in exe_path for loc in ("\\temp\\", "\\appdata\\local\\temp", "\\users\\public\\")
123             )
124             if is_suspicious_location and (p_cpu > 15.0 or trigger_type is not None):
125                 suspects.append({
126                     "pid": p.get("pid"),
127                     "name": p.get("name"),
128                     "path": exe_path,
129                     "cpu_percent": p_cpu,
130                     "memory_mb": p.get("memory_mb"),
131                     "reason": "Запуск из временного каталога с повышенной активностью",
132                 })
133                 if trigger_type is None and p_cpu > 30.0:
134                     trigger_type = "suspicious_process"
135                     severity = "critical"
136                     title = f"Подозрительный процесс из Temp: {p.get('name')} (PID {p.get('pid')})"
137                     description = f"Процесс {p.get('name')} выполняется из {exe_path} и потребляет {p_cpu:.1f}% CPU."
138
139                 elif p_cpu >= 50.0:
140                     suspects.append({
141                         "pid": p.get("pid"),
142                         "name": p.get("name"),
143                         "path": exe_path,
144                         "cpu_percent": p_cpu,
145                         "memory_mb": p.get("memory_mb"),
146                         "reason": f"Высокое индивидуальное потребление CPU: {p_cpu:.1f}%",
147                     })
148

```

```

149 |         # 5. Проверка событий безопасности (Defender / ETW)
150 |         matched_events: List[Dict[str, Any]] = []
151 |         if recent_events:
152 |             for ev in recent_events:
153 |                 ev_type = str(ev.get("event_type") or "").lower()
154 |                 ev_str = str(ev).lower()
155 |                 if "defender" in ev_type or "threat" in ev_str
or "malware" in ev_str:
156 |                     matched_events.append(ev)
157 |                     if trigger_type is None:
158 |                         trigger_type = "defender_alert"
159 |                         severity = "critical"
160 |                         title = "Срабатывание Windows Defender
/ Защиты от угроз"
161 |                         description = f"Зафиксировано событие
безопасности: {ev.get('name') or ev_type}"
162 |
163 |                     if not trigger_type:
164 |                         return None
165 |
166 |         # Проверка кулдауна, чтобы не генерировать дубликаты е
жесекундно
167 |         if (now - self._last_incident_epoch) < self._cooldown_
seconds:
168 |             return None
169 |
170 |         self._last_incident_epoch = now
171 |         inc_id = f"INC-{datetime.now(timezone.utc).strftime('%
Y%m%d%H%M%S')}-{uuid.uuid4().hex[:4].upper()}"
172 |
173 |         # Захватываем срез из кольцевого буфера (например, 5 м
инут до события)
174 |         raw_window = self.ring_buffer.get_window(center_timest
amp=now, before_seconds=300.0, after_seconds=0.0)
175 |
176 |         incident = TelemetryIncident(
177 |             incident_id=inc_id,

```

```

178         timestamp=datetime.now(timezone.utc).isoformat(),
179         trigger_type=trigger_type,
180         severity=severity,
181         title=title,
182         description=description,
183         trigger_metrics={
184             "cpu_percent": cpu_pct,
185             "disk_write_mbs": disk_w_mbs,
186             "network_tx_mbs": net_tx_mbs,
187             "network_rx_mbs": net_rx_mbs,
188         },
189         suspect_processes=suspects,
190         related_events=matched_events or (recent_events[:1
191 0] if recent_events else []),
192         metrics_summary={
193             "buffer_samples_captured": len(raw_window),
194             "peak_cpu": cpu_pct,
195             "peak_disk_write_mbs": disk_w_mbs,
196         },
197         raw_window=raw_window,
198     )
199     logger.warning(f"Зафиксирован инцидент телеметрии [{incident.incident_id}]: {incident.title}")
200     return incident
201
202     @staticmethod
203     def format_incident_markdown_for_llm(incident: TelemetryIncident) -> str:
204         """Форматирует инцидент в компактный структурированный
205         Markdown для AI / LLM анализа.
206
207         Args:
208             incident: Объект TelemetryIncident.
209
210         Returns:
211             str: Готовый markdown блок для контекста языковой
212             модели.

```

```

211         """
212         lines = [
213             f"### □ ДИАГНОСТИЧЕСКИЙ ИНЦИДЕНТ: [{incident.incident_id}]",
214             f"- **Тип триггера:** `{incident.trigger_type}` |",
215             f"- **Критичность:** `{incident.severity.upper()}`",
216             f"- **Время:** {incident.timestamp}",
217             f"- **Сводка:** {incident.title}",
218             f"- **Описание:** {incident.description}",
219             "",
220             f"#### □ Метрики в момент инцидента:",
221             f"- CPU: `{incident.trigger_metrics.get('cpu_percent', 0.0)}%`",
222             f"- Диск (Запись): `{incident.trigger_metrics.get('disk_write_mbs', 0.0)} МБ/с`",
223             f"- Сеть (TX / RX): `{incident.trigger_metrics.get('network_tx_mbs', 0.0)} МБ/с` / `{incident.trigger_metrics.get('network_rx_mbs', 0.0)} МБ/с`",
224             f"- Сохранено секундных замеров в буфере: `{incident.metrics_summary.get('buffer_samples_captured', 0)}%`",
225             "",
226         ]
227         if incident.suspect_processes:
228             lines.append("#### □ Подозрительные процессы:")
229             lines.append("| PID | Имя | Путь | CPU % | Причина")
230             lines.append("|---|---|---|---|---|")
231             for sp in incident.suspect_processes:
232                 lines.append(
233                     f"| {sp.get('pid', '-')} | `{sp.get('name', '-')}` | `{sp.get('path', '-')}` | {sp.get('cpu_percent', 0.0)}% | {sp.get('reason', '-')}"
234                 )
235             lines.append("")
236         if incident.related_events:
237             lines.append("#### □ Сопутствующие события систем")

```

```
    ы (Events):")
239         for ev in incident.related_events[:5]:
240             ev_name = ev.get("name") or ev.get("event_type")
                or "event"
241             ev_time = ev.get("timestamp", "")
242             lines.append(f"- `{ev_time}`: **{ev_name}** ({
ev.get('path') or ev.get('details') or ''})")
243             lines.append("")
244
245             lines.append("Инструкция: Оцените потенциальную угрозу
(вирус/майнер/зависание/утечка) и предложите конкретные меры
противодействия.")
246         return "\n".join(lines)
247
```

init_db.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Init Db
4 # =====
5 # Description:
6 #   Модуль проверки, создания и валидации целостности структур
7 #   ы базы данных
8 #
9 # Usage Examples:
10 #   CLI:
11 #       python -m apps.windows.telemetry.init_db
12 #   Python API:
13 #       from apps.windows.telemetry.init_db import get_default_t
14 #       telemetry_db_path
15 #
16 #       res = get_default_telemetry_db_path()
17 #
18 # File: init_db.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-06 07:50:00
24 # =====
25
26
27 from __future__ import annotations
28 """Модуль проверки, создания и валидации целостности структуры
29   базы данных"""
30
31 import argparse
32 import json
33 import os
34 from pathlib import Path
```

```

31 import sqlite3
32 import sys
33 from typing import Any, Dict, List, Optional, Union
34
35 _project_root = str(Path(__file__).resolve().parents[3])
36 if _project_root not in sys.path:
37     sys.path.insert(0, _project_root)
38 if __package__ in (None, ''):
39     __package__ = 'apps.windows.telemetry'
40
41 try:
42     from logger import logger
43 except ImportError:
44     import logging
45     logger = logging.getLogger(__name__)
46
47 from .sqlite import TelemetryStorage
48
49
50 def get_default_telemetry_db_path() -> Path:
51     """Возвращает стандартный путь к файлу базы данных telemetry.db.
52
53     Returns:
54         Path: Путь к файлу telemetry.db в каталоге логов приложения.
55     """
56     programdata = os.environ.get('ProgramData') or os.environ.get('ALLUSERSPROFILE')
57     if programdata and os.path.exists(programdata):
58         base_dir = Path(programdata)
59     else:
60         appdata = os.environ.get('APPDATA') or os.environ.get('LOCALAPPDATA')
61         if appdata and os.path.exists(appdata):
62             base_dir = Path(appdata)
63         else:
64             base_dir = Path.home() / '.config'

```

```

65 |     return base_dir / 'AI-Breadboard' / 'apps' / 'windows' / '
    | telemetry' / 'logs' / 'telemetry.db'
66 |
67 |
68 | def init_telemetry_database(
69 |     db_path: Optional[Union[str, Path]] = None,
70 |     force: bool = False,
71 |     check_integrity: bool = True,
72 | ) -> Dict[str, Any]:
73 |     """Инициализирует базу данных telemetry.db, создавая все т
    | аблицы и индексы при отсутствии.
74 |
75 |     Args:
76 |         db_path: Путь к целевому файлу SQLite базы данных (по
    | умолчанию стандартный путь).
77 |         force: Принудительно переинициализировать структуру та
    | блиц.
78 |         check_integrity: Выполнить проверку целостности PRAGMA
    | integrity_check.
79 |
80 |     Returns:
81 |         Dict[str, Any]: Словарь с результатами инициализации и
    | метаданными БД.
82 |     """
83 |     target_path = Path(db_path) if db_path else get_default_te
    |lemetry_db_path()
84 |     target_path.parent.mkdir(parents=True, exist_ok=True)
85 |
86 |     was_created = False
87 |     if not target_path.exists() or force:
88 |         logger.debug(f'Инициализация новой базы данных телемет
    |рии по пути: {target_path}')
89 |         storage = TelemetryStorage(db_path=target_path)
90 |         was_created = True
91 |     else:
92 |         storage = TelemetryStorage.get_instance(db_path=target
    |_path)
93 |

```



```
94 | # Получение списка созданных таблиц
95 | tables: List[str] = []
96 | integrity_ok = True
97 | integrity_msg = 'ok'
98 |
99 | try:
100 |     with sqlite3.connect(str(target_path)) as conn:
101 |         cursor = conn.cursor()
102 |         cursor.execute("SELECT name FROM sqlite_master WHE
103 | RE type='table' AND name NOT LIKE 'sqlite_%';")
104 |         tables = sorted([row[0] for row in cursor.fetchall
105 | (())])
106 |
107 |         if check_integrity:
108 |             cursor.execute('PRAGMA integrity_check;')
109 |             res = cursor.fetchone()
110 |             if res and res[0] != 'ok':
111 |                 integrity_ok = False
112 |                 integrity_msg = str(res[0])
113 | except Exception as ex:
114 |     logger.error(f'Ошибка проверки схемы и целостности БД
115 | {target_path}: {ex}')
116 |     integrity_ok = False
117 |     integrity_msg = str(ex)
118 |
119 |     size_bytes = target_path.stat().st_size if target_path.exi
120 | sts() else 0
121 |     size_mb = round(size_bytes / (1024 * 1024), 3)
122 |
123 |     result = {
124 |         'status': 'success' if integrity_ok else 'warning',
125 |         'db_path': str(target_path),
126 |         'exists': target_path.exists(),
127 |         'created': was_created,
128 |         'size_bytes': size_bytes,
129 |         'size_mb': size_mb,
130 |         'tables_count': len(tables),
131 |         'tables': tables,
```

```
128         'integrity_ok': integrity_ok,
129         'integrity_message': integrity_msg,
130     }
131
132     if was_created:
133         logger.info(f'    База данных telemetry.db успешно созда
на: {len(tables)} таблиц ({size_mb} МБ)')
134     else:
135         logger.debug(f'База данных telemetry.db подтверждена:
{len(tables)} таблиц ({size_mb} МБ)')
136
137     return result
138
139
140 def main() -> int:
141     """Точка входа CLI для инициализации и проверки базы данны
х телеметрии."""
142     parser = argparse.ArgumentParser(
143         description='Инициализатор и верификатор SQLite базы д
анных системной телеметрии (telemetry.db)'
144     )
145     parser.add_argument('--db-path', type=str, default=None, h
elp='Пользовательский путь к telemetry.db')
146     parser.add_argument('--force', action='store_true', help='
Принудительно переинициализировать структуру')
147     parser.add_argument('--check-only', action='store_true', h
elp='Только проверить существующую БД без создания')
148     parser.add_argument('--json', action='store_true', help='В
ывод результата в формате JSON')
149
150     args = parser.parse_args()
151
152     target_path = Path(args.db_path) if args.db_path else get_
default_telemetry_db_path()
153
154     if args.check_only and not target_path.exists():
155         if args.json:
156             print(json.dumps({'status': 'error', 'message': f'
```

```

    Файл не найден: {target_path}', 'exists': False}, ensure_ascii
    =False))
157         else:
158             print(f'❌ База данных телеметрии не найдена: {targ
    et_path}')
159         return 1
160
161     res = init_telemetry_database(db_path=args.db_path, force=
    args.force)
162
163     if args.json:
164         print(json.dumps(res, ensure_ascii=False, indent=2))
165     else:
166         status_icon = '✅ ' if res['integrity_ok'] else '❌ '
167         print(f"\n{status_icon} База данных телеметрии: {res['
    db_path']}")
168         print(f"    • Существует:      {res['exists']}")
169         print(f"    • Вновь создана:  {res['created']}")
170         print(f"    • Размер файла:   {res['size_mb']} МБ ({res
    ['size_bytes']} байт)")
171         print(f"    • Таблиц в схеме: {res['tables_count']}")
172         print(f"    • Целостность:    {res['integrity_message']
    }")
173         if res['tables']:
174             print(f"    • Таблицы:      {'', '.join(res['table
    s'][:8])){'...' if len(res['tables']) > 8 else ''}")
175             print('')
176
177     return 0 if res['integrity_ok'] else 1
178
179
180 if __name__ == '__main__':
181     sys.exit(main())
182

```

internet_speed.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Internet Speed
4 # =====
5 # Description:
6 #   Сенсор телеметрии скорости интернета и задержки сети на базе FAST.com (Netflix CDN).
7 #   Выполняет периодический замер (1 раз в час / 3600 сек) и кэширует результаты.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.internet_speed import InternetSpeedSensor
12 #
13 #       sensor = InternetSpeedSensor()
14 #       metrics = sensor.measure_internet_speed()
15 #       print(metrics['download_mbps'], metrics['bufferbloat_ms'])
16 #
17 # File: internet_speed.py
18 # Project: ai-breadboard
19 # Package: apps.windows.telemetry
20 # Author: hypo69
21 # Copyright: © 2026 hypo69
22 # Updated: 2026-10-04 07:35:00
23 # =====
24
25 from __future__ import annotations
26 """Сенсор телеметрии скорости интернета и задержки сети на базе FAST.com."""
27
```

```

28 import asyncio
29 import threading
30 import time
31 from typing import Any, Dict, List, Optional
32
33 from logger import logger
34 from apps.windows.modules.network.speedtest import NetworkSpeedTester
35
36 _speed_cache: Optional[tuple[float, Dict[str, Any]]] = None
37 _CACHE_TTL = 3600 # 1 час между замерами скорости для снижения нагрузки на сеть
38 _cache_lock = threading.Lock()
39 _bg_test_running = False
40 _bg_lock = threading.Lock()
41
42
43 class InternetSpeedSensor:
44     """Сенсор измерения скорости интернета и характеристик задержки (FAST.com)."""
45
46     def __init__(self, cache_ttl: int = _CACHE_TTL) -> None:
47         """Инициализация сенсора скорости интернета.
48
49         Args:
50             cache_ttl: Время жизни кеша замера в секундах (по умолчанию 3600 сек = 1 час).
51
52         """
53         self.cache_ttl = cache_ttl
54         self._speedtester = NetworkSpeedTester()
55
56     @classmethod
57     def clear_cache(cls) -> None:
58         """Очистить глобальный кеш замеров скорости интернета.
59
60         """
61         global _speed_cache
62         with _cache_lock:
63             _speed_cache = None

```

```

61 |
62 |     def _run_speedtest_sync(self) -> Dict[str, Any]:
63 |         """Синхронно выполнить полный цикл теста скорости FAST
64 |         .com."""
65 |         try:
66 |             loop = None
67 |             try:
68 |                 loop = asyncio.get_event_loop()
69 |             except RuntimeError:
70 |                 loop = None
71 |
72 |             if loop and loop.is_running():
73 |                 # Если вызывается внутри уже работающего цикла
74 |                 событий (например, в async worker)
75 |                 import concurrent.futures
76 |                 with concurrent.futures.ThreadPoolExecutor(max
77 |                 _workers=1) as executor:
78 |                     future = executor.submit(lambda: asyncio.r
79 |                     un(self._speedtester.run_full_speedtest()))
80 |                     return future.result(timeout=60.0)
81 |             else:
82 |                 return asyncio.run(self._speedtester.run_full_
83 |                 speedtest())
84 |         except Exception as ex:
85 |             logger.error(f'Ошибка выполнения теста скорости FA
86 |             ST.com в сенсоре телеметрии: {ex}')
87 |             return {
88 |                 'download': {'speed_mbps': 0.0, 'status': 'ERR
89 |                 OR'},
90 |                 'upload': {'speed_mbps': 0.0, 'status': 'ERROR
91 |                 '},
92 |                 'ping_ms': 0.0,
93 |                 'latency_unloaded_ms': 0.0,
94 |                 'latency_loaded_ms': 0.0,
95 |                 'bufferbloat_ms': 0.0,
96 |                 'meta': {'ip': 'N/A', 'isp': 'Unknown'},
97 |                 'provider': 'fast.com',
98 |                 'server': 'N/A'

```

```

91         }
92
93     def _perform_measurement_and_cache(self) -> Dict[str, Any]
94     :
95         """Выполняет замер скорости и сохраняет его в кэш."""
96         global _speed_cache, _bg_test_running
97         try:
98             report = self._run_speedtest_sync()
99             download_mbps = float(report.get('download', {}).g
100 et('speed_mbps', 0.0))
101             upload_mbps = float(report.get('upload', {}).get('
102 speed_mbps', 0.0))
103             ping_ms = float(report.get('ping_ms', report.get('
104 latency_unloaded_ms', 0.0)))
105             latency_loaded_ms = float(report.get('latency_load
106 ed_ms', 0.0))
107             bufferbloat_ms = float(report.get('bufferbloat_ms'
108 , 0.0))
109             meta = report.get('meta', {})
110
111             result: Dict[str, Any] = {
112                 'available': report.get('download', {}).get('s
113 tatus') == 'SUCCESS' or download_mbps > 0.0,
114                 'download_mbps': download_mbps,
115                 'upload_mbps': upload_mbps,
116                 'ping_ms': ping_ms,
117                 'latency_unloaded_ms': float(report.get('laten
118 cy_unloaded_ms', ping_ms)),
119                 'latency_loaded_ms': latency_loaded_ms,
120                 'bufferbloat_ms': bufferbloat_ms,
121                 'provider': 'fast.com',
122                 'server': report.get('server', 'FAST.com CDN')
123             },
124             'ip': meta.get('ip', 'N/A'),
125             'isp': meta.get('isp', 'Unknown ISP'),
126             'measured_at': time.time(),
127         }
128     }
129

```

```

120         with _cache_lock:
121             _speed_cache = (time.monotonic(), result)
122
123         logger.info(
124             f' [Телеметрия FAST.com] Скорость: ↓{download_
125             d_mbps:.1f} Mbps, '
126             f'↑{upload_mbps:.1f} Mbps, пинг: {ping_ms:.1f}
127             mc, bufferbloat: {bufferbloat_ms:.1f} mc'
128         )
129         return dict(result)
130     finally:
131         with _bg_lock:
132             _bg_test_running = False
133
134     def trigger_async_measure(self) -> None:
135         """Запускает измерение скорости в фоновом потоке, если
136         оно еще не выполняется."""
137         global _bg_test_running
138         with _bg_lock:
139             if _bg_test_running:
140                 return
141             _bg_test_running = True
142
143         thread = threading.Thread(
144             target=self._perform_measurement_and_cache,
145             name="InternetSpeedTestWorker",
146             daemon=True
147         )
148         thread.start()
149
150     def measure_internet_speed(self, force_refresh: bool = False,
151                               sync: bool = False) -> Dict[str, Any]:
152         """Измеряет профиль интернет-соединения через FAST.com
153         с кешированием на 1 час.
154
155         По умолчанию выполняется асинхронно в фоне и не блокир
156         ует вызывающий поток.

```



```

152 |         Args:
153 |             force_refresh: Принудительно инициировать новый за
мер.
154 |             sync: Выполнить замер синхронно с блокировкой теку
щего потока.
155 |
156 |         Returns:
157 |             Dict[str, Any]: Словарь метрик (download_mbps, upl
oad_mbps, ping_ms, bufferbloat_ms, etc.).
158 |             """
159 |             global _speed_cache
160 |             with _cache_lock:
161 |                 if not force_refresh and _speed_cache is not None:
162 |                     cached_at, cached_result = _speed_cache
163 |                     if time.monotonic() - cached_at < self.cache_t
tl:
164 |                         return dict(cached_result)
165 |
166 |             if sync:
167 |                 return self._perform_measurement_and_cache()
168 |
169 |             # Асинхронный запуск замера в фоне
170 |             self.trigger_async_measure()
171 |
172 |             with _cache_lock:
173 |                 if _speed_cache is not None:
174 |                     _, cached_result = _speed_cache
175 |                     return dict(cached_result)
176 |
177 |             # Возвращаем заглушку по умолчанию без блокировки пото
ка
178 |             return {
179 |                 'available': True,
180 |                 'download_mbps': 0.0,
181 |                 'upload_mbps': 0.0,
182 |                 'ping_ms': 0.0,
183 |                 'latency_unloaded_ms': 0.0,
184 |                 'latency_loaded_ms': 0.0,

```

```

185         'bufferbloat_ms': 0.0,
186         'provider': 'fast.com',
187         'server': 'FAST.com CDN (замер в фоне...)',
188         'ip': 'N/A',
189         'isp': 'Unknown ISP',
190         'measured_at': 0.0,
191     }
192
193
194 def get_internet_speed_sensors() -> List[Dict[str, Any]]:
195     """Получить показания сетевых метрик скорости в формате сп
иска сенсоров.
196
197     Returns:
198         List[Dict[str, Any]]: Список метрик сенсоров интернета
199     """
200     sensor = InternetSpeedSensor()
201     metrics = sensor.measure_internet_speed()
202     sensors = [
203         {
204             'sensor_id': 'internet_ping',
205             'name': 'Internet Ping Latency',
206             'category': 'network',
207             'value': metrics.get('ping_ms', 0.0),
208             'unit': 'ms'
209         },
210         {
211             'sensor_id': 'internet_download',
212             'name': 'Internet Download Speed',
213             'category': 'network',
214             'value': metrics.get('download_mbps', 0.0),
215             'unit': 'Mbps'
216         },
217         {
218             'sensor_id': 'internet_upload',
219             'name': 'Internet Upload Speed',
220             'category': 'network',

```

```
221         'value': metrics.get('upload_mbps', 0.0),
222         'unit': 'Mbps'
223     },
224     {
225         'sensor_id': 'internet_bufferbloat',
226         'name': 'Internet Bufferbloat',
227         'category': 'network',
228         'value': metrics.get('bufferbloat_ms', 0.0),
229         'unit': 'ms'
230     },
231     {
232         'sensor_id': 'internet_latency_loaded',
233         'name': 'Internet Loaded Latency',
234         'category': 'network',
235         'value': metrics.get('latency_loaded_ms', 0.0),
236         'unit': 'ms'
237     },
238 ]
239 return sensors
```

json_logger.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Json Lo
  gger
4 # =====
5 # Description:
6 #   Модуль JSON-логгера телеметрии Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.json_logger import Telemetry
      JsonLogger
11 #
12 #       service = TelemetryJsonLogger()
13 #
14 # File: json_logger.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модуль JSON-логгера телеметрии Windows."""
24
25 import json
26 import logging
27 import os
28 import time
29 from pathlib import Path
30 from typing import List, Optional
31
```

```

32 _logger = logging.getLogger(__name__)
33
34
35 class TelemetryJsonLogger:
36     """Записывает телеметрические события в JSON-Lines файл.
37
38     Параметры
39     -----
40     log_dir: str
41         Путь к директории, где будут храниться лог-файлы.
42     filename: str
43         Базовое имя лог-файла (без расширения).
44     max_file_size_mb: float, optional
45         Максимальный размер файла в мегабайтах до ротации. По
46         умолчанию 5 МБ.
47     """
48     def __init__(self, log_dir: str, filename: str, max_file_s
49         ize_mb: float = 5.0) -> None:
50         self.log_dir = Path(log_dir)
51         self.log_dir.mkdir(parents=True, exist_ok=True)
52         self.base_filename = filename if filename.endswith('.j
53         sonl') else f'{filename}.jsonl'
54         self.max_bytes = int(max_file_size_mb * 1024 * 1024)
55         self.current_path = self.log_dir / self.base_filename
56         _logger.debug('Инициализирован TelemetryJsonLogger: di
57         r=%s, file=%s, max_bytes=%d', self.log_dir, self.base_filename
58         , self.max_bytes)
59
60     def _rotate_if_needed(self) -> None:
61         """Ротирует текущий лог-файл, если его размер превышает
62         лимит.
63
64         Новый файл получает суффикс с меткой времени.
65     """
66         if self.current_path.exists() and self.current_path.st
67         at().st_size >= self.max_bytes:
68             timestamp = time.strftime('%Y%m%d%H%M%S')
69             rotated_name = f"{self.base_filename.rstrip('.json

```

```

l')}_{{timestamp}}.jsonl"
63         rotated_path = self.log_dir / rotated_name
64         try:
65             self.current_path.rename(rotated_path)
66             _logger.info('Файл лога ротирован: %s -> %s',
self.current_path.name, rotated_name)
67         except OSError as exc:
68             _logger.error('Не удалось ротировать лог-файл:
%s', exc)
69
70     def _write_record(self, record: dict) -> None:
71         """Записывает одну строку JSON в текущий файл.
72         Файл открывается в режиме добавления.
73         """
74         line = json.dumps(record, ensure_ascii=False)
75         with self.current_path.open('a', encoding='utf-8') as
f:
76             f.write(line + '\n')
77         _logger.debug('Записана запись в %s: %s', self.current
_path.name, line)
78
79     def log(self, record: dict) -> bool:
80         """Записать одну запись.
81
82         Возвращает ``True`` при успешной записи.
83         """
84         try:
85             self._rotate_if_needed()
86             self._write_record(record)
87             return True
88         except Exception as exc:
89             _logger.exception('Ошибка записи лога: %s', exc)
90             return False
91
92     def log_batch(self, records: List[dict]) -> int:
93         """Записать список записей.
94
95         Возвращает количество успешно записанных элементов.

```

```

96         """
97         count = 0
98         for rec in records:
99             if self.log(rec):
100                 count += 1
101         return count
102
103     def get_last_measurement(self) -> Optional[dict]:
104         """Вернуть последнюю запись из текущего лог-файла, если
и файл существует.
105         """
106         if not self.current_path.exists():
107             return None
108         try:
109             with self.current_path.open('rb') as f:
110                 f.seek(0, os.SEEK_END)
111                 offset = -1
112                 while -offset < f.tell():
113                     f.seek(offset, os.SEEK_END)
114                     if f.read(1) == b'\n' and offset != -1:
115                         break
116                     offset -= 1
117                 f.seek(offset + 1, os.SEEK_END)
118                 last_line = f.readline().decode('utf-8').strip
119             ()
120                 return json.loads(last_line) if last_line else
None
121         except Exception as exc:
122             _logger.exception('Не удалось прочитать последнюю
запись: %s', exc)
123             return None
124
125     def get_log_file_size(self) -> int:
126         """Вернуть размер текущего лог-файла в байтах.
127         """
128         return self.current_path.stat().st_size if self.curren
t_path.exists() else 0

```

main.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Main
4 # =====
5 # Description:
6 #   Главная точка входа для сервиса сбора телеметрии Windows.
7 #
8 # Usage Examples:
9 #   CLI:
10 #       python -m apps.windows.telemetry.main
11 #   Python API:
12 #       from apps.windows.telemetry.main import parse_arguments
13 #
14 #       res = parse_arguments()
15 #
16 # File: main.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-06 07:50:00
22 # =====
23
24 from __future__ import annotations
25 """Главная точка входа для сервиса сбора телеметрии Windows."""
26
27 import argparse
28 import os
29 import signal
30 import sys
31 import threading
32 import time
```



```

33 from datetime import datetime, timezone
34 from pathlib import Path
35 from typing import Any, Dict, List, Optional
36 _project_root = str(Path(__file__).resolve().parents[3])
37 if _project_root not in sys.path:
38     sys.path.insert(0, _project_root)
39 if __package__ in (None, ''):
40     __package__ = 'apps.windows.telemetry'
41 try:
42     from logger import logger
43 except ImportError:
44     from logger import logger
45 _stop_event = threading.Event()
46
47 def parse_arguments() -> argparse.Namespace:
48     """Парсит аргументы командной строки.
49
50     Returns:
51         argparse.Namespace: Распарсенные аргументы.
52     """
53     parser = argparse.ArgumentParser(description='Windows Tele
54     metry Service (Minimal, Hybrid, Full)', formatter_class=argpar
55     se.RawDescriptionHelpFormatter)
56     parser.add_argument('--mode', choices=['minimal', 'hybrid'
57     , 'full'], default=None, help='Режим работы: minimal (легковес
58     ный), hybrid (минимал + тяжелые периодически), full (полный ст
59     ек)')
60     parser.add_argument('--minimal', action='store_true', help
61     ='Запустить в легковесном режиме (эквивалентно --mode minimal)
62     ')
63     parser.add_argument('--interval', type=float, default=None
64     , help='Интервал сбора быстрой телеметрии в секундах (по умолч
65     анию из config.json)')
66     parser.add_argument('--heavy-interval', type=float, defaul
67     t=None, help='Интервал сбора тяжелых сенсоров для hybrid режим
68     а в секундах (по умолчанию из config.json)')
69     parser.add_argument('--top-processes', type=int, default=N
70     one, help='Количество сохраняемых процессов с наибольшей нагру

```

```

    зкой (по умолчанию 10)')
59     parser.add_argument('--config', type=str, default=None, help='Путь к файлу конфигурации config.json')
60     parser.add_argument('--log-dir', type=str, default=None, help='Путь к директории для логов и SQLite БД телеметрии')
61     parser.add_argument('--verbose', '-v', action='store_true', help='Включить подробный вывод (DEBUG)')
62     parser.add_argument('--once', action='store_true', help='Выполнить однократный опрос метрик и завершить работу')
63     return parser.parse_args()
64
65 def signal_handler(signum: int, frame: Optional[object]) -> None:
66     """Обработчик сигналов для корректной остановки сервиса.
67
68     Args:
69         signum: Номер сигнала.
70         frame: Стек вызовов.
71     """
72     logger.info(f'Получен сигнал {signum}, остановка телеметрии и...')
73     _stop_event.set()
74
75 def _set_low_priority() -> None:
76     """Устанавливает пониженный приоритет процессу для экономии ресурсов CPU."""
77     try:
78         import psutil
79         p = psutil.Process()
80         p.nice(psutil.BELOW_NORMAL_PRIORITY_CLASS)
81         logger.debug('Установлен приоритет процесса: BELOW_NORMAL')
82     except Exception as ex:
83         logger.debug(f'Не удалось понизить приоритет процесса: {ex}')
84
85 def run_telemetry_service(
86     mode: str = 'hybrid',

```

```

87     interval: float = 5.0,
88     heavy_interval: float = 60.0,
89     top_processes: int = 10,
90     heavy_collectors: Optional[Dict[str, bool]] = None,
91     log_dir: Optional[str] = None,
92     config_path: Optional[str] = None,
93     once: bool = False,
94 ) -> int:
95     """Запускает универсальный цикл сбора телеметрии (Minimal,
    Hybrid или Full).
96
97     Args:
98         mode: Режим сбора ('minimal', 'hybrid', 'full').
99         interval: Быстрый интервал сбора базовых метрик в секундах.
100         heavy_interval: Периодический интервал сбора тяжелых сенсоров (для hybrid).
101         top_processes: Количество процессов в топе.
102         heavy_collectors: Словарь активных тяжелых сенсоров.
103         log_dir: Каталог сохранения БД и логов.
104         config_path: Путь к файлу конфигурации.
105
106     Returns:
107         int: Код завершения.
108     """
109     import psutil
110     from apps.windows.modules.hardware.gpu_prober import GpuProber
111     from .models import CpuMetrics, DiskIoMetrics, DiskPartitionMetrics, GpuMetrics, MemoryMetrics, NetworkInterfaceMetrics, ProcessMetrics, SystemSnapshot
112     from .sqlite import TelemetryStorage
113     from .telemetry_config import TelemetryConfigManager
114     _set_low_priority()
115     cfg_mgr = TelemetryConfigManager(config_path=config_path)
116     resolved_log_dir = log_dir or str(cfg_mgr.get_data_path())
117     os.makedirs(resolved_log_dir, exist_ok=True)
118     db_path = os.path.join(resolved_log_dir, 'telemetry.db')

```

```
119     storage = TelemetryStorage(db_path=db_path)
120     gpu_prober = GpuProber()
121     hostname = os.environ.get('COMPUTERNAME', 'localhost')
122     boot_time = psutil.boot_time()
123     psutil.cpu_percent(interval=None)
124
125     custom_mode_set = mode is not None and ('--mode' in sys.ar
gv or '--minimal' in sys.argv)
126     custom_interval_set = interval is not None and ('--interva
l' in sys.argv or '-i' in sys.argv)
127     custom_heavy_interval_set = heavy_interval is not None and
 '--heavy-interval' in sys.argv
128     custom_top_processes_set = top_processes is not None and '
--top-processes' in sys.argv
129
130     if not custom_mode_set and cfg_mgr:
131         mode = cfg_mgr.get_mode()
132     if not custom_interval_set and cfg_mgr:
133         interval = cfg_mgr.get_interval_seconds()
134     if not custom_heavy_interval_set and cfg_mgr:
135         heavy_interval = cfg_mgr.get_heavy_interval_seconds()
136     if not custom_top_processes_set and cfg_mgr:
137         top_processes = cfg_mgr.get_effective_process_limit()
138
139     heavy_disk_interval_sec = cfg_mgr.get_heavy_disk_scan_inte
rval()
140     max_heavy_duration_sec = cfg_mgr.get_heavy_mode_max_durati
on_days() * 24.0 * 3600.0
141     auto_switch_enabled = cfg_mgr.is_heavy_auto_switch_enabled
()
142
143     h_collectors = heavy_collectors or cfg_mgr.get_heavy_colle
ctors()
144     sensor_collector = None
145     if mode in ('hybrid', 'full') and h_collectors.get('hardwa
re_sensors', True):
146         try:
147             from .sensor_collector import SensorCollector
```

```

148         sensor_collector = SensorCollector(config_manager=
| cfg_mgr)
149         logger.info('SensorCollector (LHM/Hardware) инициа
| лизирован для тяжелых опросов')
150         except Exception as sc_err:
151             logger.debug(f'SensorCollector не доступен: {sc_er
| r}')
152     system_collector = None
153     deep_diag = None
154     try:
155         from .collector import SystemCollector
156         from apps.windows.telemetry_research.deep_diagnostics
| import DeepDiagnosticsEngine
157         system_collector = SystemCollector(storage=storage)
158         deep_diag = DeepDiagnosticsEngine()
159         identity = system_collector.get_system_identity()
160         disks_health = system_collector.get_physical_disks_heal
| th(force=True)
161         import asyncio
162         hw_tree = asyncio.run(system_collector.get_hardware_tr
| ee_async(force=True))
163         logger.info(f"□ Хост: {identity.get('hostname')} | По
| льзователь: {identity.get('username')} | ОС: Windows {identity
| .get('os_build')}")
164         logger.info(f"□ Язык: {identity.get('system_language'
| )} | Локаль: {identity.get('user_locale')} | Раскладки: {' , '
| .join(identity.get('input_languages', []))}")
165         if disks_health:
166             d_summary = " , ".join([f"{d.model} ({d.media_type}
| , {d.size_gb}GB, SMART:{d.health_status})" for d in disks_heal
| th])
167             logger.info(f"□ Накопители SMART (Опрос при запус
| ке): {d_summary}")
168             if hw_tree:
169                 logger.info(f"□ [АППАРАТНАЯ КОНФИГУРАЦИЯ] Специфи
| кация всего железа ПК определена при старте процесса ({len(hw_
| tree)} компонентов)")
170     except Exception as init_err:

```

```

171         logger.debug(f"Инициализация системного коллектора: {i
nit_err}")
172
173     # Проверка критических аудитов при запуске
174     try:
175         from apps.windows.telemetry_research.audit_startup_che
cker import run_startup_audit
176         startup_audit = run_startup_audit(
177             check_integrity=True,
178             check_performance=True,
179             check_drivers=False,
180             check_eventlog=False
181         )
182         if not startup_audit.is_healthy:
183             logger.error(
184                 f"⚠ STARTUP AUDIT: Обнаружены критические про
блемы! "
185                 f"Critical: {startup_audit.critical_count}, Wa
rnings: {startup_audit.warning_count}"
186             )
187             for finding in startup_audit.findings:
188                 logger.warning(f" - [{finding['severity']}.upp
er()] {finding['domain']}: {finding['title']}")
189             elif startup_audit.warning_count > 0:
190                 logger.warning(
191                     f"⚠ STARTUP AUDIT: Предупреждения: {startup_au
dit.warning_count} "
192                     f"({startup_audit.duration_ms}ms)"
193                 )
194             else:
195                 logger.info(f"⚠ STARTUP AUDIT: Все проверки пройде
ны ({startup_audit.duration_ms}ms)")
196         except Exception as audit_err:
197             logger.debug(f"Проверка аудитов при запуске: {audit_er
r}")
198
199     # Инициализация W64/ETW сборщиков системных событий
200     w64_collector = None

```

```
201     w64_etw_collector = None
202     w64_cfg = cfg_mgr.get_config().get('w64_collector', {}) if
    cfg_mgr else {}
203     if w64_cfg.get('enabled', True):
204         try:
205             from .w64_collector import AIW64Collector
206             from .w64_etw_collector import AIW64ETWCollector
207             w64_collector = AIW64Collector(
208                 storage=storage,
209                 enable_file_monitoring=w64_cfg.get('enable_fil
    e_monitoring', True),
210                 enable_process_monitoring=w64_cfg.get('enable_
    process_monitoring', True),
211                 enable_registry_monitoring=w64_cfg.get('enable
    _registry_monitoring', True),
212                 enable_network_monitoring=w64_cfg.get('enable_
    network_monitoring', True),
213                 enable_event_log_monitoring=w64_cfg.get('enabl
    e_event_log_monitoring', True),
214             )
215             w64_collector.start()
216             w64_etw_collector = AIW64ETWCollector(
217                 storage=storage,
218                 enable_process_trace=w64_cfg.get('enable_proce
    ss_trace', True),
219                 enable_disk_trace=w64_cfg.get('enable_disk_tra
    ce', True),
220                 enable_network_trace=w64_cfg.get('enable_netwo
    rk_trace', True),
221                 enable_registry_trace=w64_cfg.get('enable_regi
    stry_trace', True),
222             )
223             w64_etw_collector.start()
224             logger.info('  Сборщики событий W64 и ETW запущен
    ы в составе телеметрии')
225         except Exception as w64_err:
226             logger.debug(f'Не удалось запустить W64 сборщики:
    {w64_err}')
```

```

227
228     logger.info('=' * 60)
229     logger.info(f'Запущен сервис телеметрии Windows [Режим: {mode.upper()}]')
230     logger.info(f'Быстрый интервал: {interval} с | Топ процесс
ов: {top_processes}')
231     if mode == 'hybrid':
232         logger.info(f'Периодический тяжелый опрос: каждые {heavy_interval} с')
233     logger.info(f'База данных: {db_path}')
234     logger.info('=' * 60)
235     tick = 0
236     prev_disk_io = psutil.disk_io_counters()
237     prev_net_io = psutil.net_io_counters()
238     prev_io_time = time.time()
239     last_gpu_probe_time: Optional[float] = None
240     cached_gpu_metrics: List[GpuMetrics] = []
241     last_partition_check: float = 0.0
242     cached_part_list: List[Any] = []
243     last_heavy_time = time.time()
244     last_heavy_disk_scan_time = time.time()
245     heavy_mode_start_time: Optional[float] = time.time() if mode.lower() in ('full', 'heavy') else None
246     heavy_poll_running = False
247     heavy_poll_lock = threading.Lock()
248     db_cleanup_interval_sec = cfg_mgr.get_db_cleanup_interval_seconds()
249     last_db_cleanup_time = time.time()
250     aggregation_interval_sec = getattr(cfg_mgr, 'get_aggregation_interval', lambda: 300.0)() if cfg_mgr else 300.0
251     last_aggregation_time = time.time()
252     known_pids: Dict[int, str] = {}
253     flashing_log_path = Path(resolved_log_dir) / 'flashing_processes.log'
254
255     try:
256         while not _stop_event.is_set():
257             loop_start = time.time()

```



```

258
259         # Динамическая перезагрузка параметров из config.j
    son 'на лету'
260         cfg_mgr.check_tc_mode_auto_switch()
261         if cfg_mgr.check_and_reload():
262             if not custom_interval_set:
263                 interval = cfg_mgr.get_interval_seconds()
264             if not custom_heavy_interval_set:
265                 heavy_interval = cfg_mgr.get_heavy_interva
l_seconds()
266             if not custom_top_processes_set:
267                 top_processes = cfg_mgr.get_effective_proc
ess_limit()
268             if not custom_mode_set:
269                 mode = cfg_mgr.get_mode()
270                 heavy_disk_interval_sec = cfg_mgr.get_heavy_di
sk_scan_interval()
271                 db_cleanup_interval_sec = cfg_mgr.get_db_clean
up_interval_seconds()
272                 max_heavy_duration_sec = cfg_mgr.get_heavy_mod
e_max_duration_days() * 24.0 * 3600.0
273                 auto_switch_enabled = cfg_mgr.is_heavy_auto_sw
itch_enabled()
274                 h_collectors = cfg_mgr.get_heavy_collectors()
275                 logger.info(
276                     f" [Телеметрия CLI] Конфигурация config.
json обновлена 'на лету': "
277                     f"интервал={interval}с, тяжелый={heavy_int
erval}с, топ={top_processes}, режим={mode}, tel_mode={cfg_mgr.
get_telemetry_mode()}"
278                 )
279
280         now_dt = datetime.now(timezone.utc)
281         now_iso = now_dt.isoformat()
282         tick += 1
283         cpu_pct = psutil.cpu_percent(interval=None)
284         cpu_freq = psutil.cpu_freq()
285         freq_mhz = cpu_freq.current if cpu_freq else 0.0

```

```

286         cpu_model = ''
287         try:
288             cpu_model = os.environ.get('PROCESSOR_IDENTIFI
ER', '')
289         except Exception:
290             pass
291         cpu_metrics = CpuMetrics(
292             model=cpu_model,
293             physical_cores=psutil.cpu_count(logical=False)
or 1,
294             logical_cores=psutil.cpu_count(logical=True) o
r 1,
295             total_percent=cpu_pct,
296             frequency_mhz=freq_mhz,
297             temperature_celsius=None,
298         )
299         mem = psutil.virtual_memory()
300         mem_metrics = MemoryMetrics(total_gb=round(mem.tot
al / (1024 ** 3), 2), used_gb=round(mem.used / (1024 ** 3), 2)
, available_gb=round(mem.available / (1024 ** 3), 2), percent=
mem.percent)
301         now_sec = time.time()
302         if last_gpu_probe_time is None or now_sec - last_g
pu_probe_time >= 10.0:
303             try:
304                 gpu_data = gpu_prober.probe()
305                 cached_gpu_metrics = [
306                     GpuMetrics(
307                         name=getattr(g, 'name', 'GPU'),
308                         load_percent=getattr(g, 'load_perc
ent', None) or getattr(g, 'utilization_gpu_pct', None),
309                         memory_total_gb=round((getattr(g,
'memory_total_mb', 0.0) or 0.0) / 1024.0, 2),
310                         memory_used_gb=round((getattr(g, '
memory_used_mb', 0.0) or 0.0) / 1024.0, 2),
311                         temperature_celsius=getattr(g, 'te
mperature_celsius', None) or getattr(g, 'temperature_gpu_c', N
one),

```

```

312         ) for g in gpu_data
313     ]
314     except Exception:
315         cached_gpu_metrics = []
316         last_gpu_probe_time = now_sec
317         gpu_metrics_list = cached_gpu_metrics
318         io_elapsed = max(0.001, now_sec - prev_io_time)
319         cur_disk_io = psutil.disk_io_counters()
320         disk_read_kb = 0.0
321         disk_write_kb = 0.0
322         if cur_disk_io and prev_disk_io:
323             disk_read_kb = round((cur_disk_io.read_bytes -
324             prev_disk_io.read_bytes) / 1024.0 / io_elapsed, 1)
325             disk_write_kb = round((cur_disk_io.write_bytes
326             - prev_disk_io.write_bytes) / 1024.0 / io_elapsed, 1)
327             prev_disk_io = cur_disk_io
328             if not cached_part_list or now_sec - last_partitio
329             n_check >= 60.0:
330                 try:
331                     cached_part_list = psutil.disk_partitions(
332                     all=False)
333                     last_partition_check = now_sec
334                 except Exception:
335                     pass
336                 disk_partitions: List[DiskPartitionMetrics] = []
337                 for part in cached_part_list:
338                     try:
339                         usage = psutil.disk_usage(part.mountpoint)
340                         disk_partitions.append(DiskPartitionMetric
341                         s(device=part.device, mountpoint=part.mountpoint, fstype=part.
342                         fstype, total_gb=round(usage.total / 1024 ** 3, 2), used_gb=ro
343                         und(usage.used / 1024 ** 3, 2), free_gb=round(usage.free / 102
344                         4 ** 3, 2), percent=usage.percent))
345                     except (PermissionError, OSError):
346                         continue
347                 cur_net_io = psutil.net_io_counters()
348                 net_recv_kb = 0.0
349                 net_sent_kb = 0.0

```

```

342         if cur_net_io and prev_net_io:
343             net_recv_kb = round((cur_net_io.bytes_recv - p
rev_net_io.bytes_recv) / 1024.0 / io_elapsed, 1)
344             net_sent_kb = round((cur_net_io.bytes_sent - p
rev_net_io.bytes_sent) / 1024.0 / io_elapsed, 1)
345             prev_net_io = cur_net_io
346             prev_io_time = now_sec
347             net_metrics_list = [NetworkInterfaceMetrics(interf
ace_name='total', bytes_sent_sec=net_sent_kb * 1024.0, bytes_r
ecv_sec=net_recv_kb * 1024.0, packets_sent_sec=0.0, packets_re
cv_sec=0.0, errors_in=cur_net_io.errin if cur_net_io else 0, e
rrors_out=cur_net_io.errout if cur_net_io else 0)]
348             raw_procs = []
349             current_proc_map: Dict[int, Dict[str, Any]] = {}
350             for p in psutil.process_iter(['pid', 'name', 'memo
ry_info', 'username']):
351                 try:
352                     info = p.info
353                     pid = info.get('pid', 0)
354                     name = info.get('name') or 'unknown'
355                     if pid:
356                         current_proc_map[pid] = {'name': name,
'proc': p, 'info': info}
357                     mem_rss = (info.get('memory_info') or geta
ttr(p, 'memory_info', lambda: None()).rss if info.get('memory
_info') else 0
358                     raw_procs.append((mem_rss, p, info))
359                     except (psutil.NoSuchProcess, psutil.AccessDen
ied):
360                         continue
361
362             # Детектор вспыхивающих процессов (новые PID, заре
гистрированные на текущем тике)
363             if tick > 1:
364                 new_pids = set(current_proc_map.keys()) - set(
known_pids.keys())
365                 if new_pids:
366                     for n_pid in new_pids:

```

```

367 |         p_entry = current_proc_map[n_pid]
368 |         p_obj = p_entry['proc']
369 |         p_name = p_entry['name']
370 |         exe_path = 'N/A'
371 |         cmdline_str = 'N/A'
372 |         parent_info = 'N/A'
373 |         try:
374 |             exe_path = p_obj.exe()
375 |         except Exception:
376 |             pass
377 |         try:
378 |             cmdline_str = " ".join(p_obj.cmdli
ne())
379 |         except Exception:
380 |             pass
381 |         try:
382 |             ppid = p_obj.ppid()
383 |             parent_name = psutil.Process(ppid)
.name() if ppid else 'Unknown'
384 |             parent_info = f"{parent_name} (PID
: {ppid})"
385 |         except Exception:
386 |             pass
387 |
388 |         alert_msg = (
389 |             f" [ВСПЫШКА / НОВЫЙ ПРОЦЕСС] PID:
{n_pid} | {p_name} | "
390 |             f"Путь: {exe_path} | Родитель: {pa
rent_info} | Команда: {cmdline_str}"
391 |         )
392 |         logger.warning(alert_msg)
393 |         try:
394 |             with open(flashing_log_path, 'a',
encoding='utf-8') as f_out:
395 |                 f_out.write(f"[{datetime.now()
.strftime('%Y-%m-%d %H:%M:%S')}] PID: {n_pid} | {p_name} | Exe
: {exe_path} | Parent: {parent_info} | Cmd: {cmdline_str}\n")
396 |         except Exception:

```

```

397 |                                     pass
398 |
399 |         known_pids = {pid: data['name'] for pid, data in c
current_proc_map.items()}
400 |
401 |         raw_procs.sort(key=lambda x: x[0], reverse=True)
402 |         top_candidates = raw_procs[:top_processes]
403 |         top_procs: List[ProcessMetrics] = []
404 |         for mem_rss, p, info in top_candidates:
405 |             proc_cpu = 0.0
406 |             try:
407 |                 proc_cpu = p.cpu_percent(interval=None)
408 |             except Exception:
409 |                 pass
410 |             mem_mb = round(mem_rss / (1024 * 1024), 1)
411 |             raw_user = info.get('username')
412 |             clean_user = raw_user if isinstance(raw_user,
str) else ''
413 |             top_procs.append(ProcessMetrics(pid=info.get('
pid', 0), name=info.get('name') or 'unknown', cpu_percent=proc
_cpu, memory_mb=mem_mb, threads_count=1, status='running', use
rname=clean_user))
414 |             snapshot = SystemSnapshot(timestamp=now_iso, hostn
ame=hostname, uptime_seconds=round(time.time() - boot_time, 1)
, cpu=cpu_metrics, memory=mem_metrics, gpus=gpu_metrics_list,
disks=disk_partitions, network=net_metrics_list, top_processes
=top_procs)
415 |             try:
416 |                 storage.save_snapshot(snapshot, top_n=top_proc
esses)
417 |             except Exception as db_err:
418 |                 logger.warning(f'Ошибка сохранения снимка в SQ
Lite: {db_err}')
419 |
420 |         # Формирование информативного пошагового лога тика
421 |         active_win_str = "N/A (Фоновый режим)"
422 |         if deep_diag:
423 |             try:

```

```

424         forensics = deep_diag.collect_forensics_activity()
425         win_info = forensics.foreground_window
426         if win_info and win_info.get('title'):
427             active_win_str = f"\{win_info.get('title')}\\" ({win_info.get('process_name') or 'N/A'}, PID:{win_info.get('pid', 0)}) | Простой: {forensics.user_idle_seconds:.1f}c"
428         except Exception:
429             pass
430
431         disk_vol_str = ", ".join([f"{d.mountpoint} {d.used_gb:.0f}/{d.total_gb:.0f}GB ({d.percent}%)" for d in disk_partitions[:3]]) or "N/A"
432         top_procs_summary = ", ".join([f"{p.name} ({p.memory_mb:.0f}MB, {p.cpu_percent:.1f}% CPU)" for p in top_procs[:4]])
433
434         logger.info(f" [ТЕЛЕМЕТРИЯ #{tick}] Режим: {mode.upper()} | {datetime.now().strftime('%H:%M:%S')}")
435         logger.info(f" └─ CPU: {cpu_pct:.1f}% @ {freq_mhz:.0f}MHz ({cpu_metrics.physical_cores}P/{cpu_metrics.logical_cores}L) | RAM: {mem_metrics.used_gb:.1f}/{mem_metrics.total_gb:.1f}GB ({mem_metrics.percent}%)"
436         if gpu_metrics_list:
437             g0 = gpu_metrics_list[0]
438             logger.info(f" └─ GPU: {g0.name} | Нагрузка: {g0.load_percent or 0.0}% | Температура: {g0.temperature_celsius or 0.0}°C")
439             logger.info(f" └─ Диски: Чтение {disk_read_kb:.1f} KB/s | Запись {disk_write_kb:.1f} KB/s | Занято: {disk_vol_str}")
440             logger.info(f" └─ Сеть: ↓{net_recv_kb:.1f} KB/s | ↑{net_sent_kb:.1f} KB/s | Активность: {active_win_str}")
441             logger.info(f" └─ Топ процессов: {top_procs_summary}")
442             logger.info(f" └─ Буфер БД: снимок #{tick} добавлен (в буфере: {storage.get_buffered_count()} записей, БД:

```



```

collect_all_sensors()
467         if heavy_readings:
468             heavy_readings_total.extend(heavy_readings)
469         try:
470             storage.save_sensor_polls(heavy_readings)
471         except Exception as sp_err:
472             logger.debug(f'Ошибка сохранения sensor_polls в БД: {sp_err}')
473
474         # Разгруппировка и красивый вывод LHM сенсоров в консоль
475         lhm_temps = [f"{s.get('sensor_name')}: {s.get('value')}{s.get('unit')}" for s in heavy_readings if s.get('sensor_category') in ('Temperatures', 'Temperature')]
476         lhm_powers = [f"{s.get('sensor_name')}: {s.get('value')}{s.get('unit')}" for s in heavy_readings if s.get('sensor_category') in ('Powers', 'Power')]
477         lhm_fans = [f"{s.get('sensor_name')}: {s.get('value')}{s.get('unit')}" for s in heavy_readings if s.get('sensor_category') in ('Fans', 'Fan')]
478         lhm_clocks = [f"{s.get('sensor_name')}: {s.get('value')}{s.get('unit')}" for s in heavy_readings if s.get('sensor_category') in ('Clocks', 'Clock')]
479
480         logger.info(f" [LHM СЕНСОРЫ]
Опрошено {len(heavy_readings)} аппаратных датчиков:")
481         if lhm_temps:
482             logger.info(f"    └─ Температуры ({len(lhm_temps)}): {' '.join(lhm_temps[:8])}")
483         if lhm_powers:
484             logger.info(f"    └─ Энергопотребление ({len(lhm_powers)}): {' '.join(lhm_powers[:6])}")
485         if lhm_fans:
486             logger.info(f"    └─ Вентиляторы ({len(lhm_fans)}): {' '.join(lhm_fans)}")

```

```

487         if lhm_clocks:
488             logger.info(f"    └ Частот
ы ({len(lhm_clocks)}): {' ', ' '.join(lhm_clocks[:6])}")
489
490             # Детальный аудит портов, сетевых соедин
ений и утечек дескрипторов
491             if system_collector:
492                 try:
493                     ports = system_collector.get_l
istening_ports(limit=10)
494                     if ports:
495                         p_str = ", ".join([f"{p.po
rt}/{p.protocol} ({p.process_name or 'System'})" for p in port
s[:5]])
496                         logger.info(f"    [ТЯЖЕЛЫЙ
ОПРОС] Открытые слушающие порты ({len(ports)}): {p_str}")
497                     except Exception:
498                         pass
499                     try:
500                         net_conns = system_collector.g
et_process_network_activity(limit=5, only_internet=True)
501                         if net_conns:
502                             c_str = ", ".join([f"{c.na
me}->{c.remote_address} ({c.service_type})" for c in net_conns
[:3]])
503                             logger.info(f"    [ТЯЖЕЛЫЙ
ОПРОС] Активные соединения с Интернет ({len(net_conns)}): {c_s
tr}")
504                     except Exception:
505                         pass
506
507             if deep_diag:
508                 try:
509                     leaks = deep_diag.collect_proc
ess_leaks(limit=5)
510                     if leaks and leaks.top_handle_
hogs:
511                         top_h = leaks.top_handle_h

```

```

ogs[0]
512         logger.info(f" [ТЯЖЕЛЫЙ
ОПРОС] Топ процесса по дескрипторам: {top_h.name} (PID:{top_h.
pid}) -> {top_h.handles_count} handles, {top_h.gdi_objects} GD
I, {top_h.user_objects} USER")
513         except Exception:
514             pass
515
516         # Тяжелый опрос накопителей SMART и на
дежности (2 раза в день / каждые 12 часов)
517         h_now = time.time()
518         if system_collector and h_collectors.g
et('storage_smart', True) and (h_now - last_heavy_disk_scan_ti
me >= heavy_disk_interval_sec):
519             try:
520                 heavy_disks = system_collector
.get_physical_disks_health(force=True)
521                 if heavy_disks:
522                     d_summary = ", ".join([f"{
d.model} ({d.media_type}, {d.size_gb}GB, SMART:{d.health_statu
s})" for d in heavy_disks])
523                     logger.info(f" [ТЯЖЕЛЫЙ
ОПРОС (SMART)] Накопители SMART: {d_summary}")
524                     last_heavy_disk_scan_time = h_
now
525             except Exception as d_err:
526                 logger.debug(f"Ошибка тяжелого
сканирования накопителей: {d_err}")
527
528                 heavy_dur = time.time() - heavy_start
529                 logger.info(f' [{{mode.upper()}}] Перио
дический тяжелый опрос завершен за {heavy_dur:.2f}с (Всего сен
соров: {len(heavy_readings_total)})')
530             except Exception as h_err:
531                 logger.error(f'Ошибка периодического т
яжелого опроса: {h_err}')
532             finally:
533                 with heavy_poll_lock:

```

```

534 |             heavy_poll_running = False
535 |
536 |         if once:
537 |             _execute_heavy_poll_job()
538 |         else:
539 |             with heavy_poll_lock:
540 |                 if not heavy_poll_running:
541 |                     heavy_poll_running = True
542 |                     worker_thread = threading.Thread(
543 |                         target=_execute_heavy_poll_job
544 |                         ,
545 |                         name="TelemetryHeavyPollWorker
546 |                         ",
547 |                         daemon=True
548 |                     )
549 |                     worker_thread.start()
550 |
551 |             # Фоновый периодический контроль размера SQLite ба
552 |             зы данных telemetry.db
553 |             if now_sec - last_db_cleanup_time >= db_cleanup_in
554 |             terval_sec:
555 |                 try:
556 |                     prune_res = storage.enforce_size_limit()
557 |                     if prune_res.get('pruned'):
558 |                         logger.warning(
559 |                             f" [КОНТРОЛЬ РАЗМЕРА БД] Усечени
560 |                             е базы данных telemetry.db: "
561 |                             f"{prune_res.get('initial_size_mb'
562 |                             )} МБ -> {prune_res.get('final_size_mb')} МБ "
563 |                             f"(удалено снимков: {prune_res.get
564 |                             ('deleted_snapshots')})")
565 |                         )
566 |                     last_db_cleanup_time = now_sec
567 |                 except Exception as db_clean_err:
568 |                     logger.debug(f"Ошибка периодического контр
569 |                     оля размера БД: {db_clean_err}")
570 |
571 |             # Фоновая периодическая многоуровневая SQL-агрегац

```

```

    ия (hourly -> daily -> weekly -> monthly -> yearly)
564         if now_sec - last_aggregation_time >= aggregation_
interval_sec:
565             try:
566                 storage.flush()
567                 agg_res = storage.run_aggregation_pipeline
(
568                     h_cnt = agg_res.get('hourly', {}).get('buc
kets_aggregated', 0)
569                     sp_cnt = agg_res.get('hourly', {}).get('sp
ikes_recorded', 0)
570                     if h_cnt > 0 or sp_cnt > 0:
571                         logger.info(f" [SQL АГРЕГАЦИЯ] Обнов
лены бакеты: hourly={h_cnt}, зафиксировано всплесков={sp_cnt}"
)
572                         last_aggregation_time = now_sec
573                     except Exception as agg_err:
574                         logger.debug(f"Ошибка периодической SQL-аг
регации: {agg_err}")
575
576             if once:
577                 storage.flush()
578                 try:
579                     storage.run_aggregation_pipeline()
580                 except Exception:
581                     pass
582                 logger.info(' Однократный опрос системной тел
еметрии успешно выполнен (--once)')
583                 break
584
585             elapsed = time.time() - loop_start
586             sleep_time = max(0.05, interval - elapsed)
587             if _stop_event.wait(timeout=sleep_time):
588                 logger.info('Получен сигнал остановки (_stop_e
vent установлен)')
589                 break
590         except KeyboardInterrupt:
591             logger.info('Прервано пользователем (KeyboardInterrupt

```

```

    ')')
592     except Exception as ex:
593         logger.error(f'Критическая ошибка в цикле телеметрии:
    {ex}', exc_info=True)
594     finally:
595         if w64_collector:
596             try:
597                 w64_collector.stop()
598             except Exception:
599                 pass
600         if w64_etw_collector:
601             try:
602                 w64_etw_collector.stop()
603             except Exception:
604                 pass
605         logger.info('Процесс телеметрии успешно завершен')
606     return 0
607
608 def run_minimal_telemetry(interval: float=5.0, top_processes:
    int=10, log_dir: Optional[str]=None) -> int:
609     """Обертка для обратной совместимости: запуск в режиме min
    imal."""
610     return run_telemetry_service(mode='minimal', interval=inte
    rval, top_processes=top_processes, log_dir=log_dir)
611
612 def main() -> int:
613     """Главная точка входа сервиса телеметрии.
614
615     Returns:
616         int: Код завершения.
617     """
618     args = parse_arguments()
619     if args.verbose:
620         logger.setLevel('DEBUG')
621     signal.signal(signal.SIGINT, signal_handler)
622     signal.signal(signal.SIGTERM, signal_handler)
623     from .telemetry_config import TelemetryConfigManager
624     try:

```

```
625 |         config_manager = TelemetryConfigManager(config_path=args.config)
626 |     except Exception as e:
627 |         logger.warning(f'Не удалось загрузить конфигурацию ({e
628 |         config_manager = None
629 |     if args.minimal:
630 |         mode = 'minimal'
631 |     elif args.mode:
632 |         mode = args.mode
633 |     elif config_manager:
634 |         mode = config_manager.get_mode()
635 |     else:
636 |         mode = 'hybrid'
637 |     interval = args.interval or (config_manager.get_interval_s
638 |     heavy_interval = args.heavy_interval or (config_manager.ge
639 |     top_processes = args.top_processes or (config_manager.get_
640 |     heavy_collectors = config_manager.get_heavy_collectors() i
641 |     return run_telemetry_service(mode=mode, interval=interval,
642 |     heavy_interval=heavy_interval, top_processes=top_processes, h
643 |     sys.exit(main())
```

models.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Models
4 # =====
5 # Description:
6 #   Data models for system metrics, hardware specs, sensors, a
7 #   nd telemetry.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.models import TelemetryProvi
11 #       der
12 #       service = TelemetryProvider()
13 #
14 # File: models.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 13:55:00
20 # =====
21
22 from __future__ import annotations
23 """Data models for system metrics, hardware specs, sensors, an
24 d telemetry."""
25
26 from abc import ABC, abstractmethod
27 from datetime import datetime, timezone
28 from enum import Enum
29 from typing import Any, Dict, List, Optional
30 from pydantic import BaseModel, ConfigDict, Field
31 from apps.windows.modules.startup.core.models import StartupAr
```



```

| chiveEntry, StartupChangeItem
31 |
32 |
33 | class CpuInventoryInfo(BaseModel):
34 |     """Паспорт физического процессора (CPU Inventory)."""
35 |     processor_id: int = 0
36 |     name: str = ''
37 |     vendor: str = ''
38 |     architecture: str = 'x86_64'
39 |     physical_cores: int = 1
40 |     logical_cores: int = 1
41 |     base_frequency_mhz: float = 0.0
42 |     max_frequency_mhz: float = 0.0
43 |     l2_cache_kb: Optional[int] = None
44 |     l3_cache_kb: Optional[int] = None
45 |     socket: str = ''
46 |     features: List[str] = Field(default_factory=list)
47 |
48 |
49 | class CpuTelemetrySample(BaseModel):
50 |     """Срез телеметрии нагрузки и температур процессора."""
51 |     timestamp: str = ''
52 |     created_at: float = 0.0
53 |     processor_id: int = 0
54 |     total_percent: float = 0.0
55 |     user_percent: float = 0.0
56 |     kernel_percent: float = 0.0
57 |     frequency_mhz: float = 0.0
58 |     temperature_c: Optional[float] = None
59 |     package_power_w: Optional[float] = None
60 |     core_utilization: List[float] = Field(default_factory=list
| )
61 |     core_temperatures: List[float] = Field(default_factory=lis
| t)
62 |
63 |
64 | class RamModuleInventoryInfo(BaseModel):
65 |     """Паспорт физического модуля (планки) оперативной памяти

```

```

    (SPD)."""
66     slot_id: int = 0
67     bank_label: str = 'DIMM'
68     device_locator: str = ''
69     serial_number: str = ''
70     part_number: str = ''
71     manufacturer: str = ''
72     capacity_bytes: int = 0
73     capacity_gb: float = 0.0
74     speed_mhz: int = 0
75     memory_type: str = 'DDR4'
76     form_factor: str = 'DIMM'
77     configured_voltage: Optional[float] = None
78
79
80 class RamTelemetrySample(BaseModel):
81     """Срез телеметрии использования физической и виртуальной
    памяти."""
82     timestamp: str = ''
83     created_at: float = 0.0
84     total_bytes: int = 0
85     total_gb: float = 0.0
86     used_bytes: int = 0
87     used_gb: float = 0.0
88     available_bytes: int = 0
89     available_gb: float = 0.0
90     percent_used: float = 0.0
91     swap_total_gb: float = 0.0
92     swap_used_gb: float = 0.0
93     swap_percent: float = 0.0
94     pool_paged_mb: Optional[float] = None
95     pool_nonpaged_mb: Optional[float] = None
96
97
98 class GpuInventoryInfo(BaseModel):
99     """Паспорт графического ускорителя (GPU Inventory)."""
100     gpu_id: int = 0
101     pci_device_id: str = ''

```

```
102     name: str = ''
103     vendor: str = ''
104     driver_version: str = ''
105     driver_date: str = ''
106     vram_bytes: int = 0
107     vram_gb: float = 0.0
108     pci_bus_id: str = ''
109     bios_version: str = ''
110     cuda_cores: Optional[int] = None
111     directml_supported: bool = True
112
113
114 class GpuTelemetrySample(BaseModel):
115     """Срез телеметрии нагрузки, памяти и температур GPU."""
116     timestamp: str = ''
117     created_at: float = 0.0
118     gpu_id: int = 0
119     name: str = ''
120     load_percent: Optional[float] = None
121     memory_used_mb: Optional[float] = None
122     memory_total_mb: Optional[float] = None
123     memory_percent: Optional[float] = None
124     temperature_gpu_c: Optional[float] = None
125     temperature_memory_c: Optional[float] = None
126     fan_speed_pct: Optional[float] = None
127     power_draw_w: Optional[float] = None
128     clock_graphics_mhz: Optional[float] = None
129     clock_memory_mhz: Optional[float] = None
130
131
132 class NetworkAdapterInventoryInfo(BaseModel):
133     """Паспорт сетевого адаптера / интерфейса (Network Adapter
134     Inventory)."""
135     adapter_id: int = 0
136     adapter_guid: str = ''
137     name: str = ''
138     interface_name: str = ''
139     mac_address: str = ''
```

```

139     adapter_type: str = 'Ethernet'
140     is_physical: bool = True
141     is_wireless: bool = False
142     max_speed_mbps: int = 0
143     driver_name: str = ''
144     driver_version: str = ''
145
146
147 class NetworkAdapterSample(BaseModel):
148     """Срез показателей сетевой активности и ошибок адаптера."
149     """
150     timestamp: str = ''
151     created_at: float = 0.0
152     adapter_name: str = ''
153     bytes_recv_sec: float = 0.0
154     bytes_sent_sec: float = 0.0
155     packets_recv_sec: float = 0.0
156     packets_sent_sec: float = 0.0
157     errors_in_sec: float = 0.0
158     errors_out_sec: float = 0.0
159     link_speed_mbps: int = 0
160     is_connected: bool = True
161
162 class TelemetryProvider(ABC):
163     """Абстрактный базовый класс для всех компонентов системы,
164     предоставляющих телеметрию."""
165
166     @abstractmethod
167     def get_sensors(self) -> List[HardwareSensor]:
168         """Возвращает список сенсоров, предоставляемых данным
169         компонентом."""
170         pass
171
172 class CpuMetrics(BaseModel):
173     """CPU usage and architecture metrics."""
174     model: str = Field(default='', description='CPU model name
175 ')
176     architecture: str = Field(default='x86_64', description='C

```

```

    PU architecture')
173     physical_cores: int = Field(default=1, description='Physical
    core count')
174     logical_cores: int = Field(default=1, description='Logical
    core count (threads)')
175     total_percent: float = Field(default=0.0, description='Overall
    CPU usage percentage')
176     per_core_percent: List[float] = Field(default_factory=list
    , description='Usage percentage per logical core')
177     frequency_mhz: float = Field(default=0.0, description='Current
    CPU frequency in MHz')
178     temperature_celsius: Optional[float] = Field(default=None,
    description='Package temperature in Celsius')
179
180 class MemoryMetrics(BaseModel):
181     """RAM and Swap memory metrics."""
182     total_gb: float = Field(default=0.0, description='Total physical
    RAM in GB')
183     available_gb: float = Field(default=0.0, description='Available
    RAM in GB')
184     used_gb: float = Field(default=0.0, description='Used RAM in
    GB')
185     percent: float = Field(default=0.0, description='Memory utilization
    percentage')
186     swap_total_gb: float = Field(default=0.0, description='Total swap
    space in GB')
187     swap_used_gb: float = Field(default=0.0, description='Used swap
    space in GB')
188     swap_percent: float = Field(default=0.0, description='Swap utilization
    percentage')
189
190 class GpuMetrics(BaseModel):
191     """GPU accelerator telemetry."""
192     name: str = Field(default='Unknown GPU', description='GPU device
    model name')
193     load_percent: Optional[float] = Field(default=None, description='GPU
    core load percentage')
194     memory_total_gb: float = Field(default=0.0, description='Total
    GPU memory in GB')

```

```

total VRAM in GB')
195     memory_used_gb: float = Field(default=0.0, description='Used VRAM in GB')
196     memory_free_gb: float = Field(default=0.0, description='Free VRAM in GB')
197     temperature_celsius: Optional[float] = Field(default=None, description='GPU core temperature in Celsius')
198     has_cuda: bool = Field(default=False, description='CUDA support availability')
199     has_directml: bool = Field(default=False, description='DirectML accelerator availability')
200     engines: Dict[str, float] = Field(default_factory=dict, description='Подсистемы и движки GPU (3D, Decode, Copy, etc.)')
201
202 class DiskPartitionMetrics(BaseModel):
203     """Storage partition metrics."""
204     device: str = Field(default='', description='Partition mount device or drive letter')
205     mountpoint: str = Field(default='', description='Mount point path')
206     fstype: str = Field(default='', description='Filesystem type')
207     total_gb: float = Field(default=0.0, description='Total capacity in GB')
208     used_gb: float = Field(default=0.0, description='Used space in GB')
209     free_gb: float = Field(default=0.0, description='Free space in GB')
210     percent: float = Field(default=0.0, description='Utilization percentage')
211     volume_name: Optional[str] = Field(default=None, description='Метка тома или описание аккаунта')
212     is_virtual: bool = Field(default=False, description='Признак виртуального/облачного тома (Google Drive, OneDrive и др.)')
213     drive_type: str = Field(default='Fixed', description='Тип диска (Fixed, Google Drive, Cloud / Virtual, Removable)')
214

```

```

215 class DiskIoMetrics(BaseModel):
216     """Disk read and write I/O rates."""
217     read_bytes_per_sec: float = Field(default=0.0, description
    ='Read throughput in bytes/s')
218     write_bytes_per_sec: float = Field(default=0.0, descriptio
    n='Write throughput in bytes/s')
219     read_count_per_sec: float = Field(default=0.0, description
    ='Read operations per second')
220     write_count_per_sec: float = Field(default=0.0, descriptio
    n='Write operations per second')
221
222 class NetworkAdapterStatistics(BaseModel):
223     """Накопительная статистика сетевого адаптера (Get-NetAdap
    terStatistics / MIB_IF_ROW2)."""
224     name: str = Field(default='', description='Имя адаптера/ин
    терфейса')
225     received_bytes: int = Field(default=0, description='Всего
    получено байт')
226     sent_bytes: int = Field(default=0, description='Всего отпр
    авлено байт')
227     received_packets: int = Field(default=0, description='Всер
    о получено пакетов')
228     sent_packets: int = Field(default=0, description='Всего от
    правлено пакетов')
229     received_discarded: int = Field(default=0, description='От
    брошено входящих пакетов')
230     received_errors: int = Field(default=0, description='Ошибо
    к входящих пакетов')
231     outbound_discarded: int = Field(default=0, description='От
    брошено исходящих пакетов')
232     outbound_errors: int = Field(default=0, description='Ошибо
    к исходящих пакетов')
233
234 class NetworkPerformanceCounter(BaseModel):
235     """Счётчики производительности интерфейса (Windows Perform
    ance Counters)."""
236     interface_name: str = Field(default='', description='Имя с
    етевого интерфейса в Performance Counters')

```

```
237     bytes_recv_per_sec: float = Field(default=0.0, description
    = 'Скорость получения (байты/сек)')
238     bytes_sent_per_sec: float = Field(default=0.0, description
    = 'Скорость отправки (байты/сек)')
239     bytes_total_per_sec: float = Field(default=0.0, description
    = 'Суммарная скорость (байты/сек)')
240
241 class AppNetworkUsageItem(BaseModel):
242     """Потребление сетевого трафика конкретным приложением/про
    цессом."""
243     process_name: str = Field(default='', description='Имя исп
    олняемого файла процесса (например, chrome.exe)')
244     pid: Optional[int] = Field(default=None, description='Иден
    тификатор процесса (если активен)')
245     rx_bytes: int = Field(default=0, description='Получено бай
    т за период')
246     tx_bytes: int = Field(default=0, description='Отправлено б
    айт за период')
247     total_bytes: int = Field(default=0, description='Всего пер
    еданного трафика в байтах')
248     rx_formatted: str = Field(default='0 B', description='Отфо
    рматированный объем полученных данных (MB/GB)')
249     tx_formatted: str = Field(default='0 B', description='Отфо
    рматированный объем отправленных данных (MB/GB)')
250     share_percent: float = Field(default=0.0, description='Дол
    я трафика от общего объема системы (%)')
251
252 class NetworkUsagePeriodReport(BaseModel):
253     """Агрегированный отчет об использовании сети за выбранный
    интервал времени (10 мин, 1 час, 24 часа)."""
254     period_minutes: int = Field(default=1440, description='Дли
    тельность периода в минутах (например, 1440 = 24 часа)')
255     total_rx_bytes: int = Field(default=0, description='Суммар
    но получено байт за период')
256     total_tx_bytes: int = Field(default=0, description='Суммар
    но отправлено байт за период')
257     total_rx_formatted: str = Field(default='0 B', description
    = 'Человекочитаемый объем входящего трафика')
```



```

258     total_tx_formatted: str = Field(default='0 B', description
    = 'Человекочитаемый объем исходящего трафика')
259     total_formatted: str = Field(default='0 B', description='С
    уммарный объем трафика (RX + TX)')
260     adapter_stats: List[NetworkAdapterStatistics] = Field(defa
    ult_factory=list, description='Статистика по адаптерам')
261     performance_counters: List[NetworkPerformanceCounter] = Fi
    eld(default_factory=list, description='Текущие Performance Cou
    nters')
262     top_apps: List[AppNetworkUsageItem] = Field(default_factor
    y=list, description='Детализация по приложениям за период')
263     summary_text: str = Field(default='', description='Сводный
    текстовый отчёт для LLM/ai-diagnostics')
264
265 class NetworkInterfaceMetrics(BaseModel):
266     """Network adapter telemetry."""
267     name: str = Field(default='', description='Adapter interfa
    ce name')
268     is_up: bool = Field(default=True, description='Interface l
    ink status')
269     speed_mbps: int = Field(default=0, description='Link speed
    in Mbps')
270     bytes_sent_per_sec: float = Field(default=0.0, description
    = 'Upload rate in bytes/s')
271     bytes_recv_per_sec: float = Field(default=0.0, description
    = 'Download rate in bytes/s')
272     ip_addresses: List[str] = Field(default_factory=list, desc
    ription='Assigned IP addresses')
273     statistics: Optional[NetworkAdapterStatistics] = Field(def
    ault=None, description='Накопительная статистика адаптера')
274
275 class ProcessMetrics(BaseModel):
276     """Метрики процесса с данными об идентичности и происхожде
    нии (Process Identity & Provenance)."""
277     pid: int = Field(..., description='Process identifier')
278     name: str = Field(..., description='Process binary name')
279     status: str = Field(default='running', description='Proces
    s execution status')

```

```
280     cpu_percent: float = Field(default=0.0, description='CPU u
| sage percentage')
281     memory_mb: float = Field(default=0.0, description='Residen
| t memory in Megabytes')
282     memory_percent: float = Field(default=0.0, description='Me
| mory usage percentage')
283     num_threads: int = Field(default=1, description='Number of
| active threads')
284     num_handles: int = Field(default=0, description='Количеств
| о дескрипторов (handles/fds)')
285     username: Optional[str] = Field(default=None, description=
| 'Owner username')
286     cmdline: Optional[str] = Field(default=None, description='
| Command line invocation')
287     read_bytes_sec: float = Field(default=0.0, description='Di
| sk read rate in bytes/sec')
288     write_bytes_sec: float = Field(default=0.0, description='D
| isk write rate in bytes/sec')
289     # Поля безопасности и токена
290     integrity_level: Optional[str] = Field(default=None, descr
| iption='Уровень целостности процесса (Low, Medium, High, Syste
| m)')
291     elevation: Optional[bool] = Field(default=None, descriptio
| n='Статус повышения привилегий UAC')
292     sid: Optional[str] = Field(default=None, description='Иден
| тификатор безопасности пользователя (User SID)')
293     session_id: Optional[int] = Field(default=None, descriptio
| n='Идентификатор сессии Windows (0=Службы, 1=Интерактивная)')
294     # Поля происхождения и родословной (Process Provenance)
295     ppid: Optional[int] = Field(default=None, description='Иде
| нтификатор родительского процесса (Parent PID)')
296     parent_name: Optional[str] = Field(default=None, descripti
| on='Имя родительского процесса')
297     executable_path: Optional[str] = Field(default=None, descr
| iption='Полный путь к исполняемому файлу процесса')
298     creation_time: Optional[str] = Field(default=None, descrip
| tion='Время создания процесса (ISO 8601 UTC)')
299     process_guid: Optional[str] = Field(default=None, descript
```

```

        ion='Уникальный GUID экземпляра процесса (PID + CreationTime)'
    )
300     ancestor_chain: Optional[str] = Field(default=None, description='Цепочка предков запуска (напр. explorer -> pwsh -> python)')
301     launch_reason: Optional[str] = Field(default=None, description='Определенный вектор/причина запуска (Служба, Задача, Терминал, Проводник)')
302
303
304 class ProcessTokenInfo(BaseModel):
305     """Информация о токене процесса для сбора security-данных.
306     """
307     pid: int = Field(..., description='Идентификатор процесса')
308     name: str = Field(..., description='Имя процесса')
309     elevation: Optional[bool] = Field(default=None, description='Признак повышения привилегий (UAC)')
310     integrity_level: Optional[str] = Field(default=None, description='Уровень целостности процесса')
311     privileges: List[str] = Field(default_factory=list, description='Список привилегий токена')
312     sid: Optional[str] = Field(default=None, description='Идентификатор безопасности (SID)')
313     session_id: Optional[int] = Field(default=None, description='Идентификатор сессии Windows')
314
315 class ProcessProvenanceInfo(BaseModel):
316     """Детальный паспорт идентичности, происхождения и цепочки вызовов процесса (Process Provenance)."""
317     pid: int = Field(..., description='Идентификатор процесса (PID)')
318     ppid: Optional[int] = Field(default=None, description='Идентификатор родительского процесса (PPID)')
319     name: str = Field(..., description='Имя процесса (например, python.exe)')
320     executable_path: Optional[str] = Field(default=None, descr

```

```
    iption='Полный путь к бинарному исполняемому файлу')
321     command_line: Optional[str] = Field(default=None, description='Полная командная строка запуска с аргументами')
322     user: Optional[str] = Field(default=None, description='Имя учетной записи (DOMAIN\\User)')
323     sid: Optional[str] = Field(default=None, description='Идентификатор безопасности пользователя (SID)')
324     session_id: Optional[int] = Field(default=None, description='Идентификатор сессии Windows (Session 0=Службы, Session 1+= Пользователь)')
325     integrity_level: Optional[str] = Field(default=None, description='Уровень целостности токена (Untrusted, Low, Medium, High, System)')
326     elevation: Optional[bool] = Field(default=None, description='Признак повышенных привилегий администратора (Elevated)')
327     creation_time: Optional[str] = Field(default=None, description='Время создания процесса в формате ISO 8601 UTC')
328     parent_start_time: Optional[str] = Field(default=None, description='Время создания родительского процесса')
329     process_guid: str = Field(default='', description='Уникальный GUID экземпляра процесса (исключает коллизии PID reuse)')
330     parent_guid: Optional[str] = Field(default=None, description='GUID родительского экземпляра процесса')
331     parent_name: Optional[str] = Field(default=None, description='Имя родительского процесса (например, WindowsTerminal.exe)')
332     parent_cmdline: Optional[str] = Field(default=None, description='Командная строка родителя')
333     ancestor_chain: List[str] = Field(default_factory=list, description='Список узлов в цепочке предков (от корня к процессу)')
334     ancestor_chain_str: str = Field(default='', description='Строковое представление цепочки предков (explorer → pwsh → python)')
335     who_runs_it: str = Field(default='', description='Кем исполняется (User / SID / Token / Session)')
336     who_created_it: str = Field(default='', description='Кто создал (Родительский процесс / Дерево предков)')
```

```
337     why_was_it_created: str = Field(default='', description='П
ричина/вектор создания (Служба, Задача, Интерактивно, IDE, CLI
, Автозапуск)')
338
339
340 class ProcessLifecycleEvent(BaseModel):
341     """Событие жизненного цикла процесса (создание, завершение
, форензика)."""
342     event_id: str = Field(default='', description='Уникальный
идентификатор события')
343     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время события UTC')
344     event_type: str = Field(..., description='Тип события: Pro
cessCreated, ProcessTerminated, Snapshot')
345     pid: int = Field(..., description='Идентификатор процесса'
)
346     ppid: Optional[int] = Field(default=None, description='Иде
нтификатор родительского процесса')
347     process_guid: str = Field(default='', description='Уникаль
ный GUID процесса')
348     parent_guid: Optional[str] = Field(default=None, descripti
on='GUID родительского процесса')
349     name: str = Field(..., description='Имя процесса')
350     executable_path: Optional[str] = Field(default=None, descr
iption='Путь к исполняемому файлу')
351     command_line: Optional[str] = Field(default=None, descript
ion='Аргументы командной строки')
352     user: Optional[str] = Field(default=None, description='Уче
тная запись пользователя')
353     sid: Optional[str] = Field(default=None, description='Иден
тификатор безопасности (SID)')
354     session_id: Optional[int] = Field(default=None, descriptio
n='Идентификатор сессии')
355     integrity_level: Optional[str] = Field(default=None, descr
iption='Уровень целостности токена')
356     elevation: Optional[bool] = Field(default=None, descriptio
n='Статус повышения привилегий')
357     parent_name: Optional[str] = Field(default=None, descripti
```

```

on='Имя родительского процесса')
358     parent_cmdline: Optional[str] = Field(default=None, description='Командная строка родителя')
359     ancestor_chain: Optional[str] = Field(default=None, description='Строка цепочки предков')
360     launch_reason: Optional[str] = Field(default=None, description='Определенный вектор запуска')
361     source: str = Field(default='system', description='Источник события: w64, etw, psutil, wevtapi')
362     raw_json: Optional[str] = Field(default=None, description='Сырые данные события в формате JSON')
363
364
365 class ProcessProvenanceReport(BaseModel):
366     """Сводный аналитический отчет Process Provenance для AI Diagnostics."""
367     timestamp: str = Field(default_factory=lambda: datetime.now(timezone.utc).isoformat(), description='Время формирования отчета')
368     total_processes: int = Field(default=0, description='Общее число активных процессов')
369     active_provenance: List[ProcessProvenanceInfo] = Field(default_factory=list, description='Список паспортов происхождения процессов')
370     recent_events: List[ProcessLifecycleEvent] = Field(default_factory=list, description='Недавние события создания/завершения процессов')
371     summary: str = Field(default='', description='Аналитическая сводка для LLM/ai-diagnostics')
372 class BatteryMetrics(BaseModel):
373     """Battery and power state telemetry."""
374     has_battery: bool = Field(default=False, description='Whether host has battery power')
375     percent: Optional[float] = Field(default=None, description='Battery charge percentage')
376     power_plugged: Optional[bool] = Field(default=None, description='Whether AC power is connected')
377     secs_left: Optional[int] = Field(default=None, description=

```

```

    = 'Seconds of battery remaining')
378     power_profile: str = Field(default='Balanced', description
    = 'Active Windows power scheme')
379
380
381
382
383 class AppDiskUsageItem(BaseModel):
384     """Потребление дискового I/O конкретным приложением/процес
    сом."""
385     process_name: str = Field(default='', description='Имя исп
    олняемого файла процесса (например, chrome.exe)')
386     pid: Optional[int] = Field(default=None, description='Иден
    тификатор процесса (если активен)')
387     read_bytes: int = Field(default=0, description='Прочитано
    байт за период')
388     write_bytes: int = Field(default=0, description='Записано
    байт за период')
389     total_bytes: int = Field(default=0, description='Всего дис
    кового I/O в байтах')
390     read_formatted: str = Field(default='0 B', description='От
    форматированный объем прочитанных данных')
391     write_formatted: str = Field(default='0 B', description='О
    тформатированный объем записанных данных')
392     share_percent: float = Field(default=0.0, description='Дол
    я дискового I/O от общего объема системы (%)')
393
394 class DiskUsagePeriodReport(BaseModel):
395     """Агрегированный отчет об использовании дискового I/O и S
    MART накопителей за период."""
396     period_minutes: int = Field(default=1440, description='Дли
    тельность периода в минутах (например, 1440 = 24 часа)')
397     total_read_bytes: int = Field(default=0, description='Сумм
    арно прочитано байт за период')
398     total_write_bytes: int = Field(default=0, description='Сум
    марно записано байт за период')
399     total_read_formatted: str = Field(default='0 B', descripti
    on='Человекочитаемый объем прочитанных данных')

```

```
400     total_write_formatted: str = Field(default='0 B', description='Человекочитаемый объем записанных данных')
401     total_formatted: str = Field(default='0 B', description='Суммарный объем I/O (Read + Write)')
402     top_writers: List[AppDiskUsageItem] = Field(default_factory=list, description='Топ процессов по объему записи за период')
403     top_readers: List[AppDiskUsageItem] = Field(default_factory=list, description='Топ процессов по объему чтения за период')
404     physical_disks: List[PhysicalDiskHealth] = Field(default_factory=list, description='Состояние и SMART-метрики физических дисков')
405     smart_delta_write_bytes: Optional[int] = Field(default=None, description='Изменение объема записи в SMART за период (байт)')
406     smart_delta_write_formatted: Optional[str] = Field(default=None, description='Отформатированное изменение записи SMART')
407     io_difference_bytes: Optional[int] = Field(default=None, description='Разница между SMART Delta и суммарным I/O процессом (байт)')
408     summary_text: str = Field(default='', description='Сводный текстовый отчет для LLM/ai-diagnostics')
409
410 class PhysicalDiskHealth(BaseModel):
411     """Physical drive SMART and health telemetry."""
412     device_id: str = Field(default='', description='Drive identifier or disk index')
413     model: str = Field(default='Physical Drive', description='Drive model name')
414     serial_number: Optional[str] = Field(default=None, description='Серийный номер накопителя')
415     media_type: str = Field(default='SSD', description='Media type (NVMe, SSD, HDD)')
416     size_gb: float = Field(default=0.0, description='Drive total capacity in GB')
417     health_status: str = Field(default='Healthy', description='Drive health status (Healthy, Warning, Unhealthy)')
```



```

418     operational_status: str = Field(default='OK', description=
| 'Operational status')
419     temperature_celsius: Optional[float] = Field(default=None,
| description='Drive temperature if available')
420     interface_type: Optional[str] = Field(default=None, descri
| ption='Drive interface or bus type (NVMe, SATA, USB, etc.)')
421     lifetime_read_bytes: Optional[int] = Field(default=None, d
| escription='Всего прочитано накопителем за время жизни (байт)'
| )
422     lifetime_write_bytes: Optional[int] = Field(default=None,
| description='Всего записано накопителем за время жизни (байт)'
| )
423     lifetime_read_tb: Optional[float] = Field(default=None, de
| scription='Всего прочитано накопителем за время жизни (TB)')
424     lifetime_write_tb: Optional[float] = Field(default=None, d
| escription='Всего записано накопителем за время жизни (TB)')
425     power_on_hours: Optional[int] = Field(default=None, descri
| ption='Время наработки диска в часах')
426     wear_percentage: Optional[float] = Field(default=None, des
| cription='Процент износа накопителя (0-100%)')
427
428 class RamStickInfo(BaseModel):
429     """Physical RAM stick SPD details."""
430     bank_label: str = Field(default='DIMM', description='Memor
| y slot label')
431     capacity_gb: float = Field(default=0.0, description='Modul
| e capacity in GB')
432     speed_mhz: int = Field(default=0, description='Configured
| clock speed in MHz/MTs')
433     manufacturer: str = Field(default='Generic', description='
| Memory manufacturer')
434     part_number: str = Field(default='', description='Module p
| art number')
435     memory_type: str = Field(default='DDR4/DDR5', description=
| 'Memory technology type')
436
437 class NetworkPortMetrics(BaseModel):
438     """Active listening TCP/UDP port item."""

```

```
439     port: int = Field(..., description='Listening port number'
440 )
441     protocol: str = Field(default='TCP', description='Protocol
: TCP / UDP')
442     address: str = Field(default='0.0.0.0', description='Bind
IP address')
443     pid: Optional[int] = Field(default=None, description='Owni
ng process PID')
444     process_name: Optional[str] = Field(default=None, descript
ion='Owning process executable name')
445
446 class ProcessNetworkActivity(BaseModel):
447     """Сетевая активность процесса (интернет-соединения, отпра
вка и прием трафика)."""
448     pid: int = Field(default=0, description='Process ID')
449     name: str = Field(default='', description='Process executab
le name')
450     user: str = Field(default='', description='Process owner u
sername')
451     local_address: str = Field(default='', description='Local
IP:Port')
452     remote_address: str = Field(default='', description='Remot
e IP:Port')
453     remote_host: Optional[str] = Field(default=None, descripti
on='Resolved remote hostname or service name')
454     protocol: str = Field(default='TCP', description='Protocol
: TCP / UDP')
455     status: str = Field(default='ESTABLISHED', description='Co
nnection state (ESTABLISHED, LISTEN, etc.)')
456     is_internet: bool = Field(default=True, description='Wheth
er remote address is external Internet vs local loopback')
457     service_type: str = Field(default='Web/HTTPS', description
='Recognized protocol service: HTTPS, HTTP, DNS, Cloud Sync, e
tc.')
458     sent_kb: float = Field(default=0.0, description='Всего отп
равлено/записано процессом (КБ)')
459     recv_kb: float = Field(default=0.0, description='Всего пол
учено/скачано/прочитано процессом (КБ)')
```

```

459     delta_sent_kb: float = Field(default=0.0, description='Объ
    ем отправленных данных за последний измеряемый период (КБ)')
460     delta_recv_kb: float = Field(default=0.0, description='Объ
    ем скачанных/полученных данных за последний измеряемый период
    (КБ)')
461     sent_rate_kbs: float = Field(default=0.0, description='Ско
    рость отдачи/отправки данных (КБ/с)')
462     recv_rate_kbs: float = Field(default=0.0, description='Ско
    рость скачивания/приема данных (КБ/с)')
463     sent_summary: str = Field(default='', description='Человек
    опонятное описание отправляемого (что шлет)')
464     recv_summary: str = Field(default='', description='Человек
    опонятное описание принимаемого (что принимает)')
465
466 class SystemHealthAlerts(BaseModel):
467     """System reliability and event log alerts."""
468     reboot_pending: bool = Field(default=False, description='W
    hether a system reboot is pending')
469     critical_events_count: int = Field(default=0, description=
    'Critical event logs in last 24h')
470     latest_alert: str = Field(default='Система стабильна', des
    cription='Summary of latest health event')
471
472 class HardwareSensor(BaseModel):
473     """Hardware sensor reading (AIDA64 style) with provider tr
    acking for deduplication."""
474     model_config = ConfigDict(extra='allow')
475
476     sensor_id: str = Field(..., description='Unique sensor ide
    ntifier')
477     name: str = Field(..., description='Human readable sensor
    name')
478     category: str = Field(default='temperature', description='
    Category: temperature, fan, voltage, power')
479     value: float = Field(..., description='Current numeric sen
    sor reading')
480     unit: str = Field(default='°C', description='Measurement u
    nit')

```

```

481     min_value: Optional[float] = Field(default=None, description=
    on='Recorded minimum')
482     max_value: Optional[float] = Field(default=None, description=
    on='Recorded maximum')
483     provider: Optional[Any] = Field(default=None, description=
    'Provider name or ID for deduplication (SensorProvider enum or
    int/str)')
484
485 class HardwareNode(BaseModel):
486     """AIDA64-like hardware component tree item."""
487     category: str = Field(..., description='Component category
    (Motherboard, CPU, GPU, Memory, Storage)')
488     name: str = Field(..., description='Device or component na
    me')
489     properties: Dict[str, Any] = Field(default_factory=dict, d
    escription='Detailed hardware properties')
490     children: List[HardwareNode] = Field(default_factory=list,
    description='Sub-components or devices')
491
492 class MonitorInfo(BaseModel):
493     """Connected display monitor details."""
494     device: str = Field(default='', description='Display devic
    e name (e.g. \\\\.\\DISPLAY1)')
495     name: str = Field(default='Monitor', description='Friendly
    monitor name or model')
496     adapter: str = Field(default='', description='Display adap
    ter name')
497     width: int = Field(default=1920, description='Display reso
    lution width')
498     height: int = Field(default=1080, description='Display res
    olution height')
499     frequency_hz: int = Field(default=60, description='Refresh
    rate in Hz')
500     bits_per_pixel: int = Field(default=32, description='Color
    bit depth')
501     is_primary: bool = Field(default=False, description='Wheth
    er this is the primary display')
502

```

```

503 class WindowsUpdateInfo(BaseModel):
504     """Windows Update status and recent hotfixes."""
505     status: str = Field(default='Up to date', description='Overall update status')
506     installed_kb_count: int = Field(default=0, description='Total installed KBs')
507     recent_hotfixes: List[str] = Field(default_factory=list, description='Recent KB identifiers')
508     latest_installed_on: Optional[str] = Field(default=None, description='Date of most recent update')
509
510 class OfficeSuiteInfo(BaseModel):
511     """Microsoft Office and productivity suite detection telemetry."""
512     installed: bool = Field(default=False, description='Whether an Office suite is installed')
513     product_name: Optional[str] = Field(default=None, description='Product suite name (e.g. Microsoft 365, Office 2021)')
514     version: Optional[str] = Field(default=None, description='Office suite release version string')
515     publisher: Optional[str] = Field(default=None, description='Software vendor/publisher name')
516     status: str = Field(default='Не установлен', description='Human-readable office status summary')
517
518 class CloudStorageInfo(BaseModel):
519     """Cloud drive and synchronization storage telemetry (OneDrive)."""
520     installed: bool = Field(default=False, description='Whether cloud storage drive is detected and active')
521     name: str = Field(default='OneDrive', description='Cloud storage service name')
522     path: Optional[str] = Field(default=None, description='Local synchronized root folder path')
523     total_gb: Optional[float] = Field(default=None, description='Total storage volume capacity in GB')
524     used_gb: Optional[float] = Field(default=None, description='Used space in GB')

```

```
525     free_gb: Optional[float] = Field(default=None, description
    ='Free available space in GB')
526     percent_used: Optional[float] = Field(default=None, descri
    ption='Storage disk utilization percentage')
527     status: str = Field(default='Не настроено', description='H
    uman-readable cloud storage status')
528
529 class SystemSnapshot(BaseModel):
530     """Complete system and hardware telemetry snapshot."""
531     timestamp: str = Field(default_factory=lambda: datetime.no
    w(timezone.utc).isoformat(), description='ISO 8601 UTC timesta
    mp of snapshot')
532     hostname: str = Field(default='', description='Machine hos
    t name')
533     workgroup: str = Field(default='WORKGROUP', description='M
    achine workgroup or domain')
534     username: str = Field(default='', description='Current log
    ged in username')
535     os_name: str = Field(default='Windows', description='Opera
    ting system name and version')
536     os_build: str = Field(default='', description='Operating s
    ystem build number')
537     system_language: str = Field(default='', description='Syst
    em UI language')
538     os_install_language: str = Field(default='', description='
    Original OS install language')
539     user_locale: str = Field(default='', description='User loc
    ale')
540     system_locale: str = Field(default='', description='System
    locale')
541     timezone: str = Field(default='', description='System time
    zone')
542     codepage: str = Field(default='', description='Active syst
    em code pages')
543     input_languages: List[str] = Field(default_factory=list, d
    escription='Installed keyboard input layouts')
544     os_install_date: str = Field(default='', description='Oper
    ating system installation date')
```

```
545     uptime_seconds: float = Field(default=0.0, description='System uptime in seconds')
546     cpu: CpuMetrics = Field(default_factory=CpuMetrics, description='CPU metrics')
547     memory: MemoryMetrics = Field(default_factory=MemoryMetrics, description='RAM and Swap metrics')
548     ram_sticks: List[RamStickInfo] = Field(default_factory=list, description='Installed physical RAM modules')
549     gpus: List[GpuMetrics] = Field(default_factory=list, description='Detected GPU accelerators')
550     monitors: List[MonitorInfo] = Field(default_factory=list, description='Connected display monitors')
551     updates: WindowsUpdateInfo = Field(default_factory=WindowsUpdateInfo, description='Windows Update information')
552     office: OfficeSuiteInfo = Field(default_factory=OfficeSuiteInfo, description='Office suite telemetry')
553     onedrive: CloudStorageInfo = Field(default_factory=CloudStorageInfo, description='OneDrive cloud storage telemetry')
554     disks: List[DiskPartitionMetrics] = Field(default_factory=list, description='Disk partitions')
555     physical_disks: List[PhysicalDiskHealth] = Field(default_factory=list, description='Physical storage disks and SMART health')
556     disk_io: DiskIoMetrics = Field(default_factory=DiskIoMetrics, description='Aggregate disk I/O rates')
557     network: List[NetworkInterfaceMetrics] = Field(default_factory=list, description='Network interfaces')
558     listening_ports: List[NetworkPortMetrics] = Field(default_factory=list, description='Active listening ports and sockets')
559     network_activity: List[ProcessNetworkActivity] = Field(default_factory=list, description='Сетевая активность программ (соединения и трафик)')
560     battery: BatteryMetrics = Field(default_factory=BatteryMetrics, description='Battery and power state')
561     alerts: SystemHealthAlerts = Field(default_factory=SystemHealthAlerts, description='System health and reliability alerts')
```

```

562     sensors: List[HardwareSensor] = Field(default_factory=list
, description='Hardware sensor readings')
563     top_processes: List[ProcessMetrics] = Field(default_factor
y=list, description='Top active processes')
564     hardware_audit: Dict[str, Any] = Field(default_factory=dic
t, description='Full hardware audit data (manufacturer, serial
, etc.)')
565
566 class WindowsFeaturesSnapshot(BaseModel):
567     """Снимок состояния Windows Optional Features при старте т
елеметрии."""
568     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='ISO-8601 метка време
ни')
569     features: List[Dict[str, Any]] = Field(default_factory=lis
t, description='Список компонентов Windows Optional Features')
570
571
572 class AnomalyItem(BaseModel):
573     """Specific detected anomaly or performance bottleneck."""
574     subsystem: str = Field(..., description='Subsystem (CPU, R
AM, GPU, Disk, Network, Process)')
575     severity: str = Field(default='warning', description='Seve
rity level: info, warning, critical')
576     title: str = Field(..., description='Brief anomaly summary
')
577     description: str = Field(..., description='Detailed diagno
stic description')
578
579 class SystemDiagnosticReport(BaseModel):
580     """AI-powered diagnostic audit report."""
581     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Audit timestamp')
582     health_score: int = Field(default=100, description='System
health score (0-100)')
583     summary: str = Field(..., description='High-level health a
nd performance summary')
584     anomalies: List[AnomalyItem] = Field(default_factory=list,

```



```

        description='List of detected anomalies')
585     recommendations: List[str] = Field(default_factory=list, d
description='Actionable optimization suggestions')
586     ai_model_used: str = Field(default='heuristic', descriptio
n='AI Model identifier or heuristic engine')
587     system_instruction: Optional[str] = Field(default=None, de
scription='System instruction used for AI model')
588     generated_prompt: Optional[str] = Field(default=None, desc
ription='Exact prompt sent to AI model')
589     raw_response: Optional[str] = Field(default=None, descript
ion='Raw response text from AI provider')
590     error: Optional[str] = Field(default=None, description='Er
ror details if inference failed')
591     stages: List[Dict[str, Any]] = Field(default_factory=list,
description='Step-by-step audit stages')
592
593 class DriverInfo(BaseModel):
594     """Сведения об установленном драйвере устройства и его акт
уальности."""
595     name: str = Field(default='', description='Имя драйвера ил
и сервиса')
596     driver_version: str = Field(default='', description='Верси
я установленного драйвера')
597     driver_date: Optional[str] = Field(default=None, descripti
on='Дата релиза драйвера (ISO или YYYY-MM-DD)')
598     provider: str = Field(default='Unknown', description='Пост
авщик драйвера (NVIDIA, Intel, Microsoft и др.)')
599     inf_name: Optional[str] = Field(default=None, description=
'Имя INF-файла драйвера (oem*.inf)')
600     is_signed: bool = Field(default=True, description='Наличие
цифровой подписи WHQL')
601     is_inbox: bool = Field(default=False, description='Являетс
я ли драйвер стандартным (inbox) от Microsoft')
602     age_days: Optional[int] = Field(default=None, description=
'Возраст драйвера в днях')
603     currency_status: str = Field(default='Актуален', descripti
on='Статус актуальности: Актуален, Устарел, Базовый драйвер ОС
')

```

```

604
605 class HardwareDeviceAudit(BaseModel):
606     """Единица аудита аппаратного устройства с драйверами, дат
        ами и сенсорами."""
607     device_id: str = Field(..., description='Уникальный иденти
        фикатор инстанса устройства (PnP Instance ID)')
608     name: str = Field(..., description='Понятное наименование
        устройства')
609     device_class: str = Field(default='Device', description='К
        ласс устройства (Display, Processor, Net, DiskDrive и др.)')
610     manufacturer: str = Field(default='Unknown', description='
        Производитель оборудования')
611     install_date: Optional[str] = Field(default=None, descript
        ion='Дата первой/текущей установки устройства в системе')
612     status: str = Field(default='OK', description='Рабочий ста
        тус устройства (OK, Problem, Error, Disabled)')
613     problem_code: int = Field(default=0, description='Код ошиб
        ки PnP (0 если исправно, 10, 43, 28 при сбое)')
614     driver: Optional[DriverInfo] = Field(default=None, descrip
        tion='Сведения об используемом драйвере')
615     sensors: List[HardwareSensor] = Field(default_factory=list
        , description='Сенсоры, привязанные к данному оборудованию')
616     properties: Dict[str, Any] = Field(default_factory=dict, d
        escription='Дополнительные аппаратные параметры')
617
618 class HardwareChangeItem(BaseModel):
619     """Зафиксированное изменение в аппаратной конфигурации или
        драйверах."""
620     change_type: str = Field(..., description='Тип изменения:
        added, removed, driver_updated, status_changed, hardware_alter
        ed')
621     device_id: str = Field(..., description='Идентификатор зат
        ронутного устройства')
622     device_name: str = Field(..., description='Наименование ус
        тройства')
623     description: str = Field(..., description='Подробное описа
        ние изменения на русском языке')
624     previous_value: Optional[Dict[str, Any]] = Field(default=N

```

```

one, description='Предыдущее состояние параметра')
625     current_value: Optional[Dict[str, Any]] = Field(default=None, description='Новое состояние параметра')
626     timestamp: str = Field(default_factory=lambda: datetime.now(timezone.utc).isoformat(), description='Временная метка обнаружения изменения')
627
628 class HardwareAuditReport(BaseModel):
629     """Полный отчет аудита аппаратного обеспечения с привязкой сенсоров и драйверов."""
630     timestamp: str = Field(default_factory=lambda: datetime.now(timezone.utc).isoformat(), description='Время проведения аудита (UTC)')
631     devices_count: int = Field(default=0, description='Общее количество обнаруженных устройств')
632     problem_devices_count: int = Field(default=0, description='Количество устройств с ошибками PnP')
633     outdated_drivers_count: int = Field(default=0, description='Количество устройств с устаревшими драйверами')
634     devices: List[HardwareDeviceAudit] = Field(default_factory=list, description='Список проаудированных устройств')
635     summary: str = Field(default='', description='Сводка состояния аппаратной части')
636     changes_since_last_archive: List[HardwareChangeItem] = Field(default_factory=list, description='Изменения, обнаруженные по сравнению с последним сохраненным архивом')
637
638 class HardwareArchiveEntry(BaseModel):
639     """Запись в архиве истории оборудования."""
640     archive_id: str = Field(..., description='Уникальный идентификатор архивного снимка')
641     timestamp: str = Field(..., description='Время создания архива')
642     devices_count: int = Field(default=0, description='Количество устройств в архиве')
643     changes_count: int = Field(default=0, description='Количество изменений, зафиксированных в этом архиве')
644     report: HardwareAuditReport = Field(..., description='Полн

```

```

        ый отчет аудита оборудования')
645
646 class ProcessLeakItem(BaseModel):
647     """Метрики скрытой диагностики и утечек отдельного процесс
        а."""
648     pid: int = Field(..., description='Идентификатор процесса'
        )
649     name: str = Field(..., description='Имя процесса')
650     status: str = Field(default='running', description='Статус
        процесса')
651     handles_count: int = Field(default=0, description='Количес
        тво открытых дескрипторов (Handles)')
652     gdi_objects: int = Field(default=0, description='Количеств
        о GDI-объектов (шрифты, битмапы, кисти, лимит 10 000)')
653     user_objects: int = Field(default=0, description='Количест
        во USER-объектов (окна, меню, курсоры)')
654     page_faults_total: int = Field(default=0, description='Общ
        ее число страничных ошибок (Page Faults)')
655     peak_working_set_mb: float = Field(default=0.0, descriptio
        n='Пиковый рабочий набор памяти в МБ')
656     memory_mb: float = Field(default=0.0, description='Текущая
        память (RSS) в МБ')
657     threads_count: int = Field(default=1, description='Количес
        тво потоков процесса')
658     cpu_percent: float = Field(default=0.0, description='Нагру
        зка на CPU %')
659     leak_risk_score: str = Field(default='normal', description
        ='Уровень риска утечки: normal, warning, critical')
660     leak_risk_reasons: List[str] = Field(default_factory=list,
        description='Список признаков аномального поведения')
661
662 class ProcessLeakDiagnosticsReport(BaseModel):
663     """Сводный отчет скрытой диагностики процессов, утечек дес
        крипторов и ресурсов UI."""
664     timestamp: str = Field(default_factory=lambda: datetime.no
        w(timezone.utc).isoformat(), description='Время сбора среза')
665     total_processes: int = Field(default=0, description='Всего
        проанализировано процессов')

```

```
666     suspicious_count: int = Field(default=0, description='Коли  
чество процессов с признаками утечки')  
667     top_handle_hogs: List[ProcessLeakItem] = Field(default_fac  
tory=list, description='Топ процессов по открытым дескрипторам  
' )  
668     top_gdi_hogs: List[ProcessLeakItem] = Field(default_factor  
y=list, description='Топ процессов по GDI объектам')  
669     top_page_fault_hogs: List[ProcessLeakItem] = Field(default  
_factory=list, description='Топ процессов по обращениям к файл  
у подкачки')  
670     all_processes: List[ProcessLeakItem] = Field(default_facto  
ry=list, description='Полный список проанализированных процесс  
ов')  
671  
672 class ForensicsActivityReport(BaseModel):  
673     """Поведенческая телеметрия и форензика активности пользов  
ателя."""  
674     timestamp: str = Field(default_factory=lambda: datetime.no  
w(timezone.utc).isoformat(), description='Время фиксации')  
675     foreground_window: Dict[str, Any] = Field(default_factory=  
dict, description='Активное окно в фокусе (заголовок, процесс,  
PID, время)')  
676     user_idle_seconds: float = Field(default=0.0, description=  
'Время бездействия пользователя (без ввода с клавиатуры/мыши)  
' )  
677     camera_active_apps: List[Dict[str, Any]] = Field(default_f  
actory=list, description='Приложения, использующие или недавно  
вызывавшие камеру')  
678     microphone_active_apps: List[Dict[str, Any]] = Field(defau  
lt_factory=list, description='Приложения, использующие микрофо  
н')  
679     userassist_top_apps: List[Dict[str, Any]] = Field(default_  
factory=list, description='История запусков из реестра UserAss  
ist (счетчик и фокусное время)')  
680  
681 class KernelThrottlingReport(BaseModel):  
682     """Качество работы ядра, прерываний и аппаратного троттлин  
га."""
```

```

683     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время среза')
684     dpc_latency_pct: float = Field(default=0.0, description='П
роцент времени ЦП в DPC (отложенных вызовах процедур)')
685     interrupt_latency_pct: float = Field(default=0.0, descript
ion='Процент времени ЦП в аппаратных прерываниях (ISR)')
686     dpc_status: str = Field(default='optimal', description='Ст
атус латентности прерываний: optimal, elevated, severe')
687     thermal_throttling_detected: bool = Field(default=False, d
escription='Обнаружен термический троттинг ядер (PROCHOT)')
688     power_limit_throttling_detected: bool = Field(default=Fals
e, description='Обнаружено ограничение по питанию (PL1/PL2 Lim
it)')
689     system_uptime_seconds: float = Field(default=0.0, descript
ion='Время непрерывной работы системы (Uptime)')
690     uptime_formatted: str = Field(default='0 дн 0 ч', descript
ion='Отформатированное время Uptime')
691     last_bsod_crashes: List[Dict[str, Any]] = Field(default_fa
ctory=list, description='Последние зафиксированные падения сис
темы (BugCheck / Minidump)')
692     gpu_pcie_link: Dict[str, Any] = Field(default_factory=dict
, description='Режим шины PCIe для видеокарты (Link Speed / Wi
dth)')
693
694 class StorageBatteryWearReport(BaseModel):
695     """Телеметрия износа накопителей SSD/NVMe и батареи питани
я."""
696     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время формирования')
697     disks_wear: List[Dict[str, Any]] = Field(default_factory=l
ist, description='Метрики износа SSD, TBW и здоровье накопител
ей')
698     battery_wear: Dict[str, Any] = Field(default_factory=dict,
description='Глубокая телеметрия износа аккумулятора (Design
vs Full Charge)')
699
700 class PeripheralsNetworkReport(BaseModel):
701     """Телеметрия сети (Wi-Fi, интерфейсы) и периферийных устр

```

```

ойств (USB, Audio)."""
702     timestamp: str = Field(default_factory=lambda: datetime.now(
timezone.utc).isoformat(), description='Время сбора')
703     usb_devices: List[Dict[str, Any]] = Field(default_factory=
list, description='Подключенные USB-устройства и контроллеры')
704     wifi_telemetry: Dict[str, Any] = Field(default_factory=dic
t, description='Параметры текущей беспроводной сети Wi-Fi (SSI
D, BSSID, RSSI, канал)')
705     audio_endpoints: List[Dict[str, Any]] = Field(default_fact
ory=list, description='Аудиоустройства и активные конечные точ
ки')
706
707 class SystemCoreMetrics(BaseModel):
708     """Быстрые базовые метрики хоста (CPU, RAM, GPU, Disk I/O,
Battery)."""
709     cpu: CpuMetrics
710     memory: MemoryMetrics
711     gpus: List[GpuMetrics] = Field(default_factory=list, descr
iption='Метрики графических ускорителей')
712     disk_io: DiskIoMetrics
713     battery: BatteryMetrics
714     uptime_seconds: float = Field(default=0.0, description='Вр
емя непрерывной работы в секундах')
715
716 class SystemHardwareQuick(BaseModel):
717     """Сводка состояния физических компонентов (диски, слоты п
амяти, открытые сокет)."""
718     ram_sticks: List[RamStickInfo] = Field(default_factory=lis
t, description='Конфигурация планок RAM (SPD)')
719     physical_disks: List[PhysicalDiskHealth] = Field(default_f
actory=list, description='Состояние физических накопителей (SM
ART)')
720     listening_ports: List[NetworkPortMetrics] = Field(default_
factory=list, description='Список активных слушающих сокетов')
721     alerts: SystemHealthAlerts = Field(default_factory=SystemH
ealthAlerts, description='Системные предупреждения и флаг пере
загрузки')
722

```

```

723 class W64SystemEvent(BaseModel):
724     """Модель отдельного системного события AI W64 Collector."
725     """
726     event_id: str = Field(default='', description='Уникальный
идентификатор события')
727     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время события UTC')
728     event_type: str = Field(..., description='Тип события (pro
cess_start, file_modified, registry_change, network_open и др.
)')
729     path: Optional[str] = Field(default=None, description='Пут
ь к файлу или ветке реестра')
730     pid: Optional[int] = Field(default=None, description='Иден
тификатор процесса')
731     name: Optional[str] = Field(default=None, description='Имя
процесса/файла/параметра')
732     details: Dict[str, Any] = Field(default_factory=dict, desc
ription='Дополнительные атрибуты события')
733
734     def to_dict(self) -> Dict[str, Any]:
735         """Сериализация события в словарь."""
736         res = self.model_dump()
737         if self.details:
738             res.update(self.details)
739         return res
740
741 class ETWTraceEvent(BaseModel):
742     """Модель низкоуровневого события ETW (Event Tracing for W
indows)."""
743     event_id: str = Field(default='', description='Уникальный
идентификатор события')
744     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время события UTC')
745     event_type: str = Field(..., description='Тип ETW трассиро
вки (etw_process_create, etw_file_io, etw_network и др.)')
746     provider: str = Field(default='Microsoft-Windows-Security-
Auditing', description='Провайдер ETW')
747     payload: Dict[str, Any] = Field(default_factory=dict, desc

```



```

    ription='Сырые поля события из ETW / Windows Event Log')
747
748     def to_dict(self) -> Dict[str, Any]:
749         """Сериализация ETW события в словарь."""
750         return self.model_dump()
751
752 class W64CollectorStatus(BaseModel):
753     """Статус и статистика работы сборщиков W64/ETW."""
754     running: bool = Field(default=False, description='Флаг ак
ивности сборщика')
755     events_count: int = Field(default=0, description='Общее ко
личество зафиксированных событий')
756     last_event_time: Optional[str] = Field(default=None, descr
iption='Время последнего зафиксированного события')
757
758
759 class SamplingMode(str, Enum):
760     """Режимы сбора телеметрии и адаптивного масштабирования ч
астоты."""
761     NORMAL = 'normal'
762     MONITORING = 'monitoring'
763     BOOT_AGGRESSIVE = 'boot_aggressive'
764     INCIDENT = 'incident'
765     FORENSIC = 'forensic'
766
767
768 class SystemMetricRollup(BaseModel):
769     """Сжатый временной срез системных метрик с расширенной ст
атистикой (avg, min, max, p95, total)."""
770     period_start: str = Field(..., description='Начало периода
агрегации (ISO timestamp)')
771     period_end: str = Field(..., description='Окончание период
а агрегации (ISO timestamp)')
772     duration_seconds: float = Field(default=60.0, description=
'Длительность интервала в секундах')
773     sample_count: int = Field(default=0, description='Количес
тво объединенных замеров')
774     tier: str = Field(default='1m', description='Уровень сжати

```

```
я: 1s, 2s, 5s, 10s, 30s, 1m, 5m, 15m, 1h, 6h, 24h')
775
776     # CPU
777     cpu_avg: float = Field(default=0.0, description='Средняя н
агрузка CPU %')
778     cpu_min: float = Field(default=0.0, description='Минимальн
ая нагрузка CPU %')
779     cpu_max: float = Field(default=0.0, description='Пиковая н
агрузка CPU %')
780     cpu_p95: float = Field(default=0.0, description='95-й перц
ентиль нагрузки CPU %')
781
782     # RAM
783     ram_avg_gb: float = Field(default=0.0, description='Средни
й объем занятой RAM в GB')
784     ram_min_gb: float = Field(default=0.0, description='Минима
льный объем занятой RAM в GB')
785     ram_max_gb: float = Field(default=0.0, description='Максим
альный объем занятой RAM в GB')
786     ram_percent_avg: float = Field(default=0.0, description='С
редний процент занятой памяти')
787     ram_percent_max: float = Field(default=0.0, description='М
аксимальный процент занятой памяти')
788
789     # Disk
790     disk_read_avg_mbs: float = Field(default=0.0, description=
'Средняя скорость чтения с диска в МБ/с')
791     disk_read_max_mbs: float = Field(default=0.0, description=
'Пиковая скорость чтения с диска в МБ/с')
792     disk_read_total_mb: float = Field(default=0.0, description
='Суммарно прочитано за период в МБ')
793     disk_write_avg_mbs: float = Field(default=0.0, description
='Средняя скорость записи на диск в МБ/с')
794     disk_write_max_mbs: float = Field(default=0.0, description
='Пиковая скорость записи на диск в МБ/с')
795     disk_write_total_mb: float = Field(default=0.0, descriptio
n='Суммарно записано за период в МБ')
796
```

```
797     # Network
798     network_rx_avg_mbs: float = Field(default=0.0, description
    ='Средняя скорость входящего трафика в МБ/с')
799     network_rx_max_mbs: float = Field(default=0.0, description
    ='Пиковая скорость входящего трафика в МБ/с')
800     network_rx_total_mb: float = Field(default=0.0, descriptio
    n='Суммарно получено входящего трафика в МБ')
801     network_tx_avg_mbs: float = Field(default=0.0, description
    ='Средняя скорость исходящего трафика в МБ/с')
802     network_tx_max_mbs: float = Field(default=0.0, description
    ='Пиковая скорость исходящего трафика в МБ/с')
803     network_tx_total_mb: float = Field(default=0.0, descriptio
    n='Суммарно отправлено исходящего трафика в МБ')
804
805
806 class ProcessMetricRollup(BaseModel):
807     """Сжатый временной срез per-process метрик за период набл
    юдения."""
808     name: str = Field(..., description='Имя исполняемого файла
    процесса')
809     pid: Optional[int] = Field(default=None, description='PID
    процесса (если отслеживался конкретный экземпляр)')
810     period_start: str = Field(..., description='Начало периода
    (ISO timestamp)')
811     period_end: str = Field(..., description='Конец периода (I
    SO timestamp)')
812     duration_seconds: float = Field(default=60.0, description=
    'Длительность наблюдения')
813     sample_count: int = Field(default=0, description='Число за
    меров')
814     path: Optional[str] = Field(default=None, description='Пут
    ь к исполняемому файлу')
815
816     cpu_avg: float = Field(default=0.0, description='Средний C
    PU %')
817     cpu_max: float = Field(default=0.0, description='Пиковый C
    PU %')
818     cpu_p95: float = Field(default=0.0, description='95-й перц
```

```
    ентить CPU %')
819
820     ram_avg_mb: float = Field(default=0.0, description='Средни
й объем RAM в МБ')
821     ram_max_mb: float = Field(default=0.0, description='Пиковы
й объем RAM в МБ')
822
823     disk_read_total_mb: float = Field(default=0.0, description
='Суммарно прочитано с диска в МБ')
824     disk_write_total_mb: float = Field(default=0.0, descriptio
n='Суммарно записано на диск в МБ')
825     network_rx_total_mb: float = Field(default=0.0, descriptio
n='Суммарно получено по сети в МБ')
826     network_tx_total_mb: float = Field(default=0.0, descriptio
n='Суммарно отправлено по сети в МБ')
827
828
829 class TelemetryIncident(BaseModel):
830     """Модель диагностического инцидента аномалии с привязкой
контекста и кольцевого буфера."""
831     incident_id: str = Field(..., description='Уникальный иден
тификатор инцидента (напр. INC-20261001-01)')
832     timestamp: str = Field(default_factory=lambda: datetime.no
w(timezone.utc).isoformat(), description='Время детекции инцид
ента')
833     trigger_type: str = Field(..., description='Тип триггера:
cpu_spike, disk_burst, network_burst, suspicious_process, dpc_
degradation, defender_alert')
834     severity: str = Field(default='warning', description='Крит
ичность: info, warning, critical')
835     title: str = Field(..., description='Краткое название инци
дента на русском языке')
836     description: str = Field(default='', description='Подробно
е описание выявленной аномалии')
837
838     trigger_metrics: Dict[str, Any] = Field(default_factory=dict, description='Значения метрик в момент срабатывания триггера')
```

```

839     suspect_processes: List[Dict[str, Any]] = Field(default_factory=list, description='Список подозрительных процессов, связанных с аномалией')
840     related_events: List[Dict[str, Any]] = Field(default_factory=list, description='Дискретные события (запуск процессов, ETW, Defender) за период')
841     metrics_summary: Dict[str, Any] = Field(default_factory=dict, description='Агрегированная сводка метрик до и после инцидента')
842     raw_window: List[Dict[str, Any]] = Field(default_factory=list, description='Срез сырых секундных замеров из кольцевого буфера (pre/post window)')
843
844
845 class ShutdownType(str, Enum):
846     """Типы завершения работы и перезагрузки Windows."""
847     PLANNED = 'PLANNED'
848     UNEXPECTED = 'UNEXPECTED'
849     CRASH_BSOD = 'CRASH_BSOD'
850     UPDATE_RESTART = 'UPDATE_RESTART'
851     USER_INITIATED = 'USER_INITIATED'
852     HARDWARE_POWER_BUTTON = 'HARDWARE_POWER_BUTTON'
853     DIRTY_POWER_LOSS = 'DIRTY_POWER_LOSS'
854     HYBRID_SHUTDOWN = 'HYBRID_SHUTDOWN'
855     UNKNOWN = 'UNKNOWN'
856
857
858 class RebootCorrelatedEvent(BaseModel):
859     """Событие, связанное с перезагрузкой или завершением работы."""
860     event_id: int = Field(..., description='Event ID (например, 1074, 41, 6008, 1001)')
861     provider: str = Field(..., description='Источник/провайдер события (User32, Kernel-Power, EventLog и др.)')
862     timestamp: str = Field(..., description='Время регистрации события в формате ISO/строка')
863     level: str = Field(default='Information', description='Уровень события (Critical, Error, Warning, Information)')

```

```
864     description: str = Field(default='', description='Краткое
| описание смысла события на русском языке')
865     details: Dict[str, Any] = Field(default_factory=dict, desc
| ription='Извлеченные структурированные поля события')
866
867
868 class RebootSession(BaseModel):
869     """Структурированная запись об одной сессии перезагрузки /
| выключения Windows."""
870     boot_id: str = Field(..., description='Уникальный идентифи
| катор сессии перезагрузки (напр. boot-20261001-014218)')
871     boot_time: str = Field(..., description='Время запуска сис
| темы (OS Startup)')
872     previous_boot_time: Optional[str] = Field(default=None, de
| scription='Время предыдущего запуска системы')
873     uptime_seconds: Optional[float] = Field(default=None, desc
| ription='Время непрерывной работы предыдущей сессии в секундах
| ')
874     uptime_human: Optional[str] = Field(default=None, descript
| ion='Человекочитаемое время работы (напр. "7h 25m")')
875     shutdown_type: ShutdownType = Field(default=ShutdownType.U
| NKNOWN, description='Классифицированный тип завершения работы'
| )
876     likely_class: str = Field(default='unknown', description='
| Короткий машинный класс причины')
877     conclusion: str = Field(default='', description='Итоговое
| аналитическое заключение на русском языке')
878     initiating_process: Optional[str] = Field(default=None, de
| scription='Процесс, инициировавший перезагрузку (из Event 1074
| )')
879     initiating_user: Optional[str] = Field(default=None, descr
| iption='Учетная запись, от имени которой инициирован рестарт')
880     shutdown_action: Optional[str] = Field(default=None, descr
| iption='Действие: restart, power off, shutdown, hybrid')
881     reason_text: Optional[str] = Field(default=None, descripti
| on='Текстовая причина перезапуска')
882     reason_code: Optional[str] = Field(default=None, descripti
| on='Шестнадцатеричный код причины (напр. 0x80020010)')
```

```
883     bugcheck_code: Optional[str] = Field(default=None, descrip
      tion='Код остановки BSOD (напр. 0x000000D1)')
884     bugcheck_params: List[str] = Field(default_factory=list, d
      escription='Параметры BugCheck (p1..p4)')
885     power_button_timestamp: Optional[int] = Field(default=None
      , description='Метка нажатия кнопки питания из Event 41')
886     windows_update_kb: Optional[str] = Field(default=None, des
      cription='Номер установленного обновления Windows (Event 19)')
887     service_installed: Optional[str] = Field(default=None, des
      cription='Имя недавно установленной службы (Event 7045)')
888     evidence: List[str] = Field(default_factory=list, descript
      ion='Список доказательных фактов и событий')
889     events_chain: List[RebootCorrelatedEvent] = Field(default_
      factory=list, description='Хронологическая цепочка коррелирова
      нных событий')
890
891
892 class RebootAnalysisReport(BaseModel):
893     """Сводный отчет анализа стабильности и причин перезагрузо
      к системы."""
894     generated_at: str = Field(default_factory=lambda: datetime
      .now(timezone.utc).isoformat(), description='Время формировани
      я отчета')
895     hostname: str = Field(default='', description='Имя компьют
      ера')
896     current_boot_time: str = Field(default='', description='Вр
      емя текущей загрузки Windows')
897     current_uptime_seconds: float = Field(default=0.0, descrip
      tion='Текущий аптайм в секундах')
898     current_uptime_human: str = Field(default='', description=
      'Текущий аптайм в читаемом формате')
899     total_reboots_analyzed: int = Field(default=0, description
      ='Количество проанализированных перезагрузок')
900     planned_count: int = Field(default=0, description='Количес
      тво плановых перезагрузок')
901     unexpected_count: int = Field(default=0, description='Коли
      чество неожиданных/аварийных выключений')
902     bsod_count: int = Field(default=0, description='Количество
```

```

    сбоях с синим экраном (BSOD)')
903     update_reboot_count: int = Field(default=0, description='К
    оличество перезагрузок из-за обновлений Windows')
904     stability_score: float = Field(default=100.0, description=
    'Оценка стабильности завершений работы (0..100)')
905     sessions: List[RebootSession] = Field(default_factory=list
    , description='Список сессий перезагрузок (от новых к старым)'
    )
906     summary_ru: str = Field(default='', description='Общее рез
    юме надежности на русском языке')
907
908
909 class DefenderTelemetrySummary(BaseModel):
910     """Сводка телеметрии встроенного защитника Windows Defende
    r."""
911     cfa_enabled: bool = Field(default=False, description='Вклю
    чена ли защита Controlled Folder Access (от Ransomware)')
912     cfa_mode: str = Field(default='Disabled', description='Реж
    им CFA: Enabled, Disabled, Audit, Warn')
913     cfa_protected_folders_count: int = Field(default=0, descri
    ption='Количество защищенных папок')
914     asr_rules_count: int = Field(default=0, description='Колич
    ество настроенных правил ASR')
915     asr_enabled_count: int = Field(default=0, description='Кол
    ичество активных правил ASR')
916     exclusions_count: int = Field(default=0, description='Обще
    е количество исключений антивируса')
917     path_exclusions: List[str] = Field(default_factory=list, d
    escription='Список путей, исключенных из проверки')
918     process_exclusions: List[str] = Field(default_factory=list
    , description='Список процессов, исключенных из проверки')
919     active_threats_count: int = Field(default=0, description='
    Количество активных необработанных угроз')
920     threat_history_count: int = Field(default=0, description='
    Количество зафиксированных угроз в истории')
921
922
923 class StartupTelemetrySummary(BaseModel):

```



```

924     """Сводка телеметрии точек автозапуска и персистентности п
    рограмм."""
925     total_entries: int = Field(default=0, description='Общее к
    оличество программ и элементов в автозагрузке')
926     registry_run_count: int = Field(default=0, description='За
    писи автозапуска в реестре Run/RunOnce')
927     startup_folders_count: int = Field(default=0, description=
    'Ярлыки в папках Startup (пользователь/система)')
928     scheduled_tasks_count: int = Field(default=0, description=
    'Задачи планировщика со стартом при входе')
929     startup_services_count: int = Field(default=0, description
    ='Службы Windows с автозапуском')
930     entries: List[Dict[str, Any]] = Field(default_factory=list
    , description='Список записей автозапуска с параметрами')
931
932 class VssTelemetrySummary(BaseModel):
933     """Сводка состояния теневых копий томов VSS и точек восста
    новления."""
934     total_snapshots_count: int = Field(default=0, description=
    'Количество существующих теневых копий')
935     protected_volumes: List[str] = Field(default_factory=list,
    description='Список защищенных томов (буквы дисков)')
936     latest_snapshot_time: Optional[str] = Field(default=None,
    description='Время создания последней теневой копии')
937     snapshots: List[Dict[str, Any]] = Field(default_factory=li
    st, description='Список снимков VSS')
938
939
940 class UserAccountsTelemetrySummary(BaseModel):
941     """Сводка аудита локальных пользователей и привилегий."""
942     total_users_count: int = Field(default=0, description='Общ
    ее число локальных учетных записей')
943     admin_users_count: int = Field(default=0, description='Чис
    ло учетных записей с правами администратора')
944     locked_users_count: int = Field(default=0, description='Чи
    сло заблокированных учетных записей')
945     admin_usernames: List[str] = Field(default_factory=list, d
    escription='Имена локальных администраторов')

```

```
946     logged_in_users: List[str] = Field(default_factory=list, d
description='Активные вошедшие пользователи')
947     users: List[Dict[str, Any]] = Field(default_factory=list,
description='Сводный список учетных записей')
948
949
950 class ExtendedSystemAuditReport(BaseModel):
951     """Комплексный отчет расширенного аудита безопасности, авт
озагрузки, VSS и пользователей."""
952     timestamp: str = Field(default_factory=lambda: datetime.no
w(datetime.utcnow().isoformat(), description='Время аудита')
953     hostname: str = Field(default='', description='Имя компьют
ера')
954     defender: DefenderTelemetrySummary = Field(default_factory
=DefenderTelemetrySummary, description='Сводка Windows Defende
r')
955     startup: StartupTelemetrySummary = Field(default_factory=S
tartupTelemetrySummary, description='Сводка автозагрузки')
956     vss: VssTelemetrySummary = Field(default_factory=VssTeleme
trySummary, description='Сводка теневых копий VSS')
957     users: UserAccountsTelemetrySummary = Field(default_factor
y=UserAccountsTelemetrySummary, description='Сводка локальных
пользователей')
958
959
960
961
```

process_token_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Process
  Token Collector
4 # =====
5 # Description:
6 #   Коллектор токенов процессов, уровней целостности, прав UAC
  , SID и Session ID.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.process_token_collector impo
  rt ProcessTokenCollector
11 #
12 #       collector = ProcessTokenCollector()
13 #       tokens = collector.collect()
14 #
15 # File: process_token_collector.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-04 01:45:00
21 # =====
22
23 """Коллектор токенов процессов, уровней целостности, прав UAC,
  SID и Session ID."""
24
25 from __future__ import annotations
26
27 import ctypes
28 import ctypes.wintypes as wtypes
29 import os
```

```
30 from typing import Dict, List, Optional
31 try:
32     import psutil
33     PSUTIL_AVAILABLE = True
34 except ImportError:
35     psutil = None
36     PSUTIL_AVAILABLE = False
37
38 from .models import ProcessTokenInfo
39
40 TOKEN_QUERY = 8
41 TokenUser = 1
42 TokenGroups = 2
43 TokenPrivileges = 3
44 TokenElevation = 20
45 TokenIntegrityLevel = 25
46
47
48 class TOKEN_ELEVATION(ctypes.Structure):
49     _fields_ = [('TokenIsElevated', ctypes.DWORD)]
50
51
52 class LUID(ctypes.Structure):
53     _fields_ = [('LowPart', ctypes.DWORD), ('HighPart', ctypes
54         .LONG)]
55
56 class LUID_AND_ATTRIBUTES(ctypes.Structure):
57     _fields_ = [('Luid', LUID), ('Attributes', ctypes.DWORD)]
58
59
60 class TOKEN_MANDATORY_LABEL(ctypes.Structure):
61     _fields_ = [('Label', LUID_AND_ATTRIBUTES)]
62
63
64 class SID_AND_ATTRIBUTES(ctypes.Structure):
65     _fields_ = [('Sid', ctypes.c_void_p), ('Attributes', ctypes
66         .DWORD)]
```

```

66 |
67 |
68 | class TOKEN_USER(ctypes.Structure):
69 |     _fields_ = [('User', SID_AND_ATTRIBUTES)]
70 |
71 |
72 | def _get_process_token(handle: Any) -> wtypes.HANDLE:
73 |     """Получить дескриптор токена процесса."""
74 |     token = wtypes.HANDLE()
75 |     if not ctypes.windll.advapi32.OpenProcessToken(handle, TOK
76 | EN_QUERY, ctypes.byref(token)):
77 |         raise ctypes.WinError()
78 |     return token
79 |
80 | def _get_token_elevation(token: Any) -> bool:
81 |     """Определить статус повышения прав UAC."""
82 |     elevation = TOKEN_ELEVATION()
83 |     size = wtypes.DWORD(ctypes.sizeof(elevation))
84 |     ret = ctypes.windll.advapi32.GetTokenInformation(
85 |         token, TokenElevation, ctypes.byref(elevation), size,
86 | ctypes.byref(size)
87 |     )
88 |     if not ret:
89 |         raise ctypes.WinError()
90 |     return bool(elevation.TokenIsElevated)
91 |
92 | def _get_integrity_level(token: Any) -> str:
93 |     """Получить уровень целостности процесса (Untrusted, Low,
94 | Medium, High, System)."""
95 |     size = wtypes.DWORD(0)
96 |     ctypes.windll.advapi32.GetTokenInformation(token, TokenInt
97 | egrityLevel, None, 0, ctypes.byref(size))
98 |     if size.value == 0:
99 |         return 'Medium'
100 |     buf = ctypes.create_string_buffer(size.value)
101 |     if not ctypes.windll.advapi32.GetTokenInformation(token, T

```

```

    okenIntegrityLevel, buf, size, ctypes.byref(size)):
100         raise ctypes.WinError()
101     label = ctypes.cast(buf, ctypes.POINTER(TOKEN_MANDATORY_LA
    BEL)).contents
102     integrity = label.Label.Luid.LowPart
103     if integrity < 4096:
104         return 'Untrusted'
105     elif integrity < 8192:
106         return 'Low'
107     elif integrity < 12288:
108         return 'Medium'
109     elif integrity < 16384:
110         return 'High'
111     else:
112         return 'System'
113
114
115 def _get_token_sid(token: Any) -> Optional[str]:
116     """Извлечь строковый SID пользователя из токена процесса."""
117     try:
118         size = wtypes.DWORD(0)
119         ctypes.windll.advapi32.GetTokenInformation(token, Toke
    nUser, None, 0, ctypes.byref(size))
120         if size.value == 0:
121             return None
122         buf = ctypes.create_string_buffer(size.value)
123         if not ctypes.windll.advapi32.GetTokenInformation(toke
    n, TokenUser, buf, size, ctypes.byref(size)):
124             return None
125         user_struct = ctypes.cast(buf, ctypes.POINTER(TOKEN_US
    ER)).contents
126         sid_ptr = user_struct.User.Sid
127         if not sid_ptr:
128             return None
129         str_sid = wtypes.LPWSTR()
130         if ctypes.windll.advapi32.ConvertSidToStringSidW(sid_p
    tr, ctypes.byref(str_sid)):

```

```

131         sid_val = str_sid.value
132         ctypes.windll.kernel32.LocalFree(str_sid)
133         return sid_val
134     except Exception:
135         pass
136     return None
137
138
139 def get_process_session_id(pid: int) -> Optional[int]:
140     """Получить Session ID процесса (Session 0=Службы, Session
141     1+=Интерактивный пользователь)."""
142     if os.name != 'nt':
143         return None
144     try:
145         session_id = wtypes.DWORD(0)
146         if ctypes.windll.kernel32.ProcessIdToSessionId(pid, ct
147         ypes.byref(session_id)):
148             return int(session_id.value)
149     except Exception:
150         pass
151     return None
152
153 class ProcessTokenCollector:
154     """Коллектор сведений о безопасности токенов, правах UAC,
155     SID и сессиях процессов Windows."""
156
157     def get_process_token_info(self, pid: int, name: str = '')
158     -> Optional[ProcessTokenInfo]:
159         """Собрать информацию о токене для конкретного PID.
160
161         Args:
162             pid: Идентификатор процесса.
163             name: Имя процесса (опционально).
164
165         Returns:
166             Optional[ProcessTokenInfo]: Данные токена или None
167             при отсутствии доступа.

```

```

164         """
165         if os.name != 'nt':
166             return ProcessTokenInfo(pid=pid, name=name, elevat
ion=False, integrity_level='Medium', sid=None, session_id=1)
167
168         session_id = get_process_session_id(pid)
169         handle = ctypes.windll.kernel32.OpenProcess(1024, Fals
e, pid)
170         if not handle:
171             return ProcessTokenInfo(
172                 pid=pid,
173                 name=name,
174                 elevation=None,
175                 integrity_level=None,
176                 privileges=[],
177                 sid=None,
178                 session_id=session_id,
179             )
180         try:
181             token = _get_process_token(handle)
182             elevated = _get_token_elevation(token)
183             integrity = _get_integrity_level(token)
184             sid = _get_token_sid(token)
185             privileges: List[str] = []
186             return ProcessTokenInfo(
187                 pid=pid,
188                 name=name,
189                 elevation=bool(elevated),
190                 integrity_level=integrity,
191                 privileges=privileges,
192                 sid=sid,
193                 session_id=session_id,
194             )
195         except Exception:
196             return ProcessTokenInfo(
197                 pid=pid,
198                 name=name,
199                 elevation=None,

```



```

200         integrity_level=None,
201         privileges=[],
202         sid=None,
203         session_id=session_id,
204     )
205     finally:
206         ctypes.windll.kernel32.CloseHandle(handle)
207
208     def collect(self) -> List[ProcessTokenInfo]:
209         """Собрать сведения о токенах для всех активных процес
210 сов в системе.
211
212         Returns:
213             List[ProcessTokenInfo]: Список токенов процессов.
214         """
215         results: List[ProcessTokenInfo] = []
216         if not PSUTIL_AVAILABLE:
217             current_pid = os.getpid()
218             info = self.get_process_token_info(current_pid, 'p
219 ython.exe')
220             if info:
221                 results.append(info)
222             return results
223
224         for proc in psutil.process_iter(attrs=['pid', 'name']):
225             :
226             try:
227                 pid = proc.info['pid']
228                 name = proc.info['name'] or 'unknown'
229                 info = self.get_process_token_info(pid, name)
230                 if info:
231                     results.append(info)
232             except Exception:
233                 continue
234         return results

```

ring_buffer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Ring Bu
  ffer
4 # =====
5 # Description:
6 #   Потокбезопасный кольцевой буфер в оперативной памяти для
  удержания
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.ring_buffer import Telemetry
  RingBuffer
11 #
12 #       service = TelemetryRingBuffer()
13 #
14 # File: ring_buffer.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 05:32:00
20 # =====
21
22 from __future__ import annotations
23 """Потокбезопасный кольцевой буфер в оперативной памяти для у
  держания сырой телеметрии."""
24
25 import collections
26 import json
27 import sys
28 import threading
29 import time
```

```

30 from datetime import datetime, timezone
31 from typing import Any, Dict, List, Optional, Tuple
32
33
34 class TelemetryRingBuffer:
35     """Потокобезопасный кольцевой буфер для хранения сырых мет
    | рик телеметрии в RAM с ограничением по объему памяти."""
36
37     def __init__(self, capacity: int = 600, max_bytes: int = 3
    | 2 * 1024 * 1024) -> None:
38         """Инициализирует кольцевой буфер заданной емкости и м
    | аксимального объема RAM.
39
40         Args:
41             capacity: Максимальное количество удерживаемых зап
    | исей (по умолчанию 600 = 10 мин при 1с).
42             max_bytes: Максимальный совокупный объем памяти бу
    | фера в байтах (по умолчанию 32 МБ).
43         """
44         self._capacity = max(10, capacity)
45         self._max_bytes = max(1024 * 1024, max_bytes)
46         self._current_bytes = 0
47         self._buffer: collections.deque[Tuple[float, Dict[str,
    | Any], int]] = collections.deque(maxlen=self._capacity)
48         self._lock = threading.Lock()
49
50     @property
51     def capacity(self) -> int:
52         """Возвращает максимальную емкость буфера в количестве
    | записей."""
53         return self._capacity
54
55     @property
56     def max_bytes(self) -> int:
57         """Возвращает максимальный лимит памяти буфера в байта
    | х."""
58         return self._max_bytes
59

```

```

60 |     @property
61 |     def current_bytes(self) -> int:
62 |         """Возвращает текущий ориентировочный объем занимаемой
        |         памяти буфера в байтах."""
63 |         with self._lock:
64 |             return self._current_bytes
65 |
66 |     def _estimate_size(self, item: Dict[str, Any]) -> int:
67 |         """Оценивает размер словаря с метриками в байтах."""
68 |         try:
69 |             return len(json.dumps(item, ensure_ascii=False, de
        |             fault=str).encode('utf-8')) + 128
70 |         except Exception:
71 |             return sys.getsizeof(item) + 256
72 |
73 |     def append(self, item: Dict[str, Any], epoch_timestamp: Op
        |     tional[float] = None) -> None:
74 |         """Добавляет запись телеметрии в кольцевой буфер с кон
        |         тролем лимитов RAM и количества.
75 |
76 |         Args:
77 |             item: Словарь с телеметрией или системным снимком.
78 |             epoch_timestamp: Unix epoch timestamp в секундах (
        |             если None, берется текущее время).
79 |         """
80 |         ts = epoch_timestamp if epoch_timestamp is not None el
        |         se time.time()
81 |         item_size = self._estimate_size(item)
82 |         with self._lock:
83 |             # Если буфер заполнен по длине, deque сам вытеснит
        |             старый элемент, но нам нужно учесть его размер
84 |             if len(self._buffer) == self._capacity and self._b
        |             uffer:
85 |                 old_ts, old_item, old_size = self._buffer[0]
86 |                 self._current_bytes = max(0, self._current_byt
        |                 es - old_size)
87 |
88 |                 self._buffer.append((ts, item, item_size))

```

```

89         self._current_bytes += item_size
90
91         # Контроль жесткого байтового лимита RAM: вытеснен
    ие самых старых записей при переполнении
92         while self._current_bytes > self._max_bytes and le
    n(self._buffer) > 1:
93             old_ts, old_item, old_size = self._buffer.popl
    eft()
94             self._current_bytes = max(0, self._current_byt
    es - old_size)
95
96     def get_recent(self, seconds: float = 300.0) -> List[Dict[
    str, Any]]:
97         """Возвращает список записей за последние N секунд.
98
99         Args:
100             seconds: Длительность временного окна назад от тек
    ущего момента.
101
102         Returns:
103             List[Dict[str, Any]]: Список замеров от старых к н
    ОВЫМ.
104
105         """
106         now = time.time()
107         cutoff = now - max(0.0, seconds)
108         with self._lock:
109             return [item for ts, item, _ in self._buffer if ts
    >= cutoff]
110
111     def get_window(
112         self,
113         center_timestamp: float,
114         before_seconds: float = 300.0,
115         after_seconds: float = 0.0,
116     ) -> List[Dict[str, Any]]:
117         """Извлекает срез телеметрии вокруг заданного момента
    времени (например, T-5m .. T+20m).

```

```
118 |         Args:
119 |             center_timestamp: Центральный момент времени событ
    | ия (Unix epoch seconds).
120 |             before_seconds: Длительность окна ДО события в сек
    | ундах.
121 |             after_seconds: Длительность окна ПОСЛЕ события в с
    | екундах.
122 |
123 |         Returns:
124 |             List[Dict[str, Any]]: Список записей, попавших в и
    | нтервал.
125 |             """
126 |             start_ts = center_timestamp - max(0.0, before_seconds)
127 |             end_ts = center_timestamp + max(0.0, after_seconds)
128 |             with self._lock:
129 |                 return [item for ts, item, _ in self._buffer if st
    | art_ts <= ts <= end_ts]
130 |
131 |     def get_all(self) -> List[Dict[str, Any]]:
132 |         """Возвращает все текущие записи в буфере.
133 |
134 |         Returns:
135 |             List[Dict[str, Any]]: Полный текущий срез буфера.
136 |             """
137 |             with self._lock:
138 |                 return [item for _, item, _ in self._buffer]
139 |
140 |     def count(self) -> int:
141 |         """Возвращает текущее число записей в буфере."""
142 |         with self._lock:
143 |             return len(self._buffer)
144 |
145 |     def clear(self) -> None:
146 |         """Очищает буфер."""
147 |         with self._lock:
148 |             self._buffer.clear()
149 |             self._current_bytes = 0
150 |
```

sampling_controller.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sampling
  # Controller
4 # =====
5 # Description:
6 #   Контроллер адаптивного изменения частоты сбора телеметрии
  # (Sampling Controller).
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sampling_controller import S
  # amplingController
11 #
12 #       service = SamplingController()
13 #
14 # File: sampling_controller.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 05:34:00
20 # =====
21
22 from __future__ import annotations
23 """Контроллер адаптивного изменения частоты сбора телеметрии (
  # Sampling Controller)."""
24
25 import time
26 from datetime import datetime, timezone
27 from typing import Any, Dict, Optional
28
29 try:
```

```

30     import psutil
31     BOOT_TIME = psutil.boot_time()
32 except Exception:
33     BOOT_TIME = time.time()
34
35 try:
36     from logger import logger
37 except ImportError:
38     from logger import logger
39
40 from .models import SamplingMode
41
42
43 class SamplingController:
44     """Контроллер режимов сбора телеметрии и адаптивного интер
45 вала."""
46
47     def __init__(
48         self,
49         initial_mode: SamplingMode = SamplingMode.BOOT_AGGRESS
50 IVE,
51         boot_epoch: Optional[float] = None,
52     ) -> None:
53         """Инициализирует контроллер режимов сбора.
54
55         Args:
56             initial_mode: Начальный режим работы (по умолчанию
57 BOOT_AGGRESSIVE).
58             boot_epoch: Время старта ОС или сервиса в секундах
59 Unix epoch.
60         """
61         self._mode = initial_mode
62         self._boot_epoch = boot_epoch if boot_epoch is not Non
63 e else BOOT_TIME
64         self._start_epoch = time.time()
65         self._incident_expiry_epoch: Optional[float] = None
66         self._previous_mode: SamplingMode = SamplingMode.MONIT
67 ORING

```



```

62 |
63 |     @property
64 |     def mode(self) -> SamplingMode:
65 |         """Возвращает текущий режим сбора с учетом авто-истече
        ния инцидентов."""
66 |         self._check_incident_expiration()
67 |         return self._mode
68 |
69 |     def set_mode(self, mode: SamplingMode) -> None:
70 |         """Принудительно устанавливает режим сбора телеметрии.
71 |
72 |         Args:
73 |             mode: Новый режим работы.
74 |         """
75 |         self._mode = mode
76 |         if mode not in (SamplingMode.INCIDENT, SamplingMode.FO
        RENSIC):
77 |             self._incident_expiry_epoch = None
78 |             logger.info(f"Режим сбора телеметрии переключен на: {s
        elf._mode.value}")
79 |
80 |     def trigger_incident_mode(self, duration_seconds: float =
        1200.0) -> None:
81 |         """Переводит сборщик в режим INCIDENT (1-секундный зах
        ват) на заданную длительность.
82 |
83 |         Args:
84 |             duration_seconds: Длительность удержания режима ин
        цидента (по умолчанию 20 минут = 1200 сек).
85 |         """
86 |         if self._mode not in (SamplingMode.INCIDENT, SamplingM
        ode.FORENSIC):
87 |             self._previous_mode = self._mode
88 |             self._mode = SamplingMode.INCIDENT
89 |             self._incident_expiry_epoch = time.time() + max(0.01,
        duration_seconds)
90 |             logger.warning(f"Активирован режим INCIDENT на {durati
        on_seconds} сек.")

```

```

91
92     def trigger_forensic_mode(self, duration_seconds: float =
93     300.0) -> None:
94         """Переводит сборщик в режим FORENSIC (250-мс глубокий
95         захват) на заданную длительность.
96
97         Args:
98             duration_seconds: Длительность удержания форензик-
99             режима в секундах (по умолчанию 5 минут = 300 сек).
100         """
101         if self._mode not in (SamplingMode.INCIDENT, SamplingM
102         ode.FORENSIC):
103             self._previous_mode = self._mode
104             self._mode = SamplingMode.FORENSIC
105             self._incident_expiry_epoch = time.time() + max(0.01,
106             duration_seconds)
107             logger.warning(f"Активирован высокоскоростной режим FO
108             RENSIC (250мс) на {duration_seconds} сек.")
109
110     def get_current_interval(self) -> float:
111         """Возвращает текущий рекомендуемый интервал опроса те
112         леметрии в секундах.
113
114         Returns:
115             float: Интервал в секундах.
116         """
117         self._check_incident_expiration()
118
119         if self._mode == SamplingMode.FORENSIC:
120             return 0.25
121
122         if self._mode == SamplingMode.INCIDENT:
123             return 1.0
124
125         if self._mode == SamplingMode.BOOT_AGGRESSIVE:
126             # Адаптивная кривая после старта Windows / сервиса
127             uptime = time.time() - self._boot_epoch
128             if uptime < 0:

```

```

122         uptime = time.time() - self._start_epoch
123
124         if uptime <= 600.0:  # 0-10 минут: 1-секундные дан
ные
125             return 1.0
126         elif uptime <= 3600.0:  # 10-60 минут: 5-секундные
127             return 5.0
128         elif uptime <= 86400.0:  # 1-24 часа: 30-секундные
129             return 30.0
130         else:
131             # После 24 часов переходим в дежурный монитори
нг
132             return 60.0
133
134         if self._mode == SamplingMode.MONITORING:
135             return 30.0
136
137         if self._mode == SamplingMode.NORMAL:
138             return 60.0
139
140         return 30.0
141
142     def should_collect_per_process_details(self) -> bool:
143         """Определяет, требуется ли собирать глубокий срез по
всем процессам на текущем тике."""
144         return True
145
146     def _check_incident_expiration(self) -> None:
147         """Проверяет истечение времени режима инцидента/форенз
ики и возвращает предыдущий режим."""
148         if self._mode in (SamplingMode.INCIDENT, SamplingMode.
FORENSIC) and self._incident_expiry_epoch is not None:
149             if time.time() >= self._incident_expiry_epoch:
150                 logger.info("Время фиксации инцидента/форензик
и истекло. Возврат в штатный режим.")
151                 self._mode = self._previous_mode
152                 self._incident_expiry_epoch = None
153

```

```
154 |     def get_status(self) -> Dict[str, Any]:
155 |         """Возвращает статус контроллера опроса.
156 |
157 |         Returns:
158 |             Dict[str, Any]: Словарь с текущим режимом, интерва
лом и временем старта.
159 |         """
160 |         return {
161 |             "mode": self.mode.value,
162 |             "current_interval_sec": self.get_current_interval(
163 |             ),
164 |             "boot_epoch": self._boot_epoch,
165 |             "incident_active": self._mode in (SamplingMode.INC
IDENT, SamplingMode.FORENSIC),
166 |             "incident_remaining_sec": max(0.0, (self._incident
_expiry_epoch - time.time())) if self._incident_expiry_epoch e
lse 0.0,
167 |         }
```

sensors.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensors
4 # =====
5 # Description:
6 #   Движок считывания показаний температурных датчиков, напряж
   ений и вентиляторов.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensors import get_hardware_
   sensors
11 #
12 #       res = get_hardware_sensors()
13 #
14 # File: sensors.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:00:00
20 # =====
21
22 from __future__ import annotations
23 """Движок считывания показаний аппаратных сенсоров, сетевых ме
   трик и скорости FAST.com."""
24
25 import json
26 import os
27 import subprocess
28 from typing import List, Optional
29 try:
30     import wmi
```

```

31 |     _WMI_AVAILABLE = True
32 | except ImportError:
33 |     _WMI_AVAILABLE = False
34 | from logger import logger
35 | from .models import HardwareSensor
36 | from .internet_speed import InternetSpeedSensor
37 | from .sensor_registry import SensorProvider
38 |
39 | # NOTE: GPU temperature, power, and fan sensors are sourced fr
    | om:
40 | # 1. GpuProber._probe_nvidia() -> HardwareMonitor.get_gpu_me
    | trics()
41 | # 2. sensor_collector.extract_sensor_readings() normalizes G
    | puMetrics to sensor_id format
42 |
43 | def _probe_acpi_thermal_zones() -> List[HardwareSensor]:
44 |     """Сбор данных температурных зон Windows ACPI через CIM с
    | fallback на WMI.
45 |
46 |     Returns:
47 |         List[HardwareSensor]: Список показаний термальных зон
    | ACPI.
48 |     """
49 |     sensors: List[HardwareSensor] = []
50 |     if os.name != 'nt':
51 |         return sensors
52 |
53 |     # 1. Первичный сбор через CIM (Get-CimInstance)
54 |     try:
55 |         cmd = [
56 |             'powershell.exe', '-NoProfile', '-NonInteractive',
    | '-Command',
57 |             'Get-CimInstance -Namespace root/wmi -ClassName MS
    | Acpi_ThermalZoneTemperature -ErrorAction SilentlyContinue | '
58 |             'Select-Object InstanceName, CurrentTemperature |
    | ConvertTo-Json -Compress'
59 |         ]
60 |         res = subprocess.run(cmd, capture_output=True, text=Tr

```

```

ue, timeout=5)
61         if res.returncode == 0 and res.stdout.strip():
62             raw_data = json.loads(res.stdout.strip())
63             items = [raw_data] if isinstance(raw_data, dict) e
lse (raw_data if isinstance(raw_data, list) else [])
64             for idx, zone in enumerate(items):
65                 raw_temp = zone.get('CurrentTemperature')
66                 if raw_temp and raw_temp > 0:
67                     celsius = round(float(raw_temp) / 10.0 - 2
73.15, 1)
68                     if -20.0 <= celsius <= 125.0:
69                         zone_name = zone.get('InstanceName') o
r f'Thermal Zone {idx}'
70                         sensor = HardwareSensor(
71                             sensor_id=f'acpi_thermal_{idx}',
72                             name=f'ACPI {zone_name}',
73                             category='temperature',
74                             value=celsius,
75                             unit='°C'
76                         )
77                         sensor.provider = SensorProvider.CIM_S
YSTEM
78                         sensors.append(sensor)
79                     if sensors:
80                         return sensors
81             except Exception as ex:
82                 logger.debug(f'CIM ACPI thermal probe skipped or error
: {ex}')
83
84     # 2. Fallback / слой совместимости через WMI
85     try:
86         import wmi
87         import pythoncom
88         pythoncom.CoInitialize()
89         w = wmi.WMI(namespace='root\\wmi')
90         for idx, zone in enumerate(w.MSAcpi_ThermalZoneTempera
ture()):
91             raw_temp = getattr(zone, 'CurrentTemperature', Non

```

```

e)
92         if raw_temp and raw_temp > 0:
93             celsius = round(float(raw_temp) / 10.0 - 273.1
5, 1)
94             if -20.0 <= celsius <= 125.0:
95                 zone_name = getattr(zone, 'InstanceName',
f'Thermal Zone {idx}')
96                 sensor = HardwareSensor(
97                     sensor_id=f'acpi_thermal_{idx}',
98                     name=f'ACPI {zone_name}',
99                     category='temperature',
100                     value=celsius,
101                     unit='°C'
102                 )
103                 sensor.provider = SensorProvider.WMI_FALLB
ACK
104                 sensors.append(sensor)
105     except Exception as ex:
106         logger.debug(f'WMI ACPI thermal fallback probe skipped
: {ex}')
107
108     return sensors
109
110 def _probe_network_sensors() -> List[HardwareSensor]:
111     """Probe network interface metrics using Windows native to
ols.
112
113     Returns:
114         List[HardwareSensor]: Network interface telemetry as s
ensors with provider marker.
115     """
116     sensors: List[HardwareSensor] = []
117     try:
118         import psutil
119         current_net_io = psutil.net_io_counters(pernic=True)
120         if current_net_io:
121             for name, stats in current_net_io.items():
122                 # Create sensors with provider marker

```



```
123         s1 = HardwareSensor(sensor_id=f'net_{name}_byt
    es_recv', name=f'Network {name} Bytes Received', category='net
    work', value=round(stats.bytes_recv, 2), unit='B')
124         s1.provider = SensorProvider.NETWORK_SENSOR
125         sensors.append(s1)
126
127         s2 = HardwareSensor(sensor_id=f'net_{name}_byt
    es_sent', name=f'Network {name} Bytes Sent', category='network
    ', value=round(stats.bytes_sent, 2), unit='B')
128         s2.provider = SensorProvider.NETWORK_SENSOR
129         sensors.append(s2)
130
131         s3 = HardwareSensor(sensor_id=f'net_{name}_pac
    kets_recv', name=f'Network {name} Packets Received', category=
    'network', value=stats.packets_recv, unit='pkts')
132         s3.provider = SensorProvider.NETWORK_SENSOR
133         sensors.append(s3)
134
135         s4 = HardwareSensor(sensor_id=f'net_{name}_pac
    kets_sent', name=f'Network {name} Packets Sent', category='net
    work', value=stats.packets_sent, unit='pkts')
136         s4.provider = SensorProvider.NETWORK_SENSOR
137         sensors.append(s4)
138
139         s5 = HardwareSensor(sensor_id=f'net_{name}_err
    ors_recv', name=f'Network {name} Errors Received', category='n
    etwork', value=stats.errin, unit='errors')
140         s5.provider = SensorProvider.NETWORK_SENSOR
141         sensors.append(s5)
142
143         s6 = HardwareSensor(sensor_id=f'net_{name}_err
    ors_sent', name=f'Network {name} Errors Sent', category='netwo
    rk', value=stats.errout, unit='errors')
144         s6.provider = SensorProvider.NETWORK_SENSOR
145         sensors.append(s6)
146
147         s7 = HardwareSensor(sensor_id=f'net_{name}_dro
    pped_recv', name=f'Network {name} Dropped Received', category=
```

```

    'network', value=stats.dropin, unit='pkts')
148         s7.provider = SensorProvider.NETWORK_SENSOR
149         sensors.append(s7)
150
151         s8 = HardwareSensor(sensor_id=f'net_{name}_dro
pped_sent', name=f'Network {name} Dropped Sent', category='net
work', value=stats.dropout, unit='pkts')
152         s8.provider = SensorProvider.NETWORK_SENSOR
153         sensors.append(s8)
154     except Exception as ex:
155         logger.debug(f'Network sensor probe failed: {ex}')
156     return sensors
157
158 def _probe_internet_speed_sensors() -> List[HardwareSensor]:
159     """Собрать показания скорости интернета и характеристик се
ти FAST.com.
160
161     Returns:
162         List[HardwareSensor]: Метрики скорости интернета и зад
ержки с маркером провайдера.
163     """
164     sensors: List[HardwareSensor] = []
165     try:
166         sensor = InternetSpeedSensor()
167         metrics = sensor.measure_internet_speed()
168
169         s1 = HardwareSensor(
170             sensor_id='internet_ping',
171             name='Internet Ping',
172             category='network',
173             value=round(metrics.get('ping_ms', 0.0), 2),
174             unit='ms'
175         )
176         s1.provider = SensorProvider.INTERNET_SPEED
177         sensors.append(s1)
178
179         s2 = HardwareSensor(
180             sensor_id='internet_download',

```

```

181         name='Internet Download Speed',
182         category='network',
183         value=round(metrics.get('download_mbps', 0.0), 2),
184         unit='Mbps'
185     )
186     s2.provider = SensorProvider.INTERNET_SPEED
187     sensors.append(s2)
188
189     s3 = HardwareSensor(
190         sensor_id='internet_upload',
191         name='Internet Upload Speed',
192         category='network',
193         value=round(metrics.get('upload_mbps', 0.0), 2),
194         unit='Mbps'
195     )
196     s3.provider = SensorProvider.INTERNET_SPEED
197     sensors.append(s3)
198
199     s4 = HardwareSensor(
200         sensor_id='internet_bufferbloat',
201         name='Internet Bufferbloat',
202         category='network',
203         value=round(metrics.get('bufferbloat_ms', 0.0), 2)
204         ,
205         unit='ms'
206     )
207     s4.provider = SensorProvider.INTERNET_SPEED
208     sensors.append(s4)
209
210     s5 = HardwareSensor(
211         sensor_id='internet_latency_loaded',
212         name='Internet Loaded Latency',
213         category='network',
214         value=round(metrics.get('latency_loaded_ms', 0.0),
215         2),
216         unit='ms'
217     )
218     s5.provider = SensorProvider.INTERNET_SPEED

```

```

217         sensors.append(s5)
218     except Exception as ex:
219         logger.debug(f'Ошибка опроса сенсора скорости интернет
a: {ex}')
220     return sensors
221
222 def _probe_storage_sensors() -> List[HardwareSensor]:
223     """Сбор сенсоров температуры и надежности накопителей.
224
225     :return: Список объектов HardwareSensor для дисков с provi
der marker.
226     """
227     sensors: List[HardwareSensor] = []
228     if os.name != 'nt':
229         return sensors
230     try:
231         from apps.windows.modules.storage_manager.core.windows
_storage_sensor import WindowsStorageSensor
232         sensor = WindowsStorageSensor(timeout_sec=30)
233         disks = sensor.get_physical_disks()
234         for disk in disks:
235             disk_slug = disk.device_id.replace('\\', '_').repl
ace('.', '_')
236             if disk.temperature_c is not None:
237                 s1 = HardwareSensor(sensor_id=f'disk_temp_{dis
k_slug}', name=f'{disk.friendly_name} Temperature', category='
temperature', value=float(disk.temperature_c), unit='°C')
238                 s1.provider = SensorProvider.WINDOWS_STORAGE
239                 sensors.append(s1)
240             if disk.wear_percentage is not None:
241                 s2 = HardwareSensor(sensor_id=f'disk_wear_{dis
k_slug}', name=f'{disk.friendly_name} Wear', category='wear',
value=float(disk.wear_percentage), unit='%')
242                 s2.provider = SensorProvider.WINDOWS_STORAGE
243                 sensors.append(s2)
244     except Exception as ex:
245         logger.debug(f'Ошибка сбора сенсоров накопителей: {ex}
')
```

```

246     return sensors
247
248 def _probe_cloud_storage_sensors() -> List[HardwareSensor]:
249     """Сбор сенсоров свободного и занятого места для облачного
    хранилища OneDrive.
250
251     Returns:
252         List[HardwareSensor]: Список сенсоров для дисков OneDr
ive с provider marker.
253     """
254     import shutil
255     sensors: List[HardwareSensor] = []
256     od_paths: List[str] = []
257     for env_var in ('OneDrive', 'OneDriveConsumer', 'OneDriveC
ommerical'):
258         val = os.environ.get(env_var)
259         if val and os.path.exists(val) and (val not in od_path
s):
260             od_paths.append(val)
261     if not od_paths:
262         home = os.path.expanduser('~')
263         candidate = os.path.join(home, 'OneDrive')
264         if os.path.exists(candidate):
265             od_paths.append(candidate)
266     if os.name == 'nt':
267         try:
268             import winreg
269             for subkey in ('Software\\Microsoft\\OneDrive\\Acc
ounts\\Personal', 'Software\\Microsoft\\OneDrive\\Accounts\\Bu
siness1', 'Software\\Microsoft\\OneDrive'):
270                 try:
271                     with winreg.OpenKey(winreg.HKEY_CURRENT_US
ER, subkey) as k:
272                         val, _ = winreg.QueryValueEx(k, 'UserF
older')
273                         if val and os.path.exists(val) and (va
l not in od_paths):
274                             od_paths.append(val)

```

```

275         except OSError:
276             pass
277     except Exception:
278         pass
279     for idx, od_path in enumerate(od_paths):
280         try:
281             usage = shutil.disk_usage(od_path)
282             free_gb = round(usage.free / 1024 ** 3, 2)
283             total_gb = round(usage.total / 1024 ** 3, 2)
284             used_pct = round(usage.used / max(usage.total, 1)
* 100.0, 1)
285             slug = f'_{idx + 1}' if len(od_paths) > 1 else ''
286             label_suffix = f' ({os.path.basename(od_path)})' i
f len(od_paths) > 1 else ''
287
288             s1 = HardwareSensor(sensor_id=f'onedrive{slug}_fre
e_gb', name=f'OneDrive Free Space{label_suffix}', category='st
orage', value=free_gb, unit='GB')
289             s1.provider = SensorProvider.SENSOR_COLLECTOR
290             sensors.append(s1)
291
292             s2 = HardwareSensor(sensor_id=f'onedrive{slug}_use
d_percent', name=f'OneDrive Used Percent{label_suffix}', categ
ory='storage', value=used_pct, unit='%')
293             s2.provider = SensorProvider.SENSOR_COLLECTOR
294             sensors.append(s2)
295
296             s3 = HardwareSensor(sensor_id=f'onedrive{slug}_tot
al_gb', name=f'OneDrive Total Capacity{label_suffix}', categor
y='storage', value=total_gb, unit='GB')
297             s3.provider = SensorProvider.SENSOR_COLLECTOR
298             sensors.append(s3)
299         except Exception as ex:
300             logger.debug(f'Ошибка сбора сенсора для OneDrive (
{od_path}): {ex}')
301     return sensors
302
303 def get_hardware_sensors() -> List[HardwareSensor]:

```

```
304 |     """Retrieve all available hardware sensors across all supp
    | orted backends.
305 |
306 |     NOTE: GPU sensors (temperature, power, fan) are NOT includ
    | ed here to avoid
307 |     duplication. They are collected via GpuProber in HardwareM
    | onitor.get_gpu_metrics()
308 |     and then normalized by SensorCollector.extract_sensor_read
    | ings().
309 |
310 |     Returns:
311 |         List[HardwareSensor]: Consolidated list of active hard
    | ware sensors.
312 |     """
313 |     all_sensors: List[HardwareSensor] = []
314 |     # GPU sensors excluded - collected via GpuProber to avoid
    | duplicate nvidia-smi calls
315 |     all_sensors.extend(_probe_acpi_thermal_zones())
316 |     all_sensors.extend(_probe_storage_sensors())
317 |     all_sensors.extend(_probe_cloud_storage_sensors())
318 |     all_sensors.extend(_probe_network_sensors())
319 |     all_sensors.extend(_probe_internet_speed_sensors())
320 |     return all_sensors
```

sensor_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensor
  Collector
4 # =====
5 # Description:
6 #   Сбор данных со всех доступных сенсоров и аппаратных провай
  деров.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensor_collector import Sens
    orCollector
11 #
12 #       service = SensorCollector()
13 #
14 # File: sensor_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 08:10:00
20 # =====
21
22 from __future__ import annotations
23 """Сбор данных со всех доступных сенсоров и аппаратных провай
  деров."""
24
25 import re
26 import threading
27 import time
28 from datetime import datetime, timezone
29 from typing import Any, Dict, List, Optional
```



```

30 | try:
31 |     from logger import logger
32 | except ImportError:
33 |     from logger import logger
34 | from .telemetry_config import TelemetryConfigManager
35 | from .sensor_registry import SensorDeduplicator, SensorProvide
   | r
36 | try:
37 |     from apps.windows.modules.hardware.hardware_monitor import
   | HardwareMonitor
38 |     from apps.windows.modules.hardware.lhm_service import LhmS
   | ervice
39 |     from .internet_speed import InternetSpeedSensor
40 | except ImportError as e:
41 |     logger.warning(f'Не удалось импортировать зависимости сенс
   | оров: {e}')
42 |     HardwareMonitor = None
43 |     LhmService = None
44 |     InternetSpeedSensor = None
45 |
46 | class SensorCollector:
47 |     """Коллектор данных со всех сенсоров с нормализованной схе
   | мой датчиков."""
48 |
49 |     def __init__(
50 |         self,
51 |         config_manager: Optional[TelemetryConfigManager] = Non
   | e,
52 |         lhm_service: Optional[Any] = None,
53 |     ) -> None:
54 |         """Инициализирует коллектор сенсоров.
55 |
56 |         Args:
57 |             config_manager: Менеджер конфигурации для настроек
   | и сенсоров.
58 |             lhm_service: Сервис LibreHardwareMonitor (опционал
   | ьно).
59 |         """

```

```

60         self.config_manager = config_manager
61         self._lhm_service = lhm_service or (LhmService() if Lh
mService else None)
62         self._hardware_monitor = HardwareMonitor() if Hardware
Monitor else None
63         self._internet_speed_sensor = InternetSpeedSensor() if
InternetSpeedSensor else None
64         self._last_hardware_snapshot: Optional[Dict[str, Any]]
= None
65         self._lock = threading.Lock()
66
67     def _get_hardware_data(self) -> Dict[str, Any]:
68         """Получает данные от HardwareMonitor.
69
70         Returns:
71             Dict[str, Any]: Данные HardwareMonitor.
72         """
73         if self._hardware_monitor is None:
74             return {}
75         try:
76             snapshot = self._hardware_monitor.get_snapshot(inc
lude_smart=True)
77             return snapshot.to_dict()
78         except Exception as e:
79             logger.error(f'Ошибка получения данных от Hardware
Monitor: {e}')
80             return {}
81
82     def _get_internet_speed_data(self) -> Dict[str, Any]:
83         """Получает данные о скорости интернета и задержке чер
ез FAST.com.
84
85         Returns:
86             Dict[str, Any]: Данные скорости интернета (downloa
d, upload, ping, bufferbloat, loaded latency).
87         """
88         if self._internet_speed_sensor is None:
89             return {'available': False, 'error': 'InternetSpee

```

```

    dSensor not available'}
90         try:
91             metrics = self._internet_speed_sensor.measure_inte
internet_speed()
92             return {
93                 'available': metrics.get('available', True),
94                 'ping_ms': metrics.get('ping_ms', 0.0),
95                 'download_mbps': metrics.get('download_mbps',
0.0),
96                 'upload_mbps': metrics.get('upload_mbps', 0.0)
,
97                 'latency_unloaded_ms': metrics.get('latency_un
loaded_ms', metrics.get('ping_ms', 0.0)),
98                 'latency_loaded_ms': metrics.get('latency_load
ed_ms', 0.0),
99                 'bufferbloat_ms': metrics.get('bufferbloat_ms'
, 0.0),
100                 'provider': metrics.get('provider', 'fast.com'
),
101                 'server': metrics.get('server', ''),
102                 'ip': metrics.get('ip', ''),
103                 'isp': metrics.get('isp', ''),
104             }
105         except Exception as e:
106             logger.error(f'Ошибка получения данных скорости ин
тернета: {e}')
107             return {'available': False, 'error': str(e)}
108
109     def _detect_provider(self, sensor_id: str, category: str,
hardware_type: str) -> SensorProvider:
110         """Определяет провайдера сенсора на основе его ID, кат
егории и источника.
111
112         Args:
113             sensor_id: Идентификатор сенсора.
114             category: Категория сенсора (Temperatures, Load, e
tc.).
115             hardware_type: Тип оборудования (cpu, gpu, etc.).

```

```
116 |
117 |     Returns:
118 |         SensorProvider: Провайдер сенсора.
119 |         """
120 |         # GPU sensors from GpuProber
121 |         if sensor_id.startswith('gpu_') and ('_temp' in sensor
122 | _id or '_load' in sensor_id):
123 |             return SensorProvider.GPU_PROBER
124 |
125 |         # Storage sensors
126 |         if sensor_id.startswith('disk_') or sensor_id.startswi
127 | th('storage_'):
128 |             return SensorProvider.WINDOWS_STORAGE
129 |
130 |         # Internet speed sensors
131 |         if sensor_id.startswith('internet_'):
132 |             return SensorProvider.INTERNET_SPEED
133 |
134 |         # Network sensors
135 |         if sensor_id.startswith('net_'):
136 |             return SensorProvider.NETWORK_SENSOR
137 |
138 |         # CPU/RAM utilization from sensor_collector
139 |         if sensor_id in ('cpu_util_total', 'ram_util_pct', 'sw
140 | ap_util_pct'):
141 |             return SensorProvider.SENSOR_COLLECTOR
142 |
143 |         if sensor_id.startswith('cpu_core_') and '_load' in se
144 | nsor_id:
145 |             return SensorProvider.SENSOR_COLLECTOR
146 |
147 |         if sensor_id.startswith('cpu_freq'):
148 |             return SensorProvider.SENSOR_COLLECTOR
149 |
150 |         # OneDrive sensors
151 |         if sensor_id.startswith('onedrive_'):
152 |             return SensorProvider.SENSOR_COLLECTOR
```

```

150         # Network rates
151         if sensor_id in ('network_sent_rate', 'network_recv_ra
te'):
152             return SensorProvider.SENSOR_COLLECTOR
153
154         # Storage usage
155         if sensor_id.startswith('storage_usage_') or sensor_id
in ('storage_read_rate', 'storage_write_rate'):
156             return SensorProvider.SENSOR_COLLECTOR
157
158         # Default based on category
159         if category in ('Temperatures', 'temperature'):
160             return SensorProvider.ACPI_THERMAL
161
162         return SensorProvider.SENSOR_COLLECTOR
163
164     def extract_sensor_readings(
165         self, hardware_data: Dict[str, Any], *args: Any, inter
net_data: Optional[Dict[str, Any]] = None, **kwargs: Any
166     ) -> List[Dict[str, Any]]:
167         """Извлекает нормализованный список всех текущих показ
аний датчиков с дедупликацией.
168
169         Каждый элемент содержит: id, hardware_name, hardware_t
ype, sensor_category,
170         sensor_name, unit, value, _provider.
171
172         Args:
173             hardware_data: Данные HardwareMonitor.
174             *args: Опциональные extra_sensors_data и/или inter
net_data.
175             internet_data: Данные скорости интернета (если пер
еданы явно).
176
177         Returns:
178             List[Dict[str, Any]]: Нормализованный и дедуплицир
ованный список сенсоров.
179         """

```

```

180         extra_sensors_data = None
181         if len(args) == 1:
182             if isinstance(args[0], dict) and ("ping_ms" in args[0] or "download_mbps" in args[0] or "available" in args[0]):
183                 internet_data = args[0]
184             else:
185                 extra_sensors_data = args[0]
186         elif len(args) >= 2:
187             extra_sensors_data = args[0]
188             internet_data = args[1]
189
190         raw_readings: List[Dict[str, Any]] = []
191
192         # Дополнительные внешние сенсоры
193         if extra_sensors_data and isinstance(extra_sensors_data, dict) and "sensors" in extra_sensors_data:
194             for s in extra_sensors_data["sensors"]:
195                 if isinstance(s, dict):
196                     sensor_id = s.get("id", s.get("sensor_name", "sensor"))
197                     category = s.get("sensor_category", "Metric")
198                     hardware_type = s.get("hardware_type", "sensor")
199                     provider = self._detect_provider(str(sensor_id), category, hardware_type)
200                     raw_readings.append({
201                         "id": sensor_id,
202                         "hardware_name": s.get("hardware_name", "Hardware"),
203                         "hardware_type": hardware_type,
204                         "sensor_category": category,
205                         "sensor_name": s.get("sensor_name", "Sensor"),
206                         "unit": s.get("unit", ""),
207                         "value": s.get("value_num", s.get("value", 0.0)),
208                         "_provider": provider,

```

```

209         })
210
211         if 'cpu' in hardware_data:
212             cpu = hardware_data['cpu']
213             cpu_name = cpu.get('model') or cpu.get('model_name
214             ') or 'Intel Core Processor'
215             raw_readings.append({
216                 'id': 'cpu_util_total',
217                 'hardware_name': cpu_name,
218                 'hardware_type': 'cpu',
219                 'sensor_category': 'Load',
220                 'sensor_name': 'CPU Total',
221                 'unit': '%',
222                 'value': cpu.get('utilization_pct', 0.0),
223                 '_provider': SensorProvider.SENSOR_COLLECTOR
224             })
225             if 'per_core_pct' in cpu and isinstance(cpu['per_c
226             ore_pct'], list):
227                 for idx, core_val in enumerate(cpu['per_core_p
228                 ct']):
229                     raw_readings.append({
230                         'id': f'cpu_core_{idx}_load',
231                         'hardware_name': cpu_name,
232                         'hardware_type': 'cpu',
233                         'sensor_category': 'Load',
234                         'sensor_name': f'CPU Core #{idx}',
235                         'unit': '%',
236                         'value': core_val,
237                         '_provider': SensorProvider.SENSOR_COL
238                         LECTOR
239                     })
240             if 'frequency_current_mhz' in cpu and cpu['frequen
241             cy_current_mhz']:
242                 raw_readings.append({
243                     'id': 'cpu_freq_current',
244                     'hardware_name': cpu_name,
245                     'hardware_type': 'cpu',
246                     'sensor_category': 'Clocks',

```

```

242         'sensor_name': 'CPU Core Frequency',
243         'unit': 'MHz',
244         'value': cpu['frequency_current_mhz'],
245         '_provider': SensorProvider.SENSOR_COLLECT
OR
246     })
247
248     if 'memory' in hardware_data:
249         ram = hardware_data['memory']
250         raw_readings.append({
251             'id': 'ram_util_pct',
252             'hardware_name': 'System Memory',
253             'hardware_type': 'memory',
254             'sensor_category': 'Load',
255             'sensor_name': 'Memory Used',
256             'unit': '%',
257             'value': ram.get('utilization_pct', 0.0),
258             '_provider': SensorProvider.SENSOR_COLLECTOR
259         })
260         if 'swap_utilization_pct' in ram and ram['swap_utilization_pct'] is not None:
261             raw_readings.append({
262                 'id': 'swap_util_pct',
263                 'hardware_name': 'System Memory',
264                 'hardware_type': 'memory',
265                 'sensor_category': 'Load',
266                 'sensor_name': 'Swap Used',
267                 'unit': '%',
268                 'value': ram['swap_utilization_pct'],
269                 '_provider': SensorProvider.SENSOR_COLLECT
OR
270             })
271
272     # GPU metrics: temperature, load, memory, and subsystem engines
273     if 'gpus' in hardware_data and isinstance(hardware_data['gpus'], list):
274         for gpu in hardware_data['gpus']:

```



```

275 |         gpu_name = gpu.get('name', 'GPU')
276 |         gpu_idx = gpu.get('index', 0)
277 |         # GPU temperature - use hardware_data from Gpu
    | Prober
278 |         if 'temperature_gpu_c' in gpu and gpu['tempera
    | ture_gpu_c'] is not None:
279 |             raw_readings.append({
280 |                 'id': f'gpu_{gpu_idx}_temp',
281 |                 'hardware_name': gpu_name,
282 |                 'hardware_type': 'gpu',
283 |                 'sensor_category': 'Temperatures',
284 |                 'sensor_name': 'GPU Core Temperature',
285 |                 'unit': '°C',
286 |                 'value': gpu['temperature_gpu_c'],
287 |                 '_provider': SensorProvider.GPU_PROBER
288 |             })
289 |         # GPU load
290 |         if 'utilization_gpu_pct' in gpu and gpu['utili
    | zation_gpu_pct'] is not None:
291 |             raw_readings.append({
292 |                 'id': f'gpu_{gpu_idx}_load',
293 |                 'hardware_name': gpu_name,
294 |                 'hardware_type': 'gpu',
295 |                 'sensor_category': 'Load',
296 |                 'sensor_name': 'GPU Core Load',
297 |                 'unit': '%',
298 |                 'value': gpu['utilization_gpu_pct'],
299 |                 '_provider': SensorProvider.GPU_PROBER
300 |             })
301 |         # GPU VRAM / Memory
302 |         if 'memory_used_mb' in gpu and gpu['memory_use
    | d_mb'] is not None:
303 |             raw_readings.append({
304 |                 'id': f'gpu_{gpu_idx}_mem_used',
305 |                 'hardware_name': gpu_name,
306 |                 'hardware_type': 'gpu',
307 |                 'sensor_category': 'Data',
308 |                 'sensor_name': 'GPU Memory Used',

```

```

309         'unit': 'MB',
310         'value': float(gpu['memory_used_mb']),
311         '_provider': SensorProvider.GPU_PROBER
312     })
313     if 'memory_total_mb' in gpu and gpu['memory_to
tal_mb'] is not None:
314         raw_readings.append({
315             'id': f'gpu_{gpu_idx}_mem_total',
316             'hardware_name': gpu_name,
317             'hardware_type': 'gpu',
318             'sensor_category': 'Data',
319             'sensor_name': 'GPU Memory Total',
320             'unit': 'MB',
321             'value': float(gpu['memory_total_mb'])
322         },
323         '_provider': SensorProvider.GPU_PROBER
324     })
325     # GPU Subsystem Engines (3D, Video Decode, Vid
eo Processing, Copy, etc.)
326     engines = gpu.get('engines')
327     if engines and isinstance(engines, dict):
328         for eng_name, eng_val in engines.items():
329             clean_id = re.sub(r'^[a-zA-Z0-9]+', '_
', eng_name.lower()).strip('_')
330             raw_readings.append({
331                 'id': f'gpu_{gpu_idx}_eng_{clean_i
d}',
332                 'hardware_name': gpu_name,
333                 'hardware_type': 'gpu',
334                 'sensor_category': 'Load',
335                 'sensor_name': eng_name,
336                 'unit': '%',
337                 'value': float(eng_val),
338                 '_provider': SensorProvider.GPU_PR
OBER
339             })
340     if 'storage' in hardware_data:

```

```

341         storage = hardware_data['storage']
342         if 'partitions' in storage and isinstance(storage[
| 'partitions'], list):
343             for part in storage['partitions']:
344                 dev = part.get('device', 'Disk')
345                 raw_readings.append({
346                     'id': f"storage_usage_{dev.replace(':', '
| , ' ').replace(chr(92), ' ')}",
347                     'hardware_name': f'Storage ({dev})',
348                     'hardware_type': 'storage',
349                     'sensor_category': 'Load',
350                     'sensor_name': f'Used Space {dev}',
351                     'unit': '%',
352                     'value': part.get('utilization_pct', 0
| .0),
353                     '_provider': SensorProvider.SENSOR_COL
| LECTOR
354                 })
355         if 'io_rates' in storage and storage['io_rates']:
356             io = storage['io_rates']
357             raw_readings.append({
358                 'id': 'storage_read_rate',
359                 'hardware_name': 'Storage',
360                 'hardware_type': 'storage',
361                 'sensor_category': 'Throughput',
362                 'sensor_name': 'Disk Read Rate',
363                 'unit': 'B/s',
364                 'value': io.get('read_bytes_sec', 0.0),
365                 '_provider': SensorProvider.SENSOR_COLLECT
| OR
366             })
367             raw_readings.append({
368                 'id': 'storage_write_rate',
369                 'hardware_name': 'Storage',
370                 'hardware_type': 'storage',
371                 'sensor_category': 'Throughput',
372                 'sensor_name': 'Disk Write Rate',
373                 'unit': 'B/s',

```

```

374         'value': io.get('write_bytes_sec', 0.0),
375         '_provider': SensorProvider.SENSOR_COLLECT
OR
376     })
377
378     # Network metrics: aggregated throughput rates
379     # Note: Detailed per-interface stats are collected by
    _probe_network_sensors() in sensors.py
380     if 'network' in hardware_data:
381         net = hardware_data['network']
382         raw_readings.append({
383             'id': 'network_sent_rate',
384             'hardware_name': 'Network',
385             'hardware_type': 'network',
386             'sensor_category': 'Throughput',
387             'sensor_name': 'Network Upload Speed',
388             'unit': 'B/s',
389             'value': net.get('bytes_sent_sec', 0.0),
390             '_provider': SensorProvider.SENSOR_COLLECTOR
391         })
392         raw_readings.append({
393             'id': 'network_recv_rate',
394             'hardware_name': 'Network',
395             'hardware_type': 'network',
396             'sensor_category': 'Throughput',
397             'sensor_name': 'Network Download Speed',
398             'unit': 'B/s',
399             'value': net.get('bytes_recv_sec', 0.0),
400             '_provider': SensorProvider.SENSOR_COLLECTOR
401         })
402
403     # Internet speed metrics
404     if internet_data and internet_data.get('available'):
405         raw_readings.append({
406             'id': 'internet_download_speed',
407             'hardware_name': 'Internet',
408             'hardware_type': 'network',
409             'sensor_category': 'Throughput',

```

```

410         'sensor_name': 'Internet Download Bandwidth',
411         'unit': 'Mbps',
412         'value': internet_data.get('download_mbps', 0.
0),
413         '_provider': SensorProvider.INTERNET_SPEED
414     })
415     raw_readings.append({
416         'id': 'internet_upload_speed',
417         'hardware_name': 'Internet',
418         'hardware_type': 'network',
419         'sensor_category': 'Throughput',
420         'sensor_name': 'Internet Upload Bandwidth',
421         'unit': 'Mbps',
422         'value': internet_data.get('upload_mbps', 0.0)
423     },
424         '_provider': SensorProvider.INTERNET_SPEED
425     })
426     raw_readings.append({
427         'id': 'internet_ping',
428         'hardware_name': 'Internet',
429         'hardware_type': 'network',
430         'sensor_category': 'Throughput',
431         'sensor_name': 'Internet Ping Latency',
432         'unit': 'ms',
433         'value': internet_data.get('ping_ms', 0.0),
434         '_provider': SensorProvider.INTERNET_SPEED
435     })
436     raw_readings.append({
437         'id': 'internet_bufferbloat',
438         'hardware_name': 'Internet',
439         'hardware_type': 'network',
440         'sensor_category': 'Throughput',
441         'sensor_name': 'Internet Bufferbloat',
442         'unit': 'ms',
443         'value': internet_data.get('bufferbloat_ms', 0
.0),
444         '_provider': SensorProvider.INTERNET_SPEED
445     })

```

```

445         raw_readings.append({
446             'id': 'internet_latency_loaded',
447             'hardware_name': 'Internet',
448             'hardware_type': 'network',
449             'sensor_category': 'Throughput',
450             'sensor_name': 'Internet Loaded Latency',
451             'unit': 'ms',
452             'value': internet_data.get('latency_loaded_ms'
    , 0.0),
453             '_provider': SensorProvider.INTERNET_SPEED
454         })
455
456     # Дедупликация базовых показаний по приоритету провайде
    р
457     deduplicator = SensorDeduplicator()
458     unique_readings: List[Dict[str, Any]] = []
459
460     for reading in raw_readings:
461         sensor_id = str(reading.get('id', ''))
462         value = reading.get('value', 0.0)
463         provider = reading.get('_provider', SensorProvider
    .SENSOR_COLLECTOR)
464
465         if deduplicator.add(sensor_id, provider, value):
466             unique_readings.append(reading)
467
468     # Обогащение из LibreHardwareMonitor (добавляются ТОЛЬ
    КО сенсоры, отсутствующие в базовом пуле CIM/WMI)
469     if self._lhm_service and self._lhm_service.is_running(
    ):
470         try:
471             lhm_items = self._lhm_service.get_flattened_se
    nsors()
472             existing_ids = {str(r.get('id', '')) for r in
    unique_readings}
473             for item in lhm_items:
474                 hw_name = item.get("hardware_name", "Hardw
    are")

```

```

475         hw_type = item.get("hardware_type", "sensor")
476         cat = item.get("sensor_category", "General")
477         s_name = item.get("sensor_name", "Sensor")
478
479         if "Distance to TjMax" in s_name:
480             continue
481
482         val_num = item.get("value_num")
483         if val_num is None:
484             continue
485
486         vraw = str(item.get("value_raw") or "").lower()
487         unit = ""
488         if "°c" in vraw or "c" in vraw:
489             unit = "°C"
490         elif "%" in vraw:
491             unit = "%"
492         elif "rpm" in vraw:
493             unit = "RPM"
494         elif "v" in vraw and "w" not in vraw:
495             unit = "V"
496         elif "w" in vraw:
497             unit = "W"
498         elif "mhz" in vraw:
499             unit = "MHz"
500         elif "b/s" in vraw:
501             unit = "B/s"
502         elif "gb" in vraw:
503             unit = "GB"
504
505         clean_hw = re.sub(r'^[a-zA-Z0-9]+', '_', hw_name.lower()).strip('_')
506         clean_name = re.sub(r'^[a-zA-Z0-9]+', '_', s_name.lower()).strip('_')
507         sid = f"lhm_{clean_hw}_{clean_name}"

```

```

508 |
509 |             # Пропускаем, если такой датчик уже есть и
    | ли это дубликат общей нагрузки CPU/RAM
510 |             if sid in existing_ids or (cat == "Load" a
    | nd "total" in clean_name and any("util_total" in eid for eid i
    | n existing_ids)):
511 |                 continue
512 |
513 |             existing_ids.add(sid)
514 |             unique_readings.append({
515 |                 'id': sid,
516 |                 'hardware_name': hw_name,
517 |                 'hardware_type': hw_type,
518 |                 'sensor_category': cat,
519 |                 'sensor_name': s_name,
520 |                 'unit': unit,
521 |                 'value': float(val_num),
522 |                 '_provider': SensorProvider.LHM_ENRICH
    | ED,
523 |                 })
524 |             except Exception as lhm_err:
525 |                 logger.debug(f'Ошибка обогащения сенсорами LHM
    | : {lhm_err}')
526 |
527 |             return unique_readings
528 |
529 |     def get_hardware_snapshot(self) -> Dict[str, Any]:
530 |         """Получает полный снапшот аппаратного состояния со сп
    | иском сенсоров.
531 |
532 |         Returns:
533 |             Dict[str, Any]: Снапшот со списком показаний сенсо
    | ров.
534 |         """
535 |         with self._lock:
536 |             now_iso = datetime.now(timezone.utc).isoformat()
537 |             hardware_data = self._get_hardware_data()
538 |             internet_data = self._get_internet_speed_data()

```



```

539         sensor_readings = self.extract_sensor_readings(har
|         dware_data, internet_data)
540         sensors_payload = []
541         for r in sensor_readings:
542             sensors_payload.append({'id': r.get('id'), 'ha
|             rdware_name': r.get('hardware_name'), 'hardware_type': r.get('
|             hardware_type'), 'sensor_category': r.get('sensor_category'),
|             'sensor_name': r.get('sensor_name'), 'unit': r.get('unit'), 'v
|             alues': [{'num': r.get('value', 0.0), 'time': now_iso}]})
543             result = {'timestamp': now_iso, 'sensors': sensors
|             _payload, 'hardware_monitor': hardware_data if hardware_data e
|             lse {}, 'internet_speed': internet_data if internet_data else
|             {}}
544             self._last_hardware_snapshot = result
545             return result
546
547     def get_last_snapshot(self) -> Optional[Dict[str, Any]]:
548         """Возвращает последний собранный снапшот.
549
550         Returns:
551             Optional[Dict[str, Any]]: Последний снапшот или No
|         ne.
552         """
553         return self._last_hardware_snapshot
554
555     def collect_all_sensors(self) -> List[Dict[str, Any]]:
556         """Собирает показания всех доступных датчиков.
557
558         Returns:
559             List[Dict[str, Any]]: Список прочитанных сенсоров.
560         """
561         with self._lock:
562             hardware_data = self._get_hardware_data()
563             internet_data = self._get_internet_speed_data()
564             return self.extract_sensor_readings(hardware_data,
|             internet_data)
565
566     def collect_lhm_cpu_readings(self) -> List[Dict[str, Any]]

```

```

:
567 |         """Извлекает нормализованные показания температур ядер
      | CPU и Package из LHM.
568 |
569 |         Returns:
570 |             List[Dict[str, Any]]: Список показаний датчиков те
      | мператур.
571 |         """
572 |         from apps.windows.modules.hardware.lhm_service import
      | LhmService
573 |         lhm = self._lhm_service or LhmService()
574 |         items = lhm.get_flattened_sensors()
575 |         results: List[Dict[str, Any]] = []
576 |         core_idx = 0
577 |         for item in items:
578 |             if item.get("hardware_type") != "cpu" or item.get(
      | "sensor_category") != "Temperatures":
579 |                 continue
580 |             name = item.get("sensor_name", "")
581 |             if "Distance to TjMax" in name:
582 |                 continue
583 |             val = item.get("value_num")
584 |             if val is None:
585 |                 continue
586 |             if "Package" in name:
587 |                 results.append({
588 |                     "id": "cpu_package_temp",
589 |                     "hardware_name": item.get("hardware_name",
      | "CPU"),
590 |                     "hardware_type": "cpu",
591 |                     "sensor_category": "Temperatures",
592 |                     "sensor_name": name,
593 |                     "unit": "°C",
594 |                     "value": val,
595 |                     "value_num": val,
596 |                 })
597 |             elif "Core" in name:
598 |                 results.append({

```

```
599 |         "id": f"cpu_core_{core_idx}_temp",
600 |         "hardware_name": item.get("hardware_name",
    | "CPU"),
601 |         "hardware_type": "cpu",
602 |         "sensor_category": "Temperatures",
603 |         "sensor_name": name,
604 |         "unit": "°C",
605 |         "value": val,
606 |         "value_num": val,
607 |     })
608 |     core_idx += 1
609 | return results
610 |
```

sensor_registry.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sensor
  Registry
4 # =====
5 # Description:
6 #   Реестр сенсоров и политика дедупликации.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sensor_registry import SensorProvider
11 #
12 #       service = SensorProvider()
13 #
14 # File: sensor_registry.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-04 07:00:00
20 # =====
21
22 from __future__ import annotations
23 """Реестр сенсоров и политика дедупликации."""
24
25 from dataclasses import dataclass, field
26 from enum import Enum
27 from typing import Dict, List, Optional, Set, Tuple
28 from datetime import datetime, timezone
29
30
31 class SensorProvider(Enum):
```

```

32 |     """Приоритеты и категории провайдеров сенсоров.
33 |
34 |     Приоритеты распределены так:
35 |     - INTERNET_SPEED (55): Выделенный speedtest
36 |     - CIM_SYSTEM (50): Основной современный нативный интерфейс
    |     Windows (CIM/MSFT)
37 |     - WINDOWS_STORAGE (45): WindowsStorageSensor (диски, SMART
    |     , температуры NVMe)
38 |     - GPU_PROBER (40): Прямой зонд GPU (NVIDIA/AMD/Intel)
39 |     - ACPI_THERMAL (35): Системные ACPI ThermalZones
40 |     - HARDWARE_MONITOR (35): Агрегатор аппаратных метрик
41 |     - SENSOR_COLLECTOR (30): Базовый коллектор системных метри
    |     к
42 |     - WMI_FALLBACK (30): WMI слой совместимости / fallback при
    |     недоступности CIM
43 |     - NETWORK_SENSOR (25): Сетевые интерфейсы psutil
44 |     - LHM_ENRICHED (20): Обогащение отсутствующими в CIM/WMI с
    |     енсорами из LibreHardwareMonitor
45 |     - NVIDIA_SMI_DIRECT (10): Прямой вызов утилит
46 |     """
47 |     INTERNET_SPEED = 55           # InternetSpeedSensor - приори
    |     тет для speedtest метрик
48 |     CIM_SYSTEM = 50               # Основной интерфейс CIM (Powe
    |     rShell / CimSession)
49 |     WINDOWS_STORAGE = 45         # WindowsStorageSensor (SMART,
    |     temps) - приоритет для storage
50 |     GPU_PROBER = 40               # GpuProber - приоритет для GP
    |     U метрик
51 |     ACPI_THERMAL = 35             # ACPI thermal zones
52 |     HARDWARE_MONITOR = 35        # HardwareMonitor (агрегатор)
53 |     SENSOR_COLLECTOR = 30        # SensorCollector.extract_sens
    |     or_readings()
54 |     WMI_FALLBACK = 30            # WMI слой совместимости
55 |     NETWORK_SENSOR = 25          # Network sensors через psutil
56 |     LHM_ENRICHED = 20            # Обогащение детальными датчик
    |     ами из LibreHardwareMonitor
57 |     NVIDIA_SMI_DIRECT = 10       # Прямой вызов nvidia-smi (leg
    |     асу)

```

```

58 |
59 |
60 | @dataclass
61 | class SensorDefinition:
62 |     """Определение сенсора с политикой дедупликации."""
63 |     sensor_id: str
64 |     category: str
65 |     preferred_provider: SensorProvider
66 |     description: str = ""
67 |     unit: str = ""
68 |     allow_multiple_sources: bool = False # Если True, дублика
ты разрешены с разными провайдерами
69 |     dedup_window_seconds: float = 1.0 # Окно дедупликации
по времени
70 |
71 |
72 | class SensorRegistry:
73 |     """Централизованный реестр сенсоров с политикой дедупликац
ии."""
74 |
75 |     # Реестр известных сенсоров с предпочтительными провайдера
ми
76 |     _registry: Dict[str, SensorDefinition] = {}
77 |
78 |     # Мappings категорий на предпочтительные провайдеры
79 |     _category_providers: Dict[str, SensorProvider] = {
80 |         "temperature": SensorProvider.ACPI_THERMAL,
81 |         "Temperatures": SensorProvider.ACPI_THERMAL,
82 |         "gpu_temp": SensorProvider.GPU_PROBER,
83 |         "gpu_load": SensorProvider.GPU_PROBER,
84 |         "gpu_power": SensorProvider.GPU_PROBER,
85 |         "gpu_fan": SensorProvider.GPU_PROBER,
86 |         "cpu_temp": SensorProvider.ACPI_THERMAL,
87 |         "cpu_load": SensorProvider.SENSOR_COLLECTOR,
88 |         "Load": SensorProvider.SENSOR_COLLECTOR, # CPU/RAM ut
ilization
89 |         "Clocks": SensorProvider.SENSOR_COLLECTOR,
90 |         "network_throughput": SensorProvider.SENSOR_COLLECTOR,

```

```

91         "network_bytes": SensorProvider.NETWORK_SENSOR,
92         "Throughput": SensorProvider.SENSOR_COLLECTOR,
93         "internet": SensorProvider.INTERNET_SPEED,
94         "storage_temp": SensorProvider.WINDOWS_STORAGE,
95         "storage_wear": SensorProvider.WINDOWS_STORAGE,
96         "storage_usage": SensorProvider.SENSOR_COLLECTOR,
97         "storage": SensorProvider.SENSOR_COLLECTOR,
98     }
99
100     # Паттерны дублирующихся sensor_id
101     _duplicate_patterns: Dict[str, List[str]] = {
102         # GPU метрики - собираются предпочтительно через GpuPr
103         "gpu_*": ["gpu_prober", "sensor_collector", "nvidia_s
104         mi_direct"],
105         # CPU метрики
106         "cpu_*": ["acpi_thermal", "sensor_collector"],
107         # Internet ping - дублируется в sensors.py и sensor_co
108         llector
109         "internet_ping": ["internet_speed", "sensor_collector"
110         ],
111         # Network throughput - дублируется в sensors.py и sens
112         or_collector
113         "network_*_rate": ["network_sensor", "sensor_collecto
114         r"],
115     }
116
117     @classmethod
118     def register(cls, definition: SensorDefinition) -> None:
119         """Регистрирует сенсор в реестре.
120
121         Args:
122             definition: Определение сенсора с политикой.
123         """
124         cls._registry[definition.sensor_id] = definition
125
126     @classmethod
127     def get_preferred_provider(cls, sensor_id: str, category:

```

```

    str) -> Optional[SensorProvider]:
123     """Возвращает предпочтительный провайдер для данного с
    енсора.
124
125     Args:
126         sensor_id: Идентификатор сенсора.
127         category: Категория сенсора.
128
129     Returns:
130         Preferred provider или None если не определен.
131     """
132     # 1. Проверяем явную регистрацию
133     if sensor_id in cls._registry:
134         return cls._registry[sensor_id].preferred_provider
135
136     # 2. Определяем по специфическому паттерну sensor_id
137     import re
138     for pattern, providers in cls._duplicate_patterns.item
139 s():
140         if re.match(pattern, sensor_id):
141             # Возвращаем первый из списка как предпочтител
142             ьный
143             provider_name = providers[0].upper()
144             try:
145                 return SensorProvider(provider_name)
146             except KeyError:
147                 pass
148
149     # 3. Проверяем по категории
150     if category in cls._category_providers:
151         return cls._category_providers[category]
152
153     return None
154
155     @classmethod
156     def should_keep_sensor(
157         cls,
158         sensor_id: str,

```



```

157         provider: SensorProvider,
158         existing_sensors: List[Tuple[str, SensorProvider]]
159     ) -> bool:
160         """Определяет, нужно ли сохранить показание сенсора ил
и отклонить как дубликат.
161
162     Args:
163         sensor_id: Идентификатор сенсора.
164         provider: Провайдер, предоставивший показание.
165         existing_sensors: Список уже собранных (sensor_id,
provider) пар.
166
167     Returns:
168         True если сенсор нужно сохранить, False если это д
убликат.
169         """
170         # Проверяем, есть ли уже такой sensor_id
171         for existing_id, existing_provider in existing_sensors
:
172             if existing_id == sensor_id:
173                 # Дубликат найден - сравниваем приоритеты пров
айдеров
174                 preferred = cls.get_preferred_provider(sensor_
id, "")
175                 if preferred:
176                     # Сохраняем только от предпочтительного пр
овайдера
177                     return provider == preferred
178                 else:
179                     # Нет явной политики - сохраняем от провай
дера с высшим приоритетом
180                     return provider.value > existing_provider.
value
181
182         # Дубликатов нет - сохраняем
183         return True
184
185

```

```

186 class SensorDeduplicator:
187     """Дедупликатор показаний сенсоров."""
188
189     def __init__(self) -> None:
190         """Инициализирует дедупликатор с пустым кешем."""
191         self._seen: Dict[str, Tuple[float, SensorProvider, flo
192 at]] = {} # sensor_id -> (timestamp, provider, value)
193
194     def add(
195         self,
196         sensor_id: str,
197         provider: SensorProvider,
198         value: float,
199         timestamp: Optional[float] = None,
200         dedup_window: float = 1.0
201     ) -> bool:
202         """Пытается добавить показание сенсора с проверкой на
203 дубликаты.
204
205     Args:
206         sensor_id: Идентификатор сенсора.
207         provider: Провайдер показания.
208         value: Значение показания.
209         timestamp: Временная метка (epoch seconds).
210         dedup_window: Окно дедупликации в секундах.
211
212     Returns:
213         True если показание добавлено, False если отклонен
214 о как дубликат.
215
216     """
217     if timestamp is None:
218         timestamp = datetime.now(timezone.utc).timestamp()
219
220     # Проверяем наличие в кеше
221     if sensor_id in self._seen:
222         cached_ts, cached_provider, cached_value = self._s
223 een[sensor_id]

```

```

220         # Проверяем временное окно
221         if abs(timestamp - cached_ts) < dedup_window:
222             # В пределах окна - проверяем приоритеты
223             preferred = SensorRegistry.get_preferred_provi
der(sensor_id, "")
224             if preferred:
225                 # Если текущий провайдер предпочтительнее
- обновляем
226                 if provider == preferred:
227                     self._seen[sensor_id] = (timestamp, pr
ovider, value)
228                     return True
229                 else:
230                     # Отклоняем
231                     return False
232             else:
233                 # Сравниваем по значению приоритета
234                 if provider.value > cached_provider.value:
235                     self._seen[sensor_id] = (timestamp, pr
ovider, value)
236                     return True
237                 else:
238                     return False
239
240         # Нет дубликата или вне окна - добавляем
241         self._seen[sensor_id] = (timestamp, provider, value)
242         return True
243
244     def get_unique(self) -> Dict[str, Tuple[SensorProvider, fl
oat]]:
245         """Возвращает уникальные показания сенсоров.
246
247         Returns:
248             Словарь {sensor_id: (provider, value)}.
249         """
250         return {sid: (prov, val) for sid, (_, prov, val) in se
lf._seen.items()}
251

```

```
252 |     def clear(self) -> None:
253 |         """Очищает кеш дедупликатора."""
254 |         self._seen.clear()
255 |
256 |
257 | def deduplicate_sensor_readings(
258 |     readings: List[Dict],
259 |     provider: SensorProvider = SensorProvider.SENSOR_COLLECTOR
260 | ) -> List[Dict]:
261 |     """Дедуплицирует список показаний сенсоров.
262 |
263 |     Args:
264 |         readings: Список показаний с полями 'id' и 'value'.
265 |         provider: Провайдер показаний.
266 |
267 |     Returns:
268 |         Дедуплицированный список показаний.
269 |     """
270 |     deduplicator = SensorDeduplicator()
271 |     unique_readings: List[Dict] = []
272 |
273 |     for reading in readings:
274 |         sensor_id = reading.get('id', '')
275 |         value = reading.get('value', 0.0)
276 |
277 |         if deduplicator.add(sensor_id, provider, value):
278 |             unique_readings.append(reading)
279 |
280 |     return unique_readings
281 |
```

```

service.py
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Service
4 # =====
5 # Description:
6 #   Фоновый сервис посекундного сбора системных метрик с сохра
   нением в SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.service import TelemetryLogg
   erService
11 #
12 #       instance = TelemetryLoggerService.get_instance()
13 #
14 # File: service.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 13:59:00
20 # =====
21
22 from __future__ import annotations
23 """Фоновый сервис периодического сбора системных метрик с сохр
   анением в SQLite."""
24
25 import asyncio
26 import threading
27 import time
28 from typing import Any, Dict, List, Optional
29 try:
30     from logger import logger

```

```

31 except ImportError:
32     from logger import logger
33 from .collector import SystemCollector
34 from apps.windows.telemetry_research.hardware_history_manager
    import HardwareHistoryManager
35 from .sqlite import TelemetryStorage
36 from .telemetry_config import TelemetryConfigManager
37 from .w64_collector import AIW64Collector
38 from .w64_etw_collector import AIW64ETWCollector
39
40 class TelemetryLoggerService:
41     """Сервис периодического сбора системной телеметрии, аудит
    а оборудования и W64/ETW событий в SQLite."""
42     _instance: Optional[TelemetryLoggerService] = None
43
44     def __init__(
45         self,
46         interval_sec: Optional[float] = None,
47         top_processes: Optional[int] = None,
48         collector: Optional[SystemCollector] = None,
49         storage: Optional[TelemetryStorage] = None,
50         hardware_audit_interval_sec: Optional[float] = None,
51         rollup_interval_sec: Optional[float] = None,
52         enable_w64: Optional[bool] = None,
53         config_manager: Optional[TelemetryConfigManager] = Non
    e,
54         db_cleanup_interval_sec: Optional[float] = None,
55     ) -> None:
56         """Инициализирует сервис сбора системных метрик.
57
58         Args:
59             interval_sec: Интервал между замерами телеметрии в
    секундах (по умолчанию из config.json).
60             top_processes: Лимит сохраняемых активных процессо
    в (по умолчанию из config.json).
61             collector: Экземпляр сборщика SystemCollector (опц
    ионально).
62             storage: Экземпляр хранилища TelemetryStorage (опц

```

ионально).

```
63         hardware_audit_interval_sec: Интервал периодическо
го аудита железа (по умолчанию из config.json).
64         rollup_interval_sec: Интервал запуска обобщения ст
арых данных (по умолчанию из config.json).
65         enable_w64: Включение подсистемы сбора событий W64
/ETW (по умолчанию из config.json).
66         config_manager: Менеджер конфигурации.
67         db_cleanup_interval_sec: Интервал фонового контрол
я размера базы данных (сек).
68         """
69         self.config_manager = config_manager or TelemetryConfi
gManager()
70         self._custom_interval_set = (interval_sec is not None)
71         self.interval_sec = max(
72             0.1,
73             interval_sec if interval_sec is not None else self
.config_manager.get_interval_seconds()
74         )
75         self.top_processes = max(
76             1,
77             top_processes if top_processes is not None else se
lf.config_manager.get_effective_process_limit()
78         )
79         self.hardware_audit_interval_sec = max(
80             1.0,
81             hardware_audit_interval_sec if hardware_audit_inte
rval_sec is not None else self.config_manager.get_heavy_interv
al_seconds()
82         )
83         self.rollup_interval_sec = max(
84             1.0,
85             rollup_interval_sec if rollup_interval_sec is not
None else self.config_manager.get_aggregation_interval()
86         )
87         self.collector = collector or SystemCollector()
88         self.storage = storage or TelemetryStorage.get_instanc
e()
```

```

89         self.db_cleanup_interval_sec = max(
90             5.0,
91             db_cleanup_interval_sec if db_cleanup_interval_sec
is not None else self.config_manager.get_db_cleanup_interval_
seconds()
92         )
93
94         cfg = self.config_manager.get_config()
95         w64_cfg = cfg.get('w64_collector', {})
96         self.enable_w64 = enable_w64 if enable_w64 is not None
else w64_cfg.get('enabled', True)
97
98         self._w64_collector: Optional[AIW64Collector] = None
99         self._w64_etw_collector: Optional[AIW64ETWCollector] =
None
100
101         self._running = False
102         self._thread: Optional[threading.Thread] = None
103         self._stop_event = threading.Event()
104         self._ticks_count = 0
105         self._start_time: Optional[float] = None
106         self._last_tick_time: Optional[float] = None
107         self._last_hw_audit_time: Optional[float] = None
108         self._last_rollup_time: Optional[float] = None
109         self._last_db_cleanup_time: Optional[float] = None
110         self._last_error: Optional[str] = None
111         self._last_snapshot: Optional[Any] = None
112
113     @classmethod
114     def get_instance(cls) -> TelemetryLoggerService:
115         """Возвращает глобальный синглтон сервиса телеметрии.
116
117         Returns:
118             TelemetryLoggerService: Единственный экземпляр сер
виса.
119         """
120         if cls._instance is None:
121             cls._instance = TelemetryLoggerService()

```



```

122         return cls._instance
123
124     @property
125     def is_running(self) -> bool:
126         """Проверяет, запущен ли фоновый сбор телеметрии.
127
128         Returns:
129             bool: True если сбор активен, иначе False.
130         """
131         return self._running and self._thread is not None and
self._thread.is_alive()
132
133     @property
134     def history_manager(self) -> HardwareHistoryManager:
135         """Возвращает менеджер истории оборудования."""
136         return self.collector.history_manager
137
138     @property
139     def w64_collector(self) -> Optional[AIW64Collector]:
140         """Возвращает сборщик W64 событий (создается лениво пр
и enable_w64)."""
141         if self._w64_collector is None and self.enable_w64:
142             self._init_w64_collectors()
143         return self._w64_collector
144
145     @w64_collector.setter
146     def w64_collector(self, val: Optional[AIW64Collector]) ->
None:
147         self._w64_collector = val
148
149     @property
150     def w64_etw_collector(self) -> Optional[AIW64ETWCollector]
:
151         """Возвращает сборщик ETW событий (создается лениво пр
и enable_w64)."""
152         if self._w64_etw_collector is None and self.enable_w64
:
153             self._init_w64_collectors()

```

```

154         return self._w64_etw_collector
155
156     @w64_etw_collector.setter
157     def w64_etw_collector(self, val: Optional[AIW64ETWCollector]) -> None:
158         self._w64_etw_collector = val
159
160     def _init_w64_collectors(self) -> None:
161         """Инициализирует W64 и ETW сборщики при необходимости
162         ."""
163         if not self.enable_w64:
164             return
165         cfg = self.config_manager.get_config()
166         w64_cfg = cfg.get('w64_collector', {})
167         if self._w64_collector is None:
168             self._w64_collector = AIW64Collector(
169                 storage=self.storage,
170                 enable_file_monitoring=w64_cfg.get('enable_file_monitoring', True),
171                 enable_process_monitoring=w64_cfg.get('enable_process_monitoring', True),
172                 enable_registry_monitoring=w64_cfg.get('enable_registry_monitoring', True),
173                 enable_network_monitoring=w64_cfg.get('enable_network_monitoring', True),
174                 enable_event_log_monitoring=w64_cfg.get('enable_event_log_monitoring', True),
175             )
176         if self._w64_etw_collector is None:
177             self._w64_etw_collector = AIW64ETWCollector(
178                 storage=self.storage,
179                 enable_process_trace=w64_cfg.get('enable_process_trace', True),
180                 enable_disk_trace=w64_cfg.get('enable_disk_trace', True),
181                 enable_network_trace=w64_cfg.get('enable_network_trace', True),
182                 enable_registry_trace=w64_cfg.get('enable_registry_trace', True)

```

```

stry_trace', True),
182         )
183
184     def start(self) -> bool:
185         """Запускает фоновый поток сбора телеметрии, аудита об
орудования и W64/ETW сборщиков.
186
187         Returns:
188             bool: True если сервис успешно запущен, False если
уже работал.
189         """
190         if self.is_running:
191             logger.debug('Сервис сбора телеметрии уже запущен.
')
192             return False
193         self._stop_event.clear()
194         self._running = True
195         self._start_time = time.time()
196         self._ticks_count = 0
197         self._last_error = None
198         self._last_hw_audit_time = None
199         self._last_db_cleanup_time = None
200
201         if self.enable_w64:
202             self._init_w64_collectors()
203             if self.w64_collector:
204                 self.w64_collector.start()
205             if self.w64_etw_collector:
206                 self.w64_etw_collector.start()
207
208         # При каждом старте телеметрии фиксируем снимок автоза
пуска в хранилище
209         try:
210             startup_entry = self.collector.archive_startup_sta
te(auto_diff=True)
211             if startup_entry:
212                 self.storage.save_startup_archive(startup_entr
y)

```

```

213         except Exception as st_init_ex:
214             logger.debug(f'Фиксация снимка автозапуска при ста
рте телеметрии: {st_init_ex}')
215
216         self._thread = threading.Thread(target=self._worker_lo
op, name='TelemetryLoggerWorker', daemon=True)
217         self._thread.start()
218         logger.info(f'Фоновый сервис телеметрии запущен в БД (
интервал: {self.interval_sec}с, аудит железа: {self.hardware_a
udit_interval_sec}с, W64: {self.enable_w64}, БД: {self.storage
.db_path})')
219         return True
220
221     def stop(self) -> bool:
222         """Останавливает фоновый поток сбора телеметрии и W64/
ETW сборщиков.
223
224         Returns:
225             bool: True если сервис был остановлен, False если
он не работал.
226         """
227         if not self._running:
228             return False
229         self._running = False
230         self._stop_event.set()
231
232         if self.w64_collector:
233             self.w64_collector.stop()
234         if self.w64_etw_collector:
235             self.w64_etw_collector.stop()
236
237         if self._thread and self._thread.is_alive():
238             self._thread.join(timeout=3.0)
239         self._thread = None
240         try:
241             self.storage.flush()
242         except Exception:
243             pass

```

```

244         logger.info(f'Фоновый сервис телеметрии остановлен. Вс
его тиков: {self._ticks_count}')
245         return True
246
247     def sync_config_from_manager(self) -> bool:
248         """Синхронизирует интервалы и настройки сервиса из мен
еджера конфигурации при их изменении.
249
250         Returns:
251             bool: True если конфигурация изменилась и была при
менена, иначе False.
252         """
253         self.config_manager.check_tc_mode_auto_switch()
254         if self.config_manager.check_and_reload():
255             if not self._custom_interval_set:
256                 self.interval_sec = max(0.1, self.config_manag
er.get_interval_seconds())
257                 self.top_processes = max(1, self.config_manager.ge
t_effective_process_limit())
258                 self.hardware_audit_interval_sec = max(1.0, self.c
onfig_manager.get_heavy_interval_seconds())
259                 self.db_cleanup_interval_sec = max(5.0, self.conf
ig_manager.get_db_cleanup_interval_seconds())
260                 self.rollup_interval_sec = max(1.0, self.config_ma
nager.get_aggregation_interval())
261                 logger.info(
262                     f" [Сервис телеметрии] Конфигурация config.j
son обновлена 'на лету': "
263                     f"базовый={self.interval_sec}с, аудит железа={
self.hardware_audit_interval_sec}с, "
264                     f"очистка БД={self.db_cleanup_interval_sec}с,
топ процессов={self.top_processes}, mode={self.config_manager.
get_telemetry_mode()}"
265                 )
266                 return True
267             return False
268
269     def _worker_loop(self) -> None:

```

[illegible]

```

    (archive_entry)
299         except Exception as hw_ex:
300             logger.debug(f'Ошибка при периодическо
м аудите железа: {hw_ex}')
301         try:
302             startup_archive = self.collector.archi
ve_startup_state(auto_diff=True)
303             if startup_archive:
304                 self.storage.save_startup_archive(
startup_archive)
305         except Exception as st_ex:
306             logger.debug(f'Ошибка при периодическо
м аудите автозапуска: {st_ex}')
307         try:
308             self.collector.get_extended_system_aud
it()
309         except Exception as ext_ex:
310             logger.debug(f'Ошибка при периодическо
м расширенном аудите: {ext_ex}')
311             self._last_hw_audit_time = loop_start
312             if self._last_rollup_time is None or loop_star
t - self._last_rollup_time >= self.rollup_interval_sec:
313                 try:
314                     self.run_rollups()
315                     self._last_rollup_time = loop_start
316                 except Exception as roll_ex:
317                     logger.debug(f'Ошибка при периодическо
м обобщении процессов: {roll_ex}')
318             if self._last_db_cleanup_time is None or loop_
start - self._last_db_cleanup_time >= self.db_cleanup_interval
_sec:
319                 try:
320                     self.cleanup_db()
321                     self._last_db_cleanup_time = loop_star
t
322                 except Exception as clean_ex:
323                     logger.debug(f'Ошибка при периодическо
м контроле размера БД: {clean_ex}')

```

```

324         except (RuntimeError, ValueError) as shut_ex:
325             if 'shutdown' in str(shut_ex).lower() or 'clos
ed file' in str(shut_ex).lower():
326                 break
327             self._last_error = str(shut_ex)
328             logger.debug(f'Ошибка при сборе системной теле
метрии: {shut_ex}')
329         except Exception as ex:
330             self._last_error = str(ex)
331             logger.debug(f'Ошибка при сборе системной теле
метрии: {ex}')
332         elapsed = time.time() - loop_start
333         sleep_time = max(0.01, self.interval_sec - elapsed
)
334         if self._stop_event.wait(timeout=sleep_time):
335             break
336
337     def run_rollups(self) -> Dict[str, Any]:
338         """Принудительно запускает процедуры обобщения устарев
ших метрик процессов.
339
340         Returns:
341             Dict[str, Any]: Сводная статистика обобщения за 2
минуты и за 1 день.
342         """
343         short_res = self.storage.aggregate_process_metrics_2mi
n(cutoff_seconds=120)
344         daily_res = self.storage.aggregate_process_metrics_dai
ly(cutoff_days=1)
345         return {'rollup_2min': short_res, 'rollup_daily': dail
y_res}
346
347     def cleanup_db(self) -> Dict[str, Any]:
348         """Запускает процедуру контроля размера и усечения баз
ы данных телеметрии.
349
350         Returns:
351             Dict[str, Any]: Отчет о результатах контроля разме

```



```

pa.
352         """
353         return self.storage.enforce_size_limit()
354
355     def get_status(self) -> Dict[str, Any]:
356         """Возвращает текущий статус сервиса телеметрии.
357
358         Returns:
359             Dict[str, Any]: Словарь с состоянием и статистикой
360                             работы сервиса.
361         """
362         uptime = round(time.time() - self._start_time, 1) if s
363         elf._start_time and self.is_running else 0.0
364         return {'is_running': self.is_running, 'interval_sec':
365                 self.interval_sec, 'top_processes_limit': self.top_processes,
366                 'hardware_audit_interval_sec': self.hardware_audit_interval_s
367                 ec, 'ticks_recorded': self._ticks_count, 'uptime_seconds': upt
368                 ime, 'last_tick_epoch': self._last_tick_time, 'last_hw_audit_e
369                 poch': self._last_hw_audit_time, 'last_error': self._last_erro
370                 r, 'storage_stats': self.storage.get_storage_stats()}
371
372     def get_last_snapshot(self) -> Optional[Any]:
373         """Возвращает последний собранный снапшот.
374
375         Returns:
376             Optional[Any]: Последний снапшот или None.
377         """
378         return self._last_snapshot
379
380     def get_history(self, limit: int=60) -> List[Dict[str, Any
381     ]]:
382         """Возвращает историю системных снимков из SQLite базы
383         данных.
384
385         Args:
386             limit: Количество последних записей.
387
388         Returns:

```

```
379 |         List[Dict[str, Any]]: Список исторических срезов.
380 |         """
381 |         return self.storage.get_snapshots(limit=limit)
382 |
383 |     def record_event(self, event_type: str, event_details: Dict[str, Any], severity: str='info') -> int:
384 |         """Записывает событие в SQLite базу данных телеметрии.
385 |
386 |         Args:
387 |             event_type: Тип события (hardware_change, process_start, driver_update).
388 |             event_details: Детали события.
389 |             severity: Уровень серьезности события.
390 |
391 |         Returns:
392 |             int: ID добавленной записи события.
393 |         """
394 |         logger.info(f'[Событие Telemetry] {event_type}: {event_details}')
395 |         try:
396 |             return self.storage.save_event(event_type, event_details, severity=severity)
397 |         except Exception as ex:
398 |             logger.error(f'Не удалось записать событие в базу данных: {ex}')
399 |             return 0
```

aggregator.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Aggregator
4 # =====
5 # Description:
6 #   Многоуровневый SQL-агрегатор временных рядов телеметрии (r
  aw -> hourly ->
7 #   daily -> weekly -> monthly -> yearly) и планировщик агрега
  ции.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.sqlite.aggregator import Tel
  emetrySqlAggregator
12 #
13 #       agg = TelemetrySqlAggregator(connection_manager)
14 #       agg.run_pipeline()
15 #
16 # File: aggregator.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry.sqlite
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-06 05:32:00
22 # =====
23
24 from __future__ import annotations
25
26 """Многоуровневый SQL-агрегатор временных рядов телеметрии и п
  ланировщик агрегации."""
27
28 import sqlite3
```

```

29 import threading
30 import time
31 from datetime import datetime, timezone
32 from enum import Enum
33 from typing import Any, Dict, List, Optional, Union
34
35 from .connection import TelemetryConnectionManager
36
37 try:
38     from logger import logger
39 except ImportError:
40     import logging
41     logger = logging.getLogger(__name__)
42
43
44 class AggregationLevel(str, Enum):
45     """Уровни иерархической агрегации временных рядов телеметр
46     ии."""
47
48     HOURLY = "hourly"
49     DAILY = "daily"
50     WEEKLY = "weekly"
51     MONTHLY = "monthly"
52     YEARLY = "yearly"
53
54 def sensors_aggregate(
55     conn: sqlite3.Connection,
56     level: Union[AggregationLevel, str],
57     start_epoch: Optional[float] = None,
58     end_epoch: Optional[float] = None,
59     detect_spikes: bool = True,
60 ) -> Dict[str, int]:
61     """Выполняет единую SQL-агрегацию сенсоров для указанного
62     уровня иерархии.
63
64     Args:
65         conn: Активное подключение к базе данных SQLite.

```

```

65 |         level: Уровень агрегации ('hourly', 'daily', 'weekly',
    |         'monthly', 'yearly').
66 |         start_epoch: Начальная граница выборки в секундах epoch
    |         h (опционально).
67 |         end_epoch: Конечная граница выборки в секундах epoch (
    |         опционально).
68 |         detect_spikes: Флаг сохранения статистических всплеско
    |         в/аномалий в telemetry_spikes.
69 |
70 |     Returns:
71 |         Dict[str, int]: Словарь с количеством обработанных бак
    |         етов и сохраненных аномалий.
72 |         """
73 |         lvl_str = str(level.value if isinstance(level, Aggregation
    |         Level) else level).lower()
74 |         cursor = conn.cursor()
75 |         now_epoch = time.time()
76 |         res = {"level": lvl_str, "buckets_aggregated": 0, "spikes_
    |         recorded": 0}
77 |
78 |         if lvl_str == AggregationLevel.HOURLY.value:
79 |             s_epoch = float(start_epoch) if start_epoch is not Non
    |             e else 0.0
80 |             e_epoch = float(end_epoch) if end_epoch is not None el
    |             se (int(now_epoch // 3600) * 3600.0)
81 |
82 |         # 1. SQL агрегация сырых измерений в часовые бакеты
83 |         cursor.execute("""
84 |             INSERT INTO telemetry_hourly (
85 |                 sensor_id,
86 |                 bucket_start,
87 |                 bucket_end,
88 |                 sample_count,
89 |                 value_min,
90 |                 value_max,
91 |                 value_avg,
92 |                 value_sum,
93 |                 value_sum_sq,

```

```

94         value_stddev,
95         value_first,
96         value_last
97     )
98     WITH raw_window AS (
99         SELECT
100             sensor_id,
101             CAST(created_at / 3600 AS INTEGER) * 3600
102         AS b_start,
103             created_at,
104             value,
105             FIRST_VALUE(value) OVER (
106                 PARTITION BY sensor_id, CAST(created_a
107             t / 3600 AS INTEGER) * 3600
108                 ORDER BY created_at ASC
109             ) AS f_val,
110             FIRST_VALUE(value) OVER (
111                 PARTITION BY sensor_id, CAST(created_a
112             t / 3600 AS INTEGER) * 3600
113                 ORDER BY created_at DESC
114             ) AS l_val
115         FROM sensor_polls
116         WHERE created_at >= ? AND created_at < ? AND v
117         alue IS NOT NULL
118     )
119     SELECT
120         sensor_id,
121         b_start AS bucket_start,
122         b_start + 3600 AS bucket_end,
123         COUNT(*) AS sample_count,
124         MIN(value) AS value_min,
125         MAX(value) AS value_max,
126         AVG(value) AS value_avg,
127         SUM(value) AS value_sum,
128         SUM(value * value) AS value_sum_sq,
129         SQRT(MAX(0.0, (SUM(value * value) - (SUM(value
130         ) * SUM(value)) / COUNT(*)) / COUNT(*))) AS value_stddev,
131         MIN(f_val) AS value_first,

```

```

127         MAX(l_val) AS value_last
128     FROM raw_window
129     GROUP BY sensor_id, b_start
130     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
131         bucket_end = excluded.bucket_end,
132         sample_count = excluded.sample_count,
133         value_min = excluded.value_min,
134         value_max = excluded.value_max,
135         value_avg = excluded.value_avg,
136         value_sum = excluded.value_sum,
137         value_sum_sq = excluded.value_sum_sq,
138         value_stddev = excluded.value_stddev,
139         value_first = excluded.value_first,
140         value_last = excluded.value_last;
141     """ , (s_epoch, e_epoch))
142     res["buckets_aggregated"] = cursor.rowcount if cursor.
rowcount > 0 else 0
143
144     # 2. Выявление и сохранение аномальных всплесков
145     if detect_spikes:
146         cursor.execute("""
147             INSERT INTO telemetry_spikes (
148                 sensor_id, timestamp, created_at, value, s
pike_type,
149                 baseline_avg, baseline_stddev, details
150             )
151             SELECT
152                 p.sensor_id,
153                 p.timestamp,
154                 p.created_at,
155                 p.value,
156                 CASE WHEN p.value >= h.value_avg THEN 'hig
h' ELSE 'low' END,
157                 h.value_avg,
158                 h.value_stddev,
159                 'Всплеск: знач=' || ROUND(p.value, 2) || '
, avg=' || ROUND(h.value_avg, 2) || ', std=' || ROUND(h.value_
stddev, 2)

```

```

160         FROM sensor_polls p
161         JOIN telemetry_hourly h
162         ON p.sensor_id = h.sensor_id
163         AND (CAST(p.created_at / 3600 AS INTEGER) * 3
164         600) = h.bucket_start
165         WHERE h.value_stddev > 0.01
166         AND ABS(p.value - h.value_avg) >= 2.5 * h.va
167         lue_stddev
168         AND p.created_at >= ? AND p.created_at < ?
169         AND NOT EXISTS (
170             SELECT 1 FROM telemetry_spikes s
171             WHERE s.sensor_id = p.sensor_id AND s.cr
172             eated_at = p.created_at
173             );
174         "", (s_epoch, e_epoch))
175         res["spikes_recorded"] = cursor.rowcount if cursor
176         .rowcount > 0 else 0
177
178     elif lvl_str == AggregationLevel.DAILY.value:
179         s_epoch = float(start_epoch) if start_epoch is not Non
180         e else 0.0
181         e_epoch = float(end_epoch) if end_epoch is not None el
182         se (int(now_epoch // 86400) * 86400.0)
183
184         cursor.execute("""
185             INSERT INTO telemetry_daily (
186                 sensor_id,
187                 bucket_start,
188                 bucket_end,
189                 sample_count,
190                 value_min,
191                 value_max,
192                 value_avg,
193                 value_sum,
194                 value_sum_sq,
195                 value_stddev,
196                 value_first,
197                 value_last

```



```

192 |         )
193 |         WITH hourly_window AS (
194 |             SELECT
195 |                 sensor_id,
196 |                 CAST(bucket_start / 86400 AS INTEGER) * 86
197 | 400 AS b_start,
198 |                 bucket_start,
199 |                 sample_count,
200 |                 value_min,
201 |                 value_max,
202 |                 value_sum,
203 |                 value_sum_sq,
204 |                 value_first,
205 |                 value_last,
206 |                 FIRST_VALUE(value_first) OVER (
207 |                     PARTITION BY sensor_id, CAST(bucket_start
208 | / 86400 AS INTEGER) * 86400
209 |                     ORDER BY bucket_start ASC
210 |                 ) AS f_val,
211 |                 FIRST_VALUE(value_last) OVER (
212 |                     PARTITION BY sensor_id, CAST(bucket_start
213 | / 86400 AS INTEGER) * 86400
214 |                     ORDER BY bucket_start DESC
215 |                 ) AS l_val
216 |             FROM telemetry_hourly
217 |             WHERE bucket_start >= ? AND bucket_start < ?
218 |         )
219 |         SELECT
220 |             sensor_id,
221 |             b_start AS bucket_start,
222 |             b_start + 86400 AS bucket_end,
223 |             SUM(sample_count) AS sample_count,
224 |             MIN(value_min) AS value_min,
225 |             MAX(value_max) AS value_max,
226 |             SUM(value_sum) / SUM(sample_count) AS value_av
227 | g,
228 |             SUM(value_sum) AS value_sum,
229 |             SUM(value_sum_sq) AS value_sum_sq,

```

```

226 |             Sqrt(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
| sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
| ))) AS value_stddev,
227 |             MIN(f_val) AS value_first,
228 |             MAX(l_val) AS value_last
229 | FROM hourly_window
230 | GROUP BY sensor_id, b_start
231 | ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
232 |     bucket_end = excluded.bucket_end,
233 |     sample_count = excluded.sample_count,
234 |     value_min = excluded.value_min,
235 |     value_max = excluded.value_max,
236 |     value_avg = excluded.value_avg,
237 |     value_sum = excluded.value_sum,
238 |     value_sum_sq = excluded.value_sum_sq,
239 |     value_stddev = excluded.value_stddev,
240 |     value_first = excluded.value_first,
241 |     value_last = excluded.value_last;
242 |     """, (s_epoch, e_epoch))
243 |     res["buckets_aggregated"] = cursor.rowcount if cursor.
| rowcount > 0 else 0
244 |
245 |     elif lvl_str == AggregationLevel.WEEKLY.value:
246 |         s_epoch = float(start_epoch) if start_epoch is not Non
| e else 0.0
247 |         e_epoch = float(end_epoch) if end_epoch is not None el
| se (int(now_epoch // 604800) * 604800.0)
248 |
249 |         cursor.execute("""
250 |             INSERT INTO telemetry_weekly (
251 |                 sensor_id,
252 |                 bucket_start,
253 |                 bucket_end,
254 |                 sample_count,
255 |                 value_min,
256 |                 value_max,
257 |                 value_avg,
258 |                 value_sum,

```

```

259         value_sum_sq,
260         value_stddev,
261         value_first,
262         value_last
263     )
264     WITH daily_window AS (
265         SELECT
266             sensor_id,
267             CAST(bucket_start / 604800 AS INTEGER) * 6
268             04800 AS b_start,
269             bucket_start,
270             sample_count,
271             value_min,
272             value_max,
273             value_sum,
274             value_sum_sq,
275             value_first,
276             value_last,
277             FIRST_VALUE(value_first) OVER (
278                 PARTITION BY sensor_id, CAST(bucket_start / 604800 AS INTEGER) * 604800
279                 ORDER BY bucket_start ASC
280             ) AS f_val,
281             FIRST_VALUE(value_last) OVER (
282                 PARTITION BY sensor_id, CAST(bucket_start / 604800 AS INTEGER) * 604800
283                 ORDER BY bucket_start DESC
284             ) AS l_val
285         FROM telemetry_daily
286         WHERE bucket_start >= ? AND bucket_start < ?
287     )
288     SELECT
289         sensor_id,
290         b_start AS bucket_start,
291         b_start + 604800 AS bucket_end,
292         SUM(sample_count) AS sample_count,
293         MIN(value_min) AS value_min,
294         MAX(value_max) AS value_max,

```

```

294         SUM(value_sum) / SUM(sample_count) AS value_av
    g,
295         SUM(value_sum) AS value_sum,
296         SUM(value_sum_sq) AS value_sum_sq,
297         SQRT(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
))) AS value_stddev,
298         MIN(f_val) AS value_first,
299         MAX(l_val) AS value_last
300     FROM daily_window
301     GROUP BY sensor_id, b_start
302     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
303         bucket_end = excluded.bucket_end,
304         sample_count = excluded.sample_count,
305         value_min = excluded.value_min,
306         value_max = excluded.value_max,
307         value_avg = excluded.value_avg,
308         value_sum = excluded.value_sum,
309         value_sum_sq = excluded.value_sum_sq,
310         value_stddev = excluded.value_stddev,
311         value_first = excluded.value_first,
312         value_last = excluded.value_last;
313     """, (s_epoch, e_epoch))
314     res["buckets_aggregated"] = cursor.rowcount if cursor.
rowcount > 0 else 0
315
316     elif lvl_str == AggregationLevel.MONTHLY.value:
317         s_epoch = float(start_epoch) if start_epoch is not Non
e else 0.0
318         e_epoch = float(end_epoch) if end_epoch is not None el
se now_epoch
319
320     cursor.execute("""
321         INSERT INTO telemetry_monthly (
322             sensor_id,
323             bucket_start,
324             bucket_end,
325             sample_count,

```

```

326         value_min,
327         value_max,
328         value_avg,
329         value_sum,
330         value_sum_sq,
331         value_stddev,
332         value_first,
333         value_last
334     )
335     WITH daily_window AS (
336         SELECT
337             sensor_id,
338             CAST(strftime('%s', datetime(bucket_start,
339 'unixepoch', 'start of month')) AS INTEGER) AS b_start,
340             CAST(strftime('%s', datetime(bucket_start,
341 'unixepoch', 'start of month', '+1 month')) AS INTEGER) AS b_
342 end,
343             bucket_start,
344             sample_count,
345             value_min,
346             value_max,
347             value_sum,
348             value_sum_sq,
349             value_first,
350             value_last,
351             FIRST_VALUE(value_first) OVER (
352                 PARTITION BY sensor_id, strftime('%Y-%
353 m', datetime(bucket_start, 'unixepoch'))
354                 ORDER BY bucket_start ASC
355             ) AS f_val,
356             FIRST_VALUE(value_last) OVER (
357                 PARTITION BY sensor_id, strftime('%Y-%
358 m', datetime(bucket_start, 'unixepoch'))
359                 ORDER BY bucket_start DESC
360             ) AS l_val
361         FROM telemetry_daily
362         WHERE bucket_start >= ? AND bucket_start < ?
363     )

```

```

359         SELECT
360             sensor_id,
361             b_start AS bucket_start,
362             MAX(b_end) AS bucket_end,
363             SUM(sample_count) AS sample_count,
364             MIN(value_min) AS value_min,
365             MAX(value_max) AS value_max,
366             SUM(value_sum) / SUM(sample_count) AS value_av
367         g,
368             SUM(value_sum) AS value_sum,
369             SUM(value_sum_sq) AS value_sum_sq,
370             Sqrt(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
371 sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
372 ))) AS value_stddev,
373             MIN(f_val) AS value_first,
374             MAX(l_val) AS value_last
375     FROM daily_window
376     GROUP BY sensor_id, b_start
377     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
378         bucket_end = excluded.bucket_end,
379         sample_count = excluded.sample_count,
380         value_min = excluded.value_min,
381         value_max = excluded.value_max,
382         value_avg = excluded.value_avg,
383         value_sum = excluded.value_sum,
384         value_sum_sq = excluded.value_sum_sq,
385         value_stddev = excluded.value_stddev,
386         value_first = excluded.value_first,
387         value_last = excluded.value_last;
388     "", (s_epoch, e_epoch))
389     res["buckets_aggregated"] = cursor.rowcount if cursor.
390 rowcount > 0 else 0
391
392     elif lvl_str == AggregationLevel.YEARLY.value:
393         s_epoch = float(start_epoch) if start_epoch is not Non
394 e else 0.0
395         e_epoch = float(end_epoch) if end_epoch is not None el
396 se now_epoch

```

```

391 |
392 |         cursor.execute("""
393 |             INSERT INTO telemetry_yearly (
394 |                 sensor_id,
395 |                 bucket_start,
396 |                 bucket_end,
397 |                 sample_count,
398 |                 value_min,
399 |                 value_max,
400 |                 value_avg,
401 |                 value_sum,
402 |                 value_sum_sq,
403 |                 value_stddev,
404 |                 value_first,
405 |                 value_last
406 |             )
407 |             WITH monthly_window AS (
408 |                 SELECT
409 |                     sensor_id,
410 |                     CAST(strftime('%s', datetime(bucket_start,
411 | 'unixepoch', 'start of year')) AS INTEGER) AS b_start,
412 |                     CAST(strftime('%s', datetime(bucket_start,
413 | 'unixepoch', 'start of year', '+1 year')) AS INTEGER) AS b_end,
414 |                     bucket_start,
415 |                     sample_count,
416 |                     value_min,
417 |                     value_max,
418 |                     value_sum,
419 |                     value_sum_sq,
420 |                     value_first,
421 |                     value_last,
422 |                     FIRST_VALUE(value_first) OVER (
423 |                         PARTITION BY sensor_id, strftime('%Y',
424 | datetime(bucket_start, 'unixepoch'))
425 |                         ORDER BY bucket_start ASC
426 |                     ) AS f_val,
427 |                     FIRST_VALUE(value_last) OVER (

```

```

425 |             PARTITION BY sensor_id, strftime('%Y',
|             datetime(bucket_start, 'unixepoch'))
426 |             ORDER BY bucket_start DESC
427 |             ) AS l_val
428 |         FROM telemetry_monthly
429 |         WHERE bucket_start >= ? AND bucket_start < ?
430 |     )
431 |     SELECT
432 |         sensor_id,
433 |         b_start AS bucket_start,
434 |         MAX(b_end) AS bucket_end,
435 |         SUM(sample_count) AS sample_count,
436 |         MIN(value_min) AS value_min,
437 |         MAX(value_max) AS value_max,
438 |         SUM(value_sum) / SUM(sample_count) AS value_av
|     g,
439 |         SUM(value_sum) AS value_sum,
440 |         SUM(value_sum_sq) AS value_sum_sq,
441 |         SQRT(MAX(0.0, (SUM(value_sum_sq) - (SUM(value_
| sum) * SUM(value_sum)) / SUM(sample_count)) / SUM(sample_count
| ))) AS value_stddev,
442 |         MIN(f_val) AS value_first,
443 |         MAX(l_val) AS value_last
444 |     FROM monthly_window
445 |     GROUP BY sensor_id, b_start
446 |     ON CONFLICT(sensor_id, bucket_start) DO UPDATE SET
447 |         bucket_end = excluded.bucket_end,
448 |         sample_count = excluded.sample_count,
449 |         value_min = excluded.value_min,
450 |         value_max = excluded.value_max,
451 |         value_avg = excluded.value_avg,
452 |         value_sum = excluded.value_sum,
453 |         value_sum_sq = excluded.value_sum_sq,
454 |         value_stddev = excluded.value_stddev,
455 |         value_first = excluded.value_first,
456 |         value_last = excluded.value_last;
457 |     "", (s_epoch, e_epoch))
458 |     res["buckets_aggregated"] = cursor.rowcount if cursor.

```



```

    rowcount > 0 else 0
459
460     else:
461         raise ValueError(f"Неизвестный уровень агрегации: {lvl
    _str}")
462
463     return res
464
465
466 class TelemetrySqlAggregator:
467     """Управляет вызовами SQL-агрегации и каскадным выполнением
    цепочки агрегаторов."""
468
469     def __init__(self, connection_manager: TelemetryConnection
    Manager) -> None:
470         """Инициализирует SQL-агрегатор телеметрии.
471
472         Args:
473             connection_manager: Менеджер подключений SQLite.
474         """
475         self._cm = connection_manager
476         self._scheduler_timer: Optional[threading.Timer] = None
477
478         self._scheduler_interval: float = 300.0
479         self._scheduler_running: bool = False
480
481     def start_background_scheduler(self, interval_seconds: float = 300.0) -> None:
482         """Запускает фоновый регламентный планировщик агрегации.
483
484         Args:
485             interval_seconds: Периодичность запуска каскада агрегации в секундах (по умолчанию 300с = 5 мин).
486         """
487         if self._cm.read_only:
488             return
489         self.stop_background_scheduler()

```

```

489         self._scheduler_interval = max(10.0, float(interval_se
conds))
490         self._scheduler_running = True
491         self._schedule_next_run()
492         logger.info(f"Запущен фоновый регламентный планировщик
агрегации (квант: {self._scheduler_interval}с)")
493
494     def stop_background_scheduler(self) -> None:
495         """Останавливает фоновый регламентный планировщик агре
гации."""
496         self._scheduler_running = False
497         if self._scheduler_timer:
498             try:
499                 self._scheduler_timer.cancel()
500             except Exception:
501                 pass
502             self._scheduler_timer = None
503
504     def _schedule_next_run(self) -> None:
505         if not self._scheduler_running or self._cm.read_only:
506             return
507         self._scheduler_timer = threading.Timer(self._schedule
r_interval, self._on_scheduler_tick)
508         self._scheduler_timer.daemon = True
509         self._scheduler_timer.start()
510
511     def _on_scheduler_tick(self) -> None:
512         if not self._scheduler_running or self._cm.read_only:
513             return
514         try:
515             self.run_pipeline()
516         except Exception as ex:
517             logger.debug(f"Ошибка регламентной фоновой агрегац
ии: {ex}")
518         finally:
519             self._schedule_next_run()
520
521     def aggregate(

```

```

522         self,
523         level: Union[AggregationLevel, str],
524         start_epoch: Optional[float] = None,
525         end_epoch: Optional[float] = None,
526         detect_spikes: bool = True,
527     ) -> Dict[str, Any]:
528         """Запускает агрегацию указанного уровня в транзакции.
529
530         Args:
531             level: Уровень агрегации ('hourly', 'daily', 'weekly', 'monthly', 'yearly').
532             start_epoch: Начальная граница диапазона.
533             end_epoch: Конечная граница диапазона.
534             detect_spikes: Флаг фиксации всплесков.
535
536         Returns:
537             Dict[str, Any]: Результаты агрегации.
538         """
539         if self._cm.read_only:
540             return {"level": str(level), "buckets_aggregated":
541                     0, "spikes_recorded": 0, "read_only": True}
542
543         with self._cm.lock, self._cm.get_connection() as conn:
544             result = sensors_aggregate(
545                 conn=conn,
546                 level=level,
547                 start_epoch=start_epoch,
548                 end_epoch=end_epoch,
549                 detect_spikes=detect_spikes,
550             )
551             conn.commit()
552             return result
553
554     def aggregate_hourly(self, start_epoch: Optional[float] =
555         None, end_epoch: Optional[float] = None, detect_spikes: bool =

```

```

    poch=start_epoch, end_epoch=end_epoch, detect_spikes=detect_spikes)
556
557     def aggregate_daily(self, start_epoch: Optional[float] = None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
558         """Выполняет суточную агрегацию из почасовых бакетов."""
559         return self.aggregate(AggregationLevel.DAILY, start_epoch=start_epoch, end_epoch=end_epoch)
560
561     def aggregate_weekly(self, start_epoch: Optional[float] = None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
562         """Выполняет недельную агрегацию из суточных бакетов."""
563         return self.aggregate(AggregationLevel.WEEKLY, start_epoch=start_epoch, end_epoch=end_epoch)
564
565     def aggregate_monthly(self, start_epoch: Optional[float] = None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
566         """Выполняет месячную агрегацию из суточных бакетов."""
567         return self.aggregate(AggregationLevel.MONTHLY, start_epoch=start_epoch, end_epoch=end_epoch)
568
569     def aggregate_yearly(self, start_epoch: Optional[float] = None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
570         """Выполняет годовую агрегацию из месячных бакетов."""
571         return self.aggregate(AggregationLevel.YEARLY, start_epoch=start_epoch, end_epoch=end_epoch)
572
573     def run_pipeline(
574         self,
575         start_epoch: Optional[float] = None,
576         end_epoch: Optional[float] = None,
577         detect_spikes: bool = True,
578     ) -> Dict[str, Any]:
579         """Выполняет полный каскад агрегации: raw -> hourly -> daily -> weekly -> monthly -> yearly в одной транзакции.

```

```

580 |
581 |     Args:
582 |         start_epoch: Начальная граница диапазона.
583 |         end_epoch: Конечная граница диапазона.
584 |         detect_spikes: Флаг фиксации всплесков.
585 |
586 |     Returns:
587 |         Dict[str, Any]: Сводная статистика всех уровней аг-
    | регации.
588 |         """
589 |         if self._cm.read_only:
590 |             return {"read_only": True}
591 |
592 |         with self._cm.lock, self._cm.get_connection() as conn:
593 |             h_res = sensors_aggregate(conn, AggregationLevel.H
    | OURLY, start_epoch, end_epoch, detect_spikes)
594 |             d_res = sensors_aggregate(conn, AggregationLevel.D
    | AILY, start_epoch, end_epoch, False)
595 |             w_res = sensors_aggregate(conn, AggregationLevel.W
    | EEKLY, start_epoch, end_epoch, False)
596 |             m_res = sensors_aggregate(conn, AggregationLevel.M
    | ONTHLY, start_epoch, end_epoch, False)
597 |             y_res = sensors_aggregate(conn, AggregationLevel.Y
    | EARLY, start_epoch, end_epoch, False)
598 |             conn.commit()
599 |
600 |         return {
601 |             "hourly": h_res,
602 |             "daily": d_res,
603 |             "weekly": w_res,
604 |             "monthly": m_res,
605 |             "yearly": y_res,
606 |         }
607 |

```

buffer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Buffer
4 # =====
5 # Description:
6 #   Управление буферизацией и пакетной записью (batching) пото
  ка телеметрии в SQLite.
7 #
8 #   Зачем нужен этот модуль:
9 #       1. Снижение нагрузки на диск (I/O) и ускорение SQLite:
10 #          Объединяет одиночные события метрик в пачки, выполняя
  одну быструю
11 #          транзакцию вместо сотен мелких операций записи с част
  ыми fsync/commit.
12 #       2. Предотвращение блокировок базы данных:
13 #          Минимизирует время удержания эксклюзивных транзакций
  SQLite, предотвращая
14 #          ошибки 'database is locked' при одновременном чтении
  данных агентами и UI.
15 #       3. Защита от потери данных (Fail-Safe / Disaster Recover
  y):
16 #          При сбоях записи или аварийном завершении удерживает
  данные во внешнем
17 #          журнале и автоматически восстанавливает/дописывает их
  при следующем старте.
18 #
19 #   Поддерживаемые режимы буферизации (buffer_mode):
20 #       - 'memory' (По умолчанию):
21 #          Накапливает записи в оперативной памяти (список в RA
  M).
22 #          Сброс в БД происходит по достижении buffer_size или
  по таймеру flush_interval_seconds.
23 #          Обеспечивает максимальную скорость работы.
```

```
24 # - 'file':
25 #     Каждая входящая запись немедленно сериализуется и до
    # писывается на диск
26 #     в аварийный JSONL-файл (telemetry_buffer.jsonl) с вы
    # зовом fsync.
27 #     По достижении buffer_size или по таймеру весь файл с
    # брасывается в SQLite
28 #     и усекается. Защищает от потери данных при внезапном
    # падении процесса/питания.
29 # - 'direct':
30 #     Отключает фоновый периодический таймер автосброса. За
    # писи накапливаются в RAM
31 #     и сбрасываются только при заполнении buffer_size или
    # явном вызове flush().
32 #
33 # Usage Examples:
34 # Python API:
35 # from apps.windows.telemetry.sqlite.buffer import TelemetryBuffer
36 # from apps.windows.telemetry.sqlite.connection import TelemetryConnectionManager
37 # from apps.windows.telemetry.sqlite.writer import TelemetryWriter
38 #
39 # cm = TelemetryConnectionManager("telemetry.db")
40 # writer = TelemetryWriter(cm)
41 # buf = TelemetryBuffer(
42 #     connection_manager=cm,
43 #     writer=writer,
44 #     buffer_mode='memory', # 'memory' | 'file' | 'direct'
    #
45 #     buffer_size=50,
46 #     flush_interval_seconds=30.0
47 # )
48 # buf.enqueue({"timestamp": "...", "cpu_percent": 12.5})
49 #
50 # File: buffer.py
51 # Project: ai-breadboard
```

```

52 # Package: apps.windows.telemetry.sqlite
53 # Author: hypo69
54 # Copyright: © 2026 hypo69
55 # Updated: 2026-10-06 17:47:00
56 # =====
    =====
57
58 from __future__ import annotations
59
60 import json
61 import os
62 import sys
63 import threading
64 from pathlib import Path
65 from typing import Any, Dict, List, Optional, Union
66
67 from .connection import TelemetryConnectionManager
68 from .writer import TelemetryWriter
69
70 try:
71     from logger import logger
72 except ImportError:
73     import logging
74     logger = logging.getLogger(__name__)
75
76
77 class TelemetryBuffer:
78     """Управляет буферизацией событий телеметрии в памяти и аварийном JSONL файле с контролем лимитов RAM."""
79
80     def __init__(
81         self,
82         connection_manager: TelemetryConnectionManager,
83         writer: TelemetryWriter,
84         buffer_mode: str = 'memory',
85         buffer_size: int = 50,
86         flush_interval_seconds: float = 30.0,
87         buffer_file_path: Optional[Union[str, Path]] = None,

```



```

88         auto_flush: bool = True,
89         max_buffer_bytes: int = 32 * 1024 * 1024,
90     ) -> None:
91         """Инициализирует буфер сброса телеметрии.
92
93         Args:
94             connection_manager: Менеджер подключений SQLite.
95             writer: Писатель пакетов телеметрии.
96             buffer_mode: Режим буферизации ('memory', 'file',
97             'direct').
98             buffer_size: Максимальный размер пачки до сброса.
99             flush_interval_seconds: Интервал таймера сброса в
100             секундах.
101             buffer_file_path: Путь к файлу аварийного буфера.
102             auto_flush: Флаг автоматического запуска таймера с
103             броска.
104             max_buffer_bytes: Максимальный лимит памяти для за
105             писей буфера в байтах (по умолчанию 32 МБ).
106         """
107         self._cm = connection_manager
108         self._writer = writer
109         self._buffer_mode = buffer_mode.lower() if buffer_mode
110         in ('memory', 'file', 'direct') else 'memory'
111         self._buffer_size = max(1, int(buffer_size))
112         self._max_buffer_bytes = max(1024 * 1024, int(max_buff
113         er_bytes))
114         self._current_buffer_bytes: int = 0
115         self._flush_interval_seconds = max(0.5, float(flush_in
116         terval_seconds))
117         self._auto_flush = bool(auto_flush) and not self._cm.r
118         ead_only
119
120         if buffer_file_path:
121             self._buffer_file_path = Path(buffer_file_path)
122         else:
123             self._buffer_file_path = self._cm.db_path.parent /
124             'telemetry_buffer.jsonl'

```

```
117         if not self._cm.read_only:
118             self._buffer_file_path.parent.mkdir(parents=True,
119             exist_ok=True)
120
121         self._buffer: List[Dict[str, Any]] = []
122         self._file_buffer_count: int = 0
123         self._is_running = True
124         self._flush_timer: Optional[threading.Timer] = None
125
126         if not self._cm.read_only:
127             self._recover_file_buffer()
128             if self._auto_flush and self._buffer_mode != 'direct':
129                 self._start_flush_timer()
130
131     @property
132     def buffer_mode(self) -> str:
133         """Текущий режим буферизации."""
134         return self._buffer_mode
135
136     @property
137     def buffer_size(self) -> int:
138         """Максимальный размер буфера в количестве записей."""
139         return self._buffer_size
140
141     @property
142     def max_buffer_bytes(self) -> int:
143         """Максимальный лимит памяти буфера в байтах."""
144         return self._max_buffer_bytes
145
146     @property
147     def current_buffer_bytes(self) -> int:
148         """Текущий объем занимаемой памяти буфера в байтах."""
149         with self._cm.lock:
150             return self._current_buffer_bytes
151
152     @property
153     def flush_interval_seconds(self) -> float:
```

```

153         """Интервал таймера автосброса в секундах."""
154         return self._flush_interval_seconds
155
156     @property
157     def buffer_file_path(self) -> Path:
158         """Путь к файлу аварийного JSONL буфера."""
159         return self._buffer_file_path
160
161     def _estimate_size(self, record: Dict[str, Any]) -> int:
162         """Оценивает объем памяти записи в байтах."""
163         try:
164             return len(json.dumps(record, ensure_ascii=False,
165 default=str).encode('utf-8')) + 128
166         except Exception:
167             return sys.getsizeof(record) + 256
168
169     def set_flush_interval_seconds(self, interval: float) -> None:
170         """Динамически изменяет интервал периодического сброса
171         буфера в SQLite."""
172         new_interval = max(0.5, float(interval))
173         if abs(self._flush_interval_seconds - new_interval) >
174 0.01:
175             with self._cm.lock:
176                 self._flush_interval_seconds = new_interval
177                 if self._auto_flush and self._buffer_mode != '
178 direct' and not self._cm.read_only:
179                     self._start_flush_timer()
180                     logger.info(f"Интервал сброса буфера телеметри
181 и обновлен: {self._flush_interval_seconds}с")
182
183     def set_buffer_mode(self, mode: str) -> None:
184         """Переключает режим буферизации."""
185         m = mode.lower()
186         if m in ('memory', 'file', 'direct'):
187             with self._cm.lock:
188                 self.flush()
189                 self._buffer_mode = m

```

```

185         if m == 'direct':
186             self._stop_flush_timer()
187         elif self._auto_flush and not self._cm.read_only:
188             self._start_flush_timer()
189             logger.info(f'Режим буферизации переключен на:
190 {m}')
191     def get_buffered_count(self) -> int:
192         """Возвращает число накопленных записей в буфере."""
193         with self._cm.lock:
194             return len(self._buffer) + self._file_buffer_count
195
196     def _start_flush_timer(self) -> None:
197         if self._cm.read_only or not self._is_running or self._buffer_mode == 'direct':
198             return
199         self._stop_flush_timer()
200         self._flush_timer = threading.Timer(self._flush_interval_seconds, self._on_flush_timer)
201         self._flush_timer.daemon = True
202         self._flush_timer.start()
203
204     def _stop_flush_timer(self) -> None:
205         if self._flush_timer is not None:
206             try:
207                 self._flush_timer.cancel()
208             except Exception:
209                 pass
210             self._flush_timer = None
211
212     def _on_flush_timer(self) -> None:
213         if not self._is_running or self._cm.read_only:
214             return
215         try:
216             self.flush()
217         except Exception as ex:
218             logger.debug(f'Ошибка фонового сброса буфера телем

```

```

    етрии: {ex}')
219         finally:
220             with self._cm.lock:
221                 if self._is_running and self._auto_flush and s
self._buffer_mode != 'direct':
222                     self._start_flush_timer()
223
224     def _recover_file_buffer(self) -> None:
225         if self._cm.read_only:
226             return
227         if self._buffer_file_path.exists() and self._buffer_fi
le_path.stat().st_size > 0:
228             try:
229                 with open(self._buffer_file_path, 'r', encodin
g='utf-8') as f:
230                     count = sum(1 for line in f if line.strip(
))
231                     self._file_buffer_count = count
232                     if count > 0:
233                         logger.info(f'Обнаружено {count} сохраненн
ых записей в {self._buffer_file_path.name}. Сброс в БД...')
234                         self.flush()
235                     except Exception as ex:
236                         logger.warning(f'Ошибка проверки файла буфера
телеметрии: {ex}')
```

```

237
238     def enqueue(self, record: Dict[str, Any]) -> None:
239         """Добавляет запись в буфер в зависимости от выбранног
о режима."""
240         if self._cm.read_only:
241             logger.warning('Попытка добавления записи в буфер
в режиме Read-Only')
242             return
243
244         rec_size = self._estimate_size(record)
245         with self._cm.lock:
246             if self._buffer_mode == 'file':
247                 try:

```

```

248         line = json.dumps(record, ensure_ascii=False, default=str)
249         with open(self._buffer_file_path, 'a', encoding='utf-8') as f:
250             f.write(line + '\n')
251             f.flush()
252             os.fsync(f.fileno())
253             self._file_buffer_count += 1
254         except Exception as ex:
255             logger.error(f'Ошибка записи в файл буфера
телеметрии: {ex}')
256             self._buffer.append(record)
257             self._current_buffer_bytes += rec_size
258
259             if self._file_buffer_count >= self._buffer_size or self._current_buffer_bytes >= self._max_buffer_bytes:
260                 self.flush()
261             else:
262                 self._buffer.append(record)
263                 self._current_buffer_bytes += rec_size
264                 if len(self._buffer) >= self._buffer_size or self._current_buffer_bytes >= self._max_buffer_bytes:
265                     self.flush()
266
267     def flush(self) -> int:
268         """Принудительно сбрасывает записи в SQLite."""
269         if self._cm.read_only:
270             return 0
271
272         with self._cm.lock:
273             records_to_save: List[Dict[str, Any]] = []
274
275             if self._buffer_file_path.exists() and self._buffer_file_path.stat().st_size > 0:
276                 try:
277                     with open(self._buffer_file_path, 'r', encoding='utf-8') as f:
278                         for line in f:

```

```

279 |             s = line.strip()
280 |             if s:
281 |                 try:
282 |                     records_to_save.append(jso
n.loads(s))
283 |                 except Exception:
284 |                     pass
285 |             except Exception as read_err:
286 |                 logger.error(f'Ошибка чтения файла буфера:
{read_err}')
287 |
288 |         if self._buffer:
289 |             records_to_save.extend(self._buffer)
290 |             self._buffer = []
291 |             self._current_buffer_bytes = 0
292 |
293 |         if not records_to_save:
294 |             return 0
295 |
296 |         inserted_count = self._writer.batch_insert_records
(records_to_save)
297 |
298 |         if self._buffer_file_path.exists():
299 |             try:
300 |                 with open(self._buffer_file_path, 'w', enc
oding='utf-8') as f:
301 |                     f.truncate(0)
302 |                     self._file_buffer_count = 0
303 |             except Exception as trunc_err:
304 |                 logger.warning(f'Ошибка усечения файла буф
ера: {trunc_err}')
305 |
306 |         if inserted_count > 0:
307 |             logger.debug(f'[Хранилище SQLite] Пакетный сброс буфера: {inserted_count} записей зафиксировано в {self._cm.db_path.name}')
308 |
309 |         return inserted_count

```

```
310 |
311 |     def close(self) -> None:
312 |         """Останавливает таймер и выполняет финальный сброс бу
313 |         фера."""
314 |         self._is_running = False
315 |         self._stop_flush_timer()
316 |         if not self._cm.read_only:
317 |             try:
318 |                 self.flush()
319 |             except Exception:
320 |                 pass
```


connection.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Connection
4 # =====
5 # Description:
6 #   Менеджер подключений к SQLite базе данных с поддержкой реж
  има только чтение.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.connection import Tel
  emetryConnectionManager
11 #
12 #       cm = TelemetryConnectionManager(db_path=Path("logs/telem
  etry.db"), read_only=True)
13 #       with cm.get_connection() as conn:
14 #           ...
15 #
16 # File: connection.py
17 # Project: ai-breadboard
18 # Package: apps.windows.telemetry.sqlite
19 # Author: hypo69
20 # Copyright: © 2026 hypo69
21 # Updated: 2026-10-06 05:32:00
22 # =====
23
24 from __future__ import annotations
25
26 """Управление подключениями к базе данных SQLite с поддержкой
  режимов Read-Only и WAL."""
27
28 import sqlite3
```

```
29 import threading
30 from pathlib import Path
31 from typing import Optional, Union
32
33 try:
34     from logger import logger
35 except ImportError:
36     import logging
37     logger = logging.getLogger(__name__)
38
39
40 class TelemetryConnectionManager:
41     """Менеджер создания и настройки соединений к SQLite базе
42     данных телеметрии."""
43
44     def __init__(
45         self,
46         db_path: Union[str, Path],
47         read_only: bool = False,
48         timeout: float = 15.0,
49         lock: Optional[threading.RLock] = None,
50     ) -> None:
51         """Инициализирует менеджер соединений.
52
53         Args:
54             db_path: Путь к файлу SQLite базы данных.
55             read_only: Флаг строгого режима только чтения (бло
56             кирует запись).
57             timeout: Таймаут ожидания освобождения базы данных
58             в секундах.
59             lock: Разделяемый RLock (опционально).
60
61         """
62         self.db_path = Path(db_path)
63         self.read_only = bool(read_only)
64         self.timeout = float(timeout)
65         self._lock = lock or threading.RLock()
66
67         if not self.read_only:
```

```

64         self.db_path.parent.mkdir(parents=True, exist_ok=True)
65
66     @property
67     def lock(self) -> threading.RLock:
68         """Возвращает разделяемый поток-безопасный лок."""
69         return self._lock
70
71     def get_connection(self) -> sqlite3.Connection:
72         """Создает и настраивает соединение SQLite с необходим
73         ыми PRAGMA директивами.
74
75         Returns:
76             sqlite3.Connection: Настроенное соединение.
77
78         """
79         if self.read_only:
80             db_uri = f"file:{self.db_path.as_posix()}?mode=ro"
81             try:
82                 conn = sqlite3.connect(db_uri, uri=True, timeout=self.timeout,
83                                         check_same_thread=False)
84             except sqlite3.OperationalError:
85                 conn = sqlite3.connect(str(self.db_path), timeout=self.timeout,
86                                         check_same_thread=False)
87             try:
88                 conn.execute('PRAGMA query_only = ON;')
89             except Exception:
90                 pass
91         else:
92             conn = sqlite3.connect(str(self.db_path), timeout=self.timeout,
93                                     check_same_thread=False)
94             conn.execute('PRAGMA journal_mode = WAL;')
95             conn.execute('PRAGMA synchronous = NORMAL;')
96             conn.execute('PRAGMA foreign_keys = ON;')
97             conn.execute('PRAGMA busy_timeout = 15000;')
98             conn.execute('PRAGMA cache_size = -64000;')
99             conn.execute('PRAGMA temp_store = MEMORY;')
100            conn.execute('PRAGMA mmap_size = 268435456;')
101            conn.execute('PRAGMA wal_autocheckpoint = 1000;')

```

```
97 |  
98 |     conn.row_factory = sqlite3.Row  
99 |     return conn  
100 |
```

maintenance.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Maintenance
4 # =====
5 # Description:
6 #   Обслуживание, очистка, агрегация и контроль размера базы д
  анных SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.maintenance import Te
    lemetryMaintenance
11 #
12 #       maint = TelemetryMaintenance(connection_manager)
13 #       stats = maint.get_storage_stats()
14 #
15 # File: maintenance.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 01:59:00
21 # =====
22
23 from __future__ import annotations
24
25 """Методы обслуживания, очистки старых записей, дефрагментации
  и агрегации телеметрии в SQLite."""
26
27 import sqlite3
28 import time
29 from collections import defaultdict
```

```

30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional
33
34 from .connection import TelemetryConnectionManager
35
36 try:
37     from logger import logger
38 except ImportError:
39     import logging
40     logger = logging.getLogger(__name__)
41
42
43 class TelemetryMaintenance:
44     """Выполняет регламентные операции: очистка, контроль разм
    ера, вакуумирование и агрегации."""
45
46     def __init__(
47         self,
48         connection_manager: TelemetryConnectionManager,
49         max_db_size_mb: float = 50.0,
50         retention_days: int = 7,
51         auto_vacuum_enabled: bool = True,
52         buffer_file_path: Optional[Path] = None,
53     ) -> None:
54         """Инициализирует менеджер обслуживания базы данных.
55
56         Args:
57             connection_manager: Менеджер соединений SQLite.
58             max_db_size_mb: Максимальный размер файлов БД в МБ
59             .
60             retention_days: Срок хранения сырых записей в днях
61             .
62             auto_vacuum_enabled: Флаг автоматического запуска
63             VACUUM.
64             buffer_file_path: Путь к файлу буфера.
65         """
66         self._cm = connection_manager

```

```

64 |         self.max_db_size_mb = max(1.0, float(max_db_size_mb))
65 |         self.retention_days = max(1, int(retention_days))
66 |         self.auto_vacuum_enabled = bool(auto_vacuum_enabled)
67 |         self.buffer_file_path = buffer_file_path or (self._cm.
db_path.parent / 'telemetry_buffer.jsonl')
68 |
69 |     def get_db_file_sizes(self) -> Dict[str, int]:
70 |         """Возвращает физические размеры файлов БД на диске (D
B, WAL, SHM) в байтах."""
71 |         db_path = self._cm.db_path
72 |         db_bytes = db_path.stat().st_size if db_path.exists()
else 0
73 |         wal_path = db_path.with_name(db_path.name + '-wal')
74 |         shm_path = db_path.with_name(db_path.name + '-shm')
75 |         wal_bytes = wal_path.stat().st_size if wal_path.exists
() else 0
76 |         shm_bytes = shm_path.stat().st_size if shm_path.exists
() else 0
77 |         return {
78 |             'db_bytes': db_bytes,
79 |             'wal_bytes': wal_bytes,
80 |             'shm_bytes': shm_bytes,
81 |             'total_bytes': db_bytes + wal_bytes + shm_bytes,
82 |         }
83 |
84 |     def get_total_db_size_mb(self) -> float:
85 |         """Возвращает совокупный физический размер файлов БД в
МБ."""
86 |         sizes = self.get_db_file_sizes()
87 |         return round(sizes['total_bytes'] / (1024.0 * 1024.0),
3)
88 |
89 |     def checkpoint_wal(self) -> bool:
90 |         """Сбрасывает WAL-журнал в основной файл базы данных."
""
91 |         if self._cm.read_only:
92 |             return False
93 |         with self._cm.lock:

```

```

94         try:
95             with self._cm.get_connection() as conn:
96                 conn.execute('PRAGMA wal_checkpoint(TRUNCATE);')
97             return True
98         except Exception as e:
99             logger.debug(f'Ошибка сброса WAL-журнала: {e}')
100     )
101
102     def vacuum(self) -> bool:
103         """Выполняет сжатие и дефрагментацию базы данных."""
104         if self._cm.read_only:
105             return False
106         with self._cm.lock:
107             self.checkpoint_wal()
108             try:
109                 conn = sqlite3.connect(str(self._cm.db_path),
110                                         timeout=30.0, isolation_level=None)
111                 try:
112                     conn.execute('VACUUM;')
113                     finally:
114                         conn.close()
115                 logger.info(f'Выполнен VACUUM для базы данных
116                             телеметрии ({self._cm.db_path})')
117                 return True
118             except Exception as e:
119                 logger.warning(f'Ошибка при выполнении VACUUM:
120                                {e}')
121             return False
122
123     def get_storage_stats(self, buffered_count: int = 0, buffer_mode: str = 'memory', buffer_size: int = 50, flush_interval_seconds: float = 30.0) -> Dict[str, Any]:
124         """Возвращает агрегированную статистику таблиц, размера и параметров."""
125         with self._cm.lock, self._cm.get_connection() as conn:
126             cursor = conn.cursor()

```



```

124 |         cursor.execute('SELECT COUNT(*) FROM system_snapsh
    | ots;')
125 |         snap_count = cursor.fetchone()[0]
126 |         cursor.execute('SELECT COUNT(*) FROM process_snaps
    | hots;')
127 |         proc_count = cursor.fetchone()[0]
128 |         cursor.execute('SELECT COUNT(*) FROM sensor_polls;
    | ')
129 |         sensors_count = cursor.fetchone()[0]
130 |         cursor.execute('SELECT COUNT(*) FROM telemetry_eve
    | nts;')
131 |         events_count = cursor.fetchone()[0]
132 |         cursor.execute('SELECT COUNT(*) FROM hardware_audi
    | ts;')
133 |         audits_count = cursor.fetchone()[0]
134 |         cursor.execute('SELECT COUNT(*) FROM app_polls;')
135 |         app_polls_count = cursor.fetchone()[0]
136 |         cursor.execute('SELECT COUNT(*) FROM app_events;')
137 |         app_events_count = cursor.fetchone()[0]
138 |         cursor.execute('SELECT COUNT(*) FROM app_param_cha
    | nges;')
139 |         app_params_count = cursor.fetchone()[0]
140 |         cursor.execute('SELECT COUNT(*) FROM custom_record
    | s;')
141 |         custom_count = cursor.fetchone()[0]
142 |         cursor.execute('SELECT COUNT(*) FROM device_events
    | ;')
143 |         device_events_count = cursor.fetchone()[0]
144 |         cursor.execute('SELECT COUNT(*) FROM device_invent
    | ory;')
145 |         device_inventory_count = cursor.fetchone()[0]
146 |         cursor.execute('SELECT COUNT(*) FROM w64_events;')
147 |         w64_events_count = cursor.fetchone()[0]
148 |         cursor.execute('SELECT COUNT(*) FROM process_outli
    | ers;')
149 |         outliers_count = cursor.fetchone()[0]
150 |         cursor.execute('SELECT COUNT(*) FROM process_rollu
    | ps_2min;')

```

```

151 |         rollups_2min_count = cursor.fetchone()[0]
152 |         cursor.execute('SELECT COUNT(*) FROM process_rollu
    | ps_daily;')
153 |         rollups_daily_count = cursor.fetchone()[0]
154 |         cursor.execute('SELECT COUNT(*) FROM incidents;')
155 |         incidents_count = cursor.fetchone()[0]
156 |         cursor.execute('SELECT COUNT(*) FROM telemetry_rol
    | lups;')
157 |         rollups_count = cursor.fetchone()[0]
158 |
159 |         def _safe_count(table: str) -> int:
160 |             try:
161 |                 cursor.execute(f'SELECT COUNT(*) FROM {tab
    | le};')
162 |                 return cursor.fetchone()[0]
163 |             except Exception:
164 |                 return 0
165 |
166 |         telemetry_hourly_count = _safe_count('telemetry_ho
    | urly')
167 |         telemetry_daily_count = _safe_count('telemetry_dai
    | ly')
168 |         telemetry_weekly_count = _safe_count('telemetry_we
    | ekly')
169 |         telemetry_monthly_count = _safe_count('telemetry_m
    | onthly')
170 |         telemetry_yearly_count = _safe_count('telemetry_ye
    | arly')
171 |         telemetry_spikes_count = _safe_count('telemetry_sp
    | ikes')
172 |         disk_inventory_count = _safe_count('disk_inventory
    | ')
173 |         volume_inventory_count = _safe_count('volume_inven
    | tory')
174 |         disk_health_count = _safe_count('disk_health_snaps
    | hots')
175 |         disk_perf_count = _safe_count('disk_performance_sa
    | mples')

```

```
176         disk_io_count = _safe_count('disk_io_events')
177         cpu_inventory_count = _safe_count('cpu_inventory')
178         ram_module_inventory_count = _safe_count('ram_modu
le_inventory')
179         gpu_inventory_count = _safe_count('gpu_inventory')
180         network_adapter_inventory_count = _safe_count('net
work_adapter_inventory')
181         cpu_samples_count = _safe_count('cpu_telemetry_sam
ples')
182         ram_samples_count = _safe_count('ram_telemetry_sam
ples')
183         gpu_samples_count = _safe_count('gpu_telemetry_sam
ples')
184         network_samples_count = _safe_count('network_adapt
er_samples')
185
186         file_sizes = self.get_db_file_sizes()
187         size_mb = round(file_sizes['db_bytes'] / (1024 * 1
024), 2)
188         wal_size_mb = round(file_sizes['wal_bytes'] / (102
4 * 1024), 2)
189         total_size_mb = round(file_sizes['total_bytes'] /
(1024 * 1024), 2)
190         buffer_file_size = self.buffer_file_path.stat().st
_size if self.buffer_file_path.exists() else 0
191
192         return {
193             'db_path': str(self._cm.db_path),
194             'snapshots_count': snap_count,
195             'process_snapshots_count': proc_count,
196             'process_outliers_count': outliers_count,
197             'process_rollups_2min_count': rollups_2min_cou
nt,
198             'process_rollups_daily_count': rollups_daily_c
ount,
199             'incidents_count': incidents_count,
200             'telemetry_rollups_count': rollups_count,
201             'telemetry_hourly_count': telemetry_hourly_cou
```

```

    nt,
202         'telemetry_daily_count': telemetry_daily_count
    ,
203         'telemetry_weekly_count': telemetry_weekly_cou
    nt,
204         'telemetry_monthly_count': telemetry_monthly_c
    ount,
205         'telemetry_yearly_count': telemetry_yearly_cou
    nt,
206         'telemetry_spikes_count': telemetry_spikes_cou
    nt,
207         'disk_inventory_count': disk_inventory_count,
208         'volume_inventory_count': volume_inventory_cou
    nt,
209         'disk_health_count': disk_health_count,
210         'disk_perf_count': disk_perf_count,
211         'disk_io_count': disk_io_count,
212         'cpu_inventory_count': cpu_inventory_count,
213         'ram_module_inventory_count': ram_module_inven
    tory_count,
214         'gpu_inventory_count': gpu_inventory_count,
215         'network_adapter_inventory_count': network_ada
    pter_inventory_count,
216         'cpu_samples_count': cpu_samples_count,
217         'ram_samples_count': ram_samples_count,
218         'gpu_samples_count': gpu_samples_count,
219         'network_samples_count': network_samples_count
    ,
220         'sensor_polls_count': sensors_count,
221         'events_count': events_count,
222         'hardware_audits_count': audits_count,
223         'app_polls_count': app_polls_count,
224         'app_events_count': app_events_count,
225         'app_param_changes_count': app_params_count,
226         'custom_records_count': custom_count,
227         'device_events_count': device_events_count,
228         'device_inventory_count': device_inventory_cou
    nt,

```

```

229         'w64_events_count': w64_events_count,
230         'file_size_mb': size_mb,
231         'wal_size_mb': wal_size_mb,
232         'total_size_mb': total_size_mb,
233         'max_db_size_mb': self.max_db_size_mb,
234         'retention_days': self.retention_days,
235         'size_limit_exceeded': total_size_mb > self.ma
x_db_size_mb,
236         'buffer_mode': buffer_mode,
237         'buffered_count': buffered_count,
238         'buffer_size': buffer_size,
239         'flush_interval_seconds': flush_interval_secon
ds,
240         'buffer_file_path': str(self.buffer_file_path)
,
241         'buffer_file_size_bytes': buffer_file_size,
242     }
243
244     def cleanup_old_records(self, retention_days: Optional[int
] = None, vacuum_after: bool = False) -> int:
245         """Удаляет устаревшие записи телеметрии."""
246         if self._cm.read_only:
247             return 0
248         days = retention_days if retention_days is not None el
se self.retention_days
249         threshold_epoch = datetime.now(timezone.utc).timestamp
() - (float(days) * 86400.0 if days > 0 else -1.0)
250         with self._cm.lock, self._cm.get_connection() as conn:
251             cursor = conn.cursor()
252             cursor.execute('SELECT id FROM system_snapshots WH
ERE created_at < ?', (threshold_epoch,))
253             old_ids = [row[0] for row in cursor.fetchall()]
254             if old_ids:
255                 cursor.execute('DELETE FROM process_snapshots
WHERE snapshot_id IN (SELECT id FROM system_snapshots WHERE cr
eated_at < ?)', (threshold_epoch,))
256                 cursor.execute('DELETE FROM system_snapshots W
HERE created_at < ?', (threshold_epoch,))

```

```
257 |         cursor.execute('DELETE FROM sensor_polls WHERE cre
| ated_at < ?', (threshold_epoch,))
258 |         cursor.execute('DELETE FROM telemetry_events WHERE
| created_at < ?', (threshold_epoch,))
259 |         cursor.execute('DELETE FROM app_polls WHERE create
| d_at < ?', (threshold_epoch,))
260 |         cursor.execute('DELETE FROM app_events WHERE creat
| ed_at < ?', (threshold_epoch,))
261 |         cursor.execute('DELETE FROM app_param_changes WHER
| E created_at < ?', (threshold_epoch,))
262 |         cursor.execute('DELETE FROM custom_records WHERE c
| reated_at < ?', (threshold_epoch,))
263 |         cursor.execute('DELETE FROM w64_events WHERE creat
| ed_at < ?', (threshold_epoch,))
264 |         cursor.execute('DELETE FROM process_outliers WHERE
| created_at < ?', (threshold_epoch,))
265 |         cursor.execute('DELETE FROM device_events WHERE cr
| eated_at < ?', (threshold_epoch,))
266 |         cursor.execute('DELETE FROM system_extended_audits
| WHERE created_at < ?', (threshold_epoch,))
267 |         cursor.execute('DELETE FROM disk_io_events WHERE c
| reated_at < ?', (threshold_epoch,))
268 |         cursor.execute('DELETE FROM disk_performance_sampl
| es WHERE created_at < ?', (threshold_epoch,))
269 |         cursor.execute('DELETE FROM disk_health_snapshots
| WHERE created_at < ?', (threshold_epoch,))
270 |         cursor.execute('DELETE FROM cpu_telemetry_samples
| WHERE created_at < ?', (threshold_epoch,))
271 |         cursor.execute('DELETE FROM ram_telemetry_samples
| WHERE created_at < ?', (threshold_epoch,))
272 |         cursor.execute('DELETE FROM gpu_telemetry_samples
| WHERE created_at < ?', (threshold_epoch,))
273 |         cursor.execute('DELETE FROM network_adapter_sample
| s WHERE created_at < ?', (threshold_epoch,))
274 |         conn.commit()
275 |         deleted_count = len(old_ids)
276 |         logger.info(f'Очищено {deleted_count} устаревших с
| нимков телеметрии (старше {days} дн.)')
```

```

277
278         if vacuum_after and self.auto_vacuum_enabled:
279             self.vacuum()
280
281         return deleted_count
282
283     def enforce_size_limit(self, max_size_mb: Optional[float]
= None, target_ratio: float = 0.85) -> Dict[str, Any]:
284         """Ограничивает совокупный размер БД телеметрии."""
285         if self._cm.read_only:
286             return {'pruned': False, 'read_only': True}
287         limit_mb = max_size_mb if max_size_mb is not None else
self.max_db_size_mb
288         initial_sizes = self.get_db_file_sizes()
289         initial_size_mb = round(initial_sizes['total_bytes'] /
(1024.0 * 1024.0), 3)
290
291         if initial_size_mb <= limit_mb:
292             return {
293                 'pruned': False,
294                 'initial_size_mb': initial_size_mb,
295                 'final_size_mb': initial_size_mb,
296                 'max_size_mb': limit_mb,
297                 'deleted_snapshots': 0,
298                 'deleted_records': 0,
299             }
300
301         target_mb = max(0.5, limit_mb * target_ratio)
302         logger.warning(f'⚠ Превышен лимит размера базы данных:
{initial_size_mb:.2f} МБ > {limit_mb:.2f} МБ.')
303
304         total_deleted_snaps = self.cleanup_old_records(retenti
on_days=self.retention_days, vacuum_after=False)
305         self.checkpoint_wal()
306
307         with self._cm.lock, self._cm.get_connection() as conn:
308             cursor = conn.cursor()
309             while True:

```

```

310         cursor.execute('SELECT COUNT(*) FROM system_sn
    | apshots;')
311         snap_count = cursor.fetchone()[0]
312         if not snap_count or snap_count <= 5:
313             break
314         batch_size = max(5, int(snap_count * 0.25))
315         cursor.execute('SELECT id, created_at FROM sys
    | tem_snapshots ORDER BY created_at ASC LIMIT ?', (batch_size,))
316         rows = cursor.fetchall()
317         if not rows:
318             break
319         ids_to_del = [r[0] for r in rows]
320         cutoff_t = rows[-1][1]
321         placeholders = ','.join('?' for _ in ids_to_de
    | l)
322         cursor.execute(f'DELETE FROM process_snapshots
    | WHERE snapshot_id IN ({placeholders})', ids_to_del)
323         cursor.execute(f'DELETE FROM system_snapshots
    | WHERE id IN ({placeholders})', ids_to_del)
324         cursor.execute('DELETE FROM sensor_polls WHERE
    | created_at <= ?', (cutoff_t,))
325         cursor.execute('DELETE FROM w64_events WHERE c
    | reated_at <= ?', (cutoff_t,))
326         cursor.execute('DELETE FROM app_polls WHERE cr
    | eated_at <= ?', (cutoff_t,))
327         cursor.execute('DELETE FROM custom_records WHE
    | RE created_at <= ?', (cutoff_t,))
328         conn.commit()
329         total_deleted_snaps += len(ids_to_del)
330         if snap_count - len(ids_to_del) <= 5:
331             break
332
333         if self.auto_vacuum_enabled:
334             self.vacuum()
335         else:
336             self.checkpoint_wal()
337
338         final_sizes = self.get_db_file_sizes()

```



```

339         final_size_mb = round(final_sizes['total_bytes'] / (10
24.0 * 1024.0), 3)
340
341         return {
342             'pruned': True,
343             'initial_size_mb': initial_size_mb,
344             'final_size_mb': final_size_mb,
345             'max_size_mb': limit_mb,
346             'target_size_mb': target_mb,
347             'deleted_snapshots': total_deleted_snaps,
348         }
349
350     def aggregate_process_metrics_2min(self, cutoff_seconds: i
nt = 120, outlier_cpu_threshold: float = 30.0) -> Dict[str, in
t]:
351         """Обобщает процессы старше двух минут по средним пока
зателям."""
352         if self._cm.read_only:
353             return {'rollups_created': 0, 'outliers_saved': 0,
'raw_deleted': 0}
354         now_epoch = datetime.now(timezone.utc).timestamp()
355         cutoff_epoch = now_epoch - cutoff_seconds
356         with self._cm.lock, self._cm.get_connection() as conn:
357             cursor = conn.cursor()
358             cursor.execute('''
359                 SELECT p.id, p.snapshot_id, p.timestamp, p.pid
, p.name, p.status,
360                     p.cpu_percent, p.memory_mb, p.memory_pe
rcent, p.num_threads,
361                     p.username, s.created_at as snap_epoch
362                 FROM process_snapshots p
363                 JOIN system_snapshots s ON p.snapshot_id = s.i
d
364                 WHERE s.created_at < ?
365                 ORDER BY p.name, p.pid, s.created_at ASC
366                 ''', (cutoff_epoch,))
367         raw_rows = cursor.fetchall()
368         if not raw_rows:

```

```

369         return {'rollups_created': 0, 'outliers_saved'
370 : 0, 'raw_deleted': 0}
371
372         outlier_rows = []
373         for r in raw_rows:
374             cpu_val = float(r['cpu_percent'] or 0.0)
375             if cpu_val >= outlier_cpu_threshold:
376                 outlier_rows.append((
377                     r['snapshot_id'], r['timestamp'], r['s
378 nap_epoch'], r['pid'], r['name'],
379                     r['status'] or 'running', cpu_val, flo
380 at(r['memory_mb'] or 0.0),
381                     float(r['memory_percent'] or 0.0), int
382 (r['num_threads'] or 1),
383                     r['username'] or '', f'Outlier: CPU={c
384 pu_val:.1f}% ',
385                     ))
386
387         outliers_saved = 0
388         if outlier_rows:
389             cursor.executemany('''
390                 INSERT INTO process_outliers (
391                     snapshot_id, timestamp, created_at, pi
392 d, name, status,
393                     cpu_percent, memory_mb, memory_percent
394 , num_threads, username, details
395                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
396                 ?)
397
398             ''', outlier_rows)
399             outliers_saved = len(outlier_rows)
400
401         grouped = defaultdict(list)
402         for r in raw_rows:
403             grouped[(r['name'], r['pid'])].append(r)
404
405         rollup_rows = []
406         for (p_name, p_pid), procs in grouped.items():
407             cnt = len(procs)
408             cpus = [float(p['cpu_percent'] or 0.0) for p i

```

```

n procs]
399         mems = [float(p['memory_mb'] or 0.0) for p in
procs]
400         threads = [int(p['num_threads'] or 1) for p in
procs]
401         proc_outliers = sum(1 for c in cpus if c >= ou
tlier_cpu_threshold)
402         rollup_rows.append((
403             procs[0]['timestamp'], procs[-1]['timestam
p'], procs[0]['snap_epoch'], procs[-1]['snap_epoch'],
404             p_name, p_pid, procs[-1]['username'] or ' '
, cnt, round(sum(cpus) / cnt, 2),
405             round(max(cpus), 2), round(min(cpus), 2),
round(sum(mems) / cnt, 2),
406             round(max(mems), 2), round(min(mems), 2),
round(sum(threads) / cnt, 1), proc_outliers,
407             ))
408
409         if rollup_rows:
410             cursor.executemany('''
411                 INSERT INTO process_rollups_2min (
412                     period_start, period_end, period_start
_epoch, period_end_epoch,
413                     name, pid, username, sample_count, avg
_cpu_percent, max_cpu_percent,
414                     min_cpu_percent, avg_memory_mb, max_me
mory_mb, min_memory_mb,
415                     avg_num_threads, outliers_count
416                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?)
417                 ''', rollup_rows)
418
419         raw_ids = [r['id'] for r in raw_rows]
420         cursor.execute(f'''
421             DELETE FROM process_snapshots WHERE id IN ({',
'.join(['?'] * len(raw_ids))})
422             ''', raw_ids)
423         conn.commit()

```

```

424         return {'rollups_created': len(rollup_rows), 'outliers_saved': outliers_saved, 'raw_deleted': len(raw_ids)}
425
426     def aggregate_process_metrics_daily(self, cutoff_days: int = 1) -> Dict[str, int]:
427         """Обобщает агрегаты процессов в суточную статистику."""
428
429         if self._cm.read_only:
430             return {'daily_rollups_created': 0, '2min_rollups_cleaned': 0}
431
432         now_epoch = datetime.now(timezone.utc).timestamp()
433         cutoff_epoch = now_epoch - cutoff_days * 86400
434         with self._cm.lock, self._cm.get_connection() as conn:
435             cursor = conn.cursor()
436             cursor.execute('SELECT * FROM process_rollups_2min WHERE period_end_epoch < ? ORDER BY name, period_start_epoch ASC', (cutoff_epoch,))
437             old_rollups = cursor.fetchall()
438             if not old_rollups:
439                 return {'daily_rollups_created': 0, '2min_rollups_cleaned': 0}
440
441             grouped_daily = defaultdict(list)
442             for r in old_rollups:
443                 d_str = r['period_start'][:10] if len(r['period_start']) >= 10 else 'unknown'
444                 grouped_daily[(d_str, r['name'])].append(r)
445
446             daily_rows = []
447             for (d_str, p_name), items in grouped_daily.items():
448                 total_samples = sum(int(it['sample_count']) for it in items)
449                 if total_samples <= 0:
450                     continue
451                 weighted_cpu = sum(float(it['avg_cpu_percent']) * int(it['sample_count']) for it in items) / total_samples
452                 weighted_mem = sum(float(it['avg_memory_mb'])

```

```

    * int(it['sample_count']) for it in items) / total_samples
451 |         daily_rows.append((
452 |             d_str, p_name, total_samples, round(weight
    ed_cpu, 2), round(max(float(it['max_cpu_percent']) for it in i
    tems), 2),
453 |             round(weighted_mem, 2), round(max(float(it
    ['max_memory_mb']) for it in items), 2),
454 |             sum(int(it['outliers_count'] or 0) for it
    in items), max(it['period_end'] for it in items),
455 |             ))
456 |
457 |         if daily_rows:
458 |             cursor.executemany(''
459 |                 INSERT INTO process_rollups_daily (
460 |                     date, name, sample_count, avg_cpu_perc
    ent, max_cpu_percent,
461 |                     avg_memory_mb, max_memory_mb, outliers
    _count, last_seen
462 |                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
463 |             '', daily_rows)
464 |
465 |         del_ids = [r['id'] for r in old_rollups]
466 |         cursor.execute(f'DELETE FROM process_rollups_2min
    WHERE id IN ({','.join(['?'] * len(del_ids))})', del_ids)
467 |         conn.commit()
468 |         return {'daily_rollups_created': len(daily_rows),
    '2min_rollups_cleaned': len(del_ids)}
469 |

```

reader.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Reader
4 # =====
5 # Description:
6 #   Класс TelemetryReader для чтения и выборки данных телеметр
  ии из SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.reader import Telemet
  ryReader
11 #
12 #       reader = TelemetryReader(connection_manager)
13 #       snapshots = reader.get_snapshots(limit=10)
14 #
15 # File: reader.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 19:48:00
21 # =====
22
23 from __future__ import annotations
24
25 """Класс TelemetryReader для выполнения выборок, аналитики и ч
  тения данных телеметрии."""
26
27 import csv
28 import json
29 import sqlite3
```

```

30 import time
31 from datetime import datetime, timezone
32 from pathlib import Path
33 from typing import Any, Dict, List, Optional, Union
34
35 from .connection import TelemetryConnectionManager
36
37 try:
38     from logger import logger
39 except ImportError:
40     import logging
41     logger = logging.getLogger(__name__)
42
43
44 class TelemetryReader:
45     """Выполняет запросы чтения данных телеметрии, процессов,
46     сенсоров и аудита из базы SQLite."""
47
48     def __init__(self, connection_manager: TelemetryConnection
49     Manager) -> None:
50         """Инициализирует ридер телеметрии.
51
52         Args:
53             connection_manager: Менеджер подключений SQLite.
54         """
55         self._cm = connection_manager
56
57     def get_snapshots(self, limit: int = 60, since_epoch: Opti
58     onal[float] = None) -> List[Dict[str, Any]]:
59         """Извлекает исторические срезы системной телеметрии."
60
61         ""
62
63         with self._cm.lock, self._cm.get_connection() as conn:
64             cursor = conn.cursor()
65             if since_epoch is not None:
66                 cursor.execute(
67                     'SELECT * FROM system_snapshots WHERE crea
68                     ted_at >= ? ORDER BY id DESC LIMIT ?',
69                     (since_epoch, limit)

```

```

63         )
64         else:
65             cursor.execute('SELECT * FROM system_snapshots
ORDER BY id DESC LIMIT ?', (limit,))
66             return [dict(row) for row in cursor.fetchall()]
67
68     def get_latest_cpu(self) -> Optional[Dict[str, Any]]:
69         """Извлекает последние метрики процессора из таблицы s
system_snapshots."""
70         with self._cm.lock, self._cm.get_connection() as conn:
71             cursor = conn.cursor()
72             cursor.execute('''
73                 SELECT id, timestamp, created_at, hostname, cp
u_total_percent, cpu_frequency_mhz
74                 FROM system_snapshots
75                 ORDER BY id DESC
76                 LIMIT 1;
77             ''')
78             row = cursor.fetchone()
79             return dict(row) if row else None
80
81     def get_cpu_history(self, limit: int = 60, since_epoch: Op
tional[float] = None) -> List[Dict[str, Any]]:
82         """Извлекает историю нагрузки процессора."""
83         with self._cm.lock, self._cm.get_connection() as conn:
84             cursor = conn.cursor()
85             if since_epoch is not None:
86                 cursor.execute('''
87                     SELECT id, timestamp, created_at, cpu_tota
l_percent, cpu_frequency_mhz
88                     FROM system_snapshots
89                     WHERE created_at >= ?
90                     ORDER BY id DESC
91                     LIMIT ?;
92                 ''', (since_epoch, limit))
93             else:
94                 cursor.execute('''
95                     SELECT id, timestamp, created_at, cpu_tota

```



```

    l_percent, cpu_frequency_mhz
96         FROM system_snapshots
97         ORDER BY id DESC
98         LIMIT ?;
99         ''' , (limit,))
100         return [dict(row) for row in cursor.fetchall()]
101
102     def get_latest_storage_data(self) -> Dict[str, Any]:
103         """Извлекает последние данные о накопителях и разделах
104         ."""
105         with self._cm.lock, self._cm.get_connection() as conn:
106             cursor = conn.cursor()
107             cursor.execute('''
108                 SELECT id, timestamp, created_at, disks_json,
109                 physical_disks_json
110                 FROM system_snapshots
111                 ORDER BY id DESC
112                 LIMIT 1;
113             ''')
114             row = cursor.fetchone()
115             if not row:
116                 return {'partitions': [], 'physical_disks': []}
117
118             partitions: List[Dict[str, Any]] = []
119             if row['disks_json']:
120                 try:
121                     partitions = json.loads(row['disks_json'])
122                 except Exception:
123                     partitions = []
124
125             physical_disks: List[Dict[str, Any]] = []
126             if row['physical_disks_json']:
127                 try:
128                     physical_disks = json.loads(row['physical_
129                     disks_json'])
130                 except Exception:
131                     physical_disks = []

```

```

129
130         return {
131             'id': row['id'],
132             'timestamp': row['timestamp'],
133             'created_at': row['created_at'],
134             'partitions': partitions,
135             'physical_disks': physical_disks,
136         }
137
138     def get_latest_snapshot_full(self) -> Optional[Dict[str, Any]]:
139         """Извлекает и десериализует последний системный снимок со всеми метаданными и процессами."""
140         with self._cm.lock, self._cm.get_connection() as conn:
141             cursor = conn.cursor()
142             cursor.execute('SELECT * FROM system_snapshots ORDER BY id DESC LIMIT 1;')
143             row = cursor.fetchone()
144             if not row:
145                 return None
146
147             snap_row = dict(row)
148             snap_id = snap_row.get('id')
149
150             # 1. Попытка десериализации raw_json
151             raw_json_str = snap_row.get('raw_json')
152             full_dict: Dict[str, Any] = {}
153             if raw_json_str and isinstance(raw_json_str, str):
154                 try:
155                     full_dict = json.loads(raw_json_str)
156                 except Exception:
157                     full_dict = {}
158
159             # 2. Обогащение/fallback из именованных колонок
160             for field in ('hostname', 'username', 'uptime_seconds', 'os_name', 'os_build', 'os_install_date',
161                         'system_language', 'os_install_language', 'user_locale', 'system_locale', 'timezone', 'codepage',

```

```

162         'cpu_total_percent', 'cpu_frequency_
| mhz', 'memory_total_gb', 'memory_used_gb', 'memory_percent',
163         'swap_percent', 'gpu_load_percent',
| 'gpu_temp_c', 'disk_read_bytes_sec', 'disk_write_bytes_sec',
164         'disk_read_count_sec', 'disk_write_c
| ount_sec', 'network_sent_bytes_sec', 'network_recv_bytes_sec',
| 'timestamp'):
165         if snap_row.get(field) is not None and (field
| not in full_dict or not full_dict[field]):
166             full_dict[field] = snap_row[field]
167
168     # 3. Десериализация JSON-колонок
169     json_column_mappings = {
170         'input_languages_json': 'input_languages',
171         'disks_json': 'disks',
172         'physical_disks_json': 'physical_disks',
173         'monitors_json': 'monitors',
174         'updates_json': 'updates',
175         'office_json': 'office',
176         'onedrive_json': 'onedrive',
177         'battery_json': 'battery',
178         'ram_sticks_json': 'ram_sticks',
179         'listening_ports_json': 'listening_ports',
180         'alerts_json': 'alerts',
181         'hardware_audit': 'hardware_audit',
182     }
183     for col_name, target_key in json_column_mappings.i
| tems():
184         val_str = snap_row.get(col_name)
185         if val_str and isinstance(val_str, str) and (t
| arget_key not in full_dict or not full_dict[target_key]):
186             try:
187                 full_dict[target_key] = json.loads(val
| _str)
188             except Exception:
189                 pass
190
191     # 4. Процессы (если не были в raw_json)

```

```

192         if 'top_processes' not in full_dict or not full_dict.get('top_processes'):
193             if snap_id:
194                 cursor.execute(
195                     'SELECT pid, name, status, cpu_percent
196                     , memory_mb, memory_percent, num_threads, num_handles, '
197                     'username, read_bytes_sec, write_bytes_sec, integrity_level, elevation, ppid, parent_name, '
198                     'executable as executable_path, cmdline, sid, session_id, creation_time, process_guid, ancestor_chain, launch_reason '
199                     'FROM process_snapshots WHERE snapshot_id = ? ORDER BY cpu_percent DESC LIMIT 50;',
200                     (snap_id,)
201                 )
202                 full_dict['top_processes'] = [dict(p) for p in cursor.fetchall()]
203
204             # 5. Структурирование CPU и Memory если отсутствуют объекты
205             if 'cpu' not in full_dict or not isinstance(full_dict.get('cpu'), dict):
206                 full_dict['cpu'] = {
207                     'total_percent': float(snap_row.get('cpu_total_percent') or 0.0),
208                     'frequency_mhz': float(snap_row.get('cpu_frequency_mhz') or 0.0),
209                     'model': full_dict.get('cpu_model', 'Intel / AMD Processor'),
210                     'architecture': 'x86_64',
211                 }
212             if 'memory' not in full_dict or not isinstance(full_dict.get('memory'), dict):
213                 tot_m = float(snap_row.get('memory_total_gb') or 0.0)
214                 used_m = float(snap_row.get('memory_used_gb') or 0.0)
215                 full_dict['memory'] = {

```

```

215         'total_gb': tot_m,
216         'used_gb': used_m,
217         'available_gb': max(0.0, tot_m - used_m),
218         'percent': float(snap_row.get('memory_perc
ent') or 0.0),
219         'swap_percent': float(snap_row.get('swap_p
ercent') or 0.0),
220     }
221     if 'disk_io' not in full_dict or not isinstance(fu
ll_dict.get('disk_io'), dict):
222         full_dict['disk_io'] = {
223             'read_bytes_per_sec': float(snap_row.get('
disk_read_bytes_sec') or 0.0),
224             'write_bytes_per_sec': float(snap_row.get(
'disk_write_bytes_sec') or 0.0),
225             'read_count_per_sec': float(snap_row.get('
disk_read_count_sec') or 0.0),
226             'write_count_per_sec': float(snap_row.get(
'disk_write_count_sec') or 0.0),
227         }
228
229     return full_dict
230
231     def get_snapshot_processes(self, snapshot_id: int) -> List
[Dict[str, Any]]:
232         """Извлекает список процессов, зафиксированных в снимк
е."""
233         with self._cm.lock, self._cm.get_connection() as conn:
234             cursor = conn.cursor()
235             cursor.execute(
236                 'SELECT * FROM process_snapshots WHERE snapsho
t_id = ? ORDER BY cpu_percent DESC, memory_mb DESC',
237                 (snapshot_id,)
238             )
239             return [dict(row) for row in cursor.fetchall()]
240
241     def get_process_history(
242         self,

```

```

243         name: Optional[str] = None,
244         pid: Optional[int] = None,
245         limit: int = 100,
246     ) -> List[Dict[str, Any]]:
247         """Извлекает историю поведения процесса по имени или P
ID."""
248         with self._cm.lock, self._cm.get_connection() as conn:
249             cursor = conn.cursor()
250             if pid is not None and name is not None:
251                 cursor.execute(
252                     'SELECT * FROM process_snapshots WHERE pid
= ? AND name LIKE ? ORDER BY id DESC LIMIT ?',
253                     (pid, f'%{name}%', limit)
254                 )
255             elif pid is not None:
256                 cursor.execute(
257                     'SELECT * FROM process_snapshots WHERE pid
= ? ORDER BY id DESC LIMIT ?',
258                     (pid, limit)
259                 )
260             elif name is not None:
261                 cursor.execute(
262                     'SELECT * FROM process_snapshots WHERE nam
e LIKE ? ORDER BY id DESC LIMIT ?',
263                     (f'%{name}%', limit)
264                 )
265             else:
266                 cursor.execute('SELECT * FROM process_snapshots
ORDER BY id DESC LIMIT ?', (limit,))
267             return [dict(row) for row in cursor.fetchall()]
268
269     def get_latest_processes(self, limit: int = 50, sort_by: str = 'cpu') -> List[Dict[str, Any]]:
270         """Извлекает процессы из самого последнего зафиксированного снимка."""
271         sort_key = (sort_by or 'cpu').lower()
272         if sort_key in ('handles', 'descriptors'):
273             order_col = 'num_handles DESC, cpu_percent DESC'

```

```

274         elif sort_key in ('memory', 'ram', 'memory_mb'):
275             order_col = 'memory_mb DESC, cpu_percent DESC'
276         elif sort_key == 'memory_percent':
277             order_col = 'memory_percent DESC, cpu_percent DESC'
278         elif sort_key == 'name':
279             order_col = 'name ASC'
280         elif sort_key == 'pid':
281             order_col = 'pid ASC'
282         else:
283             order_col = 'cpu_percent DESC, memory_mb DESC'
284
285         with self._cm.lock, self._cm.get_connection() as conn:
286             cursor = conn.cursor()
287             cursor.execute('SELECT MAX(id) FROM system_snapshots;')
288             row = cursor.fetchone()
289             last_snap_id = row[0] if row and row[0] is not None else None
290             if last_snap_id is None:
291                 cursor.execute('SELECT MAX(snapshot_id) FROM process_snapshots;')
292                 row = cursor.fetchone()
293                 last_snap_id = row[0] if row and row[0] is not None else None
294             if last_snap_id is None:
295                 return []
296
297             sql = f'''
298             SELECT pid, name, status, cpu_percent, memory_
299             mb, memory_percent,
300             num_threads, num_handles, username, rea
301             d_bytes_sec, write_bytes_sec,
302             integrity_level, elevation, ppid, paren
303             t_name, executable, cmdline, timestamp
304             FROM process_snapshots
305             WHERE snapshot_id = ?
306             ORDER BY {order_col}

```

```

304         '''
305         if limit is not None and limit > 0:
306             cursor.execute(sql + ' LIMIT ?', (last_snap_id
307 | , limit))
308         else:
309             cursor.execute(sql, (last_snap_id,))
310             return [dict(r) for r in cursor.fetchall()]
311
312     def get_process_stats(self, name: Optional[str] = None, li
313 | mit: int = 50) -> Dict[str, Any]:
314         """Возвращает агрегированную статистику процессов и вы
315 | бросы."""
316         with self._cm.lock, self._cm.get_connection() as conn:
317             cursor = conn.cursor()
318             if name:
319                 cursor.execute('SELECT * FROM process_rollups_
320 | 2min WHERE name LIKE ? ORDER BY id DESC LIMIT ?', (f'%{name}%'
321 | , limit))
322             else:
323                 cursor.execute('SELECT * FROM process_rollups_
324 | 2min ORDER BY id DESC LIMIT ?', (limit,))
325             rollups_2min = [dict(r) for r in cursor.fetchall()
326 | ]
327
328             if name:
329                 cursor.execute('SELECT * FROM process_rollups_
330 | daily WHERE name LIKE ? ORDER BY date DESC LIMIT ?', (f'%{name}%'
331 | , limit))
332             else:
333                 cursor.execute('SELECT * FROM process_rollups_
334 | daily ORDER BY date DESC LIMIT ?', (limit,))
335             daily_stats = [dict(r) for r in cursor.fetchall()]
336
337             if name:
338                 cursor.execute('SELECT * FROM process_outliers
339 | WHERE name LIKE ? ORDER BY id DESC LIMIT ?', (f'%{name}%', li
340 | mit))
341             else:

```



```

330         cursor.execute('SELECT * FROM process_outliers
ORDER BY id DESC LIMIT ?', (limit,))
331         outliers = [dict(r) for r in cursor.fetchall()]
332
333         return {
334             'name_filter': name,
335             'rollups_2min': rollups_2min,
336             'daily_stats': daily_stats,
337             'outliers': outliers,
338             'total_rollups_count': len(rollups_2min),
339             'total_outliers_count': len(outliers),
340         }
341
342     def get_events(self, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
343         """Извлекает список зафиксированных событий телеметрии
344         ."""
345         with self._cm.lock, self._cm.get_connection() as conn:
346             cursor = conn.cursor()
347             if event_type:
348                 cursor.execute('SELECT * FROM telemetry_events
WHERE event_type = ? ORDER BY id DESC LIMIT ?', (event_type,
limit))
349             else:
350                 cursor.execute('SELECT * FROM telemetry_events
ORDER BY id DESC LIMIT ?', (limit,))
351             return [dict(row) for row in cursor.fetchall()]
352
353     def get_sensor_history(
354         self,
355         sensor_id: Optional[str] = None,
356         category: Optional[str] = None,
357         limit: int = 100,
358     ) -> List[Dict[str, Any]]:
359         """Извлекает историю показаний сенсоров."""
360         with self._cm.lock, self._cm.get_connection() as conn:
361             cursor = conn.cursor()
362             if sensor_id and category:

```

```

362         cursor.execute(
363             'SELECT * FROM sensor_polls WHERE sensor_id = ? AND sensor_category = ? ORDER BY id DESC LIMIT ?',
364             (sensor_id, category, limit)
365         )
366         elif sensor_id:
367             cursor.execute('SELECT * FROM sensor_polls WHERE sensor_id = ? ORDER BY id DESC LIMIT ?', (sensor_id, limit)
368             )
369         elif category:
370             cursor.execute('SELECT * FROM sensor_polls WHERE sensor_category = ? ORDER BY id DESC LIMIT ?', (category, limit))
371         else:
372             cursor.execute('SELECT * FROM sensor_polls ORDER BY id DESC LIMIT ?', (limit,))
373         return [dict(row) for row in cursor.fetchall()]
374
375     def get_latest_sensors(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
376         """Извлекает последнее актуальное значение каждого сенсора."""
377         with self._cm.lock, self._cm.get_connection() as conn:
378             cursor = conn.cursor()
379             query = '''
380                 SELECT s1.* FROM sensor_polls s1
381                 INNER JOIN (
382                     SELECT sensor_id, MAX(id) as max_id
383                     FROM sensor_polls
384                     GROUP BY sensor_id
385                 ) s2 ON s1.id = s2.max_id
386                 ORDER BY s1.sensor_category, s1.sensor_name
387             '''
388             if limit is not None:
389                 cursor.execute(query + ' LIMIT ?', (limit,))
390             else:
391                 cursor.execute(query)
392             return [dict(row) for row in cursor.fetchall()]

```

```

392
393     def get_cpu_hierarchy_metrics(self) -> Dict[str, Any]:
394         """Извлекает структурированные иерархические метрики п
процессора из SQLite.
395
396         Выбирает последние замеры сенсоров CPU, системные сним
ки и историю нагрузки/температур.
397
398         Returns:
399             Dict[str, Any]: Словарь с сенсорами, снимками, пас
портом и историей.
400         """
401         with self._cm.lock, self._cm.get_connection() as conn:
402             cursor = conn.cursor()
403
404             # 1. Последние сенсоры CPU из sensor_polls (исключ
ая дисковые и GPU метрики)
405             query_sensors = '''
406                 SELECT s1.* FROM sensor_polls s1
407                 INNER JOIN (
408                     SELECT sensor_id, MAX(id) as max_id
409                     FROM sensor_polls
410                     WHERE hardware_type = 'cpu' OR (
411                         hardware_name NOT LIKE '%GPU%'
412                         AND hardware_name NOT LIKE '%Storage%'
413                         AND hardware_name NOT LIKE '%Disk%'
414                         AND sensor_id NOT LIKE 'disk_speed_%'
415                         AND (hardware_type = 'system' OR senso
r_category IN ('Voltages', 'Powers', 'Clocks', 'Temperatures',
'Load', 'voltages', 'powers', 'clocks', 'temperatures', 'load
'))
416                     )
417                 GROUP BY sensor_id
418                 ) s2 ON s1.id = s2.max_id
419                 ORDER BY s1.sensor_category, s1.sensor_name
420             '''
421             cursor.execute(query_sensors)
422             sensors = [dict(row) for row in cursor.fetchall()]

```

```

423 |
424 |         # 2. Последний системный снимок
425 |         cursor.execute('SELECT * FROM system_snapshots ORD
ER BY id DESC LIMIT 1')
426 |         snap_row = cursor.fetchone()
427 |         latest_snapshot = dict(snap_row) if snap_row else
None
428 |
429 |         # 3. Данные инвентаризации CPU
430 |         cursor.execute('SELECT * FROM cpu_inventory ORDER
BY processor_id ASC LIMIT 1')
431 |         inv_row = cursor.fetchone()
432 |         if inv_row:
433 |             cpu_inv = dict(inv_row)
434 |         else:
435 |             cpu_inv = self._probe_and_persist_cpu_inventor
y(cursor)
436 |
437 |         # 4. История снимков для графиков
438 |         cursor.execute('''
439 |             SELECT timestamp, created_at, cpu_total_perce
nt, cpu_frequency_mhz
440 |             FROM system_snapshots
441 |             ORDER BY id DESC
442 |             LIMIT 60
443 |         ''')
444 |         history_rows = [dict(r) for r in cursor.fetchall()
]
445 |         history_rows.reverse()
446 |
447 |         return {
448 |             'sensors': sensors,
449 |             'snapshot': latest_snapshot,
450 |             'inventory': cpu_inv,
451 |             'history': history_rows,
452 |         }
453 |
454 |     def _probe_and_persist_cpu_inventory(self, cursor: sqlite3

```

```

.Cursor) -> Dict[str, Any]:
455     """Опрашивает спецификации CPU и сохраняет их в cpu_in
    ventory."""
456     import os
457     import platform
458     import time
459     from datetime import datetime, timezone
460     try:
461         import psutil
462     except ImportError:
463         psutil = None
464
465     vendor = 'Intel'
466     model = platform.processor() or 'Processor'
467     arch = 'x86_64' if platform.machine() in ('AMD64', 'x8
6_64') else platform.machine()
468     phys_cores = psutil.cpu_count(logical=False) if psutil
    else 1
469     log_cores = psutil.cpu_count(logical=True) if psutil e
    lse 1
470     freq = psutil.cpu_freq() if psutil else None
471     base_mhz = float(freq.current if freq else 2900.0)
472     max_mhz = float(freq.max if (freq and freq.max) else b
    ase_mhz)
473     socket = ''
474     l2_kb = None
475     l3_kb = None
476     stepping = ''
477     features: List[str] = []
478
479     if os.name == 'nt':
480         try:
481             import winreg
482             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
                r'HARDWARE\DESCRIPTION\System\CentralProcessor\0') as k:
483                 reg_name, _ = winreg.QueryValueEx(k, 'Proc
                essorNameString')
484                 reg_vendor, _ = winreg.QueryValueEx(k, 'Ve

```

```

ndorIdentifier')
485         reg_mhz, _ = winreg.QueryValueEx(k, '~MHz'
    )
486         reg_ident, _ = winreg.QueryValueEx(k, 'Ide
ntifier')
487         if reg_name:
488             model = str(reg_name).strip()
489         if reg_vendor:
490             v_str = str(reg_vendor).strip()
491             vendor = 'Intel' if 'intel' in v_str.l
ower() else ('AMD' if 'amd' in v_str.lower() else v_str)
492         if reg_mhz:
493             base_mhz = float(reg_mhz)
494         if reg_ident:
495             stepping = str(reg_ident).strip()
496     except Exception:
497         pass
498
499     try:
500         import win32com.client
501         wmi_obj = win32com.client.GetObject('winmgmts:
    ')
502         for p in wmi_obj.InstancesOf('Win32_Processor'
    ):
503             if getattr(p, 'Name', None):
504                 model = str(p.Name).strip()
505             if getattr(p, 'SocketDesignation', None):
506                 socket = str(p.SocketDesignation).stri
p()
507             if getattr(p, 'L2CacheSize', None):
508                 l2_kb = int(p.L2CacheSize)
509             if getattr(p, 'L3CacheSize', None):
510                 l3_kb = int(p.L3CacheSize)
511             if getattr(p, 'MaxClockSpeed', None):
512                 max_mhz = float(p.MaxClockSpeed)
513             if getattr(p, 'Manufacturer', None):
514                 m_str = str(p.Manufacturer).strip()
515                 vendor = 'Intel' if 'intel' in m_str.l

```

```

    ower() else ('AMD' if 'amd' in m_str.lower() else m_str)
516         if getattr(p, 'NumberOfCores', None):
517             phys_cores = int(p.NumberOfCores)
518             if getattr(p, 'NumberOfLogicalProcessors',
None):
519                 log_cores = int(p.NumberOfLogicalProce
ssors)
520                 break
521         except Exception:
522             pass
523
524         now_iso = datetime.now(timezone.utc).isoformat()
525         now_epoch = time.time()
526
527         cpu_inv = {
528             'processor_id': 0,
529             'name': model,
530             'vendor': vendor,
531             'architecture': arch,
532             'physical_cores': phys_cores or 1,
533             'logical_cores': log_cores or 1,
534             'base_frequency_mhz': base_mhz,
535             'max_frequency_mhz': max_mhz,
536             'l2_cache_kb': l2_kb,
537             'l3_cache_kb': l3_kb,
538             'socket': socket,
539             'stepping': stepping,
540             'features_json': json.dumps(features),
541             'first_seen': now_iso,
542             'last_seen': now_iso,
543             'updated_at': now_epoch,
544         }
545
546         try:
547             cursor.execute('''
548                 INSERT OR REPLACE INTO cpu_inventory (
549                     processor_id, name, vendor, architecture,
physical_cores, logical_cores,

```

```

550         base_frequency_mhz, max_frequency_mhz, l2_
    cache_kb, l3_cache_kb, socket,
551         features_json, first_seen, last_seen, upda
    ted_at
552         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
    ?, ?, ?)
553         ''' , (
554             cpu_inv['processor_id'], cpu_inv['name'], cpu_
    inv['vendor'], cpu_inv['architecture'],
555             cpu_inv['physical_cores'], cpu_inv['logical_co
    res'], cpu_inv['base_frequency_mhz'],
556             cpu_inv['max_frequency_mhz'], cpu_inv['l2_cach
    e_kb'], cpu_inv['l3_cache_kb'],
557             cpu_inv['socket'], cpu_inv['features_json'], c
    pu_inv['first_seen'],
558             cpu_inv['last_seen'], cpu_inv['updated_at']
559         ))
560     except Exception as ex:
561         logger.debug(f"[reader] Не удалось сохранить cpu_i
    nventory: {ex}")
562
563     return cpu_inv
564
565     def get_app_polls(self, app: Optional[str] = None, metric_
    name: Optional[str] = None, limit: int = 100) -> List[Dict[str
    , Any]]:
566         """Извлекает историю опросов приложений."""
567         with self._cm.lock, self._cm.get_connection() as conn:
568             cursor = conn.cursor()
569             if app and metric_name:
570                 cursor.execute('SELECT * FROM app_polls WHERE
    app = ? AND metric_name = ? ORDER BY id DESC LIMIT ?', (app, m
    etric_name, limit))
571             elif app:
572                 cursor.execute('SELECT * FROM app_polls WHERE
    app = ? ORDER BY id DESC LIMIT ?', (app, limit))
573             else:
574                 cursor.execute('SELECT * FROM app_polls ORDER

```



```

        BY id DESC LIMIT ?', (limit,))
575         return [dict(row) for row in cursor.fetchall()]
576
577     def get_app_events(self, app: Optional[str] = None, event_
type: Optional[str] = None, limit: int = 100) -> List[Dict[str
, Any]]:
578         """Извлекает события приложений."""
579         with self._cm.lock, self._cm.get_connection() as conn:
580             cursor = conn.cursor()
581             if app and event_type:
582                 cursor.execute('SELECT * FROM app_events WHERE
app = ? AND event_type = ? ORDER BY id DESC LIMIT ?', (app, e
vent_type, limit))
583             elif app:
584                 cursor.execute('SELECT * FROM app_events WHERE
app = ? ORDER BY id DESC LIMIT ?', (app, limit))
585             else:
586                 cursor.execute('SELECT * FROM app_events ORDER
BY id DESC LIMIT ?', (limit,))
587             return [dict(row) for row in cursor.fetchall()]
588
589     def get_app_param_changes(self, app: Optional[str] = None,
limit: int = 100) -> List[Dict[str, Any]]:
590         """Извлекает изменения параметров приложений."""
591         with self._cm.lock, self._cm.get_connection() as conn:
592             cursor = conn.cursor()
593             if app:
594                 cursor.execute('SELECT * FROM app_param_change
s WHERE app = ? ORDER BY id DESC LIMIT ?', (app, limit))
595             else:
596                 cursor.execute('SELECT * FROM app_param_change
s ORDER BY id DESC LIMIT ?', (limit,))
597             return [dict(row) for row in cursor.fetchall()]
598
599     def export_app_polls_to_csv(self, app: Optional[str] = Non
e, output_path: Optional[Union[str, Path]] = None, limit: int
= 50000) -> Path:
600         """Экспортирует историю опросов приложений в CSV-файл.

```

```

"""
601         polls = self.get_app_polls(app=app, limit=limit)
602         target = Path(output_path) if output_path else Path('a
pp_polls.csv')
603         target.parent.mkdir(parents=True, exist_ok=True)
604         headers = ['timestamp', 'app', 'poll_type', 'metric_na
me', 'value', 'unit', 'status', 'details']
605         with open(target, 'w', newline='', encoding='utf-8-sig
') as f:
606             writer = csv.writer(f)
607             writer.writerow(headers)
608             for p in polls:
609                 writer.writerow([
610                     p.get('timestamp', ''), p.get('app', ''),
p.get('poll_type', ''),
611                     p.get('metric_name', ''), p.get('value', '
'), p.get('unit', ''),
612                     p.get('status', ''), p.get('details', ''),
613                     ])
614         return target
615
616     def export_app_events_to_csv(self, app: Optional[str] = No
ne, output_path: Optional[Union[str, Path]] = None, limit: int
= 50000) -> Path:
617         """Экспортирует историю событий приложений в CSV-файл.
"""
618         events = self.get_app_events(app=app, limit=limit)
619         target = Path(output_path) if output_path else Path('a
pp_events.csv')
620         target.parent.mkdir(parents=True, exist_ok=True)
621         headers = ['timestamp', 'app', 'event_type', 'status',
'user', 'details']
622         with open(target, 'w', newline='', encoding='utf-8-sig
') as f:
623             writer = csv.writer(f)
624             writer.writerow(headers)
625             for e in events:
626                 writer.writerow([

```

```

627         e.get('timestamp', ''), e.get('app', ''),
        e.get('event_type', ''),
628         e.get('status', ''), e.get('user', ''), e.
        get('details', ''),
629     ])
630     return target
631
632     def export_app_param_changes_to_csv(self, app: Optional[str] = None, output_path: Optional[Union[str, Path]] = None, limit: int = 50000) -> Path:
633         """Экспортирует историю изменения параметров в CSV-файл."""
634         params = self.get_app_param_changes(app=app, limit=limit)
635         target = Path(output_path) if output_path else Path('app_param_changes.csv')
636         target.parent.mkdir(parents=True, exist_ok=True)
637         headers = ['timestamp', 'app', 'param_name', 'old_value', 'new_value', 'status', 'user', 'details']
638         with open(target, 'w', newline='', encoding='utf-8-sig') as f:
639             writer = csv.writer(f)
640             writer.writerow(headers)
641             for p in params:
642                 writer.writerow([
643                     p.get('timestamp', ''), p.get('app', ''),
        p.get('param_name', ''),
644                     p.get('old_value', ''), p.get('new_value',
        ''), p.get('status', ''),
645                     p.get('user', ''), p.get('details', ''),
646                 ])
647             return target
648
649     def get_custom_records(self, source_file: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
650         """Извлекает кастомные записи логов."""
651         with self._cm.lock, self._cm.get_connection() as conn:
652             cursor = conn.cursor()

```

```

653         if source_file:
654             cursor.execute('SELECT * FROM custom_records W
HERE source_file = ? ORDER BY id DESC LIMIT ?', (source_file,
limit))
655         else:
656             cursor.execute('SELECT * FROM custom_records O
RDER BY id DESC LIMIT ?', (limit,))
657         return [dict(row) for row in cursor.fetchall()]
658
659     def get_device_events(self, event_type: Optional[str] = No
ne, device_instance_id: Optional[str] = None, limit: int = 100
) -> List[Dict[str, Any]]:
660         """Извлекает события устройств."""
661         with self._cm.lock, self._cm.get_connection() as conn:
662             cursor = conn.cursor()
663             if event_type and device_instance_id:
664                 cursor.execute('SELECT * FROM device_events WH
ERE event_type = ? AND device_instance_id = ? ORDER BY id DESC
LIMIT ?', (event_type, device_instance_id, limit))
665             elif event_type:
666                 cursor.execute('SELECT * FROM device_events WH
ERE event_type = ? ORDER BY id DESC LIMIT ?', (event_type, lim
it))
667             elif device_instance_id:
668                 cursor.execute('SELECT * FROM device_events WH
ERE device_instance_id = ? ORDER BY id DESC LIMIT ?', (device_
instance_id, limit))
669             else:
670                 cursor.execute('SELECT * FROM device_events OR
DER BY id DESC LIMIT ?', (limit,))
671             return [dict(row) for row in cursor.fetchall()]
672
673     def get_device_install_date(self, device_instance_id: str)
-> Optional[str]:
674         """Возвращает дату установки устройства из таблицы dev
ice_inventory."""
675         norm_id = str(device_instance_id).strip().upper()
676         if not norm_id:

```

```

677         return None
678     with self._cm.lock, self._cm.get_connection() as conn:
679         cursor = conn.cursor()
680         cursor.execute('SELECT install_date FROM device_in
ventory WHERE UPPER(device_instance_id) = ?', (norm_id,))
681         row = cursor.fetchone()
682         return str(row[0]) if (row and row[0]) else None
683
684     def get_w64_events(self, event_type: Optional[str] = None,
provider: Optional[str] = None, limit: int = 100) -> List[Dic
t[str, Any]]:
685         """Извлекает системные события W64/ETW."""
686         query = 'SELECT raw_json FROM w64_events WHERE 1=1'
687         params: List[Any] = []
688         if event_type:
689             query += ' AND event_type = ?'
690             params.append(event_type)
691         if provider:
692             query += ' AND provider = ?'
693             params.append(provider)
694         query += ' ORDER BY created_at DESC LIMIT ?'
695         params.append(max(1, limit))
696
697         with self._cm.lock, self._cm.get_connection() as conn:
698             cursor = conn.cursor()
699             cursor.execute(query, tuple(params))
700             results = []
701             for row in cursor.fetchall():
702                 try:
703                     results.append(json.loads(row[0]))
704                 except Exception:
705                     continue
706             return results
707
708     def get_incidents(self, limit: int = 50, trigger_type: Opt
ional[str] = None, severity: Optional[str] = None) -> List[Dic
t[str, Any]]:
709         """Возвращает список зафиксированных инцидентов."""

```

```

710 |         query = 'SELECT * FROM incidents WHERE 1=1'
711 |         params: List[Any] = []
712 |         if trigger_type:
713 |             query += ' AND trigger_type = ?'
714 |             params.append(trigger_type)
715 |         if severity:
716 |             query += ' AND severity = ?'
717 |             params.append(severity)
718 |         query += ' ORDER BY created_at DESC LIMIT ?'
719 |         params.append(limit)
720 |
721 |         results: List[Dict[str, Any]] = []
722 |         try:
723 |             with self._cm.lock, self._cm.get_connection() as c
724 |             conn:
725 |                 cursor = conn.cursor()
726 |                 cursor.execute(query, tuple(params))
727 |                 for row in cursor.fetchall():
728 |                     item = dict(row)
729 |                     for f in ('trigger_metrics_json', 'suspect
730 | _processes_json', 'related_events_json', 'metrics_summary_json
731 | ', 'raw_window_json'):
732 |                         if item.get(f):
733 |                             try:
734 |                                 item[f.replace('_json', '')] =
735 | json.loads(item[f])
736 |                             except Exception:
737 |                                 item[f.replace('_json', '')] =
738 | []
739 |                             results.append(item)
740 |         except Exception as ex:
741 |             logger.error(f'Ошибка при получении инцидентов: {e
742 | x}')
743 |         return results
744 |
745 |     def get_system_rollups(self, tier: str = '1m', start_epoch
746 | : Optional[float] = None, end_epoch: Optional[float] = None, l
747 | imit: int = 100) -> List[Dict[str, Any]]:

```

```

740 |         """Возвращает сжатые бакеты метрик за период."""
741 |         query = 'SELECT * FROM telemetry_rollups WHERE tier =
|         ?'
742 |         params: List[Any] = [tier]
743 |         if start_epoch is not None:
744 |             query += ' AND period_start_epoch >= ?'
745 |             params.append(start_epoch)
746 |         if end_epoch is not None:
747 |             query += ' AND period_end_epoch <= ?'
748 |             params.append(end_epoch)
749 |         query += ' ORDER BY period_start_epoch DESC LIMIT ?'
750 |         params.append(limit)
751 |
752 |         results: List[Dict[str, Any]] = []
753 |         try:
754 |             with self._cm.lock, self._cm.get_connection() as c
|         onn:
755 |                 cursor = conn.cursor()
756 |                 cursor.execute(query, tuple(params))
757 |                 for row in cursor.fetchall():
758 |                     results.append(dict(row))
759 |         except Exception as ex:
760 |             logger.error(f'Ошибка при получении rollups: {ex}')
761 |         )
762 |         return results
763 |
764 |     def get_reboot_history(self, limit: int = 50) -> List[Dict
|     [str, Any]]:
765 |         """Извлекает историю сессий перезагрузок."""
766 |         results: List[Dict[str, Any]] = []
767 |         try:
768 |             with self._cm.lock, self._cm.get_connection() as c
|         onn:
769 |                 cursor = conn.cursor()
770 |                 cursor.execute('SELECT * FROM reboot_history O
|                 RDER BY boot_time DESC, created_at DESC LIMIT ?', (limit,))
771 |                 for row in cursor.fetchall():
|                     item = dict(row)

```

```

772         for f in ('bugcheck_params_json', 'evidenc
e_json', 'events_chain_json', 'raw_json'):
773             if item.get(f):
774                 try:
775                     item[f.replace('_json', '')] =
json.loads(item[f])
776                 except Exception:
777                     pass
778                 results.append(item)
779             except Exception as ex:
780                 logger.error(f'Ошибка извлечения истории перезагру
зок: {ex}')
781         return results
782
783     def get_latest_reboot_session(self) -> Optional[Dict[str,
Any]]:
784         """Возвращает самую последнюю запись сессии перезагруз
ки."""
785         history = self.get_reboot_history(limit=1)
786         return history[0] if history else None
787
788     def get_extended_audits(self, limit: int = 50) -> List[Dic
t[str, Any]]:
789         """Возвращает историю расширенных системных аудитов."""
790
791         results: List[Dict[str, Any]] = []
792         try:
793             with self._cm.lock, self._cm.get_connection() as c
onn:
794                 cursor = conn.cursor()
795                 cursor.execute('SELECT * FROM system_extended_
audits ORDER BY created_at DESC LIMIT ?', (limit,))
796                 for row in cursor.fetchall():
797                     item = dict(row)
798                     if item.get('raw_json'):
799                         try:
800                             item['data'] = json.loads(item['ra
w_json'])

```



```

800         except Exception:
801             pass
802             results.append(item)
803     except Exception as ex:
804         logger.error(f'Ошибка извлечения расширенных аудит
ов: {ex}')
805     return results
806
807     def get_latest_extended_audit(self) -> Optional[Dict[str,
Any]]:
808         """Возвращает последний расширенный системный аудит."""
809
810         audits = self.get_extended_audits(limit=1)
811         return audits[0] if audits else None
812
813     def get_hardware_audits(self, limit: int = 50) -> List[Dic
t[str, Any]]:
814         """Возвращает историю аудитов аппаратного обеспечения
из таблицы hardware_audits."""
815         results: List[Dict[str, Any]] = []
816         try:
817             with self._cm.lock, self._cm.get_connection() as c
onn:
818                 cursor = conn.cursor()
819                 cursor.execute('SELECT * FROM hardware_audits
ORDER BY created_at DESC LIMIT ?', (limit,))
820                 for row in cursor.fetchall():
821                     item = dict(row)
822                     if item.get('raw_json'):
823                         try:
824                             item['data'] = json.loads(item['ra
w_json'])
825                         except Exception:
826                             pass
827                         results.append(item)
828         except Exception as ex:
829             logger.error(f'Ошибка извлечения аудитов оборудова
ния: {ex}')

```

```

829         return results
830
831     def get_latest_hardware_audit(self) -> Optional[Dict[str,
Any]]:
832         """Возвращает самый последний сохраненный аудит аппара
тного обеспечения."""
833         audits = self.get_hardware_audits(limit=1)
834         return audits[0] if audits else None
835
836     def get_startup_archives(self, limit: int = 50) -> List[Di
ct[str, Any]]:
837         """Возвращает историю архивных снимков автозапуска из
таблицы startup_audit_archives."""
838         results: List[Dict[str, Any]] = []
839         try:
840             with self._cm.lock, self._cm.get_connection() as c
onn:
841                 cursor = conn.cursor()
842                 cursor.execute('SELECT * FROM startup_audit_ar
chives ORDER BY created_at DESC LIMIT ?', (limit,))
843                 for row in cursor.fetchall():
844                     item = dict(row)
845                     if item.get('raw_json'):
846                         try:
847                             item['data'] = json.loads(item['ra
w_json'])
848                         except Exception:
849                             pass
850                         results.append(item)
851         except Exception as ex:
852             logger.error(f'Ошибка извлечения архивов автозапус
ка: {ex}')
853         return results
854
855     def get_latest_startup_archive(self) -> Optional[Dict[str,
Any]]:
856         """Возвращает самый последний сохраненный снимок автоз
апуска."""

```

```

857         audits = self.get_startup_archives(limit=1)
858         return audits[0] if audits else None
859
860     def get_startup_changes(self, limit: int = 100, archive_id
: Optional[str] = None) -> List[Dict[str, Any]]:
861         """Возвращает историю зафиксированных отличий/изменени
й в автозапуске."""
862         results: List[Dict[str, Any]] = []
863         try:
864             with self._cm.lock, self._cm.get_connection() as c
onn:
865                 cursor = conn.cursor()
866                 if archive_id:
867                     cursor.execute(
868                         'SELECT * FROM startup_changes WHERE a
rchive_id = ? ORDER BY created_at DESC, id DESC LIMIT ?',
869                         (archive_id, limit)
870                     )
871                 else:
872                     cursor.execute(
873                         'SELECT * FROM startup_changes ORDER B
Y created_at DESC, id DESC LIMIT ?',
874                         (limit,)
875                     )
876                 for row in cursor.fetchall():
877                     item = dict(row)
878                     if item.get('previous_value'):
879                         try:
880                             item['previous_value'] = json.load
s(item['previous_value'])
881                         except Exception:
882                             pass
883                     if item.get('current_value'):
884                         try:
885                             item['current_value'] = json.loads
(item['current_value'])
886                         except Exception:
887                             pass

```

```

888         results.append(item)
889     except Exception as ex:
890         logger.error(f'Ошибка извлечения изменений автозап
уска: {ex}')
891     return results
892
893     def get_process_provenance_history(
894         self,
895         guid: Optional[str] = None,
896         pid: Optional[int] = None,
897         name: Optional[str] = None,
898         user: Optional[str] = None,
899         event_type: Optional[str] = None,
900         hours: Optional[int] = None,
901         limit: int = 100,
902     ) -> List[Dict[str, Any]]:
903         """Извлечение исторических событий provenance процессо
В."""
904         results: List[Dict[str, Any]] = []
905         query = 'SELECT * FROM process_provenance_events WHERE
1=1'
906         params: List[Any] = []
907
908         if hours is not None:
909             cutoff = time.time() - (hours * 3600)
910             query += ' AND created_at >= ?'
911             params.append(cutoff)
912         if guid:
913             query += ' AND process_guid = ?'
914             params.append(guid)
915         if pid is not None:
916             query += ' AND pid = ?'
917             params.append(pid)
918         if name:
919             query += ' AND name LIKE ?'
920             params.append(f'%{name}%')
921         if user:
922             query += ' AND (user LIKE ? OR sid LIKE ?)'

```

```

923         params.extend([f'#{user}%', f'#{user}%'])
924     if event_type:
925         query += ' AND event_type = ?'
926         params.append(event_type)
927
928     query += ' ORDER BY created_at DESC LIMIT ?'
929     params.append(limit)
930
931     try:
932         with self._cm.lock, self._cm.get_connection() as c
933         onn:
934             cursor = conn.cursor()
935             cursor.execute(query, tuple(params))
936             for row in cursor.fetchall():
937                 item = dict(row)
938                 if item.get('details_json'):
939                     try:
940                         item['details'] = json.loads(item[
941                         'details_json'])
942                     except Exception:
943                         item['details'] = item['details_js
944                         on']
945                 results.append(item)
946     except Exception as ex:
947         logger.error(f'Ошибка извлечения provenance процес
948         сов: {ex}')
949     return results
950
951     def get_process_lineage(self, guid_or_pid: Union[str, int]
952     ) -> List[Dict[str, Any]]:
953         """Построение полной восходящей цепочки предков (linea
954         ge) процесса."""
955         str_val = str(guid_or_pid)
956         chain: List[Dict[str, Any]] = []
957         visited_guids: set = set()
958         visited_pids: set = set()
959
960         history = self.get_process_provenance_history(guid=str

```

```

    _val, limit=1)
955         if not history and str_val.isdigit():
956             history = self.get_process_provenance_history(pid=
int(str_val), limit=1)
957
958         current_node: Optional[Dict[str, Any]] = history[0] if
history else None
959
960         while current_node:
961             node_guid = current_node.get('process_guid')
962             node_pid = current_node.get('pid')
963
964             if (node_guid and node_guid in visited_guids) or (
node_pid and node_pid in visited_pids):
965                 break
966
967             if node_guid:
968                 visited_guids.add(node_guid)
969             if node_pid:
970                 visited_pids.add(node_pid)
971
972             chain.append(current_node)
973             parent_guid = current_node.get('parent_guid')
974             parent_pid = current_node.get('ppid') or current_n
ode.get('parent_pid')
975
976             parent_node = None
977             if parent_guid:
978                 parent_hist = self.get_process_provenance_hist
ory(guid=parent_guid, limit=1)
979                 if parent_hist:
980                     parent_node = parent_hist[0]
981
982             if not parent_node and parent_pid:
983                 parent_hist = self.get_process_provenance_hist
ory(pid=int(parent_pid), limit=1)
984                 if parent_hist:
985                     parent_node = parent_hist[0]

```

```

986 |
987 |         current_node = parent_node
988 |
989 |         chain.reverse()
990 |         return chain
991 |
992 |     def get_telemetry_rollups(
993 |         self,
994 |         level: str = 'hourly',
995 |         sensor_id: Optional[str] = None,
996 |         start_epoch: Optional[float] = None,
997 |         end_epoch: Optional[float] = None,
998 |         limit: int = 100,
999 |     ) -> List[Dict[str, Any]]:
1000 |         """Извлекает агрегированные показатели сенсоров для за
данного уровня (hourly, daily, weekly, monthly, yearly).
1001 |
1002 |         Args:
1003 |             level: Уровень иерархии ('hourly', 'daily', 'weekl
y', 'monthly', 'yearly').
1004 |             sensor_id: Фильтр по идентификатору сенсора (опцио
нально).
1005 |             start_epoch: Нижняя граница времени бакета bucket_
start (опционально).
1006 |             end_epoch: Верхняя граница времени бакета bucket_s
tart (опционально).
1007 |             limit: Максимальное число возвращаемых записей.
1008 |
1009 |         Returns:
1010 |             List[Dict[str, Any]]: Список агрегированных бакето
в со статистикой.
1011 |         """
1012 |         lvl_clean = str(level).strip().lower()
1013 |         table_name = f'telemetry_{lvl_clean}'
1014 |         valid_tables = {'telemetry_hourly', 'telemetry_daily',
'telemetry_weekly', 'telemetry_monthly', 'telemetry_yearly'}
1015 |         if table_name not in valid_tables:
1016 |             table_name = 'telemetry_hourly'

```

```

1017
1018     conditions = []
1019     params: List[Any] = []
1020
1021     if sensor_id is not None:
1022         conditions.append('sensor_id = ?')
1023         params.append(sensor_id)
1024     if start_epoch is not None:
1025         conditions.append('bucket_start >= ?')
1026         params.append(float(start_epoch))
1027     if end_epoch is not None:
1028         conditions.append('bucket_start < ?')
1029         params.append(float(end_epoch))
1030
1031     where_clause = f"WHERE {' AND ' .join(conditions)}" if
conditions else ""
1032     query = f"SELECT * FROM {table_name} {where_clause} OR
DER BY bucket_start DESC LIMIT ?"
1033     params.append(int(limit))
1034
1035     with self._cm.lock, self._cm.get_connection() as conn:
1036         cursor = conn.cursor()
1037         cursor.execute(query, params)
1038         return [dict(row) for row in cursor.fetchall()]
1039
1040     def get_telemetry_spikes(
1041         self,
1042         sensor_id: Optional[str] = None,
1043         start_epoch: Optional[float] = None,
1044         limit: int = 100,
1045     ) -> List[Dict[str, Any]]:
1046         """Извлекает зафиксированные статистические всплески и
аномалии сенсоров.
1047
1048         Args:
1049             sensor_id: Фильтр по идентификатору сенсора (опцио
нально).
1050             start_epoch: Нижняя граница времени регистрации (о

```


пционально).

```
1051         limit: Максимальное число возвращаемых записей.
1052
1053     Returns:
1054         List[Dict[str, Any]]: Список аномальных записей.
1055     """
1056     conditions = []
1057     params: List[Any] = []
1058
1059     if sensor_id is not None:
1060         conditions.append('sensor_id = ?')
1061         params.append(sensor_id)
1062     if start_epoch is not None:
1063         conditions.append('created_at >= ?')
1064         params.append(float(start_epoch))
1065
1066     where_clause = f"WHERE {' AND '.join(conditions)}" if
conditions else ""
1067     query = f"SELECT * FROM telemetry_spikes {where_clause}
ORDER BY created_at DESC LIMIT ?"
1068     params.append(int(limit))
1069
1070     with self._cm.lock, self._cm.get_connection() as conn:
1071         cursor = conn.cursor()
1072         cursor.execute(query, params)
1073         return [dict(row) for row in cursor.fetchall()]
1074
1075     def get_disk_inventory(self, disk_id: Optional[int] = None
) -> List[Dict[str, Any]]:
1076         """Извлекает паспортные данные физических дисков из та
блицы disk_inventory."""
1077         with self._cm.lock, self._cm.get_connection() as conn:
1078             cursor = conn.cursor()
1079             if disk_id is not None:
1080                 cursor.execute('SELECT * FROM disk_inventory W
HERE disk_id = ? ORDER BY disk_id ASC', (disk_id,))
1081             else:
1082                 cursor.execute('SELECT * FROM disk_inventory 0
```

```

1082 |         ORDER BY disk_id ASC')
1083 |         return [dict(row) for row in cursor.fetchall()]
1084 |
1085 |     def get_volume_inventory(self) -> List[Dict[str, Any]]:
1086 |         """Извлекает паспортные данные логических томов из таб
лицы volume_inventory."""
1087 |         with self._cm.lock, self._cm.get_connection() as conn:
1088 |             cursor = conn.cursor()
1089 |             cursor.execute('SELECT * FROM volume_inventory ORD
ER BY drive_letter ASC')
1090 |             return [dict(row) for row in cursor.fetchall()]
1091 |
1092 |     def get_disk_health_history(
1093 |         self,
1094 |         disk_id: Optional[int] = None,
1095 |         serial_number: Optional[str] = None,
1096 |         limit: int = 100,
1097 |         since_epoch: Optional[float] = None,
1098 |     ) -> List[Dict[str, Any]]:
1099 |         """Извлекает историю показателей здоровья и износа нак
опителей."""
1100 |         conditions = []
1101 |         params: List[Any] = []
1102 |
1103 |         if disk_id is not None:
1104 |             conditions.append('disk_id = ?')
1105 |             params.append(disk_id)
1106 |         if serial_number is not None:
1107 |             conditions.append('serial_number = ?')
1108 |             params.append(serial_number)
1109 |         if since_epoch is not None:
1110 |             conditions.append('created_at >= ?')
1111 |             params.append(float(since_epoch))
1112 |
1113 |         where_clause = f"WHERE {' AND '.join(conditions)}" if
conditions else ""
1114 |         query = f"SELECT * FROM disk_health_snapshots {where_c
lause} ORDER BY created_at DESC LIMIT ?"

```

```

1115         params.append(int(limit))
1116
1117         with self._cm.lock, self._cm.get_connection() as conn:
1118             cursor = conn.cursor()
1119             cursor.execute(query, params)
1120             return [dict(row) for row in cursor.fetchall()]
1121
1122     def get_disk_performance_samples(
1123         self,
1124         disk_name: Optional[str] = None,
1125         limit: int = 100,
1126         since_epoch: Optional[float] = None,
1127     ) -> List[Dict[str, Any]]:
1128         """Извлекает историю показателей производительности ди
1129 сков (IOPS, MB/s)."""
1130         conditions = []
1131         params: List[Any] = []
1132
1133         if disk_name is not None:
1134             conditions.append('disk_name = ?')
1135             params.append(disk_name)
1136         if since_epoch is not None:
1137             conditions.append('created_at >= ?')
1138             params.append(float(since_epoch))
1139
1140         where_clause = f"WHERE {' AND '}.join(conditions))" if
1141 conditions else ""
1142         query = f"SELECT * FROM disk_performance_samples {wher
1143 e_clause} ORDER BY created_at DESC LIMIT ?"
1144         params.append(int(limit))
1145
1146         with self._cm.lock, self._cm.get_connection() as conn:
1147             cursor = conn.cursor()
1148             cursor.execute(query, params)
1149             return [dict(row) for row in cursor.fetchall()]
1150
1151     def get_disk_io_events(
1152         self,

```

```

1150         pid: Optional[int] = None,
1151         disk_id: Optional[int] = None,
1152         limit: int = 100,
1153         since_epoch: Optional[float] = None,
1154     ) -> List[Dict[str, Any]]:
1155         """Извлекает события прямого ввода-вывода процессов."""
1156
1157         conditions = []
1158         params: List[Any] = []
1159
1160         if pid is not None:
1161             conditions.append('pid = ?')
1162             params.append(pid)
1163         if disk_id is not None:
1164             conditions.append('disk_id = ?')
1165             params.append(disk_id)
1166         if since_epoch is not None:
1167             conditions.append('created_at >= ?')
1168             params.append(float(since_epoch))
1169
1170         where_clause = f"WHERE {' AND ' .join(conditions)}" if
conditions else ""
1171         query = f"SELECT * FROM disk_io_events {where_clause}
ORDER BY created_at DESC LIMIT ?"
1172         params.append(int(limit))
1173
1174         with self._cm.lock, self._cm.get_connection() as conn:
1175             cursor = conn.cursor()
1176             cursor.execute(query, params)
1177             return [dict(row) for row in cursor.fetchall()]
1178
1179     def get_cpu_inventory(self, processor_id: Optional[int] =
None) -> List[Dict[str, Any]]:
1180         """Извлекает паспортные данные процессоров из таблицы
cpu_inventory."""
1181
1182         with self._cm.lock, self._cm.get_connection() as conn:
1183             cursor = conn.cursor()
1184             if processor_id is not None:

```

```

1183         cursor.execute('SELECT * FROM cpu_inventory WH
ERE processor_id = ? ORDER BY processor_id ASC', (processor_id
,))
1184     else:
1185         cursor.execute('SELECT * FROM cpu_inventory OR
DER BY processor_id ASC')
1186         return [dict(row) for row in cursor.fetchall()]
1187
1188     def get_ram_module_inventory(self) -> List[Dict[str, Any]]
:
1189         """Извлекает паспортные данные физических планок памят
и из таблицы ram_module_inventory."""
1190         with self._cm.lock, self._cm.get_connection() as conn:
1191             cursor = conn.cursor()
1192             cursor.execute('SELECT * FROM ram_module_inventory
ORDER BY slot_id ASC')
1193             return [dict(row) for row in cursor.fetchall()]
1194
1195     def get_gpu_inventory(self, gpu_id: Optional[int] = None)
-> List[Dict[str, Any]]:
1196         """Извлекает паспортные данные графических ускорителей
из таблицы gpu_inventory."""
1197         with self._cm.lock, self._cm.get_connection() as conn:
1198             cursor = conn.cursor()
1199             if gpu_id is not None:
1200                 cursor.execute('SELECT * FROM gpu_inventory WH
ERE gpu_id = ? ORDER BY gpu_id ASC', (gpu_id,))
1201             else:
1202                 cursor.execute('SELECT * FROM gpu_inventory OR
DER BY gpu_id ASC')
1203             return [dict(row) for row in cursor.fetchall()]
1204
1205     def get_network_adapter_inventory(self) -> List[Dict[str,
Any]]:
1206         """Извлекает паспортные данные сетевых интерфейсов из
таблицы network_adapter_inventory."""
1207         with self._cm.lock, self._cm.get_connection() as conn:
1208             cursor = conn.cursor()

```

```

1209         cursor.execute('SELECT * FROM network_adapter_inve
ntory ORDER BY adapter_id ASC')
1210         return [dict(row) for row in cursor.fetchall()]
1211
1212     def get_cpu_telemetry_samples(
1213         self,
1214         processor_id: Optional[int] = None,
1215         limit: int = 100,
1216         since_epoch: Optional[float] = None,
1217     ) -> List[Dict[str, Any]]:
1218         """Извлекает историю показателей нагрузки и температур
процессоров."""
1219         conditions = []
1220         params: List[Any] = []
1221
1222         if processor_id is not None:
1223             conditions.append('processor_id = ?')
1224             params.append(processor_id)
1225         if since_epoch is not None:
1226             conditions.append('created_at >= ?')
1227             params.append(float(since_epoch))
1228
1229         where_clause = f"WHERE {' AND '.join(conditions)}" if
conditions else ""
1230         query = f"SELECT * FROM cpu_telemetry_samples {where_c
lause} ORDER BY created_at DESC LIMIT ?"
1231         params.append(int(limit))
1232
1233         with self._cm.lock, self._cm.get_connection() as conn:
1234             cursor = conn.cursor()
1235             cursor.execute(query, params)
1236             return [dict(row) for row in cursor.fetchall()]
1237
1238     def get_ram_telemetry_samples(
1239         self,
1240         limit: int = 100,
1241         since_epoch: Optional[float] = None,
1242     ) -> List[Dict[str, Any]]:

```

```

1243 |         """Извлекает историю показателей использования оперативной
1244 |         памяти."""
1245 |         conditions = []
1246 |         params: List[Any] = []
1247 |
1248 |         if since_epoch is not None:
1249 |             conditions.append('created_at >= ?')
1250 |             params.append(float(since_epoch))
1251 |
1252 |         where_clause = f"WHERE {' AND '.join(conditions)}" if conditions else ""
1253 |         query = f"SELECT * FROM ram_telemetry_samples {where_clause} ORDER BY created_at DESC LIMIT ?"
1254 |         params.append(int(limit))
1255 |
1256 |         with self._cm.lock, self._cm.get_connection() as conn:
1257 |             cursor = conn.cursor()
1258 |             cursor.execute(query, params)
1259 |             return [dict(row) for row in cursor.fetchall()]
1260 |
1261 |     def get_gpu_telemetry_samples(
1262 |         self,
1263 |         gpu_id: Optional[int] = None,
1264 |         name: Optional[str] = None,
1265 |         limit: int = 100,
1266 |         since_epoch: Optional[float] = None,
1267 |     ) -> List[Dict[str, Any]]:
1268 |         """Извлекает историю показателей нагрузки, памяти и температуры GPU."""
1269 |         conditions = []
1270 |         params: List[Any] = []
1271 |
1272 |         if gpu_id is not None:
1273 |             conditions.append('gpu_id = ?')
1274 |             params.append(gpu_id)
1275 |
1276 |         if name is not None:
1277 |             conditions.append('name = ?')
1278 |             params.append(name)

```

```

1277         if since_epoch is not None:
1278             conditions.append('created_at >= ?')
1279             params.append(float(since_epoch))
1280
1281         where_clause = f"WHERE {' AND '.join(conditions)}" if
conditions else ""
1282         query = f"SELECT * FROM gpu_telemetry_samples {where_c
lause} ORDER BY created_at DESC LIMIT ?"
1283         params.append(int(limit))
1284
1285         with self._cm.lock, self._cm.get_connection() as conn:
1286             cursor = conn.cursor()
1287             cursor.execute(query, params)
1288             return [dict(row) for row in cursor.fetchall()]
1289
1290     def get_network_adapter_samples(
1291         self,
1292         adapter_name: Optional[str] = None,
1293         limit: int = 100,
1294         since_epoch: Optional[float] = None,
1295     ) -> List[Dict[str, Any]]:
1296         """Извлекает историю показателей сетевой активности ад
аптеров."""
1297         conditions = []
1298         params: List[Any] = []
1299
1300         if adapter_name is not None:
1301             conditions.append('adapter_name = ?')
1302             params.append(adapter_name)
1303         if since_epoch is not None:
1304             conditions.append('created_at >= ?')
1305             params.append(float(since_epoch))
1306
1307         where_clause = f"WHERE {' AND '.join(conditions)}" if
conditions else ""
1308         query = f"SELECT * FROM network_adapter_samples {where
_clause} ORDER BY created_at DESC LIMIT ?"
1309         params.append(int(limit))

```



```

1310
1311         with self._cm.lock, self._cm.get_connection() as conn:
1312             cursor = conn.cursor()
1313             cursor.execute(query, params)
1314             return [dict(row) for row in cursor.fetchall()]
1315
1316         # =====
1317         # Подсистемы рефакторинга: Data-First SQLite Readers
1318         # =====
1319
1320         # 1. Windows Event Logs
1321         def get_latest_event_log_channels(self) -> List[Dict[str,
1322             Any]]:
1323             """Извлечение последнего снимка каналов журналов событ
1324             ий."""
1325             with self._cm.lock, self._cm.get_connection() as conn:
1326                 cursor = conn.cursor()
1327                 cursor.execute('''
1328                     SELECT snapshot_id FROM event_log_channel_snap
1329                     shots
1330                     ORDER BY id DESC LIMIT 1
1331                     ''')
1332                 row = cursor.fetchone()
1333                 if not row:
1334                     return []
1335                 snap_id = row['snapshot_id'] if isinstance(row, sq
1336                 lite3.Row) else row[0]
1337                 cursor.execute('''
1338                     SELECT channel_name AS name, is_enabled AS ena
1339                     bled, record_count,
1340                     size_bytes, channel_type, display_name,
1341                     description
1342                     FROM event_log_channel_snapshots
1343                     WHERE snapshot_id = ?
1344                     ORDER BY record_count DESC
1345                     ''', (snap_id,))

```

```

1340         return [dict(r) for r in cursor.fetchall()]
1341
1342     def get_event_log_entries(
1343         self,
1344         channel: str = 'System',
1345         level: str = '',
1346         limit: int = 50,
1347         hours: int = 24,
1348     ) -> List[Dict[str, Any]]:
1349         """Выборка записей журнала событий из кеша по каналу и
1350         уровню."""
1351         conditions = ['channel = ?']
1352         params: List[Any] = [channel]
1353
1354         if level:
1355             conditions.append('LOWER(level) = LOWER(?)')
1356             params.append(level)
1357
1358         if hours > 0:
1359             since_time = time.time() - (hours * 3600)
1360             conditions.append('created_at >= ?')
1361             params.append(since_time)
1362
1363         where_clause = f"WHERE {' AND '.join(conditions)}"
1364         query = f'''
1365             SELECT channel, event_id, level, provider_name, ti
1366             me_created, message
1367             FROM event_log_entries_cache
1368             {where_clause}
1369             ORDER BY id DESC
1370             LIMIT ?
1371         '''
1372         params.append(int(limit))
1373
1374         with self._cm.lock, self._cm.get_connection() as conn:
1375             cursor = conn.cursor()
1376             cursor.execute(query, params)
1377             rows = [dict(r) for r in cursor.fetchall()]

```

```

1376         if not rows and hours > 0:
1377             # Fallback без фильтра по времени, если база б
        ыла заполнена раньше
1378             params[-2] = 0.0
1379             cursor.execute(query, params)
1380             rows = [dict(r) for r in cursor.fetchall()]
1381         return rows
1382
1383     def get_latest_event_log_intelligence_profile(self, channel: str = 'System') -> Optional[Dict[str, Any]]:
1384         """Извлечение последнего сохраненного профиля Log Intelligence."""
1385         with self._cm.lock, self._cm.get_connection() as conn:
1386             cursor = conn.cursor()
1387             cursor.execute('''
1388                 SELECT * FROM event_log_intelligence_profiles
1389                 WHERE channel = ?
1390                 ORDER BY id DESC LIMIT 1
1391             ''', (channel,))
1392             row = cursor.fetchone()
1393             if not row:
1394                 return None
1395             d = dict(row)
1396             d['critical_incidents'] = json.loads(d['critical_incidents_json']) if d.get('critical_incidents_json') else []
1397             d['top_clusters'] = json.loads(d['top_clusters_json']) if d.get('top_clusters_json') else []
1398             d['bursts'] = json.loads(d['bursts_json']) if d.get('bursts_json') else []
1399             decision_gate = json.loads(d['decision_gate_json']) if d.get('decision_gate_json') else {}
1400             profile_data = {
1401                 'total_events': d.get('total_analyzed', 0),
1402                 'unique_templates_count': d.get('unique_patterns_count', 0),
1403                 'critical_count': len(d['critical_incidents'])
1404             },
            'health_score': 100 - min(100, len(d['critical

```

```

        _incidents']) * 10),
1405         'top_patterns': d['top_clusters'],
1406         'bursts': d['bursts'],
1407     }
1408     d['profile'] = profile_data
1409     d['decision'] = decision_gate
1410     return d
1411
1412     def get_latest_event_log_report(self) -> Optional[Dict[str
, Any]]:
1413         """Формирование сводного отчета о журналах и ошибках и
з базы данных."""
1414         channels = self.get_latest_event_log_channels()
1415         if not channels:
1416             return None
1417         with self._cm.lock, self._cm.get_connection() as conn:
1418             cursor = conn.cursor()
1419             cursor.execute('''
1420                 SELECT channel, event_id, level, provider_name
, time_created, message
1421                 FROM event_log_entries_cache
1422                 WHERE LOWER(level) IN ('error', 'critical', 'w
arning')
1423                 ORDER BY id DESC LIMIT 20
1424             ''')
1425             errors = [dict(r) for r in cursor.fetchall()]
1426
1427             crit_count = sum(1 for e in errors if str(e.get('l
evel', ''')).lower() == 'critical')
1428             err_count = sum(1 for e in errors if str(e.get('le
vel', ''')).lower() == 'error')
1429             warn_count = sum(1 for e in errors if str(e.get('l
evel', ''')).lower() == 'warning')
1430
1431             return {
1432                 'total_channels': len(channels),
1433                 'critical_events_24h': crit_count,
1434                 'error_events_24h': err_count,

```

```

1435         'warning_events_24h': warn_count,
1436         'channels': channels,
1437         'recent_errors': errors,
1438         'timestamp': datetime.now(timezone.utc).isofor
mat(),
1439     }
1440
1441     # 2. Windows Firewall Manager
1442     def get_latest_firewall_profiles(self) -> Optional[Dict[str, Any]]:
1443         """Извлечение состояния профилей брандмауэра Windows."""
1444         with self._cm.lock, self._cm.get_connection() as conn:
1445             cursor = conn.cursor()
1446             cursor.execute('''
1447                 SELECT domain_enabled, private_enabled, public
_enabled,
1448                        domain_default_inbound, private_default
_inbound, public_default_inbound,
1449                        stealth_mode_enabled, timestamp
1450                 FROM firewall_profile_snapshots
1451                 ORDER BY id DESC LIMIT 1
1452             ''')
1453             row = cursor.fetchone()
1454             return dict(row) if row else None
1455
1456     def get_latest_firewall_rules(self, direction: Optional[str] = None, limit: int = 200) -> List[Dict[str, Any]]:
1457         """Выборка правил фильтрации брандмауэра Windows из по
следнего снимка."""
1458         with self._cm.lock, self._cm.get_connection() as conn:
1459             cursor = conn.cursor()
1460             cursor.execute('SELECT snapshot_id FROM firewall_r
ule_snapshots ORDER BY id DESC LIMIT 1')
1461             row = cursor.fetchone()
1462             if not row:
1463                 return []
1464             snap_id = row['snapshot_id'] if isinstance(row, sq

```

```

lite3.Row) else row[0]
1465
1466         conditions = ['snapshot_id = ?']
1467         params: List[Any] = [snap_id]
1468         if direction:
1469             conditions.append('LOWER(direction) = LOWER(?)
1470         ')
1471             params.append(direction)
1472
1473         where_clause = f"WHERE {' AND '}.join(conditions)}"
1474         query = f'''
1475             SELECT rule_name AS name, display_name, direct
1476             ion, action, enabled,
1477             protocol, local_port, remote_port, prog
1478             ram_path, profile_mask
1479             FROM firewall_rule_snapshots
1480             {where_clause}
1481             ORDER BY id ASC
1482             LIMIT ?
1483             '''
1484         params.append(int(limit))
1485         cursor.execute(query, params)
1486         return [dict(r) for r in cursor.fetchall()]
1487
1488     # 3. Windows Services Manager
1489     def get_latest_services_list(self, status: Optional[str] =
1490     None, limit: int = 500) -> List[Dict[str, Any]]:
1491         """Извлечение списка служб Windows из последнего снимк
1492         а."""
1493         with self._cm.lock, self._cm.get_connection() as conn:
1494             cursor = conn.cursor()
1495             cursor.execute('SELECT snapshot_id FROM services_s
1496             napshots ORDER BY id DESC LIMIT 1')
1497             row = cursor.fetchone()
1498             if not row:
1499                 return []
1500             snap_id = row['snapshot_id'] if isinstance(row, sq
1501             lite3.Row) else row[0]

```

```

1495
1496         conditions = ['snapshot_id = ?']
1497         params: List[Any] = [snap_id]
1498         if status:
1499             conditions.append('LOWER(state) = LOWER(?)')
1500             params.append(status)
1501
1502         where_clause = f"WHERE {' AND '}.join(conditions)}"
1503         query = f'''
1504             SELECT service_name AS name, display_name, sta
1505 te AS status,
1506             start_type AS startup_type, pid, binary
1507 _path AS binpath,
1508             account AS user_account, is_orphaned
1509 FROM services_snapshots
1510 {where_clause}
1511 ORDER BY service_name ASC
1512 LIMIT ?
1513 '''
1514         params.append(int(limit))
1515         cursor.execute(query, params)
1516         return [dict(r) for r in cursor.fetchall()]
1517
1518     def get_latest_services_report(self) -> Optional[Dict[str,
1519 Any]]:
1520         """Формирование сводного отчета о службах Windows."""
1521         services = self.get_latest_services_list(limit=1000)
1522         if not services:
1523             return None
1524         total = len(services)
1525         running = sum(1 for s in services if str(s.get('status
1526 ', '')).upper() in ('RUNNING', '4', 'SERVICE_RUNNING'))
1527         stopped = sum(1 for s in services if str(s.get('status
1528 ', '')).upper() in ('STOPPED', '1', 'SERVICE_STOPPED'))
1529         orphaned = sum(1 for s in services if s.get('is_orphan
1530 ed'))
1531
1532         return {

```

```

1527         'total_services': total,
1528         'running_services': running,
1529         'stopped_services': stopped,
1530         'orphaned_services': orphaned,
1531         'services': services,
1532         'timestamp': datetime.now(timezone.utc).isoformat(
1533     ),
1534     }
1535
1536     def get_service_change_events(self, limit: int = 100) -> List[Dict[str, Any]]:
1537         """Извлечение истории изменений служб."""
1538         with self._cm.lock, self._cm.get_connection() as conn:
1539             cursor = conn.cursor()
1540             cursor.execute(''
1541                 SELECT timestamp, service_name, display_name,
1542                 action,
1543                 old_state, new_state, performed_by, details_json
1544                 FROM service_change_events
1545                 ORDER BY id DESC LIMIT ?
1546             ''', (int(limit),))
1547             rows = []
1548             for r in cursor.fetchall():
1549                 d = dict(r)
1550                 if d.get('details_json'):
1551                     d['details'] = json.loads(d['details_json'])
1552             rows.append(d)
1553             return rows
1554
1555     # 4. CPU Throttling & Power Limits
1556     def get_latest_throttling_snapshot(self) -> Optional[Dict[str, Any]]:
1557         """Извлечение последнего снимка троттлинга процессора
и термических зон."""
1558         with self._cm.lock, self._cm.get_connection() as conn:
1559             cursor = conn.cursor()

```



```

1558         cursor.execute('''
1559             SELECT snapshot_id, timestamp, prochot_active,
1560             pl1_limit_watts,
1561             pl2_limit_watts, current_power_watts, m
1562             ax_core_temp_c,
1563             package_temp_c, dpc_latency_us, isr_lat
1564             ency_us,
1565             throttling_reasons_json
1566             FROM cpu_throttling_snapshots
1567             ORDER BY id DESC LIMIT 1
1568         ''')
1569         row = cursor.fetchone()
1570         if not row:
1571             return None
1572         d = dict(row)
1573         d['throttling_reasons'] = json.loads(d['throttling
1574         _reasons_json']) if d.get('throttling_reasons_json') else []
1575
1576         snap_id = d['snapshot_id']
1577         cursor.execute('''
1578             SELECT zone_name, temperature_c, critical_limi
1579             t_c, throttling_limit_c, sensor_provider
1580             FROM thermal_zone_snapshots
1581             WHERE snapshot_id = ?
1582             ''', (snap_id,))
1583         d['thermal_zones'] = [dict(z) for z in cursor.fetc
1584         hall()]
1585
1586         return d
1587
1588     # 5. Behavioral Forensics
1589     def get_latest_forensics_snapshot(self) -> Optional[Dict[s
1590     tr, Any]]:
1591         """Извлечение последнего снимка поведенческой форензик
1592         и."""
1593
1594         with self._cm.lock, self._cm.get_connection() as conn:
1595             cursor = conn.cursor()
1596             cursor.execute('''
1597                 SELECT foreground_window_title, foreground_pro

```

```

cess_name,
1588         foreground_pid, user_idle_seconds, came
ra_active_apps_json,
1589         microphone_active_apps_json, userassist
_top_apps_json, timestamp
1590         FROM forensics_snapshots
1591         ORDER BY id DESC LIMIT 1
1592         '', )
1593         row = cursor.fetchone()
1594         if not row:
1595             return None
1596         d = dict(row)
1597         d['camera_active_apps'] = json.loads(d['camera_act
ive_apps_json']) if d.get('camera_active_apps_json') else []
1598         d['microphone_active_apps'] = json.loads(d['microp
hone_active_apps_json']) if d.get('microphone_active_apps_json
') else []
1599         d['userassist_top_apps'] = json.loads(d['userassis
t_top_apps_json']) if d.get('userassist_top_apps_json') else [
]
1600         return d
1601
1602     # 6. Process Leaks & Resource Starvation
1603     def get_latest_process_leak_report(self, limit: int = 20)
-> Optional[Dict[str, Any]]:
1604         """Извлечение последнего снимка утечек ресурсов процес
сов."""
1605         with self._cm.lock, self._cm.get_connection() as conn:
1606             cursor = conn.cursor()
1607             cursor.execute('''
1608                 SELECT snapshot_id, timestamp, total_processes
, suspicious_count
1609                 FROM process_leak_snapshots
1610                 ORDER BY id DESC LIMIT 1
1611             ''')
1612             row = cursor.fetchone()
1613             if not row:
1614                 return None

```

```

1615         snap_id = row['snapshot_id'] if isinstance(row, sq
|lite3.Row) else row[0]
1616         report = dict(row)
1617
1618         cursor.execute('''
1619             SELECT pid, name, handles_count, gdi_objects,
|user_objects,
1620             page_faults, working_set_mb, leak_risk_
|score, leak_risk_reasons_json
1621             FROM process_leak_items
1622             WHERE snapshot_id = ?
1623             ORDER BY handles_count DESC
1624             LIMIT ?
1625             ''', (snap_id, int(limit)))
1626         items = []
1627         for r in cursor.fetchall():
1628             d = dict(r)
1629             d['leak_risk_reasons'] = json.loads(d['leak_r
|isk_reasons_json']) if d.get('leak_risk_reasons_json') else []
1630             items.append(d)
1631             report['all_processes'] = items
1632             report['top_handle_hogs'] = sorted(items, key=lamb
|da x: x.get('handles_count', 0), reverse=True)[:10]
1633             report['top_gdi_hogs'] = sorted(items, key=lambda
|x: x.get('gdi_objects', 0), reverse=True)[:10]
1634             report['top_page_fault_hogs'] = sorted(items, key=
|lambda x: x.get('page_faults', 0), reverse=True)[:10]
1635             report['suspicious_processes'] = [x for x in items
|if x.get('leak_risk_score') != 'normal']
1636             return report
1637
1638     # 7. Defender Security
1639     def get_latest_defender_status(self) -> Optional[Dict[str,
|Any]]:
1640         """Извлечение статуса Защитника Windows."""
1641         with self._cm.lock, self._cm.get_connection() as conn:
1642             cursor = conn.cursor()
1643             cursor.execute('''

```

```

1644         SELECT * FROM defender_snapshots
1645         ORDER BY id DESC LIMIT 1
1646     '')
1647     row = cursor.fetchone()
1648     if not row:
1649         return None
1650     d = dict(row)
1651     if d.get('raw_status_json'):
1652         try:
1653             d.update(json.loads(d['raw_status_json']))
1654         except Exception:
1655             pass
1656     return d
1657
1658     def get_latest_defender_exclusions(self) -> List[Dict[str,
Any]]:
1659         """Извлечение исключений антивируса Defender."""
1660         with self._cm.lock, self._cm.get_connection() as conn:
1661             cursor = conn.cursor()
1662             cursor.execute('SELECT snapshot_id FROM defender_e
xclusions ORDER BY id DESC LIMIT 1')
1663             row = cursor.fetchone()
1664             if not row:
1665                 return []
1666             snap_id = row['snapshot_id'] if isinstance(row, sq
lite3.Row) else row[0]
1667             cursor.execute('''
1668                 SELECT exclusion_type, exclusion_value, risk_l
evel
1669                 FROM defender_exclusions
1670                 WHERE snapshot_id = ?
1671             ''', (snap_id,))
1672             return [dict(r) for r in cursor.fetchall()]
1673
1674     def get_latest_defender_asr_rules(self) -> List[Dict[str,
Any]]:
1675         """Извлечение правил ASR Защитника Windows."""
1676         with self._cm.lock, self._cm.get_connection() as conn:

```

```

1677         cursor = conn.cursor()
1678         cursor.execute('SELECT snapshot_id FROM defender_a
| sr_rules ORDER BY id DESC LIMIT 1')
1679         row = cursor.fetchone()
1680         if not row:
1681             return []
1682         snap_id = row['snapshot_id'] if isinstance(row, sq
| lite3.Row) else row[0]
1683         cursor.execute('''
1684             SELECT rule_guid, rule_name, rule_action
1685             FROM defender_asr_rules
1686             WHERE snapshot_id = ?
1687             ''', (snap_id,))
1688         return [dict(r) for r in cursor.fetchall()]
1689
1690     def get_latest_defender_threats(self) -> List[Dict[str, An
| y]]:
1691         """Извлечение обнаруженных угроз Defender."""
1692         with self._cm.lock, self._cm.get_connection() as conn:
1693             cursor = conn.cursor()
1694             cursor.execute('''
1695                 SELECT threat_id, threat_name, severity, categ
| ory, resources_json, detection_time
1696                 FROM defender_threats
1697                 ORDER BY id DESC LIMIT 50
1698             ''')
1699             rows = []
1700             for r in cursor.fetchall():
1701                 d = dict(r)
1702                 d['resources'] = json.loads(d['resources_json'
| ]) if d.get('resources_json') else []
1703                 rows.append(d)
1704             return rows
1705
1706     def get_latest_defender_diagnostics(self) -> Dict[str, Any
| ]:
1707         """Сводная диагностика безопасности Defender."""
1708         status = self.get_latest_defender_status() or {}

```

```

1709     exclusions = self.get_latest_defender_exclusions()
1710     asr_rules = self.get_latest_defender_asr_rules()
1711     threats = self.get_latest_defender_threats()
1712     return {
1713         'status': status,
1714         'exclusions': exclusions,
1715         'asr_rules': asr_rules,
1716         'threats': threats,
1717         'total_exclusions': len(exclusions),
1718         'total_asr_rules': len(asr_rules),
1719         'active_threats_count': len(threats),
1720     }
1721
1722     # 8. Process Network & Process Manager
1723     def get_latest_process_network_activity(
1724         self,
1725         limit: int = 100,
1726         only_internet: bool = True,
1727     ) -> List[Dict[str, Any]]:
1728         """Извлечение сетевой активности процессов из process_
1729         network_snapshots."""
1730         with self._cm.lock, self._cm.get_connection() as conn:
1731             cursor = conn.cursor()
1732             cursor.execute('SELECT snapshot_id FROM process_ne
1733             twork_snapshots ORDER BY id DESC LIMIT 1')
1734             row = cursor.fetchone()
1735             if not row:
1736                 return []
1737             snap_id = row['snapshot_id'] if isinstance(row, sq
1738             lite3.Row) else row[0]
1739
1740             conditions = ['snapshot_id = ?']
1741             params: List[Any] = [snap_id]
1742             if only_internet:
1743                 conditions.append("LOWER(service_type) = 'inte
1744                 rnet' OR remote_address != ''")
1745
1746             where_clause = f"WHERE {' AND '.join(conditions)}"

```

```

1743         query = f'''
1744             SELECT pid, process_name, local_address, remot
1745             e_address, protocol,
1746             status, service_type, sent_kb, recv_kb,
1747             read_speed_kbs, write_speed_kbs
1748             FROM process_network_snapshots
1749             {where_clause}
1750             ORDER BY sent_kb + recv_kb DESC
1751             LIMIT ?
1752         '''
1753         params.append(int(limit))
1754         cursor.execute(query, params)
1755         return [dict(r) for r in cursor.fetchall()]
1756
1757     # 9. Software Transparency Scanner
1758     def get_software_inventory_from_db(self, limit: int = 200,
1759 offset: int = 0) -> List[Dict[str, Any]]:
1760         """Извлечение инвентаря установленного ПО с агрегацией
1761         данных из SQLite."""
1762         with self._cm.lock, self._cm.get_connection() as conn:
1763             cursor = conn.cursor()
1764             cursor.execute('''
1765                 SELECT s.*,
1766                     (SELECT COUNT(*) FROM software_storage_
1767 locations l WHERE l.app_id = s.app_id) AS storage_locations_co
1768 unt,
1769                     (SELECT COALESCE(SUM(size_bytes), 0) FR
1770 OM software_storage_locations l WHERE l.app_id = s.app_id) AS
1771 total_storage_bytes,
1772                     (SELECT summary FROM software_ai_resear
1773 ch r WHERE r.app_id = s.app_id) AS ai_summary,
1774                     (SELECT confidence_level FROM software_
1775 ai_research r WHERE r.app_id = s.app_id) AS ai_confidence
1776                 FROM software_inventory s
1777                 ORDER BY s.display_name ASC
1778                 LIMIT ? OFFSET ?
1779             ''', (int(limit), int(offset)))
1780         return [dict(r) for r in cursor.fetchall()]

```

```

1771
1772     def get_software_app_details_from_db(self, app_id: str) ->
1773         Optional[Dict[str, Any]]:
1774             """Извлечение полной карточки установленной программы
1775             со всеми связанными данными."""
1776             with self._cm.lock, self._cm.get_connection() as conn:
1777                 cursor = conn.cursor()
1778                 cursor.execute('SELECT * FROM software_inventory W
1779 HERE app_id = ?', (app_id,))
1780                 row = cursor.fetchone()
1781                 if not row:
1782                     return None
1783                 app_data = dict(row)
1784
1785                 cursor.execute('SELECT category, path, size_bytes,
1786 file_count, last_updated FROM software_storage_locations WHER
1787 E app_id = ?', (app_id,))
1788                 app_data['storage_locations'] = [dict(r) for r in
1789 cursor.fetchall()]
1790
1791                 cursor.execute('SELECT file_path, format, size_byt
1792 es, snippet FROM software_config_files WHERE app_id = ?', (app
1793 _id,))
1794                 app_data['config_files'] = [dict(r) for r in curso
1795 r.fetchall()]
1796
1797                 cursor.execute('SELECT * FROM software_ai_research
1798 WHERE app_id = ?', (app_id,))
1799                 ai_row = cursor.fetchone()
1800                 if ai_row:
1801                     ai_dict = dict(ai_row)
1802                     ai_dict['confirmed_facts'] = json.loads(ai_dic
1803 t['confirmed_facts_json']) if ai_dict.get('confirmed_facts_jso
1804 n') else []
1805                     ai_dict['inferred_facts'] = json.loads(ai_dict
1806 ['inferred_facts_json']) if ai_dict.get('inferred_facts_json')
1807 else []
1808                 app_data['ai_research'] = ai_dict

```



```
1795 |         else:
1796 |             app_data['ai_research'] = None
1797 |
1798 |     return app_data
1799 |
1800 |
1801 |
```

schema.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Schema
4 # =====
5 # Description:
6 #   Схема таблиц, индексов и миграций SQLite базы данных систе
  мной телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.schema import init_da
  tabase_schema
11 #
12 #       init_database_schema(connection)
13 #
14 # File: schema.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.sqlite
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 17:30:00
20 # =====
21
22 from __future__ import annotations
23
24 """Схема таблиц, индексов и процедур миграции базы данных теле
  метрии SQLite."""
25
26 import sqlite3
27 from typing import Set
28
29 try:
```

```
30     from logger import logger
31 except ImportError:
32     import logging
33     logger = logging.getLogger(__name__)
34
35
36 def init_database_schema(conn: sqlite3.Connection) -> None:
37     """Инициализирует таблицы, индексы и выполняет миграции дл
    я базы данных телеметрии.
38
39     Args:
40         conn: Активное подключение к базе данных SQLite.
41     """
42     cursor = conn.cursor()
43
44     # 1. Системные снимки (основная таблица метрик)
45     cursor.execute('''
46         CREATE TABLE IF NOT EXISTS system_snapshots (
47             id INTEGER PRIMARY KEY AUTOINCREMENT,
48             timestamp TEXT NOT NULL,
49             created_at REAL NOT NULL,
50             hostname TEXT,
51             username TEXT,
52             uptime_seconds REAL,
53             os_name TEXT,
54             os_build TEXT,
55             os_install_date TEXT,
56             system_language TEXT,
57             os_install_language TEXT,
58             user_locale TEXT,
59             system_locale TEXT,
60             timezone TEXT,
61             codepage TEXT,
62             input_languages_json TEXT,
63             disks_json TEXT,
64             physical_disks_json TEXT,
65             monitors_json TEXT,
66             updates_json TEXT,
```

```

67         office_json TEXT,
68         onedrive_json TEXT,
69         battery_json TEXT,
70         ram_sticks_json TEXT,
71         listening_ports_json TEXT,
72         alerts_json TEXT,
73         cpu_total_percent REAL,
74         cpu_frequency_mhz REAL,
75         memory_total_gb REAL,
76         memory_used_gb REAL,
77         memory_percent REAL,
78         swap_percent REAL,
79         gpu_load_percent REAL,
80         gpu_temp_c REAL,
81         disk_read_bytes_sec REAL,
82         disk_write_bytes_sec REAL,
83         disk_read_count_sec REAL,
84         disk_write_count_sec REAL,
85         network_sent_bytes_sec REAL,
86         network_recv_bytes_sec REAL,
87         hardware_audit TEXT,
88         raw_json TEXT
89     );
90 '''
91
92 # 2. Сенсоры (аппаратные датчики)
93 cursor.execute('''
94     CREATE TABLE IF NOT EXISTS sensor_polls (
95         id INTEGER PRIMARY KEY AUTOINCREMENT,
96         sensor_id TEXT NOT NULL,
97         timestamp TEXT NOT NULL,
98         created_at REAL NOT NULL,
99         hardware_name TEXT,
100        hardware_type TEXT,
101        sensor_category TEXT,
102        sensor_name TEXT,
103        unit TEXT,
104        value REAL,

```

```

105         provider TEXT,
106         provider_priority INTEGER DEFAULT 30,
107         raw_json TEXT,
108         UNIQUE(sensor_id, timestamp)
109     );
110     ''')
111
112 # 3. Снимки процессов
113 cursor.execute('''
114     CREATE TABLE IF NOT EXISTS process_snapshots (
115         id INTEGER PRIMARY KEY AUTOINCREMENT,
116         snapshot_id INTEGER NOT NULL,
117         timestamp TEXT NOT NULL,
118         pid INTEGER NOT NULL,
119         name TEXT NOT NULL,
120         status TEXT,
121         cpu_percent REAL,
122         memory_mb REAL,
123         memory_percent REAL,
124         num_threads INTEGER,
125         num_handles INTEGER DEFAULT 0,
126         username TEXT,
127         read_bytes_sec REAL,
128         write_bytes_sec REAL,
129         integrity_level TEXT,
130         elevation INTEGER DEFAULT 0,
131         ppid INTEGER,
132         parent_name TEXT,
133         executable TEXT,
134         cmdline TEXT,
135         sid TEXT,
136         session_id INTEGER,
137         creation_time TEXT,
138         process_guid TEXT,
139         ancestor_chain TEXT,
140         launch_reason TEXT,
141         FOREIGN KEY (snapshot_id) REFERENCES system_snapsh
ots(id) ON DELETE CASCADE

```

```

142         );
143     '')
144
145     # 4. Происхождение и жизненный цикл процессов
146     cursor.execute('''
147         CREATE TABLE IF NOT EXISTS process_provenance_events (
148             id INTEGER PRIMARY KEY AUTOINCREMENT,
149             event_id TEXT NOT NULL,
150             timestamp TEXT NOT NULL,
151             created_at REAL NOT NULL,
152             event_type TEXT NOT NULL,
153             pid INTEGER NOT NULL,
154             ppid INTEGER,
155             process_guid TEXT NOT NULL,
156             parent_guid TEXT,
157             name TEXT NOT NULL,
158             executable_path TEXT,
159             command_line TEXT,
160             user TEXT,
161             sid TEXT,
162             session_id INTEGER,
163             integrity_level TEXT,
164             elevation INTEGER,
165             parent_name TEXT,
166             parent_cmdline TEXT,
167             ancestor_chain TEXT,
168             launch_reason TEXT,
169             source TEXT DEFAULT 'system',
170             details_json TEXT,
171             raw_json TEXT
172         );
173     '')
174
175     # 5. Почасовые агрегаты сенсоров
176     cursor.execute('''
177         CREATE TABLE IF NOT EXISTS sensor_aggregates_hourly (
178             id INTEGER PRIMARY KEY AUTOINCREMENT,
179             sensor_id TEXT NOT NULL,

```

```

180         period_start TEXT NOT NULL,
181         period_end TEXT NOT NULL,
182         avg_value REAL,
183         min_value REAL,
184         max_value REAL,
185         count INTEGER,
186         raw_json TEXT
187     );
188 '''
189
190 # 6. Двухминутные агрегаты процессов
191 cursor.execute('''
192     CREATE TABLE IF NOT EXISTS process_rollups_2min (
193         id INTEGER PRIMARY KEY AUTOINCREMENT,
194         period_start TEXT NOT NULL,
195         period_end TEXT NOT NULL,
196         period_start_epoch REAL,
197         period_end_epoch REAL,
198         name TEXT NOT NULL,
199         pid INTEGER,
200         username TEXT,
201         sample_count INTEGER NOT NULL,
202         avg_cpu_percent REAL NOT NULL,
203         max_cpu_percent REAL NOT NULL,
204         min_cpu_percent REAL,
205         avg_memory_mb REAL NOT NULL,
206         max_memory_mb REAL NOT NULL,
207         min_memory_mb REAL,
208         avg_num_threads REAL,
209         outliers_count INTEGER DEFAULT 0
210     );
211 '''
212
213 # 7. Суточные агрегаты процессов
214 cursor.execute('''
215     CREATE TABLE IF NOT EXISTS process_rollups_daily (
216         id INTEGER PRIMARY KEY AUTOINCREMENT,
217         date TEXT NOT NULL,

```

```

218         name TEXT NOT NULL,
219         sample_count INTEGER NOT NULL,
220         avg_cpu_percent REAL NOT NULL,
221         max_cpu_percent REAL NOT NULL,
222         avg_memory_mb REAL NOT NULL,
223         max_memory_mb REAL NOT NULL,
224         outliers_count INTEGER DEFAULT 0,
225         last_seen TEXT
226     );
227 '''
228
229 # 8. Бакеты сжатия временных рядов телеметрии
230 cursor.execute('''
231     CREATE TABLE IF NOT EXISTS telemetry_rollups (
232         id INTEGER PRIMARY KEY AUTOINCREMENT,
233         period_start TEXT NOT NULL,
234         period_end TEXT NOT NULL,
235         period_start_epoch REAL NOT NULL,
236         period_end_epoch REAL NOT NULL,
237         duration_seconds REAL NOT NULL,
238         sample_count INTEGER NOT NULL,
239         tier TEXT NOT NULL,
240         cpu_avg REAL,
241         cpu_min REAL,
242         cpu_max REAL,
243         cpu_p95 REAL,
244         ram_avg_gb REAL,
245         ram_min_gb REAL,
246         ram_max_gb REAL,
247         ram_percent_avg REAL,
248         ram_percent_max REAL,
249         disk_read_avg_mbs REAL,
250         disk_read_max_mbs REAL,
251         disk_read_total_mb REAL,
252         disk_write_avg_mbs REAL,
253         disk_write_max_mbs REAL,
254         disk_write_total_mb REAL,
255         network_rx_avg_mbs REAL,

```



```

256         network_rx_max_mbs REAL,
257         network_rx_total_mb REAL,
258         network_tx_avg_mbs REAL,
259         network_tx_max_mbs REAL,
260         network_tx_total_mb REAL,
261         raw_json TEXT
262     );
263 ''')
264
265 # 9. Аудит оборудования
266 cursor.execute('''
267     CREATE TABLE IF NOT EXISTS hardware_audits (
268         id INTEGER PRIMARY KEY AUTOINCREMENT,
269         archive_id TEXT UNIQUE NOT NULL,
270         timestamp TEXT NOT NULL,
271         created_at REAL NOT NULL,
272         devices_count INTEGER DEFAULT 0,
273         problem_devices_count INTEGER DEFAULT 0,
274         outdated_drivers_count INTEGER DEFAULT 0,
275         changes_count INTEGER DEFAULT 0,
276         raw_json TEXT
277     );
278 ''')
279
280 # 10. История перезагрузок
281 cursor.execute('''
282     CREATE TABLE IF NOT EXISTS reboot_history (
283         id INTEGER PRIMARY KEY AUTOINCREMENT,
284         boot_id TEXT UNIQUE NOT NULL,
285         boot_time TEXT NOT NULL,
286         previous_boot_time TEXT,
287         uptime_seconds REAL,
288         uptime_human TEXT,
289         shutdown_type TEXT NOT NULL,
290         likely_class TEXT,
291         conclusion TEXT,
292         initiating_process TEXT,
293         initiating_user TEXT,

```

```

294         shutdown_action TEXT,
295         reason_text TEXT,
296         reason_code INTEGER,
297         bugcheck_code TEXT,
298         bugcheck_params_json TEXT,
299         power_button_timestamp TEXT,
300         windows_update_kb TEXT,
301         service_installed TEXT,
302         evidence_json TEXT,
303         events_chain_json TEXT,
304         raw_json TEXT,
305         created_at REAL NOT NULL
306     );
307 '''
308
309 # 11. Инциденты аномалий
310 cursor.execute('''
311     CREATE TABLE IF NOT EXISTS incidents (
312         id INTEGER PRIMARY KEY AUTOINCREMENT,
313         incident_id TEXT UNIQUE NOT NULL,
314         timestamp TEXT NOT NULL,
315         created_at REAL NOT NULL,
316         trigger_type TEXT NOT NULL,
317         severity TEXT DEFAULT 'warning',
318         title TEXT NOT NULL,
319         description TEXT,
320         trigger_metrics_json TEXT,
321         suspect_processes_json TEXT,
322         related_events_json TEXT,
323         metrics_summary_json TEXT,
324         raw_window_json TEXT
325     );
326 '''
327
328 # 12. События телеметрии
329 cursor.execute('''
330     CREATE TABLE IF NOT EXISTS telemetry_events (
331         id INTEGER PRIMARY KEY AUTOINCREMENT,

```

```

332         timestamp TEXT NOT NULL,
333         created_at REAL NOT NULL,
334         event_type TEXT NOT NULL,
335         severity TEXT DEFAULT 'info',
336         event_details TEXT,
337         raw_json TEXT
338     );
339     '''
340
341     # 13. Опросы приложений
342     cursor.execute('''
343         CREATE TABLE IF NOT EXISTS app_polls (
344             id INTEGER PRIMARY KEY AUTOINCREMENT,
345             timestamp TEXT NOT NULL,
346             created_at REAL NOT NULL,
347             app TEXT NOT NULL,
348             poll_type TEXT,
349             metric_name TEXT,
350             value REAL,
351             unit TEXT,
352             status TEXT,
353             details TEXT,
354             raw_json TEXT
355         );
356     ''')
357
358     # 14. События приложений
359     cursor.execute('''
360         CREATE TABLE IF NOT EXISTS app_events (
361             id INTEGER PRIMARY KEY AUTOINCREMENT,
362             timestamp TEXT NOT NULL,
363             created_at REAL NOT NULL,
364             app TEXT NOT NULL,
365             event_type TEXT NOT NULL,
366             status TEXT,
367             message TEXT,
368             details TEXT,
369             raw_json TEXT

```

```

370         );
371     '')
372
373     # 15. Изменения параметров приложений
374     cursor.execute('''
375         CREATE TABLE IF NOT EXISTS app_param_changes (
376             id INTEGER PRIMARY KEY AUTOINCREMENT,
377             timestamp TEXT NOT NULL,
378             created_at REAL NOT NULL,
379             app TEXT NOT NULL,
380             param_name TEXT NOT NULL,
381             old_value TEXT,
382             new_value TEXT,
383             status TEXT,
384             user TEXT,
385             details TEXT,
386             raw_json TEXT
387         );
388     '')
389
390     # 16. Пользовательские записи
391     cursor.execute('''
392         CREATE TABLE IF NOT EXISTS custom_records (
393             id INTEGER PRIMARY KEY AUTOINCREMENT,
394             timestamp TEXT NOT NULL,
395             created_at REAL NOT NULL,
396             source_file TEXT,
397             payload_json TEXT
398         );
399     '')
400
401     # 17. События устройств (PnP / Device Events)
402     cursor.execute('''
403         CREATE TABLE IF NOT EXISTS device_events (
404             id INTEGER PRIMARY KEY AUTOINCREMENT,
405             timestamp TEXT NOT NULL,
406             created_at REAL NOT NULL,
407             event_type TEXT NOT NULL,

```

```

408         device_instance_id TEXT,
409         friendly_name TEXT,
410         device_class TEXT,
411         category TEXT,
412         has_problem INTEGER DEFAULT 0,
413         problem_code INTEGER DEFAULT 0,
414         status_code INTEGER DEFAULT 0,
415         manufacturer TEXT,
416         flapping_count INTEGER DEFAULT 0,
417         uptime_seconds REAL DEFAULT 0.0,
418         raw_json TEXT
419     );
420 '''
421
422 # 18. Инвентарь устройств
423 cursor.execute('''
424     CREATE TABLE IF NOT EXISTS device_inventory (
425         id INTEGER PRIMARY KEY AUTOINCREMENT,
426         device_instance_id TEXT UNIQUE NOT NULL,
427         friendly_name TEXT,
428         device_class TEXT,
429         install_date TEXT NOT NULL,
430         created_at REAL NOT NULL,
431         updated_at REAL NOT NULL
432     );
433 '''
434
435 # 19. Системные события W64/ETW
436 cursor.execute('''
437     CREATE TABLE IF NOT EXISTS w64_events (
438         id INTEGER PRIMARY KEY AUTOINCREMENT,
439         event_id INTEGER,
440         timestamp TEXT NOT NULL,
441         created_at REAL NOT NULL,
442         event_type TEXT,
443         path TEXT,
444         pid INTEGER,
445         name TEXT,

```

```
446         provider TEXT,
447         raw_json TEXT
448     );
449     '''
450
451 # 20. Аномальные процессы
452 cursor.execute('''
453     CREATE TABLE IF NOT EXISTS process_outliers (
454         id INTEGER PRIMARY KEY AUTOINCREMENT,
455         snapshot_id INTEGER,
456         timestamp TEXT NOT NULL,
457         created_at REAL NOT NULL,
458         pid INTEGER,
459         name TEXT,
460         status TEXT,
461         cpu_percent REAL,
462         memory_mb REAL,
463         memory_percent REAL,
464         num_threads INTEGER,
465         username TEXT,
466         details TEXT
467     );
468     ''')
469
470 # 21. Расширенный системный аудит
471 cursor.execute('''
472     CREATE TABLE IF NOT EXISTS system_extended_audits (
473         id INTEGER PRIMARY KEY AUTOINCREMENT,
474         timestamp TEXT NOT NULL,
475         created_at REAL NOT NULL,
476         hostname TEXT,
477         defender_cfa_enabled INTEGER DEFAULT 0,
478         defender_asr_count INTEGER DEFAULT 0,
479         defender_exclusions_count INTEGER DEFAULT 0,
480         defender_threats_count INTEGER DEFAULT 0,
481         startup_entries_count INTEGER DEFAULT 0,
482         vss_snapshots_count INTEGER DEFAULT 0,
483         users_total_count INTEGER DEFAULT 0,
```

```

484         users_admin_count INTEGER DEFAULT 0,
485         raw_json TEXT
486     );
487     '')
488
489 # 22. Почасовые агрегаты сенсоров (telemetry_hourly)
490 cursor.execute('''
491     CREATE TABLE IF NOT EXISTS telemetry_hourly (
492         sensor_id TEXT NOT NULL,
493         bucket_start INTEGER NOT NULL,
494         bucket_end INTEGER NOT NULL,
495         sample_count INTEGER NOT NULL,
496         value_min REAL,
497         value_max REAL,
498         value_avg REAL,
499         value_sum REAL,
500         value_sum_sq REAL,
501         value_stddev REAL,
502         value_first REAL,
503         value_last REAL,
504         PRIMARY KEY (sensor_id, bucket_start)
505     );
506     '')
507
508 # 23. Суточные агрегаты сенсоров (telemetry_daily)
509 cursor.execute('''
510     CREATE TABLE IF NOT EXISTS telemetry_daily (
511         sensor_id TEXT NOT NULL,
512         bucket_start INTEGER NOT NULL,
513         bucket_end INTEGER NOT NULL,
514         sample_count INTEGER NOT NULL,
515         value_min REAL,
516         value_max REAL,
517         value_avg REAL,
518         value_sum REAL,
519         value_sum_sq REAL,
520         value_stddev REAL,
521         value_first REAL,

```

```

522         value_last REAL,
523         PRIMARY KEY (sensor_id, bucket_start)
524     );
525     '')
526
527 # 24. Недельные агрегаты сенсоров (telemetry_weekly)
528 cursor.execute('''
529     CREATE TABLE IF NOT EXISTS telemetry_weekly (
530         sensor_id TEXT NOT NULL,
531         bucket_start INTEGER NOT NULL,
532         bucket_end INTEGER NOT NULL,
533         sample_count INTEGER NOT NULL,
534         value_min REAL,
535         value_max REAL,
536         value_avg REAL,
537         value_sum REAL,
538         value_sum_sq REAL,
539         value_stddev REAL,
540         value_first REAL,
541         value_last REAL,
542         PRIMARY KEY (sensor_id, bucket_start)
543     );
544     ''')
545
546 # 25. Месячные агрегаты сенсоров (telemetry_monthly)
547 cursor.execute('''
548     CREATE TABLE IF NOT EXISTS telemetry_monthly (
549         sensor_id TEXT NOT NULL,
550         bucket_start INTEGER NOT NULL,
551         bucket_end INTEGER NOT NULL,
552         sample_count INTEGER NOT NULL,
553         value_min REAL,
554         value_max REAL,
555         value_avg REAL,
556         value_sum REAL,
557         value_sum_sq REAL,
558         value_stddev REAL,
559         value_first REAL,

```



```

560         value_last REAL,
561         PRIMARY KEY (sensor_id, bucket_start)
562     );
563     '')
564
565 # 26. Годовые агрегаты сенсоров (telemetry_yearly)
566 cursor.execute('''
567     CREATE TABLE IF NOT EXISTS telemetry_yearly (
568         sensor_id TEXT NOT NULL,
569         bucket_start INTEGER NOT NULL,
570         bucket_end INTEGER NOT NULL,
571         sample_count INTEGER NOT NULL,
572         value_min REAL,
573         value_max REAL,
574         value_avg REAL,
575         value_sum REAL,
576         value_sum_sq REAL,
577         value_stddev REAL,
578         value_first REAL,
579         value_last REAL,
580         PRIMARY KEY (sensor_id, bucket_start)
581     );
582     ''')
583
584 # 27. Фиксация аномальных всплесков сенсоров (telemetry_sp
ikes)
585 cursor.execute('''
586     CREATE TABLE IF NOT EXISTS telemetry_spikes (
587         id INTEGER PRIMARY KEY AUTOINCREMENT,
588         sensor_id TEXT NOT NULL,
589         timestamp TEXT NOT NULL,
590         created_at REAL NOT NULL,
591         value REAL NOT NULL,
592         spike_type TEXT DEFAULT 'high',
593         baseline_avg REAL,
594         baseline_stddev REAL,
595         details TEXT,
596         raw_json TEXT

```

```

597         );
598     '')
599
600     # 28. События прямого ввода-вывода (disk_io_events / ETW /
        Process I/O)
601     cursor.execute('''
602         CREATE TABLE IF NOT EXISTS disk_io_events (
603             id INTEGER PRIMARY KEY AUTOINCREMENT,
604             timestamp TEXT NOT NULL,
605             created_at REAL NOT NULL,
606             pid INTEGER NOT NULL,
607             process_name TEXT NOT NULL,
608             disk_id INTEGER NOT NULL,
609             operation TEXT NOT NULL,
610             bytes_count INTEGER NOT NULL,
611             duration_ms REAL,
612             offset INTEGER,
613             file_path TEXT
614         );
615     '')
616
617     # 29. Снимки здоровья и износа накопителей (disk_health_sn
        apshots / NVMe SMART / TBW)
618     cursor.execute('''
619         CREATE TABLE IF NOT EXISTS disk_health_snapshots (
620             id INTEGER PRIMARY KEY AUTOINCREMENT,
621             timestamp TEXT NOT NULL,
622             created_at REAL NOT NULL,
623             disk_id INTEGER NOT NULL,
624             serial_number TEXT,
625             model TEXT,
626             bus_type TEXT,
627             temperature_c REAL,
628             wear_percent REAL,
629             available_spare REAL,
630             tbw_written_tb REAL,
631             tbw_read_tb REAL,
632             power_on_hours INTEGER,

```

```

633         power_cycles INTEGER,
634         unsafe_shutdowns INTEGER,
635         media_errors INTEGER,
636         error_log_entries INTEGER,
637         health_status TEXT DEFAULT 'Healthy',
638         raw_smart_json TEXT
639     );
640     ''')
641
642     # 30. Паспорт и конфигурация физических накопителей (disk_
inventory)
643     cursor.execute('''
644         CREATE TABLE IF NOT EXISTS disk_inventory (
645             id INTEGER PRIMARY KEY AUTOINCREMENT,
646             disk_id INTEGER NOT NULL,
647             device_path TEXT NOT NULL,
648             serial_number TEXT UNIQUE,
649             vendor TEXT,
650             product TEXT,
651             revision TEXT,
652             bus_type TEXT,
653             media_type TEXT,
654             size_bytes INTEGER,
655             size_gb REAL,
656             sector_size INTEGER,
657             removable INTEGER DEFAULT 0,
658             partition_style TEXT,
659             gpt_guid TEXT,
660             partitions_json TEXT,
661             first_seen TEXT NOT NULL,
662             last_seen TEXT NOT NULL,
663             updated_at REAL NOT NULL
664         );
665     ''')
666
667     # 31. Паспорт логических томов (volume_inventory)
668     cursor.execute('''
669         CREATE TABLE IF NOT EXISTS volume_inventory (

```

```

670         id INTEGER PRIMARY KEY AUTOINCREMENT,
671         volume_guid TEXT UNIQUE NOT NULL,
672         drive_letter TEXT,
673         mount_point TEXT,
674         label TEXT,
675         filesystem TEXT,
676         filesystem_flags INTEGER,
677         cluster_size_bytes INTEGER,
678         sector_size_bytes INTEGER,
679         total_bytes INTEGER,
680         total_gb REAL,
681         disk_id INTEGER,
682         first_seen TEXT NOT NULL,
683         last_seen TEXT NOT NULL,
684         updated_at REAL NOT NULL
685     );
686 '''
687
688 # 32. Сэмплы производительности дисков (disk_performance_s
amples)
689 cursor.execute('''
690     CREATE TABLE IF NOT EXISTS disk_performance_samples (
691         id INTEGER PRIMARY KEY AUTOINCREMENT,
692         timestamp TEXT NOT NULL,
693         created_at REAL NOT NULL,
694         disk_name TEXT NOT NULL,
695         read_bytes_sec REAL,
696         write_bytes_sec REAL,
697         read_iops REAL,
698         write_iops REAL,
699         read_time_ms REAL,
700         write_time_ms REAL,
701         percent_disk_time REAL,
702         queue_length REAL
703     );
704 '''
705
706 # 33. Паспорт процессоров (cpu_inventory)

```

```

707 cursor.execute('''
708     CREATE TABLE IF NOT EXISTS cpu_inventory (
709         id INTEGER PRIMARY KEY AUTOINCREMENT,
710         processor_id INTEGER NOT NULL,
711         name TEXT UNIQUE NOT NULL,
712         vendor TEXT,
713         architecture TEXT,
714         physical_cores INTEGER,
715         logical_cores INTEGER,
716         base_frequency_mhz REAL,
717         max_frequency_mhz REAL,
718         l2_cache_kb INTEGER,
719         l3_cache_kb INTEGER,
720         socket TEXT,
721         features_json TEXT,
722         first_seen TEXT NOT NULL,
723         last_seen TEXT NOT NULL,
724         updated_at REAL NOT NULL
725     );
726 ''')
727
728 # 34. Паспорт планок памяти (ram_module_inventory)
729 cursor.execute('''
730     CREATE TABLE IF NOT EXISTS ram_module_inventory (
731         id INTEGER PRIMARY KEY AUTOINCREMENT,
732         slot_id INTEGER NOT NULL,
733         bank_label TEXT NOT NULL,
734         device_locator TEXT,
735         serial_number TEXT UNIQUE,
736         part_number TEXT,
737         manufacturer TEXT,
738         capacity_bytes INTEGER,
739         capacity_gb REAL,
740         speed_mhz INTEGER,
741         memory_type TEXT,
742         form_factor TEXT,
743         configured_voltage REAL,
744         first_seen TEXT NOT NULL,

```

```

745         last_seen TEXT NOT NULL,
746         updated_at REAL NOT NULL
747     );
748     '')
749
750 # 35. Паспорт графических ускорителей (gpu_inventory)
751 cursor.execute('''
752     CREATE TABLE IF NOT EXISTS gpu_inventory (
753         id INTEGER PRIMARY KEY AUTOINCREMENT,
754         gpu_id INTEGER NOT NULL,
755         pci_device_id TEXT UNIQUE,
756         name TEXT NOT NULL,
757         vendor TEXT,
758         driver_version TEXT,
759         driver_date TEXT,
760         vram_bytes INTEGER,
761         vram_gb REAL,
762         pci_bus_id TEXT,
763         bios_version TEXT,
764         cuda_cores INTEGER,
765         directml_supported INTEGER DEFAULT 1,
766         first_seen TEXT NOT NULL,
767         last_seen TEXT NOT NULL,
768         updated_at REAL NOT NULL
769     );
770     '')
771
772 # 36. Паспорт сетевых интерфейсов (network_adapter_invento
ry)
773 cursor.execute('''
774     CREATE TABLE IF NOT EXISTS network_adapter_inventory (
775         id INTEGER PRIMARY KEY AUTOINCREMENT,
776         adapter_id INTEGER NOT NULL,
777         adapter_guid TEXT UNIQUE NOT NULL,
778         name TEXT NOT NULL,
779         interface_name TEXT,
780         mac_address TEXT,
781         adapter_type TEXT,

```

```

782         is_physical INTEGER DEFAULT 1,
783         is_wireless INTEGER DEFAULT 0,
784         max_speed_mbps INTEGER,
785         driver_name TEXT,
786         driver_version TEXT,
787         first_seen TEXT NOT NULL,
788         last_seen TEXT NOT NULL,
789         updated_at REAL NOT NULL
790     );
791 '''
792
793 # 37. Сэмплы телеметрии процессора (cpu_telemetry_samples)
794 cursor.execute('''
795     CREATE TABLE IF NOT EXISTS cpu_telemetry_samples (
796         id INTEGER PRIMARY KEY AUTOINCREMENT,
797         timestamp TEXT NOT NULL,
798         created_at REAL NOT NULL,
799         processor_id INTEGER DEFAULT 0,
800         total_percent REAL,
801         user_percent REAL,
802         kernel_percent REAL,
803         frequency_mhz REAL,
804         temperature_c REAL,
805         package_power_w REAL,
806         core_utilization_json TEXT,
807         core_temperatures_json TEXT
808     );
809 '''
810
811 # 38. Сэмплы телеметрии оперативной памяти (ram_telemetry_
812 samples)
813 cursor.execute('''
814     CREATE TABLE IF NOT EXISTS ram_telemetry_samples (
815         id INTEGER PRIMARY KEY AUTOINCREMENT,
816         timestamp TEXT NOT NULL,
817         created_at REAL NOT NULL,
818         total_bytes INTEGER,
819         total_gb REAL,

```

```

819         used_bytes INTEGER,
820         used_gb REAL,
821         available_bytes INTEGER,
822         available_gb REAL,
823         percent_used REAL,
824         swap_total_gb REAL,
825         swap_used_gb REAL,
826         swap_percent REAL,
827         pool_paged_mb REAL,
828         pool_nonpaged_mb REAL
829     );
830 '''
831
832 # 39. Сэмплы телеметрии GPU (gpu_telemetry_samples)
833 cursor.execute('''
834     CREATE TABLE IF NOT EXISTS gpu_telemetry_samples (
835         id INTEGER PRIMARY KEY AUTOINCREMENT,
836         timestamp TEXT NOT NULL,
837         created_at REAL NOT NULL,
838         gpu_id INTEGER DEFAULT 0,
839         name TEXT NOT NULL,
840         load_percent REAL,
841         memory_used_mb REAL,
842         memory_total_mb REAL,
843         memory_percent REAL,
844         temperature_gpu_c REAL,
845         temperature_memory_c REAL,
846         fan_speed_pct REAL,
847         power_draw_w REAL,
848         clock_graphics_mhz REAL,
849         clock_memory_mhz REAL
850     );
851 '''
852
853 # 40. Сэмплы сетевых интерфейсов (network_adapter_samples)
854 cursor.execute('''
855     CREATE TABLE IF NOT EXISTS network_adapter_samples (
856         id INTEGER PRIMARY KEY AUTOINCREMENT,

```



```

857         timestamp TEXT NOT NULL,
858         created_at REAL NOT NULL,
859         adapter_name TEXT NOT NULL,
860         bytes_recv_sec REAL,
861         bytes_sent_sec REAL,
862         packets_recv_sec REAL,
863         packets_sent_sec REAL,
864         errors_in_sec REAL,
865         errors_out_sec REAL,
866         link_speed_mbps INTEGER,
867         is_connected INTEGER DEFAULT 1
868     );
869     '''
870
871     # 41. Архивные снимки автозапуска Windows (startup_audit_a
archives)
872     cursor.execute('''
873         CREATE TABLE IF NOT EXISTS startup_audit_archives (
874             id INTEGER PRIMARY KEY AUTOINCREMENT,
875             archive_id TEXT UNIQUE NOT NULL,
876             timestamp TEXT NOT NULL,
877             created_at REAL NOT NULL,
878             total_entries INTEGER DEFAULT 0,
879             active_entries INTEGER DEFAULT 0,
880             disabled_entries INTEGER DEFAULT 0,
881             broken_entries INTEGER DEFAULT 0,
882             health_score INTEGER DEFAULT 100,
883             changes_count INTEGER DEFAULT 0,
884             raw_json TEXT
885         );
886     ''')
887
888     # 42. Отличия и изменения в автозапуске между снимками (st
artup_changes)
889     cursor.execute('''
890         CREATE TABLE IF NOT EXISTS startup_changes (
891             id INTEGER PRIMARY KEY AUTOINCREMENT,
892             archive_id TEXT NOT NULL,

```

```

893         timestamp TEXT NOT NULL,
894         created_at REAL NOT NULL,
895         change_type TEXT NOT NULL,
896         entry_id TEXT NOT NULL,
897         name TEXT NOT NULL,
898         description TEXT,
899         previous_value TEXT,
900         current_value TEXT
901     );
902     '''
903
904     # 43. Снимки каналов журналов событий (event_log_channel_s
napshots)
905     cursor.execute('''
906         CREATE TABLE IF NOT EXISTS event_log_channel_snapshots
907     (
908         id INTEGER PRIMARY KEY AUTOINCREMENT,
909         snapshot_id TEXT NOT NULL,
910         timestamp DATETIME NOT NULL,
911         channel_name TEXT NOT NULL,
912         display_name TEXT,
913         description TEXT,
914         record_count INTEGER DEFAULT 0,
915         size_bytes INTEGER DEFAULT 0,
916         channel_type TEXT DEFAULT 'Admin',
917         is_enabled INTEGER DEFAULT 1,
918         created_at REAL NOT NULL
919     );
920     '''
921
922     # 44. Кеш записей журналов событий (event_log_entries_cach
e)
923     cursor.execute('''
924         CREATE TABLE IF NOT EXISTS event_log_entries_cache (
925             id INTEGER PRIMARY KEY AUTOINCREMENT,
926             snapshot_id TEXT NOT NULL,
927             timestamp DATETIME NOT NULL,
928             channel TEXT NOT NULL,

```

```

928         event_id INTEGER DEFAULT 0,
929         level TEXT DEFAULT 'Information',
930         provider_name TEXT,
931         time_created TEXT,
932         message TEXT,
933         raw_json TEXT,
934         created_at REAL NOT NULL
935     );
936     '''
937
938     # 45. Скомпилированные профили Log Intelligence (event_log
939     _intelligence_profiles)
940     cursor.execute('''
941         CREATE TABLE IF NOT EXISTS event_log_intelligence_prof
942         iles (
943             id INTEGER PRIMARY KEY AUTOINCREMENT,
944             snapshot_id TEXT NOT NULL,
945             timestamp DATETIME NOT NULL,
946             channel TEXT NOT NULL,
947             total_analyzed INTEGER DEFAULT 0,
948             unique_patterns_count INTEGER DEFAULT 0,
949             critical_incidents_json TEXT,
950             top_clusters_json TEXT,
951             bursts_json TEXT,
952             decision_gate_json TEXT,
953             created_at REAL NOT NULL
954         );
955     ''')
956
957     # 46. Профили брандмауэра (firewall_profile_snapshots)
958     cursor.execute('''
959         CREATE TABLE IF NOT EXISTS firewall_profile_snapshots
960         (
961             id INTEGER PRIMARY KEY AUTOINCREMENT,
962             snapshot_id TEXT NOT NULL,
963             timestamp DATETIME NOT NULL,
964             domain_enabled INTEGER NOT NULL,
965             private_enabled INTEGER NOT NULL,

```

```

963         public_enabled INTEGER NOT NULL,
964         domain_default_inbound TEXT,
965         private_default_inbound TEXT,
966         public_default_inbound TEXT,
967         stealth_mode_enabled INTEGER DEFAULT 1,
968         created_at REAL NOT NULL
969     );
970     ''')
971
972 # 47. Правила брандмауэра (firewall_rule_snapshots)
973 cursor.execute('''
974     CREATE TABLE IF NOT EXISTS firewall_rule_snapshots (
975         id INTEGER PRIMARY KEY AUTOINCREMENT,
976         snapshot_id TEXT NOT NULL,
977         rule_name TEXT NOT NULL,
978         display_name TEXT,
979         direction TEXT NOT NULL,
980         action TEXT NOT NULL,
981         enabled INTEGER NOT NULL,
982         protocol TEXT,
983         local_port TEXT,
984         remote_port TEXT,
985         program_path TEXT,
986         profile_mask INTEGER,
987         created_at REAL NOT NULL
988     );
989     ''')
990
991 # 48. Снимки состояния служб Windows (services_snapshots)
992 cursor.execute('''
993     CREATE TABLE IF NOT EXISTS services_snapshots (
994         id INTEGER PRIMARY KEY AUTOINCREMENT,
995         snapshot_id TEXT NOT NULL,
996         timestamp DATETIME NOT NULL,
997         service_name TEXT NOT NULL,
998         display_name TEXT,
999         state TEXT NOT NULL,
1000         state_code INTEGER,

```

```

1001         start_type TEXT,
1002         pid INTEGER,
1003         binary_path TEXT,
1004         account TEXT,
1005         is_orphaned INTEGER DEFAULT 0,
1006         created_at REAL NOT NULL
1007     );
1008 ''')
1009
1010 # 49. События изменений служб (service_change_events)
1011 cursor.execute('''
1012     CREATE TABLE IF NOT EXISTS service_change_events (
1013         id INTEGER PRIMARY KEY AUTOINCREMENT,
1014         timestamp DATETIME NOT NULL,
1015         service_name TEXT NOT NULL,
1016         display_name TEXT,
1017         action TEXT NOT NULL,
1018         old_state TEXT,
1019         new_state TEXT,
1020         performed_by TEXT DEFAULT 'SYSTEM',
1021         details_json TEXT,
1022         created_at REAL NOT NULL
1023     );
1024 ''')
1025
1026 # 50. Снимки троттлинга и электропитания CPU (cpu_throttli
ng_snapshots)
1027 cursor.execute('''
1028     CREATE TABLE IF NOT EXISTS cpu_throttling_snapshots (
1029         id INTEGER PRIMARY KEY AUTOINCREMENT,
1030         snapshot_id TEXT NOT NULL,
1031         timestamp DATETIME NOT NULL,
1032         prohot_active BOOLEAN NOT NULL DEFAULT 0,
1033         pl1_limit_watts REAL,
1034         pl2_limit_watts REAL,
1035         current_power_watts REAL,
1036         max_core_temp_c REAL,
1037         package_temp_c REAL,

```

```

1038         dpc_latency_us INTEGER DEFAULT 0,
1039         isr_latency_us INTEGER DEFAULT 0,
1040         throttling_reasons_json TEXT,
1041         created_at REAL NOT NULL
1042     );
1043 '''
1044
1045 # 51. Снимки температурных зон (thermal_zone_snapshots)
1046 cursor.execute('''
1047     CREATE TABLE IF NOT EXISTS thermal_zone_snapshots (
1048         id INTEGER PRIMARY KEY AUTOINCREMENT,
1049         snapshot_id TEXT NOT NULL,
1050         timestamp DATETIME NOT NULL,
1051         zone_name TEXT NOT NULL,
1052         temperature_c REAL NOT NULL,
1053         critical_limit_c REAL,
1054         throttling_limit_c REAL,
1055         sensor_provider TEXT NOT NULL,
1056         created_at REAL NOT NULL
1057     );
1058 '''
1059
1060 # 52. Снимки поведенческой форе́нзики (forensics_snapshots)
1061 cursor.execute('''
1062     CREATE TABLE IF NOT EXISTS forensics_snapshots (
1063         id INTEGER PRIMARY KEY AUTOINCREMENT,
1064         snapshot_id TEXT NOT NULL,
1065         timestamp DATETIME NOT NULL,
1066         foreground_window_title TEXT,
1067         foreground_process_name TEXT,
1068         foreground_pid INTEGER,
1069         user_idle_seconds INTEGER DEFAULT 0,
1070         camera_active_apps_json TEXT,
1071         microphone_active_apps_json TEXT,
1072         userassist_top_apps_json TEXT,
1073         created_at REAL NOT NULL
1074     );
1075 '''

```

```

1076
1077 # 53. Сводные снимки утечек ресурсов (process_leak_snapsho
ts)
1078 cursor.execute('''
1079     CREATE TABLE IF NOT EXISTS process_leak_snapshots (
1080         id INTEGER PRIMARY KEY AUTOINCREMENT,
1081         snapshot_id TEXT NOT NULL UNIQUE,
1082         timestamp DATETIME NOT NULL,
1083         total_processes INTEGER NOT NULL,
1084         suspicious_count INTEGER NOT NULL,
1085         created_at REAL NOT NULL
1086     );
1087 ''')
1088
1089 # 54. Детализация процессов с утечками (process_leak_items
)
1090 cursor.execute('''
1091     CREATE TABLE IF NOT EXISTS process_leak_items (
1092         id INTEGER PRIMARY KEY AUTOINCREMENT,
1093         snapshot_id TEXT NOT NULL,
1094         pid INTEGER NOT NULL,
1095         name TEXT NOT NULL,
1096         handles_count INTEGER NOT NULL,
1097         gdi_objects INTEGER NOT NULL,
1098         user_objects INTEGER NOT NULL,
1099         page_faults INTEGER NOT NULL,
1100         working_set_mb REAL NOT NULL,
1101         leak_risk_score TEXT NOT NULL,
1102         leak_risk_reasons_json TEXT,
1103         created_at REAL NOT NULL
1104     );
1105 ''')
1106
1107 # 55. Снимки защитника Defender (defender_snapshots)
1108 cursor.execute('''
1109     CREATE TABLE IF NOT EXISTS defender_snapshots (
1110         id INTEGER PRIMARY KEY AUTOINCREMENT,
1111         snapshot_id TEXT NOT NULL,

```

```

1112         timestamp DATETIME NOT NULL,
1113         real_time_protection_enabled INTEGER NOT NULL,
1114         cloud_protection_enabled INTEGER NOT NULL,
1115         behavior_monitoring_enabled INTEGER NOT NULL,
1116         tamper_protection_enabled INTEGER NOT NULL,
1117         pua_protection_enabled INTEGER NOT NULL,
1118         antivirus_enabled INTEGER NOT NULL,
1119         antispyware_enabled INTEGER NOT NULL,
1120         engine_version TEXT,
1121         av_signature_version TEXT,
1122         last_quick_scan_datetime DATETIME,
1123         last_full_scan_datetime DATETIME,
1124         security_score INTEGER DEFAULT 100,
1125         cfa_state INTEGER DEFAULT 0,
1126         raw_status_json TEXT,
1127         created_at REAL NOT NULL
1128     );
1129 '''
1130
1131 # 56. Исключения защитника (defender_exclusions)
1132 cursor.execute('''
1133     CREATE TABLE IF NOT EXISTS defender_exclusions (
1134         id INTEGER PRIMARY KEY AUTOINCREMENT,
1135         snapshot_id TEXT NOT NULL,
1136         exclusion_type TEXT NOT NULL,
1137         exclusion_value TEXT NOT NULL,
1138         risk_level TEXT DEFAULT 'SAFE',
1139         created_at REAL NOT NULL
1140     );
1141 '''
1142
1143 # 57. Правила ASR (defender_asr_rules)
1144 cursor.execute('''
1145     CREATE TABLE IF NOT EXISTS defender_asr_rules (
1146         id INTEGER PRIMARY KEY AUTOINCREMENT,
1147         snapshot_id TEXT NOT NULL,
1148         rule_guid TEXT NOT NULL,
1149         rule_name TEXT NOT NULL,

```



```

1150         rule_action TEXT NOT NULL,
1151         created_at REAL NOT NULL
1152     );
1153     '''
1154
1155     # 58. Обнаруженные угрозы (defender_threats)
1156     cursor.execute('''
1157         CREATE TABLE IF NOT EXISTS defender_threats (
1158             id INTEGER PRIMARY KEY AUTOINCREMENT,
1159             snapshot_id TEXT NOT NULL,
1160             threat_id TEXT,
1161             threat_name TEXT NOT NULL,
1162             severity TEXT NOT NULL,
1163             category TEXT,
1164             resources_json TEXT,
1165             detection_time DATETIME,
1166             created_at REAL NOT NULL
1167         );
1168     ''')
1169
1170     # 59. Снимки сетевых сокетов процессов (process_network_snapshots)
1171     cursor.execute('''
1172         CREATE TABLE IF NOT EXISTS process_network_snapshots (
1173             id INTEGER PRIMARY KEY AUTOINCREMENT,
1174             snapshot_id TEXT NOT NULL,
1175             timestamp DATETIME NOT NULL,
1176             pid INTEGER NOT NULL,
1177             process_name TEXT NOT NULL,
1178             local_address TEXT,
1179             remote_address TEXT,
1180             protocol TEXT,
1181             status TEXT,
1182             service_type TEXT,
1183             sent_kb REAL DEFAULT 0.0,
1184             recv_kb REAL DEFAULT 0.0,
1185             read_speed_kbs REAL DEFAULT 0.0,
1186             write_speed_kbs REAL DEFAULT 0.0,

```

```

1187         created_at REAL NOT NULL
1188     );
1189     ''')
1190
1191     # 60. Инвентарь установленного ПО (software_inventory)
1192     cursor.execute('''
1193         CREATE TABLE IF NOT EXISTS software_inventory (
1194             app_id TEXT PRIMARY KEY,
1195             display_name TEXT NOT NULL,
1196             version TEXT,
1197             publisher TEXT,
1198             install_date TEXT,
1199             install_location TEXT,
1200             architecture TEXT,
1201             is_system_component INTEGER DEFAULT 0,
1202             first_seen DATETIME NOT NULL,
1203             last_scanned_at DATETIME NOT NULL
1204         );
1205     ''')
1206
1207     # 61. Категоризированные места хранения ПО (software_storage_locations)
1208     cursor.execute('''
1209         CREATE TABLE IF NOT EXISTS software_storage_locations
1210     (
1211         id INTEGER PRIMARY KEY AUTOINCREMENT,
1212         app_id TEXT NOT NULL,
1213         category TEXT NOT NULL,
1214         path TEXT NOT NULL,
1215         size_bytes INTEGER DEFAULT 0,
1216         file_count INTEGER DEFAULT 0,
1217         last_updated DATETIME NOT NULL,
1218         FOREIGN KEY (app_id) REFERENCES software_inventory
1219         (app_id) ON DELETE CASCADE
1220     );
1221     ''')
1222
1223     # 62. Файлы конфигурации ПО (software_config_files)

```

```

1222 cursor.execute('''
1223     CREATE TABLE IF NOT EXISTS software_config_files (
1224         id INTEGER PRIMARY KEY AUTOINCREMENT,
1225         app_id TEXT NOT NULL,
1226         file_path TEXT NOT NULL,
1227         format TEXT,
1228         size_bytes INTEGER DEFAULT 0,
1229         snippet TEXT,
1230         FOREIGN KEY (app_id) REFERENCES software_inventory
1231         (app_id) ON DELETE CASCADE
1232     );
1233     ''')
1234
1235 # 63. Сетевые снимки ПО (software_network_snapshots)
1236 cursor.execute('''
1237     CREATE TABLE IF NOT EXISTS software_network_snapshots
1238     (
1239         id INTEGER PRIMARY KEY AUTOINCREMENT,
1240         snapshot_id TEXT NOT NULL,
1241         timestamp DATETIME NOT NULL,
1242         app_id TEXT NOT NULL,
1243         pid INTEGER NOT NULL,
1244         local_address TEXT,
1245         remote_address TEXT,
1246         protocol TEXT,
1247         domain_name TEXT,
1248         sent_kb REAL DEFAULT 0.0,
1249         recv_kb REAL DEFAULT 0.0,
1250         FOREIGN KEY (app_id) REFERENCES software_inventory
1251         (app_id) ON DELETE CASCADE
1252     );
1253     ''')
1254
1255 # 64. Анализ ПО с помощью ИИ (software_ai_research)
1256 cursor.execute('''
1257     CREATE TABLE IF NOT EXISTS software_ai_research (
1258         app_id TEXT PRIMARY KEY,
1259         summary TEXT,

```

```

1257         config_purpose_explanation TEXT,
1258         data_storage_explanation TEXT,
1259         network_activity_explanation TEXT,
1260         confirmed_facts_json TEXT,
1261         inferred_facts_json TEXT,
1262         confidence_level REAL DEFAULT 0.0,
1263         updated_at DATETIME NOT NULL,
1264         FOREIGN KEY (app_id) REFERENCES software_inventory
    (app_id) ON DELETE CASCADE
1265     );
1266     '')
1267
1268     # Индексы для ускорения выборок
1269     cursor.execute('CREATE INDEX IF NOT EXISTS idx_snapshots_c
    reated_at ON system_snapshots(created_at);')
1270     cursor.execute('CREATE INDEX IF NOT EXISTS idx_startup_arc
    hives_created_at ON startup_audit_archives(created_at);')
1271     cursor.execute('CREATE INDEX IF NOT EXISTS idx_startup_arc
    hives_archive_id ON startup_audit_archives(archive_id);')
1272     cursor.execute('CREATE INDEX IF NOT EXISTS idx_startup_cha
    nges_created_at ON startup_changes(created_at);')
1273     cursor.execute('CREATE INDEX IF NOT EXISTS idx_startup_cha
    nges_archive_id ON startup_changes(archive_id);')
1274     cursor.execute('CREATE INDEX IF NOT EXISTS idx_snapshots_t
    imESTAMP ON system_snapshots(timestamp);')
1275     cursor.execute('CREATE INDEX IF NOT EXISTS idx_snapshots_c
    reated_host ON system_snapshots(created_at, hostname);')
1276     cursor.execute('CREATE INDEX IF NOT EXISTS idx_sensor_poll
    s_sensor_time ON sensor_polls(sensor_id, created_at);')
1277     cursor.execute('CREATE INDEX IF NOT EXISTS idx_sensor_poll
    s_category ON sensor_polls(sensor_category, created_at);')
1278     cursor.execute('CREATE INDEX IF NOT EXISTS idx_processes_s
    nAPSHOT_id ON process_snapshots(snapshot_id);')
1279     cursor.execute('CREATE INDEX IF NOT EXISTS idx_processes_n
    ame_pid ON process_snapshots(name, pid);')
1280     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_snap_s
    nAPSHOT_cpu ON process_snapshots(snapshot_id, cpu_percent DESC
    );')

```

```
1281 cursor.execute('CREATE INDEX IF NOT EXISTS idx_aggregates_  
sensor_time ON sensor_aggregates_hourly(sensor_id, period_star  
t);')  
1282 cursor.execute('CREATE INDEX IF NOT EXISTS idx_rollups_dai  
ly_date ON process_rollups_daily(date, name);')  
1283 cursor.execute('CREATE INDEX IF NOT EXISTS idx_rollups_tie  
r_start_end ON telemetry_rollups(tier, period_start_epoch, per  
iod_end_epoch);')  
1284 cursor.execute('CREATE INDEX IF NOT EXISTS idx_hardware_au  
dits_archive_id ON hardware_audits(archive_id);')  
1285 cursor.execute('CREATE INDEX IF NOT EXISTS idx_reboot_hist  
ory_boot_time ON reboot_history(boot_time);')  
1286 cursor.execute('CREATE INDEX IF NOT EXISTS idx_incidents_c  
reated_at ON incidents(created_at);')  
1287 cursor.execute('CREATE INDEX IF NOT EXISTS idx_incidents_t  
rigger ON incidents(trigger_type);')  
1288 cursor.execute('CREATE INDEX IF NOT EXISTS idx_incidents_s  
ev_time ON incidents(severity, created_at);')  
1289 cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_prov_g  
uid ON process_provenance_events(process_guid);')  
1290 cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_prov_c  
reated_at ON process_provenance_events(created_at);')  
1291 cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_prov_n  
ame_pid ON process_provenance_events(name, pid);')  
1292 cursor.execute('CREATE INDEX IF NOT EXISTS idx_device_even  
ts_created_at ON device_events(created_at);')  
1293 cursor.execute('CREATE INDEX IF NOT EXISTS idx_device_inve  
ntory_dev_id ON device_inventory(device_instance_id);')  
1294 cursor.execute('CREATE INDEX IF NOT EXISTS idx_w64_events_  
created_at ON w64_events(created_at);')  
1295 cursor.execute('CREATE INDEX IF NOT EXISTS idx_process_out  
liers_created_at ON process_outliers(created_at);')  
1296 cursor.execute('CREATE INDEX IF NOT EXISTS idx_extended_au  
dits_created_at ON system_extended_audits(created_at);')  
1297 cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_h  
ourly_time ON telemetry_hourly(bucket_start);')  
1298 cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_h  
ourly_sensor ON telemetry_hourly(sensor_id, bucket_start);')
```

```
1299      cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_d
aily_time ON telemetry_daily(bucket_start);')
1300      cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_d
aily_sensor ON telemetry_daily(sensor_id, bucket_start);')
1301      cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_w
eekly_time ON telemetry_weekly(bucket_start);')
1302      cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_w
eekly_sensor ON telemetry_weekly(sensor_id, bucket_start);')
1303      cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_m
onthly_time ON telemetry_monthly(bucket_start);')
1304      cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_m
onthly_sensor ON telemetry_monthly(sensor_id, bucket_start);')
1305      cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_y
early_time ON telemetry_yearly(bucket_start);')
1306      cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_y
early_sensor ON telemetry_yearly(sensor_id, bucket_start);')
1307      cursor.execute('CREATE INDEX IF NOT EXISTS idx_telemetry_s
pikes_time ON telemetry_spikes(created_at, sensor_id);')
1308      cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_io_pid
_time ON disk_io_events(pid, created_at);')
1309      cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_io_dis
k_time ON disk_io_events(disk_id, created_at);')
1310      cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_health
_disk_time ON disk_health_snapshots(disk_id, created_at);')
1311      cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_health
_serial_time ON disk_health_snapshots(serial_number, created_a
t);')
1312      cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_inv_se
rial ON disk_inventory(serial_number);')
1313      cursor.execute('CREATE INDEX IF NOT EXISTS idx_volume_inv_
guid ON volume_inventory(volume_guid);')
1314      cursor.execute('CREATE INDEX IF NOT EXISTS idx_disk_perf_n
ame_time ON disk_performance_samples(disk_name, created_at);')
1315      cursor.execute('CREATE INDEX IF NOT EXISTS idx_cpu_inv_nam
e ON cpu_inventory(name);')
1316      cursor.execute('CREATE INDEX IF NOT EXISTS idx_ram_mod_ser
ial ON ram_module_inventory(serial_number);')
1317      cursor.execute('CREATE INDEX IF NOT EXISTS idx_gpu_inv_pci
```

```
ON gpu_inventory(pci_device_id);')
1318 cursor.execute('CREATE INDEX IF NOT EXISTS idx_net_inv_gui
d ON network_adapter_inventory(adapter_guid);')
1319 cursor.execute('CREATE INDEX IF NOT EXISTS idx_cpu_samples
_time ON cpu_telemetry_samples(created_at);')
1320 cursor.execute('CREATE INDEX IF NOT EXISTS idx_ram_samples
_time ON ram_telemetry_samples(created_at);')
1321 cursor.execute('CREATE INDEX IF NOT EXISTS idx_gpu_samples
_time ON gpu_telemetry_samples(created_at);')
1322 cursor.execute('CREATE INDEX IF NOT EXISTS idx_net_samples
_time ON network_adapter_samples(created_at);')
1323
1324 # Индексы для новых подсистем
1325 cursor.execute('CREATE INDEX IF NOT EXISTS idx_evt_chan_sn
ap_id ON event_log_channel_snapshots(snapshot_id);')
1326 cursor.execute('CREATE INDEX IF NOT EXISTS idx_evt_chan_na
me ON event_log_channel_snapshots(channel_name);')
1327 cursor.execute('CREATE INDEX IF NOT EXISTS idx_evt_entries
_snap_id ON event_log_entries_cache(snapshot_id);')
1328 cursor.execute('CREATE INDEX IF NOT EXISTS idx_evt_entries
_chan_lvl ON event_log_entries_cache(channel, level);')
1329 cursor.execute('CREATE INDEX IF NOT EXISTS idx_evt_intel_c
han ON event_log_intelligence_profiles(channel);')
1330
1331 cursor.execute('CREATE INDEX IF NOT EXISTS idx_fw_prof_sna
pshot ON firewall_profile_snapshots(snapshot_id);')
1332 cursor.execute('CREATE INDEX IF NOT EXISTS idx_fw_rule_sna
pshot ON firewall_rule_snapshots(snapshot_id);')
1333 cursor.execute('CREATE INDEX IF NOT EXISTS idx_fw_rule_nam
e ON firewall_rule_snapshots(rule_name);')
1334
1335 cursor.execute('CREATE INDEX IF NOT EXISTS idx_services_sn
ap_id ON services_snapshots(snapshot_id);')
1336 cursor.execute('CREATE INDEX IF NOT EXISTS idx_services_na
me ON services_snapshots(service_name);')
1337 cursor.execute('CREATE INDEX IF NOT EXISTS idx_services_st
ate ON services_snapshots(state);')
1338 cursor.execute('CREATE INDEX IF NOT EXISTS idx_svc_changes
```

```
1338 | _ts ON service_change_events(timestamp);')
1339 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_svc_changes
1340 | _name ON service_change_events(service_name);')
1341 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_throttling_
1342 | snapshot_id ON cpu_throttling_snapshots(snapshot_id);')
1343 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_throttling_
1344 | timestamp ON cpu_throttling_snapshots(timestamp);')
1345 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_thermal_zon
1346 | e_snapshot ON thermal_zone_snapshots(snapshot_id);')
1347 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_forensics_s
1348 | nAPSHOT_id ON forensics_snapshots(snapshot_id);')
1349 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_forensics_t
1350 | imestamp ON forensics_snapshots(timestamp);')
1351 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_leak_s
1352 | nAPSHOT_id ON process_leak_snapshots(snapshot_id);')
1353 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_leak_t
1354 | imestamp ON process_leak_snapshots(timestamp);')
1355 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_leak_i
1356 | tem_snapshot ON process_leak_items(snapshot_id);')
1357 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_leak_i
1358 | tem_pid ON process_leak_items(pid);')
1359 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_defender_sn
1360 | apshot_id ON defender_snapshots(snapshot_id);')
1361 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_defender_ti
1362 | mestamp ON defender_snapshots(timestamp);')
1363 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_defender_ex
1364 | clusions_snap ON defender_exclusions(snapshot_id);')
1365 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_defender_as
1366 | r_snap ON defender_asr_rules(snapshot_id);')
1367 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_defender_th
1368 | reats_snap ON defender_threats(snapshot_id);')
1369 |     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_net_sn
1370 | apshot_id ON process_network_snapshots(snapshot_id);')
```



```

1360     cursor.execute('CREATE INDEX IF NOT EXISTS idx_proc_net_pi
1361 | d ON process_network_snapshots(pid);')
1362     cursor.execute('CREATE INDEX IF NOT EXISTS idx_sw_storage_
1363 | app ON software_storage_locations(app_id);')
1364     cursor.execute('CREATE INDEX IF NOT EXISTS idx_sw_config_a
1365 | pp ON software_config_files(app_id);')
1366     cursor.execute('CREATE INDEX IF NOT EXISTS idx_sw_net_app_
1367 | ts ON software_network_snapshots(timestamp, app_id);')
1368
1369     _run_migrations(cursor)
1370     conn.commit()
1371
1372 def _run_migrations(cursor: sqlite3.Cursor) -> None:
1373     """Выполняет безопасные миграции добавления новых колонок
1374 | в существующие таблицы."""
1375     # Миграция: telemetry_events
1376     try:
1377         cursor.execute("PRAGMA table_info(telemetry_events)")
1378         columns = [row[1] for row in cursor.fetchall()]
1379         if 'severity' not in columns:
1380             cursor.execute("ALTER TABLE telemetry_events ADD C
1381 | OLUMN severity TEXT DEFAULT 'info'")
1382         if 'event_details' not in columns:
1383             cursor.execute("ALTER TABLE telemetry_events ADD C
1384 | OLUMN event_details TEXT")
1385         if 'raw_json' not in columns:
1386             cursor.execute("ALTER TABLE telemetry_events ADD C
1387 | OLUMN raw_json TEXT")
1388     except Exception as exc:
1389         logger.warning(f'Миграция telemetry_events: {exc}')
1390
1391     # Миграция: system_snapshots
1392     try:
1393         cursor.execute("PRAGMA table_info(system_snapshots)")
1394         snap_cols = {row[1] for row in cursor.fetchall()}
1395         expected_snap_cols = {

```

```
1390 |         'hostname': 'TEXT',
1391 |         'username': 'TEXT',
1392 |         'uptime_seconds': 'REAL',
1393 |         'os_name': 'TEXT',
1394 |         'os_build': 'TEXT',
1395 |         'os_install_date': 'TEXT',
1396 |         'system_language': 'TEXT',
1397 |         'os_install_language': 'TEXT',
1398 |         'user_locale': 'TEXT',
1399 |         'system_locale': 'TEXT',
1400 |         'timezone': 'TEXT',
1401 |         'codepage': 'TEXT',
1402 |         'input_languages_json': 'TEXT',
1403 |         'disks_json': 'TEXT',
1404 |         'physical_disks_json': 'TEXT',
1405 |         'monitors_json': 'TEXT',
1406 |         'updates_json': 'TEXT',
1407 |         'office_json': 'TEXT',
1408 |         'onedrive_json': 'TEXT',
1409 |         'battery_json': 'TEXT',
1410 |         'ram_sticks_json': 'TEXT',
1411 |         'listening_ports_json': 'TEXT',
1412 |         'alerts_json': 'TEXT',
1413 |         'cpu_total_percent': 'REAL',
1414 |         'cpu_frequency_mhz': 'REAL',
1415 |         'memory_total_gb': 'REAL',
1416 |         'memory_used_gb': 'REAL',
1417 |         'memory_percent': 'REAL',
1418 |         'swap_percent': 'REAL',
1419 |         'gpu_load_percent': 'REAL',
1420 |         'gpu_temp_c': 'REAL',
1421 |         'disk_read_bytes_sec': 'REAL',
1422 |         'disk_write_bytes_sec': 'REAL',
1423 |         'disk_read_count_sec': 'REAL',
1424 |         'disk_write_count_sec': 'REAL',
1425 |         'network_sent_bytes_sec': 'REAL',
1426 |         'network_recv_bytes_sec': 'REAL',
1427 |         'hardware_audit': 'TEXT',
```

```

1428         'raw_json': 'TEXT',
1429     }
1430     for col_name, col_type in expected_snap_cols.items():
1431         if col_name not in snap_cols:
1432             cursor.execute(f"ALTER TABLE system_snapshots
1433 ADD COLUMN {col_name} {col_type}")
1434     except Exception as exc:
1435         logger.warning(f'Миграция system_snapshots: {exc}')
1436
1437     # Миграция: process_snapshots
1438     try:
1439         cursor.execute("PRAGMA table_info(process_snapshots)")
1440         proc_cols = {row[1] for row in cursor.fetchall()}
1441         new_proc_columns = {
1442             'num_handles': 'INTEGER DEFAULT 0',
1443             'read_bytes_sec': 'REAL',
1444             'write_bytes_sec': 'REAL',
1445             'integrity_level': 'TEXT',
1446             'elevation': 'INTEGER DEFAULT 0',
1447             'ppid': 'INTEGER',
1448             'parent_name': 'TEXT',
1449             'executable': 'TEXT',
1450             'cmdline': 'TEXT',
1451             'sid': 'TEXT',
1452             'session_id': 'INTEGER',
1453             'creation_time': 'TEXT',
1454             'process_guid': 'TEXT',
1455             'ancestor_chain': 'TEXT',
1456             'launch_reason': 'TEXT',
1457         }
1458         for col_name, col_def in new_proc_columns.items():
1459             if col_name not in proc_cols:
1460                 cursor.execute(f"ALTER TABLE process_snapshots
1461 ADD COLUMN {col_name} {col_def}")
1462     except Exception as exc:
1463         logger.warning(f'Миграция process_snapshots: {exc}')

```

storage.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Storage
4 # =====
5 # Description:
6 #   Главный фасад TelemetryStorage базы данных SQLite телеметр
  ии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite import TelemetryStora
    ge
11 #
12 #       storage = TelemetryStorage.get_instance(read_only=True)
13 #       snapshots = storage.get_snapshots(limit=10)
14 #
15 # File: storage.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 19:49:00
21 # =====
22
23 from __future__ import annotations
24
25 """Главный класс TelemetryStorage, объединяющий подсистемы чте
  ния, записи, буферизации и обслуживания."""
26
27 import atexit
28 import os
29 import sqlite3
```

```
30 import threading
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional, Union
33
34 from .aggregator import AggregationLevel, TelemetrySqlAggregator
35 from .buffer import TelemetryBuffer
36 from .connection import TelemetryConnectionManager
37 from .maintenance import TelemetryMaintenance
38 from .reader import TelemetryReader
39 from .schema import init_database_schema
40 from .writer import TelemetryWriter
41 from ..models import (
42     HardwareArchiveEntry,
43     ProcessLifecycleEvent,
44     ProcessProvenanceInfo,
45     StartupArchiveEntry,
46     SystemMetricRollup,
47     SystemSnapshot,
48     TelemetryIncident,
49 )
50 from ..telemetry_config import TelemetryConfigManager
51
52 try:
53     from logger import logger
54 except ImportError:
55     import logging
56     logger = logging.getLogger(__name__)
57
58
59 class TelemetryStorage:
60     """Универсальный фасад персистентного хранения и выборки т
    елеметрии в SQLite."""
61
62     _instance: Optional[TelemetryStorage] = None
63     _ro_instance: Optional[TelemetryStorage] = None
64     _lock = threading.RLock()
65
```

```

66     def __init__(
67         self,
68         db_path: Optional[Union[str, Path]] = None,
69         buffer_mode: Optional[str] = None,
70         buffer_size: Optional[int] = None,
71         flush_interval_seconds: Optional[float] = None,
72         buffer_file_path: Optional[Union[str, Path]] = None,
73         auto_flush: bool = True,
74         max_db_size_mb: Optional[float] = None,
75         retention_days: Optional[int] = None,
76         auto_vacuum: Optional[bool] = None,
77         read_only: bool = False,
78     ) -> None:
79         """Инициализирует подключение к SQLite базе данных тел
80         еметрии."""
81
82         self.read_only = bool(read_only)
83
84         # 1. Определение путей базы данных
85         if db_path is None:
86             programdata = os.environ.get('ProgramData') or os.
87             environ.get('ALLUSERSPROFILE')
88             if programdata and os.path.exists(programdata):
89                 base_dir = Path(programdata)
90             else:
91                 appdata = os.environ.get('APPDATA') or os.envi
92                 ron.get('LOCALAPPDATA')
93                 base_dir = Path(appdata) if appdata and os.pat
94                 h.exists(appdata) else (Path.home() / '.config')
95
96                 ai_tel_dir = base_dir / 'AITelemetry' / 'data'
97                 if (ai_tel_dir / 'telemetry.db').is_file() or ai_t
98                 el_dir.is_dir():
99                     target_dir = ai_tel_dir
100                 else:
101                     target_dir = base_dir / 'AI-Breadboard' / 'app
102                     s' / 'windows' / 'telemetry' / 'logs'
103
104                 if not self.read_only:

```

```
98         target_dir.mkdir(parents=True, exist_ok=True)
99         self.db_path = target_dir / 'telemetry.db'
100     else:
101         self.db_path = Path(db_path)
102         if not self.read_only:
103             self.db_path.parent.mkdir(parents=True, exist_
ok=True)
104
105     # 2. Конфигурация
106     cfg_manager = None
107     try:
108         cfg_manager = TelemetryConfigManager()
109     except Exception:
110         cfg_manager = None
111
112     raw_mode = buffer_mode or (cfg_manager.get_buffer_mode
() if cfg_manager else 'memory')
113     b_mode = raw_mode.lower() if raw_mode in ('memory', 'f
ile', 'direct') else 'memory'
114     b_size = int(buffer_size if buffer_size is not None el
se (cfg_manager.get_buffer_size() if cfg_manager else 50))
115     f_interval = float(flush_interval_seconds if flush_int
erval_seconds is not None else (cfg_manager.get_flush_interval
_seconds() if cfg_manager else 30.0))
116     m_size = float(max_db_size_mb if max_db_size_mb is not
None else (cfg_manager.get_max_db_size_mb() if cfg_manager el
se 50.0))
117     r_days = int(retention_days if retention_days is not N
one else (cfg_manager.get_retention_days() if cfg_manager else
7))
118     a_vacuum = bool(auto_vacuum if auto_vacuum is not None
else (cfg_manager.is_auto_vacuum_enabled() if cfg_manager els
e True))
119
120     # 3. Инициализация менеджера подключений
121     self._cm = TelemetryConnectionManager(db_path=self.db_
path, read_only=self.read_only, lock=self._lock)
122
```

```

123 |         # 4. Инициализация схемы (только в режиме записи)
124 |         if not self.read_only:
125 |             try:
126 |                 with self._cm.lock, self._cm.get_connection()
| as conn:
127 |                     init_database_schema(conn)
128 |             except Exception as schema_err:
129 |                 logger.warning(f'Ошибка инициализации схемы ба
| зы данных: {schema_err}')
130 |
131 |         # 5. Инициализация подсистем
132 |         self._reader = TelemetryReader(self._cm)
133 |         self._writer = TelemetryWriter(self._cm)
134 |         self._buffer = TelemetryBuffer(
135 |             connection_manager=self._cm,
136 |             writer=self._writer,
137 |             buffer_mode=b_mode,
138 |             buffer_size=b_size,
139 |             flush_interval_seconds=f_interval,
140 |             buffer_file_path=buffer_file_path,
141 |             auto_flush=auto_flush,
142 |         )
143 |         self._maintenance = TelemetryMaintenance(
144 |             connection_manager=self._cm,
145 |             max_db_size_mb=m_size,
146 |             retention_days=r_days,
147 |             auto_vacuum_enabled=a_vacuum,
148 |             buffer_file_path=self._buffer.buffer_file_path,
149 |         )
150 |         self._aggregator = TelemetrySqlAggregator(self._cm)
151 |
152 |         if not self.read_only:
153 |             try:
154 |                 atexit.register(self.close)
155 |             except Exception:
156 |                 pass
157 |
158 |         @classmethod

```



```

159     def get_instance(
160         cls,
161         db_path: Optional[Union[str, Path]] = None,
162         read_only: bool = False,
163     ) -> TelemetryStorage:
164         """Возвращает синглтон-экземпляр хранилища телеметрии
165         (для записи или только для чтения)."""
166         with cls._lock:
167             if read_only:
168                 if cls._ro_instance is None or (db_path is not
169                 None and cls._ro_instance.db_path != Path(db_path)):
170                     cls._ro_instance = TelemetryStorage(db_path=
171                     db_path, read_only=True, auto_flush=False)
172                 return cls._ro_instance
173             else:
174                 if cls._instance is None or (db_path is not No
175                 ne and cls._instance.db_path != Path(db_path)):
176                     cls._instance = TelemetryStorage(db_path=db
177                     _path, read_only=False)
178                 return cls._instance
179
180     def _get_connection(self) -> sqlite3.Connection:
181         """Создает и настраивает соединение с базой данных SQL
182         ite."""
183         return self._cm.get_connection()
184
185     @property
186     def reader(self) -> TelemetryReader:
187         """Подсистема чтения данных."""
188         return self._reader
189
190     @property
191     def writer(self) -> TelemetryWriter:
192         """Подсистема записи данных."""
193         return self._writer
194
195     @property
196     def buffer(self) -> TelemetryBuffer:

```

```

191         """Подсистема буферизации."""
192         return self._buffer
193
194     @property
195     def maintenance(self) -> TelemetryMaintenance:
196         """Подсистема регламентного обслуживания."""
197         return self._maintenance
198
199     @property
200     def aggregator(self) -> TelemetrySqlAggregator:
201         """Подсистема многоуровневой SQL-агрегации."""
202         return self._aggregator
203
204     # -----
205     # Делегирование методов буфера
206     # -----
207
208     def set_buffer_mode(self, mode: str) -> None:
209         self._buffer.set_buffer_mode(mode)
210
211     def get_buffer_mode(self) -> str:
212         return self._buffer.buffer_mode
213
214     def set_flush_interval_seconds(self, interval: float) -> N
one:
215         """Обновляет интервал таймера сброса буфера в SQLite."
""
216         self._buffer.set_flush_interval_seconds(interval)
217
218     def get_buffered_count(self) -> int:
219         return self._buffer.get_buffered_count()
220
221     def flush(self) -> int:
222         return self._buffer.flush()
223
224     def close(self) -> None:

```

```

225         self._aggregator.stop_background_scheduler()
226         self._buffer.close()
227
228         # -----
229         # Делегирование методов сохранения (Writer / Buffer)
230         # -----
231
232     def save_snapshot(self, snapshot: SystemSnapshot, top_n: i
nt = 20) -> int:
233         if self._buffer.buffer_mode == 'direct':
234             with self._cm.lock, self._cm.get_connection() as c
onn:
235                 snap_id = self._writer.insert_snapshot_row(con
n.cursor(), {'data': snapshot, 'top_n': top_n})
236                 conn.commit()
237                 return snap_id
238             snap_dict = snapshot if isinstance(snapshot, dict) els
e (snapshot.model_dump() if hasattr(snapshot, 'model_dump') el
se (vars(snapshot) if hasattr(snapshot, '__dict__') else snaps
hot))
239             self._buffer.enqueue({'type': 'snapshot', 'data': snap
_dict, 'top_n': top_n})
240             return 1
241
242     def save_sensor_poll(self, sensor_item: Dict[str, Any], ti
mestamp: Optional[str] = None) -> int:
243         if self._buffer.buffer_mode == 'direct':
244             with self._cm.lock, self._cm.get_connection() as c
onn:
245                 row = self._writer.prepare_sensor_poll_row(sen
sor_item, timestamp)
246                 conn.cursor().execute('''
247                     INSERT INTO sensor_polls (
248                         sensor_id, timestamp, created_at, hard
ware_name, hardware_type,
249                         sensor_category, sensor_name, unit, va

```

```

    lue, provider, provider_priority, raw_json
250         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
    ?)
251         ON CONFLICT(sensor_id, timestamp) DO UPDAT
    E SET
252             created_at = excluded.created_at, hard
    ware_name = excluded.hardware_name,
253             hardware_type = excluded.hardware_type
    , sensor_category = excluded.sensor_category,
254             sensor_name = excluded.sensor_name, un
    it = excluded.unit, value = excluded.value,
255             provider = excluded.provider, provider
    _priority = excluded.provider_priority,
256             raw_json = excluded.raw_json
257         '', row)
258         conn.commit()
259         return 1
260         self._buffer.enqueue({'type': 'sensor_poll', 'data': s
    ensor_item, 'timestamp': timestamp})
261         return 1
262
263     def save_sensor_polls_batch(self, items: List[Dict[str, An
    y]], timestamp: Optional[str] = None, deduplicate: bool = True
    ) -> int:
264         if not items:
265             return 0
266         if deduplicate:
267             from ..sensor_registry import SensorDeduplicator,
    SensorProvider
268             deduplicator = SensorDeduplicator()
269             for item in items:
270                 sensor_id = str(item.get('id', ''))
271                 value = item.get('value', 0.0)
272                 provider_raw = item.get('_provider', SensorPro
    vider.SENSOR_COLLECTOR)
273                 if isinstance(provider_raw, str):
274                     try:
275                         provider = SensorProvider[provider_raw

```

```

        .upper()]
276         except KeyError:
277             provider = SensorProvider.SENSOR_COLLE
        CTOR
278             elif isinstance(provider_raw, SensorProvider):
279                 provider = provider_raw
280             else:
281                 provider = SensorProvider.SENSOR_COLLECTOR
282                 deduplicator.add(sensor_id, provider, value)
283                 unique_providers = deduplicator.get_unique()
284                 items = [it for it in items if str(it.get('id', ''
        )) in unique_providers]
285
286         if not items:
287             return 0
288         if self._buffer.buffer_mode == 'direct':
289             return self._writer.batch_insert_records([{'type':
        'sensor_polls_batch', 'data': items, 'timestamp': timestamp}
        ])
290         self._buffer.enqueue({'type': 'sensor_polls_batch', 'd
        ata': items, 'timestamp': timestamp})
291         return len(items)
292
293         save_sensor_polls = save_sensor_polls_batch
294
295         def save_sensor_aggregates_batch(self, aggregates: List[Di
        ct[str, Any]]) -> int:
296             if not aggregates:
297                 return 0
298             if self._buffer.buffer_mode == 'direct':
299                 return self._writer.batch_insert_records([{'type':
        'sensor_aggregates_batch', 'data': aggregates}])
300             self._buffer.enqueue({'type': 'sensor_aggregates_batch
        ', 'data': aggregates})
301             return len(aggregates)
302
303         def save_event(self, event_type: str, event_details: Dict[
        str, Any], severity: str = 'info', timestamp: Optional[str] =

```

```

None) -> int:
304         rec = {'type': 'event', 'event_type': event_type, 'details': event_details, 'severity': severity, 'timestamp': timestamp}
305         if self._buffer.buffer_mode == 'direct':
306             return self._writer.batch_insert_records([rec])
307         self._buffer.enqueue(rec)
308         return 1
309
310     def save_hardware_archive(self, archive_entry: HardwareArchiveEntry) -> int:
311         entry_dict = archive_entry if isinstance(archive_entry, dict) else (archive_entry.model_dump() if hasattr(archive_entry, 'model_dump') else (vars(archive_entry) if hasattr(archive_entry, '__dict__') else archive_entry))
312         if self._buffer.buffer_mode == 'direct':
313             return self._writer.batch_insert_records([{'type': 'hardware_archive', 'data': entry_dict}])
314         self._buffer.enqueue({'type': 'hardware_archive', 'data': entry_dict})
315         return 1
316
317     def save_startup_archive(self, archive_entry: StartupArchiveEntry) -> int:
318         """Сохраняет снимок аудита автозапуска и зафиксированные изменения в БД."""
319         entry_dict = archive_entry if isinstance(archive_entry, dict) else (archive_entry.model_dump() if hasattr(archive_entry, 'model_dump') else (vars(archive_entry) if hasattr(archive_entry, '__dict__') else archive_entry))
320         if self._buffer.buffer_mode == 'direct':
321             return self._writer.batch_insert_records([{'type': 'startup_archive', 'data': entry_dict}])
322         self._buffer.enqueue({'type': 'startup_archive', 'data': entry_dict})
323         return 1
324
325     def save_app_poll(self, app: str, poll_type: str, metric_n

```

```

    ame: str, value: Any, unit: str = '', status: str = 'OK', details: Any = '', timestamp: Optional[str] = None) -> int:
326         rec = {'type': 'app_poll', 'app': app, 'poll_type': poll_type, 'metric_name': metric_name, 'value': value, 'unit': unit, 'status': status, 'details': details, 'timestamp': timestamp}
327         if self._buffer.buffer_mode == 'direct':
328             return self._writer.batch_insert_records([rec])
329         self._buffer.enqueue(rec)
330         return 1
331
332     def save_app_event(self, app: str, event_type: str, status: str = 'OK', details: Any = '', timestamp: Optional[str] = None) -> int:
333         rec = {'type': 'app_event', 'app': app, 'event_type': event_type, 'status': status, 'details': details, 'timestamp': timestamp}
334         if self._buffer.buffer_mode == 'direct':
335             return self._writer.batch_insert_records([rec])
336         self._buffer.enqueue(rec)
337         return 1
338
339     def save_app_param_change(self, app: str, param_name: str, old_value: Any, new_value: Any, status: str = 'SUCCESS', user: str = 'system', details: Any = '', timestamp: Optional[str] = None) -> int:
340         rec = {'type': 'app_param_change', 'app': app, 'param_name': param_name, 'old_value': old_value, 'new_value': new_value, 'status': status, 'user': user, 'details': details, 'timestamp': timestamp}
341         if self._buffer.buffer_mode == 'direct':
342             return self._writer.batch_insert_records([rec])
343         self._buffer.enqueue(rec)
344         return 1
345
346     def save_app_polls_batch(self, polls: List[Dict[str, Any]]) -> int:
347         if not polls:

```

```

348         return 0
349         if self._buffer.buffer_mode == 'direct':
350             return self._writer.batch_insert_records([{'type':
| 'app_polls_batch', 'data': polls}])
351         self._buffer.enqueue({'type': 'app_polls_batch', 'data
| ': polls})
352         return len(polls)
353
354     def save_app_events_batch(self, events: List[Dict[str, Any
| ]]) -> int:
355         if not events:
356             return 0
357         if self._buffer.buffer_mode == 'direct':
358             return self._writer.batch_insert_records([{'type':
| 'app_events_batch', 'data': events}])
359         self._buffer.enqueue({'type': 'app_events_batch', 'dat
| a': events})
360         return len(events)
361
362     def save_app_param_changes_batch(self, param_changes: List
| [Dict[str, Any]]) -> int:
363         if not param_changes:
364             return 0
365         if self._buffer.buffer_mode == 'direct':
366             return self._writer.batch_insert_records([{'type':
| 'app_param_changes_batch', 'data': param_changes}])
367         self._buffer.enqueue({'type': 'app_param_changes_batch
| ', 'data': param_changes})
368         return len(param_changes)
369
370     def save_custom_record(self, source_file: str, payload: Di
| ct[str, Any], timestamp: Optional[str] = None) -> int:
371         rec = {'type': 'custom_record', 'source_file': source_
| file, 'payload': payload, 'timestamp': timestamp}
372         if self._buffer.buffer_mode == 'direct':
373             return self._writer.batch_insert_records([rec])
374         self._buffer.enqueue(rec)
375         return 1

```



```

376
377     def save_device_event(self, event: Dict[str, Any], timesta
mp: Optional[str] = None) -> int:
378         rec = {'type': 'device_event', 'data': event, 'timesta
mp': timestamp}
379         if self._buffer.buffer_mode == 'direct':
380             return self._writer.batch_insert_records([rec])
381         self._buffer.enqueue(rec)
382         return 1
383
384     def save_w64_event(self, event_data: Dict[str, Any], provi
der: str = 'w64_collector') -> int:
385         rec = {'type': 'w64_event', 'data': event_data, 'provi
der': provider}
386         if self._buffer.buffer_mode == 'direct':
387             return self._writer.batch_insert_records([rec])
388         self._buffer.enqueue(rec)
389         return 1
390
391     def save_process_provenance_event(self, event: Union[Proce
ssLifecycleEvent, ProcessProvenanceInfo, Dict[str, Any]]) -> b
ool:
392         ev_dict = event if isinstance(event, dict) else (event
.model_dump() if hasattr(event, 'model_dump') else (vars(event
) if hasattr(event, '__dict__') else event))
393         if self._buffer.buffer_mode == 'direct':
394             return self._writer.batch_insert_records([{'type':
'process_provenance_event', 'data': ev_dict}]) > 0
395         self._buffer.enqueue({'type': 'process_provenance_even
t', 'data': ev_dict})
396         return True
397
398     def save_process_provenance_batch(self, events: List[Union
[ProcessLifecycleEvent, ProcessProvenanceInfo, Dict[str, Any]]
]) -> int:
399         if not events:
400             return 0
401         ev_dicts = [e if isinstance(e, dict) else (e.model_dum

```

```

    p() if hasattr(e, 'model_dump') else (vars(e) if hasattr(e, '__dict__') else e)) for e in events]
402         if self._buffer.buffer_mode == 'direct':
403             return self._writer.batch_insert_records([{'type':
    'process_provenance_events_batch', 'data': ev_dicts}])
404         self._buffer.enqueue({'type': 'process_provenance_events_batch', 'data': ev_dicts})
405         return len(events)
406
407     def save_incident(self, incident: TelemetryIncident) -> bool:
408         return self._writer.batch_insert_records([{'type': 'incident',
409             'data': incident if isinstance(incident, dict) else (incident.model_dump() if hasattr(incident, 'model_dump') else (vars(incident) if hasattr(incident, '__dict__') else incident))
411             }]) > 0 or True
412
413     def save_system_rollup(self, rollup: SystemMetricRollup) -> bool:
414         return self._writer.batch_insert_records([{'type': 'system_rollup',
415             'data': rollup if isinstance(rollup, dict) else (rollup.model_dump() if hasattr(rollup, 'model_dump') else (vars(rollup) if hasattr(rollup, '__dict__') else rollup))
417             }]) > 0 or True
418
419     def save_reboot_session(self, session: Any) -> int:
420         return self._writer.batch_insert_records([{'type': 'reboot_session',
421             'data': session if isinstance(session, dict) else (session.model_dump() if hasattr(session, 'model_dump') else (vars(session) if hasattr(session, '__dict__') else session))
423             }]) or 1
424
425     def save_extended_audit(self, audit: Any, timestamp: Optional[str] = None) -> int:

```

```

426         return self._writer.batch_insert_records([
427             'type': 'extended_audit',
428             'data': audit if isinstance(audit, dict) else (audit.model_dump() if hasattr(audit, 'model_dump') else (vars(audit) if hasattr(audit, '__dict__') else audit)),
429             'timestamp': timestamp,
430         ]) or 1
431
432     # -----
433     # Делегирование методов чтения (Reader)
434     # -----
435
436     def get_snapshots(self, limit: int = 60, since_epoch: Optional[float] = None) -> List[Dict[str, Any]]:
437         return self._reader.get_snapshots(limit=limit, since_epoch=since_epoch)
438
439     def get_latest_snapshot_full(self) -> Optional[Dict[str, Any]]:
440         """Извлекает и десериализует последний системный снимок со всеми метаданными и процессами."""
441         return self._reader.get_latest_snapshot_full()
442
443     def get_latest_cpu(self) -> Optional[Dict[str, Any]]:
444         return self._reader.get_latest_cpu()
445
446     def get_cpu_history(self, limit: int = 60, since_epoch: Optional[float] = None) -> List[Dict[str, Any]]:
447         return self._reader.get_cpu_history(limit=limit, since_epoch=since_epoch)
448
449     def get_latest_storage_data(self) -> Dict[str, Any]:
450         return self._reader.get_latest_storage_data()
451
452     def get_snapshot_processes(self, snapshot_id: int) -> List[Dict[str, Any]]:

```

```

453         return self._reader.get_snapshot_processes(snapshot_id
=snapshot_id)
454
455     def get_process_history(self, name: Optional[str] = None,
pid: Optional[int] = None, limit: int = 100) -> List[Dict[str,
Any]]:
456         return self._reader.get_process_history(name=name, pid
=pid, limit=limit)
457
458     def get_process_provenance_history(self, **kwargs: Any) ->
List[Dict[str, Any]]:
459         return self._reader.get_process_provenance_history(**k
wargs)
460
461     def get_process_lineage(self, guid_or_pid: Union[str, int]
) -> List[Dict[str, Any]]:
462         return self._reader.get_process_lineage(guid_or_pid=gu
id_or_pid)
463
464     def get_latest_processes(self, limit: int = 50, sort_by: s
tr = 'cpu') -> List[Dict[str, Any]]:
465         return self._reader.get_latest_processes(limit=limit,
sort_by=sort_by)
466
467     def get_process_stats(self, name: Optional[str] = None, li
mit: int = 50) -> Dict[str, Any]:
468         return self._reader.get_process_stats(name=name, limit
=limit)
469
470     def get_events(self, event_type: Optional[str] = None, lim
it: int = 100) -> List[Dict[str, Any]]:
471         return self._reader.get_events(event_type=event_type,
limit=limit)
472
473     def get_sensor_history(self, sensor_id: Optional[str] = No
ne, category: Optional[str] = None, limit: int = 100) -> List[
Dict[str, Any]]:
474         return self._reader.get_sensor_history(sensor_id=senso

```

```

    r_id, category=category, limit=limit)
475
476     def get_latest_sensors(self, limit: Optional[int] = None)
    -> List[Dict[str, Any]]:
477         return self._reader.get_latest_sensors(limit=limit)
478
479     def get_cpu_hierarchy_metrics(self) -> Dict[str, Any]:
480         """Извлекает иерархические метрики процессора, ядер, п
отоков и сенсоров из базы телеметрии."""
481         return self._reader.get_cpu_hierarchy_metrics()
482
483     def get_app_polls(self, app: Optional[str] = None, metric_
name: Optional[str] = None, limit: int = 100) -> List[Dict[str
, Any]]:
484         return self._reader.get_app_polls(app=app, metric_name
=metric_name, limit=limit)
485
486     def get_app_events(self, app: Optional[str] = None, event_
type: Optional[str] = None, limit: int = 100) -> List[Dict[str
, Any]]:
487         return self._reader.get_app_events(app=app, event_type
=event_type, limit=limit)
488
489     def get_app_param_changes(self, app: Optional[str] = None,
    limit: int = 100) -> List[Dict[str, Any]]:
490         return self._reader.get_app_param_changes(app=app, lim
it=limit)
491
492     def export_app_polls_to_csv(self, app: Optional[str] = Non
e, output_path: Optional[Union[str, Path]] = None, limit: int
    = 50000) -> Path:
493         return self._reader.export_app_polls_to_csv(app=app, o
utput_path=output_path, limit=limit)
494
495     def export_app_events_to_csv(self, app: Optional[str] = No
ne, output_path: Optional[Union[str, Path]] = None, limit: int
    = 50000) -> Path:
496         return self._reader.export_app_events_to_csv(app=app,

```

```

    output_path=output_path, limit=limit)
497
498     def export_app_param_changes_to_csv(self, app: Optional[str] = None, output_path: Optional[Union[str, Path]] = None, limit: int = 50000) -> Path:
499         return self._reader.export_app_param_changes_to_csv(app=app, output_path=output_path, limit=limit)
500
501     def get_custom_records(self, source_file: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
502         return self._reader.get_custom_records(source_file=source_file, limit=limit)
503
504     def get_device_events(self, event_type: Optional[str] = None, device_instance_id: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
505         return self._reader.get_device_events(event_type=event_type, device_instance_id=device_instance_id, limit=limit)
506
507     def get_device_install_date(self, device_instance_id: str) -> Optional[str]:
508         return self._reader.get_device_install_date(device_instance_id=device_instance_id)
509
510     def get_or_create_device_install_date(
511         self,
512         device_instance_id: str,
513         friendly_name: Optional[str] = None,
514         device_class: Optional[str] = None,
515         default_date: Optional[str] = None,
516     ) -> str:
517         """Возвращает существующую дату установки устройства или сохраняет новую."""
518         existing = self._reader.get_device_install_date(device_instance_id)
519         if existing:
520             return existing
521         date_to_save = default_date or datetime.now(timezone.u

```

```

tc).strftime('%d.%m.%Y %H:%M')
522         if not self.read_only:
523             try:
524                 with self._cm.lock, self._cm.get_connection()
as conn:
525                     cursor = conn.cursor()
526                     self._writer.upsert_device_inventory(
527                         cursor=cursor,
528                         device_instance_id=device_instance_id,
529                         friendly_name=friendly_name,
530                         device_class=device_class,
531                         install_date=date_to_save,
532                     )
533                     conn.commit()
534             except Exception as ex:
535                 logger.debug(f'Ошибка сохранения инвентаря уст
ройства: {ex}')
536         return date_to_save
537
538     def get_w64_events(self, event_type: Optional[str] = None,
provider: Optional[str] = None, limit: int = 100) -> List[Dic
t[str, Any]]:
539         return self._reader.get_w64_events(event_type=event_ty
pe, provider=provider, limit=limit)
540
541     def get_incidents(self, limit: int = 50, trigger_type: Opt
ional[str] = None, severity: Optional[str] = None) -> List[Dic
t[str, Any]]:
542         return self._reader.get_incidents(limit=limit, trigger
_type=trigger_type, severity=severity)
543
544     def get_system_rollups(self, tier: str = '1m', start_epoch
: Optional[float] = None, end_epoch: Optional[float] = None, l
imit: int = 100) -> List[Dict[str, Any]]:
545         return self._reader.get_system_rollups(tier=tier, star
t_epoch=start_epoch, end_epoch=end_epoch, limit=limit)
546
547     def get_reboot_history(self, limit: int = 50) -> List[Dict

```

```

    [str, Any]]:
548         return self._reader.get_reboot_history(limit=limit)
549
550     def get_latest_reboot_session(self) -> Optional[Dict[str,
Any]]:
551         return self._reader.get_latest_reboot_session()
552
553     def get_extended_audits(self, limit: int = 50) -> List[Dic
t[str, Any]]:
554         return self._reader.get_extended_audits(limit=limit)
555
556     def get_latest_extended_audit(self) -> Optional[Dict[str,
Any]]:
557         return self._reader.get_latest_extended_audit()
558
559     def get_hardware_audits(self, limit: int = 50) -> List[Dic
t[str, Any]]:
560         return self._reader.get_hardware_audits(limit=limit)
561
562     def get_latest_hardware_audit(self) -> Optional[Dict[str,
Any]]:
563         return self._reader.get_latest_hardware_audit()
564
565     def get_startup_archives(self, limit: int = 50) -> List[Di
ct[str, Any]]:
566         return self._reader.get_startup_archives(limit=limit)
567
568     def get_latest_startup_archive(self) -> Optional[Dict[str,
Any]]:
569         return self._reader.get_latest_startup_archive()
570
571     def get_startup_changes(self, limit: int = 100, archive_id
: Optional[str] = None) -> List[Dict[str, Any]]:
572         return self._reader.get_startup_changes(limit=limit, a
rchive_id=archive_id)
573
574     def get_telemetry_rollups(self, level: str = 'hourly', sen
sor_id: Optional[str] = None, start_epoch: Optional[float] = N

```



```

one, end_epoch: Optional[float] = None, limit: int = 100) -> List[Dict[str, Any]]:
575     return self._reader.get_telemetry_rollups(level=level,
        sensor_id=sensor_id, start_epoch=start_epoch, end_epoch=end_e
        poch, limit=limit)
576
577     def get_telemetry_spikes(self, sensor_id: Optional[str] =
        None, start_epoch: Optional[float] = None, limit: int = 100) -
        > List[Dict[str, Any]]:
578         return self._reader.get_telemetry_spikes(sensor_id=sen
        sor_id, start_epoch=start_epoch, limit=limit)
579
580     # -----
        -----
581     # Делегирование методов хранилища накопителей (Storage Sub
        system)
582     # -----
        -----
583
584     def save_disk_inventory(self, disk_data: Any) -> None:
585         """Сохранить или обновить паспорт физического накопите
        ля."""
586         rec = {'type': 'disk_inventory', 'data': disk_data}
587         if self._buffer.buffer_mode == 'direct':
588             self._writer.batch_insert_records([rec])
589             return
590         self._buffer.enqueue(rec)
591
592     def save_volume_inventory(self, volume_data: Any) -> None:
593         """Сохранить или обновить паспорт логического тома."""
594         rec = {'type': 'volume_inventory', 'data': volume_data
        }
595         if self._buffer.buffer_mode == 'direct':
596             self._writer.batch_insert_records([rec])
597             return
598         self._buffer.enqueue(rec)
599
600     def save_disk_health_snapshot(self, health_data: Any) -> N

```

```

one:
601     """Сохранить снимок здоровья / SMART / TBW накопителя.
    """
602     rec = {'type': 'disk_health', 'data': health_data}
603     if self._buffer.buffer_mode == 'direct':
604         self._writer.batch_insert_records([rec])
605         return
606     self._buffer.enqueue(rec)
607
608     def save_disk_performance_sample(self, perf_data: Any) ->
None:
609         """Сохранить сэмпл производительности диска."""
610         rec = {'type': 'disk_performance', 'data': perf_data}
611         if self._buffer.buffer_mode == 'direct':
612             self._writer.batch_insert_records([rec])
613             return
614         self._buffer.enqueue(rec)
615
616     def save_disk_io_event(self, io_event_data: Any) -> None:
617         """Сохранить событие прямого ввода-вывода."""
618         rec = {'type': 'disk_io_event', 'data': io_event_data}
619         if self._buffer.buffer_mode == 'direct':
620             self._writer.batch_insert_records([rec])
621             return
622         self._buffer.enqueue(rec)
623
624     def get_disk_inventory(self, disk_id: Optional[int] = None
) -> List[Dict[str, Any]]:
625         """Извлечь паспортные данные физических дисков."""
626         return self._reader.get_disk_inventory(disk_id=disk_id
)
627
628     def get_volume_inventory(self) -> List[Dict[str, Any]]:
629         """Извлечь паспортные данные логических томов."""
630         return self._reader.get_volume_inventory()
631
632     def get_disk_health_history(
633         self,

```

```

634         disk_id: Optional[int] = None,
635         serial_number: Optional[str] = None,
636         limit: int = 100,
637         since_epoch: Optional[float] = None,
638     ) -> List[Dict[str, Any]]:
639         """Извлечь историю здоровья и SMART/TBW накопителей."""
640         return self._reader.get_disk_health_history(disk_id=di
sk_id, serial_number=serial_number, limit=limit, since_epoch=s
ince_epoch)
641
642     def get_disk_performance_samples(
643         self,
644         disk_name: Optional[str] = None,
645         limit: int = 100,
646         since_epoch: Optional[float] = None,
647     ) -> List[Dict[str, Any]]:
648         """Извлечь сэмплы производительности дисков."""
649         return self._reader.get_disk_performance_samples(disk_
name=disk_name, limit=limit, since_epoch=since_epoch)
650
651     def get_disk_io_events(
652         self,
653         pid: Optional[int] = None,
654         disk_id: Optional[int] = None,
655         limit: int = 100,
656         since_epoch: Optional[float] = None,
657     ) -> List[Dict[str, Any]]:
658         """Извлечь события ввода-вывода процессов."""
659         return self._reader.get_disk_io_events(pid=pid, disk_i
d=disk_id, limit=limit, since_epoch=since_epoch)
660
661     # -----
662     # Делегирование методов хранилища процессоров (CPU Subsys
tem)
663     # -----

```

```

664
665     def save_cpu_inventory(self, cpu_data: Any) -> None:
666         """Сохранить или обновить паспорт процессора."""
667         rec = {'type': 'cpu_inventory', 'data': cpu_data}
668         if self._buffer.buffer_mode == 'direct':
669             self._writer.batch_insert_records([rec])
670             return
671         self._buffer.enqueue(rec)
672
673     def save_cpu_telemetry_sample(self, sample_data: Any) -> N
one:
674         """Сохранить снимок телеметрии процессора."""
675         rec = {'type': 'cpu_telemetry', 'data': sample_data}
676         if self._buffer.buffer_mode == 'direct':
677             self._writer.batch_insert_records([rec])
678             return
679         self._buffer.enqueue(rec)
680
681     def get_cpu_inventory(self, processor_id: Optional[int] =
None) -> List[Dict[str, Any]]:
682         """Извлечь паспортные данные процессоров."""
683         return self._reader.get_cpu_inventory(processor_id=pro
cessor_id)
684
685     def get_cpu_telemetry_samples(
686         self,
687         processor_id: Optional[int] = None,
688         limit: int = 100,
689         since_epoch: Optional[float] = None,
690     ) -> List[Dict[str, Any]]:
691         """Извлечь историю метрик процессора."""
692         return self._reader.get_cpu_telemetry_samples(processo
r_id=processor_id, limit=limit, since_epoch=since_epoch)
693
694     # -----
-----
695     # Делегирование методов хранилища памяти (RAM Module / SPD
Subsystem)

```

```

696 |     # -----
697 |
698 |     def save_ram_module_inventory(self, module_data: Any) -> N
one:
699 |         """Сохранить или обновить паспорт планки оперативной п
амяти."""
700 |         rec = {'type': 'ram_module_inventory', 'data': module_
data}
701 |         if self._buffer.buffer_mode == 'direct':
702 |             self._writer.batch_insert_records([rec])
703 |             return
704 |         self._buffer.enqueue(rec)
705 |
706 |     def save_ram_telemetry_sample(self, sample_data: Any) -> N
one:
707 |         """Сохранить снимок телеметрии оперативной памяти."""
708 |         rec = {'type': 'ram_telemetry', 'data': sample_data}
709 |         if self._buffer.buffer_mode == 'direct':
710 |             self._writer.batch_insert_records([rec])
711 |             return
712 |         self._buffer.enqueue(rec)
713 |
714 |     def get_ram_module_inventory(self) -> List[Dict[str, Any]]
:
715 |         """Извлечь паспортные данные физических планок операти
вной памяти."""
716 |         return self._reader.get_ram_module_inventory()
717 |
718 |     def get_ram_telemetry_samples(
719 |         self,
720 |         limit: int = 100,
721 |         since_epoch: Optional[float] = None,
722 |     ) -> List[Dict[str, Any]]:
723 |         """Извлечь историю метрик оперативной памяти."""
724 |         return self._reader.get_ram_telemetry_samples(limit=li
mit, since_epoch=since_epoch)
725 |

```

```

726 |     # -----
727 |     # Делегирование методов хранилища графики (GPU Subsystem)
728 |     # -----
729 |
730 |     def save_gpu_inventory(self, gpu_data: Any) -> None:
731 |         """Сохранить или обновить паспорт графического ускорит
732 | еля."""
733 |         rec = {'type': 'gpu_inventory', 'data': gpu_data}
734 |         if self._buffer.buffer_mode == 'direct':
735 |             self._writer.batch_insert_records([rec])
736 |             return
737 |         self._buffer.enqueue(rec)
738 |
739 |     def save_gpu_telemetry_sample(self, sample_data: Any) -> N
740 | one:
741 |         """Сохранить снимок телеметрии графического ускорителя
742 | ."""
743 |         rec = {'type': 'gpu_telemetry', 'data': sample_data}
744 |         if self._buffer.buffer_mode == 'direct':
745 |             self._writer.batch_insert_records([rec])
746 |             return
747 |         self._buffer.enqueue(rec)
748 |
749 |     def get_gpu_inventory(self, gpu_id: Optional[int] = None)
750 | -> List[Dict[str, Any]]:
751 |         """Извлечь паспортные данные GPU."""
752 |         return self._reader.get_gpu_inventory(gpu_id=gpu_id)
753 |
754 |     def get_gpu_telemetry_samples(
755 | self,
756 | gpu_id: Optional[int] = None,
757 | name: Optional[str] = None,
758 | limit: int = 100,
759 | since_epoch: Optional[float] = None,
760 | ) -> List[Dict[str, Any]]:
761 |         """Извлечь историю метрик GPU."""

```

```

758         return self._reader.get_gpu_telemetry_samples(gpu_id=g
| pu_id, name=name, limit=limit, since_epoch=since_epoch)
759
760         # -----
| -----
761         # Делегирование методов хранилища сети (Network Adapter Su
| bsystem)
762         # -----
| -----
763
764         def save_network_adapter_inventory(self, adapter_data: Any
| ) -> None:
765             """Сохранить или обновить паспорт сетевого адаптера."""
| "
766             rec = {'type': 'network_adapter_inventory', 'data': ad
| apter_data}
767             if self._buffer.buffer_mode == 'direct':
768                 self._writer.batch_insert_records([rec])
769                 return
770             self._buffer.enqueue(rec)
771
772         def save_network_adapter_sample(self, sample_data: Any) ->
| None:
773             """Сохранить сэмпл показателей сетевого адаптера."""
774             rec = {'type': 'network_adapter_sample', 'data': sampl
| e_data}
775             if self._buffer.buffer_mode == 'direct':
776                 self._writer.batch_insert_records([rec])
777                 return
778             self._buffer.enqueue(rec)
779
780         def get_network_adapter_inventory(self) -> List[Dict[str,
| Any]]:
781             """Извлечь паспортные данные сетевых адаптеров."""
782             return self._reader.get_network_adapter_inventory()
783
784         def get_network_adapter_samples(
785             self,

```

```

786         adapter_name: Optional[str] = None,
787         limit: int = 100,
788         since_epoch: Optional[float] = None,
789     ) -> List[Dict[str, Any]]:
790         """Извлечь сэмплы показателей сетевых адаптеров."""
791         return self._reader.get_network_adapter_samples(adapter_name=adapter_name, limit=limit, since_epoch=since_epoch)
792
793
794     # -----
795     # Делегирование методов обслуживания (Maintenance)
796     # -----
797
798     def get_db_file_sizes(self) -> Dict[str, int]:
799         return self._maintenance.get_db_file_sizes()
800
801     def get_total_db_size_mb(self) -> float:
802         return self._maintenance.get_total_db_size_mb()
803
804     def checkpoint_wal(self) -> bool:
805         return self._maintenance.checkpoint_wal()
806
807     def vacuum(self) -> bool:
808         return self._maintenance.vacuum()
809
810     def get_storage_stats(self) -> Dict[str, Any]:
811         return self._maintenance.get_storage_stats(
812             buffered_count=self.get_buffered_count(),
813             buffer_mode=self.get_buffer_mode(),
814             buffer_size=self._buffer.buffer_size,
815             flush_interval_seconds=self._buffer.flush_interval
816         )
817
818     get_stats = get_storage_stats
819

```



```

820     def cleanup_old_records(self, retention_days: Optional[int
    ] = None, vacuum_after: bool = False) -> int:
821         return self._maintenance.cleanup_old_records(retention
    _days=retention_days, vacuum_after=vacuum_after)
822
823     def enforce_size_limit(self, max_size_mb: Optional[float]
    = None, target_ratio: float = 0.85) -> Dict[str, Any]:
824         return self._maintenance.enforce_size_limit(max_size_m
    b=max_size_mb, target_ratio=target_ratio)
825
826     def prune_to_size(self, target_size_mb: float) -> Dict[str
    , Any]:
827         return self.enforce_size_limit(max_size_mb=target_size
    _mb, target_ratio=0.95)
828
829     def aggregate_process_metrics_2min(self, cutoff_seconds: i
    nt = 120, outlier_cpu_threshold: float = 30.0) -> Dict[str, in
    t]:
830         return self._maintenance.aggregate_process_metrics_2mi
    n(cutoff_seconds=cutoff_seconds, outlier_cpu_threshold=outlier
    _cpu_threshold)
831
832     def aggregate_process_metrics_daily(self, cutoff_days: int
    = 1) -> Dict[str, int]:
833         return self._maintenance.aggregate_process_metrics_dai
    ly(cutoff_days=cutoff_days)
834
835     # -----
    -----
836     # Делегирование методов SQL-агрегации сенсоров (Aggregator
    )
837     # -----
    -----
838
839     def aggregate_sensors(self, level: Union[AggregationLevel,
    str], start_epoch: Optional[float] = None, end_epoch: Optiona
    l[float] = None, detect_spikes: bool = True) -> Dict[str, Any]
    :

```

```
840         return self._aggregator.aggregate(level=level, start_e
      | poch=start_epoch, end_epoch=end_epoch, detect_spikes=detect_sp
      | ikes)
841
842     def aggregate_hourly(self, start_epoch: Optional[float] =
      | None, end_epoch: Optional[float] = None, detect_spikes: bool =
      | True) -> Dict[str, Any]:
843         return self._aggregator.aggregate_hourly(start_epoch=s
      | tart_epoch, end_epoch=end_epoch, detect_spikes=detect_spikes)
844
845     def aggregate_daily(self, start_epoch: Optional[float] = N
      | one, end_epoch: Optional[float] = None) -> Dict[str, Any]:
846         return self._aggregator.aggregate_daily(start_epoch=st
      | art_epoch, end_epoch=end_epoch)
847
848     def aggregate_weekly(self, start_epoch: Optional[float] =
      | None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
849         return self._aggregator.aggregate_weekly(start_epoch=s
      | tart_epoch, end_epoch=end_epoch)
850
851     def aggregate_monthly(self, start_epoch: Optional[float] =
      | None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
852         return self._aggregator.aggregate_monthly(start_epoch=
      | start_epoch, end_epoch=end_epoch)
853
854     def aggregate_yearly(self, start_epoch: Optional[float] =
      | None, end_epoch: Optional[float] = None) -> Dict[str, Any]:
855         return self._aggregator.aggregate_yearly(start_epoch=s
      | tart_epoch, end_epoch=end_epoch)
856
857     def run_aggregation_pipeline(self, start_epoch: Optional[f
      | loat] = None, end_epoch: Optional[float] = None, detect_spikes
      | : bool = True) -> Dict[str, Any]:
858         return self._aggregator.run_pipeline(start_epoch=start
      | _epoch, end_epoch=end_epoch, detect_spikes=detect_spikes)
859
860     def start_background_aggregator(self, interval_seconds: fl
      | oat = 300.0) -> None:
```

```

861         """Запускает фоновый регламентный планировщик агрегаци
и."""
862         self._aggregator.start_background_scheduler(interval_s
econds=interval_seconds)
863
864     def stop_background_aggregator(self) -> None:
865         """Останавливает фоновый регламентный планировщик агре
гации."""
866         self._aggregator.stop_background_scheduler()
867
868         # -----
-----
869         # Делегирование методов подсистем рефакторинга: Data-First
SQLite
870         # -----
-----
871
872         # 1. Windows Event Logs
873         def save_event_log_snapshot(self, snapshot_id: str, channe
ls: List[Any], entries: List[Any], intelligence_profiles: Opti
onal[List[Any]] = None) -> int:
874             return self._writer.save_event_log_snapshot(snapshot_i
d, channels, entries, intelligence_profiles)
875
876         def get_latest_event_log_channels(self) -> List[Dict[str,
Any]]:
877             return self._reader.get_latest_event_log_channels()
878
879         def get_event_log_entries(self, channel: str = 'System', l
evel: str = '', limit: int = 50, hours: int = 24) -> List[Dict
[str, Any]]:
880             return self._reader.get_event_log_entries(channel=chan
nel, level=level, limit=limit, hours=hours)
881
882         def get_latest_event_log_intelligence_profile(self, channe
l: str = 'System') -> Optional[Dict[str, Any]]:
883             return self._reader.get_latest_event_log_intelligence_
profile(channel=channel)

```

```
884
885     def get_latest_event_log_report(self) -> Optional[Dict[str
    , Any]]:
886         return self._reader.get_latest_event_log_report()
887
888     # 2. Windows Firewall
889     def save_firewall_snapshot(self, snapshot_id: str, profile
    s: Any, rules: List[Any]) -> int:
890         return self._writer.save_firewall_snapshot(snapshot_id
    , profiles, rules)
891
892     def update_firewall_rule_state(self, rule_name: str, enabl
    ed: bool) -> bool:
893         return self._writer.update_firewall_rule_state(rule_na
    me, enabled)
894
895     def get_latest_firewall_profiles(self) -> Optional[Dict[st
    r, Any]]:
896         return self._reader.get_latest_firewall_profiles()
897
898     def get_latest_firewall_rules(self, direction: Optional[st
    r] = None, limit: int = 200) -> List[Dict[str, Any]]:
899         return self._reader.get_latest_firewall_rules(directio
    n=direction, limit=limit)
900
901     # 3. Windows Services
902     def save_services_snapshot(self, snapshot_id: str, service
    s: List[Any]) -> int:
903         return self._writer.save_services_snapshot(snapshot_id
    , services)
904
905     def record_service_change(self, service_name: str, display
    _name: str, action: str, old_state: Optional[str] = None, new_
    state: Optional[str] = None, performed_by: str = 'SYSTEM', det
    ails: Optional[Dict[str, Any]] = None) -> int:
906         return self._writer.record_service_change(service_name
    , display_name, action, old_state, new_state, performed_by, de
    tails)
```

```

907
908     def get_latest_services_list(self, status: Optional[str] =
909     None, limit: int = 500) -> List[Dict[str, Any]]:
910         return self._reader.get_latest_services_list(status=status,
911         limit=limit)
912
913     def get_latest_services_report(self) -> Optional[Dict[str,
914     Any]]:
915         return self._reader.get_latest_services_report()
916
917     def get_service_change_events(self, limit: int = 100) -> L
918     ist[Dict[str, Any]]:
919         return self._reader.get_service_change_events(limit=li
920         mit)
921
922     # 4. Throttling & Power
923     def save_throttling_snapshot(self, snapshot_id: str, throt
924     tling_data: Any, thermal_zones: Optional[List[Any]] = None) ->
925     int:
926         return self._writer.save_throttling_snapshot(snapshot_
927         id, throttling_data, thermal_zones)
928
929     def get_latest_throttling_snapshot(self) -> Optional[Dict[
930     str, Any]]:
931         return self._reader.get_latest_throttling_snapshot()
932
933     # 5. Forensics
934     def save_forensics_snapshot(self, snapshot_id: str, forens
935     ics_data: Any) -> int:
936         return self._writer.save_forensics_snapshot(snapshot_i
937         d, forensics_data)
938
939     def get_latest_forensics_snapshot(self) -> Optional[Dict[s
940     tr, Any]]:
941         return self._reader.get_latest_forensics_snapshot()
942
943     # 6. Process Leaks
944     def save_process_leak_snapshot(self, snapshot_id: str, tot

```

```

    al_processes: Any, suspicious_count: int = 0, leak_items: Optional[List[Any]] = None) -> int:
933         return self._writer.save_process_leak_snapshot(snapshot_id, total_processes, suspicious_count, leak_items)
934
935     def get_latest_process_leak_report(self, limit: int = 20) -> Optional[Dict[str, Any]]:
936         return self._reader.get_latest_process_leak_report(limit=limit)
937
938     # 7. Defender
939     def save_defender_snapshot(self, snapshot_id: str, defender_status: Any, exclusions: Optional[List[Any]] = None, asr_rules: Optional[List[Any]] = None, threats: Optional[List[Any]] = None) -> int:
940         return self._writer.save_defender_snapshot(snapshot_id, defender_status, exclusions, asr_rules, threats)
941
942     def get_latest_defender_status(self) -> Optional[Dict[str, Any]]:
943         return self._reader.get_latest_defender_status()
944
945     def get_latest_defender_exclusions(self) -> List[Dict[str, Any]]:
946         return self._reader.get_latest_defender_exclusions()
947
948     def get_latest_defender_asr_rules(self) -> List[Dict[str, Any]]:
949         return self._reader.get_latest_defender_asr_rules()
950
951     def get_latest_defender_threats(self) -> List[Dict[str, Any]]:
952         return self._reader.get_latest_defender_threats()
953
954     def get_latest_defender_diagnostics(self) -> Dict[str, Any]:
955         return self._reader.get_latest_defender_diagnostics()
956

```

```

957 |     # 8. Process Network & Process Manager
958 |     def save_process_network_snapshot(self, snapshot_id: str,
| network_items: List[Any]) -> int:
959 |         return self._writer.save_process_network_snapshot(snapshot_id, network_items)
960 |
961 |     def get_latest_process_network_activity(self, limit: int =
| 100, only_internet: bool = True) -> List[Dict[str, Any]]:
962 |         return self._reader.get_latest_process_network_activity(limit=limit, only_internet=only_internet)
963 |
964 |     # 9. Software Transparency
965 |     def save_software_inventory(self, apps: List[Any], storage
| _locations: Optional[List[Any]] = None, config_files: Optional
| [List[Any]] = None, ai_research: Optional[List[Any]] = None) -
| > int:
966 |         return self._writer.save_software_inventory(apps, storage_locations, config_files, ai_research)
967 |
968 |     def save_software_network_snapshots(self, snapshot_id: str
| , net_items: List[Any]) -> int:
969 |         return self._writer.save_software_network_snapshots(snapshot_id, net_items)
970 |
971 |     def get_software_inventory_from_db(self, limit: int = 200,
| offset: int = 0) -> List[Dict[str, Any]]:
972 |         return self._reader.get_software_inventory_from_db(limit=limit, offset=offset)
973 |
974 |     def get_software_app_details_from_db(self, app_id: str) ->
| Optional[Dict[str, Any]]:
975 |         return self._reader.get_software_app_details_from_db(app_id=app_id)
976 |
977 |
978 |

```

writer.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Sqlite -
  Writer
4 # =====
5 # Description:
6 #   Класс TelemetryWriter для сериализации и записи телеметрии
  в SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite.writer import Telemet
  ryWriter
11 #
12 #       writer = TelemetryWriter(connection_manager)
13 #       saved = writer.batch_insert_records(records)
14 #
15 # File: writer.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 17:30:00
21 # =====
22
23 from __future__ import annotations
24
25 """Класс TelemetryWriter для сериализации, подготовки строк и
  транзакционной записи в SQLite."""
26
27 import json
28 import sqlite3
29 import time
```



```

30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Dict, List, Optional, Union
33
34 from .connection import TelemetryConnectionManager
35 from ..models import (
36     HardwareArchiveEntry,
37     ProcessLifecycleEvent,
38     ProcessProvenanceInfo,
39     StartupArchiveEntry,
40     SystemMetricRollup,
41     SystemSnapshot,
42     TelemetryIncident,
43 )
44
45 try:
46     from logger import logger
47 except ImportError:
48     import logging
49     logger = logging.getLogger(__name__)
50
51
52 class TelemetryWriter:
53     """Выполняет пакетную и прямую вставку записей телеметрии
54     в базу данных SQLite."""
55
56     def __init__(self, connection_manager: TelemetryConnection
57     Manager) -> None:
58         """Инициализирует писатель телеметрии.
59
60         Args:
61             connection_manager: Менеджер соединений SQLite.
62
63         """
64         self._cm = connection_manager
65
66     def batch_insert_records(self, records: List[Dict[str, Any
67     ]]) -> int:
68         """Выполняет пакетную вставку разнородных записей в SQ

```

Lite в единой транзакции.

```
65
66     Args:
67         records: Список словарей записей.
68
69     Returns:
70         int: Общее число сохраненных записей.
71     """
72     if not records or self._cm.read_only:
73         return 0
74
75     saved_total = 0
76     with self._cm.lock, self._cm.get_connection() as conn:
77         cursor = conn.cursor()
78         sensor_poll_rows: List[tuple] = []
79         sensor_agg_rows: List[tuple] = []
80
81         for item in records:
82             rec_type = item.get('type')
83
84             if rec_type == 'snapshot':
85                 self.insert_snapshot_row(cursor, item)
86                 saved_total += 1
87
88             elif rec_type == 'sensor_poll':
89                 sensor_poll_rows.append(self.prepare_sensor_poll_row(item.get('data', {}), item.get('timestamp')))
90
91             elif rec_type == 'sensor_polls_batch':
92                 ts = item.get('timestamp')
93                 for s in item.get('data', []):
94                     sensor_poll_rows.append(self.prepare_sensor_poll_row(s, ts))
95
96             elif rec_type == 'sensor_aggregates_batch':
97                 for a in item.get('data', []):
98                     sensor_agg_rows.append((
99                         a.get('sensor_id', ''),
```

```

100         a.get('period_start', ''),
101         a.get('period_end', ''),
102         a.get('avg', None),
103         a.get('min', None),
104         a.get('max', None),
105         a.get('count', 0),
106         json.dumps(a, ensure_ascii=False,
default=str),
107     ))
108
109     elif rec_type == 'hardware_archive':
110         self.insert_hardware_archive_row(cursor, item.get('data', {}))
111         saved_total += 1
112
113     elif rec_type == 'startup_archive':
114         self.insert_startup_archive_row(cursor, item.get('data', {}))
115         saved_total += 1
116
117     elif rec_type == 'event':
118         ts_str = item.get('timestamp') or datetime
.now(timezone.utc).isoformat()
119         try:
120             now_epoch = datetime.fromisoformat(ts_str).timestamp()
121         except Exception:
122             now_epoch = datetime.now(timezone.utc).timestamp()
123         details_str = json.dumps(item.get('details', {}), ensure_ascii=False, default=str)
124         cursor.execute('''
125             INSERT INTO telemetry_events (
126                 timestamp, created_at, event_type,
severity, event_details, raw_json
127             ) VALUES (?, ?, ?, ?, ?, ?)
128             ''', (ts_str, now_epoch, item.get('event_type', ''), item.get('severity', 'info'), details_str, details_

```

```
str))
129         saved_total += 1
130
131         elif rec_type == 'disk_inventory':
132             self.upsert_disk_inventory(cursor, item.get(
133                 'data') or item)
134             saved_total += 1
135
136         elif rec_type == 'volume_inventory':
137             self.upsert_volume_inventory(cursor, item.
138                 get('data') or item)
139             saved_total += 1
140
141         elif rec_type == 'disk_health':
142             self.insert_disk_health_snapshot(cursor, i
143                 tem.get('data') or item)
144             saved_total += 1
145
146         elif rec_type == 'disk_performance':
147             self.insert_disk_performance_sample(cursor
148                 , item.get('data') or item)
149             saved_total += 1
150
151         elif rec_type == 'disk_io_event':
152             self.insert_disk_io_event(cursor, item.get
153                 ('data') or item)
154             saved_total += 1
155
156         elif rec_type in ('cpu_inventory', 'cpu_inv'):
157             self.upsert_cpu_inventory(cursor, item.get
158                 ('data') or item)
159             saved_total += 1
160
161         elif rec_type in ('ram_module_inventory', 'ram
162             _inventory', 'ram_module'):
163             self.upsert_ram_module_inventory(cursor, i
164                 tem.get('data') or item)
165             saved_total += 1
```

```
158|
159|         elif rec_type in ('gpu_inventory', 'gpu_inv'):
160|             self.upsert_gpu_inventory(cursor, item.get
161| ('data') or item)
162|             saved_total += 1
163|
164|         elif rec_type in ('network_adapter_inventory',
165| 'net_inventory', 'net_adapter_inv'):
166|             self.upsert_network_adapter_inventory(curs
167| or, item.get('data') or item)
168|             saved_total += 1
169|
170|         elif rec_type in ('cpu_telemetry', 'cpu_sample
171| '):
172|             self.insert_cpu_telemetry_sample(cursor, i
173| tem.get('data') or item)
174|             saved_total += 1
175|
176|         elif rec_type in ('ram_telemetry', 'ram_sample
177| '):
178|             self.insert_ram_telemetry_sample(cursor, i
179| tem.get('data') or item)
180|             saved_total += 1
181|
182|         elif rec_type in ('gpu_telemetry', 'gpu_sample
183| '):
184|             self.insert_gpu_telemetry_sample(cursor, i
185| tem.get('data') or item)
186|             saved_total += 1
187|
188|         elif rec_type in ('network_adapter_sample', 'n
189| etwork_sample', 'net_sample'):
190|             self.insert_network_adapter_sample(cursor,
191| item.get('data') or item)
192|             saved_total += 1
193|
194|         elif rec_type == 'app_poll':
195|             cursor.execute(''
```

```

185         INSERT INTO app_polls (
186             timestamp, created_at, app, poll_t
ype, metric_name, value, unit, status, details, raw_json
187         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
188         '', self.prepare_app_poll_row(item.get('d
ata') or item))
189         saved_total += 1
190
191         elif rec_type == 'app_polls_batch':
192             rows = [self.prepare_app_poll_row(p) for p
in item.get('data', [])]
193             if rows:
194                 cursor.executemany(''
195                     INSERT INTO app_polls (
196                         timestamp, created_at, app, po
ll_type, metric_name, value, unit, status, details, raw_json
197                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?)
198                 '', rows)
199                 saved_total += len(rows)
200
201         elif rec_type == 'app_event':
202             cursor.execute(''
203                 INSERT INTO app_events (
204                     timestamp, created_at, app, event_
type, status, details, raw_json
205                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
206             '', self.prepare_app_event_row(item.get('
data') or item))
207             saved_total += 1
208
209         elif rec_type == 'app_events_batch':
210             rows = [self.prepare_app_event_row(ev) for
ev in item.get('data', [])]
211             if rows:
212                 cursor.executemany(''
213                     INSERT INTO app_events (

```

```

214         timestamp, created_at, app, ev
ent_type, status, details, raw_json
215         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
216         '', rows)
217         saved_total += len(rows)
218
219         elif rec_type == 'app_param_change':
220             cursor.execute(''
221                 INSERT INTO app_param_changes (
222                     timestamp, created_at, app, param_
name, old_value, new_value, status, user, details, raw_json
223                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
224             '', self.prepare_app_param_row(item.get('
data') or item))
225             saved_total += 1
226
227         elif rec_type == 'app_param_changes_batch':
228             rows = [self.prepare_app_param_row(pc) for
pc in item.get('data', [])]
229             if rows:
230                 cursor.executemany(''
231                     INSERT INTO app_param_changes (
232                         timestamp, created_at, app, pa
ram_name, old_value, new_value, status, user, details, raw_jso
n
233                         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?)
234                     '', rows)
235             saved_total += len(rows)
236
237         elif rec_type == 'custom_record':
238             ts_str = item.get('timestamp') or datetime
.now(timezone.utc).isoformat()
239             try:
240                 now_epoch = datetime.fromisoformat(ts_
str).timestamp()
241             except Exception:

```

```

242         now_epoch = datetime.now(timezone.utc)
243         .timestamp()
244         payload_str = json.dumps(item.get('payload
245         ', {}), ensure_ascii=False, default=str)
246         cursor.execute('''
247             INSERT INTO custom_records (timestamp,
248             created_at, source_file, payload_json)
249             VALUES (?, ?, ?, ?)
250             ''', (ts_str, now_epoch, item.get('source_
251             file', ''), payload_str))
252             saved_total += 1
253
254         elif rec_type == 'device_event':
255             cursor.execute('''
256                 INSERT INTO device_events (
257                     timestamp, created_at, event_type,
258                     device_instance_id,
259                     friendly_name, device_class, categ
260                     ory, has_problem,
261                     problem_code, status_code, manufac
262                     turer,
263                     flapping_count, uptime_seconds, ra
264                     w_json
265                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
266                     , ?, ?, ?, ?)
267             ''', self.prepare_device_event_row(item.ge
268             t('data', {}), item.get('timestamp')))
269             saved_total += 1
270
271         elif rec_type == 'w64_event':
272             cursor.execute('''
273                 INSERT INTO w64_events (
274                     event_id, timestamp, created_at, e
275                     vent_type, path, pid, name, provider, raw_json
276                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
277             ''', self.prepare_w64_event_row(item.get('
278             data', {}), item.get('provider', 'w64_collector')))
279             saved_total += 1

```



```

268 |
269 |             elif rec_type == 'process_provenance_event':
270 |                 cursor.execute('''
271 |                     INSERT INTO process_provenance_events
272 |     (
273 |         event_id, timestamp, created_at, e
vent_type, process_guid, parent_guid,
274 |         pid, ppid, name, executable_path,
command_line, user, sid, session_id,
275 |         integrity_level, elevation, parent_name, parent_cmdline,
ancestor_chain, launch_reason, sou
rce, details_json, raw_json
276 |         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
277 |         ''', self.prepare_process_provenance_row(i
tem.get('data', {})))
278 |         saved_total += 1
279 |
280 |             elif rec_type == 'process_provenance_events_ba
tch':
281 |                 rows = [self.prepare_process_provenance_ro
w(ev) for ev in item.get('data', [])]
282 |                 if rows:
283 |                     cursor.executemany('''
284 |                         INSERT INTO process_provenance_eve
nts (
285 |                             event_id, timestamp, created_a
t, event_type, process_guid, parent_guid,
286 |                             pid, ppid, name, executable_pa
th, command_line, user, sid, session_id,
287 |                             integrity_level, elevation, pa
rent_name, parent_cmdline,
288 |                             ancestor_chain, launch_reason,
source, details_json, raw_json
289 |                             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
290 |                             ''', rows)
```

```

291         saved_total += len(rows)
292
293         elif rec_type == 'incident':
294             inc_data = item.get('data', {})
295             now_dt = datetime.now(timezone.utc)
296             ts_str = inc_data.get('timestamp') or item
297             .get('timestamp') or now_dt.isoformat()
298             try:
299                 now_epoch = datetime.fromisoformat(ts_
300 str).timestamp()
301             except Exception:
302                 now_epoch = now_dt.timestamp()
303                 inc_id = inc_data.get('incident_id') or f"
304 INC-{int(now_epoch)}"
305                 cursor.execute('''
306                     INSERT OR REPLACE INTO incidents (
307                         incident_id, timestamp, created_at
308 , trigger_type, severity, title,
309 description, trigger_metrics_json,
310 suspect_processes_json,
311 related_events_json, metrics_summa
312 ry_json, raw_window_json
313 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
314 , ?, ?)
315 ''', (
316     inc_id, ts_str, now_epoch,
317     inc_data.get('trigger_type', ''),
318     inc_data.get('severity', 'warning'),
319     inc_data.get('title', ''),
320     inc_data.get('description', ''),
321     json.dumps(inc_data.get('trigger_metri
322 cs', {}), ensure_ascii=False, default=str),
323     json.dumps(inc_data.get('suspect_proce
324 sses', []), ensure_ascii=False, default=str),
325     json.dumps(inc_data.get('related_event
326 s', []), ensure_ascii=False, default=str),
327     json.dumps(inc_data.get('metrics_summa
328 ry', {}), ensure_ascii=False, default=str),

```

```

318         json.dumps(inc_data.get('raw_window',
    | [], ensure_ascii=False, default=str),
319         ))
320         saved_total += 1
321
322         elif rec_type == 'system_rollup':
323             r_data = item.get('data', {})
324             p_start = r_data.get('period_start', '')
325             p_end = r_data.get('period_end', '')
326             try:
327                 p_start_epoch = datetime.fromisoformat
    | (p_start).timestamp()
328             except Exception:
329                 p_start_epoch = 0.0
330             try:
331                 p_end_epoch = datetime.fromisoformat(p
    | _end).timestamp()
332             except Exception:
333                 p_end_epoch = 0.0
334             cursor.execute('''
335                 INSERT INTO telemetry_rollups (
336                     period_start, period_end, period_s
    | tart_epoch, period_end_epoch,
337                     duration_seconds, sample_count, ti
    | er,
338                     cpu_avg, cpu_min, cpu_max, cpu_p95
    | ,
339                     ram_avg_gb, ram_min_gb, ram_max_gb
    | , ram_percent_avg, ram_percent_max,
340                     disk_read_avg_mbs, disk_read_max_m
    | bs, disk_read_total_mb,
341                     disk_write_avg_mbs, disk_write_max
    | _mbs, disk_write_total_mb,
342                     network_rx_avg_mbs, network_rx_max
    | _mbs, network_rx_total_mb,
343                     network_tx_avg_mbs, network_tx_max
    | _mbs, network_tx_total_mb,
344                     raw_json

```

```

345 |             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
346 |             , ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
347 |             ''' , (
348 |                 p_start, p_end, p_start_epoch, p_end_e
349 | poch,
350 |                 float(r_data.get('duration_seconds', 0
351 | .0) or 0.0),
352 |                 int(r_data.get('sample_count', 0) or 0
353 | ),
354 |                 str(r_data.get('tier', '1m')),
355 |                 r_data.get('cpu_avg'), r_data.get('cpu
356 | _min'), r_data.get('cpu_max'), r_data.get('cpu_p95'),
357 |                 r_data.get('ram_avg_gb'), r_data.get('
358 | ram_min_gb'), r_data.get('ram_max_gb'),
359 |                 r_data.get('ram_percent_avg'), r_data.
360 | get('ram_percent_max'),
361 |                 r_data.get('disk_read_avg_mbs'), r_dat
362 | a.get('disk_read_max_mbs'), r_data.get('disk_read_total_mb'),
363 |                 r_data.get('disk_write_avg_mbs'), r_da
364 | ta.get('disk_write_max_mbs'), r_data.get('disk_write_total_mb'
365 | ),
366 |                 r_data.get('network_rx_avg_mbs'), r_da
367 | ta.get('network_rx_max_mbs'), r_data.get('network_rx_total_mb'
368 | ),
369 |                 r_data.get('network_tx_avg_mbs'), r_da
370 | ta.get('network_tx_max_mbs'), r_data.get('network_tx_total_mb'
371 | ),
372 |                 json.dumps(r_data, ensure_ascii=False,
373 | default=str),
374 |             ))
375 |         saved_total += 1
376 |
377 |     elif rec_type == 'reboot_session':
378 |         s_data = item.get('data', {})
379 |         now_dt = datetime.now(timezone.utc)
380 |         boot_time = s_data.get('boot_time') or now
381 | _dt.isoformat()
382 |
383 |     try:

```

```

367         now_epoch = datetime.fromisoformat(boot_time).timestamp()
368     except Exception:
369         now_epoch = now_dt.timestamp()
370     cursor.execute('''
371         INSERT OR REPLACE INTO reboot_history
372         (
373             boot_id, boot_time, previous_boot_time, uptime_seconds, uptime_human,
374             shutdown_type, likely_class, conclusion, initiating_process, initiating_user,
375             shutdown_action, reason_text, reason_code, bugcheck_code, bugcheck_params_json,
376             power_button_timestamp, windows_update_kb, service_installed, evidence_json,
377             events_chain_json, raw_json, created_at
378             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
379             , ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
380     ''', (
381         s_data.get('boot_id') or f"boot_{int(now_epoch)}",
382         boot_time,
383         s_data.get('previous_boot_time'),
384         float(s_data.get('uptime_seconds', 0.0) or 0.0),
385         s_data.get('uptime_human'),
386         s_data.get('shutdown_type', 'unknown')
387         ,
388         s_data.get('likely_class'),
389         s_data.get('conclusion'),
390         s_data.get('initiating_process'),
391         s_data.get('initiating_user'),
392         s_data.get('shutdown_action'),
393         s_data.get('reason_text'),
394         s_data.get('reason_code'),
395         s_data.get('bugcheck_code'),
396         json.dumps(s_data.get('bugcheck_params

```

```

    ', {}), ensure_ascii=False, default=str),
394         s_data.get('power_button_timestamp'),
395         s_data.get('windows_update_kb'),
396         s_data.get('service_installed'),
397         json.dumps(s_data.get('evidence', {})),
    ensure_ascii=False, default=str),
398         json.dumps(s_data.get('events_chain',
    []), ensure_ascii=False, default=str),
399         json.dumps(s_data, ensure_ascii=False,
    default=str),
400         now_epoch,
401     ))
402     saved_total += 1
403
404     elif rec_type == 'extended_audit':
405         audit_data = item.get('data', {})
406         now_dt = datetime.now(timezone.utc)
407         ts_str = item.get('timestamp') or audit_data.get('timestamp') or now_dt.isoformat()
408         try:
409             now_epoch = datetime.fromisoformat(ts_str).timestamp()
410         except Exception:
411             now_epoch = now_dt.timestamp()
412
413     def _get_f(obj: Any, key: str, default: Any = None) -> Any:
414         if isinstance(obj, dict):
415             return obj.get(key, default)
416         return getattr(obj, key, default)
417
418     def_obj = _get_f(audit_data, 'defender', {})
419     startup_obj = _get_f(audit_data, 'startup', {})
420     vss_obj = _get_f(audit_data, 'vss', {})
421     users_obj = _get_f(audit_data, 'users', {})
    )

```

```

422
423         cursor.execute('''
424             INSERT INTO system_extended_audits (
425                 timestamp, created_at, hostname,
426                 defender_cfa_enabled, defender_asr
427                 _count, defender_exclusions_count, defender_threats_count,
428                 startup_entries_count, vss_snapsho
429                 ts_count, users_total_count, users_admin_count,
430                 raw_json
431                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
432                 , ?, ?)
433             ''', (
434                 ts_str, now_epoch, _get_f(audit_data,
435                 'hostname', ''),
436                 int(bool(_get_f(def_obj, 'cfa_enabled'
437                 , False))),
438                 int(_get_f(def_obj, 'asr_rules_count',
439                 0) or 0),
440                 int(_get_f(def_obj, 'exclusions_count'
441                 , len(_get_f(def_obj, 'path_exclusions', []) or [])) or 0),
442                 int(_get_f(def_obj, 'active_threats_co
443                 unt', 0) or 0),
444                 int(_get_f(startup_obj, 'total_entries
445                 ', len(_get_f(startup_obj, 'entries', []) or [])) or 0),
446                 int(_get_f(vss_obj, 'total_snapshots_c
447                 ount', len(_get_f(vss_obj, 'snapshots', []) or [])) or 0),
448                 int(_get_f(users_obj, 'total_users_cou
449                 nt', len(_get_f(users_obj, 'users', []) or [])) or 0),
450                 int(len(_get_f(users_obj, 'admin_usern
451                 ames', []) or []) or _get_f(users_obj, 'users_admin_count', 0)
452                 or 0),
453                 json.dumps(audit_data, ensure_ascii=Fa
454                 lse, default=str),
455                 ))
456         saved_total += 1
457
458     if sensor_poll_rows:
459         cursor.executemany('''

```

```

446         INSERT INTO sensor_polls (
447             sensor_id, timestamp, created_at, hard
ware_name, hardware_type,
448             sensor_category, sensor_name, unit, va
lue, provider, provider_priority, raw_json
449         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?)
450         ON CONFLICT(sensor_id, timestamp) DO UPDAT
E SET
451             created_at = excluded.created_at,
452             hardware_name = excluded.hardware_name
,
453             hardware_type = excluded.hardware_type
,
454             sensor_category = excluded.sensor_cate
gory,
455             sensor_name = excluded.sensor_name,
456             unit = excluded.unit,
457             value = excluded.value,
458             provider = excluded.provider,
459             provider_priority = excluded.provider_
priority,
460             raw_json = excluded.raw_json
''' , sensor_poll_rows)
461         saved_total += len(sensor_poll_rows)
462
463
464         if sensor_agg_rows:
465             cursor.executemany(''
466                 INSERT INTO sensor_aggregates_hourly (
467                     sensor_id, period_start, period_end, a
vg_value, min_value, max_value, count, raw_json
468                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
469             ''' , sensor_agg_rows)
470             saved_total += len(sensor_agg_rows)
471
472         conn.commit()
473     return saved_total
474

```



```

475     def prepare_sensor_poll_row(self, s: Dict[str, Any], times
|tamp: Optional[str] = None) -> tuple:
476         """Подготавливает кортеж для вставки в sensor_polls."""
|
477         now_dt = datetime.now(timezone.utc)
478         ts_str = timestamp or s.get('timestamp') or now_dt.iso
|format()
479         try:
480             now_epoch = datetime.fromisoformat(ts_str).timesta
|mp()
481         except Exception:
482             now_epoch = now_dt.timestamp()
483             sid = str(s.get('id') or s.get('sensor_id') or '')
484             s_name = s.get('sensor_name') or s.get('name') or (f'S
|ensor #{sid}' if sid else 'Sensor')
485             cat = s.get('sensor_category') or s.get('category') or
|'General'
486
487             # Определение типа оборудования с fallback
488             hw_type = s.get('hardware_type')
489             if not hw_type:
490                 sid_l = sid.lower()
491                 cat_l = cat.lower()
492                 if 'disk' in sid_l or 'storage' in sid_l or 'stora
|ge' in cat_l:
493                     hw_type = 'storage'
494                 elif 'gpu' in sid_l or 'vram' in sid_l or 'graphic
|s' in sid_l:
495                     hw_type = 'gpu'
496                 elif 'ram' in sid_l or 'memory' in sid_l:
497                     hw_type = 'memory'
498                 elif 'net' in sid_l or 'nic' in sid_l or 'network'
|in cat_l:
499                     hw_type = 'network'
500                 else:
501                     hw_type = 'cpu'
502
503             hw_name = s.get('hardware_name') or ('Storage' if hw_t

```

```

    type == 'storage' else 'System')
504     unit = s.get('unit', '')
505     raw_val = s.get('value', s.get('value_num', 0.0))
506     try:
507         val_float = round(float(raw_val), 2)
508     except (ValueError, TypeError):
509         val_float = 0.0
510
511     provider_raw = s.get('_provider') or s.get('provider')
    or 'CIM_SYSTEM'
512     if hasattr(provider_raw, 'name'):
513         provider_str = provider_raw.name
514         provider_priority = provider_raw.value
515     elif isinstance(provider_raw, str):
516         provider_str = provider_raw.upper()
517         try:
518             from ..sensor_registry import SensorProvider
519             provider_priority = SensorProvider[provider_ra
w.upper()].value
520         except (KeyError, ImportError):
521             provider_priority = 30
522     else:
523         provider_str = 'CIM_SYSTEM'
524         provider_priority = 30
525
526     raw_json_str = s.get('raw_json')
527     if not raw_json_str:
528         try:
529             raw_json_str = json.dumps(s, ensure_ascii=False
e, default=str)
530         except Exception:
531             raw_json_str = None
532
533     return (sid, ts_str, now_epoch, hw_name, hw_type, cat,
s_name, unit, val_float, provider_str, provider_priority, raw
_json_str)
534
535     def prepare_app_poll_row(self, p: Dict[str, Any]) -> tuple

```

```

:
536     now_dt = datetime.now(timezone.utc)
537     ts_str = p.get('timestamp') or now_dt.isoformat()
538     try:
539         now_epoch = datetime.fromisoformat(ts_str).timestamp()
540     except Exception:
541         now_epoch = now_dt.timestamp()
542     app = p.get('app', 'unknown')
543     poll_type = p.get('poll_type', 'poll')
544     metric_name = p.get('metric_name', 'metric')
545     val = p.get('value')
546     try:
547         val_num = float(val) if val is not None and not isinstance(val, (dict, list)) else None
548     except (ValueError, TypeError):
549         val_num = None
550     unit = p.get('unit', '')
551     status = p.get('status', 'OK')
552     details = p.get('details', '')
553     details_str = json.dumps(details, ensure_ascii=False)
554     if isinstance(details, (dict, list)) else str(details or '')
555     raw_json = json.dumps({'app': app, 'poll_type': poll_type, 'metric_name': metric_name, 'value': val, 'unit': unit, 'status': status, 'details': details}, ensure_ascii=False)
556     return (ts_str, now_epoch, app, poll_type, metric_name, val_num, unit, status, details_str, raw_json)
557
558 def prepare_app_event_row(self, ev: Dict[str, Any]) -> tuple:
559     now_dt = datetime.now(timezone.utc)
560     ts_str = ev.get('timestamp') or now_dt.isoformat()
561     try:
562         now_epoch = datetime.fromisoformat(ts_str).timestamp()
563     except Exception:
564         now_epoch = now_dt.timestamp()
565     app = ev.get('app', 'unknown')

```

```

565         event_type = ev.get('event_type', 'event')
566         status = ev.get('status', 'OK')
567         details = ev.get('details', '')
568         details_str = json.dumps(details, ensure_ascii=False)
    if isinstance(details, (dict, list)) else str(details or '')
569         raw_json = json.dumps({'app': app, 'event_type': event
    _type, 'status': status, 'details': details}, ensure_ascii=False)
570         return (ts_str, now_epoch, app, event_type, status, de
    tails_str, raw_json)
571
572     def prepare_app_param_row(self, pc: Dict[str, Any]) -> tuple:
573         now_dt = datetime.now(timezone.utc)
574         ts_str = pc.get('timestamp') or now_dt.isoformat()
575         try:
576             now_epoch = datetime.fromisoformat(ts_str).timestamp()
577         except Exception:
578             now_epoch = now_dt.timestamp()
579         app = pc.get('app', 'unknown')
580         param_name = pc.get('param_name', 'param')
581         old_val = pc.get('old_value')
582         new_val = pc.get('new_value')
583         old_str = json.dumps(old_val, ensure_ascii=False) if i
    sinstance(old_val, (dict, list)) else str(old_val or '')
584         new_str = json.dumps(new_val, ensure_ascii=False) if i
    sinstance(new_val, (dict, list)) else str(new_val or '')
585         status = pc.get('status', 'SUCCESS')
586         user = pc.get('user', 'system')
587         details = pc.get('details', '')
588         details_str = json.dumps(details, ensure_ascii=False)
    if isinstance(details, (dict, list)) else str(details or '')
589         raw_json = json.dumps({'app': app, 'param_name': param
    _name, 'old_value': old_val, 'new_value': new_val, 'status': s
    tatus, 'user': user, 'details': details}, ensure_ascii=False)
590         return (ts_str, now_epoch, app, param_name, old_str, n
    ew_str, status, user, details_str, raw_json)

```

```

591
592     def prepare_device_event_row(self, event: Dict[str, Any],
timestamp: Optional[str] = None) -> tuple:
593         now_dt = datetime.now(timezone.utc)
594         ts_str = timestamp or event.get('timestamp') or now_dt
.isoformat()
595         try:
596             now_epoch = datetime.fromisoformat(ts_str).timesta
mp()
597         except Exception:
598             now_epoch = now_dt.timestamp()
599         raw_json = json.dumps(event, ensure_ascii=False, defau
lt=str)
600         return (
601             ts_str, now_epoch, event.get('event_type', ''), ev
ent.get('device_instance_id', ''),
602             event.get('friendly_name', ''), event.get('device_
class', ''), event.get('category', ''),
603             int(bool(event.get('has_problem', False))), event.
get('problem_code', 0),
604             event.get('status_code', 0), event.get('manufactur
er', ''),
605             event.get('flapping_count_in_window', 0), event.ge
t('uptime_seconds', 0.0), raw_json,
606         )
607
608     def prepare_w64_event_row(self, event_data: Dict[str, Any]
, provider: str = 'w64_collector') -> tuple:
609         now_dt = datetime.now(timezone.utc)
610         ts_str = event_data.get('timestamp') or now_dt.isoform
at()
611         try:
612             now_epoch = datetime.fromisoformat(ts_str).timesta
mp()
613         except Exception:
614             now_epoch = now_dt.timestamp()
615         event_id = str(event_data.get('event_id') or f'evt_{in
t(now_epoch)})')

```

```

616         event_type = str(event_data.get('event_type', 'unknown
        '))
617         path_val = event_data.get('path')
618         pid_val = event_data.get('pid')
619         if pid_val is not None:
620             try:
621                 pid_val = int(pid_val)
622             except (ValueError, TypeError):
623                 pid_val = None
624         name_val = event_data.get('name')
625         raw_json = json.dumps(event_data, ensure_ascii=False,
        default=str)
626         return (event_id, ts_str, now_epoch, event_type, path_
        val, pid_val, name_val, provider, raw_json)
627
628     def prepare_process_provenance_row(self, p: Any) -> tuple:
629         """Подготавливает кортеж для вставки в process_provena
        nce_events."""
630         p_dict = p if isinstance(p, dict) else (p.model_dump()
        if hasattr(p, 'model_dump') else (vars(p) if hasattr(p, '__di
        ct__') else p))
631         now_dt = datetime.now(timezone.utc)
632         ts_str = p_dict.get('timestamp') or now_dt.isoformat()
633         try:
634             now_epoch = datetime.fromisoformat(ts_str).timesta
        mp()
635         except Exception:
636             now_epoch = now_dt.timestamp()
637
638         pid = int(p_dict.get('pid') or 0)
639         event_id = p_dict.get('event_id') or f"pp_{int(now_epo
        ch * 1000)}_{pid}"
640         event_type = p_dict.get('event_type') or 'ProcessCreat
        ed'
641         ppid = int(p_dict['ppid']) if p_dict.get('ppid') is no
        t None else None
642         name = p_dict.get('name') or 'unknown'
643         exe_path = p_dict.get('executable_path') or p_dict.get

```

```

        ('executable')
644         cmdline = p_dict.get('command_line') or p_dict.get('cm
        dline')
645         user = p_dict.get('user') or p_dict.get('username')
646         sid = p_dict.get('sid')
647         session_id = int(p_dict['session_id']) if p_dict.get('
        session_id') is not None else None
648         integrity = p_dict.get('integrity_level')
649         elevation = int(bool(p_dict.get('elevation', False)))
650         parent_name = p_dict.get('parent_name')
651         parent_cmdline = p_dict.get('parent_cmdline')
652         ancestor_chain = p_dict.get('ancestor_chain') or p_dic
        t.get('ancestor_chain_str')
653         if isinstance(ancestor_chain, list):
654             ancestor_chain = ' -> '.join(ancestor_chain)
655         launch_reason = p_dict.get('launch_reason') or p_dict.
        get('why_was_it_created')
656         source = p_dict.get('source') or 'system'
657         creation_time = p_dict.get('creation_time') or ts_str
658         process_guid = p_dict.get('process_guid') or f"proc_{p
        id}_{creation_time}"
659         parent_guid = p_dict.get('parent_guid')
660         details = p_dict.get('details') or p_dict.get('details
        _json')
661         details_json = json.dumps(details, ensure_ascii=False)
        if isinstance(details, (dict, list)) else (str(details) if de
        tails else None)
662         raw_json = json.dumps(p_dict, ensure_ascii=False, defa
        ult=str)
663
664         return (
665             event_id, ts_str, now_epoch, event_type, process_g
        uid, parent_guid,
666             pid, ppid, name, exe_path, cmdline, user, sid, ses
        sion_id,
667             integrity, elevation, parent_name, parent_cmdline,
668             ancestor_chain, launch_reason, source, details_jso
        n, raw_json

```

```

669         )
670
671     def insert_snapshot_row(self, cursor: sqlite3.Cursor, item
: Dict[str, Any]) -> int:
672         """Вставляет строку снимка и процессы в базу данных со
всеми расширенными параметрами."""
673         snap_data = item.get('data', {})
674         top_n = item.get('top_n', 20)
675         snapshot = snap_data if isinstance(snap_data, SystemSn
apshot) else None
676
677         if snapshot:
678             hostname = snapshot.hostname
679             username = snapshot.username or ''
680             uptime = snapshot.uptime_seconds
681             os_name = snapshot.os_name or 'Windows'
682             os_build = str(snapshot.os_build or '')
683             os_install_date = str(snapshot.os_install_date or
'')
684             system_lang = str(snapshot.system_language or '')
685             os_install_lang = str(snapshot.os_install_language
or '')
686             user_locale = str(snapshot.user_locale or '')
687             sys_locale = str(snapshot.system_locale or '')
688             timezone_val = str(snapshot.timezone or '')
689             codepage_val = str(snapshot.codepage or '')
690             input_langs = snapshot.input_languages or []
691             input_langs_json = json.dumps(input_langs, ensure_
ascii=False) if input_langs else None
692
693             cpu_pct = snapshot.cpu.total_percent if snapshot.c
pu else 0.0
694             cpu_freq = snapshot.cpu.frequency_mhz if snapshot.
cpu else 0.0
695             mem_tot = snapshot.memory.total_gb if snapshot.mem
ory else 0.0
696             mem_used = snapshot.memory.used_gb if snapshot.mem
ory else 0.0

```



```

697         mem_pct = snapshot.memory.percent if snapshot.memo
ry else 0.0
698         swap_pct = snapshot.memory.swap_percent if snapsho
t.memory else 0.0
699         disk_rb = snapshot.disk_io.read_bytes_per_sec if s
napshot.disk_io else 0.0
700         disk_wb = snapshot.disk_io.write_bytes_per_sec if
snapshot.disk_io else 0.0
701         disk_rc = snapshot.disk_io.read_count_per_sec if s
napshot.disk_io else 0.0
702         disk_wc = snapshot.disk_io.write_count_per_sec if
snapshot.disk_io else 0.0
703         net_sent = sum(getattr(i, 'bytes_sent_sec', 0.0) o
r 0.0 for i in (snapshot.network or []))
704         net_recv = sum(getattr(i, 'bytes_recv_sec', 0.0) o
r 0.0 for i in (snapshot.network or []))
705         gpu_load = 0.0
706         gpu_temp = 0.0
707         if snapshot.gpus:
708             gpu = snapshot.gpus[0]
709             gpu_load = getattr(gpu, 'load_percent', None)
or getattr(gpu, 'utilization_gpu_pct', 0.0) or 0.0
710             gpu_temp = getattr(gpu, 'temperature_celsius',
None) or getattr(gpu, 'temperature_gpu_c', 0.0) or 0.0
711             raw_procs = snapshot.top_processes or []
712
713         disks_list = [d if isinstance(d, dict) else (d.mod
el_dump() if hasattr(d, 'model_dump') else (vars(d) if ha
sattr(d, '__dict__') else d)) for d in (snapshot.disks or [])]
714         disks_json = json.dumps(disks_list, ensure_ascii=F
alse, default=str) if disks_list else None
715         phys_disks_list = [d if isinstance(d, dict) else (
d.model_dump() if hasattr(d, 'model_dump') else (vars(d) if ha
sattr(d, '__dict__') else d)) for d in (snapshot.physical_disk
s or [])]
716         physical_disks_json = json.dumps(phys_disks_list,
ensure_ascii=False, default=str) if phys_disks_list else None
717

```

```

718         monitors_list = [m if isinstance(m, dict) else (m.
model_dump() if hasattr(m, 'model_dump') else (vars(m) if hasa
ttr(m, '__dict__') else m)) for m in (snapshot.monitors or [])
]
719         monitors_json = json.dumps(monitors_list, ensure_a
scii=False, default=str) if monitors_list else None
720
721         updates_obj = snapshot.updates.model_dump() if has
attr(snapshot.updates, 'model_dump') else (snapshot.updates if
instance(snapshot.updates, dict) else (vars(snapshot.update
s) if hasattr(snapshot.updates, '__dict__') else None))
722         updates_json = json.dumps(updates_obj, ensure_asci
i=False, default=str) if updates_obj else None
723
724         office_obj = snapshot.office.model_dump() if hasat
tr(snapshot.office, 'model_dump') else (snapshot.office if isi
nstance(snapshot.office, dict) else (vars(snapshot.office) if
hasattr(snapshot.office, '__dict__') else None))
725         office_json = json.dumps(office_obj, ensure_ascii=
False, default=str) if office_obj else None
726
727         onedrive_obj = snapshot.onedrive.model_dump() if h
asattr(snapshot.onedrive, 'model_dump') else (snapshot.onedriv
e if isinstance(snapshot.onedrive, dict) else (vars(snapshot.o
nedrive) if hasattr(snapshot.onedrive, '__dict__') else None))
728         onedrive_json = json.dumps(onedrive_obj, ensure_as
cii=False, default=str) if onedrive_obj else None
729
730         battery_obj = snapshot.battery.model_dump() if has
attr(snapshot.battery, 'model_dump') else (snapshot.battery if
instance(snapshot.battery, dict) else (vars(snapshot.batter
y) if hasattr(snapshot.battery, '__dict__') else None))
731         battery_json = json.dumps(battery_obj, ensure_asci
i=False, default=str) if battery_obj else None
732
733         ram_sticks_list = [r if isinstance(r, dict) else (
r.model_dump() if hasattr(r, 'model_dump') else (vars(r) if ha
sattr(r, '__dict__') else r)) for r in (snapshot.ram_sticks or

```

```

    []])
734         ram_sticks_json = json.dumps(ram_sticks_list, ensu
re_ascii=False, default=str) if ram_sticks_list else None
735
736         ports_list = [p if isinstance(p, dict) else (p.mod
el_dump() if hasattr(p, 'model_dump') else (vars(p) if hasattr
(p, '__dict__') else p)) for p in (snapshot.listening_ports or
    [])]
737         listening_ports_json = json.dumps(ports_list, ensu
re_ascii=False, default=str) if ports_list else None
738
739         alerts_list = [a if isinstance(a, dict) else (a.mo
del_dump() if hasattr(a, 'model_dump') else (vars(a) if hasatt
r(a, '__dict__') else a)) for a in (snapshot.alerts or [])]
740         alerts_json = json.dumps(alerts_list, ensure_ascii
=False, default=str) if alerts_list else None
741
742         ts_str = snapshot.timestamp
743         hw_audit = getattr(snapshot, 'hardware_audit', Non
e)
744         raw_full_dict = snapshot.model_dump() if hasattr(s
napshot, 'model_dump') else (vars(snapshot) if hasattr(snapshot,
 '__dict__') else snapshot)
745         raw_json_str = json.dumps(raw_full_dict, ensure_as
cii=False, default=str)
746         else:
747             hostname = snap_data.get('hostname')
748             username = snap_data.get('username') or ''
749             uptime = snap_data.get('uptime_seconds')
750             os_name = snap_data.get('os_name') or 'Windows'
751             os_build = str(snap_data.get('os_build') or '')
752             os_install_date = str(snap_data.get('os_install_da
te') or '')
753             system_lang = str(snap_data.get('system_language')
or '')
754             os_install_lang = str(snap_data.get('os_install_la
nguage') or '')
755             user_locale = str(snap_data.get('user_locale') or

```

```

756 |         sys_locale = str(snap_data.get('system_locale') or
757 |         '')
758 |         timezone_val = str(snap_data.get('timezone') or '')
759 |         codepage_val = str(snap_data.get('codepage') or '')
760 |         input_langs = snap_data.get('input_languages', [])
761 |         input_langs_json = json.dumps(input_langs, ensure_
762 |         ascii=False) if input_langs else None
763 |         disks_val = snap_data.get('disks', [])
764 |         disks_list = [d if isinstance(d, dict) else (d.mod
765 |         el_dump() if hasattr(d, 'model_dump') else (vars(d) if hasattr
766 |         (d, '__dict__') else d)) for d in disks_val] if isinstance(dis
767 |         ks_val, list) else []
768 |         disks_json = json.dumps(disks_list, ensure_ascii=F
769 |         alse, default=str) if disks_list else None
770 |         phys_disks_val = snap_data.get('physical_disks', [
771 |         ])
772 |         phys_disks_list = [d if isinstance(d, dict) else (
773 |         d.model_dump() if hasattr(d, 'model_dump') else (vars(d) if ha
774 |         sattr(d, '__dict__') else d)) for d in phys_disks_val] if isin
775 |         stance(phys_disks_val, list) else []
776 |         physical_disks_json = json.dumps(phys_disks_list,
777 |         ensure_ascii=False, default=str) if phys_disks_list else None
778 |         mon_val = snap_data.get('monitors', [])
779 |         monitors_json = json.dumps(mon_val, ensure_ascii=F
780 |         alse, default=str) if mon_val else None
781 |         upd_val = snap_data.get('updates')
782 |         updates_json = json.dumps(upd_val, ensure_ascii=Fa
783 |         lse, default=str) if upd_val else None
784 |         off_val = snap_data.get('office')
785 |         office_json = json.dumps(off_val, ensure_ascii=Fal
786 |         se, default=str) if off_val else None

```

```
777 |
778 |         od_val = snap_data.get('onedrive')
779 |         onedrive_json = json.dumps(od_val, ensure_ascii=False, default=str) if od_val else None
780 |
781 |         bat_val = snap_data.get('battery')
782 |         battery_json = json.dumps(bat_val, ensure_ascii=False, default=str) if bat_val else None
783 |
784 |         ram_val = snap_data.get('ram_sticks', [])
785 |         ram_sticks_json = json.dumps(ram_val, ensure_ascii=False, default=str) if ram_val else None
786 |
787 |         ports_val = snap_data.get('listening_ports', [])
788 |         listening_ports_json = json.dumps(ports_val, ensure_ascii=False, default=str) if ports_val else None
789 |
790 |         alerts_val = snap_data.get('alerts', [])
791 |         alerts_json = json.dumps(alerts_val, ensure_ascii=False, default=str) if alerts_val else None
792 |
793 |         cpu_obj = snap_data.get('cpu', {})
794 |         cpu_pct = cpu_obj.get('total_percent', 0.0) if isinstance(cpu_obj, dict) else getattr(cpu_obj, 'total_percent', 0.0)
795 |         cpu_freq = cpu_obj.get('frequency_mhz', 0.0) if isinstance(cpu_obj, dict) else getattr(cpu_obj, 'frequency_mhz', 0.0)
796 |         mem_obj = snap_data.get('memory', {})
797 |         mem_tot = mem_obj.get('total_gb', 0.0) if isinstance(mem_obj, dict) else getattr(mem_obj, 'total_gb', 0.0)
798 |         mem_used = mem_obj.get('used_gb', 0.0) if isinstance(mem_obj, dict) else getattr(mem_obj, 'used_gb', 0.0)
799 |         mem_pct = mem_obj.get('percent', 0.0) if isinstance(mem_obj, dict) else getattr(mem_obj, 'percent', 0.0)
800 |         swap_pct = mem_obj.get('swap_percent', 0.0) if isinstance(mem_obj, dict) else getattr(mem_obj, 'swap_percent', 0.0)
```

```

801         disk_obj = snap_data.get('disk_io', {})
802         disk_rb = disk_obj.get('read_bytes_per_sec', 0.0)
803         if isinstance(disk_obj, dict) else getattr(disk_obj, 'read_bytes_per_sec', 0.0)
804         disk_wb = disk_obj.get('write_bytes_per_sec', 0.0)
805         if isinstance(disk_obj, dict) else getattr(disk_obj, 'write_bytes_per_sec', 0.0)
806         disk_rc = disk_obj.get('read_count_per_sec', 0.0)
807         if isinstance(disk_obj, dict) else getattr(disk_obj, 'read_count_per_sec', 0.0)
808         disk_wc = disk_obj.get('write_count_per_sec', 0.0)
809         if isinstance(disk_obj, dict) else getattr(disk_obj, 'write_count_per_sec', 0.0)
810         net_list = snap_data.get('network', [])
811         net_sent = sum((n.get('bytes_sent_sec', 0.0) if isinstance(n, dict) else getattr(n, 'bytes_sent_sec', 0.0)) or 0.0 for n in net_list)
812         net_recv = sum((n.get('bytes_recv_sec', 0.0) if isinstance(n, dict) else getattr(n, 'bytes_recv_sec', 0.0)) or 0.0 for n in net_list)
813         gpus = snap_data.get('gpus', [])
814         gpu_load = 0.0
815         gpu_temp = 0.0
816         if gpus:
817             g0 = gpus[0]
818             gpu_load = (g0.get('load_percent', 0.0) if isinstance(g0, dict) else getattr(g0, 'load_percent', 0.0)) or 0.0
819             gpu_temp = (g0.get('temperature_celsius', 0.0) if isinstance(g0, dict) else getattr(g0, 'temperature_celsius', 0.0)) or 0.0
820         raw_procs = snap_data.get('top_processes', [])
821         ts_str = snap_data.get('timestamp')
822         hw_audit = snap_data.get('hardware_audit')
823         raw_json_str = json.dumps(snap_data, ensure_ascii=False, default=str)
824         now_dt = datetime.now(timezone.utc)

```

```
822         ts_str = ts_str or now_dt.isoformat()
823         try:
824             now_epoch = datetime.fromisoformat(ts_str).timestamp()
825         except Exception:
826             now_epoch = now_dt.timestamp()
827
828         hardware_audit_json = json.dumps(hw_audit or {}, ensure_ascii=False, default=str)
829
830         cursor.execute('''
831             INSERT INTO system_snapshots (
832                 timestamp, created_at, hostname, username, uptime_seconds, os_name, os_build, os_install_date,
833                 system_language, os_install_language, user_locale, system_locale, timezone, codepage,
834                 input_languages_json, disks_json, physical_disks_json, monitors_json, updates_json, office_json,
835                 onedrive_json, battery_json, ram_sticks_json, listening_ports_json, alerts_json,
836                 cpu_total_percent, cpu_frequency_mhz, memory_total_gb, memory_used_gb,
837                 memory_percent, swap_percent, gpu_load_percent, gpu_temp_c,
838                 disk_read_bytes_sec, disk_write_bytes_sec, disk_read_count_sec,
839                 disk_write_count_sec, network_sent_bytes_sec, network_recv_bytes_sec, hardware_audit, raw_json
840             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
841         ''', (
842             ts_str, now_epoch, hostname, username, uptime, os_name, os_build, os_install_date,
843             system_lang, os_install_lang, user_locale, sys_locale, timezone_val, codepage_val,
844             input_langs_json, disks_json, physical_disks_json, monitors_json, updates_json, office_json,
```

```

845 |         onedrive_json, battery_json, ram_sticks_json, list
      | ening_ports_json, alerts_json,
846 |         cpu_pct, cpu_freq, mem_tot, mem_used,
847 |         mem_pct, swap_pct, gpu_load, gpu_temp,
848 |         disk_rb, disk_wb, disk_rc,
849 |         disk_wc, net_sent, net_recv, hardware_audit_json,
      | raw_json_str
850 |     ))
851 |     snapshot_id = cursor.lastrowid or 0
852 |
853 |     processes = raw_procs if top_n is None or top_n <= 0 e
      | lse raw_procs[:top_n]
854 |     proc_rows = []
855 |     for p in processes:
856 |         p_dict = p if isinstance(p, dict) else (p.model_du
      | mp() if hasattr(p, 'model_dump') else (vars(p) if hasattr(p, '
      | __dict__') else p))
857 |         proc_rows.append((
858 |             snapshot_id, ts_str, p_dict.get('pid', 0), p_d
      | ict.get('name', 'unknown'),
859 |             p_dict.get('status', ''), float(p_dict.get('cp
      | u_percent', 0.0) or 0.0),
860 |             float(p_dict.get('memory_mb', 0.0) or 0.0), fl
      | oat(p_dict.get('memory_percent', 0.0) or 0.0),
861 |             int(p_dict.get('num_threads', 0) or 0), int(p_
      | dict.get('num_handles', 0) or 0),
862 |             str(p_dict.get('username', '') or ''), float(p
      | _dict.get('read_bytes_sec', 0.0) or 0.0),
863 |             float(p_dict.get('write_bytes_sec', 0.0) or 0.
      | 0), p_dict.get('integrity_level', None),
864 |             int(bool(p_dict.get('elevation', False))), p_d
      | ict.get('ppid', None),
865 |             p_dict.get('parent_name', None), p_dict.get('e
      | xecutable_path', p_dict.get('executable', None)),
866 |             p_dict.get('cmdline', p_dict.get('command_line
      | ', None)), p_dict.get('sid', None),
867 |             p_dict.get('session_id', None), p_dict.get('cr
      | eation_time', None),

```



```

868         p_dict.get('process_guid', None), p_dict.get('
ancestor_chain', None),
869         p_dict.get('launch_reason', None),
870     ))
871     if proc_rows:
872         cursor.executemany('''
873             INSERT INTO process_snapshots (
874                 snapshot_id, timestamp, pid, name, status,
875                 cpu_percent,
876                 memory_mb, memory_percent, num_threads, nu
m_handles,
877                 username, read_bytes_sec, write_bytes_sec,
878                 integrity_level, elevation,
879                 ppid, parent_name, executable, cmdline, si
d, session_id,
880                 creation_time, process_guid, ancestor_chai
n, launch_reason
881             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
882             ''', proc_rows)
883
884     # Синхронизация нормализованных таблиц оборудования и
метрик
885     try:
886         # 1. CPU
887         cpu_data = getattr(snapshot, 'cpu', None) or snap_
data.get('cpu')
888         if cpu_data:
889             c_dict = cpu_data if isinstance(cpu_data, dict
) else (cpu_data.model_dump() if hasattr(cpu_data, 'model_dump
') else vars(cpu_data))
890             if c_dict.get('model') or c_dict.get('name'):
891                 self.upsert_cpu_inventory(cursor, {
892                     'processor_id': 0,
893                     'name': c_dict.get('model') or c_dict.
get('name', 'CPU'),
894                     'architecture': c_dict.get('architectu
re', 'x86_64'),

```

```

893 |             'physical_cores': c_dict.get('physical
| _cores', 1),
894 |             'logical_cores': c_dict.get('logical_c
| ores', 1),
895 |             'base_frequency_mhz': c_dict.get('freq
| uency_mhz', 0.0),
896 |         })
897 |         self.insert_cpu_telemetry_sample(cursor, {
898 |             'timestamp': ts_str,
899 |             'created_at': now_epoch,
900 |             'processor_id': 0,
901 |             'total_percent': cpu_pct,
902 |             'frequency_mhz': cpu_freq,
903 |             'temperature_c': c_dict.get('temperature_c
| elsius') or c_dict.get('temperature_c'),
904 |             'core_utilization': c_dict.get('per_core_p
| ercent', []),
905 |         })
906 |
907 |         # 2. RAM
908 |         mem_data = getattr(snapshot, 'memory', None) or sn
| ap_data.get('memory')
909 |         if mem_data:
910 |             m_dict = mem_data if isinstance(mem_data, dict
| ) else (mem_data.model_dump() if hasattr(mem_data, 'model_dump
| ') else vars(mem_data))
911 |             self.insert_ram_telemetry_sample(cursor, {
912 |                 'timestamp': ts_str,
913 |                 'created_at': now_epoch,
914 |                 'total_gb': mem_tot,
915 |                 'used_gb': mem_used,
916 |                 'available_gb': m_dict.get('available_gb',
| max(0.0, mem_tot - mem_used)),
917 |                 'percent_used': mem_pct,
918 |                 'swap_total_gb': m_dict.get('swap_total_gb
| ', 0.0),
919 |                 'swap_used_gb': m_dict.get('swap_used_gb',
| 0.0),

```

```

920         'swap_percent': swap_pct,
921     })
922
923     # 3. RAM Sticks (SPD)
924     sticks = getattr(snapshot, 'ram_sticks', None) or
snap_data.get('ram_sticks') or []
925     for s in sticks:
926         self.upsert_ram_module_inventory(cursor, s)
927
928     # 4. GPUs
929     gpus_list = getattr(snapshot, 'gpus', None) or sna
p_data.get('gpus') or []
930     for g_idx, g_item in enumerate(gpus_list):
931         g_dict = g_item if isinstance(g_item, dict) el
se (g_item.model_dump() if hasattr(g_item, 'model_dump') else
vars(g_item))
932         g_name = g_dict.get('name', f'GPU {g_idx}')
933         name_l = g_name.lower()
934         if 'nvidia' in name_l or 'geforce' in name_l o
r g_dict.get('has_cuda'):
935             vendor = 'NVIDIA'
936         elif 'amd' in name_l or 'radeon' in name_l:
937             vendor = 'AMD'
938         elif 'intel' in name_l or 'arc' in name_l or '
uhd' in name_l or 'hd graphics' in name_l:
939             vendor = 'Intel'
940         else:
941             vendor = g_dict.get('vendor') or 'Generic'
942
943         self.upsert_gpu_inventory(cursor, {
944             'gpu_id': g_idx,
945             'name': g_name,
946             'vendor': vendor,
947             'vram_gb': g_dict.get('memory_total_gb', 0
).0),
948             'directml_supported': bool(g_dict.get('has
_directml', True))),
949         })

```

```

950         self.insert_gpu_telemetry_sample(cursor, {
951             'timestamp': ts_str,
952             'created_at': now_epoch,
953             'gpu_id': g_idx,
954             'name': g_dict.get('name', f'GPU {g_idx}')
955         },
956             'load_percent': g_dict.get('load_percent')
957         ),
958             'memory_used_mb': (g_dict.get('memory_used_
959 _gb', 0.0) or 0.0) * 1024.0,
960             'memory_total_mb': (g_dict.get('memory_tot
961 al_gb', 0.0) or 0.0) * 1024.0,
962             'temperature_gpu_c': g_dict.get('temperatu
963 re_celsius') or g_dict.get('temperature_gpu_c'),
964         })
965
966         # 5. Network
967         net_items = getattr(snapshot, 'network', None) or
968         snap_data.get('network') or []
969         for n_idx, n_item in enumerate(net_items):
970             n_dict = n_item if isinstance(n_item, dict) el
971 se (n_item.model_dump() if hasattr(n_item, 'model_dump') else
972 vars(n_item))
973             self.upsert_network_adapter_inventory(cursor,
974             {
975                 'adapter_id': n_idx,
976                 'adapter_guid': n_dict.get('name', f'Adapt
977 er_{n_idx}'),
978                 'name': n_dict.get('name', f'Adapter {n_id
979 x}'),
980                 'max_speed_mbps': n_dict.get('speed_mbps',
981 0),
982             })
983             self.insert_network_adapter_sample(cursor, {
984                 'timestamp': ts_str,
985                 'created_at': now_epoch,
986                 'adapter_name': n_dict.get('name', f'Adapt
987 er {n_idx}'),

```

```

975         'bytes_recv_sec': n_dict.get('bytes_recv_p
er_sec', 0.0),
976         'bytes_sent_sec': n_dict.get('bytes_sent_p
er_sec', 0.0),
977         'link_speed_mbps': n_dict.get('speed_mbps'
, 0),
978         'is_connected': bool(n_dict.get('is_up', T
rue)),
979     })
980     except Exception as norm_err:
981         logger.debug(f'Предупреждение синхронизации нормал
изованных таблиц оборудования: {norm_err}')
982
983     return snapshot_id
984
985     def insert_hardware_archive_row(self, cursor: sqlite3.Curs
or, data: Any) -> int:
986         now_dt = datetime.now(timezone.utc)
987         if isinstance(data, HardwareArchiveEntry):
988             raw_json = data.model_dump_json()
989             report = data.report
990             prob_count = report.problem_devices_count if repor
t else 0
991             outdated_count = report.outdated_drivers_count if
report else 0
992             archive_id = data.archive_id
993             timestamp = data.timestamp
994             devices_count = data.devices_count
995             changes_count = data.changes_count
996         else:
997             raw_json = json.dumps(data, ensure_ascii=False, de
fault=str)
998             report = data.get('report', {})
999             prob_count = report.get('problem_devices_count', 0
) if isinstance(report, dict) else getattr(report, 'problem_de
vices_count', 0)
1000             outdated_count = report.get('outdated_drivers_coun
t', 0) if isinstance(report, dict) else getattr(report, 'outda

```

```

ted_drivers_count', 0)
1001         archive_id = data.get('archive_id', f'arch_{int(now_dt.timestamp())}')
1002         timestamp = data.get('timestamp', now_dt.isoformat())
1003         devices_count = data.get('devices_count', 0)
1004         changes_count = data.get('changes_count', 0)
1005
1006         try:
1007             now_epoch = datetime.fromisoformat(timestamp).timestamp()
1008         except Exception:
1009             now_epoch = now_dt.timestamp()
1010
1011         cursor.execute('''
1012             INSERT OR REPLACE INTO hardware_audits (
1013                 archive_id, timestamp, created_at, devices_count,
1014                 problem_devices_count, outdated_drivers_count,
1015                 changes_count, raw_json
1016             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1017             ''', (archive_id, timestamp, now_epoch, devices_count,
1018                 prob_count, outdated_count, changes_count, raw_json))
1019         return cursor.lastrowid or 0
1020
1021     def insert_startup_archive_row(self, cursor: sqlite3.Cursor, data: Any) -> int:
1022         """Сохранение снимка аудита автозапуска и зафиксированных изменений в БД."""
1023         now_dt = datetime.now(timezone.utc)
1024         if isinstance(data, StartupArchiveEntry):
1025             raw_json = data.model_dump_json()
1026             archive_id = data.archive_id
1027             timestamp = data.timestamp
1028             total_entries = data.total_entries
1029             health_score = data.health_score
1030             changes_count = data.changes_count
1031             report = data.report

```

```

1030 |         changes = data.changes
1031 |         if hasattr(report, 'summary') and report.summary:
1032 |             active_entries = report.summary.active_entries
1033 |             disabled_entries = report.summary.disabled_entries
1034 |             broken_entries = report.summary.broken_entries
1035 |         else:
1036 |             active_entries = 0
1037 |             disabled_entries = 0
1038 |             broken_entries = 0
1039 |     else:
1040 |         raw_json = json.dumps(data, ensure_ascii=False, default=str)
1041 |         archive_id = data.get('archive_id', f'startup_{int(
now_dt.timestamp())}')
1042 |         timestamp = data.get('timestamp', now_dt.isoformat())
1043 |         total_entries = data.get('total_entries', 0)
1044 |         health_score = data.get('health_score', 100)
1045 |         changes_count = data.get('changes_count', 0)
1046 |         report = data.get('report', {})
1047 |         summary = report.get('summary', {}) if isinstance(
report, dict) else getattr(report, 'summary', None)
1048 |         if isinstance(summary, dict):
1049 |             active_entries = summary.get('active_entries',
0)
1050 |             disabled_entries = summary.get('disabled_entries', 0)
1051 |             broken_entries = summary.get('broken_entries',
0)
1052 |         else:
1053 |             active_entries = getattr(summary, 'active_entries', 0) if summary else 0
1054 |             disabled_entries = getattr(summary, 'disabled_entries', 0) if summary else 0
1055 |             broken_entries = getattr(summary, 'broken_entries', 0) if summary else 0
1056 |         changes = data.get('changes', [])

```

```

1057
1058         try:
1059             now_epoch = datetime.fromisoformat(timestamp).time
stamp()
1060         except Exception:
1061             now_epoch = now_dt.timestamp()
1062
1063         cursor.execute('''
1064             INSERT OR REPLACE INTO startup_audit_archives (
1065                 archive_id, timestamp, created_at, total_entri
es,
1066                 active_entries, disabled_entries, broken_entri
es,
1067                 health_score, changes_count, raw_json
1068             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1069         ''', (
1070             archive_id, timestamp, now_epoch, total_entries,
1071             active_entries, disabled_entries, broken_entries,
1072             health_score, changes_count, raw_json
1073         ))
1074
1075         if changes:
1076             for ch in changes:
1077                 ch_dict = ch.model_dump(mode='json') if hasatt
r(ch, 'model_dump') else ch
1078                 cursor.execute('''
1079                     INSERT INTO startup_changes (
1080                         archive_id, timestamp, created_at, cha
nge_type,
1081                         entry_id, name, description, previous_
value, current_value
1082                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1083                 ''', (
1084                     archive_id,
1085                     ch_dict.get('timestamp', timestamp),
1086                     now_epoch,
1087                     ch_dict.get('change_type', 'modified'),
1088                     ch_dict.get('entry_id', ''),

```



```

1089         ch_dict.get('name', ''),
1090         ch_dict.get('description', ''),
1091         json.dumps(ch_dict.get('previous_value'),
ensure_ascii=False) if ch_dict.get('previous_value') is not No
ne else None,
1092         json.dumps(ch_dict.get('current_value'), e
nsure_ascii=False) if ch_dict.get('current_value') is not None
else None,
1093     ))
1094
1095     return cursor.lastrowid or 0
1096
1097     def upsert_device_inventory(
1098         self,
1099         cursor: sqlite3.Cursor,
1100         device_instance_id: str,
1101         friendly_name: Optional[str] = None,
1102         device_class: Optional[str] = None,
1103         install_date: Optional[str] = None,
1104     ) -> int:
1105         """Вставка или обновление записи в таблице device_inve
ntory."""
1106         now_dt = datetime.now(timezone.utc)
1107         now_epoch = now_dt.timestamp()
1108         norm_id = str(device_instance_id).strip().upper()
1109         date_val = install_date or now_dt.strftime('%d.%m.%Y %
H:%M')
1110         cursor.execute('''
1111             INSERT INTO device_inventory (
1112                 device_instance_id, friendly_name, device_clas
s, install_date, created_at, updated_at
1113             ) VALUES (?, ?, ?, ?, ?, ?)
1114             ON CONFLICT(device_instance_id) DO UPDATE SET
1115                 friendly_name = COALESCE(excluded.friendly_nam
e, device_inventory.friendly_name),
1116                 device_class = COALESCE(excluded.device_class,
device_inventory.device_class),
1117                 updated_at = excluded.updated_at

```

```

1118         '', (norm_id, friendly_name or '', device_class or ''
1119         , date_val, now_epoch, now_epoch))
1120
1121     def upsert_disk_inventory(self, cursor: sqlite3.Cursor, da
1122     ta: Any) -> int:
1123         """Вставка или обновление паспорта физического накопит
1124         еля (disk_inventory)."""
1125         d = data if isinstance(data, dict) else (data.model_du
1126         mp() if hasattr(data, 'model_dump') else vars(data))
1127         now_dt = datetime.now(timezone.utc)
1128         ts_str = d.get('timestamp') or now_dt.isoformat()
1129         now_epoch = now_dt.timestamp()
1130         serial = str(d.get('serial_number') or f"DISK_{d.get('
1131         disk_id', 0)}").strip()
1132
1133         cursor.execute(''
1134         INSERT INTO disk_inventory (
1135         disk_id, device_path, serial_number, vendor, p
1136         roduct,
1137         revision, bus_type, media_type, size_bytes, si
1138         ze_gb,
1139         sector_size, removable, partition_style, gpt_g
1140         uid,
1141         partitions_json, first_seen, last_seen, update
1142         d_at
1143         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1144         , ?, ?, ?, ?)
1145         ON CONFLICT(serial_number) DO UPDATE SET
1146         disk_id = excluded.disk_id,
1147         device_path = excluded.device_path,
1148         vendor = excluded.vendor,
1149         product = excluded.product,
1150         revision = excluded.revision,
1151         bus_type = excluded.bus_type,
1152         media_type = excluded.media_type,
1153         size_bytes = excluded.size_bytes,
1154         size_gb = excluded.size_gb,

```

```

1146         sector_size = excluded.sector_size,
1147         removable = excluded.removable,
1148         partition_style = excluded.partition_style,
1149         gpt_guid = excluded.gpt_guid,
1150         partitions_json = excluded.partitions_json,
1151         last_seen = excluded.last_seen,
1152         updated_at = excluded.updated_at
1153     ''' , (
1154         int(d.get('disk_id', 0)),
1155         d.get('device_path') or f"\\\\.\\PhysicalDrive{d.g
1156         et('disk_id', 0)}",
1157         serial,
1158         d.get('vendor', ''),
1159         d.get('product') or d.get('name', ''),
1160         d.get('revision', ''),
1161         d.get('bus_type', 'Unknown'),
1162         d.get('media_type', 'SSD'),
1163         int(d.get('size_bytes', 0)),
1164         float(d.get('size_gb', 0.0)),
1165         int(d.get('sector_size', 512)),
1166         int(bool(d.get('removable', False))),
1167         d.get('partition_style', 'GPT'),
1168         d.get('gpt_guid', ''),
1169         json.dumps(d.get('partitions', []), ensure_ascii=F
1170         else, default=str),
1171         ts_str,
1172         ts_str,
1173         now_epoch,
1174     ))
1175     return cursor.lastrowid or 0
1176
1177     def upsert_volume_inventory(self, cursor: sqlite3.Cursor,
1178     data: Any) -> int:
1179         """Вставка или обновление паспорта логического тома (v
1180         olume_inventory)."""
1181         v = data if isinstance(data, dict) else (data.model_du
1182         mp() if hasattr(data, 'model_dump') else vars(data))
1183         now_dt = datetime.now(timezone.utc)

```

```

1179         ts_str = v.get('timestamp') or now_dt.isoformat()
1180         now_epoch = now_dt.timestamp()
1181         guid = str(v.get('volume_guid') or v.get('volume_id')
or v.get('drive_letter', 'UNKNOWN')).strip()
1182
1183         cursor.execute('''
1184             INSERT INTO volume_inventory (
1185                 volume_guid, drive_letter, mount_point, label,
filesystem,
1186                 filesystem_flags, cluster_size_bytes, sector_s
ize_bytes,
1187                 total_bytes, total_gb, disk_id, first_seen, la
st_seen, updated_at
1188             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
1189
ON CONFLICT(volume_guid) DO UPDATE SET
1190             drive_letter = excluded.drive_letter,
1191             mount_point = excluded.mount_point,
1192             label = excluded.label,
1193             filesystem = excluded.filesystem,
1194             filesystem_flags = excluded.filesystem_flags,
1195             cluster_size_bytes = excluded.cluster_size_byt
es,
1196             sector_size_bytes = excluded.sector_size_bytes
,
1197             total_bytes = excluded.total_bytes,
1198             total_gb = excluded.total_gb,
1199             disk_id = excluded.disk_id,
1200             last_seen = excluded.last_seen,
1201             updated_at = excluded.updated_at
1202         ''', (
1203             guid,
1204             v.get('drive_letter', ''),
1205             v.get('mount_point', ''),
1206             v.get('label', ''),
1207             v.get('filesystem', 'NTFS'),
1208             int(v.get('filesystem_flags', 0) or 0),
1209             int(v.get('cluster_size_bytes', 4096) or 4096),

```

```

1210         int(v.get('sector_size_bytes', 512) or 512),
1211         int(v.get('total_bytes', 0) or 0),
1212         float(v.get('total_gb', 0.0) or 0.0),
1213         v.get('disk_id'),
1214         ts_str,
1215         ts_str,
1216         now_epoch,
1217     ))
1218     return cursor.lastrowid or 0
1219
1220     def insert_disk_health_snapshot(self, cursor: sqlite3.Cursor, data: Any) -> int:
1221         """Вставка снимка здоровья / SMART / TBW накопителя (disk_health_snapshots)."""
1222         h = data if isinstance(data, dict) else (data.model_dump() if hasattr(data, 'model_dump') else vars(data))
1223         now_dt = datetime.now(timezone.utc)
1224         ts_str = h.get('timestamp') or now_dt.isoformat()
1225         try:
1226             created_at = float(h.get('created_at') or datetime.fromisoformat(ts_str).timestamp())
1227         except Exception:
1228             created_at = now_dt.timestamp()
1229
1230         cursor.execute('''
1231             INSERT INTO disk_health_snapshots (
1232                 timestamp, created_at, disk_id, serial_number,
1233                 model, bus_type,
1234                 temperature_c, wear_percent, available_spare,
1235                 tbw_written_tb,
1236                 tbw_read_tb, power_on_hours, power_cycles, unsafe_shutdowns,
1237                 media_errors, error_log_entries, health_status,
1238                 raw_smart_json
1239             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
1240             , ?, ?, ?, ?)
1241             ''', (
1242                 ts_str,

```

```

1239         created_at,
1240         int(h.get('disk_id', 0)),
1241         h.get('serial_number'),
1242         h.get('model', ''),
1243         h.get('bus_type', 'Unknown'),
1244         h.get('temperature_c'),
1245         h.get('wear_percentage') or h.get('wear_percent'),
1246         h.get('available_spare_percent') or h.get('availab
le_spare'),
1247         h.get('tbw_written_tb') or h.get('lifetime_write_t
b'),
1248         h.get('tbw_read_tb') or h.get('lifetime_read_tb'),
1249         h.get('power_on_hours'),
1250         h.get('power_cycles'),
1251         h.get('unsafe_shutdowns'),
1252         h.get('media_errors'),
1253         h.get('error_log_entries'),
1254         h.get('health_status', 'Healthy'),
1255         json.dumps(h, ensure_ascii=False, default=str),
1256     ))
1257     return cursor.lastrowid or 0
1258
1259     def insert_disk_performance_sample(self, cursor: sqlite3.C
ursor, data: Any) -> int:
1260         """Вставка сэмпла производительности диска (disk_perfo
rmance_samples)."""
1261         p = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1262         now_dt = datetime.now(timezone.utc)
1263         ts_str = p.get('timestamp') or now_dt.isoformat()
1264         try:
1265             created_at = float(p.get('created_at') or datetime
.fromisoformat(ts_str).timestamp())
1266         except Exception:
1267             created_at = now_dt.timestamp()
1268
1269         cursor.execute(''
1270             INSERT INTO disk_performance_samples (

```

```

1271         timestamp, created_at, disk_name, read_bytes_s
1272         ec, write_bytes_sec,
1273         read_iops, write_iops, read_time_ms, write_tim
1274         e_ms, percent_disk_time, queue_length
1275     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1276     ''' , (
1277         ts_str,
1278         created_at,
1279         p.get('disk_name') or f"PhysicalDrive{p.get('disk_
1280         id', 0)}",
1281         float(p.get('read_bytes_sec', 0.0) or 0.0),
1282         float(p.get('write_bytes_sec', 0.0) or 0.0),
1283         float(p.get('read_iops', 0.0) or 0.0),
1284         float(p.get('write_iops', 0.0) or 0.0),
1285         float(p.get('read_time_ms', 0.0) or p.get('avg_rea
1286         d_latency_ms', 0.0) or 0.0),
1287         float(p.get('write_time_ms', 0.0) or p.get('avg_wr
1288         ite_latency_ms', 0.0) or 0.0),
1289         float(p.get('percent_disk_time', 0.0) or p.get('di
1290         sk_time_percent', 0.0) or 0.0),
1291         float(p.get('queue_length', 0.0) or 0.0),
1292     ))
1293     return cursor.lastrowid or 0
1294
1295     def insert_disk_io_event(self, cursor: sqlite3.Cursor, dat
1296     a: Any) -> int:
1297         """Вставка события ввода-вывода (disk_io_events)."""
1298         ev = data if isinstance(data, dict) else (data.model_d
1299         ump() if hasattr(data, 'model_dump') else vars(data))
1300         now_dt = datetime.now(timezone.utc)
1301         ts_str = ev.get('timestamp') or now_dt.isoformat()
1302         try:
1303             created_at = float(ev.get('created_at') or datetim
1304             e.fromisoformat(ts_str).timestamp())
1305         except Exception:
1306             created_at = now_dt.timestamp()
1307
1308         cursor.execute(''

```

```

1300         INSERT INTO disk_io_events (
1301             timestamp, created_at, pid, process_name, disk
1302             _id,
1303             operation, bytes_count, duration_ms, offset, f
1304             ile_path
1305         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1306     ''' , (
1307         ts_str,
1308         created_at,
1309         int(ev.get('pid', 0)),
1310         ev.get('process_name', ''),
1311         int(ev.get('disk_id', 0)),
1312         ev.get('operation', 'READ'),
1313         int(ev.get('bytes_count', 0)),
1314         ev.get('duration_ms'),
1315         ev.get('offset'),
1316         ev.get('file_path'),
1317     ))
1318     return cursor.lastrowid or 0
1319
1320     def upsert_cpu_inventory(self, cursor: sqlite3.Cursor, dat
1321     a: Any) -> int:
1322         """Вставка или обновление паспорта физического процесс
1323         opa (cpu_inventory)."""
1324         c = data if isinstance(data, dict) else (data.model_du
1325         mp() if hasattr(data, 'model_dump') else vars(data))
1326         now_dt = datetime.now(timezone.utc)
1327         ts_str = c.get('timestamp') or now_dt.isoformat()
1328         now_epoch = now_dt.timestamp()
1329         name = str(c.get('name') or f"CPU_{c.get('processor_id
1330         ', 0)}").strip()
1331
1332         cursor.execute('''
1333             INSERT INTO cpu_inventory (
1334                 processor_id, name, vendor, architecture, phys
1335                 ical_cores, logical_cores,
1336                 base_frequency_mhz, max_frequency_mhz, l2_cach
1337                 e_kb, l3_cache_kb,

```



```

1330         socket, features_json, first_seen, last_seen,
updated_at
1331     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?)
1332     ON CONFLICT(name) DO UPDATE SET
1333         processor_id = excluded.processor_id,
1334         vendor = excluded.vendor,
1335         architecture = excluded.architecture,
1336         physical_cores = excluded.physical_cores,
1337         logical_cores = excluded.logical_cores,
1338         base_frequency_mhz = excluded.base_frequency_m
hz,
1339         max_frequency_mhz = excluded.max_frequency_mhz
,
1340         l2_cache_kb = excluded.l2_cache_kb,
1341         l3_cache_kb = excluded.l3_cache_kb,
1342         socket = excluded.socket,
1343         features_json = excluded.features_json,
1344         last_seen = excluded.last_seen,
1345         updated_at = excluded.updated_at
1346     '', (
1347         int(c.get('processor_id', 0)),
1348         name,
1349         c.get('vendor', ''),
1350         c.get('architecture', 'x86_64'),
1351         int(c.get('physical_cores', 1) or 1),
1352         int(c.get('logical_cores', 1) or 1),
1353         float(c.get('base_frequency_mhz', 0.0) or 0.0),
1354         float(c.get('max_frequency_mhz', 0.0) or 0.0),
1355         c.get('l2_cache_kb'),
1356         c.get('l3_cache_kb'),
1357         c.get('socket', ''),
1358         json.dumps(c.get('features', []), ensure_ascii=False, default=str),
1359         ts_str,
1360         ts_str,
1361         now_epoch,
1362     ))

```

```

1363         return cursor.lastrowid or 0
1364
1365     def upsert_ram_module_inventory(self, cursor: sqlite3.Curs
or, data: Any) -> int:
1366         """Вставка или обновление паспорта планки оперативной
памяти (ram_module_inventory)."""
1367         r = data if isinstance(data, dict) else (data.model_du
mp() if hasattr(data, 'model_dump') else vars(data))
1368         now_dt = datetime.now(timezone.utc)
1369         ts_str = r.get('timestamp') or now_dt.isoformat()
1370         now_epoch = now_dt.timestamp()
1371         serial = str(r.get('serial_number') or f"RAM_{r.get('b
ank_label', '')}_{r.get('slot_id', 0)}").strip()
1372
1373         cursor.execute('''
1374             INSERT INTO ram_module_inventory (
1375                 slot_id, bank_label, device_locator, serial_nu
mber, part_number,
1376                 manufacturer, capacity_bytes, capacity_gb, spe
ed_mhz, memory_type,
1377                 form_factor, configured_voltage, first_seen, l
ast_seen, updated_at
1378             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?)
1379
1380             ON CONFLICT(serial_number) DO UPDATE SET
1381                 slot_id = excluded.slot_id,
1382                 bank_label = excluded.bank_label,
1383                 device_locator = excluded.device_locator,
1384                 part_number = excluded.part_number,
1385                 manufacturer = excluded.manufacturer,
1386                 capacity_bytes = excluded.capacity_bytes,
1387                 capacity_gb = excluded.capacity_gb,
1388                 speed_mhz = excluded.speed_mhz,
1389                 memory_type = excluded.memory_type,
1390                 form_factor = excluded.form_factor,
1391                 configured_voltage = excluded.configured_volta
ge,
1392                 last_seen = excluded.last_seen,

```

```

1392         updated_at = excluded.updated_at
1393     ''' , (
1394         int(r.get('slot_id', 0)),
1395         r.get('bank_label', 'DIMM'),
1396         r.get('device_locator', ''),
1397         serial,
1398         r.get('part_number', ''),
1399         r.get('manufacturer', ''),
1400         int(r.get('capacity_bytes', 0) or 0),
1401         float(r.get('capacity_gb', 0.0) or 0.0),
1402         int(r.get('speed_mhz', 0) or 0),
1403         r.get('memory_type', 'DDR4'),
1404         r.get('form_factor', 'DIMM'),
1405         r.get('configured_voltage'),
1406         ts_str,
1407         ts_str,
1408         now_epoch,
1409     ))
1410     return cursor.lastrowid or 0
1411
1412     def upsert_gpu_inventory(self, cursor: sqlite3.Cursor, data
1413 a: Any) -> int:
1414         """Вставка или обновление паспорта графического ускоре
1415         теля (gpu_inventory)."""
1416         g = data if isinstance(data, dict) else (data.model_du
1417 mp() if hasattr(data, 'model_dump') else vars(data))
1418         now_dt = datetime.now(timezone.utc)
1419         ts_str = g.get('timestamp') or now_dt.isoformat()
1420         now_epoch = now_dt.timestamp()
1421         pci_id = str(g.get('pci_device_id') or g.get('pci_bus_
1422 id') or f"GPU_{g.get('gpu_id', 0)}_{g.get('name', '')}").strip
1423 ()
1424
1425         cursor.execute('''
1426             INSERT INTO gpu_inventory (
1427                 gpu_id, pci_device_id, name, vendor, driver_ve
1428 rsion, driver_date,
1429                 vram_bytes, vram_gb, pci_bus_id, bios_version,

```

```

    cuda_cores,
1424         directml_supported, first_seen, last_seen, upd
ated_at
1425     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
, ?)
1426     ON CONFLICT(pci_device_id) DO UPDATE SET
1427         gpu_id = excluded.gpu_id,
1428         name = excluded.name,
1429         vendor = excluded.vendor,
1430         driver_version = excluded.driver_version,
1431         driver_date = excluded.driver_date,
1432         vram_bytes = excluded.vram_bytes,
1433         vram_gb = excluded.vram_gb,
1434         pci_bus_id = excluded.pci_bus_id,
1435         bios_version = excluded.bios_version,
1436         cuda_cores = excluded.cuda_cores,
1437         directml_supported = excluded.directml_support
ed,
1438         last_seen = excluded.last_seen,
1439         updated_at = excluded.updated_at
1440     '', (
1441         int(g.get('gpu_id', 0)),
1442         pci_id,
1443         g.get('name', 'Graphics Adapter'),
1444         g.get('vendor', ''),
1445         g.get('driver_version', ''),
1446         g.get('driver_date', ''),
1447         int(g.get('vram_bytes', 0) or 0),
1448         float(g.get('vram_gb', 0.0) or (g.get('memory_tota
l_mb', 0.0) / 1024.0 if g.get('memory_total_mb') else 0.0)),
1449         g.get('pci_bus_id', ''),
1450         g.get('bios_version', ''),
1451         g.get('cuda_cores'),
1452         int(bool(g.get('directml_supported', True))),
1453         ts_str,
1454         ts_str,
1455         now_epoch,
1456     ))

```

```

1457         return cursor.lastrowid or 0
1458
1459     def upsert_network_adapter_inventory(self, cursor: sqlite3
1460         .Cursor, data: Any) -> int:
1461         """Вставка или обновление паспорта сетевого адаптера (
1462         network_adapter_inventory)."""
1463         n = data if isinstance(data, dict) else (data.model_du
1464         mp() if hasattr(data, 'model_dump') else vars(data))
1465         now_dt = datetime.now(timezone.utc)
1466         ts_str = n.get('timestamp') or now_dt.isoformat()
1467         now_epoch = now_dt.timestamp()
1468         guid = str(n.get('adapter_guid') or n.get('mac_address
1469         ') or n.get('name', 'UNKNOWN')).strip()
1470
1471         cursor.execute('''
1472             INSERT INTO network_adapter_inventory (
1473                 adapter_id, adapter_guid, name, interface_name
1474                 , mac_address,
1475                 adapter_type, is_physical, is_wireless, max_sp
1476                 eed_mbps,
1477                 driver_name, driver_version, first_seen, last_
1478                 seen, updated_at
1479             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1480             )
1481
1482             ON CONFLICT(adapter_guid) DO UPDATE SET
1483                 adapter_id = excluded.adapter_id,
1484                 name = excluded.name,
1485                 interface_name = excluded.interface_name,
1486                 mac_address = excluded.mac_address,
1487                 adapter_type = excluded.adapter_type,
1488                 is_physical = excluded.is_physical,
1489                 is_wireless = excluded.is_wireless,
1490                 max_speed_mbps = excluded.max_speed_mbps,
1491                 driver_name = excluded.driver_name,
1492                 driver_version = excluded.driver_version,
1493                 last_seen = excluded.last_seen,
1494                 updated_at = excluded.updated_at
1495         ''', (

```

```

1487         int(n.get('adapter_id', 0)),
1488         guid,
1489         n.get('name', 'Network Adapter'),
1490         n.get('interface_name', ''),
1491         n.get('mac_address', ''),
1492         n.get('adapter_type', 'Ethernet'),
1493         int(bool(n.get('is_physical', True))),
1494         int(bool(n.get('is_wireless', False))),
1495         int(n.get('max_speed_mbps', 0) or 0),
1496         n.get('driver_name', ''),
1497         n.get('driver_version', ''),
1498         ts_str,
1499         ts_str,
1500         now_epoch,
1501     ))
1502     return cursor.lastrowid or 0
1503
1504     def insert_cpu_telemetry_sample(self, cursor: sqlite3.Cursor, data: Any) -> int:
1505         """Вставка снимка метрик нагрузки и температур процессора (cpu_telemetry_samples)."""
1506         c = data if isinstance(data, dict) else (data.model_dump() if hasattr(data, 'model_dump') else vars(data))
1507         now_dt = datetime.now(timezone.utc)
1508         ts_str = c.get('timestamp') or now_dt.isoformat()
1509         try:
1510             created_at = float(c.get('created_at') or datetime.fromisoformat(ts_str).timestamp())
1511         except Exception:
1512             created_at = now_dt.timestamp()
1513
1514         cursor.execute('''
1515             INSERT INTO cpu_telemetry_samples (
1516                 timestamp, created_at, processor_id, total_percent, user_percent,
1517                 kernel_percent, frequency_mhz, temperature_c, package_power_w,
1518                 core_utilization_json, core_temperatures_json

```

```

1519         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1520     ''' , (
1521         ts_str,
1522         created_at,
1523         int(c.get('processor_id', 0)),
1524         float(c.get('total_percent', 0.0) or 0.0),
1525         float(c.get('user_percent', 0.0) or 0.0),
1526         float(c.get('kernel_percent', 0.0) or 0.0),
1527         float(c.get('frequency_mhz', 0.0) or 0.0),
1528         c.get('temperature_c') or c.get('temperature_celsius'),
1529         c.get('package_power_w'),
1530         json.dumps(c.get('core_utilization') or c.get('per
1531         _core_percent', []), ensure_ascii=False, default=str),
1531         json.dumps(c.get('core_temperatures', []), ensure_
1532         ascii=False, default=str),
1532     ))
1533     return cursor.lastrowid or 0
1534
1535     def insert_ram_telemetry_sample(self, cursor: sqlite3.Curs
1536     or, data: Any) -> int:
1537         """Вставка снимка метрик использования оперативной пам
1538         яти (ram_telemetry_samples)."""
1539         r = data if isinstance(data, dict) else (data.model_du
1540         mp() if hasattr(data, 'model_dump') else vars(data))
1541         now_dt = datetime.now(timezone.utc)
1542         ts_str = r.get('timestamp') or now_dt.isoformat()
1543         try:
1544             created_at = float(r.get('created_at') or datetime
1545             .fromisoformat(ts_str).timestamp())
1546         except Exception:
1547             created_at = now_dt.timestamp()
1548
1549         cursor.execute(''
1550             INSERT INTO ram_telemetry_samples (
1551                 timestamp, created_at, total_bytes, total_gb,
1552                 used_bytes, used_gb,
1553                 available_bytes, available_gb, percent_used, s

```

```

wap_total_gb,
1549         swap_used_gb, swap_percent, pool_paged_mb, pool_nonpaged_mb
1550     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
1551 )
1552     '', (
1553         ts_str,
1554         created_at,
1555         int(r.get('total_bytes', 0) or 0),
1556         float(r.get('total_gb', 0.0) or 0.0),
1557         int(r.get('used_bytes', 0) or 0),
1558         float(r.get('used_gb', 0.0) or 0.0),
1559         int(r.get('available_bytes', 0) or 0),
1560         float(r.get('available_gb', 0.0) or 0.0),
1561         float(r.get('percent_used', 0.0) or r.get('percent
1562 ', 0.0) or 0.0),
1563         float(r.get('swap_total_gb', 0.0) or 0.0),
1564         float(r.get('swap_used_gb', 0.0) or 0.0),
1565         float(r.get('swap_percent', 0.0) or 0.0),
1566         r.get('pool_paged_mb'),
1567         r.get('pool_nonpaged_mb'),
1568     ))
1569     return cursor.lastrowid or 0
1570
1571 def insert_gpu_telemetry_sample(self, cursor: sqlite3.Cursor, data: Any) -> int:
1572     """Вставка снимка метрик GPU (gpu_telemetry_samples)."""
1573
1574     g = data if isinstance(data, dict) else (data.model_dump() if hasattr(data, 'model_dump') else vars(data))
1575     now_dt = datetime.now(timezone.utc)
1576     ts_str = g.get('timestamp') or now_dt.isoformat()
1577     try:
1578         created_at = float(g.get('created_at') or datetime.fromisoformat(ts_str).timestamp())
1579     except Exception:
1580         created_at = now_dt.timestamp()
1581

```



```

1579         cursor.execute('''
1580             INSERT INTO gpu_telemetry_samples (
1581                 timestamp, created_at, gpu_id, name, load_perc
ent, memory_used_mb,
1582                 memory_total_mb, memory_percent, temperature_g
pu_c, temperature_memory_c,
1583                 fan_speed_pct, power_draw_w, clock_graphics_mh
z, clock_memory_mhz
1584             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
)
1585         ''', (
1586             ts_str,
1587             created_at,
1588             int(g.get('gpu_id', 0) or g.get('index', 0)),
1589             g.get('name', 'GPU'),
1590             g.get('load_percent') or g.get('utilization_gpu_pc
t'),
1591             float(g.get('memory_used_mb', 0.0) or (g.get('memo
ry_used_gb', 0.0) * 1024.0 if g.get('memory_used_gb') else 0.0
)),
1592             float(g.get('memory_total_mb', 0.0) or (g.get('mem
ory_total_gb', 0.0) * 1024.0 if g.get('memory_total_gb') else
0.0)),
1593             g.get('memory_percent') or g.get('utilization_memo
ry_pct'),
1594             g.get('temperature_gpu_c') or g.get('temperature_c
elsius'),
1595             g.get('temperature_memory_c'),
1596             g.get('fan_speed_pct'),
1597             g.get('power_draw_w'),
1598             g.get('clock_graphics_mhz'),
1599             g.get('clock_memory_mhz'),
1600         ))
1601         return cursor.lastrowid or 0
1602
1603     def insert_network_adapter_sample(self, cursor: sqlite3.Cu
rsor, data: Any) -> int:
1604         """Вставка снимка метрик сетевого адаптера (network_ad

```

```

apter_samples)."""
1605         n = data if isinstance(data, dict) else (data.model_dump() if hasattr(data, 'model_dump') else vars(data))
1606         now_dt = datetime.now(timezone.utc)
1607         ts_str = n.get('timestamp') or now_dt.isoformat()
1608         try:
1609             created_at = float(n.get('created_at') or datetime.fromisoformat(ts_str).timestamp())
1610         except Exception:
1611             created_at = now_dt.timestamp()
1612
1613         cursor.execute('''
1614             INSERT INTO network_adapter_samples (
1615                 timestamp, created_at, adapter_name, bytes_received_sec, bytes_sent_sec,
1616                 packets_recv_sec, packets_sent_sec, errors_in_sec, errors_out_sec,
1617                 link_speed_mbps, is_connected
1618             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1619         ''', (
1620             ts_str,
1621             created_at,
1622             n.get('adapter_name') or n.get('name', 'Network Adapter'),
1623             float(n.get('bytes_received_sec', 0.0) or n.get('bytes_recv_per_sec', 0.0) or 0.0),
1624             float(n.get('bytes_sent_sec', 0.0) or n.get('bytes_sent_per_sec', 0.0) or 0.0),
1625             float(n.get('packets_recv_sec', 0.0) or 0.0),
1626             float(n.get('packets_sent_sec', 0.0) or 0.0),
1627             float(n.get('errors_in_sec', 0.0) or 0.0),
1628             float(n.get('errors_out_sec', 0.0) or 0.0),
1629             int(n.get('link_speed_mbps', 0) or n.get('speed_mbps', 0) or 0),
1630             int(bool(n.get('is_connected', True) if n.get('is_connected') is not None else n.get('is_up', True))),
1631         ))
1632         return cursor.lastrowid or 0

```

```

1633
1634     # =====
1635     # Подсистемы рефакторинга: Data-First SQLite Writers
1636     # =====
1637
1638     def save_event_log_snapshot(
1639         self,
1640         snapshot_id: str,
1641         channels: List[Any],
1642         entries: List[Any],
1643         intelligence_profiles: Optional[List[Any]] = None,
1644     ) -> int:
1645         """Сохранение снимка каналов, кэша событий и профилей
1646         интеллекта логов."""
1647         if self._cm.read_only:
1648             return 0
1649         now_dt = datetime.now(timezone.utc)
1650         ts_str = now_dt.isoformat()
1651         created_at = now_dt.timestamp()
1652
1653         with self._cm.lock, self._cm.get_connection() as conn:
1654             cursor = conn.cursor()
1655             for ch in channels:
1656                 d = ch if isinstance(ch, dict) else (ch.model_
1657                 dump() if hasattr(ch, 'model_dump') else vars(ch))
1658                 cursor.execute('''
1659                     INSERT INTO event_log_channel_snapshots (
1660                         snapshot_id, timestamp, channel_name,
1661                         display_name, description,
1662                         record_count, size_bytes, channel_type
1663                         , is_enabled, created_at
1664                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1665                     ''', (
1666                         snapshot_id, ts_str, d.get('name') or d.ge
1667                         t('channel_name', ''),
1668                         d.get('display_name'), d.get('description'

```

```

),
1664         int(d.get('record_count', 0) or 0), int(d.
get('size_bytes', 0) or 0),
1665         d.get('channel_type', 'Admin'), int(bool(d
.get('enabled', True) if d.get('enabled') is not None else d.g
et('is_enabled', True))),
1666         created_at,
1667     ))
1668
1669     for ev in entries:
1670         d = ev if isinstance(ev, dict) else (ev.model_
dump() if hasattr(ev, 'model_dump') else vars(ev))
1671         cursor.execute('''
1672             INSERT INTO event_log_entries_cache (
1673                 snapshot_id, timestamp, channel, event
_id, level, provider_name,
1674                 time_created, message, raw_json, creat
ed_at
1675                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1676             ''', (
1677                 snapshot_id, ts_str, d.get('channel', 'Sys
tem'),
1678                 int(d.get('event_id', 0) or 0), d.get('lev
el', 'Information'),
1679                 d.get('provider_name') or d.get('provider'
) or d.get('source', ''),
1680                 d.get('time_created') or d.get('timestamp'
, ts_str),
1681                 d.get('message', ''), json.dumps(d, ensure
_ascii=False) if isinstance(d, dict) else None,
1682                 created_at,
1683             ))
1684
1685     if intelligence_profiles:
1686         for prof in intelligence_profiles:
1687             p = prof if isinstance(prof, dict) else (p
rof.model_dump() if hasattr(prof, 'model_dump') else vars(prof
))

```

```

1688         prof_data = p.get('profile') if isinstance
| (p.get('profile'), dict) else p
1689         cursor.execute('''
1690             INSERT INTO event_log_intelligence_pro
| files (
1691                 snapshot_id, timestamp, channel, t
| otal_analyzed, unique_patterns_count,
1692                 critical_incidents_json, top_clust
| ers_json, bursts_json, decision_gate_json, created_at
1693             ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?
| )
1694             ''', (
1695                 snapshot_id, ts_str, p.get('channel',
| 'System'),
1696                 int(p.get('total_analyzed', 0) or prof
| _data.get('total_events', 0) or 0),
1697                 int(p.get('unique_patterns_count', 0)
| or prof_data.get('unique_templates_count', 0) or 0),
1698                 json.dumps(p.get('critical_incidents')
| or p.get('anomalies') or prof_data.get('critical_incidents',
| []), ensure_ascii=False),
1699                 json.dumps(p.get('top_clusters') or p.
| get('top_patterns') or prof_data.get('top_patterns', []), ensu
| re_ascii=False),
1700                 json.dumps(p.get('bursts') or prof_dat
| a.get('bursts', []), ensure_ascii=False),
1701                 json.dumps(p.get('decision') or p.get(
| 'decision_gate') or {
1702                     'strategy': p.get('strategy'),
1703                     'strategy_rationale': p.get('strat
| egy_rationale'),
1704                     'recommended_llm_action': p.get('r
| ecommended_llm_action'),
1705                     }, ensure_ascii=False),
1706                 created_at,
1707             ))
1708
1709         conn.commit()

```

```

1710         return len(channels) + len(entries)
1711
1712     def save_firewall_snapshot(self, snapshot_id: str, profile
1713     s: Any, rules: List[Any]) -> int:
1714         """Сохранение снимка профилей и правил брандмауэра Win
1715         dows."""
1716         if self._cm.read_only:
1717             return 0
1718         now_dt = datetime.now(timezone.utc)
1719         ts_str = now_dt.isoformat()
1720         created_at = now_dt.timestamp()
1721
1722         p = profiles if isinstance(profiles, dict) else (profi
1723         les.model_dump() if hasattr(profiles, 'model_dump') else vars(
1724         profiles))
1725
1726         with self._cm.lock, self._cm.get_connection() as conn:
1727             cursor = conn.cursor()
1728             cursor.execute('''
1729                 INSERT INTO firewall_profile_snapshots (
1730                     snapshot_id, timestamp, domain_enabled, pr
1731                     ivate_enabled, public_enabled,
1732                     domain_default_inbound, private_default_in
1733                     bound, public_default_inbound,
1734                     stealth_mode_enabled, created_at
1735                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1736             ''', (
1737                 snapshot_id, ts_str,
1738                 int(bool(p.get('domain_enabled', True))),
1739                 int(bool(p.get('private_enabled', True))),
1740                 int(bool(p.get('public_enabled', True))),
1741                 p.get('domain_default_inbound', 'Block'),
1742                 p.get('private_default_inbound', 'Block'),
1743                 p.get('public_default_inbound', 'Block'),
1744                 int(bool(p.get('stealth_mode_enabled', True)))
1745             ),
1746                 created_at,
1747             ))

```

```

1741
1742         for r in rules:
1743             d = r if isinstance(r, dict) else (r.model_dump
1744             p() if hasattr(r, 'model_dump') else vars(r))
1745             cursor.execute('''
1746                 INSERT INTO firewall_rule_snapshots (
1747                     snapshot_id, rule_name, display_name,
1748                     direction, action, enabled,
1749                     protocol, local_port, remote_port, pro
1750                     gram_path, profile_mask, created_at
1751                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
1752                     ?)
1753             ''', (
1754                 snapshot_id, d.get('name') or d.get('rule_
1755                 name', ''),
1756                 d.get('display_name') or d.get('name', ''),
1757                 ,
1758                 d.get('direction', 'Inbound'),
1759                 d.get('action', 'Allow'),
1760                 int(bool(d.get('enabled', True))),
1761                 d.get('protocol', 'Any'),
1762                 str(d.get('local_port', '') or ''),
1763                 str(d.get('remote_port', '') or ''),
1764                 d.get('program_path') or d.get('program',
1765                 ''),
1766                 int(d.get('profile_mask', 0) or 0),
1767                 created_at,
1768             ))
1769
1770         conn.commit()
1771         return len(rules) + 1
1772
1773     def update_firewall_rule_state(self, rule_name: str, enabl
1774     ed: bool) -> bool:
1775         """Атомарное обновление состояния правила брандмауэра
1776         в базе данных."""
1777         if self._cm.read_only:
1778             return False

```

```

1770         with self._cm.lock, self._cm.get_connection() as conn:
1771             cursor = conn.cursor()
1772             cursor.execute(
1773                 'UPDATE firewall_rule_snapshots SET enabled =
? WHERE rule_name = ?',
1774                 (int(bool(enabled)), rule_name)
1775             )
1776             conn.commit()
1777             return cursor.rowcount > 0
1778
1779     def save_services_snapshot(self, snapshot_id: str, service
s: List[Any]) -> int:
1780         """Сохранение снимка состояния служб Windows."""
1781         if self._cm.read_only or not services:
1782             return 0
1783         now_dt = datetime.now(timezone.utc)
1784         ts_str = now_dt.isoformat()
1785         created_at = now_dt.timestamp()
1786
1787         with self._cm.lock, self._cm.get_connection() as conn:
1788             cursor = conn.cursor()
1789             for s in services:
1790                 d = s if isinstance(s, dict) else (s.model_dump
p() if hasattr(s, 'model_dump') else vars(s))
1791                 cursor.execute('''
1792                     INSERT INTO services_snapshots (
1793                         snapshot_id, timestamp, service_name,
display_name, state,
1794                         state_code, start_type, pid, binary_pa
th, account, is_orphaned, created_at
1795                         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?)
1796                     ''', (
1797                         snapshot_id, ts_str,
1798                         d.get('name') or d.get('service_name', ''),
,
1799                         d.get('display_name'),
1800                         d.get('state') or d.get('status', 'STOPPED

```



```

    '),
1801         int(d.get('state_code', 0) or 0),
1802         d.get('start_type') or d.get('startup_type',
    ', 'Manual'),
1803         int(d.get('pid', 0) or 0) if d.get('pid')
    else None,
1804         d.get('binary_path') or d.get('binpath', '
    '),
1805         d.get('account') or d.get('user_account',
    ''),
1806         int(bool(d.get('is_orphaned', False))),
1807         created_at,
1808     ))
1809     conn.commit()
1810     return len(services)
1811
1812 def record_service_change(
1813     self,
1814     service_name: str,
1815     display_name: str,
1816     action: str,
1817     old_state: Optional[str] = None,
1818     new_state: Optional[str] = None,
1819     performed_by: str = 'SYSTEM',
1820     details: Optional[Dict[str, Any]] = None,
1821 ) -> int:
1822     """Фиксация события изменения состояния службы."""
1823     if self._cm.read_only:
1824         return 0
1825     now_dt = datetime.now(timezone.utc)
1826     ts_str = now_dt.isoformat()
1827     created_at = now_dt.timestamp()
1828
1829     with self._cm.lock, self._cm.get_connection() as conn:
1830         cursor = conn.cursor()
1831         cursor.execute('''
1832             INSERT INTO service_change_events (
1833                 timestamp, service_name, display_name, act

```

```

ion, old_state,
1834         new_state, performed_by, details_json, cre
ated_at
1835         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
1836         '', (
1837         ts_str, service_name, display_name, action, ol
d_state,
1838         new_state, performed_by,
1839         json.dumps(details, ensure_ascii=False) if det
ails else None,
1840         created_at,
1841         ))
1842         conn.commit()
1843         return cursor.lastrowid or 0
1844
1845     def save_throttling_snapshot(
1846         self,
1847         snapshot_id: str,
1848         throttling_data: Any,
1849         thermal_zones: Optional[List[Any]] = None,
1850     ) -> int:
1851         """Сохранение снимка троттлинга процессора и ACPI терм
ических зон."""
1852         if self._cm.read_only:
1853             return 0
1854         now_dt = datetime.now(timezone.utc)
1855         ts_str = now_dt.isoformat()
1856         created_at = now_dt.timestamp()
1857
1858         t = throttling_data if isinstance(throttling_data, dic
t) else (throttling_data.model_dump() if hasattr(throttling_da
ta, 'model_dump') else vars(throttling_data))
1859
1860         with self._cm.lock, self._cm.get_connection() as conn:
1861             cursor = conn.cursor()
1862             cursor.execute('''
1863                 INSERT INTO cpu_throttling_snapshots (
1864                     snapshot_id, timestamp, prochot_active, pl

```

```

1865 |         pl1_limit_watts, pl2_limit_watts,
1866 |         current_power_watts, max_core_temp_c, pack
1867 | age_temp_c, dpc_latency_us,
1868 |         isr_latency_us, throttling_reasons_json, c
1869 | reated_at
1870 |         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1871 |         '', (
1872 |             snapshot_id, ts_str,
1873 |             int(bool(t.get('prochot_active', False))),
1874 |             float(t.get('pl1_limit_watts', 0.0) or 0.0) if
1875 | t.get('pl1_limit_watts') is not None else None,
1876 |             float(t.get('pl2_limit_watts', 0.0) or 0.0) if
1877 | t.get('pl2_limit_watts') is not None else None,
1878 |             float(t.get('current_power_watts', 0.0) or 0.0
1879 | ) if t.get('current_power_watts') is not None else None,
1880 |             float(t.get('max_core_temp_c', 0.0) or 0.0) if
1881 | t.get('max_core_temp_c') is not None else None,
1882 |             float(t.get('package_temp_c', 0.0) or 0.0) if
1883 | t.get('package_temp_c') is not None else None,
1884 |             int(t.get('dpc_latency_us', 0) or 0),
1885 |             int(t.get('isr_latency_us', 0) or 0),
1886 |             json.dumps(t.get('throttling_reasons') or t.ge
1887 | t('reasons', []), ensure_ascii=False),
1888 |             created_at,
1889 |         ))
1890 |
1891 |         if thermal_zones:
1892 |             for z in thermal_zones:
1893 |                 zd = z if isinstance(z, dict) else (z.mode
1894 | l_dump() if hasattr(z, 'model_dump') else vars(z))
1895 |                 cursor.execute(''
1896 |                     INSERT INTO thermal_zone_snapshots (
1897 |                         snapshot_id, timestamp, zone_name,
1898 |                         temperature_c,
1899 |                         critical_limit_c, throttling_limit
1900 | _c, sensor_provider, created_at
1901 |                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1902 |                     '', (

```

```

1891 |             snapshot_id, ts_str,
1892 |             zd.get('zone_name') or zd.get('name',
| 'ThermalZone'),
1893 |             float(zd.get('temperature_c', 0.0) or
| 0.0),
1894 |             float(zd.get('critical_limit_c', 100.0
| ) or 100.0) if zd.get('critical_limit_c') is not None else Non
| e,
1895 |             float(zd.get('throttling_limit_c', 90.
| 0) or 90.0) if zd.get('throttling_limit_c') is not None else N
| one,
1896 |             zd.get('sensor_provider', 'ACPI'),
1897 |             created_at,
1898 |         ))
1899 |
1900 |         conn.commit()
1901 |         return 1 + (len(thermal_zones) if thermal_zones el
| se 0)
1902 |
1903 |     def save_forensics_snapshot(self, snapshot_id: str, forens
| ics_data: Any) -> int:
1904 |         """Сохранение снимка поведенческой форензики."""
1905 |         if self._cm.read_only:
1906 |             return 0
1907 |         now_dt = datetime.now(timezone.utc)
1908 |         ts_str = now_dt.isoformat()
1909 |         created_at = now_dt.timestamp()
1910 |
1911 |         f = forensics_data if isinstance(forensics_data, dict)
| else (forensics_data.model_dump() if hasattr(forensics_data,
| 'model_dump') else vars(forensics_data))
1912 |
1913 |         with self._cm.lock, self._cm.get_connection() as conn:
1914 |             cursor = conn.cursor()
1915 |             cursor.execute('''
1916 |                 INSERT INTO forensics_snapshots (
1917 |                     snapshot_id, timestamp, foreground_window_
| title, foreground_process_name,

```

```

1918         foreground_pid, user_idle_seconds, camera_
1919         active_apps_json,
1920         microphone_active_apps_json, userassist_to
1921         p_apps_json, created_at
1922     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1923     ''' , (
1924         snapshot_id, ts_str,
1925         f.get('foreground_window_title') or f.get('act
1926         ive_window_title', ''),
1927         f.get('foreground_process_name') or f.get('act
1928         ive_process_name', ''),
1929         int(f.get('foreground_pid', 0) or f.get('activ
1930         e_pid', 0) or 0),
1931         int(f.get('user_idle_seconds', 0) or f.get('id
1932         le_seconds', 0) or 0),
1933         json.dumps(f.get('camera_active_apps') or f.ge
1934         t('camera_apps', []), ensure_ascii=False),
1935         json.dumps(f.get('microphone_active_apps') or
1936         f.get('microphone_apps', []), ensure_ascii=False),
1937         json.dumps(f.get('userassist_top_apps') or f.g
1938         et('userassist_apps', []), ensure_ascii=False),
1939         created_at,
1940     ))
1941     conn.commit()
1942     return cursor.lastrowid or 0
1943
1944 def save_process_leak_snapshot(
1945     self,
1946     snapshot_id: str,
1947     total_processes: Any,
1948     suspicious_count: int = 0,
1949     leak_items: Optional[List[Any]] = None,
1950 ) -> int:
1951     """Сохранение снимка аудита утечек ресурсов процессов.
1952     """
1953     if self._cm.read_only:
1954         return 0
1955     now_dt = datetime.now(timezone.utc)

```

```

1946         ts_str = now_dt.isoformat()
1947         created_at = now_dt.timestamp()
1948
1949         # Если передан объект/словарь отчета
1950         if hasattr(total_processes, 'model_dump') or (isinstance(total_processes, dict) and 'total_processes' in total_processes):
1951             rep_dict = total_processes.model_dump() if hasattr(total_processes, 'model_dump') else total_processes
1952             tot_proc = int(rep_dict.get('total_processes', 0))
1953             susp_cnt = int(rep_dict.get('suspicious_count', 0))
1954             items_list = rep_dict.get('all_processes') or rep_dict.get('top_handle_hogs') or []
1955         else:
1956             tot_proc = int(total_processes or 0)
1957             susp_cnt = int(suspicious_count or 0)
1958             items_list = leak_items or []
1959
1960         with self._cm.lock, self._cm.get_connection() as conn:
1961             cursor = conn.cursor()
1962             cursor.execute('''
1963                 INSERT INTO process_leak_snapshots (
1964                     snapshot_id, timestamp, total_processes, suspicious_count, created_at
1965                     ) VALUES (?, ?, ?, ?, ?)
1966             ''', (
1967                 snapshot_id, ts_str, tot_proc, susp_cnt, created_at
1968             ))
1969
1970         for it in items_list:
1971             d = it if isinstance(it, dict) else (it.model_dump() if hasattr(it, 'model_dump') else vars(it))
1972             cursor.execute('''
1973                 INSERT INTO process_leak_items (
1974                     snapshot_id, pid, name, handles_count,
1975                     gdi_objects, user_objects,

```

```

1975         page_faults, working_set_mb, leak_risk
1976         _score, leak_risk_reasons_json, created_at
1977         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
1978         '', (
1979         snapshot_id,
1980         int(d.get('pid', 0) or 0),
1981         d.get('name') or d.get('process_name', '')
1982         ,
1983         int(d.get('handles_count', 0) or d.get('num_handles', 0) or 0),
1984         int(d.get('gdi_objects', 0) or 0),
1985         int(d.get('user_objects', 0) or 0),
1986         int(d.get('page_faults', 0) or 0),
1987         float(d.get('working_set_mb', 0.0) or d.get('memory_mb', 0.0) or 0.0),
1988         d.get('leak_risk_score', 'LOW'),
1989         json.dumps(d.get('leak_risk_reasons') or d.get('reasons', []), ensure_ascii=False),
1990         created_at,
1991         ))
1992         conn.commit()
1993         return len(leak_items) + 1
1994
1995     def save_defender_snapshot(
1996         self,
1997         snapshot_id: str,
1998         defender_status: Any,
1999         exclusions: Optional[List[Any]] = None,
2000         asr_rules: Optional[List[Any]] = None,
2001         threats: Optional[List[Any]] = None,
2002     ) -> int:
2003         """Сохранение снимка состояния Защитника Windows, исключений, ASR и угроз."""
2004         if self._cm.read_only:
2005             return 0
2006         now_dt = datetime.now(timezone.utc)
2007         ts_str = now_dt.isoformat()

```

```

2007         created_at = now_dt.timestamp()
2008
2009         s = defender_status if isinstance(defender_status, dict) else (defender_status.model_dump() if hasattr(defender_status, 'model_dump') else vars(defender_status))
2010
2011         with self._cm.lock, self._cm.get_connection() as conn:
2012             cursor = conn.cursor()
2013             cursor.execute('''
2014                 INSERT INTO defender_snapshots (
2015                     snapshot_id, timestamp, real_time_protection_enabled, cloud_protection_enabled,
2016                     behavior_monitoring_enabled, tamper_protection_enabled, pua_protection_enabled,
2017                     antivirus_enabled, antispyware_enabled, engine_version, av_signature_version,
2018                     last_quick_scan_datetime, last_full_scan_datetime, security_score,
2019                     cfa_state, raw_status_json, created_at
2020                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
2021             ''', (
2022                 snapshot_id, ts_str,
2023                 int(bool(s.get('real_time_protection_enabled', True))),
2024                 int(bool(s.get('cloud_protection_enabled', True))),
2025                 int(bool(s.get('behavior_monitoring_enabled', True))),
2026                 int(bool(s.get('tamper_protection_enabled', True))),
2027                 int(bool(s.get('pua_protection_enabled', True))),
2028                 int(bool(s.get('antivirus_enabled', True))),
2029                 int(bool(s.get('antispyware_enabled', True))),
2030                 s.get('engine_version', ''),
2031                 s.get('av_signature_version') or s.get('signature_version', ''),

```



```

2032         s.get('last_quick_scan_datetime'),
2033         s.get('last_full_scan_datetime'),
2034         int(s.get('security_score', 100) or 100),
2035         int(s.get('cfa_state', 0) or 0),
2036         json.dumps(s, ensure_ascii=False) if isinstance
e(s, dict) else None,
2037         created_at,
2038     ))
2039
2040     if exclusions:
2041         for exc in exclusions:
2042             d = exc if isinstance(exc, dict) else (exc
.model_dump() if hasattr(exc, 'model_dump') else vars(exc))
2043             cursor.execute('''
2044                 INSERT INTO defender_exclusions (
2045                     snapshot_id, exclusion_type, exclu
sion_value, risk_level, created_at
2046                 ) VALUES (?, ?, ?, ?, ?)
2047             ''', (
2048                 snapshot_id,
2049                 d.get('type') or d.get('exclusion_type
', 'path'),
2050                 d.get('value') or d.get('exclusion_val
ue', ''),
2051                 d.get('risk_level', 'SAFE'),
2052                 created_at,
2053             ))
2054
2055     if asr_rules:
2056         for asr in asr_rules:
2057             d = asr if isinstance(asr, dict) else (asr
.model_dump() if hasattr(asr, 'model_dump') else vars(asr))
2058             cursor.execute('''
2059                 INSERT INTO defender_asr_rules (
2060                     snapshot_id, rule_guid, rule_name,
rule_action, created_at
2061                 ) VALUES (?, ?, ?, ?, ?)
2062             ''', (

```

```

2063         snapshot_id,
2064         d.get('guid') or d.get('rule_guid', ''
    ),
2065         d.get('name') or d.get('rule_name', ''
    ),
2066         d.get('action') or d.get('rule_action'
    , 'Block'),
2067         created_at,
2068     ))
2069
2070     if threats:
2071         for th in threats:
2072             d = th if isinstance(th, dict) else (th.mo
del_dump() if hasattr(th, 'model_dump') else vars(th))
2073             cursor.execute('''
2074                 INSERT INTO defender_threats (
2075                     snapshot_id, threat_id, threat_nam
e, severity, category, resources_json, detection_time, created
_at
2076                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
2077             ''', (
2078                 snapshot_id,
2079                 d.get('threat_id') or d.get('id', ''),
2080                 d.get('threat_name') or d.get('name',
                ''),
2081                 d.get('severity', 'High'),
2082                 d.get('category', 'Trojan'),
2083                 json.dumps(d.get('resources', []), ens
ure_ascii=False),
2084                 d.get('detection_time'),
2085                 created_at,
2086             ))
2087
2088         conn.commit()
2089         return 1 + (len(exclusions) if exclusions else 0)
2090
2091     def save_process_network_snapshot(self, snapshot_id: str,
network_items: List[Any]) -> int:

```

```

2092 |         """Сохранение снимка сетевой активности процессов."""
2093 |         if self._cm.read_only or not network_items:
2094 |             return 0
2095 |         now_dt = datetime.now(timezone.utc)
2096 |         ts_str = now_dt.isoformat()
2097 |         created_at = now_dt.timestamp()
2098 |
2099 |         with self._cm.lock, self._cm.get_connection() as conn:
2100 |             cursor = conn.cursor()
2101 |             for it in network_items:
2102 |                 d = it if isinstance(it, dict) else (it.model_
dump() if hasattr(it, 'model_dump') else vars(it))
2103 |                 cursor.execute('''
2104 |                     INSERT INTO process_network_snapshots (
2105 |                         snapshot_id, timestamp, pid, process_n
ame, local_address,
2106 |                         remote_address, protocol, status, serv
ice_type, sent_kb,
2107 |                         recv_kb, read_speed_kbs, write_speed_k
bs, created_at
2108 |                     ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?)
2109 |                 ''', (
2110 |                     snapshot_id, ts_str,
2111 |                     int(d.get('pid', 0) or 0),
2112 |                     d.get('process_name') or d.get('name', '')
,
2113 |                     d.get('local_address', ''),
2114 |                     d.get('remote_address', ''),
2115 |                     d.get('protocol', 'TCP'),
2116 |                     d.get('status', 'ESTABLISHED'),
2117 |                     d.get('service_type', 'Internet'),
2118 |                     float(d.get('sent_kb', 0.0) or 0.0),
2119 |                     float(d.get('recv_kb', 0.0) or 0.0),
2120 |                     float(d.get('read_speed_kbs', 0.0) or 0.0)
,
2121 |                     float(d.get('write_speed_kbs', 0.0) or 0.0
),

```

```

2122         created_at,
2123     ))
2124     conn.commit()
2125     return len(network_items)
2126
2127     def save_software_inventory(
2128         self,
2129         apps: List[Any],
2130         storage_locations: Optional[List[Any]] = None,
2131         config_files: Optional[List[Any]] = None,
2132         ai_research: Optional[List[Any]] = None,
2133     ) -> int:
2134         """Сохранение инвентаря установленного ПО, мест хранения,
конфигов и AI-аналитики."""
2135         if self._cm.read_only or not apps:
2136             return 0
2137         now_dt = datetime.now(timezone.utc)
2138         ts_str = now_dt.isoformat()
2139
2140         with self._cm.lock, self._cm.get_connection() as conn:
2141             cursor = conn.cursor()
2142             for app in apps:
2143                 d = app if isinstance(app, dict) else (app.model_dump() if hasattr(app, 'model_dump') else vars(app))
2144                 app_id = d.get('app_id') or d.get('id') or d.get('name', '')
2145                 cursor.execute('''
2146                     INSERT OR REPLACE INTO software_inventory
2147                     (
2148                         app_id, display_name, version, publisher, install_date,
2149                         install_location, architecture, is_system_component, first_seen, last_scanned_at
2150                         ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, COALESCE
2151                         ((SELECT first_seen FROM software_inventory WHERE app_id = ?),

```

```

2152         d.get('display_name') or d.get('name', app
_id),
2153         d.get('version', ''),
2154         d.get('publisher', ''),
2155         d.get('install_date', ''),
2156         d.get('install_location') or d.get('path',
        ''),
2157         d.get('architecture', 'x64'),
2158         int(bool(d.get('is_system_component', Fals
e)))),
2159         app_id, ts_str, ts_str,
2160     ))
2161
2162     if storage_locations:
2163         for loc in storage_locations:
2164             d = loc if isinstance(loc, dict) else (loc
.model_dump() if hasattr(loc, 'model_dump') else vars(loc))
2165             cursor.execute('''
2166                 INSERT INTO software_storage_locations
2167                 (
2168                     app_id, category, path, size_bytes
, file_count, last_updated
2169                     ) VALUES (?, ?, ?, ?, ?, ?)
2170                 ''', (
2171                     d.get('app_id', ''),
2172                     d.get('category', 'data'),
2173                     d.get('path', ''),
2174                     int(d.get('size_bytes', 0) or 0),
2175                     int(d.get('file_count', 0) or 0),
2176                     ts_str,
2177                 ))
2178
2179     if config_files:
2180         for cf in config_files:
2181             d = cf if isinstance(cf, dict) else (cf.mo
del_dump() if hasattr(cf, 'model_dump') else vars(cf))
2182             cursor.execute('''
                INSERT INTO software_config_files (

```

```

2183         app_id, file_path, format, size_by
tes, snippet
2184         ) VALUES (?, ?, ?, ?, ?)
2185     ''' , (
2186         d.get('app_id', ''),
2187         d.get('file_path') or d.get('path', ''
),
2188         d.get('format', 'json'),
2189         int(d.get('size_bytes', 0) or 0),
2190         d.get('snippet', ''),
2191     ))
2192
2193     if ai_research:
2194         for ai in ai_research:
2195             d = ai if isinstance(ai, dict) else (ai.mo
del_dump() if hasattr(ai, 'model_dump') else vars(ai))
2196             app_id = d.get('app_id', '')
2197             cursor.execute(''
2198                 INSERT OR REPLACE INTO software_ai_res
earch (
2199                     app_id, summary, config_purpose_ex
planation, data_storage_explanation,
2200                     network_activity_explanation, conf
irmed_facts_json, inferred_facts_json,
2201                     confidence_level, updated_at
2202                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
2203             ''' , (
2204                 app_id,
2205                 d.get('summary', ''),
2206                 d.get('config_purpose_explanation', ''
),
2207                 d.get('data_storage_explanation', ''),
2208                 d.get('network_activity_explanation',
''),
2209                 json.dumps(d.get('confirmed_facts', []
), ensure_ascii=False),
2210                 json.dumps(d.get('inferred_facts', []
, ensure_ascii=False),

```



```
2244 |         ))
2245 |     conn.commit()
2246 |     return len(net_items)
2247 |
2248 |
```


__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Sqlite
  Package
4 # =====
5 # Description:
6 #   Пакет персистентного хранения и выборки системной телеметр
  ии в SQLite.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.sqlite import TelemetryStora
  ge, TelemetryReader
11 #
12 #       storage = TelemetryStorage.get_instance(read_only=True)
13 #       cpu = storage.get_latest_cpu()
14 #
15 # File: __init__.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.sqlite
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 00:20:00
21 # =====
22
23 from __future__ import annotations
24
25 """Пакет персистентного хранения, буферизации и выборки систем
  ной телеметрии в SQLite."""
26
27 from .aggregator import AggregationLevel, TelemetrySqlAggregat
  or, sensors_aggregate
28 from .buffer import TelemetryBuffer
```

```
29 | from .connection import TelemetryConnectionManager
30 | from .maintenance import TelemetryMaintenance
31 | from .reader import TelemetryReader
32 | from .schema import init_database_schema
33 | from .storage import TelemetryStorage
34 | from .writer import TelemetryWriter
35 |
36 | SQLiteTelemetryStorage = TelemetryStorage
37 |
38 | __all__ = [
39 |     'TelemetryStorage',
40 |     'SQLiteTelemetryStorage',
41 |     'TelemetryReader',
42 |     'TelemetryWriter',
43 |     'TelemetryBuffer',
44 |     'TelemetryMaintenance',
45 |     'TelemetryConnectionManager',
46 |     'TelemetrySqlAggregator',
47 |     'AggregationLevel',
48 |     'sensors_aggregate',
49 |     'init_database_schema',
50 | ]
51 |
52 |
```

storage_usage.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Storage
  Usage
4 # =====
5 # Description:
6 #   Четырехуровневый стек сбора телеметрии накопителей Windows
  :
7 #   1. DiskInventoryCollector (Win32 + IOCTL Storage Property
  & Volumes)
8 #   2. DiskHealthCollector (NVMe SMART / Health / TBW)
9 #   3. DiskPerformanceCollector (IOPS, Latency, Throughput, Qu
  eue)
10 #   4. DiskProcessIOCollector (Попроцессный I/O анализ и агрег
  аты)
11 #
12 # Usage Examples:
13 #   Python API:
14 #       from apps.windows.telemetry.storage_usage import (
15 #           DiskInventoryCollector,
16 #           DiskHealthCollector,
17 #           DiskPerformanceCollector,
18 #           DiskProcessIOCollector,
19 #           WindowsStorageUsageCollector,
20 #       )
21 #
22 #       collector = WindowsStorageUsageCollector()
23 #       disks = collector.get_physical_disks()
24 #       report = collector.get_disk_usage_period_report()
25 #
26 # File: storage_usage.py
27 # Project: ai-breadboard
28 # Package: apps.windows.telemetry
29 # Author: hypo69
```

```
30 # Copyright: © 2026 hypo69
31 # Updated: 2026-10-06 01:35:00
32 # =====
33 |=====
34 from __future__ import annotations
35 """Четырехуровневый стек сбора телеметрии накопителей Windows.
36 |"""
37 import math
38 import os
39 import time
40 from typing import Any, Dict, List, Optional, Tuple
41 |
42 try:
43 |     import psutil
44 |     PSUTIL_AVAILABLE = True
45 except ImportError:
46 |     psutil = None
47 |     PSUTIL_AVAILABLE = False
48 |
49 from logger import logger
50 from apps.windows.modules.storage_manager.core.raw_disk_io import WindowsRawDiskIO
51 from apps.windows.modules.storage_manager.core.windows_storage
52 |_sensor import WindowsStorageSensor
53 from apps.windows.telemetry.models import (
54 |     AppDiskUsageItem,
55 |     DiskUsagePeriodReport,
56 |     PhysicalDiskHealth,
57 | )
58 |
59 def _format_bytes(num_bytes: float) -> str:
60 |     """Форматирование объема байт в человекочитаемый вид (B, K
61 |     B, MB, GB, TB).
62 |
63 |     Args:
```

```

63         num_bytes: Количество байт.
64
65     Returns:
66         str: Отформатированная строка.
67     """
68     if num_bytes <= 0 or math.isnan(num_bytes):
69         return '0 B'
70     units = ['B', 'KB', 'MB', 'GB', 'TB', 'PB']
71     idx = 0
72     val = float(num_bytes)
73     while val >= 1024.0 and idx < len(units) - 1:
74         val /= 1024.0
75         idx += 1
76     return f'{val:.2f} {units[idx]}' if idx > 0 else f'{int(val)} B'
77
78
79 class DiskInventoryCollector:
80     """Слой 1: Коллектор инвентаря физических накопителей, разделов и логических томов."""
81
82     def __init__(self, raw_io: Optional[WindowsRawDiskIO] = None) -> None:
83         """Инициализация коллектора инвентаря дисков."""
84         self.raw_io = raw_io or WindowsRawDiskIO()
85
86     def collect(self) -> Dict[str, Any]:
87         """Сбор полного инвентаря накопителей и томов.
88
89         Returns:
90             Dict[str, Any]: Данные физических дисков и логических томов.
91         """
92         disks = self.raw_io.list_physical_disks()
93         volumes = self.raw_io.get_logical_volumes()
94         return {
95             'disks_count': len(disks),
96             'disks': disks,

```

```

97         'volumes_count': len(volumes),
98         'volumes': volumes,
99         'timestamp': time.time(),
100     }
101
102
103 class DiskHealthCollector:
104     """Слой 2: Коллектор SMART-показателей здоровья, износа и
105     TBW накопителей."""
106
107     def __init__(
108         self,
109         raw_io: Optional[WindowsRawDiskIO] = None,
110         storage_sensor: Optional[WindowsStorageSensor] = None,
111     ) -> None:
112         """Инициализация коллектора здоровья дисков."""
113         self.raw_io = raw_io or WindowsRawDiskIO()
114         self.storage_sensor = storage_sensor or WindowsStorage
115         Sensor()
116
117     def collect(self, force_refresh: bool = False) -> List[Dic
118     t[str, Any]]:
119         """Сбор показателей здоровья всех доступных накопители
120         й.
121
122         Returns:
123             List[Dict[str, Any]]: Показатели SMART, TBW, темпе
124             ратуры и ошибок.
125         """
126         results: List[Dict[str, Any]] = []
127         raw_disks = self.raw_io.list_physical_disks()
128
129         for d in raw_disks:
130             disk_id = d.get('disk_id', 0)
131             smart = d.get('smart_health', {})
132             results.append({
133                 'disk_id': disk_id,
134                 'device_path': d.get('device_path'),

```

```

130         'model': d.get('product'),
131         'bus_type': d.get('bus_type'),
132         'temperature_c': smart.get('temperature_c'),
133         'wear_percentage': smart.get('percentage_used'
    ),
134         'available_spare_percent': smart.get('available_spare_percent'),
135         'tbw_written_tb': smart.get('data_units_written_tb'),
136         'tbw_read_tb': smart.get('data_units_read_tb')
    ,
137         'power_on_hours': smart.get('power_on_hours'),
138         'unsafe_shutdowns': smart.get('unsafe_shutdowns'),
139         'media_errors': smart.get('media_errors'),
140         'raw_available': smart.get('raw_nvme_available', False),
141     })
142
143     # Fallback / дополнение через WindowsStorageSensor CIM
    если нативный IOCTL вернул None
144     needs_cim = any(r['temperature_c'] is None and r['wear_percentage'] is None for r in results)
145     if needs_cim:
146         try:
147             cim_disks = self.storage_sensor.get_physical_disks(force_refresh=force_refresh)
148             cim_map = {str(cd.device_id).replace(r'\\.\PHYSICALDRIVE', '').replace('Disk', ''): cd for cd in cim_disks}
149             for r in results:
150                 str_id = str(r['disk_id'])
151                 if str_id in cim_map:
152                     cd = cim_map[str_id]
153                     if r['temperature_c'] is None and cd.temperature_c:
154                         r['temperature_c'] = cd.temperature_c
155                     if r['wear_percentage'] is None and cd

```

```

155 |         .wear_percentage:
156 |             r['wear_percentage'] = cd.wear_per
157 |             centage
158 |             if r['power_on_hours'] is None and cd.
159 |             power_on_hours:
160 |                 r['power_on_hours'] = cd.power_on_
161 |                 hours
162 |                 if r['tbw_written_tb'] is None and cd.
163 |                 lifetime_write_tb:
164 |                     r['tbw_written_tb'] = cd.lifetime_
165 |                     write_tb
166 |                     if r['tbw_read_tb'] is None and cd.lif
167 |                     etime_read_tb:
168 |                         r['tbw_read_tb'] = cd.lifetime_rea
169 |                         d_tb
170 |             except Exception as exc:
171 |                 logger.debug(f"Ошибка дополнения здоровья чере
172 | з CIM: {exc}")
173 |
174 |         return results
175 |
176 |
177 | class DiskPerformanceCollector:
178 |     """Слой 3: Коллектор текущей скорости, IOPS и времени откл
179 | ика дисков."""
180 |
181 |     def __init__(self) -> None:
182 |         """Инициализация коллектора производительности дисков.
183 |
184 |
185 |         self._last_sample_time: float = time.time()
186 |         self._last_io_counters: Dict[str, Any] = {}
187 |
188 |     def collect(self) -> Dict[str, Any]:
189 |         """Сбор показателей пропускной способности и IOPS нако
190 | пителей.
191 |
192 |
193 |         Returns:
194 |
195 |         Dict[str, Any]: Метрики скорости чтения/записи и о

```


пераций в секунду.

```
182         """
183         now = time.time()
184         dt = max(0.001, now - self._last_sample_time)
185         res: Dict[str, Any] = {
186             'total_read_bytes_sec': 0.0,
187             'total_write_bytes_sec': 0.0,
188             'total_read_iops': 0.0,
189             'total_write_iops': 0.0,
190             'per_disk': {},
191             'dt_sec': dt,
192         }
193
194         if not PSUTIL_AVAILABLE:
195             self._last_sample_time = now
196             return res
197
198         try:
199             per_disk_counters = psutil.disk_io_counters(perdis
200 k=True)
201             if per_disk_counters:
202                 for disk_name, cur in per_disk_counters.items(
203 ):
204                     prev = self._last_io_counters.get(disk_name)
205                     if prev:
206                         r_bytes = max(0, cur.read_bytes - prev
207 .read_bytes) / dt
208                         w_bytes = max(0, cur.write_bytes - prev
209 v.write_bytes) / dt
210                         r_count = max(0, cur.read_count - prev
211 .read_count) / dt
212                         w_count = max(0, cur.write_count - prev
213 v.write_count) / dt
214                         r_time = max(0, cur.read_time - prev.r
215 ead_time)
216                         w_time = max(0, cur.write_time - prev.
217 write_time)
```

```

210 |
211 |             res['total_read_bytes_sec'] += r_bytes
212 |             res['total_write_bytes_sec'] += w_byte
    | s
213 |             res['total_read_iops'] += r_count
214 |             res['total_write_iops'] += w_count
215 |
216 |             res['per_disk'][disk_name] = {
217 |                 'read_bytes_sec': round(r_bytes, 1
    | ),
218 |                 'write_bytes_sec': round(w_bytes,
    | 1),
219 |                 'read_iops': round(r_count, 1),
220 |                 'write_iops': round(w_count, 1),
221 |                 'read_time_ms': r_time,
222 |                 'write_time_ms': w_time,
223 |             }
224 |             self._last_io_counters = per_disk_counters
225 |         except Exception as exc:
226 |             logger.debug(f"Ошибка сбора счетчиков производител
    | ьности дисков: {exc}")
227 |
228 |             self._last_sample_time = now
229 |             return res
230 |
231 |
232 | class DiskProcessIOCollector:
233 |     """Слой 4: Коллектор активности ввода-вывода процессов (To
    | p Writers/Readers)."""
234 |
235 |     def __init__(self) -> None:
236 |         """Инициализация коллектора I/O процессов."""
237 |         self._last_process_io: Dict[int, Dict[str, Any]] = {}
238 |         self._last_poll_time: float = time.time()
239 |
240 |     def collect(self) -> Dict[str, Dict[str, Any]]:
241 |         """Опрос I/O всех активных процессов и вычисление дель
    | т чтения/записи.

```

```

242
243     Returns:
244         Dict[str, Dict[str, Any]]: Агрегированные дельты п
о именам процессов.
245         """
246         if not PSUTIL_AVAILABLE:
247             return {}
248
249         now = time.time()
250         aggregated_by_name: Dict[str, Dict[str, Any]] = {}
251
252         try:
253             for p in psutil.process_iter(attrs=['pid', 'name']
):
254                 try:
255                     pid = p.info['pid']
256                     name = p.info.get('name') or f'PID_{pid}'
257                     io_cnt = p.io_counters()
258                     cur_r = io_cnt.read_bytes
259                     cur_w = io_cnt.write_bytes
260
261                     prev = self._last_process_io.get(pid)
262                     if prev:
263                         delta_r = max(0, cur_r - prev['read_by
tes'])
264                         delta_w = max(0, cur_w - prev['write_b
ytes'])
265                     else:
266                         delta_r = cur_r
267                         delta_w = cur_w
268
269                     self._last_process_io[pid] = {
270                         'name': name,
271                         'read_bytes': cur_r,
272                         'write_bytes': cur_w,
273                         'time': now,
274                     }
275

```

```

276         if name not in aggregated_by_name:
277             aggregated_by_name[name] = {
278                 'pid': pid,
279                 'read_bytes': 0,
280                 'write_bytes': 0,
281             }
282             aggregated_by_name[name]['read_bytes'] +=
283             delta_r
284             aggregated_by_name[name]['write_bytes'] +=
285             delta_w
286         except (psutil.NoSuchProcess, psutil.AccessDen
287         ired, AttributeError):
288             continue
289         except Exception as ex:
290             logger.debug(f'Ошибка выборки process I/O deltas:
291             {ex}')
292
293     self._last_poll_time = now
294     return aggregated_by_name
295
296 class WindowsStorageUsageCollector:
297     """Унифицированный диспетчер дисковой телеметрии, объединя
298     ющий все 4 слоя."""
299
300     def __init__(
301         self,
302         raw_io: Optional[WindowsRawDiskIO] = None,
303         storage_sensor: Optional[WindowsStorageSensor] = None,
304     ) -> None:
305         """Инициализация унифицированного коллектора дисковой
306         телеметрии."""
307
308         self.raw_io = raw_io or WindowsRawDiskIO()
309         self.storage_sensor = storage_sensor or WindowsStorage
310         Sensor()
311
312         self.inventory_collector = DiskInventoryCollector(self
313         .raw_io)

```

```

306         self.health_collector = DiskHealthCollector(self.raw_i
o, self.storage_sensor)
307         self.performance_collector = DiskPerformanceCollector(
)
308         self.process_io_collector = DiskProcessIOCollector()
309
310     def get_physical_disks(self, force_refresh: bool = False)
-> List[PhysicalDiskHealth]:
311         """Получить актуальный список физических накопителей с
показателями SMART и Lifetime."""
312         results: List[PhysicalDiskHealth] = []
313         try:
314             health_list = self.health_collector.collect(force_
refresh=force_refresh)
315             for h in health_list:
316                 results.append(PhysicalDiskHealth(
317                     device_id=f"Disk {h.get('disk_id', 0)}",
318                     model=h.get('model') or f"PhysicalDrive{h.
get('disk_id', 0)}",
319                     media_type='SSD' if 'SSD' in (h.get('model
', '')) + h.get('bus_type', '').upper() or h.get('bus_type') =
= 'NVMe' else 'HDD',
320                     size_gb=0.0,
321                     health_status='Healthy',
322                     operational_status='OK',
323                     temperature_celsius=h.get('temperature_c')
,
324                     interface_type=h.get('bus_type') or 'NVMe'
,
325                     lifetime_read_bytes=int(h['tbw_read_tb'] *
(1024 ** 4)) if h.get('tbw_read_tb') else None,
326                     lifetime_write_bytes=int(h['tbw_written_tb
'] * (1024 ** 4)) if h.get('tbw_written_tb') else None,
327                     lifetime_read_tb=h.get('tbw_read_tb'),
328                     lifetime_write_tb=h.get('tbw_written_tb'),
329                     power_on_hours=h.get('power_on_hours'),
330                     wear_percentage=h.get('wear_percentage'),
331                 ))

```

```

332         except Exception as ex:
333             logger.debug(f'Ошибка получения физических накопит
елей: {ex}')
334
335         if not results:
336             results.append(PhysicalDiskHealth(
337                 device_id='Disk 0',
338                 model='System Drive (NVMe/SSD)',
339                 media_type='SSD',
340                 size_gb=512.0,
341                 health_status='Healthy',
342                 operational_status='OK',
343                 interface_type='NVMe',
344             ))
345         return results
346
347     def poll_process_io_deltas(self) -> Dict[str, Dict[str, An
y]]:
348         """Опросить текущий I/O процессов и рассчитать накопле
нные дельты."""
349         return self.process_io_collector.collect()
350
351     def get_disk_usage_period_report(
352         self,
353         period_minutes: int = 1440,
354         force_refresh_smart: bool = False,
355     ) -> DiskUsagePeriodReport:
356         """Сформировать агрегированный отчет об использовании
диска за интервал."""
357         phys_disks = self.get_physical_disks(force_refresh=for
ce_refresh_smart)
358         proc_deltas = self.poll_process_io_deltas()
359
360         writers_list: List[AppDiskUsageItem] = []
361         readers_list: List[AppDiskUsageItem] = []
362
363         total_r = sum(item['read_bytes'] for item in proc_delt
as.values())

```

```

364         total_w = sum(item['write_bytes'] for item in proc_deltas.values())
365         total_all = total_r + total_w
366
367         # Ранжирование по записи
368         sorted_writers = sorted(proc_deltas.items(), key=lambda x: x[1]['write_bytes'], reverse=True)
369         for name, item in sorted_writers[:15]:
370             wb = item['write_bytes']
371             rb = item['read_bytes']
372             tot = wb + rb
373             if wb <= 0:
374                 continue
375             pct = round((wb / total_w) * 100.0, 1) if total_w > 0 else 0.0
376             writers_list.append(AppDiskUsageItem(
377                 process_name=name,
378                 pid=item.get('pid'),
379                 read_bytes=rb,
380                 write_bytes=wb,
381                 total_bytes=tot,
382                 read_formatted=_format_bytes(rb),
383                 write_formatted=_format_bytes(wb),
384                 share_percent=pct,
385             ))
386
387         # Ранжирование по чтению
388         sorted_readers = sorted(proc_deltas.items(), key=lambda x: x[1]['read_bytes'], reverse=True)
389         for name, item in sorted_readers[:15]:
390             wb = item['write_bytes']
391             rb = item['read_bytes']
392             tot = wb + rb
393             if rb <= 0:
394                 continue
395             pct = round((rb / total_r) * 100.0, 1) if total_r > 0 else 0.0
396             readers_list.append(AppDiskUsageItem(

```

```

397         process_name=name,
398         pid=item.get('pid'),
399         read_bytes=rb,
400         write_bytes=wb,
401         total_bytes=tot,
402         read_formatted=_format_bytes(rb),
403         write_formatted=_format_bytes(wb),
404         share_percent=pct,
405     ))
406
407     top_writer = sorted_writers[0][0] if sorted_writers and sorted_writers[0][1]['write_bytes'] > 0 else 'N/A'
408     top_reader = sorted_readers[0][0] if sorted_readers and sorted_readers[0][1]['read_bytes'] > 0 else 'N/A'
409
410     diag = (
411         f"За анализируемый период ({period_minutes} мин) за
412         аписано {_format_bytes(total_w)}, "
413         f"прочитано {_format_bytes(total_r)}. Главный источник записи: {top_writer}, чтения: {top_reader}."
414     )
415
416     return DiskUsagePeriodReport(
417         period_minutes=period_minutes,
418         total_read_bytes=total_r,
419         total_write_bytes=total_w,
420         total_read_formatted=_format_bytes(total_r),
421         total_write_formatted=_format_bytes(total_w),
422         total_formatted=_format_bytes(total_all),
423         top_writers=writers_list,
424         top_readers=readers_list,
425         physical_disks=phys_disks,
426         summary_text=diag,
427     )
428
429 __all__ = [
430     'DiskInventoryCollector',

```



```
431 | 'DiskHealthCollector',  
432 | 'DiskPerformanceCollector',  
433 | 'DiskProcessIOCollector',  
434 | 'WindowsStorageUsageCollector',  
435 | ]  
436 |
```

telemetry_config.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Telemetry Config
4 # =====
5 # Description:
6 #   Менеджер конфигурации сбора телеметрии и сенсоров с поддержкой
7 #   динамического перечитывания config.json на лету.
8 #
9 # Usage Examples:
10 #   Python API:
11 #       from apps.windows.telemetry.telemetry_config import TelemetryConfigManager
12 #
13 #       config_mgr = TelemetryConfigManager()
14 #       interval = config_mgr.get_interval_seconds()
15 #       if config_mgr.check_and_reload():
16 #           print("Конфигурация обновлена!")
17 #
18 # File: telemetry_config.py
19 # Project: ai-breadboard
20 # Package: apps.windows.telemetry
21 # Author: hypo69
22 # Copyright: © 2026 hypo69
23 # Updated: 2026-10-06 07:58:00
24 # =====
25
26 from __future__ import annotations
27 """Менеджер конфигурации сбора телеметрии и сенсоров."""
28
29 import json
30 import os
```

```
31 import re
32 import shutil
33 from pathlib import Path
34 from typing import Any, Dict, List, Optional
35 try:
36     from logger import logger
37 except ImportError:
38     import logging
39     logger = logging.getLogger(__name__)
40
41
42 def get_default_telemetry_config_path() -> Path:
43     """Возвращает путь по умолчанию к файлу конфигурации телем
    етрии в %ProgramData%.
44
45     Проверяет сначала %ProgramData%/AITelemetry/config/config.
    json,
46     затем %ProgramData%/AI-Breadboard/apps/windows/telemetry/c
    onfig.json.
47     Если целевой файл отсутствует, выполняется попытка скопиро
    вать шаблон из пакета модуля.
48
49     Returns:
50         Path: Путь к файлу конфигурации в %ProgramData%.
51     """
52     programdata = os.environ.get('ProgramData') or os.environ.
    get('ALLUSERSPROFILE')
53     if programdata and os.path.exists(programdata):
54         base_dir = Path(programdata)
55     else:
56         appdata = os.environ.get('APPDATA') or os.environ.get(
    'LOCALAPPDATA')
57         if appdata and os.path.exists(appdata):
58             base_dir = Path(appdata)
59         else:
60             base_dir = Path.home() / '.config'
61
62     ai_tel_config = base_dir / 'AITelemetry' / 'config' / 'con
```

```

fig.json'
63     if ai_tel_config.is_file():
64         return ai_tel_config
65
66     target_path = base_dir / 'AI-Breadboard' / 'apps' / 'windo
ws' / 'telemetry' / 'config.json'
67
68     if not target_path.exists():
69         bundled_template = Path(__file__).parent / 'config.jso
n'
70         if bundled_template.is_file():
71             try:
72                 target_path.parent.mkdir(parents=True, exist_o
k=True)
73                 shutil.copy2(bundled_template, target_path)
74             except Exception:
75                 return bundled_template
76     return target_path
77
78
79 def parse_interval_to_seconds(val: Any, default: float = 5.0)
-> float:
80     """Преобразует строковое или числовое значение интервала в
секунды (float).
81
82     Поддерживает:
83     - Числа (int, float): 5 -> 5.0, 0.5 -> 0.5
84     - Строковые числа: "5", "5.0" -> 5.0
85     - Строки с единицами времени:
86         - Секунды: "5s", "5 seconds", "5 сек", "5 секунд"
-> 5.0
87         - Минуты: "1m", "1 minute", "5 minutes", "2 мин" -
> 60.0 / 300.0 / 120.0
88         - Часы: "1h", "1 hour", "6 hours", "1 час" -> 3600
.0 / 21600.0
89         - Дни: "1d", "1 day", "7 days", "1 день" -> 86400.
0 / 604800.0
90

```

```

91 |     Args:
92 |         val: Значение интервала из конфигурации.
93 |         default: Значение по умолчанию в секундах.
94 |
95 |     Returns:
96 |         float: Интервал в секундах (минимум 0.001 при валидном
положительном значении).
97 |         ""
98 |     if val is None:
99 |         return float(default)
100 |     if isinstance(val, (int, float)):
101 |         return max(0.001, float(val))
102 |     if isinstance(val, str):
103 |         val_str = val.strip().lower()
104 |         if not val_str:
105 |             return float(default)
106 |         try:
107 |             return max(0.001, float(val_str))
108 |         except ValueError:
109 |             pass
110 |         match = re.match(r"^([\d.]+\s*([a-za-я]+)?$", val_str
)
111 |         if match:
112 |             try:
113 |                 num = float(match.group(1))
114 |                 unit = match.group(2) or "s"
115 |                 if unit in ("s", "sec", "second", "seconds", "
сек", "секунд", "секунды", "секунда"):
116 |                     return max(0.001, num)
117 |                 elif unit in ("m", "min", "minute", "minutes",
"мин", "минут", "минуты", "минута"):
118 |                     return max(0.001, num * 60.0)
119 |                 elif unit in ("h", "hr", "hour", "hours", "ч",
"час", "часа", "часов"):
120 |                     return max(0.001, num * 3600.0)
121 |                 elif unit in ("d", "day", "days", "д", "день",
"дня", "дней"):
122 |                     return max(0.001, num * 86400.0)

```

```

123         except ValueError:
124             pass
125     return float(default)
126
127
128 class TelemetryConfigManager:
129     """Менеджер конфигурации для централизованного управления
130     сенсорами и интервалами."""
131
132     def __init__(self, config_path: Optional[str] = None) -> None:
133         """Инициализирует менеджер конфигурации.
134
135         Args:
136             config_path: Путь к файлу конфигурации (по умолчанию: %APPDATA%/AI-Breadboard/apps/windows/telemetry/config.json).
137
138         """
139         if config_path:
140             self._config_path = str(config_path)
141         else:
142             self._config_path = str(get_default_telemetry_config_path())
143
144         self._config: Dict[str, Any] = {}
145         self._sensors_config: Dict[str, Dict[str, Any]] = {}
146         self._loggers_config: Dict[str, Dict[str, Any]] = {}
147         self._default_interval = 5.0
148         self._fast_interval = 30.0
149         self._fast_duration_days = 2
150         self._aggregation_interval = 3600.0
151         self._heavy_interval = 60.0
152         self._heavy_disk_scan_interval = 43200.0
153         self._heavy_mode_max_duration_days = 5.0
154         self._heavy_mode_auto_switch_enabled = True
155         self._mode = 'hybrid'
156         self._telemetry_mode = 'telemetry'
157         self._standard_flush_interval_seconds = 30.0
158         self._tc_flush_interval_seconds = 5.0

```

```

156         self._tc_mode_timeout_seconds = 300.0
157         self._tc_mode_activated_at: Optional[float] = None
158         self._max_db_size_mb = 100.0
159         self._retention_days = 7
160         self._db_cleanup_interval_seconds = 300.0
161         self._flush_interval_seconds = 30.0
162         self._auto_vacuum_enabled = True
163         self._last_mtime_ns: int = 0
164         self._load_config()
165
166     def _load_config(self) -> None:
167         """Загружает конфигурацию из JSON файла с фиксацией вр
168         емени модификации."""
169         try:
170             cfg_path = Path(self._config_path)
171             if cfg_path.is_file():
172                 self._last_mtime_ns = cfg_path.stat().st_mtime
173                 _ns
174             with open(self._config_path, 'r', encoding='utf-8'
175             ) as f:
176                 new_config = json.load(f)
177
178                 self._config = new_config
179                 if 'sensors' in self._config:
180                     self._sensors_config = self._config.get('senso
181                     rs', {})
182                 elif 'telemetry' in self._config and 'sensors' in
183                 self._config['telemetry']:
184                     self._sensors_config = self._config['telemetry
185                     '].get('sensors', {})
186                 else:
187                     self._sensors_config = {}
188
189                 self._loggers_config = self._config.get('loggers',
190                 {})
191
192                 raw_interval = self._config.get('interval_seconds'
193                 , self._config.get('interval', self._config.get('telemetry', {

```

```

    }).get('interval_seconds', 5.0)))
186         self._default_interval = parse_interval_to_seconds
    (raw_interval, 5.0)
187
188         raw_heavy = self._config.get('heavy_interval_seconds', self._config.get('heavy_interval', self._config.get('telemetry', {})).get('heavy_interval_seconds', 60.0)))
189         self._heavy_interval = parse_interval_to_seconds(raw_heavy, 60.0)
190
191         raw_disk_interval = self._config.get('heavy_disk_scan_interval_seconds')
192         self._heavy_disk_scan_interval = parse_interval_to_seconds(raw_disk_interval, 43200.0)
193
194         raw_fast_interval = self._config.get('fast_interval_seconds')
195         self._fast_interval = parse_interval_to_seconds(raw_fast_interval, 30.0)
196
197         self._fast_duration_days = int(self._config.get('fast_duration_days', 2))
198
199         raw_agg = self._config.get('aggregation_interval_seconds', self._config.get('rollup_interval_seconds'))
200         self._aggregation_interval = parse_interval_to_seconds(raw_agg, 3600.0)
201
202         raw_max_days = self._config.get('heavy_mode_max_duration_days')
203         self._heavy_mode_max_duration_days = float(raw_max_days) if raw_max_days is not None else 5.0
204
205         self._heavy_mode_auto_switch_enabled = bool(self._config.get('heavy_mode_auto_switch_enabled', True))
206         self._mode = str(self._config.get('mode', self._config.get('telemetry', {})).get('mode', 'hybrid')).lower()
207

```



```

208         raw_tel_mode = str(self._config.get('telemetry_mod
    e', self._config.get('profile', 'telemetry'))).lower()
209         self._telemetry_mode = raw_tel_mode if raw_tel_mod
    e in ('telemetry', 'tc') else 'telemetry'
210
211         raw_std_flush = self._config.get('standard_flush_i
    nterval_seconds', self._config.get('flush_interval_seconds', 3
    0.0))
212         self._standard_flush_interval_seconds = parse_inte
    rval_to_seconds(raw_std_flush, 30.0)
213
214         raw_tc_flush = self._config.get('tc_flush_interval
    _seconds', 5.0)
215         self._tc_flush_interval_seconds = parse_interval_t
    o_seconds(raw_tc_flush, 5.0)
216
217         raw_tc_timeout = self._config.get('tc_mode_timeout
    _seconds', self._config.get('tc_timeout_seconds', 300.0))
218         self._tc_mode_timeout_seconds = parse_interval_to_
    seconds(raw_tc_timeout, 300.0)
219
220         raw_tc_act = self._config.get('tc_mode_activated_a
    t')
221         try:
222             self._tc_mode_activated_at = float(raw_tc_act)
    if raw_tc_act is not None else None
223         except (ValueError, TypeError):
224             self._tc_mode_activated_at = None
225
226         raw_max_db = self._config.get('max_db_size_mb', se
    lf._config.get('max_file_size_mb', 100.0))
227         self._max_db_size_mb = float(raw_max_db) if raw_ma
    x_db is not None else 100.0
228
229         raw_retention = self._config.get('retention_days',
    7)
230         self._retention_days = int(raw_retention) if raw_r
    etention is not None else 7

```

```

231
232         raw_cleanup_interval = self._config.get('db_cleanup_interval_seconds')
233         self._db_cleanup_interval_seconds = parse_interval_to_seconds(raw_cleanup_interval, 300.0)
234
235         if self._telemetry_mode == 'tc':
236             self._flush_interval_seconds = self._tc_flush_interval_seconds
237         else:
238             self._flush_interval_seconds = self._standard_flush_interval_seconds
239
240         self._auto_vacuum_enabled = bool(self._config.get('auto_vacuum_enabled', True))
241     except FileNotFoundError:
242         if not self._config:
243             self._config = {}
244             self._sensors_config = {}
245             self._loggers_config = {}
246             self._default_interval = 5.0
247             self._heavy_interval = 60.0
248             self._heavy_disk_scan_interval = 43200.0
249             self._heavy_mode_max_duration_days = 5.0
250             self._heavy_mode_auto_switch_enabled = True
251             self._mode = 'hybrid'
252             self._max_db_size_mb = 50.0
253             self._retention_days = 7
254             self._db_cleanup_interval_seconds = 300.0
255             self._flush_interval_seconds = 30.0
256             self._auto_vacuum_enabled = True
257     except (json.JSONDecodeError, OSError, PermissionError) as e:
258         if not self._config:
259             raise ValueError(f'Ошибка парсинга {self._config_path}: {e}')
260         logger.warning(f'Временная ошибка считывания {self._config_path} на лету: {e}')

```

```

261
262     @property
263     def config_path(self) -> str:
264         """Возвращает путь к активному файлу конфигурации."""
265         return self._config_path
266
267     def check_and_reload(self) -> bool:
268         """Проверяет модификацию файла config.json на диске и
269         перезагружает параметры 'на лету'.
270
271         Returns:
272             bool: True если конфигурация изменилась и успешно
273             перезагружена, иначе False.
274
275         """
276         try:
277             cfg_path = Path(self._config_path)
278             if not cfg_path.is_file():
279                 return False
280             current_mtime_ns = cfg_path.stat().st_mtime_ns
281             if current_mtime_ns != self._last_mtime_ns:
282                 self._load_config()
283                 return True
284             except Exception as e:
285                 logger.debug(f'Проверка модификации config.json: {
286                     e}')
287                 return False
288
289     def reload(self) -> None:
290         """Принудительно перезагружает конфигурацию из файла."
291
292         """
293         self._load_config()
294
295     def get_data_path(self) -> Path:
296         """Возвращает настроенный или системный путь к директо
297         рии хранения данных телеметрии."""
298         custom = self._config.get('data_path') or self._config
299         .get('data_dir')
300         if not custom and isinstance(self._config.get('storage

```

```

    '), dict):
293         custom = self._config['storage'].get('data_path')
    or self._config['storage'].get('data_dir')
294         if custom:
295             return Path(os.path.expandvars(str(custom)))
296         programdata = os.environ.get('ProgramData') or os.envi
    ron.get('ALLUSERSPROFILE')
297         if programdata and os.path.exists(programdata):
298             base_dir = Path(programdata)
299         else:
300             appdata = os.environ.get('APPDATA') or os.environ.
    get('LOCALAPPDATA')
301             if appdata and os.path.exists(appdata):
302                 base_dir = Path(appdata)
303             else:
304                 base_dir = Path.home() / '.config'
305             return base_dir / 'AI-Breadboard' / 'apps' / 'windows'
    / 'telemetry' / 'logs'
306
307     def get_db_path(self) -> Path:
308         """Возвращает настроенный или системный путь к файлу S
    QLite базы данных телеметрии."""
309         if isinstance(self._config.get('storage'), dict):
310             custom_db = self._config['storage'].get('db_path')
311             if custom_db:
312                 return Path(os.path.expandvars(str(custom_db))
    )
313             custom_db = self._config.get('db_path')
314             if custom_db:
315                 return Path(os.path.expandvars(str(custom_db)))
316             return self.get_data_path() / 'telemetry.db'
317
318     def get_fast_interval(self) -> float:
319         """Возвращает интервал быстрого режима (сек)."""
320         return getattr(self, '_fast_interval', 30.0)
321
322     def get_fast_duration_days(self) -> int:
323         """Возвращает длительность быстрого режима в днях."""

```

```

324         return getattr(self, '_fast_duration_days', 2)
325
326     def get_aggregation_interval(self) -> float:
327         """Возвращает интервал агрегации (сек)."""
328         return getattr(self, '_aggregation_interval', 3600.0)
329
330     def get_mode(self) -> str:
331         """Возвращает режим сбора: minimal, hybrid, full."""
332         return self._mode if self._mode in ('minimal', 'hybrid', 'full') else 'hybrid'
333
334     def get_interval_seconds(self) -> float:
335         """Возвращает быстрый базовый интервал сбора телеметрии."""
336         return self._default_interval
337
338     def get_heavy_interval_seconds(self) -> float:
339         """Возвращает периодический интервал тяжелых сенсоров."""
340         return self._heavy_interval
341
342     def get_heavy_disk_scan_interval(self) -> float:
343         """Возвращает интервал тяжелой проверки дисков (в секундах)."""
344         return getattr(self, '_heavy_disk_scan_interval', 43200.0)
345
346     def get_heavy_mode_max_duration_days(self) -> float:
347         """Возвращает максимальную непрерывную длительность тяжелого режима (в днях)."""
348         return getattr(self, '_heavy_mode_max_duration_days', 5.0)
349
350     def is_heavy_auto_switch_enabled(self) -> bool:
351         """Возвращает флаг разрешения автопереключения тяжелого режима в легкий."""
352         return getattr(self, '_heavy_mode_auto_switch_enabled', True)

```

```

353
354     def get_heavy_collectors(self) -> Dict[str, bool]:
355         """Возвращает словарь флагов включения тяжелых коллект
опров."""
356         default_heavy = {
357             'hardware_sensors': True,
358             'deep_disk_smart': True,
359             'system_snapshot': True,
360             'hardware_tree': True,
361         }
362         custom = self._config.get('heavy_collectors', {})
363         if isinstance(custom, dict):
364             default_heavy.update(custom)
365         return default_heavy
366
367     def get_top_processes(self) -> int:
368         """Возвращает число сохраняемых процессов с наибольшей
нагрузкой."""
369         return int(self._config.get('top_processes', 25))
370
371     def get_buffer_mode(self) -> str:
372         """Возвращает режим буферизации телеметрии ('memory',
'file', 'direct')."""
373         mode = str(self._config.get('buffer_mode', 'memory')).
lower()
374         return mode if mode in ('memory', 'file', 'direct') el
se 'memory'
375
376     def get_buffer_size(self) -> int:
377         """Возвращает максимальный размер буфера перед принуди
тельным сбросом в БД."""
378         try:
379             val = int(self._config.get('buffer_size', 50))
380             return max(1, val)
381         except (ValueError, TypeError):
382             return 50
383
384     def get_telemetry_mode(self) -> str:

```

```

385 |         """Возвращает режим телеметрии: 'telemetry' (стандартн
    |   ый) или 'tc' (реальное время)."""
386 |         self.check_tc_mode_auto_switch()
387 |         return getattr(self, '_telemetry_mode', 'telemetry')
388 |
389 |     def check_tc_mode_auto_switch(self) -> bool:
390 |         """Проверяет таймаут активного режима 'tc' (реальное в
    |   ремя) и переключает в 'telemetry'.
391 |
    |   Returns:
392 |         bool: True если произошло автопереключение в 'tele
    |   metry', иначе False.
393 |         """
394 |         if getattr(self, '_telemetry_mode', 'telemetry') == 't
    |   c' and getattr(self, '_tc_mode_activated_at', None):
395 |             import time
396 |             elapsed = time.time() - self._tc_mode_activated_at
397 |             timeout = getattr(self, '_tc_mode_timeout_seconds'
    |   , 300.0)
398 |             if elapsed >= timeout:
399 |                 self._telemetry_mode = 'telemetry'
400 |                 self._tc_mode_activated_at = None
401 |                 self._flush_interval_seconds = getattr(self, '
    |   _standard_flush_interval_seconds', 30.0)
402 |                 self.save_config({
403 |                     'telemetry_mode': 'telemetry',
404 |                     'tc_mode_activated_at': None
405 |                 })
406 |                 logger.info(
407 |                     f" [Телеметрия] Режим реального времени '
    |   tc' завершен по таймауту ({int(timeout)}с). "
408 |                     f"Автоматически переключен в стандартный р
    |   ежим 'telemetry' (flush: {self._flush_interval_seconds}с, max_
    |   db: {self.get_max_db_size_mb()}МБ)"
409 |                 )
410 |                 return True
411 |             return False
412 |
413 |

```

```

414     def activate_tc_mode(self, duration_seconds: Optional[float] = None, flush_interval_seconds: Optional[float] = None) ->
    bool:
415         """Активирует режим отображения телеметрии реального в
    ремени ('tc') с задержкой сброса в N сек.
416
417         Args:
418             duration_seconds: Длительность режима ТС в секунда
    х до автопереключения в telemetry (по умолчанию 300с = 5 мин).
419             flush_interval_seconds: Интервал сброса в БД в сек
    ундах (по умолчанию 5.0с).
420
421         Returns:
422             bool: True при успешной активации.
423         """
424         import time
425         self._telemetry_mode = 'tc'
426         self._tc_mode_activated_at = time.time()
427         if flush_interval_seconds is not None:
428             self._tc_flush_interval_seconds = max(0.1, float(f
    lush_interval_seconds))
429         if duration_seconds is not None:
430             self._tc_mode_timeout_seconds = max(1.0, float(dur
    ation_seconds))
431         self._flush_interval_seconds = self._tc_flush_interval
    _seconds
432
433         updates: Dict[str, Any] = {
434             'telemetry_mode': 'tc',
435             'tc_mode_activated_at': self._tc_mode_activated_at
    ,
436             'tc_flush_interval_seconds': self._tc_flush_interv
    al_seconds,
437             'tc_mode_timeout_seconds': self._tc_mode_timeout_s
    econds,
438         }
439         logger.info(f" [Телеметрия] Активирован режим реально
    го времени 'tc' (flush: {self._tc_flush_interval_seconds}с, та

```



```

    ймаут: {self._tc_mode_timeout_seconds}c)")
440         return self.save_config(updates)
441
442     def deactivate_tc_mode(self) -> bool:
443         """Деактивирует режим 'tc' и возвращает телеметрию в с
тандартный режим 'telemetry'."""
444         self._telemetry_mode = 'telemetry'
445         self._tc_mode_activated_at = None
446         self._flush_interval_seconds = getattr(self, '_standar
d_flush_interval_seconds', 30.0)
447         updates = {
448             'telemetry_mode': 'telemetry',
449             'tc_mode_activated_at': None,
450         }
451         logger.info(f" [Телеметрия] Переключено в стандартны
й режим 'telemetry' (flush: {self._flush_interval_seconds}c)")
452         return self.save_config(updates)
453
454     def get_standard_flush_interval_seconds(self) -> float:
455         """Возвращает стандартный интервал сброса буфера в БД
(по умолчанию 30с)."""
456         return getattr(self, '_standard_flush_interval_seconds
', 30.0)
457
458     def get_tc_flush_interval_seconds(self) -> float:
459         """Возвращает интервал сброса в режиме реального време
ни ТС (по умолчанию 5с)."""
460         return getattr(self, '_tc_flush_interval_seconds', 5.0
)
461
462     def get_tc_mode_timeout_seconds(self) -> float:
463         """Возвращает таймаут действия режима ТС до автоперекл
ючения в стандартный (в секундах, по умолчанию 300с)."""
464         return getattr(self, '_tc_mode_timeout_seconds', 300.0
)
465
466     def get_flush_interval_seconds(self) -> float:
467         """Возвращает периодический интервал сброса буфера в Б

```

```

Д с учетом активного режима."""
468         self.check_tc_mode_auto_switch()
469         if getattr(self, '_telemetry_mode', 'telemetry') == 't
с':
470             return getattr(self, '_tc_flush_interval_seconds',
5.0)
471         return getattr(self, '_standard_flush_interval_seconds
', 30.0)
472
473     def get_max_db_size_mb(self) -> float:
474         """Возвращает максимальный разрешенный размер SQLite б
азы данных (в МБ, по умолчанию 100.0)."""
475         try:
476             val = float(self._config.get('max_db_size_mb', sel
f._config.get('max_file_size_mb', 100.0)))
477             return max(1.0, val)
478         except (ValueError, TypeError):
479             return 100.0
480
481     def get_retention_days(self) -> int:
482         """Возвращает срок хранения сырых записей телеметрии (
в днях)."""
483         try:
484             val = int(self._config.get('retention_days', 7))
485             return max(1, val)
486         except (ValueError, TypeError):
487             return 7
488
489     def get_db_cleanup_interval_seconds(self) -> float:
490         """Возвращает интервал периодической проверки и очистк
и БД (в секундах)."""
491         return getattr(self, '_db_cleanup_interval_seconds', 3
00.0)
492
493     def is_auto_vacuum_enabled(self) -> bool:
494         """Возвращает флаг разрешения авто-вакуума (VACUUM) SQ
Lite при усечении."""
495         return bool(self._config.get('auto_vacuum_enabled', Tr

```

```

ue))
496
497     def get_process_mode(self) -> str:
498         """Возвращает режим фильтрации процессов ('top_n' или
499         'all')."""
500         return str(self._config.get('process_mode', 'top_n')).
lower()
501
502     def get_top_processes_limit(self) -> int:
503         """Возвращает настроенный лимит количества Топ-процесс
504         ов (по умолчанию 10)."""
505         try:
506             val = int(self._config.get('top_processes', 10))
507             return max(1, val)
508         except (ValueError, TypeError):
509             return 10
510
511     def get_effective_process_limit(self) -> int:
512         """Возвращает эффективный лимит процессов для сохранен
513         ия в БД телеметрии."""
514         pm = self.get_process_mode()
515         if pm == 'all':
516             return 500
517         return self.get_top_processes_limit()
518
519     def get_sensor_config(self, sensor_name: str) -> Dict[str,
Any]:
520         """Возвращает конфигурацию конкретного сенсора."""
521         return self._sensors_config.get(sensor_name, {'enabled
': False})
522
523     def is_sensor_enabled(self, sensor_name: str) -> bool:
524         """Проверяет, включен ли сенсор."""
525         config = self.get_sensor_config(sensor_name)
526         return bool(config.get('enabled', False))
527
528     def get_sensor_interval(self, sensor_name: str) -> float:
529         """Возвращает интервал опроса сенсора в секундах."""

```

```

527         config = self.get_sensor_config(sensor_name)
528         raw = config.get('interval_seconds', config.get('inter
val', self._default_interval))
529         return parse_interval_to_seconds(raw, self._default_in
terval)
530
531     def get_sensor_metrics(self, sensor_name: str) -> List[str
]:
532         """Возвращает список метрик для сенсора."""
533         config = self.get_sensor_config(sensor_name)
534         return config.get('metrics', [])
535
536     def get_all_sensor_names(self) -> List[str]:
537         """Возвращает список всех имен сенсоров."""
538         return list(self._sensors_config.keys())
539
540     def get_enabled_sensors(self) -> List[str]:
541         """Возвращает список включенных сенсоров."""
542         return [name for name, config in self._sensors_config.
items() if config.get('enabled', False)]
543
544     def get_logger_interval(self, logger_name: str, default: f
loat = 60.0) -> float:
545         """Возвращает интервал опроса логгера приложения в сек
ундах."""
546         logger_info = self._loggers_config.get(logger_name, {}
)
547         if isinstance(logger_info, dict):
548             raw = logger_info.get('interval', self._config.get
('default_interval'))
549             return parse_interval_to_seconds(raw, default)
550         return float(default)
551
552     def is_logger_enabled(self, logger_name: str, default: boo
l = True) -> bool:
553         """Проверяет активность логгера приложения."""
554         logger_info = self._loggers_config.get(logger_name, {}
)

```

```

555         if isinstance(logger_info, dict):
556             return bool(logger_info.get('enabled', default))
557         return default
558
559     def get_all_intervals(self) -> Dict[str, Any]:
560         """Возвращает сводный словарь всех настроенных интерва
лов системы."""
561         return {
562             'telemetry_mode': self.get_telemetry_mode(),
563             'default_interval_seconds': self.get_interval_seco
nds(),
564             'heavy_interval_seconds': self.get_heavy_interval_
seconds(),
565             'fast_interval_seconds': self.get_fast_interval(),
566             'aggregation_interval_seconds': self.get_aggregati
on_interval(),
567             'heavy_disk_scan_interval_seconds': self.get_heavy
_disk_scan_interval(),
568             'db_cleanup_interval_seconds': self.get_db_cleanup
_interval_seconds(),
569             'flush_interval_seconds': self.get_flush_interval_
seconds(),
570             'standard_flush_interval_seconds': self.get_standar
d_flush_interval_seconds(),
571             'tc_flush_interval_seconds': self.get_tc_flush_int
erval_seconds(),
572             'tc_mode_timeout_seconds': self.get_tc_mode_timeou
t_seconds(),
573             'sensors': {
574                 name: self.get_sensor_interval(name)
575                 for name in self.get_all_sensor_names()
576             },
577             'loggers': {
578                 name: self.get_logger_interval(name)
579                 for name in self._loggers_config.keys()
580             }
581         }
582

```

```

583     def get_general_config(self) -> Dict[str, Any]:
584         """Возвращает общую конфигурацию (без сенсоров)."""
585         return {k: v for k, v in self._config.items() if k !=
'sensors'}
586
587     def get_config(self) -> Dict[str, Any]:
588         """Возвращает полную конфигурацию телеметрии."""
589         return self._config.copy()
590
591     @property
592     def config(self) -> Dict[str, Any]:
593         """Свойство для получения полной конфигурации."""
594         return self.get_config()
595
596     def save_config(self, new_config: Optional[Dict[str, Any]]
= None) -> bool:
597         """Сохраняет текущую или переданную конфигурацию в фай
л.
598
599         Args:
600             new_config: Опциональный словарь новых параметров.
601
602         Returns:
603             bool: True при успешном сохранении.
604         """
605         if new_config:
606             self._config.update(new_config)
607             if 'sensors' in new_config:
608                 self._sensors_config = new_config['sensors']
609             if 'loggers' in new_config:
610                 self._loggers_config = new_config['loggers']
611             if 'interval_seconds' in new_config:
612                 self._default_interval = parse_interval_to_sec
onds(new_config['interval_seconds'], 5.0)
613             if 'heavy_interval_seconds' in new_config:
614                 self._heavy_interval = parse_interval_to_secon
ds(new_config['heavy_interval_seconds'], 60.0)
615             if 'heavy_disk_scan_interval_seconds' in new_conf

```

```

g:
616         self._heavy_disk_scan_interval = parse_interva
l_to_seconds(new_config['heavy_disk_scan_interval_seconds'], 4
3200.0)
617         if 'fast_interval_seconds' in new_config:
618             self._fast_interval = parse_interval_to_second
s(new_config['fast_interval_seconds'], 30.0)
619         if 'aggregation_interval_seconds' in new_config:
620             self._aggregation_interval = parse_interval_to
_seconds(new_config['aggregation_interval_seconds'], 3600.0)
621         if 'heavy_mode_max_duration_days' in new_config:
622             self._heavy_mode_max_duration_days = float(new
_config['heavy_mode_max_duration_days'])
623         if 'heavy_mode_auto_switch_enabled' in new_config:
624             self._heavy_mode_auto_switch_enabled = bool(ne
w_config['heavy_mode_auto_switch_enabled'])
625         if 'mode' in new_config:
626             self._mode = str(new_config['mode']).lower()
627         if 'telemetry_mode' in new_config:
628             self._telemetry_mode = str(new_config['telemet
ry_mode']).lower()
629         if 'standard_flush_interval_seconds' in new_config
:
630             self._standard_flush_interval_seconds = parse_
interval_to_seconds(new_config['standard_flush_interval_second
s'], 30.0)
631         if 'tc_flush_interval_seconds' in new_config:
632             self._tc_flush_interval_seconds = parse_interv
al_to_seconds(new_config['tc_flush_interval_seconds'], 5.0)
633         if 'tc_mode_timeout_seconds' in new_config:
634             self._tc_mode_timeout_seconds = parse_interval
_to_seconds(new_config['tc_mode_timeout_seconds'], 300.0)
635         if 'tc_mode_activated_at' in new_config:
636             self._tc_mode_activated_at = float(new_config[
'tc_mode_activated_at']) if new_config['tc_mode_activated_at']
is not None else None
637         if 'max_db_size_mb' in new_config:
638             self._max_db_size_mb = float(new_config['max_d

```

```

        b_size_mb']])
639         if 'retention_days' in new_config:
640             self._retention_days = int(new_config['retenti
on_days'])
641         if 'db_cleanup_interval_seconds' in new_config:
642             self._db_cleanup_interval_seconds = parse_inte
rval_to_seconds(new_config['db_cleanup_interval_seconds'], 300
.0)
643         if 'flush_interval_seconds' in new_config:
644             self._flush_interval_seconds = parse_interval_
to_seconds(new_config['flush_interval_seconds'], 30.0)
645         if 'auto_vacuum_enabled' in new_config:
646             self._auto_vacuum_enabled = bool(new_config['a
uto_vacuum_enabled'])
647         try:
648             target_path = Path(self._config_path)
649             target_path.parent.mkdir(parents=True, exist_ok=Tr
ue)
650             with open(target_path, 'w', encoding='utf-8') as f
:
651                 json.dump(self._config, f, indent=2, ensure_as
cii=False)
652             if target_path.is_file():
653                 self._last_mtime_ns = target_path.stat().st_mt
ime_ns
654             return True
655         except Exception:
656             return False

```


w64_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - W64 Collector
4 # =====
5 # Description:
6 #   AI Windows 64-bit Collector - модуль сбора детальных системных событий телеметрии.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.w64_collector import AIW64Collector
11 #
12 #       service = AIW64Collector()
13 #
14 # File: w64_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 07:35:00
20 # =====
21
22 from __future__ import annotations
23 """AI Windows 64-bit Collector - модуль сбора детальных системных событий телеметрии."""
24
25 import json
26 import os
27 import re
28 import subprocess
29 import threading
```

```

30 import time
31 from datetime import datetime, timezone
32 from pathlib import Path
33 from typing import Any, Callable, Dict, List, Optional
34 from logger import logger
35 from .models import W64CollectorStatus, W64SystemEvent
36 from .sqlite import TelemetryStorage
37
38
39 class AIW64Collector:
40     """Максимально детальный сборщик событий Windows 64-bit дл
я подсистемы телеметрии.
41
42     Фиксирует непрерывные изменения системного состояния:
43     - Процессы (создание, завершение, активность)
44     - Файловая система (изменения в отслеживаемых путях)
45     - Реестр Windows (модификации ключей)
46     - Сетевая активность (новые соединения и открытые порты)
47     """
48
49     def __init__(
50         self,
51         enable_file_monitoring: bool = True,
52         enable_process_monitoring: bool = True,
53         enable_registry_monitoring: bool = True,
54         enable_network_monitoring: bool = True,
55         enable_event_log_monitoring: bool = True,
56         monitored_paths: Optional[List[str]] = None,
57         poll_interval_sec: float = 1.0,
58         storage: Optional[TelemetryStorage] = None,
59         on_event_callback: Optional[Callable[[Dict[str, Any]],
None]] = None,
60         **kwargs: Any,
61     ) -> None:
62         """Инициализация сборщика системных событий W64.
63
64         Args:
65             enable_file_monitoring: Флаг мониторинга изменений

```

```

        файлов.
66         enable_process_monitoring: Флаг мониторинга процес
сов.
67         enable_registry_monitoring: Флаг мониторинга реест
ра.
68         enable_network_monitoring: Флаг мониторинга сетевы
х соединений.
69         enable_event_log_monitoring: Флаг мониторинга журн
алов Windows.
70         monitored_paths: Список путей файловой системы для
отслеживания.
71         poll_interval_sec: Интервал между циклами сканиров
ания изменений (сек).
72         storage: Экземпляр хранилища TelemetryStorage для
сохранения в SQLite через буфер.
73         on_event_callback: Опциональный callback-обработчи
к каждого зафиксированного события.
74         **kwargs: Дополнительные параметры конфигурации.
75         """
76         self.enable_file_monitoring = enable_file_monitoring
77         self.enable_process_monitoring = enable_process_monito
ring
78         self.enable_registry_monitoring = enable_registry_moni
toring
79         self.enable_network_monitoring = enable_network_monito
ring
80         self.enable_event_log_monitoring = enable_event_log_mo
nitoring
81         self.poll_interval = max(0.1, poll_interval_sec)
82         self.storage = storage or TelemetryStorage.get_instanc
e()
83         self.on_event_callback = on_event_callback
84
85         self.monitored_paths = monitored_paths or [
86             os.path.expandvars(r'%USERPROFILE%\Documents'),
87             os.path.expandvars(r'%USERPROFILE%\Downloads'),
88             os.path.expandvars(r'%USERPROFILE%\Desktop'),
89             r'C:\Program Files',

```

```

90         r'C:\Program Files (x86)',
91         r'C:\Windows\System32',
92         r'C:\Windows\SysWOW64',
93     ]
94
95     self._running = False
96     self._thread: Optional[threading.Thread] = None
97     self._stop_event = threading.Event()
98     self._event_counter = 0
99     self._last_event_time: Optional[str] = None
100     self._last_baseline: Dict[str, Any] = {}
101     logger.debug('AIW64Collector инициализирован (буфериза
ция в оперативной памяти)')
102
103     def _log_event(self, event_type: str, event_data: Dict[str
, Any]) -> None:
104         """Зафиксировать событие телеметрии в оперативной памя
ти с пакетным сбросом в SQLite.
105
106         Args:
107             event_type: Тип события.
108             event_data: Словарь с атрибутами события.
109         """
110         self._event_counter += 1
111         now_iso = datetime.now(timezone.utc).isoformat()
112         event_data['event_id'] = f'evt_{int(time.time())}_{sel
f._event_counter}'
113         event_data['timestamp'] = now_iso
114         event_data['event_type'] = event_type
115         self._last_event_time = now_iso
116
117         # 1. Сохранение в буфер памяти TelemetryStorage
118         if self.storage:
119             try:
120                 self.storage.save_w64_event(event_data, provid
er='w64_collector')
121             except Exception as ex:
122                 logger.debug(f'Ошибка сохранения W64 события в

```

```

    БД: {ex}')
123
124     # 2. Передача в callback при наличии
125     if self.on_event_callback:
126         try:
127             self.on_event_callback(event_data)
128         except Exception as ex:
129             logger.debug(f'Ошибка в on_event_callback: {ex
    }')
130
131     def start(self) -> bool:
132         """Запустить фоновый поток мониторинга событий.
133
134         Returns:
135             bool: True если сборщик успешно запущен, False есл
и уже работал.
136         """
137         if self._running:
138             logger.warning('AIW64Collector уже запущен')
139             return False
140
141         self._running = True
142         self._stop_event.clear()
143         self._update_baseline()
144         self._thread = threading.Thread(target=self._monitorin
g_loop, name='AIW64CollectorWorker', daemon=True)
145         self._thread.start()
146         logger.info('AIW64Collector успешно запущен')
147         return True
148
149     def stop(self) -> bool:
150         """Остановить фоновый поток мониторинга.
151
152         Returns:
153             bool: True если сборщик остановлен, False если не
был активен.
154         """
155         if not self._running:

```

```

156         return False
157
158     self._running = False
159     self._stop_event.set()
160     if self._thread and self._thread.is_alive():
161         self._thread.join(timeout=5.0)
162     logger.info(f'AIW64Collector остановлен. Всего зафиксир
163 ровано событий: {self._event_counter}')
164     return True
165
166     def _update_baseline(self) -> None:
167         """Сформировать начальный снимок состояния системы для
168 сравнения."""
169         self._last_baseline = {
170             'processes': self._get_current_processes(),
171             'network_connections': self._get_current_network_c
172 onnections(),
173             'registry_keys': self._get_registry_keys(),
174             'file_hashes': self._get_file_hashes(),
175         }
176
177     def _monitoring_loop(self) -> None:
178         """Основной рабочий цикл мониторинга и выявления измен
179 ений."""
180         last_check = time.time()
181         while not self._stop_event.is_set():
182             try:
183                 current_time = time.time()
184                 if current_time - last_check >= self.poll_inte
185 rval:
186                     current_state = {
187                         'processes': self._get_current_process
188 es(),
189                         'network_connections': self._get_curre
190 nt_network_connections(),
191                         'registry_keys': self._get_registry_ke
192 ys(),
193                         'file_hashes': self._get_file_hashes()

```

```

186         }
187         self._detect_changes(self._last_baseline,
current_state)
188         self._last_baseline = current_state
189         last_check = current_time
190
191         self._stop_event.wait(timeout=min(0.5, self.po
ll_interval))
192     except Exception as ex:
193         logger.error(f'Ошибка в цикле мониторинга AIW6
4Collector: {ex}')
194
195     def _detect_changes(self, old_state: Dict[str, Any], new_s
tate: Dict[str, Any]) -> None:
196         """Сравнить предыдущее и текущее состояние и залогиров
ать изменения.
197
198         Args:
199             old_state: Предыдущий системный срез.
200             new_state: Текущий системный срез.
201         """
202         # Мониторинг процессов
203         if self.enable_process_monitoring:
204             old_procs = {p['pid']: p for p in old_state.get('p
rocesses', [])}
205             new_procs = {p['pid']: p for p in new_state.get('p
rocesses', [])}
206
207             for pid, proc in new_procs.items():
208                 if pid not in old_procs:
209                     self._log_event('process_start', proc)
210
211             for pid, proc in old_procs.items():
212                 if pid not in new_procs:
213                     self._log_event('process_stop', proc)
214
215         # Мониторинг сетевых соединений

```

```
216         if self.enable_network_monitoring:
217             old_conns = {c.get('key'): c for c in old_state.ge
218 t('network_connections', []) if c.get('key')}
219             new_conns = {c.get('key'): c for c in new_state.ge
220 t('network_connections', []) if c.get('key')}
221
222             for key, conn in new_conns.items():
223                 if key not in old_conns:
224                     self._log_event('network_open', conn)
225
226             for key, conn in old_conns.items():
227                 if key not in new_conns:
228                     self._log_event('network_close', conn)
229
230     # Мониторинг файлов
231     if self.enable_file_monitoring:
232         old_files = old_state.get('file_hashes', {})
233         new_files = new_state.get('file_hashes', {})
234
235         for path, h in new_files.items():
236             if path not in old_files:
237                 self._log_event('file_created', {'path': p
238 ath, 'hash': h})
239             elif old_files[path] != h:
240                 self._log_event('file_modified', {'path':
241 path, 'old_hash': old_files[path], 'new_hash': h})
242
243         for path, h in old_files.items():
244             if path not in new_files:
245                 self._log_event('file_deleted', {'path': p
246 ath, 'hash': h})
247
248     # Мониторинг реестра
249     if self.enable_registry_monitoring:
250         old_reg = old_state.get('registry_keys', {})
251         new_reg = new_state.get('registry_keys', {})
252
253         for key, val in new_reg.items():
```



```

249         if key not in old_reg:
250             self._log_event('registry_created', {'key'
: key, 'value': val})
251         elif old_reg[key] != val:
252             self._log_event('registry_modified', {'key
': key, 'old_value': old_reg[key], 'new_value': val})
253
254         for key, val in old_reg.items():
255             if key not in new_reg:
256                 self._log_event('registry_deleted', {'key'
: key, 'value': val})
257
258     def _get_current_processes(self) -> List[Dict[str, Any]]:
259         """Получить текущий список активных процессов в систем
e."""
260         processes = []
261         try:
262             import psutil
263             for proc in psutil.process_iter(['pid', 'name', 'e
xe', 'cmdline', 'username', 'create_time']):
264                 try:
265                     info = proc.info
266                     processes.append({
267                         'pid': info.get('pid'),
268                         'name': info.get('name') or '',
269                         'path': info.get('exe') or '',
270                         'cmdline': ' '.join(info.get('cmdline'
) or []) if info.get('cmdline') else '',
271                         'username': info.get('username') or ''
272                         ,
273                         'create_time': info.get('create_time')
or 0.0,
274                     })
275                 except (psutil.NoSuchProcess, psutil.AccessDen
ied):
276                     continue
277             except Exception as ex:
278                 logger.debug(f'Ошибка при получении списка процесс

```

```

    ов: {ex}')
278         return processes
279
280     def _get_current_network_connections(self) -> List[Dict[str, Any]]:
281         """Получить текущие активные сетевые подключения."""
282         connections = []
283         try:
284             import psutil
285             for conn in psutil.net_connections(kind='inet'):
286                 try:
287                     laddr = f'{conn.laddr.ip}:{conn.laddr.port}' if conn.laddr else ''
288                     raddr = f'{conn.raddr.ip}:{conn.raddr.port}' if conn.raddr else ''
289                     key = f'{conn.type}_{laddr}_{raddr}_{conn.pid}'
290                     connections.append({
291                         'key': key,
292                         'fd': conn.fd,
293                         'family': conn.family.name if hasattr(conn.family, 'name') else str(conn.family),
294                         'type': conn.type.name if hasattr(conn.type, 'name') else str(conn.type),
295                         'laddr': laddr,
296                         'raddr': raddr,
297                         'status': conn.status,
298                         'pid': conn.pid,
299                     })
300                 except Exception:
301                     continue
302             except Exception as ex:
303                 logger.debug(f'Ошибка при получении сетевых соединений: {ex}')
304         return connections
305
306     def _get_registry_keys(self) -> Dict[str, Any]:
307         """Получить контрольные ключи реестра автозагрузки и с

```

```

        ИСТЕМНЫХ ПОЛИТИК. """
308         keys = {}
309         if os.name != 'nt':
310             return keys
311         try:
312             import winreg
313             hives = [
314                 (winreg.HKEY_CURRENT_USER, r'Software\Microsof
315                 t\Windows\CurrentVersion\Run'),
316                 (winreg.HKEY_LOCAL_MACHINE, r'Software\Microso
317                 ft\Windows\CurrentVersion\Run'),
318                 (winreg.HKEY_LOCAL_MACHINE, r'Software\Microso
319                 ft\Windows\CurrentVersion\RunOnce'),
320             ]
321             for hive, subkey in hives:
322                 try:
323                     with winreg.OpenKey(hive, subkey) as k:
324                         idx = 0
325                         while True:
326                             try:
327                                 name, val, _ = winreg.EnumValu
328                                 e(k, idx)
329                                 key_path = f'{subkey}\\{name}'
330                                 keys[key_path] = str(val)
331                                 idx += 1
332                             except OSError:
333                                 break
334                         except OSError:
335                             continue
336                     except Exception as ex:
337                         logger.debug(f'Ошибка при чтении ключей реестра: {
338                         ex}')
339             return keys
340
341         def _get_file_hashes(self) -> Dict[str, str]:
342             """Получить отпечатки и размеры отслеживаемых файлов д
343             ля быстрого детекта изменений. """
344             hashes = {}

```

```

339         for root_path in self.monitored_paths:
340             p = Path(root_path)
341             if not p.exists():
342                 continue
343             try:
344                 # Ограничиваем сканирование первыми 100 файлам
и для предотвращения перегрузки
345                 count = 0
346                 for item in p.rglob('*'):
347                     if count > 100:
348                         break
349                     if item.is_file():
350                         try:
351                             stat = item.stat()
352                             hashes[str(item)] = f'{stat.st_siz
e}_{stat.st_mtime}'
353                             count += 1
354                         except OSError:
355                             continue
356                     except Exception as ex:
357                         logger.debug(f'Ошибка при проверке файлов в {r
oot_path}: {ex}')
358                 return hashes
359
360     def get_status(self) -> Dict[str, Any]:
361         """Получить диагностический статус сборщика.
362
363         Returns:
364             Dict[str, Any]: Словарь со статусом и счетчиками.
365         """
366         status = W64CollectorStatus(
367             running=self._running,
368             events_count=self._event_counter,
369             last_event_time=self._last_event_time,
370         )
371         return status.model_dump()
372
373     def get_events(self, event_type: Optional[str] = None, lim

```

```

        it: int = 100) -> List[Dict[str, Any]]:
374         """Получить список последних событий из SQLite хранилища телеметрии.
375
376         Args:
377             event_type: Фильтр по типу события.
378             limit: Максимальное число записей.
379
380         Returns:
381             List[Dict[str, Any]]: Список событий.
382         """
383         if self.storage:
384             return self.storage.get_w64_events(event_type=event_type, provider='w64_collector', limit=limit)
385         return []
386
387
388 _global_w64_collector: Optional[AIW64Collector] = None
389 _lock = threading.Lock()
390
391 def get_w64_collector(**kwargs: Any) -> AIW64Collector:
392     """Получить глобальный синглтон сборщика AIW64Collector.
393
394     Args:
395         **kwargs: Аргументы инициализации при первом создании.
396
397     Returns:
398         AIW64Collector: Синглтон сборщика.
399     """
400     global _global_w64_collector
401     with _lock:
402         if _global_w64_collector is None:
403             _global_w64_collector = AIW64Collector(**kwargs)
404         return _global_w64_collector
405
406 def start_w64_collector(**kwargs: Any) -> bool:
407     """Запустить глобальный сборщик AIW64Collector."""
408     collector = get_w64_collector(**kwargs)

```

```
409 |     return collector.start()
410 |
411 | def stop_w64_collector() -> bool:
412 |     """Остановить глобальный сборщик AIW64Collector."""
413 |     global _global_w64_collector
414 |     with _lock:
415 |         if _global_w64_collector:
416 |             res = _global_w64_collector.stop()
417 |             _global_w64_collector = None
418 |             return res
419 |     return False
420 |
```

w64_etw_collector.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - W64 Etw
  Collector
4 # =====
5 # Description:
6 #   ETW Collector для сбора низкоуровневых событий ядра и журна
  лов аудита Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.w64_etw_collector import AIW
  64ETWCollector
11 #
12 #       service = AIW64ETWCollector()
13 #
14 # File: w64_etw_collector.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 07:35:00
20 # =====
21
22 from __future__ import annotations
23 """ETW Collector для сбора низкоуровневых событий ядра и журна
  лов аудита Windows."""
24
25 import json
26 import os
27 import threading
28 import time
29 from collections import deque
```

```

30 from datetime import datetime, timezone
31 from pathlib import Path
32 from typing import Any, Callable, Dict, List, Optional
33 from logger import logger
34 from .models import ETWTraceEvent, ProcessLifecycleEvent, W64C
    collectorStatus
35 from .sqlite import TelemetryStorage
36 from .win32_fffi.wevtapi import WevtAPI
37
38
39 class AIW64ETWCollector:
40     """Сборщик низкоуровневых событий через ETW и журналы ауди
    та Windows."""
41
42     def __init__(
43         self,
44         enable_process_trace: bool = True,
45         enable_disk_trace: bool = True,
46         enable_network_trace: bool = True,
47         enable_registry_trace: bool = True,
48         poll_interval_sec: float = 5.0,
49         storage: Optional[TelemetryStorage] = None,
50         on_event_callback: Optional[Callable[[Dict[str, Any]],
    None]] = None,
51         **kwargs: Any,
52     ) -> None:
53         """Инициализация ETW сборщика телеметрии.
54
55         Args:
56             enable_process_trace: Трассировка создания/заверше
    ния процессов (ID 4688).
57             enable_disk_trace: Трассировка операций файлового
    аудита (ID 4663).
58             enable_network_trace: Трассировка сетевых подключе
    ний Windows Filtering Platform.
59             enable_registry_trace: Трассировка изменений ключе
    й аудита реестра.
60             poll_interval_sec: Интервал между выборками событи

```


й ETW (сек).

```
61         storage: Экземпляр хранилища TelemetryStorage.
62         on_event_callback: Callback-обработчик каждого зафиксированного события.
63         **kwargs: Дополнительные параметры конфигурации.
64         """
65         self.enable_process_trace = enable_process_trace
66         self.enable_disk_trace = enable_disk_trace
67         self.enable_network_trace = enable_network_trace
68         self.enable_registry_trace = enable_registry_trace
69         self.poll_interval = max(1.0, poll_interval_sec)
70         self.storage = storage or TelemetryStorage.get_instance()
71         self.on_event_callback = on_event_callback
72
73         self.wevt_api = WevtAPI() if os.name == 'nt' else None
74         self._seen_records: set[tuple[str, int]] = set()
75         self._seen_queue: deque[tuple[str, int]] = deque(maxlen=5000)
76
77         self._running = False
78         self._thread: Optional[threading.Thread] = None
79         self._stop_event = threading.Event()
80         self._event_counter = 0
81         self._last_event_time: Optional[str] = None
82         logger.debug('AIW64ETWCollector инициализирован (буферизация в оперативной памяти)')
83
84     def start(self) -> bool:
85         """Запустить фоновый поток сбора ETW событий.
86
87         Returns:
88             bool: True если сборщик успешно запущен, False если уже работал.
89         """
90         if self._running:
91             logger.warning('AIW64ETWCollector уже запущен')
92             return False
```

```

93
94     self._running = True
95     self._stop_event.clear()
96     self._thread = threading.Thread(target=self._etw_loop,
name='AIW64ETWWorker', daemon=True)
97     self._thread.start()
98     logger.info('AIW64ETWCollector успешно запущен')
99     return True
100
101     def stop(self) -> bool:
102         """Остановить фоновый поток ETW сбора.
103
104         Returns:
105             bool: True если сборщик остановлен, False если не
был активен.
106         """
107         if not self._running:
108             return False
109
110         self._running = False
111         self._stop_event.set()
112         if self._thread and self._thread.is_alive():
113             self._thread.join(timeout=5.0)
114         logger.info(f'AIW64ETWCollector остановлен. Всего зафи
ксировано событий: {self._event_counter}')
115         return True
116
117     def _etw_loop(self) -> None:
118         """Основной рабочий цикл выборки событий ETW."""
119         while not self._stop_event.is_set():
120             try:
121                 self._collect_etw_events()
122                 self._stop_event.wait(timeout=self.poll_interv
al)
123             except Exception as ex:
124                 logger.error(f'Ошибка в цикле ETW сборщика: {e
x}')
125

```

```

126     def _collect_etw_events(self) -> None:
127         """Собрать порцию событий из включенных провайдеров."""
128         if os.name != 'nt' or not self.wevt_api or not self.wevt_api.is_available():
129             return
130
131         if self.enable_process_trace:
132             self._collect_process_events()
133         if self.enable_disk_trace:
134             self._collect_file_events()
135         if self.enable_network_trace:
136             self._collect_network_events()
137         if self.enable_registry_trace:
138             self._collect_registry_events()
139
140     def _collect_security_events(self, event_ids: int | list[int], event_type: str, max_events: int = 50) -> None:
141         """Собрать порцию событий из Security журнала Windows через WevtAPI.
142
143         Args:
144             event_ids: Идентификатор или список идентификаторов событий.
145             event_type: Название типа события для логирования.
146             max_events: Максимальное количество событий в выборке.
147         """
148         if not self.wevt_api or not self.wevt_api.is_available():
149             return
150
151         try:
152             raw_events = self.wevt_api.read_events(
153                 channel='Security',
154                 limit=max_events,
155                 event_id=event_ids,
156

```

```

157         for ev in raw_events:
158             rec_id = ev.get('record_id')
159             if rec_id:
160                 key = ('Security', rec_id)
161                 if key in self._seen_records:
162                     continue
163                 if len(self._seen_queue) == self._seen_queue.
ue.maxlen:
164                     oldest = self._seen_queue[0]
165                     self._seen_records.discard(oldest)
166                     self._seen_queue.append(key)
167                     self._seen_records.add(key)
168
169                 event_data = ev.get('event_data', {})
170                 if event_data:
171                     self._log_event(event_type, event_data)
172             except Exception as ex:
173                 logger.debug(f'Ошибка сбора событий {event_type} E
TW: {ex}')
174
175     def _collect_process_events(self) -> None:
176         """Собрать события создания процессов (Security ID 468
8)."""
177         self._collect_security_events(4688, 'etw_process_creat
e')
178
179     def _collect_file_events(self) -> None:
180         """Собрать события файловых операций (Security ID 4663
)."""
181         self._collect_security_events(4663, 'etw_file_access')
182
183     def _collect_network_events(self) -> None:
184         """Собрать события сетевых блокировок/подключений WFP
(Security ID 5156/5158)."""
185         self._collect_security_events([5156, 5158], 'etw_netwo
rk_connection')
186
187     def _collect_registry_events(self) -> None:

```

```

188 |         """Собрать события аудита реестра (Security ID 4657)."""
189 |         self._collect_security_events(4657, 'etw_registry_access')
190 |
191 |     def _log_event(self, event_type: str, event_data: Dict[str, Any]) -> None:
192 |         """Зафиксировать событие ETW в оперативной памяти с пакетным сбросом в SQLite.
193 |
194 |         Args:
195 |             event_type: Тип события ETW.
196 |             event_data: Словарь с полями из журнала событий.
197 |         """
198 |         self._event_counter += 1
199 |         now_iso = datetime.now(timezone.utc).isoformat()
200 |
201 |         payload = {
202 |             'event_id': f'etw_{int(time.time())}_{self._event_counter}',
203 |             'timestamp': now_iso,
204 |             'event_type': event_type,
205 |             'provider': 'Microsoft-Windows-Security-Auditing',
206 |             'data': event_data,
207 |         }
208 |         self._last_event_time = now_iso
209 |
210 |         # 1. Сохранение в SQLite TelemetryStorage через буфер
211 |         # памяти
212 |         if self.storage:
213 |             try:
214 |                 db_record = {
215 |                     'event_id': payload['event_id'],
216 |                     'timestamp': now_iso,
217 |                     'event_type': event_type,
218 |                     'path': event_data.get('ObjectName') or event_data.get('NewProcessName'),
219 |                     'pid': event_data.get('ProcessId') or event_data.get('ProcessName')

```

```

    t_data.get('NewProcessId'),
219         'name': event_data.get('ProcessName') or e
vent_data.get('SubjectUserName'),
220         'details': event_data,
221     }
222     self.storage.save_w64_event(db_record, provide
r='w64_etw_collector')
223
224     # Специфическая фиксация создания процессов дл
я Process Provenance
225     if event_type == 'etw_process_create':
226         def _safe_pid(val: Any) -> int:
227             if val is None:
228                 return 0
229             if isinstance(val, int):
230                 return val
231             s = str(val).strip()
232             if s.startswith(('0x', '0X')):
233                 try:
234                     return int(s, 16)
235                 except ValueError:
236                     return 0
237             try:
238                 return int(s)
239             except ValueError:
240                 return 0
241
242         new_pid = _safe_pid(event_data.get('NewPro
cessId') or event_data.get('ProcessId'))
243         parent_pid = _safe_pid(event_data.get('Pro
cessId') or event_data.get('ParentProcessId'))
244         exe_path = event_data.get('NewProcessName'
) or event_data.get('ProcessName') or ''
245         proc_name = os.path.basename(exe_path) if
exe_path else 'unknown'
246         parent_exe = event_data.get('ParentProcess
Name') or ''
247         parent_name = os.path.basename(parent_exe)

```

```

    if parent_exe else None
248         u_name = event_data.get('SubjectUserName')
249         d_name = event_data.get('SubjectDomainName
    ')
250         full_user = f'{d_name}\\{u_name}' if d_name and u_name and d_name != '-' else u_name
251         u_sid = event_data.get('SubjectUserSid')
252         cmdline = event_data.get('CommandLine')
253         integrity = event_data.get('MandatoryLabel
    ')
254
255         prov_event = ProcessLifecycleEvent(
256             event_type='ProcessCreated',
257             process_guid=f'proc_{new_pid}_{int(time.
    time())}',
258             pid=new_pid,
259             name=proc_name,
260             executable_path=exe_path,
261             command_line=cmdline,
262             user=full_user,
263             sid=u_sid,
264             integrity_level=integrity,
265             parent_pid=parent_pid,
266             parent_name=parent_name,
267             created_at=now_iso,
268             details=event_data,
269         )
270         self.storage.save_process_provenance_event
    (prov_event)
271     except Exception as ex:
272         logger.debug(f'Ошибка сохранения ETW события в
    БД: {ex}')
273
274     # 2. Передача в callback
275     if self.on_event_callback:
276         try:
277             self.on_event_callback(payload)
278         except Exception as ex:

```

```

279         logger.debug(f'Ошибка в ETW on_event_callback:
    {ex}')
280
281     def get_status(self) -> Dict[str, Any]:
282         """Получить диагностический статус ETW сборщика.
283
284         Returns:
285             Dict[str, Any]: Словарь со статусом и счетчиками.
286         """
287         status = W64CollectorStatus(
288             running=self._running,
289             events_count=self._event_counter,
290             last_event_time=self._last_event_time,
291         )
292         return status.model_dump()
293
294     def get_events(self, event_type: Optional[str] = None, limit: int = 100) -> List[Dict[str, Any]]:
295         """Получить список последних событий ETW из SQLite хранилища.
296
297         Args:
298             event_type: Фильтр по типу события.
299             limit: Максимальное число записей.
300
301         Returns:
302             List[Dict[str, Any]]: Список событий.
303         """
304         if self.storage:
305             return self.storage.get_w64_events(event_type=event_type, provider='w64_etw_collector', limit=limit)
306         return []
307

```


advapi32.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  | - Advapi32
4 # =====
5 # Description:
6 #   advapi32.dll wrapper for registry, services, and security
  | functions.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ffadvapi32 import SY
  | STEMTIME
11 #
12 #       service = SYSTEMTIME()
13 #
14 # File: advapi32.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ffadvapi32
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """advapi32.dll wrapper for registry, services, and security f
  | unctions."""
23
24 import ctypes
25 import ctypes.wintypes as wintypes
26 from typing import List, Optional, Dict, Any
27 from dataclasses import dataclass
28 from enum import IntFlag
29 HKEY_CLASSES_ROOT = 2147483648
```

```

30 HKEY_CURRENT_USER = 2147483649
31 HKEY_LOCAL_MACHINE = 2147483650
32 HKEY_USERS = 2147483651
33 HKEY_PERFORMANCE_DATA = 2147483652
34 HKEY_CURRENT_CONFIG = 2147483653
35 KEY_READ = 131097
36 KEY_WRITE = 131078
37 KEY_QUERY_VALUE = 1
38 KEY_ENUMERATE_SUB_KEYS = 8
39 SERVICE_QUERY_STATUS = 4
40 SERVICE_QUERY_CONFIG = 1
41 SERVICE_AUTO_START = 2
42 SERVICE_DEMAND_START = 3
43 SERVICE_DISABLED = 4
44 SERVICE_RUNNING = 4
45 SERVICE_STOPPED = 1
46
47 class SYSTEMTIME(ctypes.Structure):
48     _fields_ = [('wYear', wintypes.WORD), ('wMonth', wintypes.
WORD), ('wDayOfWeek', wintypes.WORD), ('wDay', wintypes.WORD),
    ('wHour', wintypes.WORD), ('wMinute', wintypes.WORD), ('wSeco
nd', wintypes.WORD), ('wMilliseconds', wintypes.WORD)]
49
50 class SERVICE_STATUS(ctypes.Structure):
51     _fields_ = [('dwServiceType', wintypes.DWORD), ('dwCurrent
State', wintypes.DWORD), ('dwControlsAccepted', wintypes.DWORD
), ('dwWin32ExitCode', wintypes.DWORD), ('dwServiceSpecificExi
tCode', wintypes.DWORD), ('dwCheckPoint', wintypes.DWORD), ('d
wWaitHint', wintypes.DWORD)]
52
53 class ENUM_SERVICE_STATUS(ctypes.Structure):
54     _fields_ = [('lpServiceName', wintypes.LPWSTR), ('lpDispla
yName', wintypes.LPWSTR), ('ServiceStatus', SERVICE_STATUS)]
55
56 @dataclass
57 class RegistryValue:
58     """Registry value information."""
59     name: str

```

```

60     value_type: int
61     data: Any
62
63 @dataclass
64 class ServiceInfo:
65     """Service information."""
66     name: str
67     display_name: str
68     state: str
69     service_type: str
70     start_type: str
71
72 class Advapi32API:
73     """Wrapper for advapi32 registry and service functions."""
74
75     def __init__(self):
76         self.advapi32 = ctypes.windll.advapi32
77         self._setup_function_signatures()
78
79     def _setup_function_signatures(self):
80         """Configure ctypes function signatures."""
81         self.advapi32.RegOpenKeyExW.argtypes = [wintypes.HKEY,
82         wintypes.LPCWSTR, wintypes.DWORD, wintypes.DWORD, ctypes.POIN
83         TER(wintypes.HKEY)]
84         self.advapi32.RegOpenKeyExW.restype = wintypes.LONG
85         self.advapi32.RegQueryValueExW.argtypes = [wintypes.HK
86         EY, wintypes.LPCWSTR, wintypes.LPVOID, ctypes.POINTER(wintypes
87         .DWORD), wintypes.LPVOID, ctypes.POINTER(wintypes.DWORD)]
88         self.advapi32.RegQueryValueExW.restype = wintypes.LONG
89         self.advapi32.RegEnumKeyExW.argtypes = [wintypes.HKEY,
90         wintypes.DWORD, wintypes.LPWSTR, ctypes.POINTER(wintypes.DWOR
91         D), ctypes.POINTER(wintypes.DWORD), wintypes.LPWSTR, ctypes.PO
92         INTER(wintypes.DWORD), ctypes.POINTER(SYSTEMTIME)]
93         self.advapi32.RegEnumKeyExW.restype = wintypes.LONG
94         self.advapi32.RegCloseKey.argtypes = [wintypes.HKEY]
95         self.advapi32.RegCloseKey.restype = wintypes.LONG
96         self.advapi32.OpenSCManagerW.argtypes = [wintypes.LPCW
97         STR, wintypes.LPCWSTR, wintypes.DWORD]

```

```

90         self.advapi32.OpenSCManagerW.restype = wintypes.SC_HAN
DLE
91         self.advapi32.OpenServiceW.argtypes = [wintypes.SC_HAN
DLE, wintypes.LPCWSTR, wintypes.DWORD]
92         self.advapi32.OpenServiceW.restype = wintypes.SC_HAN
DLE
93         self.advapi32.QueryServiceStatus.argtypes = [wintypes.
SC_HANDLE, ctypes.POINTER(SERVICE_STATUS)]
94         self.advapi32.QueryServiceStatus.restype = wintypes.BO
OL
95         self.advapi32.EnumServicesStatusW.argtypes = [wintypes
.SC_HANDLE, wintypes.DWORD, wintypes.DWORD, ctypes.POINTER(ENU
M_SERVICE_STATUS), wintypes.DWORD, ctypes.POINTER(wintypes.DWO
RD), ctypes.POINTER(wintypes.DWORD), wintypes.LPVOID]
96         self.advapi32.EnumServicesStatusW.restype = wintypes.B
OOL
97         self.advapi32.CloseServiceHandle.argtypes = [wintypes.
SC_HANDLE]
98         self.advapi32.CloseServiceHandle.restype = wintypes.BO
OL
99
100     def read_registry_value(self, hive: int, path: str, value_
name: str) -> Optional[Any]:
101         """
102         Read a registry value.
103
104         Args:
105             hive: Registry hive (HKEY_*)
106             path: Registry path
107             value_name: Value name
108
109         Returns:
110             Registry value or None
111         """
112         try:
113             key = wintypes.HKEY()
114             result = self.advapi32.RegOpenKeyExW(hive, path, 0
, KEY_READ, ctypes.byref(key))

```

```

115         if result != 0:
116             return None
117         try:
118             value_type = wintypes.DWORD()
119             data_size = wintypes.DWORD(4096)
120             data = ctypes.create_unicode_buffer(4096)
121             result = self.advapi32.RegQueryValueExW(key, v
alue_name, None, ctypes.byref(value_type), data, ctypes.byref(
data_size))
122             if result == 0:
123                 return RegistryValue(name=value_name, valu
e_type=value_type.value, data=data.value)
124             return None
125         finally:
126             self.advapi32.RegCloseKey(key)
127     except Exception:
128         return None
129
130     def enumerate_registry_values(self, hive: int, path: str)
-> List[RegistryValue]:
131         """
132         Enumerate all values in a registry key.
133
134         Args:
135             hive: Registry hive
136             path: Registry path
137
138         Returns:
139             List of RegistryValue objects
140         """
141         values = []
142         try:
143             key = wintypes.HKEY()
144             result = self.advapi32.RegOpenKeyExW(hive, path, 0
, KEY_READ, ctypes.byref(key))
145             if result != 0:
146                 return values
147             try:

```

```

148         index = 0
149         while True:
150             value_name_buffer = ctypes.create_unicode_
buffer(256)
151             value_name_size = wintypes.DWORD(len(value
_name_buffer))
152             value_type = wintypes.DWORD()
153             data_size = wintypes.DWORD(4096)
154             data = ctypes.create_unicode_buffer(4096)
155             result = self.advapi32.RegEnumKeyExW(key,
index, value_name_buffer, ctypes.byref(value_name_size), None,
None, None, None)
156             if result != 0:
157                 break
158             values.append(RegistryValue(name=value_nam
e_buffer.value, value_type=value_type.value, data=data.value))
159             index += 1
160         finally:
161             self.advapi32.RegCloseKey(key)
162     except Exception:
163         pass
164     return values
165
166     def enumerate_services(self) -> List[ServiceInfo]:
167         """
168         Enumerate all services.
169
170         Returns:
171             List of ServiceInfo objects
172         """
173         services = []
174         try:
175             scm_handle = self.advapi32.OpenSCManagerW(None, No
ne, 1)
176             if not scm_handle:
177                 return services
178             try:
179                 max_services = 256

```

```

180         services_buffer = (ENUM_SERVICE_STATUS * max_s
| services)()
181         bytes_needed = wintypes.DWORD()
182         services_returned = wintypes.DWORD()
183         result = self.advapi32.EnumServicesStatusW(scm
| _handle, 48, 4, services_buffer, ctypes.sizeof(services_buffer
| ), ctypes.byref(bytes_needed), ctypes.byref(services_returned)
| , None)
184         if result:
185             for i in range(services_returned.value):
186                 svc = services_buffer[i]
187                 service_info = ServiceInfo(name=svc.lp
| ServiceName if svc.lpServiceName else 'Unknown', display_name=
| svc.lpDisplayName if svc.lpDisplayName else 'Unknown', state='
| Running' if svc.ServiceStatus.dwCurrentState == SERVICE_RUNNIN
| G else 'Stopped', service_type='Service' if svc.ServiceStatus.
| dwServiceType == 16 else 'Driver', start_type='Auto Start' if
| svc.ServiceStatus.dwCurrentState == SERVICE_AUTO_START else 'M
| anual')
188                 services.append(service_info)
189         finally:
190             self.advapi32.CloseServiceHandle(scm_handle)
191     except Exception:
192         pass
193     return services

```

error_decoder.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Error Decoder
5 # Description:
6 #   Модуль нативного декодирования системных кодов возврата Win32, HRESULT,
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.error_decoder import WindowsErrorDecoder
11 #
12 #       service = WindowsErrorDecoder()
13 #
14 # File: error_decoder.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Модуль нативного декодирования системных кодов возврата Win32, HRESULT, """
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 import os
28 import sys
29 from typing import Any, Dict, Optional, Tuple, Union
```



```
30
31 try:
32     from logger import logger
33 except ImportError:
34     import logging
35     logger = logging.getLogger(__name__)
36
37 # Флаги для FormatMessageW
38 FORMAT_MESSAGE_ALLOCATE_BUFFER = 0x00000100
39 FORMAT_MESSAGE_IGNORE_INSERTS = 0x00000200
40 FORMAT_MESSAGE_FROM_SYSTEM = 0x00001000
41 FORMAT_MESSAGE_FROM_HMODULE = 0x00000800
42
43 # Известные стандартные Stop-коды ядра Windows (BugCheck Codes
44 )
45 BUGCHECK_CODES_MAP: Dict[int, Tuple[str, str]] = {
46     0x0000000A: ('IRQL_NOT_LESS_OR_EQUAL', 'Процесс или драйве
47     р попытался обратиться к адресу памяти при недопустимо высоком
48     уровне IRQL. '),
49     0x0000001E: ('KMODE_EXCEPTION_NOT_HANDLED', 'Программа ядр
50     а или драйвер сгенерировали исключение, которое не было перехв
51     ачено обработчиком. '),
52     0x00000024: ('NTFS_FILE_SYSTEM', 'Критический сбой или пов
53     реждение в системном драйвере файловой системы ntfs.sys. '),
54     0x0000003B: ('SYSTEM_SERVICE_EXCEPTION', 'Исключение при в
55     ыполнении подпрограммы системной службы Windows. '),
56     0x00000050: ('PAGE_FAULT_IN_NONPAGED_AREA', 'Запрос недопу
57     стимой системной памяти. Ошибка вызвана некорректным адресом и
58     ли сбоем RAM. '),
59     0x0000007E: ('SYSTEM_THREAD_EXCEPTION_NOT_HANDLED', 'Систе
60     мный поток сгенерировал неперехваченное исключение (часто связ
61     ано с несовместимым драйвером). '),
62     0x0000007F: ('UNEXPECTED_KERNEL_MODE_TRAP', 'Аппаратная ло
63     вушка процессора (деление на ноль, повреждение стека или перег
64     рузка оборудования). '),
65     0x0000009F: ('DRIVER_POWER_STATE_FAILURE', 'Драйвер находи
66     тся в несогласованном или недействительном состоянии управлени
67     я электропитанием (сна/пробуждения). '),
```

53 0x000000BE: ('ATTEMPTED_WRITE_TO_READONLY_MEMORY', 'Попытка
а драйвера выполнить запись в сегмент памяти, доступный только
для чтения.'),

54 0x000000C2: ('BAD_POOL_CALLER', 'Текущий поток выполнил не
корректный запрос к пулу системной памяти ядра.'),

55 0x000000D1: ('DRIVER_IRQL_NOT_LESS_OR_EQUAL', 'Драйвер поп
ытался обратиться к выгружаемой памяти на повышенном уровне пр
ерываний IRQL.'),

56 0x000000EF: ('CRITICAL_PROCESS_DIED', 'Критически важный с
истемный процесс (csrss.exe, wininit.exe и др.) неожиданно зав
ершил свою работу.'),

57 0x00000109: ('CRITICAL_STRUCTURE_CORRUPTION', 'Ядро обнару
жило повреждение критических структур данных или кода ядра (Pa
tchGuard).'),

58 0x00000116: ('VIDEO_TDR_FAILURE', 'Сбой видеодрайвера: поп
ытка сброса зависшего графического драйвера завершилась тайм-а
утом.'),

59 0x00000117: ('VIDEO_TDR_TIMEOUT_DETECTED', 'Графический др
айвер не ответил вовремя в пределах интервала обнаружения тайм
-аута TDR.'),

60 0x00000124: ('WHEA_UNCORRECTABLE_ERROR', 'Критическая неус
транимая аппаратная ошибка архитектуры Windows Hardware Error
Architecture (CPU/RAM/PCIe).'),

61 0x00000133: ('DPC_WATCHDOG_VIOLATION', 'Сторожевой таймер
DPC зафиксировал зависание отложенного вызова процедуры драйве
ром.'),

62 0x00000139: ('KERNEL_SECURITY_CHECK_FAILURE', 'Ядро обнару
жило повреждение критической структуры данных безопасности (Bu
ffer Overflow / Stack Cookie).'),

63 0x00000154: ('UNEXPECTED_STORE_EXCEPTION', 'Сбой подсистем
ы хранения страниц памяти ядра (компрессия памяти / повреждени
е накопителя).'),

64 0x00000192: ('KERNEL_AUTO_BOOST_LOCK_ACQUISITION_WITH_RAIS
ED_IRQL', 'Попытка захвата блокировки AutoBoost ядра на повыше
нном уровне IRQL.'),

65 0x000001A7: ('STORE_DATA_STRUCTURE_CORRUPTION', 'Поврежден
ие внутренней структуры данных диспетчера хранилища страниц па
мяти.'),

```

66     0xC000021A: ('STATUS_SYSTEM_PROCESS_TERMINATED', 'Подсисте
ма пользовательского режима (WinLogon или Csrss) аварийно заве
ршилась с нарушением безопасности. '),
67     0xC0000221: ('STATUS_IMAGE_CHECKSUM_MISMATCH', 'Контрольна
я сумма системного драйвера или DLL не совпадает с заголовком
файла. '),
68 }
69
70
71 class WindowsErrorDecoder:
72     """Класс нативного декодирования ошибок Windows через Form
atMessageW."""
73
74     def __init__(self) -> None:
75         """Инициализация дескрипторов библиотек."""
76         self._kernel32 = getattr(ctypes.windll, 'kernel32', No
ne) if os.name == 'nt' else None
77         self._ntdll_handle: Optional[int] = None
78         if self._kernel32 and os.name == 'nt':
79             try:
80                 self._kernel32.GetModuleHandleW.argtypes = [wi
ntypes.LPCWSTR]
81                 self._kernel32.GetModuleHandleW.restype = wint
ypes.HMODULE
82                 self._ntdll_handle = self._kernel32.GetModuleH
andleW('ntdll.dll')
83             except Exception as ex:
84                 logger.debug(f'Не удалось получить дескриптор
ntdll.dll: {ex}')
85
86         def parse_error_code(self, code: Union[int, str]) -> Tuple
[int, str]:
87             """Преобразовать входной код ошибки (число, hex-строку
или отрицательный int) в uint32.
88
89             Args:
90                 code: Числовое или строковое представление кода ош
ибки.

```

```

91 |
92 |     Returns:
93 |         Tuple[int, str]: (нормализованный uint32 код, форм
    | атированная hex-строка 0XXXXXXXXX).
94 |     """
95 |     if isinstance(code, str):
96 |         c_str = code.strip()
97 |         try:
98 |             if c_str.lower().startswith('0x'):
99 |                 val = int(c_str, 16)
100 |             else:
101 |                 val = int(c_str, 10)
102 |         except ValueError:
103 |             return 0, '0x00000000'
104 |     else:
105 |         val = int(code)
106 |
107 |     # Преобразуем отрицательные 32-битные числа (например
    | из ctypes signed HRESULT) в беззнаковый uint32
108 |     uint_val = val & 0xFFFFFFFF
109 |     return uint_val, f'0x{uint_val:08X}'
110 |
111 |     def format_system_error(self, code: Union[int, str], lang_
    | id: int = 0) -> str:
112 |         """Получить локализованное системное описание ошибки и
    | з Windows API (Win32 / HRESULT / NTSTATUS).
113 |
114 |     Args:
115 |         code: Код ошибки (например, 5, 0x80070005, 0xC000
    | 005, '0x80240020').
116 |         lang_id: Идентификатор языка (0 - язык текущего по
    | тока/системы).
117 |
118 |     Returns:
119 |         str: Локализованное текстовое описание ошибки или
    | пустая строка, если код не распознан.
120 |     """
121 |     if not self._kernel32 or os.name != 'nt':

```

```

122         return ''
123
124     uint_code, _ = self.parse_error_code(code)
125     if uint_code == 0:
126         return 'Операция успешно завершена.'
127
128     # 1. Попытка чтения из системного репозитория (FORMAT_
MESSAGE_FROM_SYSTEM для Win32 и HRESULT)
129     buf = wintypes.LPWSTR()
130     flags = FORMAT_MESSAGE_ALLOCATE_BUFFER | FORMAT_MESSAG
E_FROM_SYSTEM | FORMAT_MESSAGE_IGNORE_INSERTS
131     chars = self._kernel32.FormatMessageW(
132         flags,
133         None,
134         wintypes.DWORD(uint_code),
135         wintypes.DWORD(lang_id),
136         ctypes.byref(buf),
137         0,
138         None,
139     )
140
141     if chars > 0 and buf.value:
142         msg = buf.value.strip().rstrip('\r\n.')
143         if self._kernel32.LocalFree:
144             self._kernel32.LocalFree(buf)
145         return msg
146
147     # 2. Если код лежит в диапазоне NTSTATUS (0xC0000000..
0xDFFFFFFF или 0x40000000..0x7FFFFFFF)
148     # запрашиваем системные сообщения из ntdll.dll
149     if self._ntdll_handle:
150         buf_nt = wintypes.LPWSTR()
151         flags_nt = FORMAT_MESSAGE_ALLOCATE_BUFFER | FORMAT
_MESSAGE_FROM_HMODULE | FORMAT_MESSAGE_IGNORE_INSERTS
152         chars_nt = self._kernel32.FormatMessageW(
153             flags_nt,
154             wintypes.HMODULE(self._ntdll_handle),
155             wintypes.DWORD(uint_code),

```

```

156 |         wintypes.DWORD(lang_id),
157 |         ctypes.byref(buf_nt),
158 |         0,
159 |         None,
160 |     )
161 |     if chars_nt > 0 and buf_nt.value:
162 |         msg_nt = buf_nt.value.strip().rstrip('\r\n.')
163 |         if self._kernel32.LocalFree:
164 |             self._kernel32.LocalFree(buf_nt)
165 |         return msg_nt
166 |
167 |     # 3. Если это HRESULT вида 0x8007XXXX (Win32 Facility
168 |     7), пробуем распаковать Win32 код ошибки
169 |     if (uint_code & 0xFFFF0000) == 0x80070000:
170 |         win32_subcode = uint_code & 0xFFFF
171 |         return self.format_system_error(win32_subcode, lan
172 | g_id=lang_id)
173 |
174 |     return ''
175 |
176 |     def decode_bugcheck_code(self, code: Union[int, str]) -> D
177 | ict[str, str]:
178 |         """Расшифровать Stop-код синего экрана (BSOD BugCheck
179 |         Code).
180 |
181 |         Args:
182 |             code: Числовой или строковый код BugCheck (наприме
183 |             p, 209, 0xD1, '0x000000D1').
184 |
185 |         Returns:
186 |             Dict[str, str]: Словарь с полями 'code_hex', 'symb
187 |             ol', 'description_ru'.
188 |
189 |         """
190 |         uint_code, hex_str = self.parse_error_code(code)
191 |         symbol = 'UNKNOWN_BUGCHECK'
192 |         desc = 'Критический сбой ядра Windows.'
193 |
194 |         if uint_code in BUGCHECK_CODES_MAP:

```

```

188         symbol, desc = BUGCHECK_CODES_MAP[uint_code]
189     else:
190         # Попробуем запросить описание через ntdll
191         nt_desc = self.format_system_error(uint_code)
192         if nt_desc:
193             desc = nt_desc
194
195     return {
196         'code_hex': hex_str,
197         'code_dec': str(uint_code),
198         'symbol': symbol,
199         'description_ru': desc,
200     }
201
202
203 # Глобальный синглтон декодера
204 _decoder_instance: Optional[WindowsErrorDecoder] = None
205
206
207 def get_error_decoder() -> WindowsErrorDecoder:
208     """Получить глобальный экземпляр WindowsErrorDecoder."""
209     global _decoder_instance
210     if _decoder_instance is None:
211         _decoder_instance = WindowsErrorDecoder()
212     return _decoder_instance
213
214
215 def format_system_error(code: Union[int, str], lang_id: int =
216     0) -> str:
217     """Удобный хелпер для форматирования системного кода ошибок
218     и."""
219     return get_error_decoder().format_system_error(code, lang_
220     id=lang_id)
221
222
223 def decode_bugcheck_code(code: Union[int, str]) -> Dict[str, s
224     tr]:
225     """Удобный хелпер для расшифровки Stop-кода ядра (BugCheck

```

```
    )."""
222     return get_error_decoder().decode_bugcheck_code(code)
223
224
225 __all__ = [
226     'WindowsErrorDecoder',
227     'get_error_decoder',
228     'format_system_error',
229     'decode_bugcheck_code',
230     'BUGCHECK_CODES_MAP',
231 ]
232
```


etw.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Etw
5 # Description:
6 #   Event Tracing for Windows (ETW) wrapper for real-time even
7 #   t monitoring.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.etw import GUID
11 #
12 #       service = GUID()
13 #
14 # File: etw.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """Event Tracing for Windows (ETW) wrapper for real-time event
23   monitoring."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 from typing import Optional, List, Callable, Dict, Any
28 from dataclasses import dataclass
29 from enum import IntFlag
30 import threading
31
32 EVENT_TRACE_FLAG_PROCESS = 1
```

```

31 EVENT_TRACE_FLAG_THREAD = 2
32 EVENT_TRACE_FLAG_IMAGE_LOAD = 4
33 EVENT_TRACE_FLAG_DISK_IO = 256
34 EVENT_TRACE_FLAG_DISK_FILE_IO = 512
35 EVENT_TRACE_FLAG_MEMORY_PAGE_FAULTS = 4096
36 EVENT_TRACE_FLAG_MEMORY_HARD_FAULTS = 8192
37 EVENT_TRACE_FLAG_NETWORK_TCPIP = 65536
38 EVENT_TRACE_FLAG_REGISTRY = 131072
39 EVENT_TRACE_FLAG_FILE_IO = 262144
40 EVENT_TRACE_FLAG_FILE_IO_INIT = 524288
41 EVENT_TRACE_REAL_TIME_MODE = 256
42 EVENT_TRACE_FILE_MODE_SEQUENTIAL = 1
43 EVENT_TRACE_FILE_MODE_CIRCULAR = 2
44 EVENT_TRACE_USE_GLOBAL_SEQUENCE = 16384
45 WNODE_FLAG_TRACED_GUID = 131072
46 WNODE_FLAG_USE_MOF_PTR = 1048576
47 PEVENT_TRACE_BUFFER_CALLBACK = ctypes.CFUNCTYPE(wintypes.ULONG
    , wintypes.LPVOID)
48 PEVENT_CALLBACK = ctypes.CFUNCTYPE(None, wintypes.LPVOID)
49
50 class GUID(ctypes.Structure):
51     _fields_ = [('Data1', wintypes.DWORD), ('Data2', wintypes.
    WORD), ('Data3', wintypes.WORD), ('Data4', ctypes.c_ubyte * 8)
    ]
52
53 class WNODE_HEADER(ctypes.Structure):
54     _fields_ = [('BufferSize', wintypes.DWORD), ('ProviderId',
    wintypes.DWORD), ('ClientContext', ctypes.c_uint64), ('Timest
    amp', ctypes.c_uint64), ('Guid', GUID), ('KernelHandle', winty
    pes.HANDLE), ('Flags', wintypes.DWORD)]
55
56 class EVENT_TRACE_PROPERTIES(ctypes.Structure):
57     _fields_ = [('Wnode', WNODE_HEADER), ('BufferSize', wintyp
    es.DWORD), ('MinimumBuffers', wintypes.DWORD), ('MaximumBuffer
    s', wintypes.DWORD), ('MaximumFileSize', wintypes.DWORD), ('Lo
    gFileMode', wintypes.DWORD), ('FlushTimer', wintypes.DWORD), (
    'EnableFlags', wintypes.DWORD), ('AgeLimit', wintypes.LONG), (
    'NumberOfBuffers', wintypes.DWORD), ('FreeBuffers', wintypes.D

```

```

WORD), ('EventsLost', wintypes.DWORD), ('BuffersWritten', wintypes.DWORD), ('LogBuffersLost', wintypes.DWORD), ('RealTimeBuffersLost', wintypes.DWORD), ('LoggerThreadId', wintypes.HANDLE)]
58
59 class EVENT_TRACE_HEADER(ctypes.Structure):
60     _fields_ = [('Size', wintypes.USHORT), ('Reserved', wintypes.USHORT), ('Class', ctypes.c_ubyte), ('Type', ctypes.c_ubyte), ('Level', ctypes.c_ubyte), ('Version', ctypes.c_ubyte), ('Marker', wintypes.DWORD), ('ThreadId', wintypes.DWORD), ('ProcessId', wintypes.DWORD), ('TimeStamp', ctypes.c_uint64), ('ProviderId', GUID), ('ClientContext', wintypes.DWORD), ('Flags', wintypes.DWORD)]
61
62 @dataclass
63 class EtwEvent:
64     """ETW event information."""
65     timestamp: int
66     pid: int
67     tid: int
68     event_class: int
69     event_type: int
70     level: int
71     data: bytes
72
73 class EtwAPI:
74     """Wrapper for ETW event tracing functions."""
75
76     def __init__(self):
77         self.advapi32 = ctypes.windll.advapi32
78         self.ntdll = ctypes.windll.ntdll
79         self._setup_function_signatures()
80         self._session_handle = None
81         self._event_callbacks: Dict[int, List[Callable]] = {}
82
83     def _setup_function_signatures(self):
84         """Configure ctypes function signatures."""
85         self.advapi32.StartTraceW.argtypes = [ctypes.POINTER(w

```

```

intypes.HANDLE), wintypes.LPCWSTR, ctypes.POINTER(EVENT_TRACE_
PROPERTIES)])
86         self.advapi32.StartTraceW.restype = wintypes.ULONG
87         self.advapi32.StopTraceW.argtypes = [wintypes.HANDLE,
wintypes.LPCWSTR, ctypes.POINTER(EVENT_TRACE_PROPERTIES)]
88         self.advapi32.StopTraceW.restype = wintypes.ULONG
89         self.advapi32.EnableTrace.argtypes = [wintypes.ULONG,
wintypes.ULONG, wintypes.ULONG, ctypes.POINTER(GUID), wintypes
.HANDLE]
90         self.advapi32.EnableTrace.restype = wintypes.ULONG
91         self.advapi32.ControlTraceW.argtypes = [wintypes.HANDL
E, wintypes.LPCWSTR, ctypes.POINTER(EVENT_TRACE_PROPERTIES), w
intypes.DWORD]
92         self.advapi32.ControlTraceW.restype = wintypes.ULONG
93
94     def start_kernel_logger(self, enable_flags: int=EVENT_TRAC
E_FLAG_PROCESS) -> bool:
95         """
96         Start the Kernel Logger session for real-time event tr
acing.
97
98         Args:
99             enable_flags: Flags for events to capture
100
101         Returns:
102             True if successful
103         """
104         try:
105             properties = EVENT_TRACE_PROPERTIES()
106             properties.Wnode.BufferSize = ctypes.sizeof(EVENT_
TRACE_PROPERTIES)
107             properties.Wnode.Guid = GUID()
108             properties.Wnode.ClientContext = 1
109             properties.Wnode.Flags = WNODE_FLAG_TRACED_GUID
110             properties.BufferSize = 128
111             properties.MinimumBuffers = 4
112             properties.MaximumBuffers = 64
113             properties.LogFileMode = EVENT_TRACE_REAL_TIME_MOD

```

```

E | EVENT_TRACE_USE_GLOBAL_SEQUENCE
114 |         properties.FlushTimer = 100
115 |         properties.EnableFlags = enable_flags
116 |         properties.AgeLimit = 0
117 |         session_handle = wintypes.HANDLE()
118 |         status = self.advapi32.StartTraceW(ctypes.byref(se
    | session_handle), 'NT Kernel Logger', ctypes.byref(properties))
119 |         if status == 0:
120 |             self._session_handle = session_handle
121 |             return True
122 |         return False
123 |     except Exception:
124 |         return False
125 |
126 |     def stop_kernel_logger(self) -> bool:
127 |         """
128 |         Stop the Kernel Logger session.
129 |
130 |         Returns:
131 |             True if successful
132 |         """
133 |         try:
134 |             if not self._session_handle:
135 |                 return False
136 |             properties = EVENT_TRACE_PROPERTIES()
137 |             properties.Wnode.BufferSize = ctypes.sizeof(EVENT_
    | TRACE_PROPERTIES)
138 |             status = self.advapi32.StopTraceW(self._session_ha
    | ndle, 'NT Kernel Logger', ctypes.byref(properties))
139 |             if status == 0:
140 |                 self._session_handle = None
141 |                 return True
142 |             return False
143 |         except Exception:
144 |             return False
145 |
146 |     def register_event_callback(self, event_type: int, callbac
    | k: Callable) -> None:

```

```

147 |         """
148 |         Register a callback for a specific event type.
149 |
150 |         Args:
151 |             event_type: ETW event type
152 |             callback: Callback function to invoke
153 |         """
154 |         if event_type not in self._event_callbacks:
155 |             self._event_callbacks[event_type] = []
156 |         self._event_callbacks[event_type].append(callback)
157 |
158 |     def unregister_event_callback(self, event_type: int, callb
159 | ack: Callable) -> None:
160 |         """
161 |         Unregister a callback.
162 |
163 |         Args:
164 |             event_type: ETW event type
165 |             callback: Callback function to remove
166 |         """
167 |         if event_type in self._event_callbacks:
168 |             try:
169 |                 self._event_callbacks[event_type].remove(callb
170 | ack)
171 |             except ValueError:
172 |                 pass
173 |
174 |     def get_session_handle(self) -> Optional[wintypes.HANDLE]:
175 |         """Get the current session handle."""
176 |         return self._session_handle

```

kernel32.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  # - Kernel32
4 # =====
5 # Description:
6 #   kernel32.dll wrapper for process enumeration and snapshot
  operations.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.kernel32 import PR
  OCESSENTRY32
11 #
12 #       service = PROCESSIONARY32()
13 #
14 # File: kernel32.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """kernel32.dll wrapper for process enumeration and snapshot o
  perations."""
23
24 import ctypes
25 import ctypes.wintypes as wintypes
26 from typing import List, Optional, Tuple
27 from dataclasses import dataclass
28 from enum import IntFlag
29 TH32CS_SNAPPROCESS = 2
```

```

30 TH32CS_SNAPTHREAD = 4
31 TH32CS_SNAPMODULE = 8
32 TH32CS_SNAPMODULE32 = 16
33 ABOVE_NORMAL_PRIORITY_CLASS = 32768
34 BELOW_NORMAL_PRIORITY_CLASS = 16384
35 HIGH_PRIORITY_CLASS = 128
36 IDLE_PRIORITY_CLASS = 64
37 NORMAL_PRIORITY_CLASS = 32
38 REALTIME_PRIORITY_CLASS = 256
39
40 class PROCESSENTRY32(ctypes.Structure):
41     _fields_ = [('dwSize', wintypes.DWORD), ('th32ProcessID',
wintypes.DWORD), ('th32ParentProcessID', wintypes.DWORD), ('dw
Priority', wintypes.LONG), ('dwThreadCount', wintypes.DWORD),
('th32ModuleID', wintypes.DWORD), ('pcbUsage', wintypes.DWORD)
, ('th32AccessKey', wintypes.LPVOID), ('szExeFile', ctypes.c_c
har * 260)]
42
43 class THREADENTRY32(ctypes.Structure):
44     _fields_ = [('dwSize', wintypes.DWORD), ('th32ThreadID', w
intypes.DWORD), ('th32OwnerProcessID', wintypes.DWORD), ('tpBa
sePri', wintypes.LONG), ('tpDeltaPri', wintypes.LONG), ('dwFla
gs', wintypes.DWORD)]
45
46 class MODULEENTRY32(ctypes.Structure):
47     _fields_ = [('dwSize', wintypes.DWORD), ('th32ModuleID', w
intypes.DWORD), ('th32ProcessID', wintypes.DWORD), ('GlpvBase'
, wintypes.LPVOID), ('dwSize_', wintypes.DWORD), ('hModule', w
intypes.HMODULE), ('szModule', ctypes.c_char * 256), ('szExePa
th', ctypes.c_char * 260)]
48
49 class FILETIME(ctypes.Structure):
50     _fields_ = [('dwLowDateTime', wintypes.DWORD), ('dwHighDat
eTime', wintypes.DWORD)]
51
52 @dataclass
53 class ProcessInfo:
54     """Process snapshot information from Toolhelp32."""

```



```

55     pid: int
56     ppid: int
57     name: str
58     priority: int
59     thread_count: int
60
61 @dataclass
62 class ThreadInfo:
63     """Thread snapshot information from Toolhelp32."""
64     tid: int
65     pid: int
66     base_priority: int
67     delta_priority: int
68
69 @dataclass
70 class ModuleInfo:
71     """Module snapshot information from Toolhelp32."""
72     name: str
73     path: str
74     base_address: int
75     size: int
76
77 class Kernel32API:
78     """Wrapper for kernel32 process and thread enumeration fun
79     ctions."""
80
81     def __init__(self):
82         self.kernel32 = ctypes.windll.kernel32
83         self._setup_function_signatures()
84
85     def _setup_function_signatures(self):
86         """Configure ctypes function signatures for proper typ
87         e checking."""
88         self.kernel32.CreateToolhelp32Snapshot.argtypes = [wintypes.DWORD, wintypes.DWORD]
89         self.kernel32.CreateToolhelp32Snapshot.restype = wintypes.HANDLE
90         self.kernel32.Process32First.argtypes = [wintypes.HAND

```

```

    LE, ctypes.POINTER(PROCESSENTRY32)]
89         self.kernel32.Process32First.restype = wintypes.BOOL
90         self.kernel32.Process32Next.argtypes = [wintypes.HANDL
    E, ctypes.POINTER(PROCESSENTRY32)]
91         self.kernel32.Process32Next.restype = wintypes.BOOL
92         self.kernel32.Thread32First.argtypes = [wintypes.HANDL
    E, ctypes.POINTER(THREADENTRY32)]
93         self.kernel32.Thread32First.restype = wintypes.BOOL
94         self.kernel32.Thread32Next.argtypes = [wintypes.HANDLE
    , ctypes.POINTER(THREADENTRY32)]
95         self.kernel32.Thread32Next.restype = wintypes.BOOL
96         self.kernel32.Module32First.argtypes = [wintypes.HANDL
    E, ctypes.POINTER(MODULEENTRY32)]
97         self.kernel32.Module32First.restype = wintypes.BOOL
98         self.kernel32.Module32Next.argtypes = [wintypes.HANDLE
    , ctypes.POINTER(MODULEENTRY32)]
99         self.kernel32.Module32Next.restype = wintypes.BOOL
100        self.kernel32.CloseHandle.argtypes = [wintypes.HANDLE]
101        self.kernel32.CloseHandle.restype = wintypes.BOOL
102        self.kernel32.GetCurrentProcess.restype = wintypes.HAN
    DLE
103        self.kernel32.GetProcessTimes.argtypes = [wintypes.HAN
    DLE, ctypes.POINTER(FILETIME), ctypes.POINTER(FILETIME), ctype
    s.POINTER(FILETIME), ctypes.POINTER(FILETIME)]
104        self.kernel32.GetProcessTimes.restype = wintypes.BOOL
105        self.kernel32.GetPriorityClass.argtypes = [wintypes.HA
    NDLE]
106        self.kernel32.GetPriorityClass.restype = wintypes.DWOR
    D
107
108    def enumerate_processes(self) -> List[ProcessInfo]:
109        """
110        Enumerate all running processes using Toolhelp32.
111
112        Returns:
113            List of ProcessInfo objects
114        """
115        processes = []

```

```

116         snapshot = self.kernel32.CreateToolhelp32Snapshot(TH32
| CS_SNAPPROCESS, 0)
117         if snapshot == -1:
118             return processes
119         try:
120             pe = PROCESSENTRY32()
121             pe.dwSize = ctypes.sizeof(PROCESSENTRY32)
122             if self.kernel32.Process32First(snapshot, ctypes.b
| yref(pe)):
123                 while True:
124                     try:
125                         process = ProcessInfo(pid=pe.th32Proce
| ssID, ppid=pe.th32ParentProcessID, name=pe.szExeFile.decode('u
| tf-8', errors='ignore'), priority=pe.dwPriority, thread_count=
| pe.dwThreadCount)
126                         processes.append(process)
127                     except Exception:
128                         pass
129                     if not self.kernel32.Process32Next(snapsho
| t, ctypes.byref(pe)):
130                         break
131                 finally:
132                     self.kernel32.CloseHandle(snapshot)
133             return processes
134
135     def enumerate_threads(self, pid: Optional[int]=None) -> Li
| st[ThreadInfo]:
136         """
137         Enumerate threads for a process or all threads if pid
| is None.
138
139         Args:
140             pid: Process ID to filter threads, or None for all
| threads
141
142         Returns:
143             List of ThreadInfo objects
144         """

```

```

145 |         threads = []
146 |         snapshot = self.kernel32.CreateToolhelp32Snapshot(TH32
| CS_SNAPTHREAD, pid or 0)
147 |         if snapshot == -1:
148 |             return threads
149 |         try:
150 |             te = THREADENTRY32()
151 |             te.dwSize = ctypes.sizeof(THREADENTRY32)
152 |             if self.kernel32.Thread32First(snapshot, ctypes.by
| ref(te)):
153 |                 while True:
154 |                     try:
155 |                         if pid is None or te.th32OwnerProcessI
| D == pid:
156 |                             thread = ThreadInfo(tid=te.th32Thr
| eadID, pid=te.th32OwnerProcessID, base_priority=te.tpBasePri,
| delta_priority=te.tpDeltaPri)
157 |                             threads.append(thread)
158 |                         except Exception:
159 |                             pass
160 |                         if not self.kernel32.Thread32Next(snapshot
| , ctypes.byref(te)):
161 |                             break
162 |             finally:
163 |                 self.kernel32.CloseHandle(snapshot)
164 |             return threads
165 |
166 |     def enumerate_modules(self, pid: int) -> List[ModuleInfo]:
167 |         """
168 |         Enumerate modules (DLLs) loaded in a process.
169 |
170 |         Args:
171 |             pid: Process ID
172 |
173 |         Returns:
174 |             List of ModuleInfo objects
175 |         """
176 |         modules = []

```

```

177 |         snapshot = self.kernel32.CreateToolhelp32Snapshot(TH32
| CS_SNAPMODULE | TH32CS_SNAPMODULE32, pid)
178 |         if snapshot == -1:
179 |             return modules
180 |         try:
181 |             me = MODULEENTRY32()
182 |             me.dwSize = ctypes.sizeof(MODULEENTRY32)
183 |             if self.kernel32.Module32First(snapshot, ctypes.by
| ref(me)):
184 |                 while True:
185 |                     try:
186 |                         module = ModuleInfo(name=me.szModule.d
| ecode('utf-8', errors='ignore'), path=me.szExePath.decode('utf
| -8', errors='ignore'), base_address=int(me.GlpvBase), size=me.
| dwSize_)
187 |                         modules.append(module)
188 |                     except Exception:
189 |                         pass
190 |                     if not self.kernel32.Module32Next(snapshot
| , ctypes.byref(me)):
191 |                         break
192 |             finally:
193 |                 self.kernel32.CloseHandle(snapshot)
194 |             return modules
195 |
196 |     def get_process_priority_class(self, pid: int) -> Optional
| [str]:
197 |         """
198 |         Get the priority class of a process.
199 |
200 |         Args:
201 |             pid: Process ID
202 |
203 |         Returns:
204 |             Priority class name or None if error
205 |         """
206 |         try:
207 |             handle = ctypes.windll.kernel32.OpenProcess(1024,

```

```

False, pid)
208         if not handle:
209             return None
210         try:
211             priority = self.kernel32.GetPriorityClass(handle)
212             priority_map = {ABOVE_NORMAL_PRIORITY_CLASS: 'Above Normal', BELOW_NORMAL_PRIORITY_CLASS: 'Below Normal', HIGH_PRIORITY_CLASS: 'High', IDLE_PRIORITY_CLASS: 'Idle', NORMAL_PRIORITY_CLASS: 'Normal', REALTIME_PRIORITY_CLASS: 'Realtime'}
213             return priority_map.get(priority, f'Unknown ({priority})')
214         finally:
215             self.kernel32.CloseHandle(handle)
216     except Exception:
217         return None
218
219     def get_process_times(self, pid: int) -> Optional[Tuple[int, int]]:
220         """
221         Get process creation and exit times.
222
223         Args:
224             pid: Process ID
225
226         Returns:
227             Tuple of (creation_time_filetime, exit_time_filetime) or None
228         """
229         try:
230             handle = ctypes.windll.kernel32.OpenProcess(1024, False, pid)
231             if not handle:
232                 return None
233             try:
234                 creation_time = FILETIME()
235                 exit_time = FILETIME()

```

```
236 |         kernel_time = FILETIME()
237 |         user_time = FILETIME()
238 |         if self.kernel32.GetProcessTimes(handle, ctypes
|         s.byref(creation_time), ctypes.byref(exit_time), ctypes.byref(
|         kernel_time), ctypes.byref(user_time)):
239 |             return (creation_time.dwLowDateTime, exit_
|         time.dwLowDateTime)
240 |         return None
241 |     finally:
242 |         self.kernel32.CloseHandle(handle)
243 | except Exception:
244 |     return None
```

nethelper.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  # - Nethelper
4 # =====
5 # Description:
6 #   Низкоуровневый интерфейс к IP Helper API (iphlpapi.dll) Wi
  ndows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ffnethelper import M
  IB_TCPROW_OWNER_PID
11 #
12 #       service = MIB_TCPROW_OWNER_PID()
13 #
14 # File: nethelper.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ffnethelper
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к IP Helper API (iphlpapi.dll) Win
  dows."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 import socket
28 import struct
29 from dataclasses import dataclass
```



```

30 from typing import Any, Dict, List, Optional
31 from logger import logger
32 AF_INET = 2
33 AF_INET6 = 23
34 TCP_TABLE_BASIC_LISTENER = 0
35 TCP_TABLE_BASIC_CONNECTIONS = 1
36 TCP_TABLE_BASIC_ALL = 2
37 TCP_TABLE_OWNER_PID_LISTENER = 3
38 TCP_TABLE_OWNER_PID_CONNECTIONS = 4
39 TCP_TABLE_OWNER_PID_ALL = 5
40 TCP_TABLE_OWNER_MODULE_LISTENER = 6
41 TCP_TABLE_OWNER_MODULE_CONNECTIONS = 7
42 TCP_TABLE_OWNER_MODULE_ALL = 8
43 UDP_TABLE_BASIC = 0
44 UDP_TABLE_OWNER_PID = 1
45 UDP_TABLE_OWNER_MODULE = 2
46 MIB_TCP_STATE_CLOSED = 1
47 MIB_TCP_STATE_LISTEN = 2
48 MIB_TCP_STATE_SYN_SENT = 3
49 MIB_TCP_STATE_SYN_RCVD = 4
50 MIB_TCP_STATE_ESTAB = 5
51 MIB_TCP_STATE_FIN_WAIT1 = 6
52 MIB_TCP_STATE_FIN_WAIT2 = 7
53 MIB_TCP_STATE_CLOSE_WAIT = 8
54 MIB_TCP_STATE_CLOSING = 9
55 MIB_TCP_STATE_LAST_ACK = 10
56 MIB_TCP_STATE_TIME_WAIT = 11
57 MIB_TCP_STATE_DELETE_TCB = 12
58 TCP_STATE_MAP = {MIB_TCP_STATE_CLOSED: 'CLOSED', MIB_TCP_STATE
    _LISTEN: 'LISTEN', MIB_TCP_STATE_SYN_SENT: 'SYN_SENT', MIB_TCP
    _STATE_SYN_RCVD: 'SYN_RCVD', MIB_TCP_STATE_ESTAB: 'ESTABLISHED
    ', MIB_TCP_STATE_FIN_WAIT1: 'FIN_WAIT1', MIB_TCP_STATE_FIN_WAI
    T2: 'FIN_WAIT2', MIB_TCP_STATE_CLOSE_WAIT: 'CLOSE_WAIT', MIB_T
    CP_STATE_CLOSING: 'CLOSING', MIB_TCP_STATE_LAST_ACK: 'LAST_ACK
    ', MIB_TCP_STATE_TIME_WAIT: 'TIME_WAIT', MIB_TCP_STATE_DELETE_
    TCB: 'DELETE_TCB'}
59 ERROR_INSUFFICIENT_BUFFER = 122
60

```

```

61 class MIB_TCPROW_OWNER_PID(ctypes.Structure):
62     """Структура записи TCP соединения с PID."""
63     _fields_ = [('dwState', wintypes.DWORD), ('dwLocalAddr', w
intypes.DWORD), ('dwLocalPort', wintypes.DWORD), ('dwRemoteAdd
r', wintypes.DWORD), ('dwRemotePort', wintypes.DWORD), ('dwOwn
ingPid', wintypes.DWORD)]
64
65 class MIB_UDPROW_OWNER_PID(ctypes.Structure):
66     """Структура записи UDP сокета с PID."""
67     _fields_ = [('dwLocalAddr', wintypes.DWORD), ('dwLocalPort
', wintypes.DWORD), ('dwOwningPid', wintypes.DWORD)]
68
69 class MIB_IF_ROW2(ctypes.Structure):
70     """Нативная структура MIB_IF_ROW2 из iphlpapi.dll."""
71     _fields_ = [
72         ('InterfaceLuid', ctypes.c_uint64),
73         ('InterfaceIndex', wintypes.DWORD),
74         ('InterfaceGuid', ctypes.c_byte * 16),
75         ('Alias', ctypes.c_wchar * 257),
76         ('Description', ctypes.c_wchar * 257),
77         ('PhysicalAddressLength', wintypes.DWORD),
78         ('PhysicalAddress', ctypes.c_byte * 32),
79         ('PermanentPhysicalAddress', ctypes.c_byte * 32),
80         ('Mtu', wintypes.DWORD),
81         ('Type', wintypes.DWORD),
82         ('TunnelType', ctypes.c_int),
83         ('MediaType', ctypes.c_int),
84         ('PhysicalMediumType', ctypes.c_int),
85         ('AccessType', ctypes.c_int),
86         ('DirectionType', ctypes.c_int),
87         ('InterfaceAndOperStatusFlags', ctypes.c_byte),
88         ('OperStatus', ctypes.c_int),
89         ('AdminStatus', ctypes.c_int),
90         ('MediaConnectState', ctypes.c_int),
91         ('NetworkGuid', ctypes.c_byte * 16),
92         ('ConnectionType', ctypes.c_int),
93         ('TransmitLinkSpeed', ctypes.c_uint64),
94         ('ReceiveLinkSpeed', ctypes.c_uint64),

```

```

95 |         ('InOctets', ctypes.c_uint64),
96 |         ('InUcastPkts', ctypes.c_uint64),
97 |         ('InNUcastPkts', ctypes.c_uint64),
98 |         ('InDiscards', ctypes.c_uint64),
99 |         ('InErrors', ctypes.c_uint64),
100 |         ('InUnknownProtos', ctypes.c_uint64),
101 |         ('InUcastOctets', ctypes.c_uint64),
102 |         ('InMulticastOctets', ctypes.c_uint64),
103 |         ('InBroadcastOctets', ctypes.c_uint64),
104 |         ('OutOctets', ctypes.c_uint64),
105 |         ('OutUcastPkts', ctypes.c_uint64),
106 |         ('OutNUcastPkts', ctypes.c_uint64),
107 |         ('OutDiscards', ctypes.c_uint64),
108 |         ('OutErrors', ctypes.c_uint64),
109 |         ('OutUcastOctets', ctypes.c_uint64),
110 |         ('OutMulticastOctets', ctypes.c_uint64),
111 |         ('OutBroadcastOctets', ctypes.c_uint64),
112 |         ('OutQLen', ctypes.c_uint64),
113 |     ]
114 |
115 | class MIB_IF_TABLE2(ctypes.Structure):
116 |     """Таблица сетевых интерфейсов MIB_IF_TABLE2."""
117 |     _fields_ = [
118 |         ('NumEntries', wintypes.DWORD),
119 |         ('Table', MIB_IF_ROW2 * 1),
120 |     ]
121 |
122 | class MIB_IPNETROW(ctypes.Structure):
123 |     """Структура записи таблицы ARP (MIB_IPNETROW)."""
124 |     _fields_ = [
125 |         ('dwIndex', wintypes.DWORD),
126 |         ('dwPhysAddrLen', wintypes.DWORD),
127 |         ('bPhysAddr', ctypes.c_ubyte * 8),
128 |         ('dwAddr', wintypes.DWORD),
129 |         ('dwType', wintypes.DWORD),
130 |     ]
131 |
132 | class MIB_IPNETTABLE(ctypes.Structure):

```

```
133 |     """Таблица записей ARP (MIB_IPNETTABLE)."""
134 |     _fields_ = [
135 |         ('dwNumEntries', wintypes.DWORD),
136 |         ('table', MIB_IPNETROW * 1),
137 |     ]
138 |
139 | IP_NET_TYPE_MAP = {
140 |     1: 'OTHER',
141 |     2: 'INVALID',
142 |     3: 'DYNAMIC',
143 |     4: 'STATIC',
144 | }
145 |
146 | @dataclass
147 | class NetworkSocketInfo:
148 |     """Нормализованные данные о сетевом соединении/сожете."""
149 |     protocol: str
150 |     local_address: str
151 |     local_port: int
152 |     remote_address: str
153 |     remote_port: int
154 |     state: str
155 |     pid: int
156 |
157 |     def to_dict(self) -> Dict[str, Any]:
158 |         """Преобразование в словарь."""
159 |         return {'protocol': self.protocol, 'local_address': self.local_address, 'local_port': self.local_port, 'remote_address': self.remote_address, 'remote_port': self.remote_port, 'state': self.state, 'pid': self.pid}
160 |
161 | class IPHelperAPI:
162 |     """Нативный интерфейс к iphlpapi.dll."""
163 |
164 |     def __init__(self) -> None:
165 |         """Инициализация функций iphlpapi.dll."""
166 |         self._iphlpapi = ctypes.windll.iphlpapi
167 |         self._GetExtendedTcpTable = self._iphlpapi.GetExtended
```

```

    TcpTable
168         self._GetExtendedTcpTable.argtypes = [ctypes.c_void_p,
    ctypes.POINTER(wintypes.DWORD), wintypes.BOOL, wintypes.ULONG
    , ctypes.c_int, wintypes.ULONG]
169         self._GetExtendedTcpTable.restype = wintypes.DWORD
170         self._GetExtendedUdpTable = self._iphlpapi.GetExtended
    UdpTable
171         self._GetExtendedUdpTable.argtypes = [ctypes.c_void_p,
    ctypes.POINTER(wintypes.DWORD), wintypes.BOOL, wintypes.ULONG
    , ctypes.c_int, wintypes.ULONG]
172         self._GetExtendedUdpTable.restype = wintypes.DWORD
173
174         # SendARP
175         self._SendARP = getattr(self._iphlpapi, 'SendARP', Non
    e)
176         if self._SendARP:
177             self._SendARP.argtypes = [wintypes.ULONG, wintypes
    .ULONG, ctypes.c_void_p, ctypes.POINTER(wintypes.ULONG)]
178             self._SendARP.restype = wintypes.DWORD
179
180         # GetIpNetTable
181         self._GetIpNetTable = getattr(self._iphlpapi, 'GetIpNe
    tTable', None)
182         if self._GetIpNetTable:
183             self._GetIpNetTable.argtypes = [ctypes.c_void_p, c
    ctypes.POINTER(wintypes.ULONG), wintypes.BOOL]
184             self._GetIpNetTable.restype = wintypes.DWORD
185
186         if hasattr(self._iphlpapi, 'GetIfTable2'):
187             self._GetIfTable2 = self._iphlpapi.GetIfTable2
188             self._GetIfTable2.argtypes = [ctypes.POINTER(ctype
    s.c_void_p)]
189             self._GetIfTable2.restype = wintypes.DWORD
190             self._FreeMibTable = getattr(self._iphlpapi, 'Free
    MibTable', None)
191             if self._FreeMibTable:
192                 self._FreeMibTable.argtypes = [ctypes.c_void_p
    ]

```

```

193         else:
194             self._GetIfTable2 = None
195
196     @staticmethod
197     def _ip_to_str(ip_int: int) -> str:
198         """Преобразование сетевого порядка байт IP в строку."""
199
200         try:
201             return socket.inet_ntoa(struct.pack('<L', ip_int))
202         except Exception:
203             return '0.0.0.0'
204
205     @staticmethod
206     def _port_to_int(port_int: int) -> int:
207         """Преобразование сетевого порядка байт порта в int."""
208
209         return port_int >> 8 & 255 | (port_int & 255) << 8
210
211     def get_adapter_statistics(self) -> List[Dict[str, Any]]:
212         """Получение накопительной статистики по всем сетевым
213         адаптерам через GetIfTable2.
214
215         Returns:
216             List[Dict[str, Any]]: Список словарей со статистик
217             ой интерфейсов.
218
219         """
220         if not self._GetIfTable2:
221             return []
222         p_table = ctypes.c_void_p()
223         res = self._GetIfTable2(ctypes.byref(p_table))
224         if res != 0 or not p_table:
225             return []
226         try:
227             num_entries = ctypes.cast(p_table, ctypes.POINTER(
228                 wintypes.DWORD)).contents.value
229             offset = ctypes.sizeof(wintypes.DWORD)
230             row_size = ctypes.sizeof(MIB_IF_ROW2)
231             base_addr = p_table.value or 0

```

```

226         if not base_addr:
227             return []
228         stats: List[Dict[str, Any]] = []
229         for i in range(num_entries):
230             row_addr = base_addr + offset + i * row_size
231             row = MIB_IF_ROW2.from_address(row_addr)
232             name = row.Alias or row.Description or f'Inter
face_{row.InterfaceIndex}'
233             stats.append({
234                 'name': name,
235                 'alias': row.Alias,
236                 'description': row.Description,
237                 'interface_index': row.InterfaceIndex,
238                 'received_bytes': row.InOctets,
239                 'sent_bytes': row.OutOctets,
240                 'received_packets': row.InUcastPkts + row.
InNUcastPkts,
241                 'sent_packets': row.OutUcastPkts + row.Out
NUcastPkts,
242                 'received_discarded': row.InDiscards,
243                 'received_errors': row.InErrors,
244                 'outbound_discarded': row.OutDiscards,
245                 'outbound_errors': row.OutErrors,
246                 'transmit_speed_bps': row.TransmitLinkSpee
d,
247                 'receive_speed_bps': row.ReceiveLinkSpeed,
248                 'oper_status': row.OperStatus,
249             })
250         return stats
251     finally:
252         if hasattr(self, '_FreeMibTable') and self._FreeMi
bTable and p_table:
253             self._FreeMibTable(p_table)
254
255     def get_tcp_connections(self) -> List[NetworkSocketInfo]:
256         """Получение всех активных TCP соединений и слушающих
портов с PID.
257

```

```

258         Returns:
259             List[NetworkSocketInfo]: Список соединений.
260         """
261         size = wintypes.DWORD(0)
262         self._GetExtendedTcpTable(None, ctypes.byref(size), True, AF_INET, TCP_TABLE_OWNER_PID_ALL, 0)
263         if size.value == 0:
264             return []
265         buf = (ctypes.c_byte * size.value)()
266         res = self._GetExtendedTcpTable(buf, ctypes.byref(size), True, AF_INET, TCP_TABLE_OWNER_PID_ALL, 0)
267         if res != 0:
268             return []
269         num_entries = ctypes.cast(buf, ctypes.POINTER(wintypes.DWORD)).contents.value
270         row_size = ctypes.sizeof(MIB_TCPROW_OWNER_PID)
271         offset = ctypes.sizeof(wintypes.DWORD)
272         connections: List[NetworkSocketInfo] = []
273         for i in range(num_entries):
274             row_ptr = ctypes.byref(buf, offset + i * row_size)
275             row = ctypes.cast(row_ptr, ctypes.POINTER(MIB_TCPROW_OWNER_PID)).contents
276             state_str = TCP_STATE_MAP.get(row.dwState, f'STATE_{row.dwState}')
277             local_ip = self._ip_to_str(row.dwLocalAddr)
278             local_port = self._port_to_int(row.dwLocalPort)
279             remote_ip = self._ip_to_str(row.dwRemoteAddr)
280             remote_port = self._port_to_int(row.dwRemotePort)
281             if row.dwState != MIB_TCP_STATE_LISTEN else 0:
282                 connections.append(NetworkSocketInfo(protocol='TCP', local_address=local_ip, local_port=local_port, remote_address=remote_ip, remote_port=remote_port, state=state_str, pid=row.dwOwningPid))
283
284         return connections
285
286     def get_udp_sockets(self) -> List[NetworkSocketInfo]:
287         """Получение всех UDP слушающих сокетов с привязкой к PID.

```



```

286
287     Returns:
288         List[NetworkSocketInfo]: Список UDP сокетов.
289         """
290         size = wintypes.DWORD(0)
291         self._GetExtendedUdpTable(None, ctypes.byref(size), True, AF_INET, UDP_TABLE_OWNER_PID, 0)
292         if size.value == 0:
293             return []
294         buf = (ctypes.c_byte * size.value)()
295         res = self._GetExtendedUdpTable(buf, ctypes.byref(size), True, AF_INET, UDP_TABLE_OWNER_PID, 0)
296         if res != 0:
297             return []
298         num_entries = ctypes.cast(buf, ctypes.POINTER(wintypes.DWORD)).contents.value
299         row_size = ctypes.sizeof(MIB_UDPROW_OWNER_PID)
300         offset = ctypes.sizeof(wintypes.DWORD)
301         sockets_list: List[NetworkSocketInfo] = []
302         for i in range(num_entries):
303             row_ptr = ctypes.byref(buf, offset + i * row_size)
304             row = ctypes.cast(row_ptr, ctypes.POINTER(MIB_UDPROW_OWNER_PID)).contents
305             local_ip = self._ip_to_str(row.dwLocalAddr)
306             local_port = self._port_to_int(row.dwLocalPort)
307             sockets_list.append(NetworkSocketInfo(protocol='UDP', local_address=local_ip, local_port=local_port, remote_address='*.*.*.*', remote_port=0, state='LISTEN', pid=row.dwOwningPid))
308         return sockets_list
309
310     def send_arp(self, dest_ip: str, src_ip: Optional[str] = None) -> Optional[str]:
311         """Отправка ARP-запроса через SendARP API Windows и получение MAC-адреса.
312
313         Args:
314             dest_ip: Целевой IPv4-адрес.

```

```

315 |         src_ip: Исходный IPv4-адрес (опционально, 0 по умо
| лчанию).
316 |
317 |     Returns:
318 |         Optional[str]: Отформатированный MAC-адрес (e.g. '
| 00:1A:2B:3C:4D:5E') или None при отсутствии ответа.
319 |         """
320 |         if not getattr(self, '_SendARP', None):
321 |             return None
322 |         try:
323 |             dest_addr = struct.unpack('<L', socket.inet_aton(d
| est_ip))[0]
324 |             src_addr = struct.unpack('<L', socket.inet_aton(sr
| c_ip))[0] if src_ip else 0
325 |             mac_buf = (ctypes.c_ubyte * 6)()
326 |             mac_len = wintypes.ULONG(6)
327 |             res = self._SendARP(dest_addr, src_addr, ctypes.by
| ref(mac_buf), ctypes.byref(mac_len))
328 |             if res == 0 and mac_len.value == 6:
329 |                 return ':'.join(f'{b:02X}' for b in mac_buf)
330 |             return None
331 |         except Exception as ex:
332 |             logger.debug(f'SendARP error for {dest_ip}: {ex}')
333 |             return None
334 |
335 |     def get_ip_net_table(self) -> List[Dict[str, Any]]:
336 |         """Получение таблицы ARP-записей соседей ядра Windows
| через GetIpNetTable.
337 |
338 |     Returns:
339 |         List[Dict[str, Any]]: Список записей ARP соседей (
| ip, mac, interface_index, type).
340 |         """
341 |         if not getattr(self, '_GetIpNetTable', None):
342 |             return []
343 |         try:
344 |             size = wintypes.ULONG(0)
345 |             self._GetIpNetTable(None, ctypes.byref(size), True

```

```

    )
346         if size.value == 0:
347             return []
348         buf = (ctypes.c_byte * size.value)()
349         res = self._GetIpNetTable(buf, ctypes.byref(size),
    True)
350         if res != 0:
351             return []
352         num_entries = ctypes.cast(buf, ctypes.POINTER(wint
types.DWORD)).contents.value
353         row_size = ctypes.sizeof(MIB_IPNETROW)
354         offset = ctypes.sizeof(wintypes.DWORD)
355         neighbors: List[Dict[str, Any]] = []
356         for i in range(num_entries):
357             row_ptr = ctypes.byref(buf, offset + i * row_s
ize)
358             row = ctypes.cast(row_ptr, ctypes.POINTER(MIB_
IPNETROW)).contents
359             ip = self._ip_to_str(row.dwAddr)
360             if row.dwPhysAddrLen == 6:
361                 mac = ':'.join(f'{b:02X}' for b in row.bPh
ysAddr[:6])
362             else:
363                 mac = ''
364             net_type = IP_NET_TYPE_MAP.get(row.dwType, f'T
YPE_{row.dwType}')
365             if ip and ip != '0.0.0.0' and not ip.startswit
h('255.255.255') and not ip.startswith('224.') and net_type !=
'INVALID':
366                 neighbors.append({
367                     'ip': ip,
368                     'mac': mac,
369                     'interface_index': row.dwIndex,
370                     'type': net_type,
371                 })
372         return neighbors
373     except Exception as ex:
374         logger.debug(f'GetIpNetTable error: {ex}')

```


ntdll.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  | - Ntdll
4 # =====
5 # Description:
6 #   ntdll.dll wrapper for native NT API functions.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.ntdll import SYSTEM
  | M_INFO
11 #
12 #       service = SYSTEM_INFO()
13 #
14 # File: ntdll.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """ntdll.dll wrapper for native NT API functions."""
23
24 import ctypes
25 import ctypes.wintypes as wintypes
26 from typing import Optional, Any
27 from dataclasses import dataclass
28 STATUS_SUCCESS = 0
29 STATUS_BUFFER_TOO_SMALL = 3221225507
30 SystemBasicInformation = 0
31 SystemProcessorInformation = 1
```

32 SystemPerformanceInformation = 2
33 SystemTimeOfDayInformation = 3
34 SystemPathInformation = 4
35 SystemProcessInformation = 5
36 SystemCallCountInformation = 6
37 SystemDeviceInformation = 7
38 SystemProcessorPerformanceInformation = 8
39 SystemFlagsInformation = 9
40 SystemCallTimeInformation = 10
41 SystemModuleInformation = 11
42 SystemLocksInformation = 12
43 SystemStackTraceInformation = 13
44 SystemPagedPoolInformation = 14
45 SystemNonPagedPoolInformation = 15
46 SystemHandleInformation = 16
47 SystemObjectInformation = 17
48 SystemPageFileInformation = 18
49 SystemVdmInstemulInformation = 19
50 SystemVdmBopInformation = 20
51 SystemFileCacheInformation = 21
52 SystemPoolTagInformation = 22
53 SystemInterruptInformation = 23
54 SystemDpcBehaviorInformation = 24
55 SystemFullMemoryInformation = 25
56 SystemLoadGdiDriverInformation = 26
57 SystemUnloadGdiDriverInformation = 27
58 SystemTimeAdjustmentInformation = 28
59 SystemSummaryMemoryInformation = 29
60 SystemMirrorInformation = 30
61 SystemPerformanceTraceInformation = 31
62 SystemCrashDumpInformation = 32
63 SystemExceptionInformation = 33
64 SystemCrashDumpStateInformation = 34
65 SystemDebuggerInformation = 35
66 SystemProcessorSpeedInformation = 36
67 SystemNUMAProximityNodeInformation = 37
68 SystemDpcWatchdogInformation = 38
69 SystemDpcWatchdogInformation2 = 39

```

70 SystemInvalidProcessorSpeedInformation = 40
71
72 class SYSTEM_INFO(ctypes.Structure):
73     _fields_ = [('dwOemId', wintypes.DWORD), ('dwPageSize', wintypes.DWORD), ('lpMinimumApplicationAddress', wintypes.LPVOID), ('lpMaximumApplicationAddress', wintypes.LPVOID), ('dwActiveProcessorMask', wintypes.DWORD), ('dwNumberOfProcessors', wintypes.DWORD), ('dwProcessorType', wintypes.DWORD), ('dwAllocationGranularity', wintypes.DWORD), ('wProcessorLevel', wintypes.WORD), ('wProcessorRevision', wintypes.WORD)]
74
75 class UNICODE_STRING(ctypes.Structure):
76     _fields_ = [('Length', wintypes.USHORT), ('MaximumLength', wintypes.USHORT), ('Buffer', wintypes.LPWSTR)]
77
78 class OBJECT_NAME_INFORMATION(ctypes.Structure):
79     _fields_ = [('Name', UNICODE_STRING)]
80
81 class PROCESS_BASIC_INFORMATION(ctypes.Structure):
82     _fields_ = [('Reserved1', wintypes.LPVOID), ('PebBaseAddress', wintypes.LPVOID), ('Reserved2_0', wintypes.LPVOID), ('Reserved2_1', wintypes.LPVOID), ('UniqueProcessId', wintypes.LPVOID), ('Reserved3', wintypes.LPVOID)]
83
84 class RTL_OSVERSIONINFOEXW(ctypes.Structure):
85     _fields_ = [('dwOSVersionInfoSize', wintypes.DWORD), ('dwMajorVersion', wintypes.DWORD), ('dwMinorVersion', wintypes.DWORD), ('dwBuildNumber', wintypes.DWORD), ('dwPlatformId', wintypes.DWORD), ('szCSDVersion', ctypes.c_wchar * 128), ('wServicePackMajor', wintypes.WORD), ('wServicePackMinor', wintypes.WORD), ('wSuiteMask', wintypes.WORD), ('wProductType', wintypes.BYTE), ('wReserved', wintypes.BYTE)]
86
87 @dataclass
88 class VersionInfo:
89     """Windows version information."""
90     major: int
91     minor: int

```

```

92     build: int
93     platform: str
94
95 class NtdllAPI:
96     """Wrapper for ntdll native API functions."""
97
98     def __init__(self):
99         self.ntdll = ctypes.windll.ntdll
100         self._setup_function_signatures()
101
102     def _setup_function_signatures(self):
103         """Configure ctypes function signatures."""
104         self.ntdll.NtQuerySystemInformation.argtypes = [wintypes.DWORD, wintypes.LPVOID, wintypes.ULONG, ctypes.POINTER(wintypes.ULONG)]
105         self.ntdll.NtQuerySystemInformation.restype = wintypes.LONG
106         self.ntdll.NtQueryObject.argtypes = [wintypes.HANDLE, wintypes.DWORD, wintypes.LPVOID, wintypes.ULONG, ctypes.POINTER(wintypes.ULONG)]
107         self.ntdll.NtQueryObject.restype = wintypes.LONG
108         self.ntdll.NtQueryInformationProcess.argtypes = [wintypes.HANDLE, wintypes.DWORD, wintypes.LPVOID, wintypes.ULONG, ctypes.POINTER(wintypes.ULONG)]
109         self.ntdll.NtQueryInformationProcess.restype = wintypes.LONG
110         self.ntdll.RtlGetVersion.argtypes = [ctypes.POINTER(RTL_OSVERSIONINFOEXW)]
111         self.ntdll.RtlGetVersion.restype = wintypes.LONG
112
113     def get_windows_version(self) -> Optional[VersionInfo]:
114         """
115         Get Windows version information.
116
117         Returns:
118             VersionInfo object or None
119         """
120         try:

```



```

121         version_info = RTL_OSVERSIONINFOEXW()
122         version_info.dwOSVersionInfoSize = ctypes.sizeof(R
TL_OSVERSIONINFOEXW)
123         status = self.ntdll.RtlGetVersion(ctypes.byref(ver
sion_info))
124         if status == STATUS_SUCCESS:
125             platform_map = {0: 'Win32s', 1: 'Windows 9x',
2: 'Windows NT/2000/XP/Vista/7/8/10/11'}
126             return VersionInfo(major=version_info.dwMajorV
ersion, minor=version_info.dwMinorVersion, build=version_info.
dwBuildNumber, platform=platform_map.get(version_info.dwPlatfo
rmId, 'Unknown'))
127         return None
128     except Exception:
129         return None
130
131     def query_system_information(self, info_class: int) -> Opt
ional[bytes]:
132         """
133         Query system information.
134
135         Args:
136             info_class: System information class
137
138         Returns:
139             System information bytes or None
140         """
141         try:
142             buffer_size = 8192
143             buffer = ctypes.create_string_buffer(buffer_size)
144             return_length = wintypes.ULONG()
145             status = self.ntdll.NtQuerySystemInformation(info_
class, buffer, buffer_size, ctypes.byref(return_length))
146             if status == STATUS_SUCCESS:
147                 return buffer.raw[:return_length.value]
148             return None
149         except Exception:
150             return None

```

```

151
152     def query_object_name(self, handle: int) -> Optional[str]:
153         """
154         Query the name of a kernel object.
155
156         Args:
157             handle: Object handle
158
159         Returns:
160             Object name or None
161         """
162         try:
163             buffer_size = 1024
164             buffer = ctypes.create_string_buffer(buffer_size)
165             return_length = wintypes.ULONG()
166             status = self.ntdll.NtQueryObject(handle, 1, buffer,
167 r, buffer_size, ctypes.byref(return_length))
168             if status == STATUS_SUCCESS:
169                 obj_name_info = ctypes.cast(buffer, ctypes.POINTER(OBJECT_NAME_INFORMATION))
170                 if obj_name_info.contents.Name.Buffer:
171                     return obj_name_info.contents.Name.Buffer
172                 return None
173             except Exception:
174                 return None
175
176     def query_process_basic_info(self, pid: int) -> Optional[PROCESS_BASIC_INFORMATION]:
177         """
178         Query basic process information.
179
180         Args:
181             pid: Process ID
182
183         Returns:
184             PROCESS_BASIC_INFORMATION structure or None
185         """
186         try:

```

```
186 |         handle = ctypes.windll.kernel32.OpenProcess(1024,  
    | False, pid)  
187 |         if not handle:  
188 |             return None  
189 |         try:  
190 |             pbi = PROCESS_BASIC_INFORMATION()  
191 |             return_length = wintypes.ULONG()  
192 |             status = self.ntdll.NtQueryInformationProcess(  
    | handle, 0, ctypes.byref(pbi), ctypes.sizeof(pbi), ctypes.byref  
    | (return_length))  
193 |             if status == STATUS_SUCCESS:  
194 |                 return pbi  
195 |             return None  
196 |         finally:  
197 |             ctypes.windll.kernel32.CloseHandle(handle)  
198 |     except Exception:  
199 |         return None
```

psapi.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Psapi
5 # Description:
6 #   psapi.dll wrapper for process memory and module enumeration.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_ffapi import PROCESS_MEMORY_COUNTERS
11 #
12 #       service = PROCESS_MEMORY_COUNTERS()
13 #
14 # File: psapi.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_ffapi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 """psapi.dll wrapper for process memory and module enumeration
23 ."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 from typing import List, Optional, Tuple
28 from dataclasses import dataclass
29 LIST_MODULES_DEFAULT = 0
30 LIST_MODULES_32BIT = 1
```

```

30 LIST_MODULES_64BIT = 2
31 LIST_MODULES_ALL = 3
32 SIZE_T = getattr(wintypes, 'SIZE_T', ctypes.c_size_t)
33
34 class PROCESS_MEMORY_COUNTERS(ctypes.Structure):
35     _fields_ = [('cb', wintypes.DWORD), ('PageFaultCount', win
types.DWORD), ('PeakWorkingSetSize', SIZE_T), ('WorkingSetSize
', SIZE_T), ('QuotaPeakPagedPoolUsage', SIZE_T), ('QuotaPagedP
oolUsage', SIZE_T), ('QuotaPeakNonPagedPoolUsage', SIZE_T), ('
QuotaNonPagedPoolUsage', SIZE_T), ('PagefileUsage', SIZE_T), (
'PeakPagefileUsage', SIZE_T)]
36
37 @dataclass
38 class MemoryInfo:
39     """Process memory statistics from psapi."""
40     page_faults: int
41     peak_working_set: int
42     working_set: int
43     paged_pool_peak: int
44     paged_pool_usage: int
45     nonpaged_pool_peak: int
46     nonpaged_pool_usage: int
47     pagefile_usage: int
48     pagefile_peak: int
49
50 class PsapiAPI:
51     """Wrapper for psapi process memory and module functions."
    ""
52
53     def __init__(self):
54         self.psapi = ctypes.windll.psapi
55         self._setup_function_signatures()
56
57     def _setup_function_signatures(self):
58         """Configure ctypes function signatures."""
59         self.psapi.EnumProcesses.argtypes = [ctypes.POINTER(wi
ntypes.DWORD), wintypes.DWORD, ctypes.POINTER(wintypes.DWORD)]
60         self.psapi.EnumProcesses.restype = wintypes.BOOL

```

```

61         self.psapi.GetProcessMemoryInfo.argtypes = [wintypes.H
HANDLE, ctypes.POINTER(PROCESS_MEMORY_COUNTERS), wintypes.DWORD
]
62         self.psapi.GetProcessMemoryInfo.restype = wintypes.BOOL
63         self.psapi.EnumProcessModules.argtypes = [wintypes.HAN
DLE, ctypes.POINTER(wintypes.HMODULE), wintypes.DWORD, ctypes.
POINTER(wintypes.DWORD)]
64         self.psapi.EnumProcessModules.restype = wintypes.BOOL
65         self.psapi.EnumProcessModulesEx.argtypes = [wintypes.H
HANDLE, ctypes.POINTER(wintypes.HMODULE), wintypes.DWORD, ctype
s.POINTER(wintypes.DWORD), wintypes.DWORD]
66         self.psapi.EnumProcessModulesEx.restype = wintypes.BOOL
67         self.psapi.GetModuleFileNameExW.argtypes = [wintypes.H
HANDLE, wintypes.HMODULE, wintypes.LPWSTR, wintypes.DWORD]
68         self.psapi.GetModuleFileNameExW.restype = wintypes.DWORD
69         self.psapi.GetProcessImageFileNameW.argtypes = [wintyp
es.HANDLE, wintypes.LPWSTR, wintypes.DWORD]
70         self.psapi.GetProcessImageFileNameW.restype = wintypes
.DWORD
71
72     def enumerate_processes(self) -> List[int]:
73         """
74         Get all process IDs.
75
76         Returns:
77             List of process IDs
78         """
79         max_pids = 1024
80         pids = (wintypes.DWORD * max_pids)()
81         count_ptr = wintypes.DWORD()
82         if self.psapi.EnumProcesses(pids, ctypes.sizeof(pids),
ctypes.byref(count_ptr)):
83             count = count_ptr.value // ctypes.sizeof(wintypes.
DWORD)
84             return list(pids[:count])

```

```

85         return []
86
87     def get_process_memory_info(self, pid: int) -> Optional[Me
memoryInfo]:
88         """
89         Get memory statistics for a process.
90
91         Args:
92             pid: Process ID
93
94         Returns:
95             MemoryInfo object or None
96         """
97         try:
98             handle = ctypes.windll.kernel32.OpenProcess(1024,
False, pid)
99             if not handle:
100                 return None
101             try:
102                 pmc = PROCESS_MEMORY_COUNTERS()
103                 pmc.cb = ctypes.sizeof(PROCESS_MEMORY_COUNTERS
)
104                 if self.psapi.GetProcessMemoryInfo(handle, cty
pes.byref(pmc), ctypes.sizeof(pmc)):
105                     return MemoryInfo(page_faults=pmc.PageFaul
tCount, peak_working_set=pmc.PeakWorkingSetSize, working_set=p
mc.WorkingSetSize, paged_pool_peak=pmc.QuotaPeakPagedPoolUsage
, paged_pool_usage=pmc.QuotaPagedPoolUsage, nonpaged_pool_peak
=pmc.QuotaPeakNonPagedPoolUsage, nonpaged_pool_usage=pmc.Quota
NonPagedPoolUsage, pagefile_usage=pmc.PagefileUsage, pagefile_
peak=pmc.PeakPagefileUsage)
106                 return None
107             finally:
108                 ctypes.windll.kernel32.CloseHandle(handle)
109         except Exception:
110             return None
111
112     def enumerate_process_modules(self, pid: int, filter_type:

```

```

    int=LIST_MODULES_ALL) -> List[str]:
113         """
114         Get loaded modules for a process.
115
116         Args:
117             pid: Process ID
118             filter_type: Module filter (32-bit, 64-bit, or all
119         )
120
121         Returns:
122             List of module paths
123         """
124         modules = []
125         try:
126             handle = ctypes.windll.kernel32.OpenProcess(4096 |
127                 1024, False, pid)
128             if not handle:
129                 return modules
130             try:
131                 max_modules = 256
132                 module_handles = (wintypes.HMODULE * max_modul
133             es)()
134                 count_ptr = wintypes.DWORD()
135                 if self.psapi.EnumProcessModulesEx(handle, mod
136             ule_handles, ctypes.sizeof(module_handles), ctypes.byref(count
137             _ptr), filter_type):
138                     count = count_ptr.value // ctypes.sizeof(w
139             intypes.HMODULE)
140                     for i in range(count):
141                         filename_buffer = ctypes.create_unicod
142             e_buffer(260)
143                         if self.psapi.GetModuleFileNameExW(han
144             dle, module_handles[i], filename_buffer, len(filename_buffer))
145             :
146                             modules.append(filename_buffer.val
147             ue)
148             finally:
149                 ctypes.windll.kernel32.CloseHandle(handle)

```



```
140         except Exception:
141             pass
142         return modules
143
144     def get_process_image_filename(self, pid: int) -> Optional
145     [str]:
146         """
147         Get the executable path of a process.
148
149         Args:
150             pid: Process ID
151
152         Returns:
153             Process image filename or None
154         """
155         try:
156             handle = ctypes.windll.kernel32.OpenProcess(1024,
157             False, pid)
158             if not handle:
159                 return None
160             try:
161                 filename_buffer = ctypes.create_unicode_buffer
162                 (260)
163                 if self.psapi.GetProcessImageFileNameW(handle,
164                 filename_buffer, len(filename_buffer)):
165                     return filename_buffer.value
166                 return None
167             finally:
168                 ctypes.windll.kernel32.CloseHandle(handle)
169         except Exception:
170             return None
```

scm.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
  - Scm
4 # =====
5 # Description:
6 #   Низкоуровневый интерфейс к Service Control Manager (advapi
  32.dll) Windows.
7 #
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.scm import SERVICE
  _STATUS_PROCESS
11 #
12 #       service = SERVICE_STATUS_PROCESS()
13 #
14 # File: scm.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к Service Control Manager (advapi3
  2.dll) Windows."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 from dataclasses import dataclass
28 from typing import Any, Dict, List, Optional
29 from logger import logger
```

```

30 SC_MANAGER_CONNECT = 1
31 SC_MANAGER_CREATE_SERVICE = 2
32 SC_MANAGER_ENUMERATE_SERVICE = 4
33 SC_MANAGER_LOCK = 8
34 SC_MANAGER_QUERY_LOCK_STATUS = 16
35 SC_MANAGER_MODIFY_BOOT_CONFIG = 32
36 SC_MANAGER_ALL_ACCESS = 983103
37 SERVICE_WIN32_OWN_PROCESS = 16
38 SERVICE_WIN32_SHARE_PROCESS = 32
39 SERVICE_DRIVER = 11
40 SERVICE_TYPE_ALL = SERVICE_WIN32_OWN_PROCESS | SERVICE_WIN32_S
    HARE_PROCESS | SERVICE_DRIVER
41 SERVICE_ACTIVE = 1
42 SERVICE_INACTIVE = 2
43 SERVICE_STATE_ALL = SERVICE_ACTIVE | SERVICE_INACTIVE
44 SERVICE_STOPPED = 1
45 SERVICE_START_PENDING = 2
46 SERVICE_STOP_PENDING = 3
47 SERVICE_RUNNING = 4
48 SERVICE_CONTINUE_PENDING = 5
49 SERVICE_PAUSE_PENDING = 6
50 SERVICE_PAUSED = 7
51 SERVICE_QUERY_CONFIG = 1
52 SERVICE_CHANGE_CONFIG = 2
53 SERVICE_QUERY_STATUS = 4
54 SERVICE_ENUMERATE_DEPENDENTS = 8
55 SERVICE_START = 16
56 SERVICE_STOP = 32
57 SERVICE_PAUSE_CONTINUE = 64
58 SERVICE_INTERROGATE = 128
59 SERVICE_USER_DEFINED_CONTROL = 256
60 SERVICE_ALL_ACCESS = 983551
61 SC_ENUM_PROCESS_INFO = 0
62 ERROR_MORE_DATA = 234
63
64 class SERVICE_STATUS_PROCESS(ctypes.Structure):
65     """Структура статуса и процесса службы."""
66     _fields_ = [('dwServiceType', wintypes.DWORD), ('dwCurrent

```

```
State', wintypes.DWORD), ('dwControlsAccepted', wintypes.DWORD), ('dwWin32ExitCode', wintypes.DWORD), ('dwServiceSpecificExitCode', wintypes.DWORD), ('dwCheckPoint', wintypes.DWORD), ('dwWaitHint', wintypes.DWORD), ('dwProcessId', wintypes.DWORD), ('dwServiceFlags', wintypes.DWORD)]
```

67

```
68 class ENUM_SERVICE_STATUS_PROCESSW(ctypes.Structure):
```

```
69     """Структура элемента перечисления служб с PID."""
```

```
70     _fields_ = [('lpServiceName', wintypes.LPWSTR), ('lpDisplayName', wintypes.LPWSTR), ('ServiceStatusProcess', SERVICE_STATUS_PROCESS)]
```

71

```
72 class QUERY_SERVICE_CONFIGW(ctypes.Structure):
```

```
73     """Структура детальной конфигурации службы."""
```

```
74     _fields_ = [('dwServiceType', wintypes.DWORD), ('dwStartType', wintypes.DWORD), ('dwErrorControl', wintypes.DWORD), ('lpBinaryPathName', wintypes.LPWSTR), ('lpLoadOrderGroup', wintypes.LPWSTR), ('dwTagId', wintypes.DWORD), ('lpDependencies', wintypes.LPWSTR), ('lpServiceStartName', wintypes.LPWSTR), ('lpDisplayName', wintypes.LPWSTR)]
```

75

```
76 @dataclass
```

```
77 class ServiceInfo:
```

```
78     """Нормализованные данные о службе Windows."""
```

```
79     name: str
```

```
80     display_name: str
```

```
81     state: str
```

```
82     state_code: int
```

```
83     pid: int
```

```
84     service_type: int
```

```
85     binary_path: str = ''
```

```
86     start_type: str = ''
```

```
87     account: str = ''
```

88

```
89     def to_dict(self) -> Dict[str, Any]:
```

```
90         """Преобразование в словарь."""
```

```
91         return {'name': self.name, 'display_name': self.display_name, 'state': self.state, 'state_code': self.state_code, 'p
```

```

id': self.pid, 'service_type': self.service_type, 'binary_path':
self.binary_path, 'start_type': self.start_type, 'account':
self.account}

92
93 class ServiceControlManager:
94     """Низкоуровневый клиент к Service Control Manager (advapi
32.dll)."""
95     STATE_MAP = {SERVICE_STOPPED: 'STOPPED', SERVICE_START_PEN
DING: 'START_PENDING', SERVICE_STOP_PENDING: 'STOP_PENDING', S
ERVICE_RUNNING: 'RUNNING', SERVICE_CONTINUE_PENDING: 'CONTINUE
_PENDING', SERVICE_PAUSE_PENDING: 'PAUSE_PENDING', SERVICE_PAU
SED: 'PAUSED'}
96     START_TYPE_MAP = {0: 'BOOT', 1: 'SYSTEM', 2: 'AUTO_START',
3: 'DEMAND_START', 4: 'DISABLED'}
97
98     def __init__(self) -> None:
99         """Инициализация функций advapi32.dll."""
100         self._advapi32 = ctypes.windll.advapi32
101         self._OpenSCManagerW = self._advapi32.OpenSCManagerW
102         self._OpenSCManagerW.argtypes = [wintypes.LPCWSTR, win
types.LPCWSTR, wintypes.DWORD]
103         self._OpenSCManagerW.restype = wintypes.SC_HANDLE
104         self._EnumServicesStatusExW = self._advapi32.EnumServi
cesStatusExW
105         self._EnumServicesStatusExW.argtypes = [wintypes.SC_HA
NDLE, wintypes.DWORD, wintypes.DWORD, wintypes.DWORD, wintypes
.LPBYTE, wintypes.DWORD, ctypes.POINTER(wintypes.DWORD), ctype
s.POINTER(wintypes.DWORD), ctypes.POINTER(wintypes.DWORD), win
types.LPCWSTR]
106         self._EnumServicesStatusExW.restype = wintypes.BOOL
107         self._OpenServiceW = self._advapi32.OpenServiceW
108         self._OpenServiceW.argtypes = [wintypes.SC_HANDLE, win
types.LPCWSTR, wintypes.DWORD]
109         self._OpenServiceW.restype = wintypes.SC_HANDLE
110         self._QueryServiceConfigW = self._advapi32.QueryServic
eConfigW
111         self._QueryServiceConfigW.argtypes = [wintypes.SC_HAN
DLE, wintypes.LPBYTE, wintypes.DWORD, ctypes.POINTER(wintypes.D

```

```

WORD)]
112         self._QueryServiceConfigW.restype = wintypes.BOOL
113         self._CloseServiceHandle = self._advapi32.CloseService
Handle
114         self._CloseServiceHandle.argtypes = [wintypes.SC_HANDL
E]
115         self._CloseServiceHandle.restype = wintypes.BOOL
116
117         def enum_services(self, service_type: int=SERVICE_WIN32_OW
N_PROCESS | SERVICE_WIN32_SHARE_PROCESS) -> List[ServiceInfo]:
118             """Быстрое пакетное перечисление всех служб через Enum
ServicesStatusExW.
119
120             Args:
121                 service_type: Маска типов служб (Win32, Driver, et
с.).
122
123             Returns:
124                 List[ServiceInfo]: Список обнаруженных служб с PID
и статусом.
125             """
126             h_scm = self._OpenSCManagerW(None, None, SC_MANAGER_EN
UMERATE_SERVICE | SC_MANAGER_CONNECT)
127             if not h_scm:
128                 logger.debug('Не удалось открыть SCManager через W
inAPI')
129                 return []
130             services: List[ServiceInfo] = []
131             try:
132                 bytes_needed = wintypes.DWORD(0)
133                 services_returned = wintypes.DWORD(0)
134                 resume_handle = wintypes.DWORD(0)
135                 self._EnumServicesStatusExW(h_scm, SC_ENUM_PROCESS
_INFO, service_type, SERVICE_STATE_ALL, None, 0, ctypes.byref(
bytes_needed), ctypes.byref(services_returned), ctypes.byref(r
esume_handle), None)
136                 if bytes_needed.value == 0:
137                     return []

```

```

138         buf = (ctypes.c_byte * bytes_needed.value)()
139         success = self._EnumServicesStatusExW(h_scm, SC_EN
UM_PROCESS_INFO, service_type, SERVICE_STATE_ALL, ctypes.cast(
buf, wintypes.LPBYTE), bytes_needed.value, ctypes.byref(bytes_
needed), ctypes.byref(services_returned), ctypes.byref(resume_
handle), None)
140         if not success and ctypes.GetLastError() != ERROR_
MORE_DATA:
141             return []
142             entry_ptr = ctypes.cast(buf, ctypes.POINTER(ENUM_S
ERVICE_STATUS_PROCESSW))
143             for i in range(services_returned.value):
144                 entry = entry_ptr[i]
145                 state_code = entry.ServiceStatusProcess.dwCurr
entState
146                 state_str = self.STATE_MAP.get(state_code, 'UN
KNOWN')
147                 pid = entry.ServiceStatusProcess.dwProcessId
148                 svc = ServiceInfo(name=entry.lpServiceName or
'', display_name=entry.lpDisplayName or entry.lpServiceName or
'', state=state_str, state_code=state_code, pid=pid, service_
type=entry.ServiceStatusProcess.dwServiceType)
149                 services.append(svc)
150             except Exception as ex:
151                 logger.debug(f'Исключение при нативном перечислени
и служб SCM: {ex}')
152             finally:
153                 self._CloseServiceHandle(h_scm)
154             return services
155
156     def get_service_config(self, service_name: str) -> Optiona
l[Dict[str, Any]]:
157         """Получение подробной конфигурации службы через Query
ServiceConfigW.
158
159         Args:
160             service_name: Системное имя службы.
161

```

```

162         Returns:
163             Optional[Dict[str, Any]]: Данные конфигурации или
None при ошибке.
164         """
165         h_scm = self._OpenSCManagerW(None, None, SC_MANAGER_CO
NNECT)
166         if not h_scm:
167             return None
168         try:
169             h_svc = self._OpenServiceW(h_scm, service_name, SE
RVICE_QUERY_CONFIG)
170             if not h_svc:
171                 return None
172             try:
173                 bytes_needed = wintypes.DWORD(0)
174                 self._QueryServiceConfigW(h_svc, None, 0, ctypes
es.byref(bytes_needed))
175                 if bytes_needed.value == 0:
176                     return None
177                 buf = (ctypes.c_byte * bytes_needed.value)()
178                 if not self._QueryServiceConfigW(h_svc, ctypes
.cast(buf, wintypes.LPBYTE), bytes_needed.value, ctypes.byref(
bytes_needed)):
179                     return None
180                 cfg = ctypes.cast(buf, ctypes.POINTER(QUERY_SE
RVICE_CONFIGW)).contents
181                 return {'binary_path': cfg.lpBinaryPathName or
'', 'start_type': self.START_TYPE_MAP.get(cfg.dwStartType, st
r(cfg.dwStartType)), 'start_type_code': cfg.dwStartType, 'acco
unt': cfg.lpServiceStartName or '', 'display_name': cfg.lpDisp
layName or service_name, 'dependencies': cfg.lpDependencies or
''}
182             finally:
183                 self._CloseServiceHandle(h_svc)
184         except Exception as ex:
185             logger.debug(f"Ошибка чтения конфигурации службы '{
{service_name}': {ex}")
186             return None

```



```
187 |         finally:  
188 |             self._CloseServiceHandle(h_scm)
```

setupapi.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Setupapi
5 # Description:
6 # Низкоуровневый интерфейс к SetupAPI и Configuration Manager PnP.
7 #
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.telemetry.win32_fffi.setupapi import GUID
11 #
12 # service = GUID()
13 #
14 # File: setupapi.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-01 13:28:28
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к SetupAPI и Configuration Manager PnP."""
24
25 import ctypes
26 import ctypes.wintypes as wintypes
27 from dataclasses import dataclass
28 from typing import Any, Dict, List, Optional
29 from logger import logger
```

```

30 DIGCF_DEFAULT = 1
31 DIGCF_PRESENT = 2
32 DIGCF_ALLCLASSES = 4
33 DIGCF_PROFILE = 8
34 DIGCF_DEVICEINTERFACE = 16
35 SPDRP_DEVICEDESC = 0
36 SPDRP_HARDWAREID = 1
37 SPDRP_CLASS = 7
38 SPDRP_CLASSGUID = 8
39 SPDRP_DRIVER = 9
40 SPDRP_FRIENDLYNAME = 12
41 SPDRP_LOCATION_INFORMATION = 13
42 SPDRP_MFG = 11
43 DN_ROOT_ENUMERATED = 1
44 DN_DRIVER_LOADED = 2
45 DN_ENUM_LOADED = 4
46 DN_STARTED = 8
47 DN_MANUAL = 16
48 DN_NEED_TO_ENUM = 32
49 DN_NOT_FIRST_TIME = 64
50 DN_HARDWARE_ENUM = 128
51 DN_LIBCONFIG = 256
52 DN_FILTERED = 512
53 DN_MOVED = 1024
54 DN_DISABLEABLE = 2048
55 DN_REMOVABLE = 4096
56 DN_HAS_PROBLEM = 1024
57 DN_PRIVATE_PROBLEM = 32768
58 CR_SUCCESS = 0
59 INVALID_HANDLE_VALUE = ctypes.c_void_p(-1).value
60
61 class GUID(ctypes.Structure):
62     """Структура Windows GUID."""
63     _fields_ = [('Data1', wintypes.DWORD), ('Data2', wintypes.
64         WORD), ('Data3', wintypes.WORD), ('Data4', wintypes.BYTE * 8)]
65
66 class SP_DEVINFO_DATA(ctypes.Structure):
67     """Структура информации об устройстве SetupAPI."""

```

```

67     _fields_ = [('cbSize', wintypes.DWORD), ('ClassGuid', GUID
    ), ('DevInst', wintypes.DWORD), ('Reserved', ctypes.c_void_p)]
68
69 @dataclass
70 class PnPDeviceInfo:
71     """Нормализованные данные об аппаратном устройстве PnP."""
72     device_instance_id: str
73     friendly_name: str
74     hardware_id: str
75     device_class: str
76     manufacturer: str
77     has_problem: bool
78     status_code: int
79     problem_code: int
80
81     def to_dict(self) -> Dict[str, Any]:
82         """Преобразование в словарь."""
83         return {'device_instance_id': self.device_instance_id,
84             'friendly_name': self.friendly_name, 'hardware_id': self.hard
85             ware_id, 'device_class': self.device_class, 'manufacturer': se
86             lf.manufacturer, 'has_problem': self.has_problem, 'status_code
87             ': self.status_code, 'problem_code': self.problem_code}
88
89 class SetupAPI:
90     """Низкоуровневый клиент к SetupAPI и CfgMgr32."""
91
92     def __init__(self) -> None:
93         """Инициализация библиотек setupapi.dll и cfgmgr32.dll
94         ."""
95         self._setupapi = ctypes.windll.setupapi
96         self._cfgmgr32 = ctypes.windll.cfgmgr32
97         self._SetupDiGetClassDevsW = self._setupapi.SetupDiGet
98         ClassDevsW
99         self._SetupDiGetClassDevsW.argtypes = [ctypes.c_void_p
100         , wintypes.LPCWSTR, wintypes.HWND, wintypes.DWORD]
101         self._SetupDiGetClassDevsW.restype = ctypes.c_void_p
102         self._SetupDiEnumDeviceInfo = self._setupapi.SetupDiEn
103         umDeviceInfo

```

```

96         self._SetupDiEnumDeviceInfo.argtypes = [ctypes.c_void_p, wintypes.DWORD, ctypes.POINTER(SP_DEVINFO_DATA)]
97         self._SetupDiEnumDeviceInfo.restype = wintypes.BOOL
98         self._SetupDiGetDeviceRegistryPropertyW = self._setupapi.SetupDiGetDeviceRegistryPropertyW
99         self._SetupDiGetDeviceRegistryPropertyW.argtypes = [ctypes.c_void_p, ctypes.POINTER(SP_DEVINFO_DATA), wintypes.DWORD, ctypes.POINTER(wintypes.DWORD), wintypes.LPBYTE, wintypes.DWORD, ctypes.POINTER(wintypes.DWORD)]
100        self._SetupDiGetDeviceRegistryPropertyW.restype = wintypes.BOOL
101        self._SetupDiDestroyDeviceInfoList = self._setupapi.SetupDiDestroyDeviceInfoList
102        self._SetupDiDestroyDeviceInfoList.argtypes = [ctypes.c_void_p]
103        self._SetupDiDestroyDeviceInfoList.restype = wintypes.BOOL
104        self._CM_Get_DevNode_Status = self._cfgmgr32.CM_Get_DevNode_Status
105        self._CM_Get_DevNode_Status.argtypes = [ctypes.POINTER(wintypes.ULONG), ctypes.POINTER(wintypes.ULONG), wintypes.DWORD, wintypes.ULONG]
106        self._CM_Get_DevNode_Status.restype = wintypes.DWORD
107        self._CM_Get_Device_IDW = self._cfgmgr32.CM_Get_Device_IDW
108        self._CM_Get_Device_IDW.argtypes = [wintypes.DWORD, wintypes.LPWSTR, wintypes.ULONG, wintypes.ULONG]
109        self._CM_Get_Device_IDW.restype = wintypes.DWORD
110
111        def _get_property_string(self, h_dev_info: ctypes.c_void_p, dev_data: SP_DEVINFO_DATA, prop: int) -> str:
112            """Получение строкового свойства устройства из реестра SetupAPI."""
113            reg_type = wintypes.DWORD(0)
114            req_size = wintypes.DWORD(0)
115            self._SetupDiGetDeviceRegistryPropertyW(h_dev_info, ctypes.byref(dev_data), prop, ctypes.byref(reg_type), None, 0, ctypes.byref(req_size))

```

```

116         if req_size.value == 0:
117             return ''
118         buf = (ctypes.c_byte * req_size.value)()
119         if not self._SetupDiGetDeviceRegistryPropertyW(h_dev_i
nfo, ctypes.byref(dev_data), prop, ctypes.byref(reg_type), cty
pes.cast(buf, wintypes.LPBYTE), req_size.value, ctypes.byref(r
eq_size)):
120             return ''
121         try:
122             return ctypes.wstring_at(buf)
123         except Exception:
124             return ''
125
126     def get_all_devices(self, only_present: bool=True) -> List
[PnPDeviceInfo]:
127         """Получение списка всех устройств PnP с их статусом и
кодами проблем.
128
129         Args:
130             only_present: Фильтровать только подключенные устр
ойства.
131
132         Returns:
133             List[PnPDeviceInfo]: Список PnP устройств.
134         """
135         flags = DIGCF_ALLCLASSES
136         if only_present:
137             flags |= DIGCF_PRESENT
138         h_dev_info = self._SetupDiGetClassDevsW(None, None, No
ne, flags)
139         if not h_dev_info or h_dev_info == INVALID_HANDLE_VALU
E:
140             logger.debug('Не удалось получить дескриптор Setup
DiGetClassDevsW')
141             return []
142         devices: List[PnPDeviceInfo] = []
143         try:
144             dev_data = SP_DEVINFO_DATA()

```

```

145         dev_data.cbSize = ctypes.sizeof(SP_DEVINFO_DATA)
146         idx = 0
147         while self._SetupDiEnumDeviceInfo(h_dev_info, idx,
        ctypes.byref(dev_data)):
148             idx += 1
149             try:
150                 friendly_name = self._get_property_string(
        h_dev_info, dev_data, SPDRP_FRIENDLYNAME)
151                 if not friendly_name:
152                     friendly_name = self._get_property_str
        ing(h_dev_info, dev_data, SPDRP_DEVICEDESC)
153                 hw_id = self._get_property_string(h_dev_in
        fo, dev_data, SPDRP_HARDWAREID)
154                 cls_name = self._get_property_string(h_dev
        _info, dev_data, SPDRP_CLASS)
155                 mfg = self._get_property_string(h_dev_info
        , dev_data, SPDRP_MFG)
156                 id_buf = (ctypes.c_uchar * 512)()
157                 cm_res = self._CM_Get_Device_IDW(dev_data.
        DevInst, id_buf, 512, 0)
158                 device_instance_id = id_buf.value if cm_re
        s == CR_SUCCESS else ''
159                 status = wintypes.ULONG(0)
160                 prob_code = wintypes.ULONG(0)
161                 cm_status = self._CM_Get_DevNode_Status(ct
        ypes.byref(status), ctypes.byref(prob_code), dev_data.DevInst,
        0)
162                 has_problem = False
163                 problem_num = 0
164                 if cm_status == CR_SUCCESS:
165                     problem_num = prob_code.value
166                     has_problem = status.value & DN_HAS_PR
        OBLEM != 0 or problem_num != 0
167                 devices.append(PnPDeviceInfo(device_instan
        ce_id=device_instance_id, friendly_name=friendly_name or devic
        e_instance_id or 'Устройство без имени', hardware_id=hw_id, de
        vice_class=cls_name, manufacturer=mfg, has_problem=has_problem
        , status_code=status.value if cm_status == CR_SUCCESS else 0,

```

```
        problem_code=problem_num))
168 |             except Exception:
169 |                 continue
170 |         finally:
171 |             self._SetupDiDestroyDeviceInfoList(h_dev_info)
172 |         return devices
173 |
174 |     def get_problem_devices(self) -> List[PnPDeviceInfo]:
175 |         """Быстрый поиск только устройств с аппаратными сбоями
        (Code 10/43/28).
176 |
177 |         Returns:
178 |             List[PnPDeviceInfo]: Список проблемных устройств.
179 |         """
180 |         all_devs = self.get_all_devices(only_present=True)
181 |         return [d for d in all_devs if d.has_problem and d.problem_code > 0]
```


tasksched.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Tasksched
5 # =====
6 # Description:
7 #   Интерфейс к Task Scheduler 2.0 COM API Windows.
8 # Usage Examples:
9 #   Python API:
10 #       from apps.windows.telemetry.win32_fffi.tasksched import T
11 #       askSchedulerAPI
12 #
13 #       service = TaskSchedulerAPI()
14 #
15 # File: tasksched.py
16 # Project: ai-breadboard
17 # Package: apps.windows.telemetry.win32_fffi
18 # Author: hypo69
19 # Copyright: © 2026 hypo69
20 # Updated: 2026-10-06 12:35:00
21 # =====
22
23 from __future__ import annotations
24
25 """Интерфейс к Task Scheduler 2.0 COM API Windows."""
26
27 import sys
28 from datetime import datetime
29 from typing import Any, Dict, List, Optional
30 from logger import logger
31
32 TASK_STATE_UNKNOWN = 0
33 TASK_STATE_DISABLED = 1
```

```

32 TASK_STATE_QUEUED = 2
33 TASK_STATE_READY = 3
34 TASK_STATE_RUNNING = 4
35 TASK_STATE_MAP = {
36     TASK_STATE_UNKNOWN: 'Unknown',
37     TASK_STATE_DISABLED: 'Disabled',
38     TASK_STATE_QUEUED: 'Queued',
39     TASK_STATE_READY: 'Ready',
40     TASK_STATE_RUNNING: 'Running'
41 }
42
43 TASK_ACTION_EXEC = 0
44 TASK_ACTION_COM_HANDLER = 5
45 TASK_ACTION_SEND_EMAIL = 6
46 TASK_ACTION_SHOW_MESSAGE = 7
47
48 TASK_TRIGGER_TYPE_MAP = {
49     0: 'On event',
50     1: 'Time',
51     2: 'Daily',
52     3: 'Weekly',
53     4: 'Monthly',
54     5: 'Monthly DOW',
55     6: 'On idle',
56     7: 'On registration',
57     8: 'At startup',
58     9: 'At logon',
59     11: 'Session state change',
60 }
61
62
63 def _format_com_date(dt_obj: Any) -> Optional[str]:
64     """Форматирует дату из COM API в читаемую строку.
65
66     Args:
67         dt_obj: COM объект даты или datetime.
68
69     Returns:

```

```

70         Optional[str]: Отформатированная дата или None.
71     """
72     if dt_obj is None:
73         return None
74     try:
75         # COM дата 1899 года означает отсутствие даты
76         if hasattr(dt_obj, 'year'):
77             if dt_obj.year < 1910 or dt_obj.year > 2100:
78                 return None
79             return dt_obj.strftime('%Y-%m-%d %H:%M:%S')
80         s = str(dt_obj).strip()
81         if '1899' in s or s == '' or s == 'None':
82             return None
83         return s
84     except Exception:
85         return None
86
87
88 class TaskSchedulerAPI:
89     """Обертка над COM-интерфейсом Task Scheduler 2.0 (Schedule.Service)."""
90
91     def __init__(self) -> None:
92         """Инициализация COM-объекта планировщика."""
93         self._service: Optional[Any] = None
94         self._is_available = self._connect()
95
96     def _connect(self) -> bool:
97         """Подключение к локальной службе планировщика через COM."""
98         try:
99             import win32com.client
100             self._service = win32com.client.Dispatch('Schedule
101 .Service')
102             self._service.Connect()
103             return True
104         except Exception as ex:
105             logger.debug(f'COM-интерфейс Schedule.Service недо

```

```

    ступен: {ex}')
```

105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133

```

                self._service = None
                return False

    @property
    def is_available(self) -> bool:
        """Доступность нативного COM-интерфейса планировщика."
        """
        return self._is_available and self._service is not None

    def get_all_tasks(self, max_tasks: int = 500) -> List[Dict
    [str, Any]]:
        """Рекурсивный сбор всех задач планировщика из всех па
        пок.

        Args:
            max_tasks: Максимальное количество задач для выбор
            ки.

        Returns:
            List[Dict[str, Any]]: Список задач с путями, дейст
            виями и статусами.
        """
        if not self.is_available or self._service is None:
            return []
        tasks: List[Dict[str, Any]] = []
        try:
            root_folder = self._service.GetFolder('\\\\')
            self._scan_folder(root_folder, tasks, max_tasks)
        except Exception as ex:
            logger.debug(f'Ошибка при рекурсивном чтении задач
            планировщика COM: {ex}')
        return tasks

    def _scan_folder(self, folder: Any, tasks: List[Dict[str,
    Any]], max_tasks: int) -> None:
        """Рекурсивный обход папки планировщика.
```

```

134
135     Args:
136         folder: COM-объект ITaskFolder.
137         tasks: Результирующий список задач.
138         max_tasks: Лимит задач.
139     """
140     if len(tasks) >= max_tasks:
141         return
142     try:
143         folder_tasks = folder.GetTasks(0)
144         for task in folder_tasks:
145             if len(tasks) >= max_tasks:
146                 break
147             try:
148                 task_info = self._parse_task(task)
149                 if task_info:
150                     tasks.append(task_info)
151             except Exception:
152                 continue
153         subfolders = folder.GetFolders(0)
154         for sub in subfolders:
155             if len(tasks) >= max_tasks:
156                 break
157             self._scan_folder(sub, tasks, max_tasks)
158     except Exception as ex:
159         logger.debug(f"Ошибка при сканировании папки '{get
160 attr(folder, 'Path', '')}': {ex}")
161
162     def _parse_task(self, task: Any) -> Optional[Dict[str, Any
163 ]]:
164         """Парсинг свойств зарегистрированной задачи (IRegiste
165 redTask)."""
166         name = getattr(task, 'Name', '') or ''
167         path = getattr(task, 'Path', '') or ''
168         state_code = getattr(task, 'State', 0)
169         state_name = TASK_STATE_MAP.get(state_code, 'Unknown')
170         enabled = getattr(task, 'Enabled', True)
171         next_run_raw = getattr(task, 'NextRunTime', None)

```

```

169 |         last_run_raw = getattr(task, 'LastRunTime', None)
170 |         last_result = getattr(task, 'LastTaskResult', 0) or 0
171 |
172 |         next_run_str = _format_com_date(next_run_raw)
173 |         last_run_str = _format_com_date(last_run_raw)
174 |
175 |         actions_list: List[Dict[str, Any]] = []
176 |         action_strs: List[str] = []
177 |         author = ''
178 |         triggers_list: List[str] = []
179 |
180 |         try:
181 |             definition = task.Definition
182 |             if hasattr(definition, 'RegistrationInfo') and def
initiation.RegistrationInfo:
183 |                 author = getattr(definition.RegistrationInfo,
184 | 'Author', '') or ''
185 |
186 |             if hasattr(definition, 'Triggers') and definition.
187 | Triggers:
188 |                 for trig in definition.Triggers:
189 |                     t_type = getattr(trig, 'Type', -1)
190 |                     t_label = TASK_TRIGGER_TYPE_MAP.get(t_type
191 | , 'Custom')
192 |                     if t_label not in triggers_list:
193 |                         triggers_list.append(t_label)
194 |
195 |             if hasattr(definition, 'Actions') and definition.A
196 | ctions:
197 |                 for act in definition.Actions:
198 |                     act_type = getattr(act, 'Type', -1)
199 |                     if act_type == TASK_ACTION_EXEC:
200 |                         exec_path = getattr(act, 'Path', '') o
201 | r ''
202 |                         args = getattr(act, 'Arguments', '') o
203 | r ''
204 |                         working_dir = getattr(act, 'WorkingDir
205 | ectory', '') or ''

```

```

199         full_cmd = f'{exec_path} {args}'.strip
    |
    |
200         actions_list.append({
201             'type': 'Exec',
202             'path': exec_path,
203             'arguments': args,
204             'working_directory': working_dir,
205             'command': full_cmd
206         })
207         action_strs.append(full_cmd)
208         elif act_type == TASK_ACTION_COM_HANDLER:
209             clsid = getattr(act, 'ClassId', '')
210             actions_list.append({'type': 'ComHandl
    |er', 'class_id': clsid})
211             action_strs.append(f'ComHandler:{clsid
    |}')
212         except Exception:
213             pass
214
215         schedule_type = ', '.join(triggers_list) if triggers_l
    |ist else 'Custom'
216         action_summary = ' ; '.join(action_strs) if action_str
    |s else ''
217
218         return {
219             'TaskName': name,
220             'task_name': name,
221             'name': name,
222             'TaskPath': path,
223             'task_path': path,
224             'State': state_name,
225             'state': state_name,
226             'status': state_name,
227             'Enabled': enabled,
228             'enabled': enabled,
229             'NextRunTime': next_run_str,
230             'next_run_time': next_run_str,
231             'LastRunTime': last_run_str,

```

```
232 |         'last_run_time': last_run_str,  
233 |         'LastTaskResult': last_result,  
234 |         'last_task_result': last_result,  
235 |         'Author': author,  
236 |         'author': author,  
237 |         'ScheduleType': schedule_type,  
238 |         'schedule_type': schedule_type,  
239 |         'Actions': action_summary,  
240 |         'action': action_summary,  
241 |         'ActionDetails': actions_list,  
242 |     }
```


wevtapi.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Wevtapi
5 # Description:
6 # Низкоуровневый интерфейс к wevtapi.dll для работы с журнал
7 # ами событий Windows.
8 # Usage Examples:
9 # Python API:
10 # from apps.windows.telemetry.win32_fffi.wevtapi import Cha
11 # nnelMetadata
12 # service = ChannelMetadata()
13 #
14 # File: wevtapi.py
15 # Project: ai-breadboard
16 # Package: apps.windows.telemetry.win32_fffi
17 # Author: hypo69
18 # Copyright: © 2026 hypo69
19 # Updated: 2026-10-06 06:52:00
20 # =====
21
22 from __future__ import annotations
23 """Низкоуровневый интерфейс к wevtapi.dll для работы с журнала
24 ми событий Windows."""
25
26 import ctypes
27 import ctypes.wintypes as wintypes
28 import struct
29 import sys
30 import winreg
```

```
30 import xml.etree.ElementTree as ET
31 from dataclasses import dataclass
32 from typing import Any, Dict, List, Optional
33 from logger import logger
34 EVT_QUERY_CHANNEL_PATH = 1
35 EVT_QUERY_FILE_PATH = 2
36 EVT_QUERY_REVERSE_DIRECTION = 512
37 EVT_RENDER_EVENT_XML = 1
38 CHANNEL_DESCRIPTIONS: Dict[str, str] = {'System': 'Системный журнал Windows: события ядра ОС, системных служб, драйверов устройств, сбоев питания и сетевого стека.', 'Application': 'Журнал приложений: события, предупреждения, сбои и ошибки пользовательских и серверных программ (Crash, Faults).', 'Security': 'Журнал безопасности: аудит входов в систему (Logon/Logoff), права доступа, повышение привилегий UAC и политики безопасности.', 'Setup': 'Журнал установки: логи процесса установки операционной системы, обновлений Windows и накопительных пакетов KB.', 'Microsoft-Windows-WindowsUpdateClient/Operational': 'Клиент обновления Windows Update: загрузка, верификация и установка патчей, исправления безопасности и драйверов.', 'Microsoft-Windows-Kernel-PnP/Configuration': 'Конфигурация оборудования Plug and Play: подключение новых устройств, установка драйверов, коды ошибок 10/43.', 'Microsoft-Windows-Kernel-Power/Operational': 'Питание ядра Windows: переходы в сон/гибернацию, сбои питания (Event 41), изменение состояний батареи и ACPI.', 'Microsoft-Windows-TaskScheduler/Operational': 'Планировщик заданий: запуск, завершение, сбои и выполнение фоновых регламентных задач Windows.', 'Microsoft-Windows-Windows Defender/Operational': 'Антивирус Microsoft Defender: обнаружение угроз, сканирование в реальном времени, обновление антивирусных сигнатур.', 'Microsoft-Windows-Hyper-V-Compute-Operational': 'Платформа виртуализации Hyper-V: жизненный цикл виртуальных машин, контейнеров WSL2 и виртуальных адаптеров.', 'Microsoft-Windows-Firewall With Advanced Security/Firewall': 'Брандмауэр Windows: блокировки входящего/исходящего трафика, сетевые правила фильтрации и аномалии сокетов.', 'Microsoft-Windows-Diagnostics-Performance/Operational': 'Диагностика производительности: анализ времени загрузки Windows, выключения и зависания системных
```

```
компонентов.', 'Microsoft-Windows-Bits-Client/Operational': 'Фоновая интеллектуальная служба передачи (BITS): загрузка файлов и обновлений по сети с контролем канала.', 'Microsoft-Windows-DNS-Client/Operational': 'DNS-клиент Windows: резолвинг доменных имен, таймауты DNS серверов и сетевые кэширования.', 'Microsoft-Windows-CodeIntegrity/Operational': 'Контроль целостности кода: проверка цифровых подписей драйверов и исполняемых системных файлов.', 'Microsoft-Windows-PowerShell/Operational': 'Исполнение скриптов PowerShell: запуск командлетов, сценариев автоматизации и модулей администрирования.', 'Microsoft-Windows-Sysmon/Operational': 'Microsoft System Monitor (Sysmon): глубокая телеметрия создания процессов (Event 1), сети (Event 3), файловых операций (11/23/26) и хэшей исполняемых файлов.'}
```

39

40 @dataclass

41 class ChannelMetadata:

42 """Метаданные о канале журнала Windows."""

43 channel_name: str

44 display_name: str

45 description: str = ''

46 is_enabled: bool = True

47 record_count: int = 0

48 category: str = 'Windows Event Log'

49

50 class WevtAPI:

51 """Ctypes-обертка для wevtapi.dll и реестра Windows Event Log."""

52

53 def __init__(self) -> None:

54 """Инициализация библиотеки wevtapi.dll."""

55 self._available = False

56 self.wevtapi: Optional[ctypes.WinDLL] = None

57 self._publisher_cache: Dict[str, Optional[wintypes.HANDLE]] = {}

58 if sys.platform == 'win32':

59 try:

60 self.wevtapi = ctypes.windll.LoadLibrary('wevtapi.dll')

```

61         self._setup_signatures()
62         self._available = True
63     except Exception as ex:
64         logger.debug(f'[WevtAPI] Не удалось загрузить
wevtapi.dll: {ex}')
65
66     def _setup_signatures(self) -> None:
67         """Настройка типов аргументов и возвращаемых значений
функций."""
68         if not self.wevtapi:
69             return
70         self.wevtapi.EvtOpenChannelEnum.argtypes = [wintypes.H
HANDLE, wintypes.DWORD]
71         self.wevtapi.EvtOpenChannelEnum.restype = wintypes.HAN
DLE
72         self.wevtapi.EvtNextChannelPath.argtypes = [wintypes.H
HANDLE, wintypes.DWORD, wintypes.LPWSTR, ctypes.POINTER(wintype
s.DWORD)]
73         self.wevtapi.EvtNextChannelPath.restype = wintypes.BOOL
L
74         self.wevtapi.EvtClose.argtypes = [wintypes.HANDLE]
75         self.wevtapi.EvtClose.restype = wintypes.BOOL
76         self.wevtapi.EvtOpenLog.argtypes = [wintypes.HANDLE, w
intypes.LPCWSTR, wintypes.DWORD]
77         self.wevtapi.EvtOpenLog.restype = wintypes.HANDLE
78         self.wevtapi.EvtGetLogInfo.argtypes = [wintypes.HANDLE
, wintypes.DWORD, wintypes.DWORD, ctypes.c_void_p, ctypes.POIN
TER(wintypes.DWORD)]
79         self.wevtapi.EvtGetLogInfo.restype = wintypes.BOOL
80         self.wevtapi.EvtQuery.argtypes = [wintypes.HANDLE, win
types.LPCWSTR, wintypes.LPCWSTR, wintypes.DWORD]
81         self.wevtapi.EvtQuery.restype = wintypes.HANDLE
82         self.wevtapi.EvtNext.argtypes = [wintypes.HANDLE, wint
ypes.DWORD, ctypes.POINTER(wintypes.HANDLE), wintypes.DWORD, w
intypes.DWORD, ctypes.POINTER(wintypes.DWORD)]
83         self.wevtapi.EvtNext.restype = wintypes.BOOL
84         self.wevtapi.EvtRender.argtypes = [wintypes.HANDLE, wi
ntypes.HANDLE, wintypes.DWORD, wintypes.DWORD, wintypes.LPVOID

```

```

    , ctypes.POINTER(wintypes.DWORD), ctypes.POINTER(wintypes.DWORD)]
85         self.wevtapi.EvtRender.restype = wintypes.BOOL
86         self.wevtapi.EvtOpenPublisherMetadata.argtypes = [wintypes.HANDLE, wintypes.LPCWSTR, wintypes.LPCWSTR, wintypes.DWORD, wintypes.DWORD]
87         self.wevtapi.EvtOpenPublisherMetadata.restype = wintypes.HANDLE
88         self.wevtapi.EvtFormatMessage.argtypes = [wintypes.HANDLE, wintypes.HANDLE, wintypes.DWORD, wintypes.DWORD, ctypes.c_void_p, wintypes.DWORD, wintypes.DWORD, wintypes.LPWSTR, ctypes.POINTER(wintypes.DWORD)]
89         self.wevtapi.EvtFormatMessage.restype = wintypes.BOOL
90
91     def is_available(self) -> bool:
92         """Проверить доступность wevtapi.dll в текущей системе
93         ."""
94
95         return self._available
96
97     def get_channel_record_count(self, channel_name: str) -> int:
98         """Мгновенно получить точное количество записей в канале через EvtGetLogInfo."""
99         if not self._available or not self.wevtapi:
100             return 0
101         h_log = self.wevtapi.EvtOpenLog(None, channel_name, 1)
102         if not h_log:
103             return 0
104         try:
105             buf = (ctypes.c_byte * 128)()
106             used = wintypes.DWORD(0)
107             if self.wevtapi.EvtGetLogInfo(h_log, 5, 128, ctypes.byref(buf), ctypes.byref(used)):
108                 return struct.unpack('<Q', bytes(buf)[:8])[0]
109             except Exception:
110                 pass
111         finally:
112             self.wevtapi.EvtClose(h_log)

```

```
111         return 0
112
113     def get_channel_description(self, channel_name: str) -> str:
114         """Получить человекочитаемое описание назначения канала."""
115         if channel_name in CHANNEL_DESCRIPTIONS:
116             return CHANNEL_DESCRIPTIONS[channel_name]
117         if 'Defender' in channel_name:
118             return 'Журнал защитных механизмов, антивирусного сканирования и предотвращения вторжений Microsoft Defender.'
119         if 'Kernel-Power' in channel_name:
120             return 'Журнал управления электропитанием, состояний ACPI, спящего режима и аварийных отключений питания.'
121         if 'Kernel-PnP' in channel_name:
122             return 'Журнал конфигурации оборудования, обнаружения и инициализации устройств Plug and Play.'
123         if 'WindowsUpdate' in channel_name:
124             return 'Журнал службы обновлений Windows, загрузки пакетов KB и применения хотфиксов.'
125         if 'TaskScheduler' in channel_name:
126             return 'Журнал планировщика задач Windows, запуска автоматических фоновых процессов и триггеров.'
127         if 'Hyper-V' in channel_name:
128             return 'Журнал подсистемы виртуализации Hyper-V, WSL2 контейнеров и гостевых служб.'
129         if 'PowerShell' in channel_name:
130             return 'Журнал среды PowerShell, выполнения скриптов автоматизации и команд администрирования.'
131         if 'Firewall' in channel_name:
132             return 'Журнал сетевого брандмауэра Windows, правил сетевого экрана и событий фильтрации трафика.'
133         if 'Storage' in channel_name or 'Disk' in channel_name:
134             return 'Журнал дисковой подсистемы, контроллеров накопителей, томов NTFS/ReFS и файловой системы.'
135         if 'Network' in channel_name or 'Wlan' in channel_name or 'NDIS' in channel_name:
```

```

136         return 'Журнал сетевых интерфейсов, Wi-Fi соединен
ий, сетевых адаптеров и стека TCP/IP.'
137         if 'Audio' in channel_name or 'Sound' in channel_name:
138             return 'Журнал аудиоподсистемы, службы Windows Aud
ио и конечных звуковых устройств.'
139         if 'Print' in channel_name or 'Spooler' in channel_nam
e:
140             return 'Журнал диспетчера очереди печати и драйвер
ов принтеров.'
141         if 'Bluetooth' in channel_name or 'BTN' in channel_nam
e:
142             return 'Журнал стека Bluetooth, сопряжения беспров
одных устройств и протоколов передачи данных.'
143         if 'USB' in channel_name:
144             return 'Журнал контроллеров USB, хабов и подключен
ных внешних периферийных устройств.'
145         return f"Специализированный системный журнал Windows д
ля компонента '{channel_name}'."
146
147     def enumerate_channels(self) -> List[ChannelMetadata]:
148         """Получить исчерпывающий список всех каналов Windows
Event Log с количеством записей."""
149         channels: List[ChannelMetadata] = []
150         seen: set[str] = set()
151         classic = [('System', 'System (Системный)'), ('Applica
tion', 'Application (Приложения)'), ('Security', 'Security (Бе
зопасность)'), ('Setup', 'Setup (Установка)'), ('Microsoft-Win
dows-WindowsUpdateClient/Operational', 'Windows Update Client'
), ('Microsoft-Windows-Kernel-PnP/Configuration', 'Kernel PnP
Configuration'), ('Microsoft-Windows-Kernel-Power/Operational'
, 'Kernel Power'), ('Microsoft-Windows-TaskScheduler/Operation
al', 'Task Scheduler'), ('Microsoft-Windows-Windows Defender/O
perational', 'Windows Defender'), ('Microsoft-Windows-Hyper-V-
Compute-Operational', 'Hyper-V Compute'), ('Microsoft-Windows-
PowerShell/Operational', 'Windows PowerShell')]
152         for cname, dname in classic:
153             rec_cnt = self.get_channel_record_count(cname)
154             channels.append(ChannelMetadata(channel_name=cname

```

```

    , display_name=dname, description=self.get_channel_description
    (cname), record_count=rec_cnt, is_enabled=True))
155         seen.add(cname.lower())
156         if self._available and self.wevtapi:
157             try:
158                 h_enum = self.wevtapi.EvtOpenChannelEnum(None,
0)
159                 if h_enum:
160                     buffer_size = 512
161                     buf = ctypes.create_unicode_buffer(buffer_
size)
162                     used = wintypes.DWORD()
163                     while True:
164                         success = self.wevtapi.EvtNextChannelP
ath(h_enum, buffer_size, buf, ctypes.byref(used))
165                         if not success:
166                             err = ctypes.GetLastError()
167                             if err == 122:
168                                 buffer_size = used.value + 10
169                                 buf = ctypes.create_unicode_bu
ffer(buffer_size)
170                                 continue
171                                 break
172                                 chan_str = buf.value.strip()
173                                 if chan_str and chan_str.lower() not i
n seen:
174                                     rec_cnt = self.get_channel_record_
count(chan_str)
175                                     channels.append(ChannelMetadata(ch
annel_name=chan_str, display_name=chan_str, description=self.g
et_channel_description(chan_str), record_count=rec_cnt, is_ena
bled=True))
176                                     seen.add(chan_str.lower())
177                                     self.wevtapi.EvtClose(h_enum)
178             except Exception as ex:
179                 logger.debug(f'[WevtAPI] Ошибка при перечислен
ии через EvtOpenChannelEnum: {ex}')
180         try:

```



```

181         reg_channels = self._enumerate_registry_channels()
182         for r_chan in reg_channels:
183             if r_chan.lower() not in seen:
184                 rec_cnt = self.get_channel_record_count(r_
chan)
185                 channels.append(ChannelMetadata(channel_na
me=r_chan, display_name=r_chan, description=self.get_channel_d
escription(r_chan), record_count=rec_cnt, is_enabled=True))
186                 seen.add(r_chan.lower())
187         except Exception as ex:
188             logger.debug(f'[WevtAPI] Ошибка чтения каналов из
реестра: {ex}')
189         return channels
190
191     def _enumerate_registry_channels(self) -> List[str]:
192         """Считать имена зарегистрированных каналов из реестра
Windows."""
193         res: List[str] = []
194         key_path = 'SOFTWARE\\Microsoft\\Windows\\CurrentVersi
on\\WINEVT\\Channels'
195         try:
196             with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE, key
_path, 0, winreg.KEY_READ | winreg.KEY_WOW64_64KEY) as key:
197                 num_subkeys, _, _ = winreg.QueryInfoKey(key)
198                 for i in range(num_subkeys):
199                     try:
200                         subkey_name = winreg.EnumKey(key, i)
201                         if subkey_name:
202                             res.append(subkey_name)
203                     except OSError:
204                         break
205         except Exception:
206             pass
207         return res
208
209     def read_events(self, channel: str='System', limit: int=10
0, level: str='', search: str='', event_id: int | list[int] |
tuple[int, ...] | set[int] = 0, hours: int=24) -> List[Dict[st

```

```

r, Any]]:
210     """Быстрое чтение событий из указанного канала через E
vtQuery / EvtRender."""
211     if not self._available or not self.wevtapi:
212         return []
213     limit = max(1, min(limit, 2000))
214     xpath_parts: List[str] = []
215     if level:
216         lvl_lower = level.lower()
217         if 'crit' in lvl_lower:
218             xpath_parts.append('Level=1')
219         elif 'err' in lvl_lower:
220             xpath_parts.append('Level=2')
221         elif 'warn' in lvl_lower:
222             xpath_parts.append('Level=3')
223         elif 'inf' in lvl_lower:
224             xpath_parts.append('Level=4')
225         elif 'verb' in lvl_lower:
226             xpath_parts.append('Level=5')
227     if isinstance(event_id, (list, tuple, set)):
228         valid_eids = [e for e in event_id if isinstance(e,
int) and e > 0]
229         if valid_eids:
230             clauses = ' or '.join(f'EventID={eid}' for eid
in valid_eids)
231             xpath_parts.append(f'({clauses})')
232     elif isinstance(event_id, int) and event_id > 0:
233         xpath_parts.append(f'EventID={event_id}')
234     if xpath_parts:
235         clause = ' and '.join(xpath_parts)
236         query_xpath = f'*[System[{clause}]]'
237     else:
238         query_xpath = '*'
239     events: List[Dict[str, Any]] = []
240     try:
241         h_query = self.wevtapi.EvtQuery(None, channel, que
ry_xpath, EVT_QUERY_CHANNEL_PATH | EVT_QUERY_REVERSE_DIRECTION
)

```

```

242         if not h_query:
243             return []
244         try:
245             event_handles = (wintypes.HANDLE * limit)()
246             returned = wintypes.DWORD(0)
247             if self.wevtapi.EvtNext(h_query, limit, event_
handles, 2000, 0, ctypes.byref(returned)):
248                 for i in range(returned.value):
249                     h_ev = event_handles[i]
250                     if h_ev:
251                         ev_dict = self._render_event_xml(h
_ev, channel)
252                     if ev_dict:
253                         if search:
254                             s_lower = search.lower()
255                             msg = str(ev_dict.get('mes
sage', ''')).lower()
256                             prov = str(ev_dict.get('pr
ovider', ''')).lower()
257                             eid = str(ev_dict.get('eve
nt_id', '''))
258                             if s_lower not in msg and
s_lower not in prov and (s_lower not in eid):
259                                 self.wevtapi.EvtClose(
h_ev)
260                                 continue
261                                 events.append(ev_dict)
262                                 self.wevtapi.EvtClose(h_ev)
263         finally:
264             self.wevtapi.EvtClose(h_query)
265     except Exception as ex:
266         logger.debug(f'[WevtAPI] Ошибка при чтении событий
({channel}): {ex}')
267     return events
268
269     def _get_publisher_metadata(self, provider_name: str) -> O
ptional[wintypes.HANDLE]:
270         """Получить кешированный дескриптор метаданных провайд

```

```

    ера событий."""
271         if not self._available or not self.wevtapi or not prov
ider_name:
272             return None
273         if provider_name in self._publisher_cache:
274             return self._publisher_cache[provider_name]
275         try:
276             h_pub = self.wevtapi.EvtOpenPublisherMetadata(None
, provider_name, None, 0, 0)
277             if h_pub:
278                 self._publisher_cache[provider_name] = h_pub
279                 return h_pub
280                 self._publisher_cache[provider_name] = None
281         except Exception:
282             self._publisher_cache[provider_name] = None
283         return None
284
285     def format_event_message(self, h_event: wintypes.HANDLE, p
rovider_name: str, flag: int = 1) -> Optional[str]:
286         """Сформировать нативное локализованное системное сооб
щение события через EvtFormatMessage.
287
288         Args:
289             h_event: Дескриптор события Windows.
290             provider_name: Имя издателя/провайдера (например,
'Microsoft-Windows-Kernel-Power').
291             flag: Флаг форматирования (1 = EvtFormatMessageEve
nt, 2 = Level, 3 = Task).
292
293         Returns:
294             Optional[str]: Отформатированная локализованная ст
рока сообщения или None.
295         """
296         if not self._available or not self.wevtapi or not h_ev
ent or not provider_name:
297             return None
298         h_pub = self._get_publisher_metadata(provider_name)
299         if not h_pub:

```

```

300         return None
301     try:
302         used = wintypes.DWORD()
303         self.wevtapi.EvtFormatMessage(h_pub, h_event, 0, 0
, None, flag, 0, None, ctypes.byref(used))
304         err = ctypes.GetLastError()
305         if err != 122 or used.value == 0:
306             return None
307         buf = ctypes.create_unicode_buffer(used.value + 2)
308         if self.wevtapi.EvtFormatMessage(h_pub, h_event, 0
, 0, None, flag, used.value, buf, ctypes.byref(used)):
309             val = buf.value
310             if val:
311                 return val.strip()
312     except Exception:
313         pass
314     return None
315
316     def close(self) -> None:
317         """Освободить все открытые дескрипторы издателей."""
318         if self._available and self.wevtapi:
319             for h in list(self._publisher_cache.values()):
320                 if h:
321                     try:
322                         self.wevtapi.EvtClose(h)
323                     except Exception:
324                         pass
325             self._publisher_cache.clear()
326
327     def __del__(self) -> None:
328         """Деструктор для гарантированного освобождения ресурс
329 ов."""
330         self.close()
331
332     def _render_event_xml(self, h_event: wintypes.HANDLE, chan
nel: str) -> Optional[Dict[str, Any]]:
333         """Отрендерить событие в XML строку и разобрать в слов
арь."""

```

```

333         if not self.wevtapi:
334             return None
335         used = wintypes.DWORD()
336         prop_count = wintypes.DWORD()
337         self.wevtapi.EvtRender(None, h_event, EVT_RENDER_EVENT
_XML, 0, None, ctypes.byref(used), ctypes.byref(prop_count))
338         err = ctypes.GetLastError()
339         if err != 122:
340             return None
341         buf = ctypes.create_unicode_buffer(used.value // 2 + 2
)
342         if not self.wevtapi.EvtRender(None, h_event, EVT_RENDE
R_EVENT_XML, used.value, ctypes.byref(buf), ctypes.byref(used)
, ctypes.byref(prop_count)):
343             return None
344         xml_str = buf.value
345         if not xml_str:
346             return None
347         res = self._parse_event_xml(xml_str, channel)
348         if res:
349             prov = res.get('provider', '')
350             if prov:
351                 formatted = self.format_event_message(h_event,
prov, flag=1)
352                 if formatted:
353                     res['formatted_message'] = formatted
354                     res['message'] = formatted
355             return res
356
357     def _parse_event_xml(self, xml_str: str, channel: str) ->
Dict[str, Any]:
358         """Разобрать XML представление Windows события в норма
лизованную структуру."""
359         res: Dict[str, Any] = {'timestamp': '', 'level': 'Info
rmation', 'event_id': 0, 'provider': '', 'computer': '', 'proc
ess_id': 0, 'thread_id': 0, 'channel': channel, 'message': '',
'raw_data': xml_str}
360         try:

```

```

361         clean_xml = ET.fromstring(xml_str)
362         system_elem = clean_xml.find('{http://schemas.micr
osoft.com/win/2004/08/events/event}System')
363         if system_elem is None:
364             system_elem = clean_xml.find('System')
365         if system_elem is not None:
366             for child in system_elem:
367                 tag = child.tag.split('}')[1]
368                 if tag == 'TimeCreated':
369                     raw_time = child.attrib.get('SystemTim
e', '')
370                     if raw_time:
371                         res['timestamp'] = raw_time.replac
e('T', ' ').split('.')[0]
372                     elif tag == 'EventID':
373                         try:
374                             res['event_id'] = int(child.text o
r '0')
375                         except ValueError:
376                             res['event_id'] = 0
377                     elif tag == 'EventRecordID':
378                         try:
379                             res['record_id'] = int(child.text
or '0')
380                         except ValueError:
381                             res['record_id'] = 0
382                     elif tag == 'Level':
383                         lvl_map = {'1': 'Critical', '2': 'Erro
r', '3': 'Warning', '4': 'Information', '5': 'Verbose'}
384                         res['level'] = lvl_map.get(str(child.t
ext or '').strip(), 'Information')
385                     elif tag == 'Provider':
386                         res['provider'] = child.attrib.get('Na
me', '')
387                     elif tag == 'Computer':
388                         res['computer'] = child.text or ''
389                     elif tag == 'Execution':
390                         try:

```

```

391         res['process_id'] = int(child.attr
ib.get('ProcessID', 0))
392         res['thread_id'] = int(child.attri
b.get('ThreadID', 0))
393         except ValueError:
394             pass
395         elif tag == 'Channel':
396             if child.text:
397                 res['channel'] = child.text
398         data_dict: Dict[str, str] = {}
399         data_texts: List[str] = []
400         for d in clean_xml.iter():
401             tag = d.tag.split(' ')[-1]
402             if tag in ('Data', 'string', 'Value') and d.te
xt:
403                 t = d.text.strip()
404                 name_attr = d.attrib.get('Name')
405                 if name_attr:
406                     data_dict[name_attr] = t
407                     data_texts.append(f'{name_attr}: {t}')
408                 elif t:
409                     data_texts.append(t)
410         res['event_data'] = data_dict
411         if data_texts:
412             res['message'] = ' | '.join(data_texts[:12])
413         else:
414             res['message'] = f"Event {res['event_id']} fro
m {res['provider']}"
415         except Exception:
416             res['message'] = f'Event recorded in {channel}'
417         return res
418
419     def query_process_events(self, limit: int=100, filter_proc
ess: Optional[str]=None, filter_user: Optional[str]=None) -> L
ist[Dict[str, Any]]:
420         """
421         Запросить структурированные события создания процессов
из Sysmon (Event 1)

```



```

422 |         или Windows Security Log (Event 4688).
423 |
424 |     Args:
425 |         limit: Максимальное количество событий.
426 |         filter_process: Фильтр по имени/пути исполняемого
427 |         файла.
428 |         filter_user: Фильтр по имени учетной записи пользо-
429 |         вателя.
430 |
431 |     Returns:
432 |         Список нормализованных словарей с информацией о за-
433 |         пусках процессов.
434 |         """
435 |         results: List[Dict[str, Any]] = []
436 |         sysmon_events = self.read_events(channel='Microsoft-Wi-
437 |         ndows-Sysmon/Operational', limit=limit, event_id=1)
438 |         for ev in sysmon_events:
439 |             ed = ev.get('event_data', {})
440 |             exe_path = ed.get('Image', '')
441 |             cmd_line = ed.get('CommandLine', '')
442 |             user = ed.get('User', '')
443 |             pid = int(ed.get('ProcessId', 0) or 0)
444 |             ppid = int(ed.get('ParentProcessId', 0) or 0)
445 |             parent_exe = ed.get('ParentImage', '')
446 |             parent_cmd = ed.get('ParentCommandLine', '')
447 |             hashes = ed.get('Hashes', '')
448 |             guid = ed.get('ProcessGuid', '')
449 |             parent_guid = ed.get('ParentProcessGuid', '')
450 |             if filter_process and filter_process.lower() not i
451 |             n exe_path.lower() and (filter_process.lower() not in cmd_line
452 |             .lower()):
453 |                 continue
454 |             if filter_user and filter_user.lower() not in user
455 |             .lower():
456 |                 continue
457 |             results.append({'source': 'Sysmon (Event 1)', 'tim
458 |             estamp': ev.get('timestamp'), 'event_id': 1, 'process_id': pid
459 |             , 'process_name': exe_path.replace('\\', '/').split('/')[-1] i

```

```

    f exe_path else '', 'executable_path': exe_path, 'command_line': cmd_line, 'user': user, 'parent_process_id': ppid, 'parent_process_name': parent_exe.replace('\\', '/').split('/')[-1] if parent_exe else '', 'parent_executable_path': parent_exe, 'parent_command_line': parent_cmd, 'process_guid': guid, 'parent_process_guid': parent_guid, 'hashes': hashes, 'raw_event': ev}
    )
451         if len(results) < limit:
452             remaining = limit - len(results)
453             sec_events = self.read_events(channel='Security',
limit=remaining, event_id=4688)
454             for ev in sec_events:
455                 ed = ev.get('event_data', {})
456                 exe_path = ed.get('NewProcessName', '')
457                 cmd_line = ed.get('CommandLine', '')
458                 user = ed.get('SubjectUserName', '')
459                 raw_pid = ed.get('NewProcessId', '0')
460                 try:
461                     pid = int(raw_pid, 16) if str(raw_pid).startswith('0x') else int(raw_pid or 0)
462                     except ValueError:
463                         pid = 0
464                     raw_ppid = ed.get('ProcessId', '0')
465                     try:
466                         ppid = int(raw_ppid, 16) if str(raw_ppid).startswith('0x') else int(raw_ppid or 0)
467                     except ValueError:
468                         ppid = 0
469                     parent_exe = ed.get('ParentProcessName', '')
470                     if filter_process and filter_process.lower() not in exe_path.lower() and (filter_process.lower() not in cmd_line.lower()):
471                         continue
472                     if filter_user and filter_user.lower() not in user.lower():
473                         continue
474                     results.append({'source': 'Security Audit (Event 4688)', 'timestamp': ev.get('timestamp'), 'event_id': 4688,

```

```

    'process_id': pid, 'process_name': exe_path.replace('\\', '/')
).split('/')[ -1] if exe_path else '', 'executable_path': exe_p
ath, 'command_line': cmd_line, 'user': user, 'parent_process_i
d': ppid, 'parent_process_name': parent_exe.replace('\\', '/')
.split('/')[ -1] if parent_exe else '', 'parent_executable_path
': parent_exe, 'parent_command_line': '', 'process_guid': '',
'parent_process_guid': '', 'hashes': '', 'raw_event': ev})
475         return results
476
477     def query_powershell_script_blocks(self, limit: int = 100,
    filter_text: Optional[str] = None) -> List[Dict[str, Any]]:
478         """Запросить блоки скриптов PowerShell (Script Block L
    ogging, Event ID 4104).
479
480         Args:
481             limit: Максимальное количество записей.
482             filter_text: Подстрока для поиска в теле скрипта и
    ли пути.
483
484         Returns:
485             List[Dict[str, Any]]: Нормализованные события с бл
    оками кода скриптов.
486         """
487         results: List[Dict[str, Any]] = []
488         ps_events = self.read_events(channel='Microsoft-Window
    s-PowerShell/Operational', limit=limit, event_id=4104)
489         for ev in ps_events:
490             ed = ev.get('event_data', {})
491             script_text = ed.get('ScriptBlockText', '') or ev.
    get('message', '')
492             script_id = ed.get('ScriptBlockId', '')
493             path = ed.get('Path', '')
494             user_id = ed.get('UserId', '')
495             msg_total = ed.get('MessageTotal', '1')
496             msg_number = ed.get('MessageNumber', '1')
497
498             if filter_text and filter_text.lower() not in scri
    pt_text.lower() and (filter_text.lower() not in path.lower()):

```

```

499         continue
500
501         results.append({
502             'source': 'PowerShell ScriptBlock (Event 4104)
503             ',
504             'timestamp': ev.get('timestamp'),
505             'event_id': 4104,
506             'script_block_id': script_id,
507             'script_text': script_text,
508             'path': path,
509             'user_id': user_id,
510             'message_number': int(msg_number) if str(msg_n
umber).isdigit() else 1,
511             'message_total': int(msg_total) if str(msg_tot
al).isdigit() else 1,
512             'raw_event': ev,
513         })
514     return results
515
516 def query_sysmon_events(
517     self,
518     limit: int = 100,
519     event_ids: Optional[List[int]] = None,
520     filter_process: Optional[str] = None,
521 ) -> List[Dict[str, Any]]:
522     """Запросить события безопасности Sysmon (Event ID 1,
523     7, 10, 11, 23, 26).
524
525     Args:
526         limit: Максимальное количество записей.
527         event_ids: Список опрашиваемых идентификаторов соб
528         ытий (по умолчанию: 1, 7, 10, 11, 23, 26).
529         filter_process: Фильтр по имени/пути исполняемого
530         файла или модуля.
531
532     Returns:
533         List[Dict[str, Any]]: Нормализованные события Sysm
534         on.

```

```

530 |         """
531 |         target_ids = event_ids or [1, 7, 10, 11, 23, 26]
532 |         all_events: List[Dict[str, Any]] = []
533 |
534 |         for eid in target_ids:
535 |             evs = self.read_events(channel='Microsoft-Windows-
Sysmon/Operational', limit=limit, event_id=eid)
536 |             for ev in evs:
537 |                 ed = ev.get('event_data', {})
538 |                 img = ed.get('Image', '') or ed.get('SourceIma
ge', '')
539 |                 if filter_process and filter_process.lower() n
ot in img.lower() and (filter_process.lower() not in str(ed).l
ower()):
540 |                     continue
541 |
542 |                 item: Dict[str, Any] = {
543 |                     'source': f'Sysmon (Event {eid})',
544 |                     'timestamp': ev.get('timestamp'),
545 |                     'event_id': eid,
546 |                     'process_id': int(ed.get('ProcessId', 0) o
r ed.get('SourceProcessId', 0) or 0),
547 |                     'image': img,
548 |                     'process_guid': ed.get('ProcessGuid') or e
d.get('SourceProcessGuid', ''),
549 |                     'user': ed.get('User', ''),
550 |                     'raw_event': ev,
551 |                 }
552 |
553 |                 if eid == 1:
554 |                     item.update({
555 |                         'event_type': 'ProcessCreate',
556 |                         'command_line': ed.get('CommandLine',
''),
557 |                         'parent_process_id': int(ed.get('Paren
tProcessId', 0) or 0),
558 |                         'parent_image': ed.get('ParentImage',
''),

```

```

559         'hashes': ed.get('Hashes', ''),
560     })
561     elif eid == 7:
562         item.update({
563             'event_type': 'ImageLoaded',
564             'image_loaded': ed.get('ImageLoaded',
565         ''),
566             'signed': ed.get('Signed', ''),
567             'signature': ed.get('Signature', ''),
568             'hashes': ed.get('Hashes', ''),
569         })
570     elif eid == 10:
571         item.update({
572             'event_type': 'ProcessAccess',
573             'target_process_id': int(ed.get('TargetProcessId', 0) or 0),
574             'target_image': ed.get('TargetImage',
575         ''),
576             'granted_access': ed.get('GrantedAccess', ''),
577             'call_trace': ed.get('CallTrace', ''),
578         })
579     elif eid in (11, 23, 26):
580         item.update({
581             'event_type': 'FileDelete' if eid in (
582         23, 26) else 'FileCreate',
583             'target_filename': ed.get('TargetFilename', ''),
584             'hashes': ed.get('Hashes', ''),
585         })
586     all_events.append(item)
587
588     all_events.sort(key=lambda x: str(x.get('timestamp', '')), reverse=True)
589     return all_events[:limit]

```

```
590 |__all__ = ['WevtAPI', 'ChannelMetadata', 'CHANNEL_DESCRIPTIONS  
    |']
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry Win32_Ffi
4 # - Init
5 # Description:
6 #   Модуль низкоуровневых биндингов Win32 FFI / ctypes.
7 #
8 # Usage Examples:
9 #
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: apps.windows.telemetry.win32_fffi
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-01 13:28:28
17 # =====
18
19 """Модуль низкоуровневых биндингов Win32 FFI / ctypes."""
20
21 from .kernel32 import Kernel32API
22 from .psapi import PsapiAPI
23 from .advapi32 import Advapi32API
24 from .ntdll import NtdllAPI
25 from .etw import EtwAPI
26 from .wevtapi import WevtAPI, ChannelMetadata, CHANNEL_DESCRIPTIONS
27 from .scm import ServiceControlManager, ServiceInfo
28 from .tasksched import TaskSchedulerAPI
29 from .nethelper import IPHelperAPI, NetworkSocketInfo
30 from .setupapi import SetupAPI, PnPDeviceInfo
31 from .error_decoder import WindowsErrorDecoder, format_system_
```



```
error, decode_bugcheck_code
32
33 __all__ = [
34     'Kernel32API',
35     'PsapiAPI',
36     'Advapi32API',
37     'NtdllAPI',
38     'EtwAPI',
39     'WevtAPI',
40     'ChannelMetadata',
41     'CHANNEL_DESCRIPTIONS',
42     'ServiceControlManager',
43     'ServiceInfo',
44     'TaskSchedulerAPI',
45     'IPHelperAPI',
46     'NetworkSocketInfo',
47     'SetupAPI',
48     'PnPDeviceInfo',
49     'WindowsErrorDecoder',
50     'format_system_error',
51     'decode_bugcheck_code',
52 ]
53
```

win_service.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Windows
  Service Entry
4 # =====
5 # Description:
6 #   Точка входа и обертка Windows Service для фоновой службы A
  ITelemetry.
7 #   Поддерживает исполнение под управлением Service Control Ma
  nager (SCM),
8 #   а также интерактивный запуск через консоль и CLI-команды w
  in32serviceutil.
9 #
10 # Usage Examples:
11 #   CLI:
12 #       AITelemetry.exe install
13 #       AITelemetry.exe start
14 #       AITelemetry.exe stop
15 #       AITelemetry.exe restart
16 #       AITelemetry.exe remove
17 #       AITelemetry.exe --mode hybrid
18 #
19 # File: win_service.py
20 # Project: ai-breadboard
21 # Package: apps.windows.telemetry
22 # Author: hypo69
23 # Copyright: © 2026 hypo69
24 # Updated: 2026-10-06 07:58:00
25 # =====
26
27 from __future__ import annotations
28 """Windows Service реализация для фонового демона сбора телеме
  трии AI-Breadboard."""
```

```
29 |
30 | import logging
31 | import os
32 | import sys
33 | import threading
34 | import time
35 | from pathlib import Path
36 | from typing import Any, Dict, List, Optional
37 |
38 | # Обеспечиваем корректные пути импорта для автономного и скомпилированного режима
39 | _current_dir = Path(__file__).resolve().parent
40 | _project_root = str(_current_dir.parents[2]) if len(_current_dir.parents) >= 3 else str(_current_dir)
41 | if _project_root not in sys.path:
42 |     sys.path.insert(0, _project_root)
43 |
44 | # Настройка системного логгера для службы
45 | def _setup_service_logging() -> None:
46 |     """Настраивает логирование службы в файл ProgramData и консоль."""
47 |     programdata = os.environ.get('ProgramData') or os.environ.get('ALLUSERSPROFILE')
48 |     if programdata and os.path.exists(programdata):
49 |         base_dir = Path(programdata)
50 |     else:
51 |         appdata = os.environ.get('APPDATA') or os.environ.get('LOCALAPPDATA')
52 |         base_dir = Path(appdata) if appdata and os.path.exists(appdata) else Path.home()
53 |
54 |     log_dir = base_dir / 'AITelemetry' / 'logs'
55 |     try:
56 |         log_dir.mkdir(parents=True, exist_ok=True)
57 |         log_file = log_dir / 'telemetry.log'
58 |         root_logger = logging.getLogger()
59 |         root_logger.setLevel(logging.INFO)
60 |
```

```

61         # File handler
62         file_handler = logging.FileHandler(str(log_file), encoding='utf-8')
63         file_handler.setFormatter(logging.Formatter(
64             '%(asctime)s [%(levelname)s] [%(name)s] %(message)s',
65             datefmt='%Y-%m-%d %H:%M:%S'
66         ))
67         root_logger.addHandler(file_handler)
68
69         # Stream handler for interactive execution
70         if not getattr(sys, 'frozen', False) or sys.stdout is not None:
71             console_handler = logging.StreamHandler()
72             console_handler.setFormatter(logging.Formatter(
73                 '%(asctime)s [%(levelname)s] %(message)s',
74                 datefmt='%H:%M:%S'
75             ))
76             root_logger.addHandler(console_handler)
77     except Exception:
78         pass
79
80
81 try:
82     from logger import logger
83 except ImportError:
84     _setup_service_logging()
85     logger = logging.getLogger('AITelemetryService')
86
87 try:
88     import win32serviceutil
89     import win32service
90     import win32event
91     import servicemanager
92     PYWIN32_AVAILABLE = True
93 except ImportError:
94     PYWIN32_AVAILABLE = False
95

```

```

96 |
97 | if PYWIN32_AVAILABLE:
98 |     class AITelemetryWindowsService(win32serviceutil.ServiceFramework):
99 |         """Windows Service адаптер для службы сбора системной
100 |         телеметрии AI-Breadboard."""
101 |         _svc_name_ = 'AITelemetry'
102 |         _svc_display_name_ = 'AI-Breadboard Telemetry Service'
103 |         _svc_description_ = 'Фоновая служба мониторинга, сбора
104 |         телеметрии и аналитики метрик AI-Breadboard'
105 |
106 |     def __init__(self, args: List[str]) -> None:
107 |         """Инициализирует экземпляр службы Windows Service
108 |         ."""
109 |         win32serviceutil.ServiceFramework.__init__(self, args)
110 |         self.hWaitStop = win32event.CreateEvent(None, 0, 0
111 |         , None)
112 |         self.is_alive = True
113 |         self._service_thread: Optional[threading.Thread] =
114 |         None
115 |
116 |     def SvcStop(self) -> None:
117 |         """Обрабатывает сигнал остановки службы от Service
118 |         Control Manager (SCM)."""
119 |         self.ReportServiceStatus(win32service.SERVICE_STOP
120 |         _PENDING)
121 |         logger.info('SCM запросил остановку службы AITelem
122 |         etry...')
123 |         self.is_alive = False
124 |         win32event.SetEvent(self.hWaitStop)
125 |
126 |         # Сигнализируем главному циклу
127 |         try:
128 |             from apps.windows.telemetry.main import _stop_
129 |             event
130 |             _stop_event.set()
131 |         except Exception:

```

```

123         pass
124
125     def SvcShutdown(self) -> None:
126         """Обрабатывает сигнал выключения ОС Windows."""
127         logger.info('Система выключается. Остановка службы
AITelemetry...')
128         self.SvcStop()
129
130     def SvcDoRun(self) -> None:
131         """Основной метод исполнения службы Windows Servic
e."""
132         _setup_service_logging()
133         logger.info('Служба AITelemetry запускается под уп
равлением Windows SCM...')
134         try:
135             servicemanager.LogMsg(
136                 servicemanager.EVENTLOG_INFORMATION_TYPE,
137                 servicemanager.PYS_SERVICE_STARTED,
138                 (self._svc_name_, ''))
139         )
140         except Exception:
141             pass
142
143         self.ReportServiceStatus(win32service.SERVICE_RUNN
ING)
144
145         try:
146             from apps.windows.telemetry.main import run_te
lemetry_service
147             from apps.windows.telemetry.telemetry_config i
mport TelemetryConfigManager
148
149             cfg_mgr = TelemetryConfigManager()
150             mode = cfg_mgr.get_mode()
151             interval = cfg_mgr.get_interval_seconds()
152             heavy_interval = cfg_mgr.get_heavy_interval_se
conds()
153             top_processes = cfg_mgr.get_effective_process_

```

```

limit()
154         heavy_collectors = cfg_mgr.get_heavy_collector
s()
155
156         logger.info(f'Запуск цикла телеметрии (режим:
{mode}, интервал: {interval}с, тяжелый: {heavy_interval}с)')
157         run_telemetry_service(
158             mode=mode,
159             interval=interval,
160             heavy_interval=heavy_interval,
161             top_processes=top_processes,
162             heavy_collectors=heavy_collectors,
163         )
164     except Exception as ex:
165         logger.error(f'Критическая ошибка при работе с
лужбы: {ex}', exc_info=True)
166     finally:
167         logger.info('Служба AITelemetry завершила рабо
ту.')
168         self.ReportServiceStatus(win32service.SERVICE_
STOPPED)
169
170
171 def run_interactive() -> int:
172     """Запускает телеметрию в интерактивном консольном режиме.
"""
173     _setup_service_logging()
174     from apps.windows.telemetry.main import main
175     return main()
176
177
178 def main_entry() -> int:
179     """Главная точка входа исполняемого файла AITelemetry.exe.
180
181     Returns:
182         int: Код возврата.
183     """
184     _setup_service_logging()

```

```

185
186     # Проверяем запуск диспетчером SCM без параметров
187     if len(sys.argv) == 1:
188         if PYWIN32_AVAILABLE:
189             try:
190                 servicemanager.Initialize()
191                 servicemanager.PrepareToHostSingle(AITelemetry
WindowsService)
192                 servicemanager.StartServiceCtrlDispatcher()
193                 return 0
194             except Exception as ex:
195                 # Если запущен не из SCM, переключаемся в конс
ольный режим
196                 logger.debug(f'Запуск через SCM не удался ({ex
}), переход в консольный режим.')
197                 return run_interactive()
198         else:
199             return run_interactive()
200
201     cmd = sys.argv[1].lower() if len(sys.argv) > 1 else ''
202
203     # Команды управления службой
204     if cmd in ('install', 'update', 'remove', 'start', 'stop',
'restart', 'status'):
205         if PYWIN32_AVAILABLE:
206             win32serviceutil.HandleCommandLine(AITelemetryWind
owsService)
207             return 0
208         else:
209             logger.error('Модуль pywin32 недоступен. Управлени
е службой через SCM невозможно.')
210             return 1
211
212     if cmd in ('run', 'interactive', '--mode', '--minimal', '--
interval', '--once', '-v', '--verbose', '--help', '-h'):
213         return run_interactive()
214
215     # По умолчанию передаем аргументы в главный CLI

```



```
216 |     return run_interactive()
217 |
218 |
219 | if __name__ == '__main__':
220 |     sys.exit(main_entry())
221 |
```

__init__.py

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # Process Name: AI-Breadboard Apps Windows Telemetry - Init
4 # =====
5 # Description:
6 #   Exports core system metrics models, sensor probers, and te
7 #  lemetry collectors.
8 #
9 # Usage Examples:
10 #
11 # File: __init__.py
12 # Project: ai-breadboard
13 # Package: apps.windows.telemetry
14 # Author: hypo69
15 # Copyright: © 2026 hypo69
16 # Updated: 2026-10-06 00:20:00
17 # =====
18
19 """Exports core system metrics models, sensor probers, and tel
20   emetry collectors."""
21
22 from .models import (
23     AnomalyItem,
24     BatteryMetrics,
25     CpuMetrics,
26     DiskIoMetrics,
27     DiskPartitionMetrics,
28     DriverInfo,
29     ForensicsActivityReport,
30     GpuMetrics,
31     HardwareArchiveEntry,
```

```
31 |     HardwareAuditReport,
32 |     HardwareChangeItem,
33 |     HardwareDeviceAudit,
34 |     HardwareNode,
35 |     HardwareSensor,
36 |     KernelThrottlingReport,
37 |     MemoryMetrics,
38 |     NetworkInterfaceMetrics,
39 |     NetworkPortMetrics,
40 |     PeripheralsNetworkReport,
41 |     PhysicalDiskHealth,
42 |     ProcessLeakDiagnosticsReport,
43 |     ProcessLeakItem,
44 |     ProcessLifecycleEvent,
45 |     ProcessMetrics,
46 |     ProcessNetworkActivity,
47 |     ProcessProvenanceInfo,
48 |     ProcessProvenanceReport,
49 |     ProcessTokenInfo,
50 |     RamStickInfo,
51 |     StorageBatteryWearReport,
52 |     SystemDiagnosticReport,
53 |     SystemHealthAlerts,
54 |     SystemSnapshot,
55 |     W64CollectorStatus,
56 |     W64SystemEvent,
57 |     ETWTraceEvent,
58 | )
59 | from .sensors import get_hardware_sensors
60 | from .sqlite import (
61 |     AggregationLevel,
62 |     TelemetryBuffer,
63 |     TelemetryConnectionManager,
64 |     TelemetryMaintenance,
65 |     TelemetryReader,
66 |     TelemetrySqlAggregator,
67 |     TelemetryStorage,
68 |     TelemetryWriter,
```

```
69     sensors_aggregate,
70 )
71 from .collector import SystemCollector
72 from .service import TelemetryLoggerService
73 from .telemetry_config import TelemetryConfigManager, get_default_telemetry_config_path
74 from .json_logger import TelemetryJsonLogger
75 from .file_collector import FileCollector
76 from .sensor_collector import SensorCollector
77 from .device_flapping_sensor import DeviceFlappingSensor, DeviceTransitionEvent
78 from apps.windows.modules.storage_manager.core.windows_storage_sensor import (
79     StorageDiskHealthInfo,
80     WindowsStorageSensor,
81     collect_storage_snapshot,
82     save_snapshot,
83 )
84 from .storage_usage import WindowsStorageUsageCollector
85 from .w64_collector import (
86     AIW64Collector,
87     get_w64_collector,
88     start_w64_collector,
89     stop_w64_collector,
90 )
91 from .w64_etw_collector import AIW64ETWCollector
92 from apps.windows.telemetry_research.aggregator import TelemetryAggregator
93 from apps.windows.telemetry_research.grouped_telemetry import GroupedTelemetryBuilder
94 from apps.windows.telemetry_research.incident_detector import IncidentDetector
95 from apps.windows.telemetry_research.reboot_analyzer import WindowsRebootAnalyzer
96 from apps.windows.telemetry_research.hardware_auditor import HardwareAuditor
97 from apps.windows.telemetry_research.deep_diagnostics import DeepDiagnosticsEngine
```

```
98 | from apps.windows.telemetry_research.hardware_history_manager
    | import HardwareHistoryManager
99 |
100 |
101 |
102 | __all__ = [
103 |     "AIW64Collector",
104 |     "AIW64ETWCollector",
105 |     "get_w64_collector",
106 |     "start_w64_collector",
107 |     "stop_w64_collector",
108 |     "CpuMetrics",
109 |     "MemoryMetrics",
110 |     "RamStickInfo",
111 |     "GpuMetrics",
112 |     "DiskPartitionMetrics",
113 |     "PhysicalDiskHealth",
114 |     "DiskIoMetrics",
115 |     "NetworkInterfaceMetrics",
116 |     "NetworkPortMetrics",
117 |     "BatteryMetrics",
118 |     "SystemHealthAlerts",
119 |     "ProcessMetrics",
120 |     "ProcessProvenanceInfo",
121 |     "ProcessLifecycleEvent",
122 |     "ProcessProvenanceReport",
123 |     "ProcessTokenInfo",
124 |     "ProcessLeakItem",
125 |     "ProcessLeakDiagnosticsReport",
126 |     "ForensicsActivityReport",
127 |     "KernelThrottlingReport",
128 |     "StorageBatteryWearReport",
129 |     "PeripheralsNetworkReport",
130 |     "HardwareSensor",
131 |     "HardwareNode",
132 |     "SystemSnapshot",
133 |     "AnomalyItem",
134 |     "SystemDiagnosticReport",
```

```
135 | "DriverInfo",
136 | "HardwareDeviceAudit",
137 | "HardwareChangeItem",
138 | "HardwareAuditReport",
139 | "HardwareArchiveEntry",
140 | "get_hardware_sensors",
141 | "HardwareAuditor",
142 | "HardwareHistoryManager",
143 | "TelemetryStorage",
144 | "TelemetrySqlAggregator",
145 | "AggregationLevel",
146 | "sensors_aggregate",
147 | "SystemCollector",
148 | "TelemetryLoggerService",
149 | "TelemetryConfigManager",
150 | "TelemetryJsonLogger",
151 | "FileCollector",
152 | "SensorCollector",
153 | "TelemetryAggregator",
154 | "DeviceFlappingSensor",
155 | "DeviceTransitionEvent",
156 | "DeepDiagnosticsEngine",
157 | "IncidentDetector",
158 | "WindowsRebootAnalyzer",
159 | "GroupedTelemetryBuilder",
160 | "TelemetryCompactor",
161 | "AuditStartupChecker",
162 | "StorageDiskHealthInfo",
163 | "WindowsStorageSensor",
164 | "WindowsStorageUsageCollector",
165 | "collect_storage_snapshot",
166 | "save_snapshot",
167 | "W64CollectorStatus",
168 | "W64SystemEvent",
169 | "ETWTraceEvent",
170 | "init_telemetry_database",
171 | "get_default_telemetry_db_path",
172 | "get_default_telemetry_config_path",
```

```
173 | ]
174 |
175 |
176 | def __getattr__(name: str):
177 |     """Ленивый импорт диагностического движка для предотвращен
    |     ия циклических зависимостей."""
178 |     if name in ("DiagnosticEngine", "SystemDiagnosticEngine"):
179 |         from apps.windows.telemetry_research.diagnostic_engine
    |         import DiagnosticEngine, SystemDiagnosticEngine
180 |         if name == "DiagnosticEngine":
181 |             return DiagnosticEngine
182 |         return SystemDiagnosticEngine
183 |     raise AttributeError(f"module '{__name__}' has no attribut
    | e '{name}'")
```